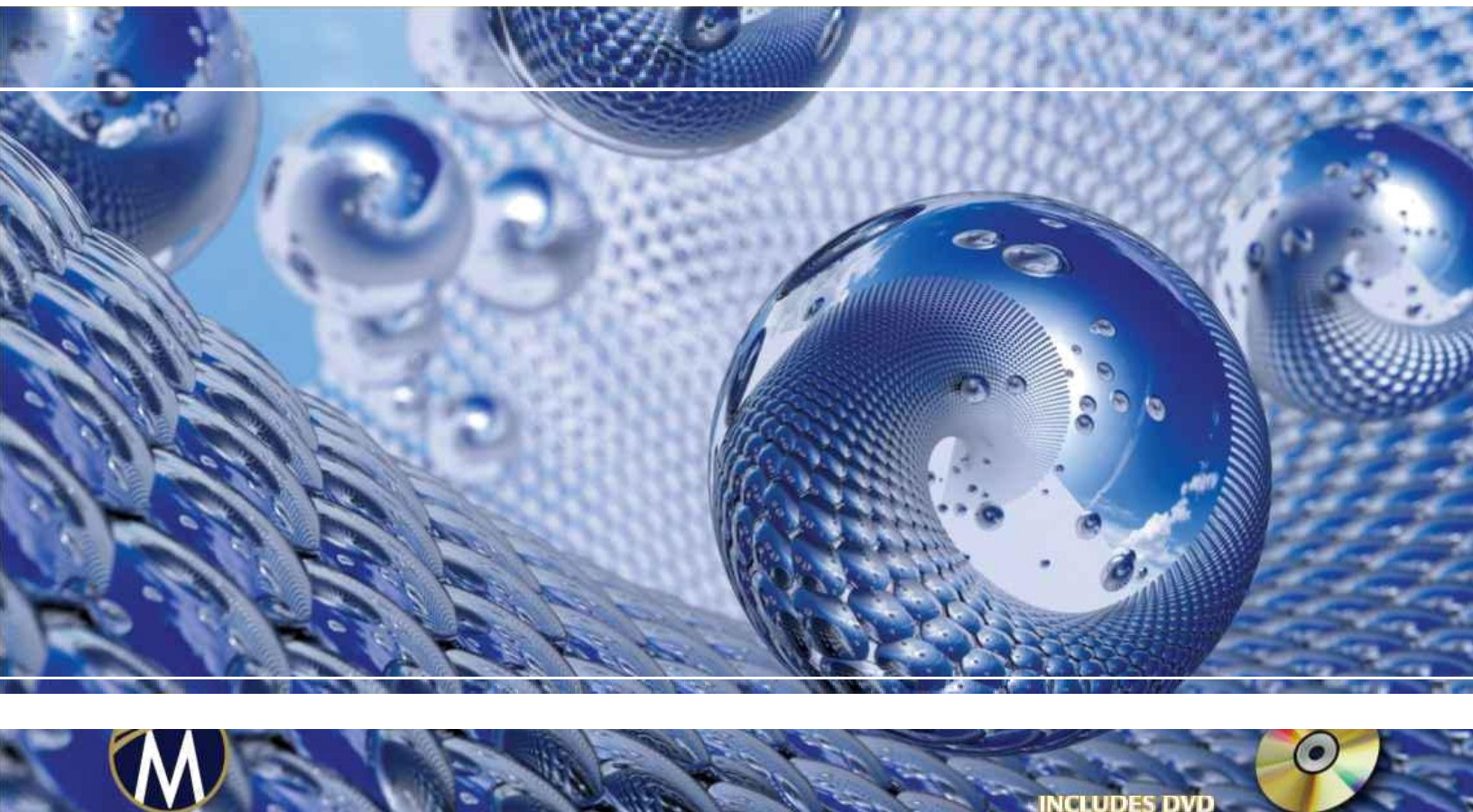


프로그래밍 입문

3D GAME

P프로그래밍

WITH DIRECTX® 11



INCLUDES DVD



FRANK D. LUNA

입문

DIRECTX® 11을 활용한 3D 게임 프로그

래밍 입문

소개

DIRECTX® 11을 활용한 3D 게임 프로그래밍

프랭크 D. 루나



머큐리 러닝 앤드 인포메이션

버지니아주 덜레스 매사추세츠

주 보스턴

저작권 ©2012 머큐리 러닝 앤드 인포메이션 LLC. **모든** 권리 보유.

본 출판물, 그 일부 또는 함께 제공되는 소프트웨어는 출판사의 사전 서면 허가 없이 복제, 모든 유형의 검색 시스템에 저장, 또는 복사, 녹음, 인터넷 게시, 스캔 등을 포함하
되 이에 국한되지 않는 모든 수단, 매체, 전자적 표시 또는 기계적 표시를 통해 전송될 수 없습니다.

발행인: David Pa lai
MERCURY LEARNING AND INFORMATION
22841 Quicksilver Drive
Du les, VA
20166info@mercleaming.com
www.mercleaming.com 1-800-
758-3756

이 책은 산성 없는 종이에 인쇄되었습니다.

프랭크 D. 루나. 《*DIRECTX 11을 활용한 3D 게임 프로그래밍 입문*》

ISBN: 978-1-9364202-2-3

출판사는 기업, 제조사 및 개발사가 자사 제품을 구별하기 위해 사용하는 상표를 인정하고 존중합니다. 본서에 언급된 **모든** 상표명 및 제품명은 해당 기업의 상표 또는 서비스 마크
입니다. 서비스 마크나 상표 등의 누락 또는 오용(어떠한 형태로든)은 타인의 재산을 침해하려는 의도가 아닙니다.

미국 의회 도서관 통제 번호: 2012931119 121314321

캐나다에서 인쇄됨

당사 출판물은 기관, 기업 등에 의해 채택, 라이선스 또는 대량 구매가 가능합니다. 자세한 내용은 고객 서비스 부서(1-800-758-3756, 무료)로 문의하십시오.

MERCURY LEARNING AND INFORMATION의 구매자에 대한 유일한 의무는 불량 재료 또는 제작 결함에 근거하여 디스크를 교체하는 것이며, 제품의 작동 또는 기능성에 근
거한 것은 아닙니다.

라이선스, 책임 부인 및 제한적 보증

본 서적(이하 "저작물")을 구매하거나 사용함으로써 귀하는 본 라이선스가 본 저작물에 포함된 내용의 사용 권한을 부여하지만, 서적 내 텍스트 콘텐츠에 대한 소유권 또는 그 안에 포함된 정보나 제품에 대한 소유권을 부여하지 않음을 동의합니다. **본 라이선스는 출판사의 서면 동의 없이 저작물을 인터넷이나 네트워크(어떠한 종류의 네트워크든)에 업로드하는 것을 허용하지 않습니다.** 본 저작물에 포함된 텍스트, 코드, 시뮬레이션, 이미지 등의 복제 또는 배포는 해당 제품의 라이선스 조건에 따라 제한되며, 저작물에 포함된 텍스트 자료의 일부를 (어떤 매체로든) 복제하거나 네트워크에 배포하려면 출판사 또는 해당 콘텐츠 소유자 등으로부터 허가를 받아야 합니다.

머큐리 러닝 앤드 인포메이션 LLC(이하 "MLI" 또는 "출판사") 및 본 저작물의 부속 디스크, 동반 알고리즘, 코드 또는 컴퓨터 프로그램(이하 "소프트웨어")의 제작, 집필 또는 생산에 관여한 모든 자, 그리고 본 저작물의 동반 웹사이트 또는 소프트웨어는 본 저작물의 내용을 사용함으로써 얻을 수 있는 성능이나 결과에 대해 보증할 수 없으며, 보증하지 않습니다. 저자, 개발자 및 출판사는 본 패키지에 포함된 텍스트 자료 및/또는 프로그램의 정확성과 기능성을 보장하기 위해 최선을 다했습니다. 그러나 우리는 이러한 내용이나 프로그램의 성능에 대해 어떠한 종류의 명시적 또는 묵시적 보증도 하지 않습니다. 본 저작물은 "있는 그대로" 보증 없이 판매됩니다(책 제작에 사용된 결함 있는 재료 또는 불량한 제작 기술로 인한 경우를 제외함).

본 저작물의 저자, 개발자, 부속 콘텐츠의 출판사 및 본 저작물의 구성, 제작, 제조에 관여한 모든 자는 본 출판물에 포함된 알고리즘, 소스 코드, 컴퓨터 프로그램 또는 텍스트 자료의 사용(또는 사용 불가)으로 인해 발생하는 어떠한 종류의 손해에 대해서도 책임을 지지 않습니다. 여기에는 본 저작물의 사용으로 인해 발생하는 수익 또는 이익의 손실, 기타 부수적, 물리적 또는 결과적 손해가 포함되나 이에 국한되지 않습니다.

모든 종류의 청구에 대한 유일한 구제책은 출판사의 재량에 따라 책을 교체하는 것으로 명시적으로 제한됩니다. "묵시적 보증" 및 특정 "제외 사항"의 사용은 주마다 다르며, 본 제품 구매자에게 적용되지 않을 수 있습니다.

조카들에게,

마릭, 한스, 맥스, 안나, 어거스터스, 프레슬리

감사의 글

서문

- 대상 독자 사전 지식
- 필수 개발 도구 및 하드웨어 D3DX 라이브러리 사용
- DirectX SDK 문서 및 SDK 샘플의 명확성
- 샘플 프로그램 및 온라인 보충 자료 Visual Studio 2010에서 데모 프로젝트 설정
 - Win32 프로젝트 생성 DirectX 라이브러리 연
 - 결 검색 경로 설정
 - 소스 코드 추가 및 프로젝트 빌드

제1부 수학적 선행 지식

제1장 벡터 대수학

- 1.1 벡터
 - 1.1.1 벡터와 좌표계
 - 1.1.2 좌손 좌표계 대 우손 좌표계
 - 1.1.3 기본 벡터 연산
- 1.2 벡터의 길이 및 단위 벡터
- 1.3 내적
 - 1.3.1 직교화
- 1.4 외적
 - 1.4.1 의사 2차원 외적
 - 1.4.2 외적에 의한 직교화
- 1.5 점
- 1.6 XNA 수학 벡터
 - 1.6.1 벡터 유형
 - 1.6.2 로드 및 저장 방법
 - 1.6.3 매개 변수 전달
 - 1.6.4 상수 벡터
 - 1.6.5 중복 정의된 연산자
 - 1.6.6 기타
 - 1.6.7 세터 함수
 - 1.6.8 벡터 함수
 - 1.6.9 부동 소수점 오류
- 1.7 요약
- 1.8 연습 문제

제2장 행렬 대수

- 2.1 정의
- 2.2 행렬 곱셈
 - 2.2.1 정의
 - 2.2.2 벡터-행렬 곱셈
 - 2.2.3 결합법칙
- 2.3 행렬의 전치
- 2.4 단위 행렬
- 2.5 행렬의 행렬식
 - 2.5.1 행렬의 소행렬
 - 2.5.2 정의

- 2.6 행렬의 애플리터
- 2.7 행렬의 역행렬
- 2.8 XNA 행렬
 - 2.8.1 행렬 유형
 - 2.8.2 행렬 함수
 - 2.8.3 XNA 행렬 샘플 프로그램
- 2.9 요약
- 2.10 연습 문제

제3장 변환

- 3.1 선형 변환
 - 3.1.1 정의
 - 3.1.2 행렬 표현
 - 3.1.3 확대/축소
 - 3.1.4 회전
- 3.2 아핀 변환
 - 3.2.1 동차 좌표
 - 3.2.2 정의와 행렬 표현
 - 3.2.2 평행 이동
 - 3.2.3 확대/축소 및 회전을 위한 아핀 행렬
 - 3.2.4 아핀 변환 행렬의 기하학적 해석.
- 3.3 변환의 합성
- 3.4 좌표계 변환
 - 3.4.1 벡터
 - 3.4.2 점
 - 3.4.3 행렬 표현
 - 3.4.4 결합성과 좌표 변환 행렬
 - 3.4.5 역행렬과 좌표계 변환 행렬
- 3.5 변환 행렬 대 좌표계 변환 행렬
- 3.6 XNA 수학 변환 함수
- 3.7 요약
- 3.8 연습 문제

제2부 DIRECT 3D 기초

제4장 Direct3D 초기화

- 4.1 준비 작업
 - 4.1.1 Direct3D 개요
 - 4.1.2 COM
 - 4.1.3 텍스처 및 데이터 리소스 형식
 - 4.1.4 스왑 체인과 페이지 플리핑
 - 4.1.5 깊이 버퍼링
 - 4.1.6 텍스처 리소스 뷰
 - 4.1.7 멀티샘플링 이론
 - 4.1.8 Direct3D에서의 멀티샘플링
 - 4.1.9 기능 레벨
- 4.2 Direct3D 초기화
 - 4.2.1 디바이스 및 컨텍스트 생성
 - 4.2.2 4X MSAA 품질 지원 확인
 - 4.2.3 스왑 체인 설명
 - 4.2.4 스왑 체인 생성
 - 4.2.5 렌더 타겟 뷰 생성
 - 4.2.6 깊이/스텐실 버퍼 생성 및 뷰
 - 4.2.7 뷰를 출력 병합 단계에 바인딩

4.2.8 뷰포트 설정

4.3 타이밍 및 애니메이션

4.3.1 성능 타이머

4.3.2 게임 타이머 클래스

- 4.3.3 프레임 간 경과 시간
- 4.3.4 총 시간
- 4.4 데모 애플리케이션 프레임워크
 - 4.4.1 D3DApp
 - 4.4.2 비프레임워크 메서드
 - 4.4.3 프레임워크 메서드
 - 4.4.4 프레임 통계
 - 4.4.5 메시지 핸들러
 - 4.4.6 전체 화면으로 전환
 - 4.4.7 "Direct3D 초기화" 데모
- 4.5 Direct3D 애플리케이션 디버깅
- 4.6 요약
- 4.7 연습 문제

제5장 렌더링 파이프라인

- 5.1 3D 환상
- 5.2 모델 표현
- 5.3 기본 컴퓨터 색상
 - 5.3.1 색상 연산
 - 5.3.2 128비트 색상
 - 5.3.3 32비트 색상
- 5.4 렌더링 파이프라인 개요
- 5.5 입력 어셈블러 단계
 - 5.5.1 정점
 - 5.5.2 프리미티브 토폴로지
 - 5.5.2.1 점 목록
 - 5.5.2.2 선 스트립
 - 5.5.2.3 선 목록
 - 5.5.2.4 삼각형 스트립
 - 5.5.2.5 삼각형 리스트
 - 5.5.2.6 인접성을 가진 기본 도형
 - 5.5.2.7 제어점 패치 리스트
 - 5.5.3 인덱스
- 5.6 버텍스 셰이더 단계
 - 5.6.1 로컬 공간과 월드 공간
 - 5.6.2 뷰 공간
 - 5.6.3 투영 및 동차 클립 공간
 - 5.6.3.1 프러스텀 정의
 - 5.6.3.2 버텍스 투영
 - 5.6.3.3 정규화된 장치 좌표(NDC)
 - 5.6.3.4 행렬을 이용한 투영 방정식 작성
 - 5.6.3.5 정규화된 깊이 값
 - 5.6.3.6 XMMatrixPerspectiveFovLH
- 5.7 테셀레이션 단계
- 5.8 지오메트리 셰이더 단계
- 5.9 클리핑
- 5.10 래스터화 단계
 - 5.10.1 뷰포트 변환
 - 5.10.2 백페이스 클리핑
 - 5.10.3 버텍스 속성 보간
- 5.11 픽셀 셰이더 단계
- 5.12 출력 병합 단계
- 5.13 요약
- 5.14 연습 문제

제6장 Direct3D에서의 그리기

6.1 버텍스와 입력 레이아웃

6.2 버텍스 버퍼

- 6.3 인덱스와 인덱스 버퍼
- 6.4 버텍스 셰이더 예제
- 6.5 상수 버퍼
- 6.6 예제 픽셀 셰이더
- 6.7 렌더 상태
- 6.8 이펙트
 - 6.8.1 이펙트 파일
 - 6.8.2 셰이더 컴파일
 - 6.8.3 C++ 애플리케이션에서 효과와 인터페이스하기
 - 6.8.4 이펙트를 사용한 그리기
 - 6.8.5 빌드 시점에 효과 컴파일하기
 - 6.8.6 "셰이더 생성기"로서의 효과 프레임워크
 - 6.8.7 어셈블리의 모습
- 6.9 박스 데모
- 6.10 언덕 데모
 - 6.10.1 그리드 정점 생성
 - 6.10.2 그리드 인덱스 생성
 - 6.10.3 높이 함수 적용
- 6.11 모양 데모
 - 6.11.1 실린더 메쉬 생성
 - 6.11.1.1 원통 측면 형상
 - 6.11.1.2 덮개 형상
 - 6.11.2 구체 메쉬 생성
 - 6.11.3 지오스피어 메쉬 생성
 - 6.11.4 데모 코드
- 6.12 파일에서 형상 로드하기
- 6.13 동적 버텍스 버퍼
- 6.14 요약
- 6.15 연습 문제

제7장 조명

- 7.1 빛과 재료의 상호작용
- 7.2 법선 벡터
 - 7.2.1 법선 벡터 계산
 - 7.2.2 법선 벡터 변환
- 7.3 람베르트 코사인 법칙
- 7.4 확산 조명
- 7.5 주변 조명
- 7.6 반사 조명
- 7.7 간략한 요약
- 7.8 재료 지정
- 7.9 평행광
- 7.10 포인트 라이트
 - 7.10.1 감쇠
 - 7.10.2 범위
- 7.11 스포트라이트
- 7.12 구현
 - 7.12.1 조명 구조
 - 7.12.2 구조물 포장
 - 7.12.3 방향성 조명 구현
 - 7.12.4 포인트 라이트 구현
 - 7.12.5 스포트라이트 구현
- 7.13 조명 데모

7.13.1 효과 파일

7.13.2 C++ 애플리케이션 코드

7.13.3 노멀 계산

7.14 Lit Skull 데모

7.15 요약

7.16 연습 문제

8장 텍스처링

- 8.1 텍스처와 리소스 복습
- 8.2 텍스처 좌표
- 8.3 텍스처 생성 및 활성화
- 8.4 필터
 - 8.4.1 확대
 - 8.4.2 축소
 - 8.4.2.1 밍맵 생성
 - 8.4.3 이방성 필터링
- 8.5 텍스처 샘플링
- 8.6 텍스처와 머티리얼
- 8.7 크레이트 데모
 - 8.7.1 텍스처 좌표 지정
 - 8.7.2 텍스처 생성
 - 8.7.3 텍스처 설정
 - 8.7.4 업데이트된 기본 효과
- 8.8 주소 모드
- 8.9 텍스처 변환
- 8.10 텍스처 처리된 언덕과 파도 데모
 - 8.10.1 그리드 텍스처 좌표 생성
 - 8.10.2 텍스처 타일링
 - 8.10.3 텍스처 애니메이션
- 8.11 압축 텍스처 형식
- 8.12 요약
- 8.13 연습 문제

제9장 블렌딩

- 9.1 블렌딩 방정식
- 9.2 블렌딩 연산
- 9.3 블렌딩 요소
- 9.4 블렌딩 상태
- 9.5 예시
 - 9.5.1 색상 쓰기 없음
 - 9.5.2 더하기/빼기
 - 9.5.3 곱하기
 - 9.5.4 투명도
 - 9.5.5 블렌딩과 깊이 버퍼
- 9.6 알파 채널
- 9.7 클리핑 픽셀
- 9.8 포그
- 9.9 요약
- 9.10 연습 문제

제10장 스텐실링

- 10.1 깊이/스텐실 형식 및 지우기
- 10.2 스텐실 테스트
- 10.3 깊이/스텐실 상태 블록
 - 10.3.1 깊이 설정
 - 10.3.2 스텐실 설정
 - 10.3.3 깊이/스텐실 상태 생성 및 바인딩
 - 10.3.4 효과 파일 내의 깊이/스텐실 상태
- 10.4 평면 거울 구현

10.4.1 미러 개요

10.4.2 미러 깊이/스텐실 상태 정의

10.4.3 썬 그리기

10.4.4 와인딩 순서와 반사

- 10.5 평면 그림자 구현
 - 10.5.1 평행광 그림자
 - 10.5.2 점광원 그림자
 - 10.5.3 일반 그림자 행렬
 - 10.5.4 스텐실 버퍼를 이용한 이중 블렌딩 방지
 - 10.5.5 그림자 코드
- 10.6 요약
- 10.7 연습 문제

제11장 기하학 셰이더

- 11.1 지오메트리 셰이더 프로그래밍
- 11.2 트리 빌보드 데모
 - 11.2.1 개요
 - 11.2.2 버텍스 구조
 - 11.2.3 이펙트 파일
 - 11.2.4 SV_PrimitiveID
 - 11.3 텍스처 배열
 - 11.3.1 개요
 - 11.3.2 텍스처 배열 샘플링
 - 11.3.3 텍스처 배열 로드
 - 11.3.4 텍스처 서브 리소스
- 11.4 알파 투 커버리지
- 11.5 요약
- 11.6 연습 문제

12장 컴퓨터 셰이더

- 12.1 스레드와 스레드 그룹
- 12.2 간단한 컴퓨터 셰이더
- 12.3 데이터 입력 및 출력 리소스
 - 12.3.1 텍스처 입력
 - 12.3.2 텍스처 출력 및 비순차적 액세스 뷰(UAV)
 - 12.3.3 인덱싱 및 텍스처 샘플링
 - 12.3.4 구조화된 버퍼 리소스
 - 12.3.5 CS 결과 복사 시스템 메모리
- 12.4 스레드 식별 시스템 값
- 12.5 버퍼 추가 및 소비
- 12.6 공유 메모리 및 동기화
- 12.7 블러 데모
 - 12.7.1 블러링 이론
 - 12.7.2 렌더-투-텍스처
 - 12.7.3 블러 구현 개요
 - 12.7.4 컴퓨터 셰이더 프로그램
- 12.8 추가 자료
- 12.9 요약
- 12.10 연습 문제

13장 테셀레이션 단계

- 13.1 테셀레이션 프리미티브 유형
 - 13.1.1 테셀레이션과 버텍스 셰이더
- 13.2 외피 셰이더
 - 13.2.1 상수 헬 셰이더
 - 13.2.2 제어점 헬 셰이더
- 13.3 테셀레이션 단계
 - 13.3.1 쿼드 패치 테셀레이션 예시

13.3.2 삼각형 패치 테셀레이션 예시

13.4 도메인 셰이더

13.5 쿼드 테셀레이션

13.6 큐빅 베지어 사각 패치

- 13.6.1 베지어 곡선
- 13.6.2 3차 베지어 표면
- 13.6.3 3차 베지어 표면 평가 코드
- 13.6.4 패치 기하 구조 정의
- 13.7 요약
- 13.8 연습 문제

제3부 주제

제14장 1인칭 카메라 구축

- 14.1 뷰 트랜스폼 복습
- 14.2 카메라 클래스
- 14.3 선택된 메서드 구현
 - 14.3.1 `XMVECTOR` 반환 변형
 - 14.3.2 `SetLens`
 - 14.3.3 파생된 프러스텀 정보
 - 14.3.4 카메라 변환
 - 14.3.5 뷰 매트릭스 구축
- 14.4 카메라 데모 코멘트
- 14.5 요약
- 14.6 연습 문제

15장 인스턴싱과 프러스텀 컬링

- 15.1 하드웨어 인스턴싱
 - 15.1.1 버텍스 셰이더
 - 15.1.2 스트리밍 인스턴스화된 데이터
 - 15.1.3 인스턴스화된 데이터 그리기
 - 15.1.4 인스턴스화된 버퍼 생성
- 15.2 바운딩 볼륨과 프러스텀
 - 15.2.1 XNA 충돌
 - 15.2.2 박스
 - 15.2.2.1 회전 및 축 정렬 바운딩 박스
 - 15.2.3 구체
 - 15.2.4 프러스텀
 - 15.2.4.1 프러스텀 평면 구성
 - 15.2.4.2 프러스텀/구체 교차
 - 15.2.4.3 프러스텀/AABB 교차
- 15.3 프러스텀 컬링
- 15.4 요약
- 15.5 연습 문제

제16장 픽킹

- 16.1 스크린에서 투영 창으로의 변환
- 16.2 월드/로컬 공간 픽킹 레이
- 16.3 레이/메쉬 교차
 - 16.3.1 레이/AABB 교차
 - 16.3.2 레이/구체 교차
 - 16.3.3 레이/삼각형 교차
- 16.4 데모 애플리케이션
- 16.5 요약
- 16.6 연습 문제

17장 큐브 매핑

- 17.1 큐브 매핑
- 17.2 환경 맵

17.2.1 Direct3D에서 큐브 맵 로딩 및 사용

17.3 하늘 텍스처링

17.4 반사 모델링

- 17.5 동적 큐브 맵
 - 17.5.1 큐브 맵 및 렌더 타겟 뷰 구축
 - 17.5.2 깊이 버퍼 및 뷰포트 구축
 - 17.5.3 큐브 맵 카메라 설정
 - 17.5.4 큐브 맵에 그리기
- 17.6 지오메트리 셰이더를 사용한 동적 큐브 맵
- 17.7 요약
- 17.8 연습 문제

제18장 노멀 매핑과 변위 매핑

- 18.1 동기 부여
- 18.2 노멀 맵
- 18.3 텍스처/접선 공간
- 18.4 벡스 탄젠트 공간
- 18.5 접선 공간과 객체 공간 간 변환
- 18.6 노멀 매핑 셰이더 코드
- 18.7 변위 매핑
- 18.8 변위 매핑 셰이더 코드
 - 18.8.1 프리미티브 타입
 - 18.8.2 벡스 셰이더
 - 18.8.3 힐 셰이더
 - 18.8.4 도메인 셰이더
- 18.9 요약
- 18.10 연습 문제

19장 지형 렌더링

- 19.1 하이트맵
 - 19.1.1 하이트맵 생성하기
 - 19.1.2 RAW 파일 로드하기
 - 19.1.3 스무딩
 - 19.1.4 하이트맵 셰이더 리소스 뷰
- 19.2 지형 테셀레이션
 - 19.2.1 그리드 구축
 - 19.2.2 지형 벡스 셰이더
 - 19.2.3 테셀레이션 팩터
 - 19.2.4 변위 매핑
 - 19.2.5 접선 및 법선 벡터 추정
- 19.3 프러스텀 클립 패치
- 19.4 텍스처링
- 19.5 지형 높이
- 19.6 요약
- 19.7 연습 문제

제20장 입자 시스템과 스트림 아웃

- 20.1 입자 표현
- 20.2 입자 운동
- 20.3 무작위성
- 20.4 블렌딩과 파티클 시스템
- 20.5 스트림 아웃
 - 20.5.1 스트림 아웃용 지오메트리 셰이더 생성
 - 20.5.2 스트림 아웃 전용
 - 20.5.3 스트림 아웃용 벡스 버퍼 생성
 - 20.5.4 스트림 아웃 스테이지에 바인딩
 - 20.5.5 스트림 아웃 스테이지에서 바인딩 해제

20.5.6 자동 드로우

20.5.7 버텍스 버퍼 핑퐁

20.6 GPU 기반 파티클 시스템

20.6.1 파티클 효과

- 20.6.2 파티클 시스템 클래스
- 20.6.3 이미터 파티클
- 20.6.4 초기화 버텍스 버퍼
- 20.6.5 업데이트/드로우 메서드

- 20.7 발사
- 20.8 비
- 20.9 요약
- 20.10 연습 문제

21장 색도 매핑

- 21.1 씬 깊이 렌더링
- 21.2 직교 투영
- 21.3 투영 텍스처 좌표
 - 21.3.1 코드 구현
 - 21.3.2 프러스텀 외부 점
 - 21.3.3 직교 투영
- 21.4 색도 매핑
 - 21.4.1 알고리즘 설명
 - 21.4.2 바이어싱 및 에일리어싱
 - 21.4.3 PCF 필터링
 - 21.4.4 색도 맵 구축
 - 21.4.5 색도우 팩터
 - 21.4.6 색도 맵 테스트
 - 21.4.7 색도 맵 렌더링
- 21.5 대형 PCF 커널
 - 21.5.1 DDX 및 DDY 함수
 - 21.5.2 대형 PCF 커널 문제에 대한 해결책
 - 21.5.3 대형 PCF 커널 문제에 대한 대안적 해결책..
- 21.6 요약
- 21.7 연습문제

제22장 앰비언트 오클루전

- 22.1 레이 캐스팅을 통한 앰비언트 오클루전
- 22.2 스크린 스페이스 앰비언트 오클루전
 - 22.2.1 노멀 패스 및 깊이 패스 렌더링
 - 22.2.2 앰비언트 오클루전 패스
 - 22.2.2.1 뷰 스페이스 위치 재구성
 - 22.2.2.2 무작위 샘플 생성
 - 22.2.2.3 잠재적 가림 지점 생성
 - 22.2.2.4 가림 테스트 수행
 - 22.2.2.5 계산 완료
 - 22.2.2.6 구현
 - 22.2.3 블러 패스
 - 22.2.4 앰비언트 오클루전 맵 사용
- 22.3 요약
- 22.4 연습 문제

23장 메쉬

- 23.1 $m3d$ 형식
 - 23.1.1 헤더
 - 23.1.2 재료
 - 23.1.3 부분 집합
 - 23.1.4 정점 및 삼각형 인덱스
- 23.2 메쉬 지오메트리

23.3 기본 모델

23.4 메쉬 뷰어 데모

23.5 요약

23.6 연습 문제

제24장 쿼터니언

- 24.1 복소수의 복습
 - 24.1.1 정의
 - 24.1.2 기하학적 해석
 - 24.1.3 극좌표 표현과 회전
- 24.2 쿼터니언 대수
 - 24.2.1 정의와 기본 연산
 - 24.2.2 특수 곱
 - 24.2.3 성질
 - 24.2.4 변환
 - 24.2.5 공액과 노름
 - 24.2.6 역원
 - 24.2.7 극좌표 표현
- 24.3 단위 사원수와 회전
 - 24.3.1 회전 연산자
 - 24.3.2 쿼터니언 회전 연산자를 행렬로 변환
 - 24.3.3 행렬을 사원수 회전 연산자로 변환
 - 24.3.4 합성
- 24.4 쿼터니언 보간
- 24.5 XNA 수학 쿼터니언 함수
- 24.6 회전 데모
- 24.7 요약
- 24.8 연습 문제

25장 캐릭터 애니메이션

- 25.1 프레임 계층 구조
 - 25.1.1 수학적 공식화
- 25.2 스킨된 메시
 - 25.2.1 정의
 - 25.2.2 뼈대-루트 변환 재구성
 - 25.2.3 오프셋 변환
 - 25.2.4 스켈레톤 애니메이션
 - 25.2.5 최종 변환 계산
- 25.3 버텍스 블렌딩
- 25.4 .m3d 애니메이션 데이터 로드
 - 25.4.1 스킨된 버텍스 데이터
 - 25.4.2 본 오프셋 변환
 - 25.4.3 계층 구조
 - 25.4.4 애니메이션 데이터
 - 25.4.5 M3DLoader
- 25.5 캐릭터 애니메이션 데모
- 25.6 요약
- 25.7 연습 문제

부록 A: Windows 프로그래밍 소개

- A.1 개요
 - A.1.1 참고 자료
 - A.1.2 이벤트, 메시지 큐, 메시지, 메시지 루프...
 - A.1.3 GUI
 - A.1.4 유니코드
- A.2 기본 Windows 애플리케이션
- A.3 기본 Windows 애플리케이션 설명
 - A.3.1 포함, 전역 변수 및 프로토타입
- A.3.2 WinMain

A.3.3 `WNDCLASS` 및 등록

A.3.4 창 생성 및 표시

A.3.5 메시지 루프

A.3.6 윈도우 프로시저

A.3.7 메시지박스 함수

A.4 더 나은 메시지 루프

A.5 요약

A.6 연습 문제

부록 B: 고급 세이더 언어 참조

변수 유형 스칼라 유형

벡터 유형 스위즐 매트

릭스 유형 배열 구조체

`typedef` 키워드 변수 접두사 형 변

환

키워드 및 연산자 키워드

연산자 프로그램 흐름

함수

사용자 정의 함수 내장 함수

부록 C: 일부 해석 기하학

C.1 광선, 직선 및 선분

C.2 평행사변형

C.3 삼각형

C.4 평면

C.4.1 XNA 수학 평면

C.4.2 점/평면의 공간적 관계

C.4.3 구성

C.4.4 평면의 정규화

C.4.5 평면 변환

C.4.6 주어진 점에 대한 평면 상의 가장 가까운 점

C.4.7 선분/평면 교차점

C.4.8 벡터 반사

C.4.9 점의 대칭

C.4.10 반사 행렬

C.5 연습문제

부록 D: 연습문제 해답 *참고문헌 및 추가 자료 색인*

이 책의 초판 검토를 도와주신 Rod Lopez, Jim Leiterman, Hanley Leung, Rick Falck, Tybon Wu, Tuomas Sandroos, Eric Sandegren께 감사드립니다. 또한 이번 판의 일부 장을 검토해 주신 Jay Tennant와 William Goschnick께도 감사드립니다. 이 책의 데모 프로그램에 사용된 3D 모델과 텍스처 일부를 제작해 주신 Tyler Drinkard에게도 감사드립니다. 또한 데일 E. 라 포스, 애덤 홀트, 게리 시몬스, 제임스 램버스, 윌리엄 친의 도움에 감사드립니다. 마지막으로 머큐리 러닝 앤 인포메이션의 직원들, 특히 데이비드 팔라이에게 감사드립니다.

Direct3D 11은 Windows 플랫폼에서 최신 그래픽 하드웨어를 활용해 고성능 3D 그래픽 애플리케이션을 작성하기 위한 렌더링 라이브러리입니다. (XBOX 360에서는 DirectX 9의 수정 버전이 사용됩니다.) Direct3D는 제어하는 기본 그래픽 하드웨어를 밀접하게 모델링하는 애플리케이션 프로그래밍 인터페이스(API)를 제공한다는 점에서 저수준 라이브러리입니다. Direct3D의 주요 사용처는 게임 산업으로, 상위 수준의 렌더링 엔진이 Direct3D 위에 구축됩니다. 그러나 의료 및 과학 시각화, 건축 시뮬레이션 등 다른 산업 분야에서도 고성능 대화형 3D 그래픽이 필요합니다. 또한 모든 신규 PC에 현대적인 그래픽 카드가 장착됨에 따라, 비(非) 3D 애플리케이션들도 GPU(그래픽 처리 장치)를 활용하여 집약적인 계산을 그래픽 카드에 오프로드하기 시작했습니다. 이를 *범용 GPU 컴퓨팅*이라고 하며, Direct3D 11은 범용 GPU 프로그램을 작성하기 위한 컴퓨터 셰이더 API를 제공합니다. Direct3D는 일반적으로 네이티브 C++로 프로그래밍되지만, 안정적인 .NET 래퍼(예: <http://slimdx.org/>)가 존재하므로 관리되는 애플리케이션에서도 이 강력한 3D 그래픽 API에 접근할 수 있습니다. 마지막으로, 마이크로소프트는 2011년 BUILD 컨퍼런스(<http://www.buildwindows.com/>)에서 Direct3D 11이 Windows 8의 고성능 3D "Metro" 애플리케이션 개발에 핵심 역할을 할 것임을 최근 시연했습니다. 종합적으로 볼 때, Direct3D 개발자들에게 미래는 밝아 보입니다.

이 책은 Direct3D 11을 사용하여 게임 개발에 중점을 둔 대화형 컴퓨터 그래픽스 프로그래밍 입문을 소개합니다. Direct3D와 셰이더 프로그래밍의 기초를 가르친 후, 독자는 더 고급 기술을 배울 준비가 될 것입니다. 이 책은 세 개의 주요 부분으로 나뉩니다. 제1부에서는 이 책 전반에 걸쳐 사용될 수학적 도구를 설명합니다. 제2부에서는 Direct3D에서 초기화, 3D 지오메트리 정의, 카메라 설정, 버텍스 셰이더/픽셀 셰이더/지오메트리 셰이더/컴퓨터 셰이더 생성, 라이팅, 텍스처링, 블렌딩, 스텐실링, 테셀레이션 등 기본 작업 구현 방법을 보여줍니다. 제3부는 Direct3D를 활용하여 다양한 흥미로운 기법과 특수 효과를 구현하는 데 중점을 둡니다. 메쉬 작업, 지형 렌더링, 피킹, 파티클 시스템, 환경 매핑, 노멀 매핑, 변위 매핑, 실시간 그림자, 앰비언트 오클루전 등이 포함됩니다.

초보자의 경우 이 책을 처음부터 끝까지 순서대로 읽는 것이 가장 좋습니다. 각 장은 난이도가 점진적으로 증가하도록 구성되어 있어 독자가 갑자기 복잡해지는 내용에 혼란스러워하지 않도록 했습니다. 일반적으로 특정 장에서는 이전에 다룬 기술과 개념을 활용합니다. 따라서 다음 장으로 넘어가기 전에 해당 장의 내용을 완전히 숙지하는 것이 중요합니다. 경험이 있는 독자는 관심 있는 장만 선택하여 읽을 수 있습니다.

마지막으로, 이 책을 읽은 후 어떤 종류의 게임을 개발할 수 있을지 궁금하실 수 있습니다. 그 답은 이 책을 훑어보며 개발된 애플리케이션 유형을 확인하는 것이 가장 좋습니다. 이를 통해 이 책에서 가르치는 기술과 여러분의 독창성을 바탕으로 개발 가능한 게임 유형을 상상해 볼 수 있을 것입니다.

대상 독자

이 책은 다음 세 가지 독자층을 염두에 두고 기획되었습니다:

1. Direct3D 최신 버전을 활용한 3D 프로그래밍 입문을 원하는 중급 C++ 프로그래머.
2. DirectX 이외의 API(예: OpenGL)에 익숙한 3D 프로그래머로서 Direct3D 11 입문 정보를 원하는 분들.
3. Direct3D 9 및 Direct3D 11에 익숙한 프로그래머로서 최신 버전의 Direct3D를 배우고자 하는 분들.

필수 조건

이 책은 Direct3D 11, 셰이더 프로그래밍, 3D 게임 프로그래밍에 대한 입문서임을 강조해야 합니다. 일반적인 컴퓨터 프로그래밍 입문서가 *아닙니다*. 독자는 다음 선행 지식을 갖추어야 합니다:

1. 고등학교 수준의 수학: 대수학, 삼각법, (수학적) 함수 등.
2. Visual Studio 숙련도: 프로젝트 생성, 파일 추가, 외부 라이브러리 링크 지정 방법 등을 알고 있어야 합니다.
3. 중급 수준의 C++ 및 데이터 구조 기술: 포인터, 배열, 연산자 중목 정의, 연결 리스트, 상속, 다형성 등에 익숙해야 합니다.
4. Win32 API를 활용한 Windows 프로그래밍에 익숙하면 도움이 되지만 필수 사항은 아닙니다. 부록 A에 Win32 입문 자료를 제공합니다.

필수 개발 도구 및 하드웨어

Direct3D 11 애플리케이션을 프로그래밍하려면 DirectX 11 SDK가 필요합니다. 최신 버전은 <http://msdn.microsoft.com/en-us/directx/default.aspx>에서 다운로드할 수 있습니다. 다운로드 후 설치 방법사의 지침을 따르십시오. 본서 집필 시점 기준 최신 SDK 버전은 2010년 6월 DirectX SDK입니다. 모든 샘플 프로그램은 Visual Studio 2010으로 작성되었습니다.

Direct3D 11은 Direct3D 11 지원 하드웨어가 필요합니다. 이 책의 데모는 Geforce GTX 460에서 테스트되었습니다.

D3DX 라이브러리 사용

버전 7.0부터 DirectX는 D3DX(Direct3D 확장) 라이브러리를 함께 제공합니다. 이 라이브러리는 수학 연산, 텍스처 및 이미지 연산, 메시 연산, 셰이더 연산(예: 컴파일 및 어셈블)과 같은 일반적인 3D 그래픽스 관련 작업을 단순화하는 함수, 클래스 및 인터페이스 세트를 제공합니다. 즉, D3DX에는 직접 구현하기 번거로운 많은 기능들이 포함되어 있습니다.

이 책에서는 D3DX 라이브러리를 사용합니다. 이를 통해 더 흥미로운 내용에 집중할 수 있기 때문입니다. 예를 들어, 다양한 이미지 형식(.bmp, .jpeg 등)을 Direct3D 텍스처 인터페이스로 로드하는 방법을 여러 페이지에 걸쳐 설명하기보다는 D3DX 함수 `D3DxiCreateTextureFromFile`를 한 번 호출하는 것으로 해결할 수 있습니다. 즉, D3DX는 생산성을 높여주고, 바퀴를 재발명하는 데 시간을 낭비하지 않고 실제 콘텐츠에 더 집중할 수 있게 해줍니다.

D3DX를 사용해야 하는 다른 이유:

1. D3DX는 범용적이며 다양한 유형의 3D 애플리케이션에 폭넓게 활용될 수 있습니다.
2. D3DX는 빠릅니다. 일반적인 기능이 가질 수 있는 속도만큼은 최소한 보장됩니다.
3. 다른 개발자들은 D3DX를 사용합니다. 따라서 D3DX를 사용하는 코드를 접하게 될 가능성이 매우 높습니다. 결과적으로 D3DX를 사용할지 여부와 관계없이, 이를 사용하는 코드를 읽을 수 있도록 D3DX에 익숙해져야 합니다.
4. D3DX는 이미 존재하며 철저히 테스트되었습니다. 또한 DirectX의 각 버전 업데이트마다 더욱 개선되고 기능이 풍부해집니다.

DIRECTX SDK 문서 및 SDK 샘플 사용하기

Direct3D는 방대한 API로, 이 한 권의 책으로 그 모든 세부 사항을 다루기는 어렵습니다. 따라서 자세한 정보를 얻으려면 DirectX SDK 문서를 활용하는 방법을 반드시 익혀야 합니다. DirectX SDK를 설치한 디렉터리(*DirectX SDK*)의 *DirectX SDK\Documentation\DirectX9* 디렉터리에서 *windows_graphics.chm* 파일을 실행하면 C++ DirectX 온라인 문서를 열 수 있습니다. 특히 Direct3D 11 섹션([그림 1](#) 참조)으로 이동해야 합니다.

DirectX 문서는 DirectX API의 거의 모든 부분을 다루고 있으므로 참고 자료로 매우 유용합니다. 그러나 문서가 깊이 있게 다루지 않거나 사전 지식을 가정하기 때문에 학습 도구로는 최적이지 않습니다. 다만, 새로운 DirectX 버전이 출시될 때마다 점점 더 나아지고 있습니다.

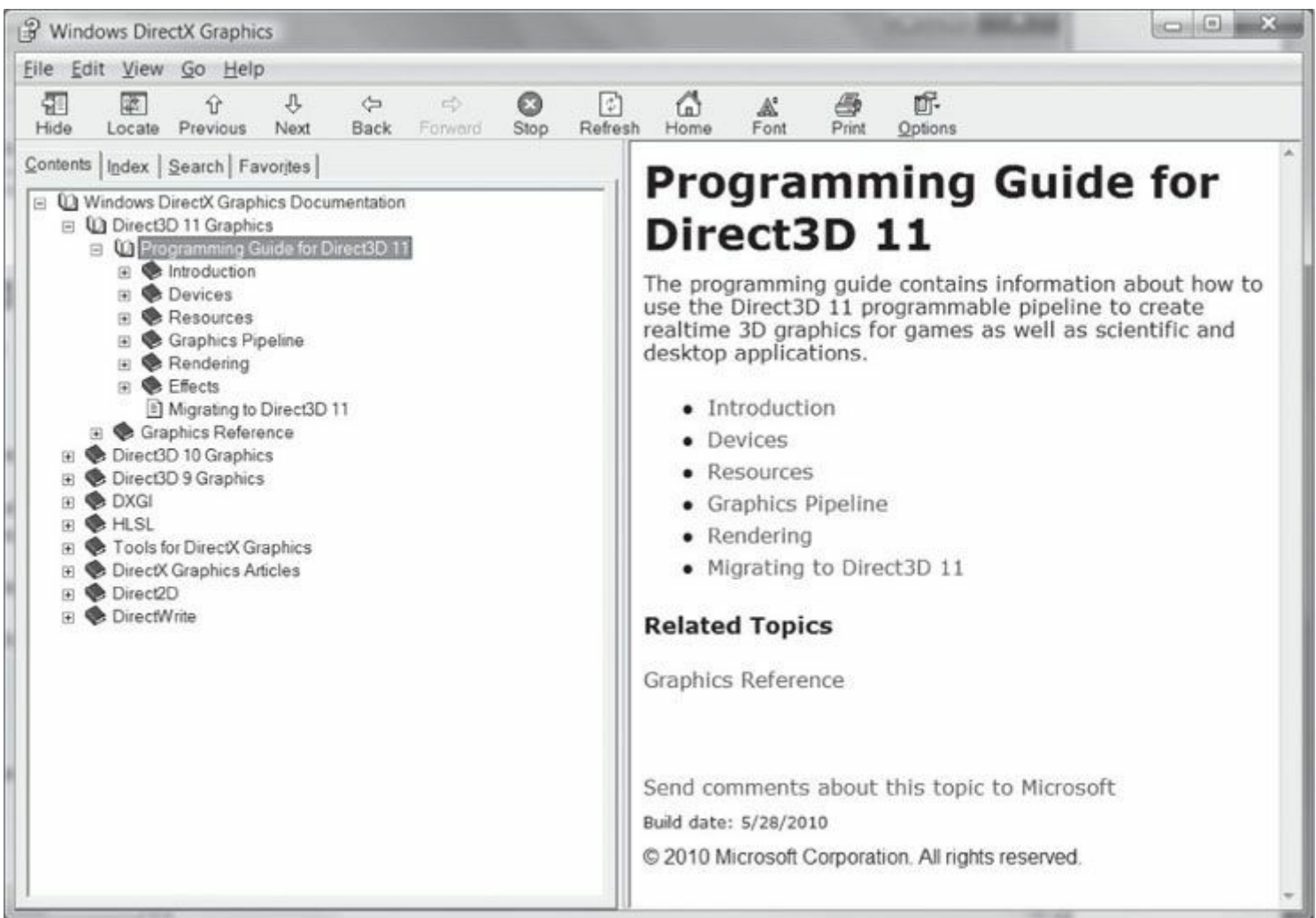


그림 1. DirectX 문서의 Direct3D 프로그래밍 가이드.

앞서 언급했듯이, 이 문서는 주로 참고 자료로 유용합니다. DirectX 관련 유형이나 함수(예: `ID3D11Device::createBuffer` 함수)를 접했을 때 더 많은 정보를 원한다면, 문서 색인을 검색하기만 하면 해당 객체 유형(이 경우에는 함수)에 대한 설명을 얻을 수 있습니다(그림 2 참조).

Note 이 책에서는 때때로 자세한 내용을 위해 여러분을 문서로 안내할 수 있습니다.

DirectX SDK에 포함된 Direct3D 샘플 프로그램도 참고하시기 바랍니다. C++ Direct3D 샘플은 `DirectX SDK\Samples\C++\Direct3D10` 및 `DirectX SDK\Samples\C++\Direct3D11` 디렉터리 에 위치합니다. 각 샘플은 Direct3D에서 특정 효과를 구현하는 방법을 보여줍니다. 이 샘플들은 그래픽 프로그래밍 초보자에게는 상당히 고급 수준이지만, 이 책을 다 읽은 후에는 충분히 연구할 수 있을 것입니다. 샘플을 살펴보는 것은 이 책을 마친 후 좋은 '다음 단계'가 될 것입니다. Direct3D 10과 Direct3D 11 샘플을 모두 언급한 점에 유의하십시오. Direct3D 11은 Direct3D 10에 새로운 기능을 추가한 것이므로, Direct3D 11 애플리케이션을 만들 때에도 Direct3D 10 기법이 여전히 적용됩니다. 따라서 특정 효과를 구현하는 방법을 확인하기 위해 Direct3D 10 샘플을 연구하는 것은 여전히 가치가 있습니다.

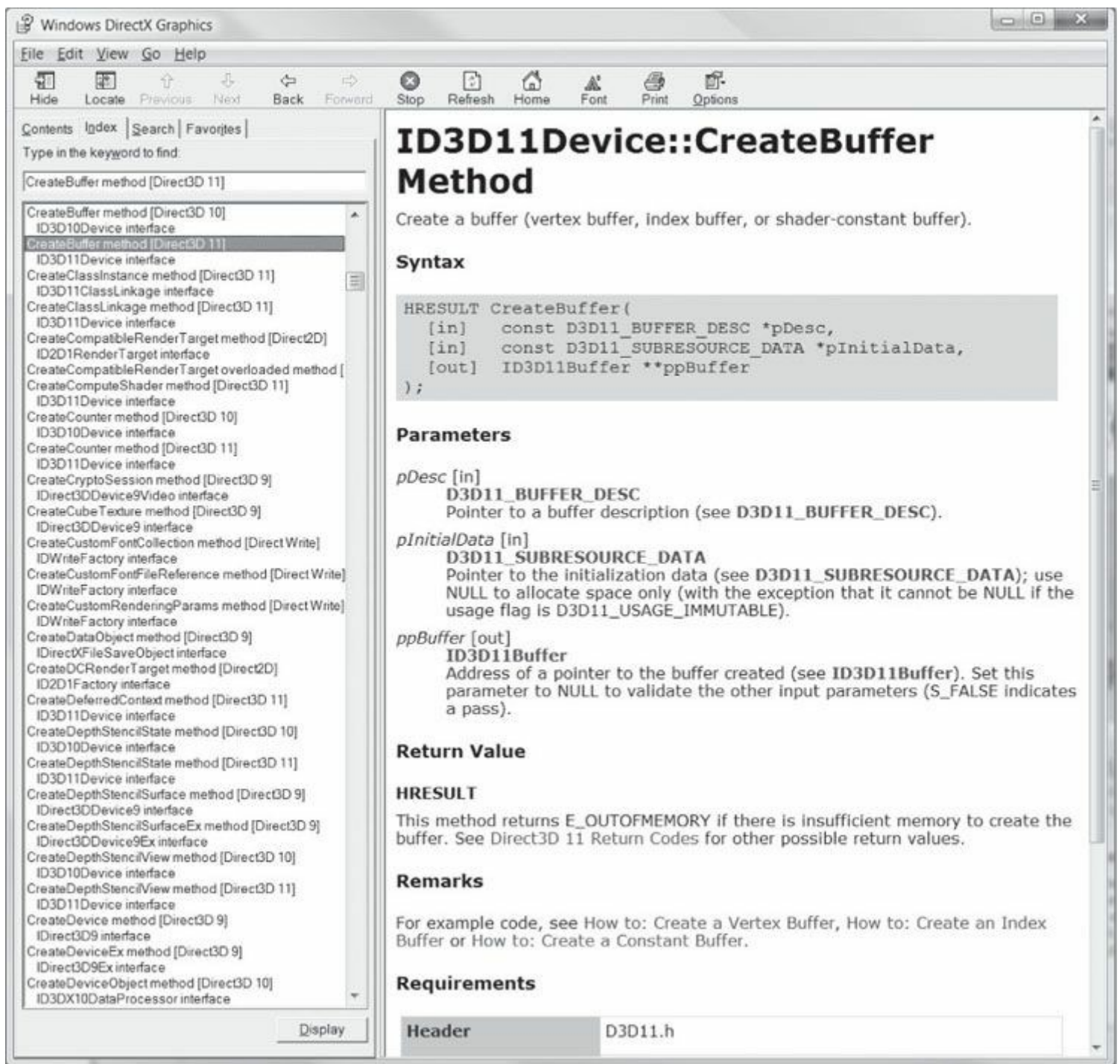


그림 2. DirectX 문서 색인.

명확성

이 책의 프로그램 샘플은 성능보다는 명확성을 염두에 두고 작성되었음을 강조하고자 합니다. 따라서 많은 샘플이 비효율적으로 구현되었을 수 있습니다. 본서의 샘플 코드를 여러분의 프로젝트에 활용하실 경우, 효율성 향상을 위해 재작업을 고려하시기 바랍니다. 또한 Direct3D API에 집중하기 위해 Direct3D 위에 최소한의 인프라만 구축하였습니다. 대규모 3D 애플리케이션에서는 Direct3D 위에 렌더링 엔진을 구현할 가능성이 높지만, 본서의 주제는 렌더링 엔진 설계가 아닌 Direct3D API입니다.

샘플 프로그램 및 온라인 보충 자료

이 책의 부록 DVD와 웹사이트(www.d3dcode.net 및 www.mercclearing.com)는 이 책을 최대한 활용하는 데 필수적인 역할을 합니다. DVD와 웹사이트에서는 이 책의 모든 예제에 대한 완전한 소스 코드와 프로젝트 파일을 찾을 수 있습니다. 대부분의 경우 DirectX 프로그램은 교재에 완전히 포함시키기에 너무 크기 때문에, 설명하는 개념에 기반한 관련 코드 조각만 포함시켰습니다. 독자 여러분께서는 해당 데모 코드를 학습하여 프로그램 전체를 살펴볼 것을 적극 권장합니다. (데모는 학습이 용이하도록 간결하고 핵심에 집중하도록 구성했습니다.) 일반적으로 독자 여러분께서는 해당 장을 읽고 데모 코드를 충분히 학습한 후, 해당 장의 데모를 스스로 구현할 수 있어야 합니다. 실제로

좋은 연습법은 책을 참고 자료로 삼고 샘플 코드를 활용하여 직접 예제를 구현해 보는 것입니다.

샘플 프로그램 외에도 웹사이트에는 게시판이 마련되어 있습니다. 독자 여러분께서 서로 소통하시고 이해가 되지 않는 주제나 명확히 설명이 필요한 주제에 대해 질문을 게시해 주시기를 권장합니다. 개념에 대한 다양한 관점과 설명을 접하는 것이 이해 속도를 높이는 경우가 많습니다. 마지막으로, 이 책에 수록하지 못한 주제에 대한 추가 프로그램 샘플과 튜토리얼을 웹사이트에 게시할 계획입니다.

VISUAL STUDIO 2010에서의 데모 프로젝트 설정

이 책의 데모는 해당 프로젝트 파일(.vcxproj) 또는 솔루션 파일(.sin)을 더블클릭하기만 하면 실행할 수 있습니다. 이 섹션에서는 Visual Studio 2010(VS10)을 사용하여 책의 데모 애플리케이션 프레임워크를 기반으로 프로젝트를 처음부터 생성하고 빌드하는 방법을 설명합니다. 작동 예시로, [6장의 "Box"](#) 데모를 재현하고 빌드하는 방법을 보여드리겠습니다.

독자는 DirectX 애플리케이션 프로그래밍에 필요한 최신 버전의 DirectX SDK(<http://msdn.microsoft.com/directx/>에서 제공)를 이미 성공적으로 다운로드하고 설치한 것으로 가정합니다. SDK 설치의 간단하며 설치 마법사가 안내해 줄 것입니다.

Win32 프로젝트 생성

먼저 VS10을 실행한 후, 메인 메뉴에서 [그림 3](#)과 같이 **파일 > 새로 만들기 > 프로젝트를** 선택합니다.

새 프로젝트 대화 상자가 나타납니다([그림 4](#)). 왼쪽의 Visual C++ 프로젝트 유형 트리 컨트롤에서 Visual C++ > **Win32**를 선택합니다. 오른쪽에서 **Win32 프로젝트를** 선택합니다. 다음으로 프로젝트 이름을 지정하고 프로젝트 폴더를 저장할 위치를 지정합니다. 또한 기본적으로 선택되어 있는 경우 **솔루션 디렉터리 생성**을 선택 해제합니다. 이제 **확인**을 누릅니다.

새로운 대화 상자가 나타납니다. 왼쪽에는 개요(Overview)와 응용 프로그램 설정(Application Settings) 두 가지 옵션이 있습니다. **응용 프로그램 설정(Application Settings)**을 선택하면 [그림 5](#)에 표시된 대화 상자가 나타납니다. 여기서 **Windows 응용 프로그램**이 선택되었는지, **빈 프로젝트(Empty project)** 상자가 선택되었는지 확인하십시오. 이제 **완료(Finish)** 버튼을 누르십시오. 이 시점에서 빈 Win32 프로젝트를 성공적으로 생성했지만, DirectX 프로젝트 데모를 빌드하기 전에 수행해야 할 작업이 몇 가지 더 있습니다.

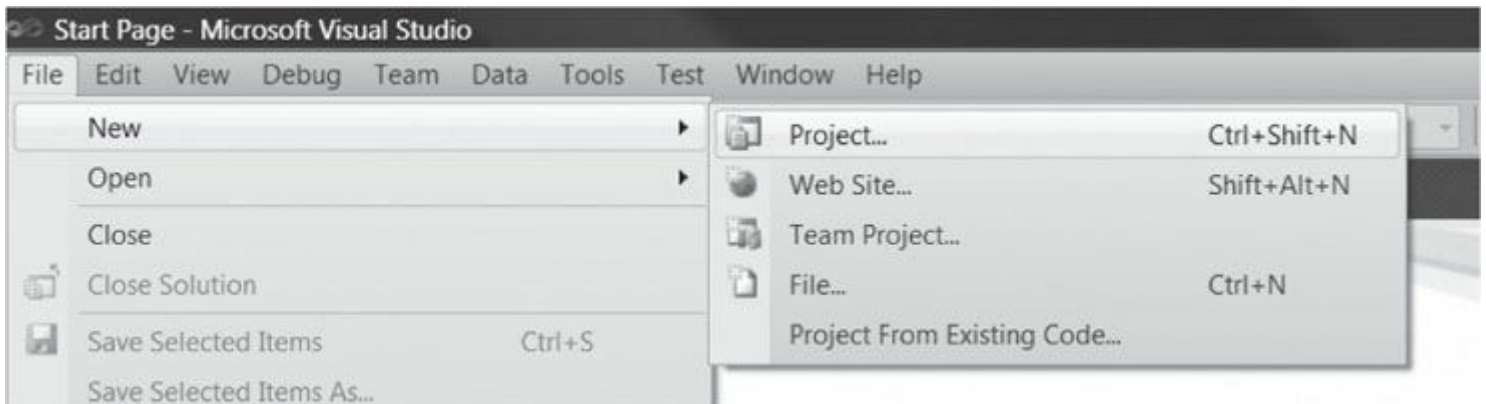


Figure 3. 새 프로젝트 생성.

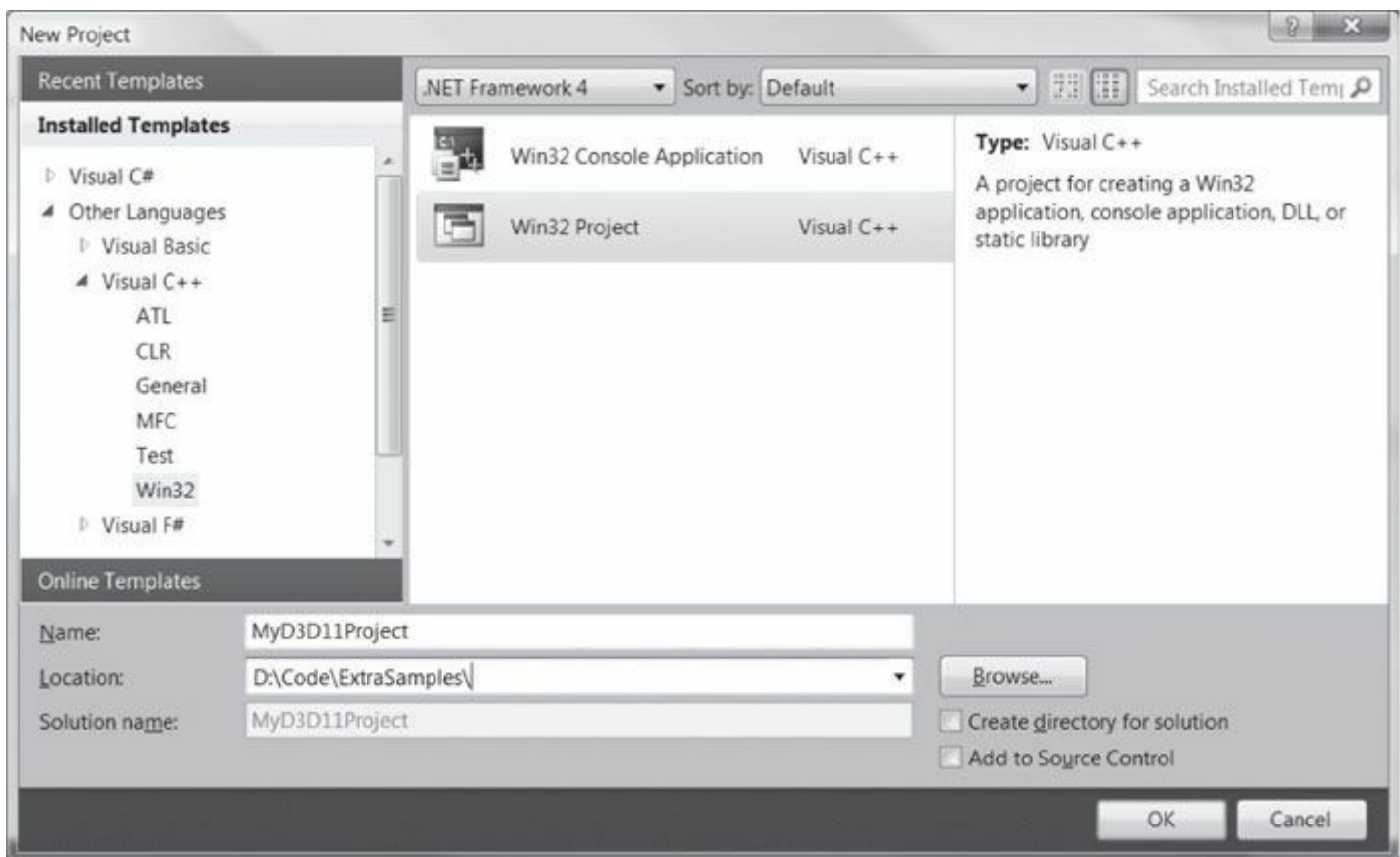


Figure 4. 새 프로젝트 설정.

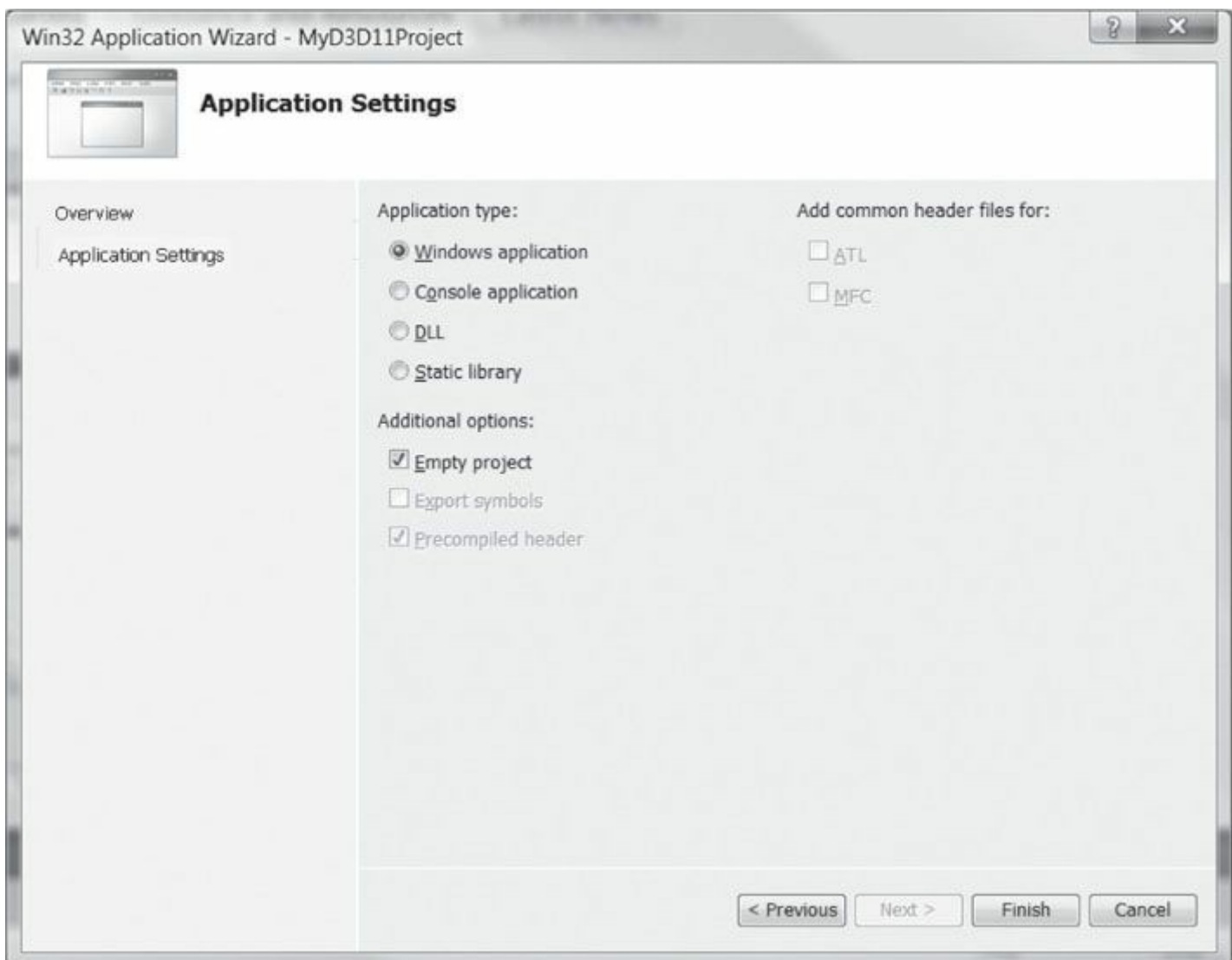


그림 5. 애플리케이션 설정.

DirectX 라이브러리 연결

이제 프로젝트에 DirectX 라이브러리를 연결해야 합니다. 디버그 빌드의 경우 다음 라이브러리를 추가하십시오:

```
d3dll.lib; d3dxl1d.lib;
D3DCompiler.lib;
Effects11d.lib;
dxerr.lib; dxgi.lib;
dxguid.lib;
```

릴리스 빌드 시에는 위와 동일한 라이브러리를 추가하되, **d3dxiid.iib** 및 Effects11.lib의 끝에 있는 'd' 문자를 제거하여

Effectsiid.iib의 끝 부분에 있는 'd' 문자를 제거하여 **d3dxii.iib**와 **Effects11.lib****로만** 남깁니다.

라이브러리 파일을 링크하려면 솔루션 탐색기에서 프로젝트 이름을 마우스 오른쪽 버튼으로 클릭하고 드롭다운 메뉴에서 **속성**을 선택합니다(그림 6). 그러면 **그림 7과 같은** 대화 상자가 표시됩니다. 왼쪽 트리 컨트롤에서 **구성 속성 > 링커 > 입력**을 선택합니다. 그런 다음 오른쪽의 **추가 종속성** 줄에 라이브러리 파일 이름을 지정합니다. **적용**을 누른 다음 **확인**을 누릅니다.

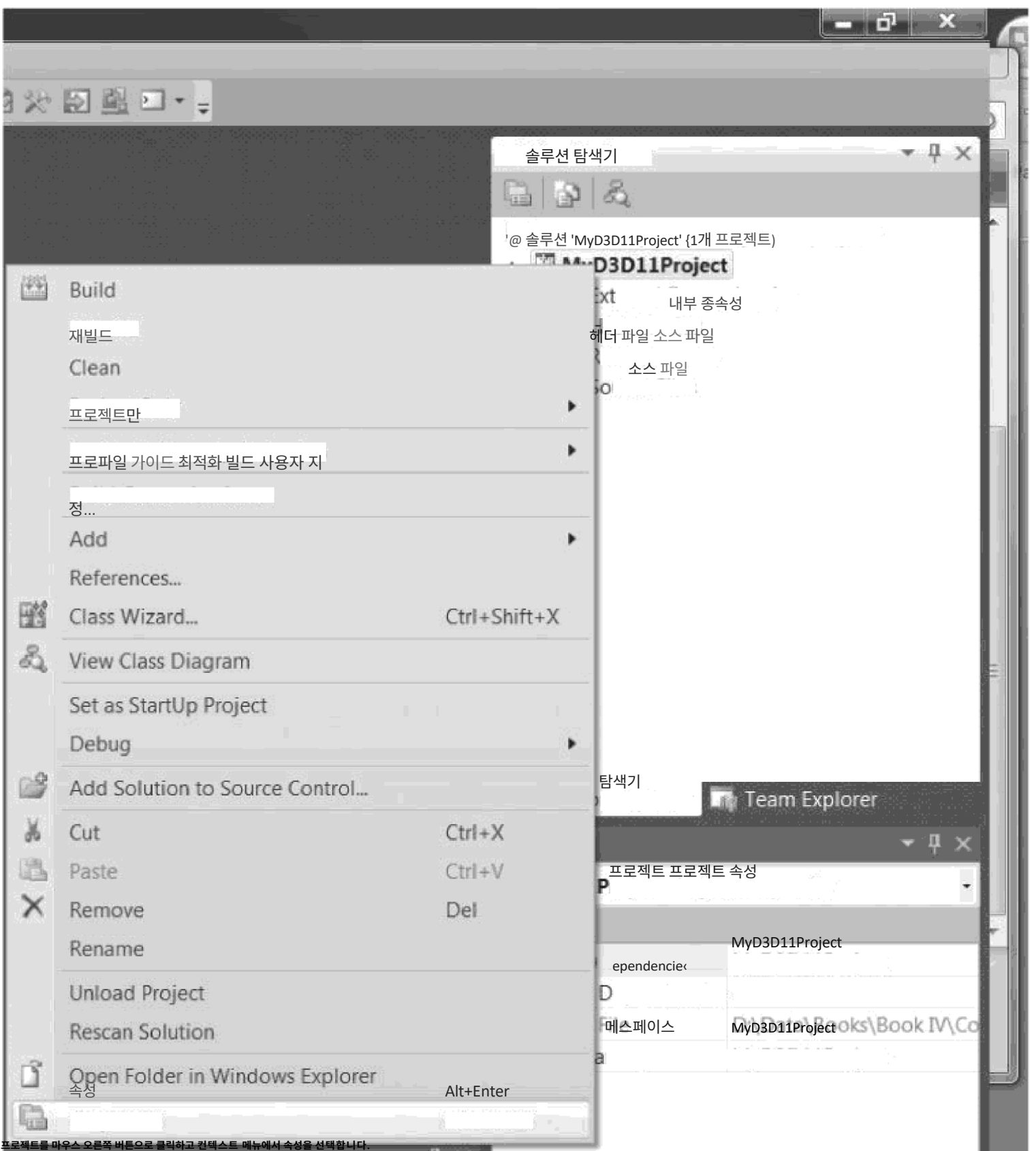
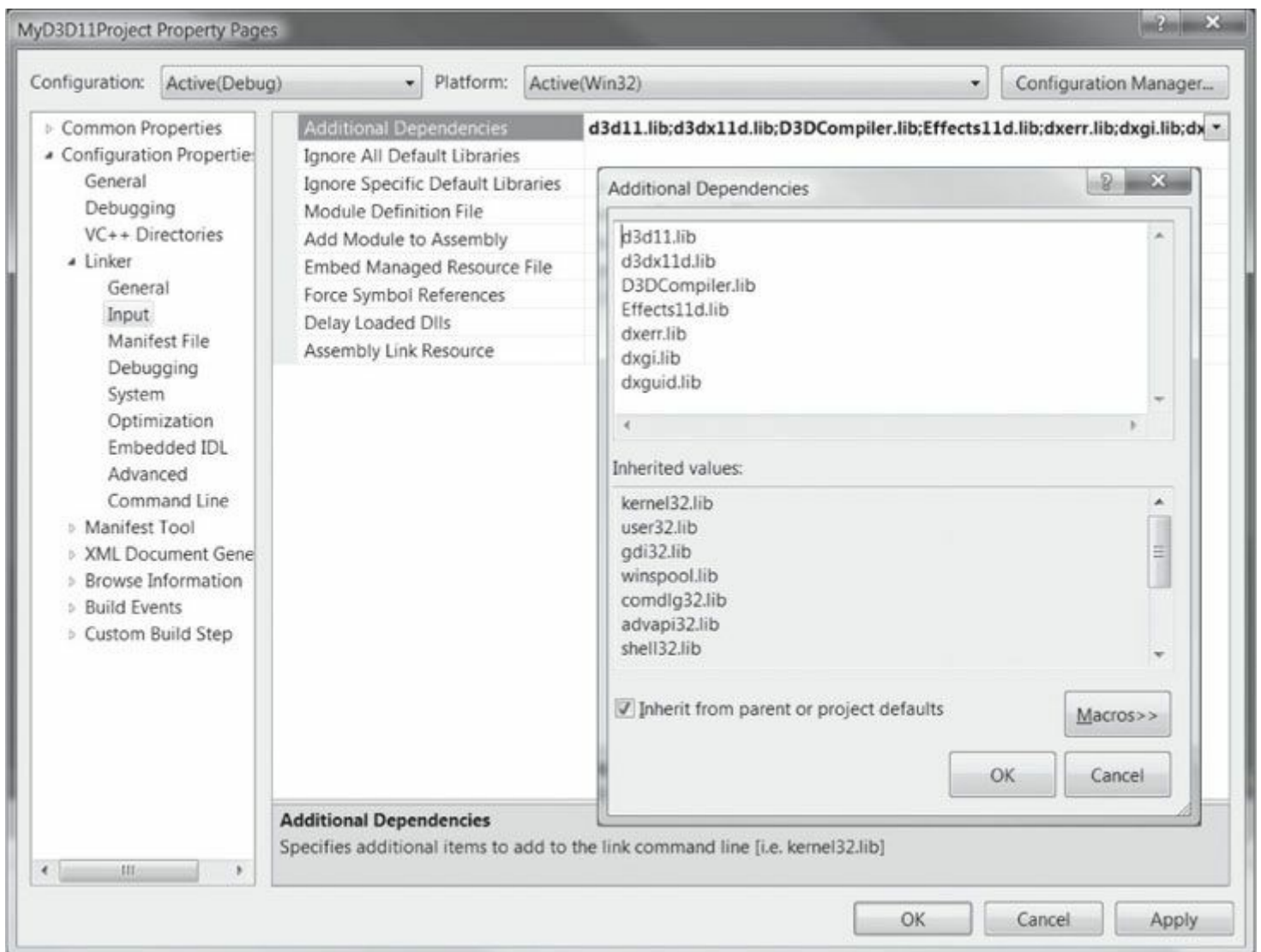


그림 6. 프로젝트를 마우스 오른쪽 버튼으로 클릭하고 컨텍스트 메뉴에서 속성을 선택합니다.



Fi 그림 7. DirectX 라이브러리를 연결합니다.

검색 경로 설정

이제 Visual Studio가 DirectX 헤더 및 라이브러리 파일을 검색할 디렉터리를 인식하도록 설정해야 합니다. 다시 솔루션 탐색기에서 프로젝트 이름을 마우스 오른쪽 버튼으로 클릭하고 드롭다운 메뉴에서 **속성**을 선택합니다(그림 6). 그러면 그림 7과 같은 대화 상자가 표시됩니다. 왼쪽 트리 컨트롤에서 **구성 속성 > VC++ 디렉터리**를 선택합니다. 그런 다음 오른쪽에서 **실행 파일 디렉터리, 포함 디렉터리 및 라이브러리 디렉터리**에 추가 항목을 추가해야 합니다(그림 8).

DirectX SDK의 정확한 경로는 설치 위치에 따라 다르며, 샘플 프로그램의 공통 디렉터리() 경로는 압축 해제 위치에 따라 달라진다는 점을 유의하십시오. 또한 공통 디렉터리를 이동하는 것은 자유롭지만, Visual Studio의 검색 경로를 해당 위치에 맞게 업데이트해야 합니다.

Common 디렉터리를 이동할 수 있지만, Visual Studio의 검색 경로를 그에 맞게 업데이트해야 합니다.

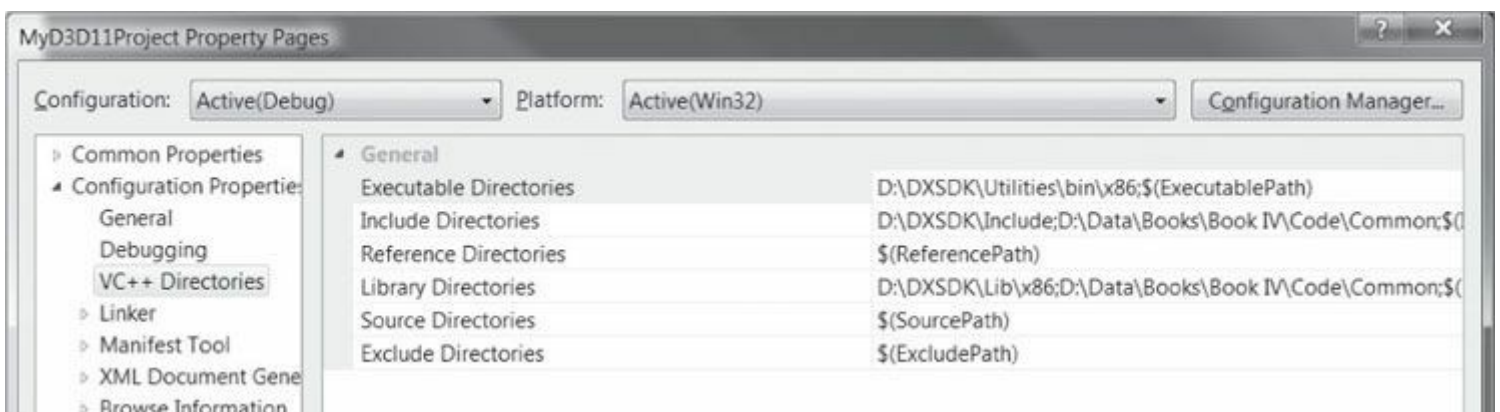
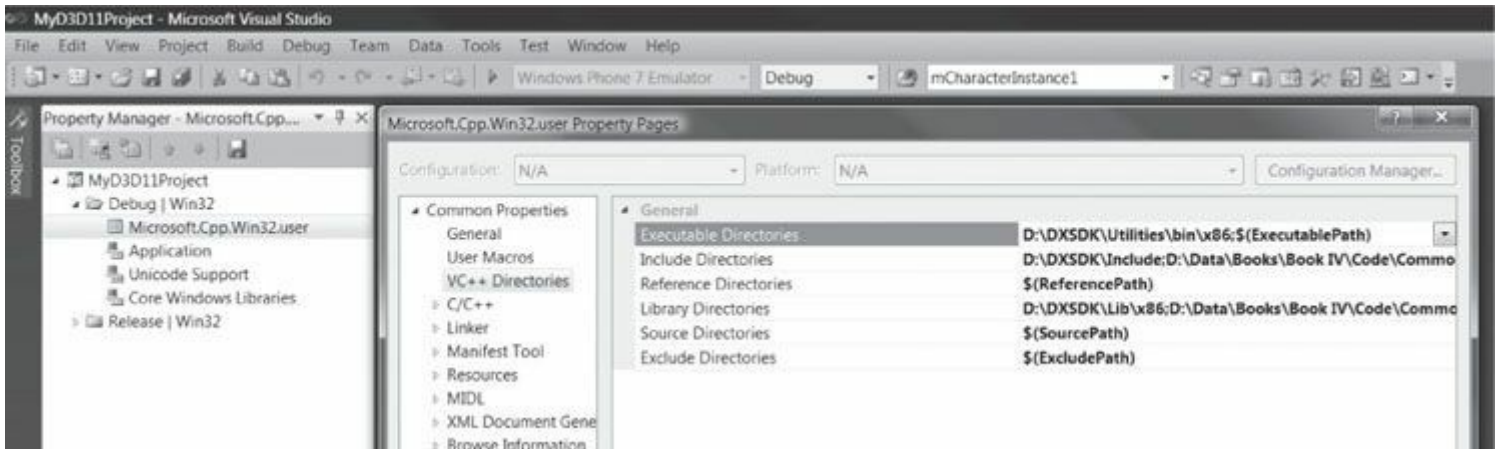


그림 8. 프로젝트별 검색 경로 설정.

Category	Additional Items
Executable Directories	1. D:\DXSDK\Utilities\bin\x86 – DirectX utility programs. In particular, we run the <i>fxc</i> utility program from the IDE.
Include Directories	1. D:\DXSDK\Include – DirectX header files. 2. D:\Data\Books\Book IV\Code\Common – Path to shared sample code for all of our demos (see “Note” following).
Library Directories	1. D:\DXSDK\Lib\x86 – DirectX library files. 2. D:\Data\Books\Book IV\Code\Common – Path to shared sample code for all of our demos (see “Note” following).

 책의 소스 코드를 다운로드하면 각 장별로 폴더가 생성되며, 공통 디렉터리(Common)가 포함됩니다. 공통 디렉터리에는 모든 데모에서 공유할 수 있는 재사용 가능한 프레임워크 코드가 포함되어 있어 프로젝트 간 소스 코드 중복을 방지합니다. VC++가 공통 폴더 내 헤더 파일과 라이브러리를 찾을 수 있도록 검색 경로에 해당 폴더를 추가해야 합니다.

Visual Studio 2010에서는 디렉터리 경로가 프로젝트별 설정으로 적용됩니다(이는 Visual Studio 2008의 동작과 다릅니다). 즉, 새 프로젝트를 생성할 때마다 각 프로젝트의 검색 경로를 설정해야 합니다. 데모 프로젝트를 많이 만드는 경우 이는 번거로울 수 있습니다. 그러나 이러한 설정을 사용자별로 영구적으로 적용하는 방법이 있습니다. Visual Studio 메뉴에서 보기 > 기타 창 > 속성 관리자를 선택합니다. [그림 9](#)와 같이 속성 관리자를 확장한 후 *Microsoft.Cpp.Win32 사용자* 항목을 더블클릭합니다. [그림 8](#)과 유사한 대화 상자가 표시되지만, 여기서 설정한 경로는 프로젝트 간에 유지되므로 매번 추가할 필요가 없습니다. 구체적으로, 새로 생성하는 모든 프로젝트는 여기서 설정한 값을 상속받게 됩니다.



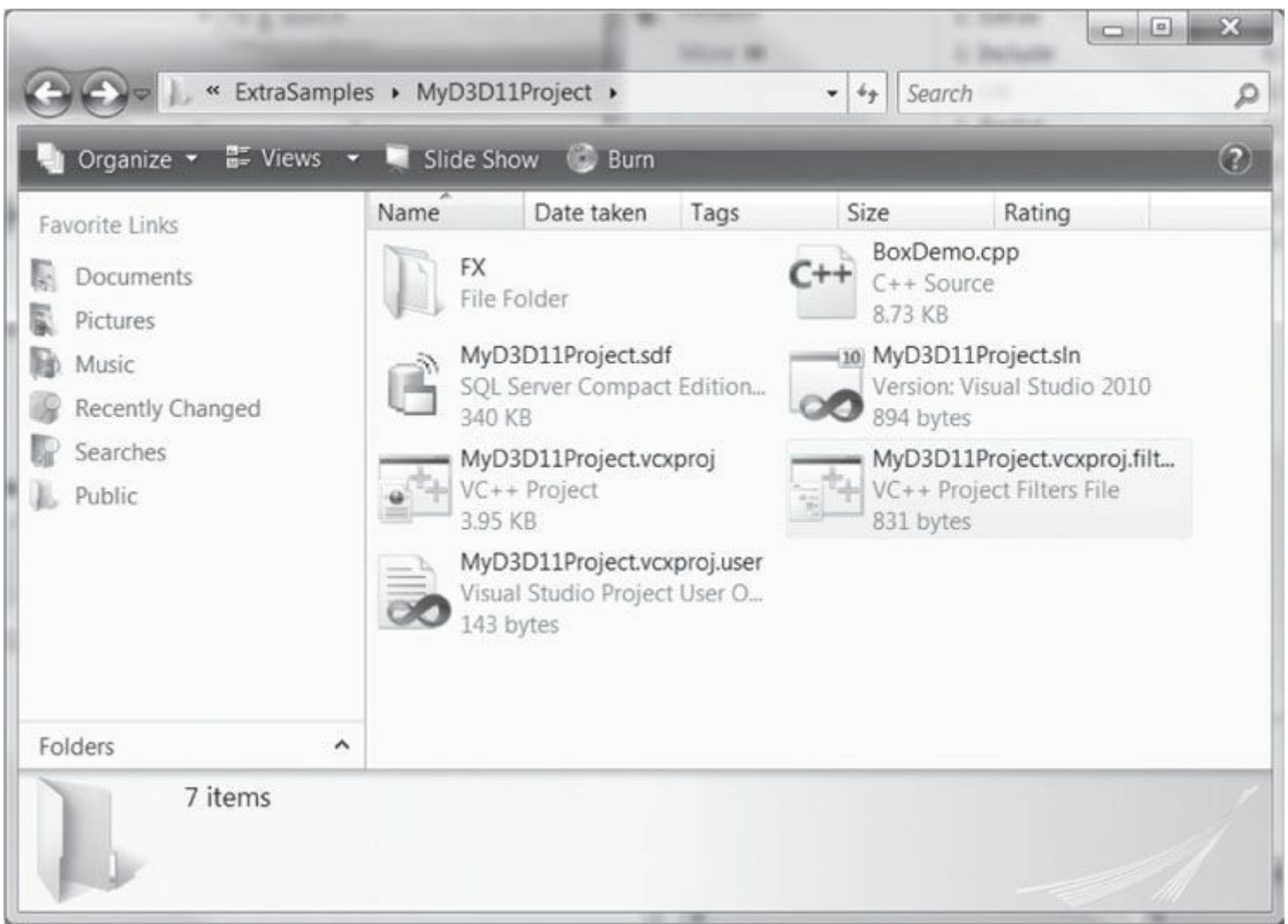
[그림 9.](#) 사용자별 검색 경로 설정.

소스 코드 추가 및 프로젝트 빌드

이제 프로젝트 설정이 완료되었습니다. 소스 코드 파일을 프로젝트에 추가하고 빌드할 수 있습니다. 먼저 "Box" 소스 코드 파일과 FX 폴더를 프로젝트 디렉터리에 복사하세요. 또한 이전 섹션에서 설명한 대로 *Common* 디렉터리를 하드 드라이브의 적절한 위치에 배치하고 해당 검색 경로를 추가했다고 가정합니다.

파일을 복사한 후 프로젝트 디렉터리는 [그림 10](#)과 같아야 합니다. 이제 다음 단계를 따라 프로젝트에 코드를 추가하세요.

1. 솔루션 탐색기에서 프로젝트 이름을 마우스 오른쪽 버튼으로 클릭하고 드롭다운 메뉴에서 **추가 > 기존 항목...**을 선택한 후 *BoxDemo.cpp*를 프로젝트에 추가합니다.
2. 솔루션 탐색기에서 프로젝트 이름을 마우스 오른쪽 버튼으로 클릭하고 **추가 > 새 필터**를 선택한 후 필터 이름을 *FX*로 지정합니다. FX 필터를 마우스 오른쪽 버튼으로 클릭하고 드롭다운 메뉴에서 **추가 > 기존 항목...**을 선택한 후 *FX\color.fx*를 프로젝트에 추가합니다.
3. 솔루션 탐색기에서 프로젝트 이름을 마우스 오른쪽 버튼으로 클릭하고 **추가 > 새 필터**를 선택한 후 필터 이름을 *Common*으로 지정합니다. *Common* 필터를 마우스 오른쪽 버튼으로 클릭하고 드롭다운 메뉴에서 **추가 > 기존 항목...**을 선택한 후, 책의 *Common* 디렉터리 코드를 저장한 위치로 이동하여 해당 디렉터리의 모든 *.h/.cpp* 파일을 프로젝트에 추가합니다.
4. 이제 소스 코드 파일이 프로젝트에 포함되었으며, 솔루션 탐색기는 [그림 11](#)과 같아야 합니다. 이제 메인 메뉴에서 **디버그 > 디버깅 시작**을 선택하여 데모를 컴파일, 링크 및 실행할 수 있습니다. [그림 12](#)의 애플리케이션이 표시되어야 합니다.



Fi 그림 10. 파일 복사 후 프로젝트 디렉터리.



Solution 'MyD3D11Project' (1 개 프로젝트) (1 project)

Header Files

- 4 공통
 - 5.3 Camera.cpp
 - Camera.h
 - 5.7 d3dApp.cpp
 - d3dApp.h
 - d3dUtil.cpp
 - d3dUtil.h
 - d3dxEffect.h
 - GameTimer.cpp
 - GameTimer.h
 - GeometryGenerator.cpp
 - GeometryGenerator.h
 - LightHelper.cpp
 - LightHelper.h
 - MathHelper.cpp
 - MathHelper.h
 - TextureMgr.cpp
 - TextureMgr.h
 - Waves.cpp
 - Waves.h

리소스 파일

1 소스 파일

BoxDemo.cpp

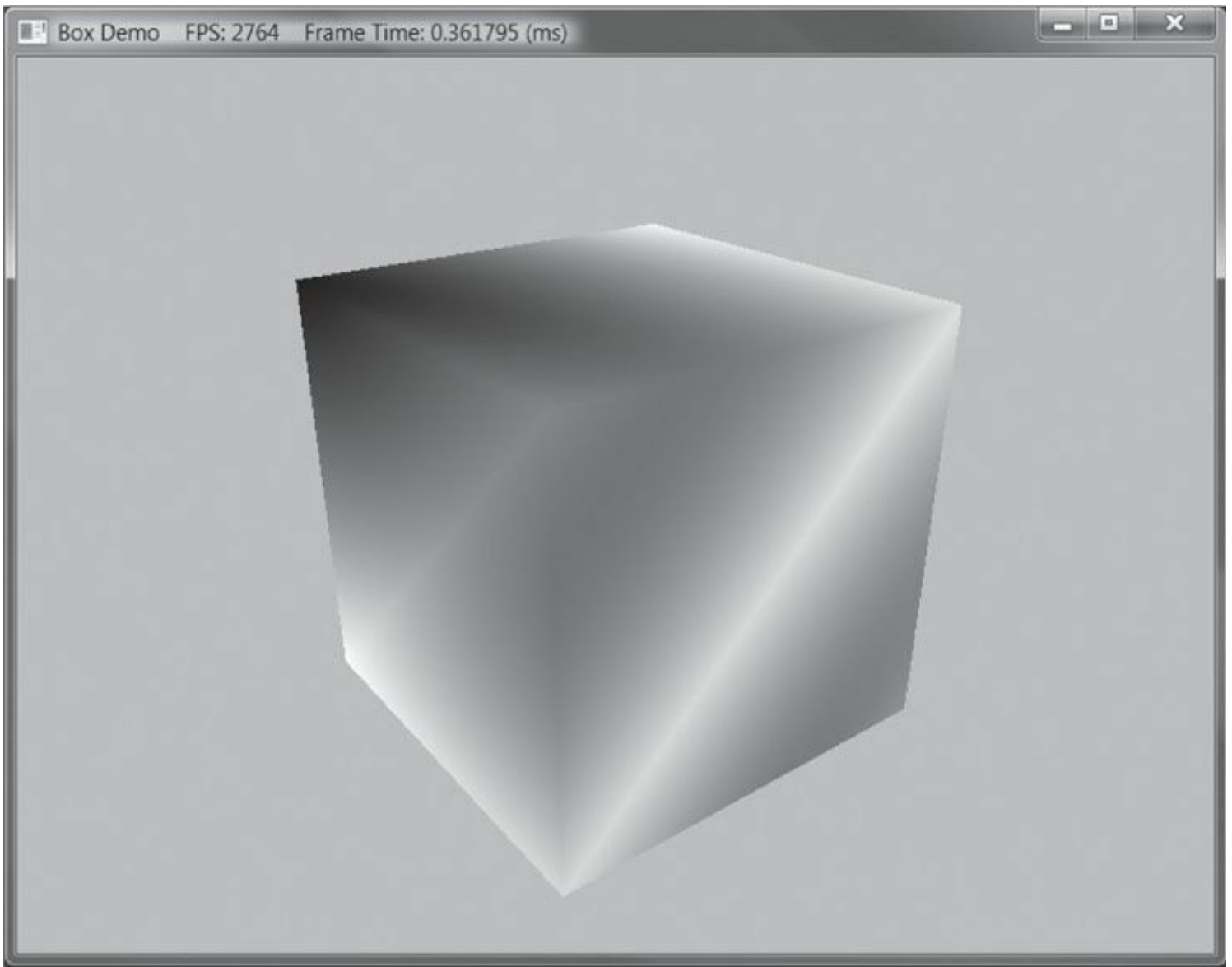
55 솔루션 탐색기

Resource Files

Source Files

BoxDemo.cpp

그림 11. 모든 파일을 추가한 후의 솔루션 탐색기.



Fi 그림 12. "Box" 데모의 스크린샷.

Note: Common 디렉토리의 많은 코드는 책의 진행 과정에 따라 점차 구축됩니다. 따라서 코드를 미리 살펴보지 않는 것이 좋습니다. 대신 해당 코드가 다루어지는 책의 해당 장을 읽을 때까지 기다리십시오.

다루는 장을 읽을 때까지 기다리시기 바랍니다.

다른 대부분의 샘플은 빌드 시점에 셰이더 컴파일러 도구를 호출합니다. 기존 **Note:** 데모를 실행하기 위해 이 작업을 수행할 필요는 없습니다. 해당 단계는 이미 구성되어 있기 때문입니다. 그러나 자체 프로젝트에서 이 작업을 수행하는 방법은

§6.9.5에서 설명합니다.

파트 1 수학적 전제 조건

"수학의 지식이 없이는 이 세상의 사물들을 알 수 없다."

로저 베이컨, 『대작(Opus Majus)』 제4부 Distinctia Prima 제1장, 1267년.

비디오 게임은 가상 세계를 시뮬레이션하려 시도한다. 그러나 컴퓨터는 본질적으로 숫자를 처리한다. 따라서 컴퓨터에 세계를 어떻게 전달할 것인가라는 문제가 발생한다. 그 해답은 우리의 세계와 그 안의 상호작용을 완전히 수학적으로 기술하는 것이다. 수학적으로 완전히 기술하는 것이다. 결과적으로 수학은 비디오 게임 개발에서 근본적인 역할을 한다. 이 선행 지식 부분에서는 이 책 전반에 걸쳐 사용될 수학적 도구를 소개한다. 벡터에 중점을 두고, 좌표계, 행렬, 변환은 이 책의 거의 모든 예제 프로그램에서 사용되는 도구이므로 반드시 숙지해야 합니다. 수학적 설명 외에도 XNA 수학 라이브러리의 관련 클래스와 함수에 대한 개요와 데모가 제공됩니다.

여기서 다루는 주제는 이 책의 나머지 부분을 이해하는 데 필수적인 것들뿐이며, 비디오 게임 수학에 대한 포괄적인 설명은 아닙니다. 이 주제를 다루는 책들이 따로 존재하기 때문입니다. 비디오 게임 수학에 대한 보다 완전한 참고 자료를 원하는 독자에게는 [Verth04]와 [Lengyel02]를 추천합니다.

제1장, 벡터 대수: 벡터는 아마도 컴퓨터 게임에서 사용되는 가장 기본적인 수학적 대상일 것입니다. 예를 들어, 우리는 벡터를 사용하여 위치, 변위, 방향, 속도 및 힘을 표현합니다. 이 장에서는 벡터와 이를 조작하는 데 사용되는 연산을 연구합니다.

제2장, 행렬 대수: 행렬은 변환을 효율적이고 간결하게 표현하는 방법을 제공합니다. 이 장에서는 행렬과 행렬에 정의된 연산에 익숙해집니다.

제3장, 변환: 이 장에서는 세 가지 기본 기하학적 변환인 크기 조정, 회전, 평행 이동을 살펴봅니다. 우리는 이러한 변환을 사용하여 공간에서 3차원 객체를 조작합니다. 또한 기하학을 나타내는 좌표를 한 좌표계에서 다른 좌표계로 변환하는 데 사용되는 좌표계 변환에 대해서도 설명합니다.

제1장 벡터 대수

벡터는 컴퓨터 그래픽스, 충돌 감지, 물리 시뮬레이션에서 핵심적인 역할을 하며, 이 모든 것은 현대 비디오 게임의 공통 요소입니다. 여기서는 비공식적이고 실용적인 접근법을 취합니다. 3D 게임/그래픽스 수학에 특화된 책으로는 [Verth04]를 추천합니다. 이 책의 거의 모든 데모 프로그램에서 벡터가 사용된다는 점을 강조함으로써 벡터의 중요성을 부각합니다.

- 목표:
- 1. 벡터가 기하학적으로 및 수치적으로 어떻게 표현되는지 학습한다.
 - 2. 벡터에 정의된 연산과 그 기하학적 응용을 발견한다.
 - 3. XNA Math 라이브러리의 벡터 함수와 클래스에 익숙해지기.

1.1 벡터

벡터는 크기와 방향을 모두 지닌 양을 의미합니다. 크기와 방향을 모두 지닌 양을 *벡터 값을 지닌 양*이라고 합니다. 벡터 값을 지닌 양의 예로는 힘(특정 방향과 특정 강도(크기)로 작용하는 힘), 변위(입자가 이동한 순방향과 거리), 속도(속도와 방향) 등이 있습니다. 따라서 벡터는 힘, 변위, 속도를 표현하는 데 사용됩니다. 또한 순수한 방향을 지정하는 데에도 벡터를 사용합니다. 예를 들어 3D 게임에서 플레이어가 바라보는 방향, 다각형이 향하는 방향, 광선이 이동하는 방향, 또는 광선이 표면에서 반사되는 방향 등이 있습니다.

벡터를 수학적으로 특성화하는 첫 단계는 기하학적 접근입니다. 우리는 방향을 지닌 선분으로 벡터를 그래픽적으로 명시합니다. (그림 1.1 참조), 여기서 길이는 벡터의 크기를 나타내고 방향은 벡터의 방향을 나타낸다. 벡터를 그리는 위치는 중요하지 않다는 점을 유의하자. 위치를 변경해도 크기와 방향(벡터가 지닌 두 속성)은 변하지 않기 때문이다. 따라서 두 벡터는 길이가 같고 방향이 같을 때만 서로 같다고 말합니다. 따라서 그림 1.1a에 그려진 벡터 u 와 v 는 길이가 같고 방향이 같으므로 실제로 같습니다. 사실 벡터에 있어 위치는 중요하지 않으므로, 우리는 항상 벡터를 이동시켜도 그 의미를 바꾸지 않을 수 있습니다(이동은 길이와 방향을 모두 바꾸지 않기 때문입니다). $u(u)$ 를 이동시켜 v 와 완전히 겹치게 할 수 있으며(반대의 경우도 마찬가지), 이로 인해 둘을 구분할 수 없게 됨으로써 이동식이 성립함을 알 수 있다. 물리적 예시로, 그림의 벡터 u 와 v 는

1.1b 두 지점 A 와 B 에 있는 개미들에게 각각 현재 위치에서 북쪽으로 10미터 이동하라고 지시합니다. 여기서도 $u = v$ 입니다. 벡터 자체는 위치와 무관하며, 단순히 개미들이 현재 위치에서 어떻게 이동할지 지시할 뿐입니다. 이 예시에서는 개미들에게 북쪽(방향)으로 10미터(길이) 이동하라고 지시합니다.

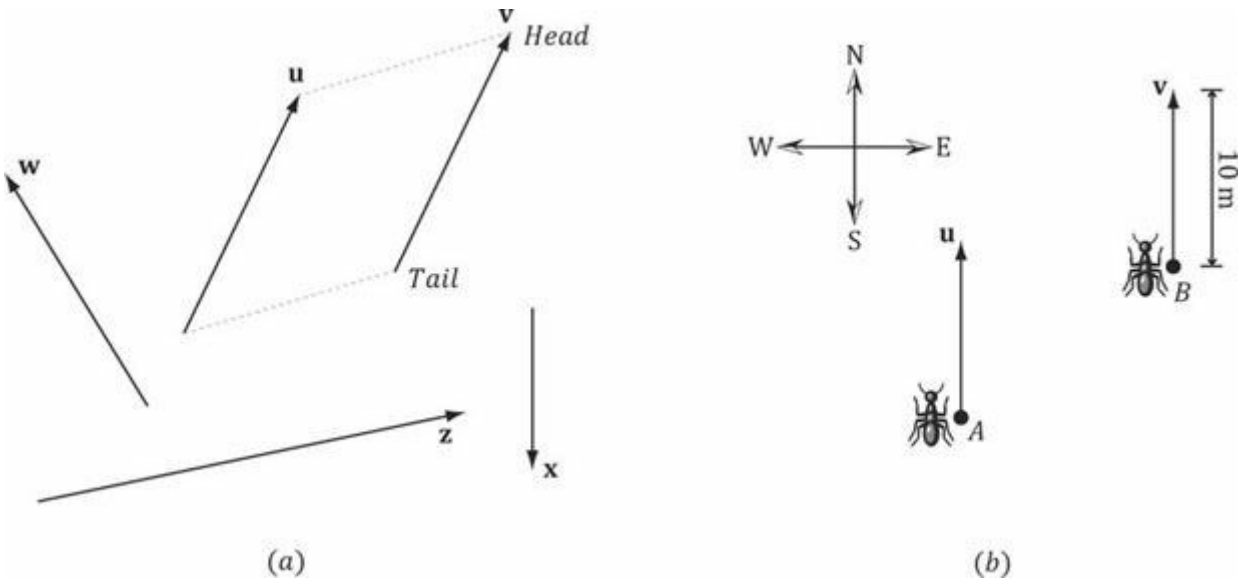


그림 1.1. (a) 2차원 평면에 그려진 벡터들. (b) 개미들에게 북쪽으로 10미터 이동하라고 지시하는 벡터들.

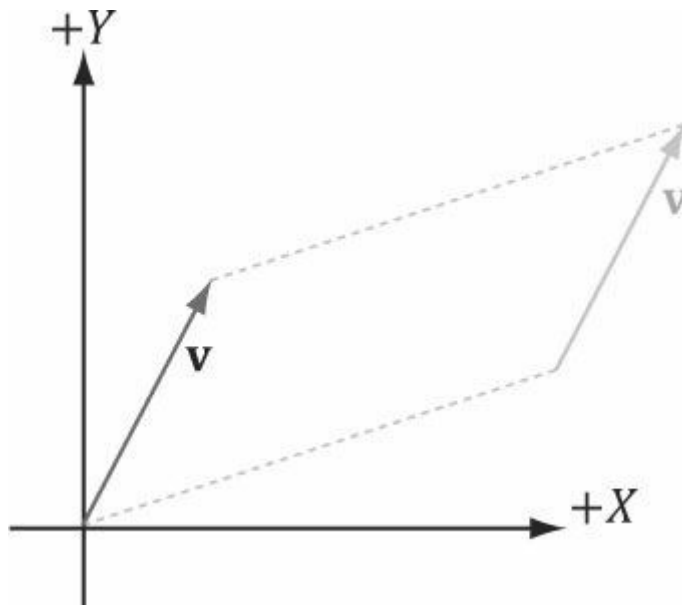


그림 1.2. 벡터 v 의 꼬리가 좌표계의 원점과 일치하도록 이동시킨다. 벡터의 꼬리가 원점과 일치할 때, 우리는 이를 표준 위치에 있다고 말한다.

1.1.1 벡터와 좌표계

이제 벡터에 대한 유용한 기하학적 연산을 정의할 수 있으며, 이를 통해 벡터 값을 갖는 양과 관련된 문제를 해결할 수 있습니다. 그러나 컴퓨터는 벡터를 기하학적으로 처리할 수 없기 때문에, 대신 벡터를 수치적으로 지정하는 방법을 찾아야 합니다. 따라서 공간에 3차원 좌표계를 도입하고, 모든 벡터의 꼬리(초점)가 원점과 일치하도록 변환합니다(그림 1.2). 그러면 벡터의 머리(시작점) 좌표를 지정하여 벡터를 식별할 수 있으며, 그림 1.3과 같이 $v = (x, y, z)$ 로 표기할 수 있습니다. 이제 컴퓨터 프로그램에서 벡터를 세 개의 부동 소수점 숫자로 표현할 수 있습니다.

2차원 작업 시에는 2차원 좌표계를 사용하며 벡터는 두 개의 좌표만 가집니다: $v = (x, y)$. 따라서 컴퓨터 프로그램에서 벡터를 두 개의 부동 소수점 수로 표현할 수 있습니다.

그림 1.4를 고려하자. 이 그림은 벡터 v 와 공간 내 두 개의 좌표계를 보여준다. (이 책에서는 '좌표계', '기준 좌표계', '공간', '좌표계'라는 용어를 모두 동일한 의미로 사용함을 유의하라.) 벡터 v 를 두 좌표계 중 어느 하나에서 표준 위치에 오도록 변환할 수 있다. 그러나 벡터 v 의 좌표계 A 에 대한 좌표는 좌표계 B 에 대한 좌표와 다르다는 점을 유의하라. 즉, 동일한 벡터 v 도 서로 다른 좌표계에서는 다른 좌표 표현을 가진다.

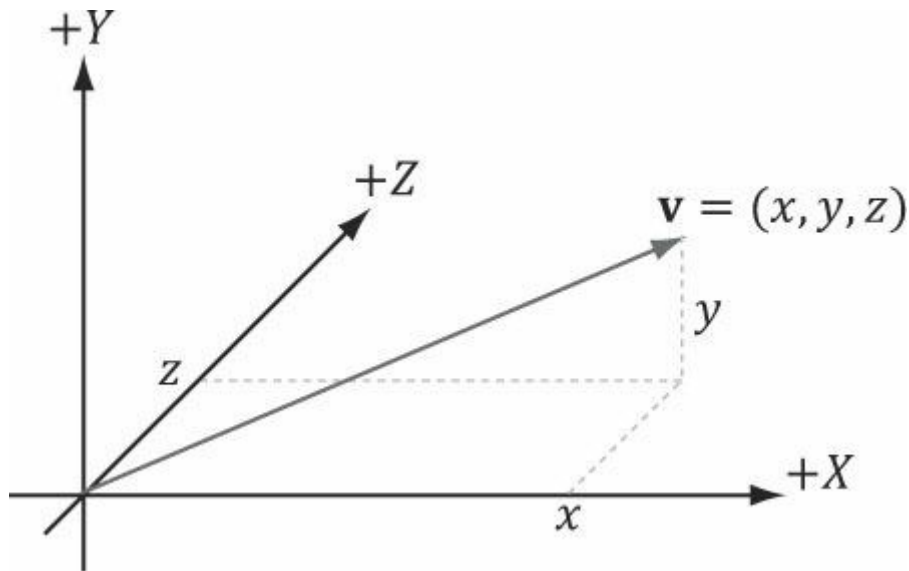


그림 1.3. 좌표계에 대한 상대적 좌표로 지정된 벡터.

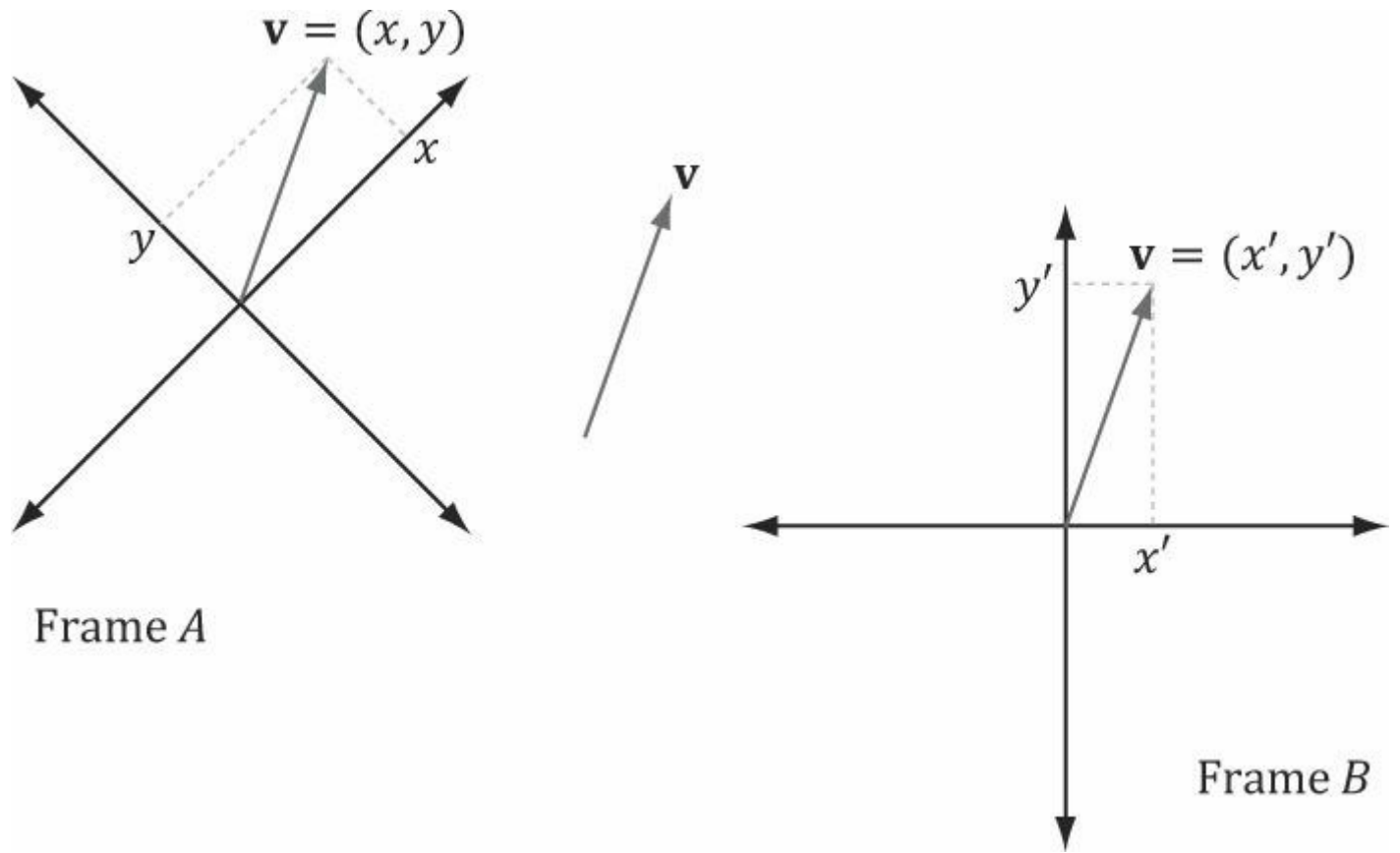


그림 1.4. 서로 다른 좌표계에 대해 설명될 때 동일한 벡터 $\mathbf{v}$ 가 다른 좌표를 가짐.

이 개념은 예를 들어 온도와 유사합니다. 물은 섭씨 100도 또는 화씨 212도에서 끓습니다. 물의 끓는점이라는 물리적 온도는 척도(즉, 다른 척도를 선택한다고 해서 끓는점을 낮출 수 없음)에 관계없이 동일하지만, 우리는 사용하는 척도에 따라 온도에 다른 스칼라 수를 할당합니다. 마찬가지로 벡터의 경우, 방향과 크기(유클리드 선분 내에 내재됨)는 변하지 않습니다. 단지 이를 설명하는 데 사용하는 기준계에 따라 좌표만 달라질 뿐입니다. 이는 벡터를 좌표로 식별할 때마다 해당 좌표가 특정 기준계에 상대적임을 의미하므로 중요합니다. 3차원 컴퓨터 그래픽스에서는 종종 하나 이상의 기준계를 활용하게 되므로, 벡터의 좌표가 어느 기준계에 상대적인지 추적해야 합니다. 또한 한 기준계에서 다른 기준계로 벡터 좌표를 변환하는 방법을 알아야 합니다.

벡터와 점 모두 좌표계에 대한 좌표 (x, y, z) 로 표현될 수 있음을 알 수 있습니다. 그러나 이 둘은 동일하지 않습니다(Note:); 점은 3차원 공간에서의 위치를 나타내는 반면, 벡터는 크기와 방향을 나타냅니다.

점들에 대해서는 §1.5에서 더 자세히 다루겠습니다.

1.1.2 좌손 좌표계 대 우손 좌표계

Direct3D는 소위 왼손 좌표계를 사용합니다. 왼손을 잡고 손가락을 양의 x 축 방향으로 향하게 한 다음, 손가락을 양의 y 축 방향으로 말아 올리면 엄지손가락이 대략 양의 z 축 방향을 가리킵니다. 그림 1.5는 왼손 좌표계와 오른손 좌표계의 차이점을 보여줍니다.

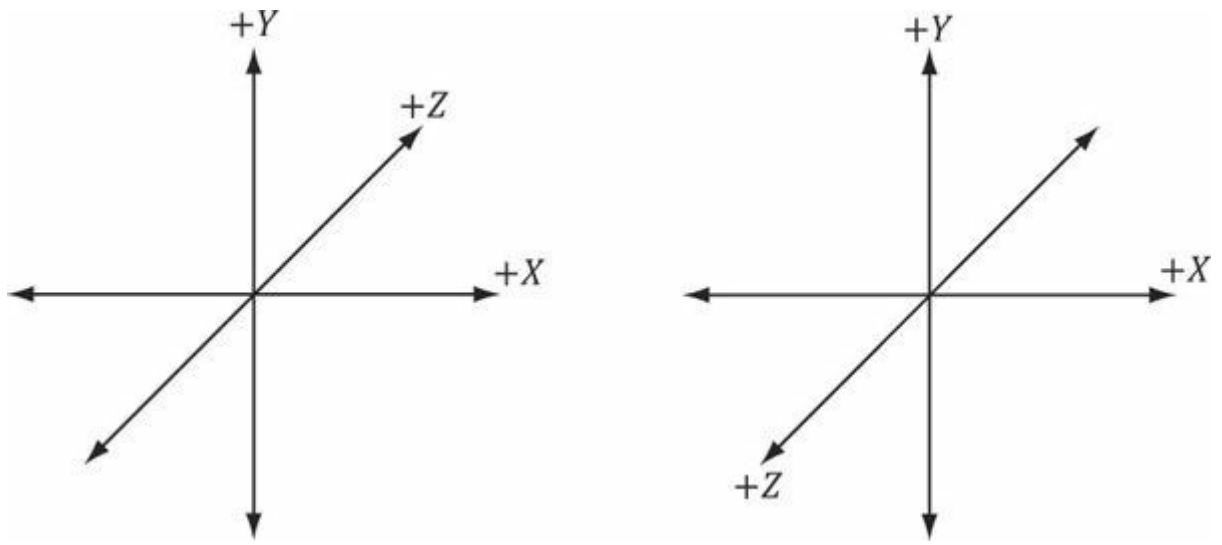


그림 1.5. 왼쪽은 왼손 좌표계입니다. 양의 z 축이 페이지 안쪽으로 향하는 것을 확인하세요. 오른쪽은 오른손 좌표계입니다. 양의 z 축이 페이지 밖으로 나오는 것을 확인하세요.

오른손 좌표계에서, 오른손을 들어 손가락을 양의 x 축 방향으로 향하게 한 다음, 손가락을 양의 y 축 방향으로 말아 올리면 엄지손가락이 대략 양의 z 축 방향을 가리킵니다.

1.1.3 벡터의 기본 연산

이제 좌표 표현을 사용하여 벡터에 대한 등호, 덧셈, 스칼라 곱셈, 뺄셈을 정의합니다. 이 네 가지 정의를 위해 $\mathbf{u} = (u_x, u_y, u_z)$ 및 $\mathbf{v} = (v_x, v_y, v_z)$ 라고 합시다.

1. 두 벡터는 그 대응하는 성분들이 같을 때만 같다. 즉, $\mathbf{u} = \mathbf{v}$ 는 $u_x = v_x, u_y = v_y, u_z = v_z$ 이다.
2. 벡터는 성분별로 더합니다: $\mathbf{u} + \mathbf{v} = (u_x + v_x, u_y + v_y, u_z + v_z)$. 동일한 차원의 벡터만 더하는 것이 의미가 있음을 유의하십시오.
3. 스칼라(즉, 실수)와 벡터를 곱하면 그 결과는 벡터가 된다. k 를 스칼라라 하면, $k\mathbf{u} = (ku_x, ku_y, ku_z)$ 이다. 이를 *스칼라 곱셈*이라 한다.
4. 벡터 덧셈과 스칼라 곱셈을 통해 뺄셈을 정의합니다. 즉, $\mathbf{u} - \mathbf{v} = \mathbf{u} + (-1 \cdot \mathbf{v}) = \mathbf{u} + (-\mathbf{v}) = (u_x - v_x, u_y - v_y, u_z - v_z)$.

예제 1.1

$\mathbf{u} = (1, 2, 3), \mathbf{v} = (1, 2, 3), \mathbf{w} = (3, 0, -2), k = 2$ 라 하자. 그러면,

1. $\mathbf{u} + \mathbf{w} = (1, 2, 3) + (3, 0, -2) = (4, 2, 1)$;
2. $\mathbf{u} = \mathbf{v}$;
3. $\mathbf{u} - \mathbf{v} = \mathbf{u} + (-\mathbf{v}) = (1, 2, 3) + (-1, -2, -3) = (0, 0, 0) = \mathbf{0}$;
4. $k\mathbf{w} = 2(3, 0, -2) = (6, 0, -4)$.

세 번째 항의 차이는 *제로 벡터*라고 불리는 특별한 벡터를 나타내며, 이 벡터는 모든 성분이 0이고 0으로 표기됩니다.

예제 1.2

이 예시를 2차원 벡터로 설명하면 그림을 더 간단하게 그릴 수 있습니다. 개념은 3차원과 동일하며, 2차원에서는 성분이 하나 적을 뿐입니다.

1. 벡터 $\mathbf{v} = (2, 1)$ 을 고려하자. $\mathbf{v}$ 와 벡터 $-\frac{1}{2}\mathbf{v}$ 는 기하학적으로 어떻게 비교될까? $-\frac{1}{2}\mathbf{v}$ 임을 알 수 있다. $\mathbf{v}$ 와 $-\frac{1}{2}\mathbf{v}$ 를 모두 그래프화하면(그림 1.6a), $-\frac{1}{2}\mathbf{v}$ 가 $\mathbf{v}$ 와 정반대 방향이며 길이는 $\mathbf{v}$ 의 1/2임을 알 수 있다. 따라서 기하학적으로 벡터의 부호 반전은 방향을 "뒤집는" 것으로, 스칼라 곱셈은 벡터 길이를 "확대/축소"하는 것으로 생각할 수 있다.
2. $\mathbf{u} = (2, \frac{1}{2})$ 이고 $\mathbf{v} = (1, 2)$ 라고 하자. 그러면 $\mathbf{u} + \mathbf{v} = (3, \frac{5}{2})$ 이다. 그림 1.6b는 벡터 덧셈이 기하학적으로 무엇을 의미하는지 보여준다: $\mathbf{u}$ 를 평행 이동시켜 그 꼬리/가 $\mathbf{v}$ 의 머리 부분과 일치하도록 한다. 그러면 합은 $\mathbf{v}$ 의 꼬리에서 시작하여 이동된 $\mathbf{u}$ 의 머리 부분에서 끝나는 벡터이다. ($\mathbf{u}$ 를 고정하고 $\mathbf{v}$ 를 이동시켜 그 꼬리가 $\mathbf{u}$ 의 머리와 일치하도록 해도 동일한 결과를 얻습니다. 이 경우 $\mathbf{u} + \mathbf{v}$ 는 $\mathbf{u}$ 의 꼬리에서 시작하여 이동된 $\mathbf{v}$ 의 머리에서 끝나는 벡터가 됩니다.) 또한 우리의

벡터 덧셈의 법칙은 힘을 합쳐 합력을 생성할 때 물리적으로 직관적으로 예상되는 결과와 일치합니다: 동일한 방향의 두 힘(벡터)을 더하면 그 방향으로 더 강한 합력(더 긴 벡터)이 생성됩니다. 서로 반대 방향의 두 힘(벡터)을 더하면 더 약한 합력(더 짧은 벡터)이 생성됩니다. 그림 1.7은 이러한 개념을 보여줍니다.

3. $\mathbf{u} = \left(2, \frac{1}{2}\right)$ 이고 $\mathbf{v} = (1, 2)$ 라고 하자. 그러면 $\mathbf{v} - \mathbf{u} = \left(-1, \frac{3}{2}\right)$ 이다. 그림 1.6c는 벡터 뺄셈이 기하학적으로 무엇을 의미하는지 보여준다. 본질적으로, 차이 $\mathbf{v} - \mathbf{u}$ 는 $\mathbf{u}$ 의 벡터 머리에서 $\mathbf{v}$ 의 벡터 머리로 향하는 벡터를 나타냅니다. $\mathbf{u}$ 와 $\mathbf{v}$ 를 점으로 해석한다면, $\mathbf{v} - \mathbf{u}$ 는 점 $\mathbf{u}$ 에서 점 $\mathbf{v}$ 로 향하는 벡터를 나타냅니다. 이 해석은 한 점에서 다른 점으로 향하는 벡터를 자주 원하게 될 것이므로 중요합니다. 또한 $\mathbf{u}$ 와 $\mathbf{v}$ 를 점으로 생각할 때, $\mathbf{v} - \mathbf{u}$ 의 길이는 $\mathbf{u}$ 에서 $\mathbf{v}$ 까지의 거리임을 유의하십시오.

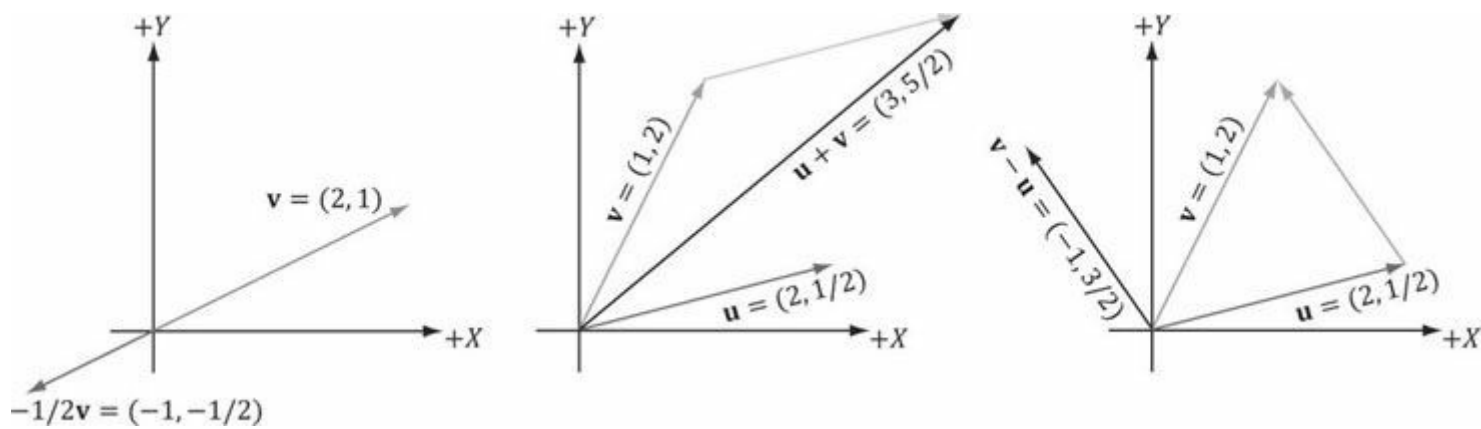


그림 1.6. (a) 스칼라 곱셈의 기하학적 해석. (b) 벡터 덧셈의 기하학적 해석. (c) 벡터 뺄셈의 기하학적 해석.

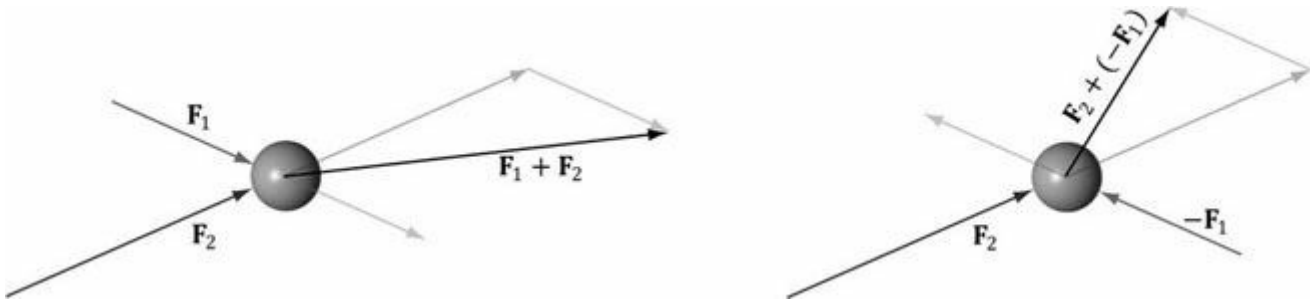


그림 1.7. 공에 작용하는 힘들. 벡터 덧셈을 사용하여 힘을 합쳐 합력을 구한다.

1.2 벡터의 길이 및 단위 벡터

기하학적으로 벡터의 크기는 방향이 지정된 선분의 길이이다. 벡터의 크기는 이중 수직 막대기(예: $\|\mathbf{u}\|$ 는 $\mathbf{u}$ 의 크기를 나타냄). 이제 벡터 $\mathbf{u} = (x, y, z)$ 가 주어졌을 때, 대수적으로 그 크기를 계산하고자 한다. 3차원 벡터의 크기는 피타고라스 정리를 두 번 적용하여 계산할 수 있다; 그림 1.8을 참조하라.

먼저 xz 평면 상에 변 x, z 와 빗변 a 를 가진 삼각형을 살펴봅시다. 피타고라스 정리에 따라 다음이 성립합니다.

$a = \sqrt{x^2 + z^2}$. 이제 변 a, y , 빗변 $\|\mathbf{u}\|$ 를 가진 삼각형을 살펴보자. 다시 피타고라스 정리를 적용하면 다음과 같은 크기 공식에 도달한다:

$$\|\mathbf{u}\| = \sqrt{y^2 + a^2} = \sqrt{y^2 + \left(\sqrt{x^2 + z^2}\right)^2} = \sqrt{x^2 + y^2 + z^2} \tag{식 1.1}$$

일부 응용 분야에서는 벡터의 길이를 고려하지 않습니다. 공간의 방향을 표현하기 위해 벡터를 사용하기 때문입니다. 이러한 경우 단위 벡터의 길이가 정확히 1이 되도록 합니다. 벡터를 단위 길이로 만들 때 우리는 벡터를 정규화한다고 말합니다. 각 성분을 그 크기로 나누어 벡터를 정규화할 수 있습니다:

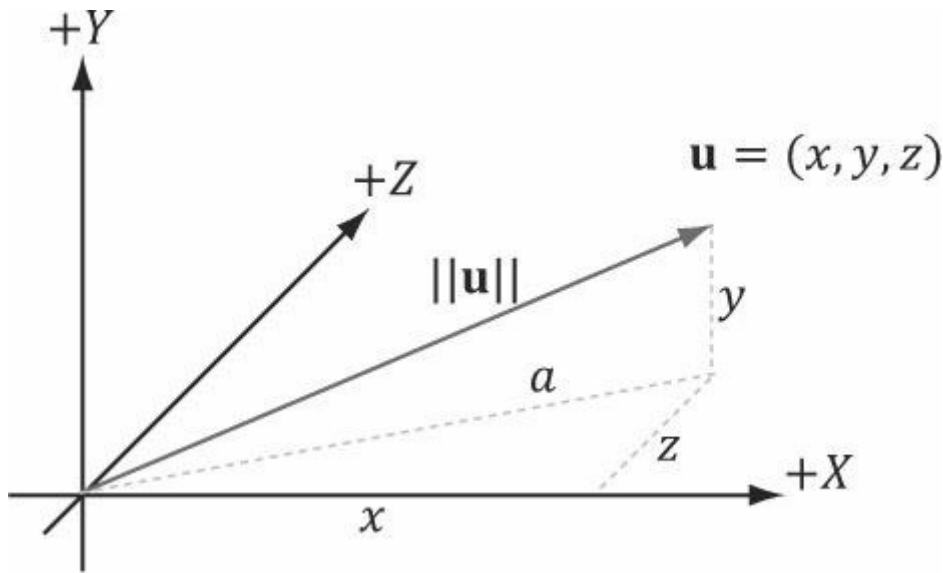


그림 1.8. 벡터의 3차원 길이는 피타고라스 정리를 두 번 적용하여 계산할 수 있다.

$$\hat{\mathbf{u}} = \frac{\mathbf{u}}{\|\mathbf{u}\|} = \left(\frac{x}{\|\mathbf{u}\|}, \frac{y}{\|\mathbf{u}\|}, \frac{z}{\|\mathbf{u}\|} \right) \quad (\text{식 1.2})$$

이 공식이 올바른지 확인하기 위해 $\hat{\mathbf{u}}$ 의 길이를 계산해 볼 수 있습니다:

$$\|\hat{\mathbf{u}}\| = \sqrt{\left(\frac{x}{\|\mathbf{u}\|}\right)^2 + \left(\frac{y}{\|\mathbf{u}\|}\right)^2 + \left(\frac{z}{\|\mathbf{u}\|}\right)^2} = \frac{\sqrt{x^2 + y^2 + z^2}}{\sqrt{\|\mathbf{u}\|^2}} = \frac{\|\mathbf{u}\|}{\|\mathbf{u}\|} = 1$$

따라서 $\hat{\mathbf{u}}$ 는 실제로 단위 벡터입니다.

예제 1.3

정규화 정규화 벡터 $\mathbf{v}$ = $(-1, 3, 4)$. 우리는 $\|\mathbf{v}\| = \sqrt{(-1)^2 + 3^2 + 4^2} = \sqrt{26}$ 따라서,

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|} = \left(-\frac{1}{\sqrt{26}}, \frac{3}{\sqrt{26}}, \frac{4}{\sqrt{26}} \right)$$

이 $\|\hat{\mathbf{v}}\|$ 이 실제로 단위 벡터임을 확인하기 위해 그 길이를 계산합니다:

$$\|\hat{\mathbf{v}}\| = \sqrt{\left(-\frac{1}{\sqrt{26}}\right)^2 + \left(\frac{3}{\sqrt{26}}\right)^2 + \left(\frac{4}{\sqrt{26}}\right)^2} = \sqrt{\frac{1}{26} + \frac{9}{26} + \frac{16}{26}} = \sqrt{1} = 1$$

1.3 내적

내적은 벡터 곱셈의 한 형태로 스칼라 값을 산출합니다. 이 때문에 스칼라 곱이라고도 불립니다. $\mathbf{u} = (u_x, u_y, u_z)$ 및 $\mathbf{v} = (v_x, v_y, v_z)$ 라고 할 때, 내적은 다음과 같이 정의됩니다:

$$\mathbf{u} \cdot \mathbf{v} = u_x v_x + u_y v_y + u_z v_z \quad (\text{식 1.3})$$

말로 표현하자면, 내적은 대응하는 성분들의 곱의 합이다.

내적의 정의는 명백한 기하학적 의미를 제시하지 않습니다. 코사인 법칙(연습문제 10 참조)을 사용하면 다음과 같은 관계를 구할 수 있습니다:

$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta \quad (\text{식 1.4})$$

여기서 θ 는 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 사이의 각도로, $0 \leq \theta \leq \pi$ 를 만족한다; 그림 1.9를 참조하라. 따라서 식 1.4는 두 벡터 사이의 내적이 그들 사이의 각도의 코사인에 벡터들의 크기로 스케일링된 값임을 말한다. 특히, $\mathbf{u}$ 와 $\mathbf{v}$ 가 모두 단위 벡터라면, $\mathbf{u} \cdot \mathbf{v}$ 는 그들 사이의 각도의 코사인이다(즉, $\mathbf{u} \cdot \mathbf{v} = \cos \theta$).

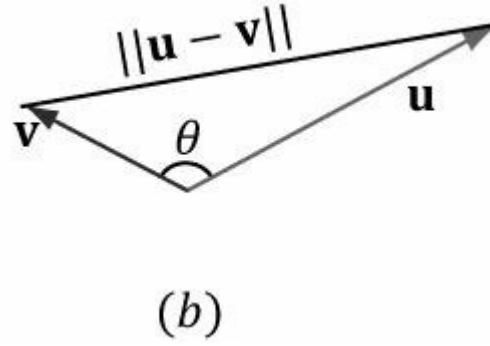
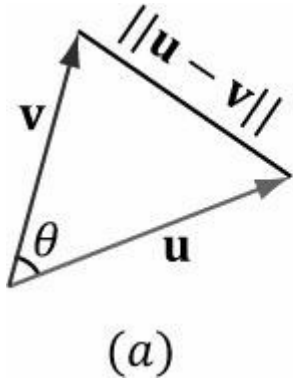


그림 1.9. 왼쪽 그림에서 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 사이의 각도 θ 는 예각이다. 오른쪽 그림에서 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 사이의 각도 θ 는 둔각이다. 두 벡터 사이의 각도를 언급할 때, 우리는 항상 가장 작은 각도, 즉 $0 \leq \theta \leq \pi$ 를 만족하는 각도 θ 를 의미한다.

식 1.4는 내적의 유용한 기하학적 성질을 알려준다:

1. $\mathbf{u} \cdot \mathbf{v} = 0$ 이면, $\mathbf{u} \perp \mathbf{v}$ (즉, 벡터들이 직교한다).
2. $\mathbf{u} \cdot \mathbf{v} > 0$ 이면 두 벡터 사이의 각도 θ 는 90도보다 작다(즉, 벡터들은 예각을 이룬다).
3. $\mathbf{u} \cdot \mathbf{v} < 0$ 이면 두 벡터 사이의 각도 θ 는 90도보다 크다(즉, 벡터들이 둔각을 이룬다).

Note: "직교하는"이라는 단어는 "수직인"의 동의어로 사용할 수 있습니다.

예제 1.4

$\mathbf{u} = (1, 2, 3)$ 이고 $\mathbf{v} = (-4, 0, -1)$ 이라고 하자. $\mathbf{u}$ 와 $\mathbf{v}$ 사이의 각도를 구하라. 먼저 다음을 계산한다:

$$\mathbf{u} \cdot \mathbf{v} = (1, 2, 3) \cdot (-4, 0, -1) = -4 - 3 = -7$$

$$\|\mathbf{u}\| = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$$

$$\|\mathbf{v}\| = \sqrt{(-4)^2 + 0^2 + (-1)^2} = \sqrt{17}$$

이제 방정식 1.4를 적용하고 θ 에 대해 풀면 다음과 같이 구할 수 있습니다:

$$\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{-7}{\sqrt{14} \sqrt{17}}$$

$$\theta = \cos^{-1} \frac{-7}{\sqrt{14} \sqrt{17}} \approx 117^\circ$$

예제 1.5

그림 1.10을 고려하라. $\mathbf{v}$ 와 단위 벡터 $\mathbf{n}$ 이 주어졌을 때, 내적을 사용하여 $\mathbf{v}$ 와 $\mathbf{n}$ 을 변수로 하는 $\mathbf{p}$ 의 공식이 무엇인지 구하라.

먼저, 그림에서 $\mathbf{p} = k\mathbf{n}$ 을 만족하는 스칼라 k 가 존재함을 관찰할 수 있다. 또한 $\|\mathbf{n}\| = 1$ 이라고 가정했으므로, $\|$

$\mathbf{p}\| = \|k\mathbf{n}\| = |k| \|\mathbf{n}\| = |k|$. (k 가 음수일 수 있는 경우는 $\mathbf{p}$ 와 $\mathbf{n}$ 이 서로 반대 방향을 향할 때뿐임에 유의하라.) 삼각법을 이용하면 $k = \|\mathbf{v}\| \cos \theta$ 임을 알 수 있으므로, $\mathbf{p} = k\mathbf{n} = (\|\mathbf{v}\| \cos \theta)\mathbf{n}$ 이다.

그러나 $\mathbf{n}$ 이 단위 벡터이므로, 이를 다른 방식으로 표현할 수 있다:

$$\mathbf{p} = (\|\mathbf{v}\| \cos \theta)\mathbf{n} = (\|\mathbf{v}\| \cdot 1 \cos \theta)\mathbf{n} = (\|\mathbf{v}\| \|\mathbf{n}\| \cos \theta)\mathbf{n} = (\mathbf{v} \cdot \mathbf{n})\mathbf{n}$$

특히, 이는 $k = \mathbf{v} \cdot \mathbf{n}$ 을 보여주며, $\mathbf{n}$ 이 단위 벡터일 때 $\mathbf{v} \cdot \mathbf{n}$ 의 기하학적 해석을 설명한다. 우리는 $\mathbf{p}$ 를 $\mathbf{n}$ 위의 $\mathbf{v}$ 의 직교 투영이라고 부르며, 일반적으로 다음과 같이 표기한다.

$$\mathbf{p} = \text{proj}_{\mathbf{n}}(\mathbf{v})$$

벡터 $\mathbf{v}$ 를 힘으로 해석한다면, $\mathbf{p}$ 는 힘 $\mathbf{v}$ 중 $\mathbf{n}$ 방향으로 작용하는 부분으로 생각할 수 있다. 마찬가지로, 벡터 $\mathbf{w} = \text{perp}_{\mathbf{n}}(\mathbf{v}) = \mathbf{v} - \mathbf{p}$ 는 힘 $\mathbf{v}$ 중 $\mathbf{n}$ 방향에 수직으로 작용하는 부분이다(이 때문에 우리는 이를 수직을 의미하는 $\text{perp}_{\mathbf{n}}(\mathbf{v})$ 로도 표기한다). $\mathbf{v} = \mathbf{p} + \mathbf{w} = \text{proj}_{\mathbf{n}}(\mathbf{v}) + \text{perp}_{\mathbf{n}}(\mathbf{v})$ 임을 관찰할 수 있는데, 이는 벡터 $\mathbf{v}$ 를 두 개의 직교 벡터 $\mathbf{p}$ 와 $\mathbf{w}$ 의 합으로 분해했음을 의미한다.

벡터 $\mathbf{n}$ 이 단위 길이가 아닐 경우, 항상 먼저 정규화하여 단위 길이로 만들 수 있습니다. $\mathbf{n}$ 을 단위 벡터

를 사용하면

$$\frac{\mathbf{n}}{\|\mathbf{n}\|}$$

보다 일반적인 투영 공식이 도출됩니다:

$$\mathbf{p} = \text{proj}_{\mathbf{n}}(\mathbf{v}) = \left(\mathbf{v} \cdot \frac{\mathbf{n}}{\|\mathbf{n}\|} \right) \frac{\mathbf{n}}{\|\mathbf{n}\|} = \frac{(\mathbf{v} \cdot \mathbf{n})}{\|\mathbf{n}\|^2} \mathbf{n}$$

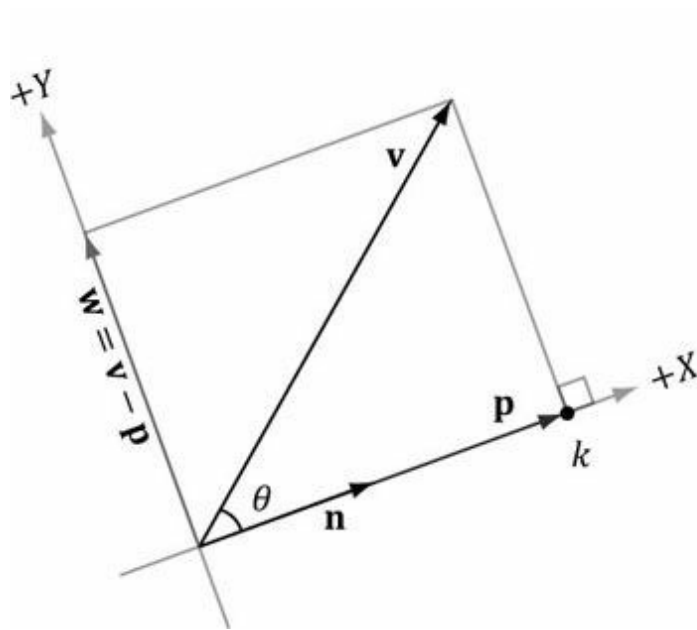
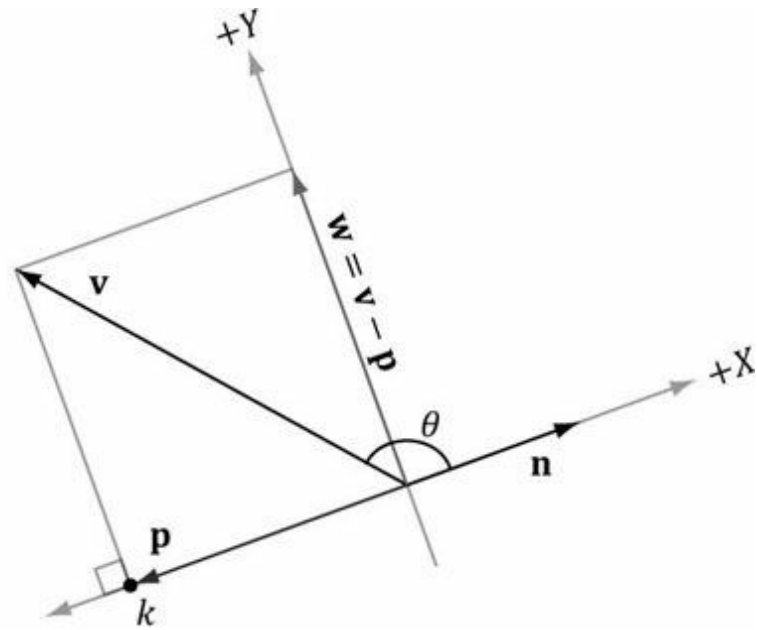


그림 1.10. $\mathbf{v}$ 의 $\mathbf{n}$ 에 대한 직교 투영.



1.3.1 직교화

벡터 집합 $\{\mathbf{v}_0, \dots, \mathbf{v}_{n-1}\}$ 은 벡터들이 서로 직교하고(집합 내 모든 벡터가 다른 모든 벡터와 직교함) 길이가 1일 때 *직교정규*라고 한다. 때로는 거의 직교정규에 가깝지만 완전히 그렇지 않은 벡터 집합이 존재한다. 흔히 이 집합을 직교화하여 직교정규로 만드는 작업이 수행된다. 3차원 컴퓨터 그래픽스에서는 정규 직교 집합으로 시작할 수 있지만, 수치 정밀도 문제로 인해 점차 정규 직교성을 잃게 된다. 우리는 주로 이 문제의 2차원 및 3차원 사례(즉, 각각 두 개와 세 개의 벡터를 포함하는 집합)에 관심을 둔다.

먼저 더 간단한 2차원 사례를 살펴보겠습니다. $\{\mathbf{v}_0, \mathbf{v}_1\}$ 이라는 벡터 집합이 있다고 가정하고, 이를 정규 직교화하여

[그림 1.11](#)에 표시된 직교 정규 집합 $\{\mathbf{w}_0, \mathbf{w}_1\}$ 을 고려한다. $\mathbf{w}_0 = \mathbf{v}_0$ 으로 시작하여 $\mathbf{v}_1$ 을 $\mathbf{w}_0$ 과 직교하도록 수정한다. 이는 $\mathbf{w}_0$ 방향으로 작용하는 $\mathbf{v}_1$ 의 성분을 빼내는 방식으로 수행된다:

$$\mathbf{w}_1 = \mathbf{v}_1 - \text{proj}_{\mathbf{w}_0}(\mathbf{v}_1)$$

이제 우리는 상호 직교하는 벡터 집합 $\{\mathbf{w}_0, \mathbf{w}_1\}$ 을 얻었습니다. 직교 정규 집합을 구성하기 위한 마지막 단계는 $\mathbf{w}_0$ 과 $\mathbf{w}_1$ 을 정규화하여 단위 길이로 만드는 것입니다.

3차원 사례는 2차원 사례와 동일한 원리로 진행되나 단계가 더 많습니다. 정규화하고자 하는 벡터 집합 $\{\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2\}$ 가 있다고 가정합니다.

이를 [그림 1.12](#)와 같이 직교 정규화 벡터 집합 $\{\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2\}$ 로 정규화하고자 한다. $\mathbf{w}_0 = \mathbf{v}_0$ 으로 시작하여 $\mathbf{v}_1$ 을 $\mathbf{w}_0$ 과 직교하도록 수정한다. 이는 $\mathbf{w}_0$ 방향으로 작용하는 $\mathbf{v}_1$ 의 성분을 빼내는 방식으로 수행된다:

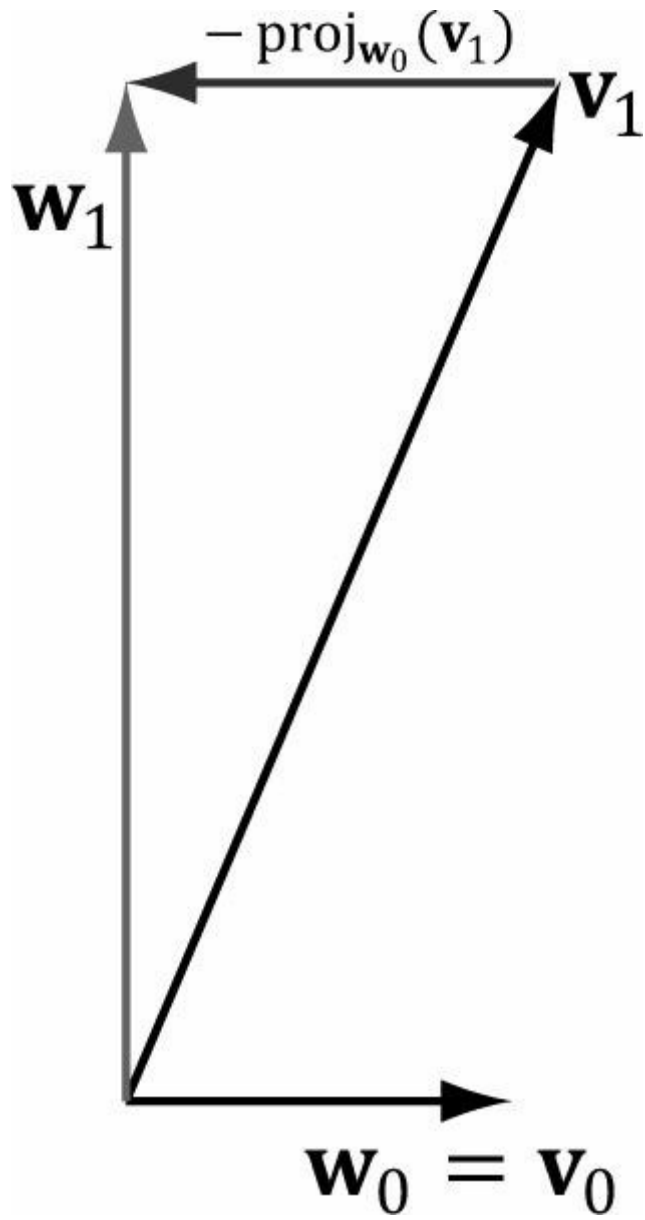


그림 1.11. 2차원 직교화.

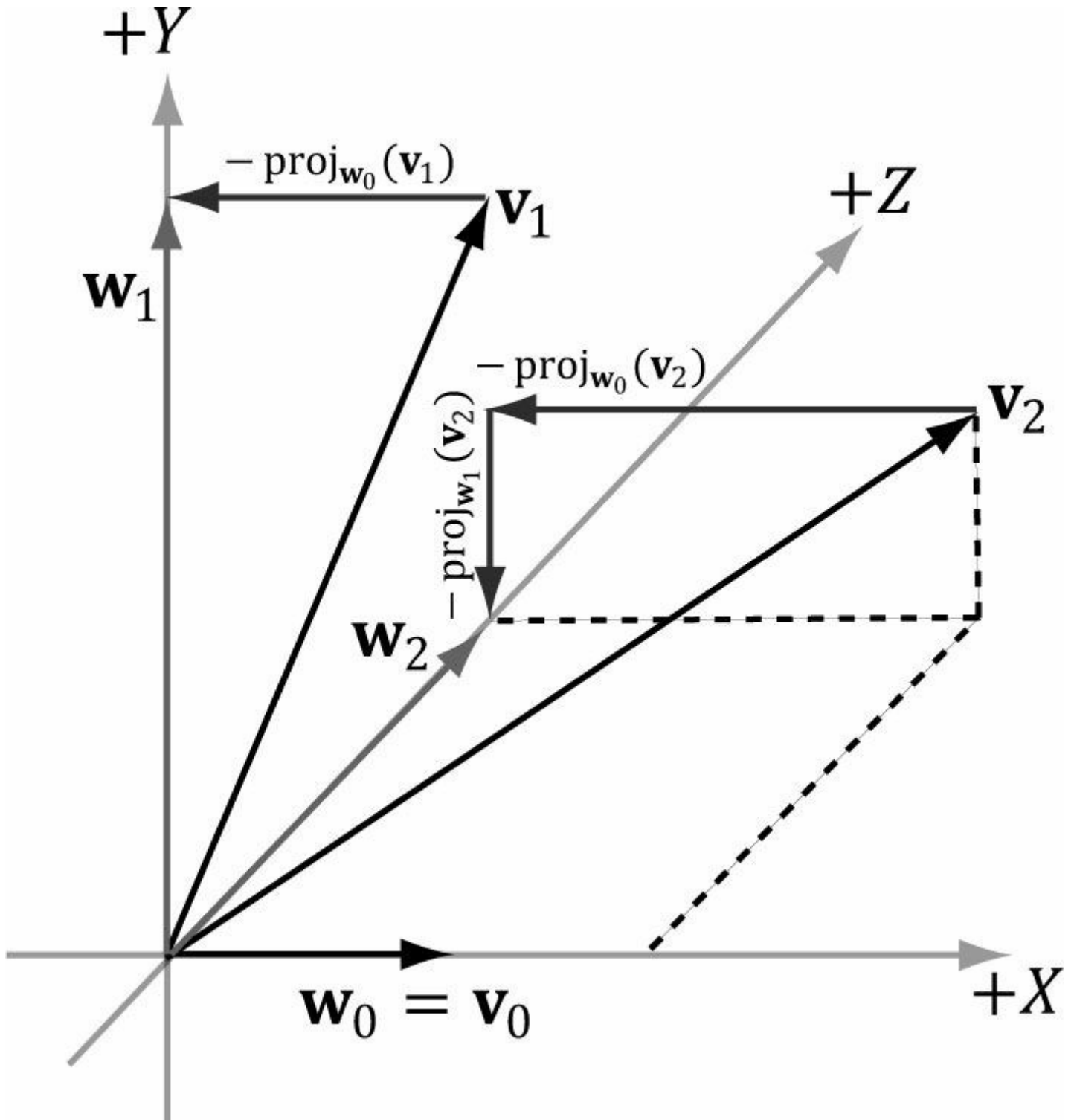


그림 1.12. 3차원 직교화.

$$w_1 = v_1 - \text{proj}_{w_0}(v_1)$$

다음으로, v_2 를 w_0 과 w_1 모두에 대해 직교하도록 수정합니다. 이는 w_0 방향으로 작용하는 v_2 의 부분과 w_1 방향으로 작용하는 v_2 의 부분을 빼내는 방식으로 수행됩니다.

방향으로 작용하는 부분과 w_1 방향으로 작용하는 부분을 빼내는 방식으로 수행됩니다:

$$w_2 = v_2 - \text{proj}_{w_0}(v_2) - \text{proj}_{w_1}(v_2)$$

이제 우리는 상호 직교하는 벡터 집합 $\{w_0, w_1, w_2\}$ 을 갖게 되었습니다. 직교 정규 집합을 구성하기 위한 마지막 단계는 w_0, w_1, w_2 를 정규화하여 길이를 1로 만드는 것입니다.

일반적인 경우, 정규 직교 집합으로 직교화하고자 하는 n 개의 벡터 집합 $\{v_0, \dots, v_{n-1}\}$ 에 대해 다음과 같은

일반적으로 *그람-슈미트 직교화* 과정이라고 불리는 절차는 다음과 같습니다:

Base Step : Set $\mathbf{w}_0 = \mathbf{v}_0$

For $1 \leq i \leq n-1$, Set $\mathbf{w}_i = \mathbf{v}_i - \sum_{j=0}^{i-1} \text{proj}_{\mathbf{w}_j}(\mathbf{v}_i)$

Normalization Step : Set $\mathbf{w}_i = \frac{\mathbf{w}_i}{\|\mathbf{w}_i\|}$

다시 말해, 직교 정규 집합에 추가할 입력 집합의 벡터 $\mathbf{v}_i$ 를 선택할 때, 추가하려는 벡터 $\mathbf{v}_i$ 가 이미 직교 정규 집합에 속한 다른 벡터들($\mathbf{w}_0, \mathbf{w}_1, \dots, \mathbf{w}_{i-1}$)의 성분을 빼내어, 새로 추가되는 벡터가 직교 정규 집합에 이미 존재하는 다른 벡터들과 직교하도록 보장해야 한다는 것입니다.

1.4 외적

벡터 연산에서 정의하는 두 번째 형태의 곱셈은 외적입니다. 스칼라 값을 반환하는 내적과 달리, 외적은 또 다른 벡터를 반환합니다. 또한 외적은 3차원 벡터에 대해서만 정의됩니다(특히 2차원 외적은 존재하지 않습니다). 두 3차원 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 의 외적을 취하면 $\mathbf{u}$ 와 $\mathbf{v}$ 에 대해 상호 직교하는 또 다른 벡터 $\mathbf{w}$ 를 얻습니다. 즉, $\mathbf{w}$ 는 $\mathbf{u}$ 에 대해 직교하고, $\mathbf{w}$ 는 $\mathbf{v}$ 에 대해 직교합니다. [그림 1.13](#)을 참조하십시오. $\mathbf{u} = (u_x, u_y, u_z)$ 이고 $\mathbf{v} = (v_x, v_y, v_z)$ 일 때, 외적은 다음과 같이 계산됩니다:

$$\mathbf{w} = \mathbf{u} \times \mathbf{v} = (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x) \quad (\text{식 1.5})$$

오른손 좌표계에서 작업 중이라면 오른손 엄지 규칙을 사용합니다. 오른손을 쥐고() 손가락을 첫 번째 벡터 방향으로 향하게 한 다음, 손가락을 $0 \leq \theta \leq \pi$ 의 각도를 따라 말면, 엄지손가락이 대략 $\mathbf{w} = \mathbf{u} \times \mathbf{v}$ 의 방향을 가리킵니다.

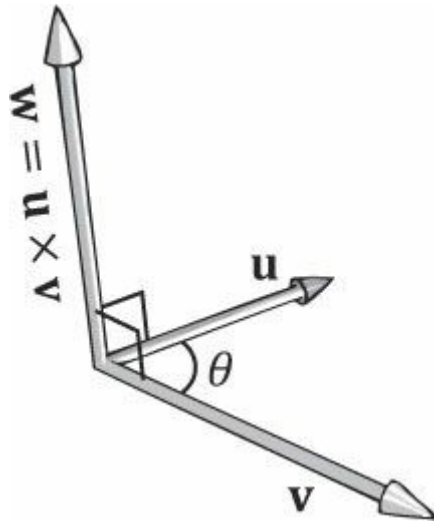


그림 1.13. 두 3차원 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 의 외적은 $\mathbf{u}$ 와 $\mathbf{v}$ 에 서로 직교하는 또 다른 벡터 $\mathbf{w}$ 를 생성한다. 왼손을 잡고 첫 번째 벡터 $\mathbf{u}$ 의 방향으로 손가락을 뻗은 다음, $0 \leq \theta \leq \pi$ 의 각도로 $\mathbf{v}$ 방향으로 손가락을 말아 올리면 엄지손가락이 대략 $\mathbf{w} = \mathbf{u} \times \mathbf{v}$ 의 방향을 가리킨다. 이를 왼손 엄지 규칙이라 한다.

예제 1.6

$\mathbf{u} = (2, 1, 3)$ 및 $\mathbf{v} = (2, 0, 0)$ 이라 하자. $\mathbf{w} = \mathbf{u} \times \mathbf{v}$ 및 $\mathbf{z} = \mathbf{v} \times \mathbf{u}$ 를 계산한 후, $\mathbf{w}$ 가 $\mathbf{u}$ 와 직교하고 $\mathbf{w}$ 가 $\mathbf{v}$ 와 직교함을 검증하라. 식 1.5를 적용하면 다음과 같다.

$$\begin{aligned} \mathbf{w} &= \mathbf{u} \times \mathbf{v} \\ &= (2, 1, 3) \times (2, 0, 0) \\ &= (1 \cdot 0 - 3 \cdot 0, 3 \cdot 2 - 2 \cdot 0, 2 \cdot 0 - 1 \cdot 2) \end{aligned}$$

$$= (0, 6, -2)$$

그리

고

$$\begin{aligned}\mathbf{z} &= \mathbf{v} \times \mathbf{u} \\ &= (2, 0, 0) \times (2, 1, 3) \\ &= (0 \cdot 3 - 0 \cdot 1, 0 \cdot 2 - 2 \cdot 3, 2 \cdot 1 - 0 \cdot 2) \\ &= (0, -6, 2)\end{aligned}$$

이 결과는 한 가지를 분명히 보여줍니다. 일반적으로 $\mathbf{u} \times \mathbf{v} \neq \mathbf{v} \times \mathbf{u}$ 입니다. 따라서 우리는 외적(cross product)이 반가환적(anti-commutative)이라고 말합니다. 사실, $\mathbf{u} \times \mathbf{v} = -\mathbf{v} \times \mathbf{u}$ 임을 증명할 수 있습니다. 외적이 반환하는 벡터는 *왼손 엄지* 규칙(left-hand thumb rule)으로 결정할 수 있습니다. 첫 번째 벡터 방향으로 손가락을 뻗은 후, 두 번째 벡터 방향으로 손가락을 말아 올릴 때(항상 각도가 가장 작은 경로를 선택), 엄지손가락이 [그림 1.13](#)과 같이 결과 벡터의 방향을 가리킵니다.

$\mathbf{w}$ 가 $\mathbf{u}$ 와 직교하고 $\mathbf{w}$ 가 $\mathbf{v}$ 와 직교함을 보이기 위해, §1.3에서 $\mathbf{u} \cdot \mathbf{v} = 0$ 이면 $\mathbf{u} \perp \mathbf{v}$ (즉, 벡터들이 서로 수직임)임을 상기합니다. 직교한다). 왜냐하면

$$\mathbf{w} \cdot \mathbf{u} = (0, 6, -2) \cdot (2, 1, 3) = 0 \cdot 2 + 6 \cdot 1 + (-2) \cdot 3 = 0$$

그리

고

$$\mathbf{w} \cdot \mathbf{v} = (0, 6, -2) \cdot (2, 0, 0) = 0 \cdot 2 + 6 \cdot 0 + (-2) \cdot 0 = 0$$

따라서 $\mathbf{w}$ 는 $\mathbf{u}$ 와 직교하며, $\mathbf{w}$ 는 $\mathbf{v}$ 와도 직교함을 알 수 있습니다.

1.4.1 의사 2차원 외적

외적은 주어진 두 3차원 벡터에 수직인 벡터를 구하는 데 사용됩니다. 2차원에서는 상황이 완전히 동일하지는 않지만, 2차원 벡터 $\mathbf{u} = (u_x, u_y)$ 가 주어졌을 때 $\mathbf{u}$ 에 수직인 벡터 $\mathbf{v}$ 를 찾는 것이 유용할 수 있습니다. [그림 1.14](#)는 $\mathbf{v} = (-u_y, u_x)$ 를 제안하는 기하학적 설정을 보여줍니다. 공식적인 증명은 간단합니다:

$$\mathbf{u} \cdot \mathbf{v} = (u_x, u_y) \cdot (-u_y, u_x) = -u_x u_y + u_y u_x = 0 \text{ 따라서 } \mathbf{u} \perp \mathbf{v} \text{이다. } \mathbf{u} \cdot$$

$(-\mathbf{v}) = u_x u_y + u_y (-u_x) = 0$ 이기도 하므로 $\mathbf{u} \perp (-\mathbf{v})$ 임을 알 수 있다.

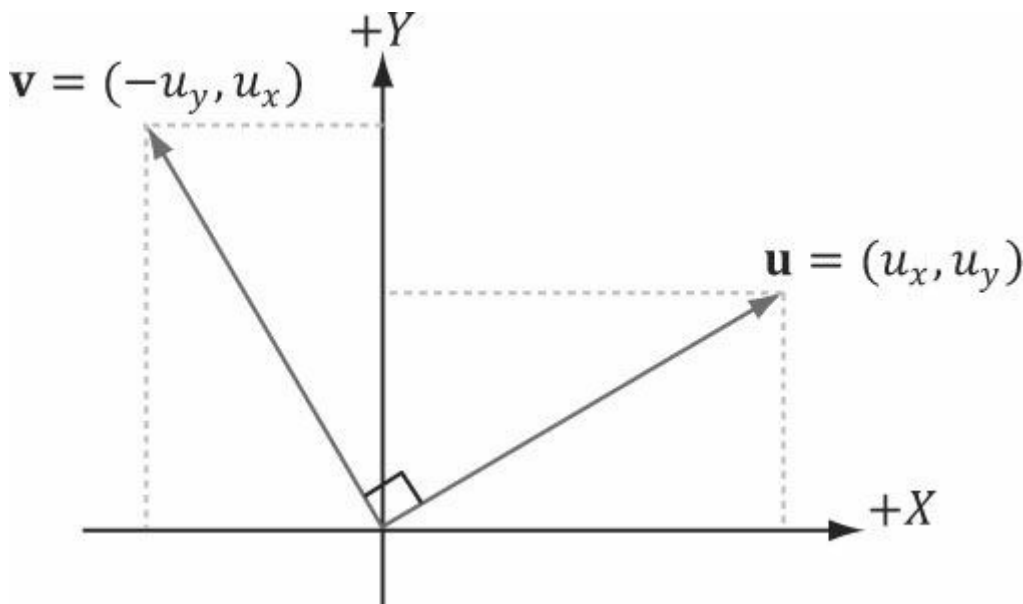


그림 1.14. 벡터 $\mathbf{u}$ 의 2차원 의사 교차 곱은 직교 벡터 $\mathbf{v}$ 를 산출한다.

1.4.2 외적에 의한 직교화

§1.3.1절에서는 과정을 이용해 벡터 집합을 직교화하는 방법을 살펴보았습니다. 3차원에서는 거의 직교정규화되었으나 누적인 수치 정밀도 오차로 인해 직교정규화가 깨진 벡터 집합 $\{\text{Gram-Schmidt } \mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2\}$ 을 직교화하는 또 다른 전략이 있습니다. 이는 외적을 활용합니다. 이 과정의 기하학적 구조는 [그림 1.15](#)를 참조하십시오:

1. Set $\mathbf{w}_0 = \frac{\mathbf{v}_0}{\|\mathbf{v}_0\|}$.
2. Set $\mathbf{w}_2 = \frac{\mathbf{w}_0 \times \mathbf{v}_1}{\|\mathbf{w}_0 \times \mathbf{v}_1\|}$.

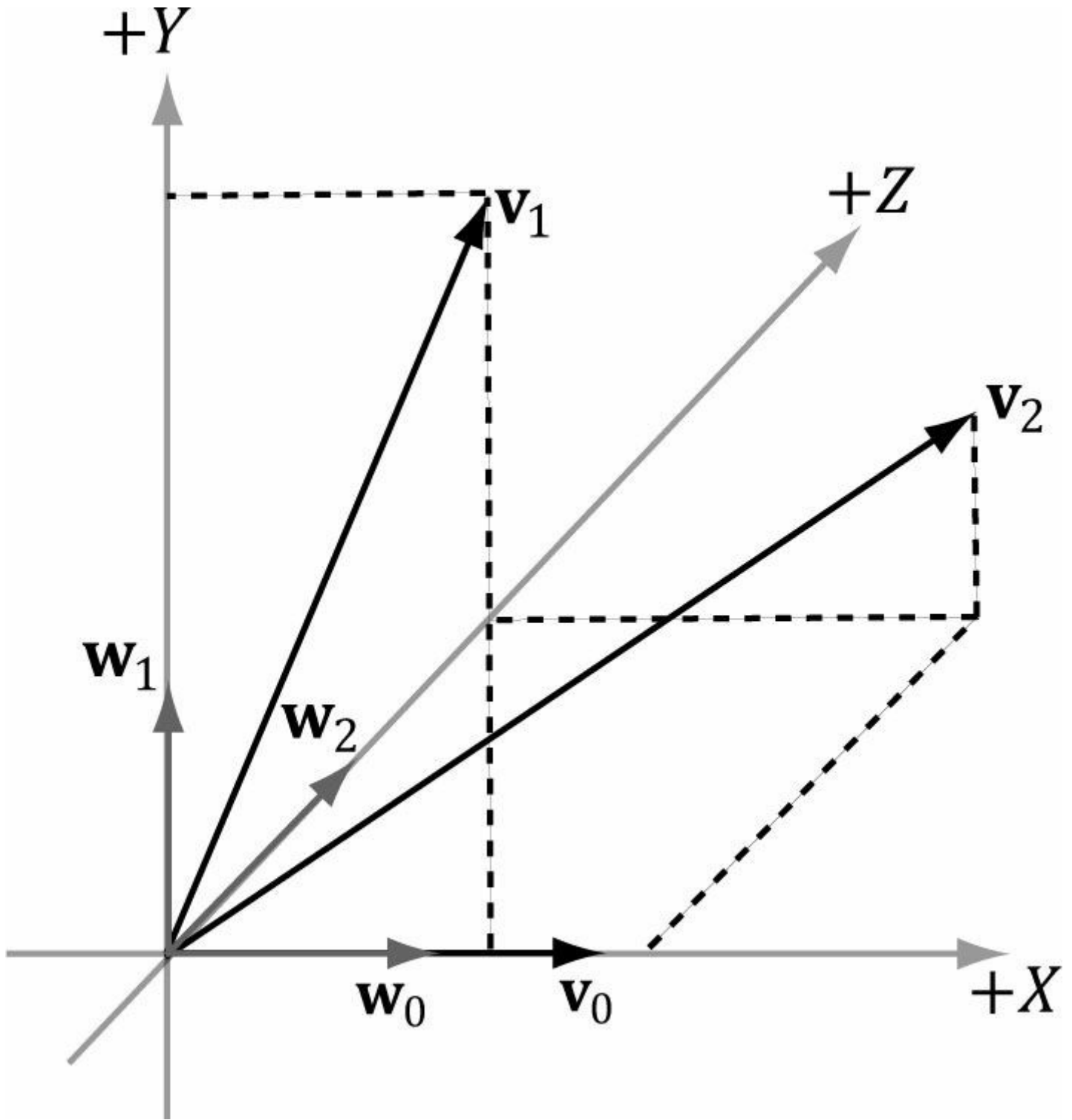


그림 1.15. 교차 곱을 이용한 3차원 직교화.

3. $w_1 = w_2 \times w_0$ 으로 설정합니다. 연습문제 14에 따르면, $\|w_2 \times w_0\| = 1$ 입니다. 왜냐하면 $w_2 \perp w_0$ 이고 $\|w_2\| = \|w_0\| = 1$ 이기 때문입니다. 따라서 이 마지막 단계에서 정규화를 수행할 필요가 없습니다.

이 시점에서 벡터 집합 $\{w_0, w_1, w_2\}$ 은 직교 정규화되어 있습니다.

이전 예시에서는 $w_0 = \frac{v_0}{\|v_0\|}$ 로 시작했는데, 이는 v_0 에서 w_0 으로 이동할 때 방향을 변경하지 않았음을 의미합니다. 단지 길이를 변경했을 뿐입니다. 그러나 w_1 과 w_2 의 방향은 각각 v_1 과 v_2 와 다를 수 있습니다. 특정 응용 분야에 따라 방향을 변경하지 않기로 선택한 벡터는 다음과 같을 수 있습니다.

Note 중요합니다. 예를 들어, 이 책의 후반부에서는 카메라의 방향을 세 개의 직교 정규화 벡터

$\{v_0, v_1, v_2\}$ 로 표현합니다. 여기서 세 번째 벡터 $v_{(2)}$ 는 카메라가 바라보는 방향을 나타냅니다. 이 벡터들을 직교화할 때, 우리는 종종 바라보는 방향을 변경하고 싶지 않습니다. 따라서 이전 알고리즘을 $v_{(2)}$ 로 시작하고 v_0 와 v_1 을 수정하여 벡터들을 직교화할 것입니다.

1.5 POINTS

지금까지 우리는 위치를 설명하지 않는 벡터에 대해 논의해 왔습니다. 그러나 3D 프로그램에서는 3D 기하학적 구조물의 위치나 3D 가상 카메라의 위치와 같이 위치를 명시해야 할 필요가 있습니다. 좌표계에 대해, 표준 위치(그림 1.16)의 벡터를 사용하여 공간 내 3D 위치를 표현할 수 있습니다. 이를 *위치 벡터*라고 부릅니다. 이 경우 벡터 끝점의 위치가 관심 대상이며, 방향이나 크기는 중요하지 않습니다. 위치 벡터만으로도 점을 식별할 수 있으므로, "위치 벡터"와 "점"이라는 용어를 상호 교환하여 사용할 것입니다.

벡터를 사용하여 점을 표현할 때, 특히 코드에서 발생하는 한 가지 부작용은 의미가 없는 벡터 연산을 수행할 수 있다는 점입니다.

점의 경우, 예를 들어, 기하학적으로 두 점의 합은 무엇을 의미해야 할까? 반면, 일부 연산은 점으로 확장될 수 있다. 예를 들어, 두 점 q 와 p 의 차이는 p 에서 q 로 향하는 벡터로 정의한다. 또한 점 p 에 벡터 v 를 더하는 것은 점 p 를 벡터 v 만큼 변위시켜 얻은 점 q 로 정의한다. 편리하게도, 좌표계에 대한 점의 위치를 벡터로 표현하기 때문에 방금 논의한 점 연산에 대해 별도의 작업이 필요하지 않다. 벡터 대수 프레임워크가 이미 이를 처리하기 때문이다; 그림 1.17을 참조하라.

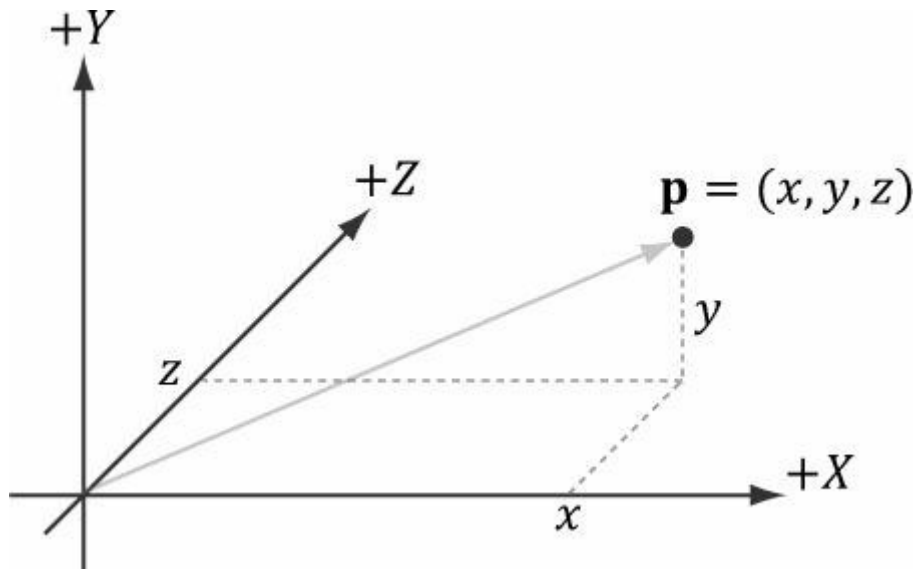
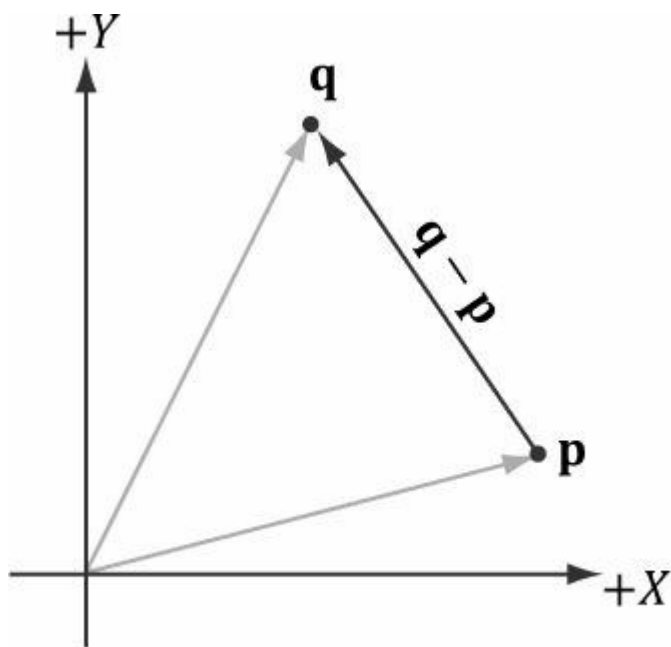
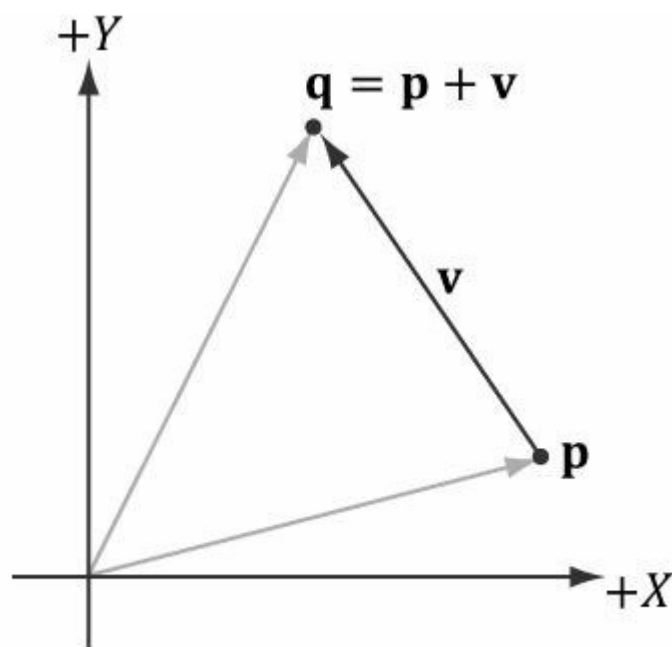


그림 1.16. 위치 벡터는 원점에서 점까지 뻗어 있으며, 좌표계에 대한 점의 위치를 완전히 설명한다.



(a)



(b)

그림 1.17. (a) 두 점 p 와 q 사이의 차이 $q - p$ 는 p 에서 q 로 향하는 벡터로 정의된다. (b) 점 p 에 벡터 v 를 더한 것은 점 p 를 벡터 v 만큼 변위시켜 얻은 점 q 로 정의된다.

Note: 실제로 점들의 특별한 합을 정의하는 기하학적으로 의미 있는 방법이 있는데, 이를 아핀 조합이라고 하며 점들의 가중 평균과 유사하다.

1.6 XNA MATH VECTORS

Direct3D 9 및 10에서는 D3DX 라이브러리가 3D 그래픽스 작업에 필요한 핵심 3D 수학 유형(벡터 포함)을 지원하는 3D 수학 유틸리티 라이브러리와 함께 제공되었습니다. 버전 11용 D3DX 라이브러리는 더 이상 3D 수학 코드를 포함하지 않습니다. 대신 XNA Math 라이브러리를 포함하는데, 이는 Direct3D와 별도로 개발된 벡터 수학 라이브러리로 Windows 및 Xbox 360의 특수 하드웨어 레지스터를 활용합니다. Windows에서는 이 라이브러리가 SSE2(Streaming SIMD Extensions 2) 명령어 세트를 사용합니다. 128비트 폭의 SIMD(단일 명령 다중 데이터) 레지스터를 통해 SIMD 명령어는 하나의 명령으로 4개의 32비트 **부동소수점** 또는 **정수** 연산을 수행할 수 있습니다. 이는 벡터 계산에 매우 유용합니다. 예를 들어 벡터 덧셈을 살펴보면:

$$\mathbf{u} + \mathbf{v} = (u_x + v_x, u_y + v_y, u_z + v_z)$$

해당 성분들만 더하면 된다는 것을 알 수 있습니다. SIMD를 사용하면 4개의 스칼라 명령어 대신 하나의 SIMD 명령어로 4차원 벡터 덧셈을 수행할 수 있습니다. 3차원에서 작업할 때도 SIMD를 사용할 수 있지만, 4번째 성분을 0으로 설정하고 무시하면 됩니다. 2차원에서도 마찬가지입니다.

XNA Math 라이브러리에 대한 우리의 설명은 포괄적이지 않으며, 이 책에 필요한 핵심 부분만 다룹니다. 모든 세부 사항에 대해서는 온라인 문서 [XNAMath2010]을 권장합니다. SIMD 벡터 라이브러리를 최적화하여 개발하는 방법과 XNA Math 라이브러리가 특정 선택을 한 이유에 대한 통찰력을 얻고자 하는 독자들을 위해, 자세한 내용은 온라인 문서 [XNAMath2010]을 참고하시기 바랍니다. SIMD 벡터 라이브러리를 최적화하여 개발하는 방법과 XNA Math 라이브러리의 설계 결정 배경에 대한 통찰을 얻고자 하는 독자께서는 [Oliveira2010]의 논문 "*Designing Fast Cross-Platform SIMD Vector Libraries*"를 추천합니다.

XNA Math 라이브러리를 사용하려면 <xnamath.h>를 #include하기만 하면 됩니다. 모든 코드가 헤더 파일에 인라인으로 구현되어 있으므로 추가 라이브러리 파일은 필요하지 않습니다. 헤더 파일에 인라인으로 구현되어 있습니다.

 <d3dx10.h>를 포함하고, d3dx10.lib로 링크하며, XNA 수학 라이브러리 대신 D3DX10 수학 함수를 사용하는 것을 막는 것은 아무것도 없습니다.

1.6.1 벡터 유형

XNA Math에서 핵심 벡터 유형은 SIMD 하드웨어 레지스터에 매핑되는 XMVECTOR입니다. 이는 단일 SIMD 명령어로 네 개의 32비트 부동 소수점을 처리할 수 있는 128비트 유형입니다. SSE2가 사용 가능한 경우 다음과 같이 정의됩니다:

```
typedef __m128 XMVECTOR;
```

여기서 **m128**은 특수한 SIMD 유형입니다. 계산을 수행할 때 SIMD의 이점을 활용하려면 벡터가 이 유형이어야 합니다. 앞서 언급한 바와 같이, SIMD의 이점을 활용하기 위해 2D 및 3D 벡터에도 이 유형을 계속 사용하지만, 사용되지 않는 구성 요소는 0으로 초기화하고 무시합니다.

XMVECTOR는 16바이트 정렬되어야 하며, 이는 지역 변수와 전역 변수에 대해 자동으로 수행됩니다. 클래스 데이터 멤버의 경우 대신 **XMFLOAT2**(2차원), **XMFLOAT3**(3차원), **XMFLOAT4**(4차원)을 사용하는 것이 권장됩니다. 이 구조체들은 아래와 같이 정의됩니다:

```
typedef struct _XMFLOAT2 {
    FLOAT x;
    FLOAT y;
} XMFLOAT2;

typedef struct _XMFLOAT3 {
    FLOAT x;
    FLOAT y;
    FLOAT z;
} XMFLOAT3;

typedef struct _XMFLOAT4 {
    FLOAT x;
    FLOAT y;
    FLOAT z;
    FLOAT w;
} XMFLOAT4;
```

그러나 이러한 유형을 직접 계산에 사용하면 SIMD의 이점을 활용하지 못합니다. SIMD를 사용하려면 해당 유형의 인스턴스를 XMVECTOR 유형으로 변환해야 합니다. 이는 XNA Math 로딩 함수를 통해 수행됩니다. 반대로,

XNA Math는 **XMVECTOR** 데이터를 앞서 설명한 **XMFLOAT*** 유형으로 변환하는 저장 함수를 제공합니다.
요약하면,

1. 로컬 또는 전역 변수에는 **XMVECTOR**를 사용하십시오.
2. 클래스 데이터 멤버에는 **XMVECTOR**, **XMVECTOR2**, **XMVECTOR3**, **XMVECTOR4**를 사용하십시오.
3. 계산을 수행하기 전에 **XMVECTOR**에서 **XMVECTOR로** 변환하기 위해 로딩 함수를 사용하십시오.
4. 계산은 **XMVECTOR** 인스턴스로 수행하십시오.
5. 저장 함수를 사용하여 **XMVECTOR**에서 **XMVECTOR로** 변환하십시오.

1.6.2 로드 및 저장 방법

XMVECTOR에서 **XMVECTOR로** 데이터를 로드하는 방법은 다음과 같습니다:

```
// XMVECTOR를 XMVECTOR로 로드
XMVECTOR XMLoadFloat2(CONST XMVECTOR *pSource);

// XMVECTOR를 XMVECTOR로 로드
XMVECTOR XMLoadFloat3(CONST XMVECTOR *pSource);

// XMVECTOR를 XMVECTOR로 로드
XMVECTOR XMLoadFloat4(CONST XMVECTOR *pSource);
```

 **XMVECTOR**에 다른 데이터 유형을 로드하는 데 사용되는 로딩 방법은 훨씬 더 많습니다. 전체 목록은 *XNA Math* 문서를 참조하십시오. 다음은 몇 가지 추가 예시입니다:

```
// 3개 요소 UINT 배열을 XMVECTOR에 로드
XMVECTOR XMLoadInt3(CONST UINT *pSource);

// XMVECTOR를 XMVECTOR로 로드
XMVECTOR XMLoadColor(CONST XMVECTOR *pSource);

// XMVECTOR를 XMVECTOR에 로드
XMVECTOR XMLoadByte4(CONST XMVECTOR *pSource);
```

XMVECTOR의 데이터를 **XMVECTOR**에 저장하기 위해 다음 메서드를 사용합니다:

```
// XMVECTOR를 XMVECTOR로 로드합니다
VOID XMStoreFloat2(XMVECTOR *pDestination, XMVECTOR V);

// XMVECTOR를 XMVECTOR에 로드합니다
VOID XMStoreFloat3(XMVECTOR *pDestination, XMVECTOR V);

// XMVECTOR를 XMVECTOR에 로드합니다
VOID XMStoreFloat4(XMVECTOR *pDestination, XMVECTOR V);
```

 **XMVECTOR**를 다른 데이터 유형에 저장하는 데 사용되는 저장 방법은 훨씬 더 많습니다. 다음은 일부 예시입니다. 전체 목록은 *XNA Math* 문서를 참조하십시오.

```
// XMVECTOR를 3개 요소 UINT 배열에 로드
VOID XMStoreInt3(UINT *pDestination, XMVECTOR V);

// XMVECTOR를 XMVECTOR에 로드합니다
VOID XMStoreColor(XMVECTOR *pDestination, XMVECTOR V);

// XMVECTOR를 XMVECTOR에 로드합니다
VOID XMStoreByte4(XMVECTOR *pDestination, XMVECTOR V);
```

때로는 **XMVECTOR**의 한 구성 요소만 가져오거나 설정하고 싶을 때가 있습니다. 다음의 게터 및 세터 함수가 이를 용이하게 합니다:

```
FLOAT XMVectorGetX(XMVECTOR V);
```

```

    FLOAT    XMVectorGetY(FXMVECTOR V);    FLOAT

XMVectorGetZ(FXMVECTOR V);    FLOAT

XMVectorGetW(FXMVECTOR V);

    XMVECTOR XMVectorSetX(FXMVECTOR V, FLOAT x); XMVECTOR

XMVectorSetY(FXMVECTOR V, FLOAT y); XMVECTOR XMVectorSetZ(FXMVECTOR V,
    FLOAT z); XMVECTOR XMVectorSetW(FXMVECTOR V, FLOAT w);

```

1.6.3 매개변수 전달

SIMD를 활용하기 위해 **XMVECTOR** 형식의 함수에 매개변수를 전달할 때 몇 가지 규칙이 있습니다. 이러한 규칙은 플랫폼에 따라 다르며, 특히 32비트 Windows, 64비트 Windows, Xbox 360 간에 차이가 있습니다. 플랫폼 독립성을 위해 **XMVECTOR** 매개변수 전달에는 **CXMVECTOR** 및 **FXMVECTOR** 형식을 사용합니다. 이 형식들은 플랫폼에 따라 적절한 형식으로 정의됩니다. Windows의 경우 다음과 같이 정의됩니다:

```

// 32비트 Windows

typedef const XMVECTOR FXMVECTOR; typedef const
XMVECTOR& CXMVECTOR;

// 64비트 Windows

typedef const XMVECTOR& FXMVECTOR; typedef const
XMVECTOR& CXMVECTOR;

```

차이점은 **XMVECTOR**의 복사본을 직접 전달할 수 있는지, 아니면 참조를 전달해야 하는지에 있습니다. 이제 **XMVECTOR** 매개변수 전달 규칙은 다음과 같습니다:

첫 번째 3개의 **XMVECTOR** 매개변수는 **FXMVECTOR** 유형이어야 하며, 추가적인 **XMVECTOR** 매개변수는 **CXMVECTOR** 유형이어야 합니다.

CXMVECTOR 유형이어야 합니다. 예시는 다음과 같습니다:

```

    XMINLINE XMATRIX XMMatrixTransformation( FXMVECTOR ScalingOrigin,
        FXMVECTOR ScalingOrientationQuaternion, FXMVECTOR
        Scaling,
        CXMVECTOR RotationOrigin, CXMVECTOR RotationQuaternion,
        CXMVECTOR Translation);

```

이 함수는 6개의 **XMVECTOR** 매개변수를 받지만, 매개변수 전달 규칙에 따라 **FXMVECTOR**와

CXMVECTOR 유형. 처음 세 개의 **XMVECTOR** 매개변수는 **FXMVECTOR** 유형이며, 추가 **XMVECTOR** 매개변수는 **CXMVECTOR** 유형입니다. **XMVECTOR** 매개변수 사이에 **XMVECTOR**가 아닌 매개변수를 포함할 수 있습니다. 이 경우 처음 3개의 **XMVECTOR** 매개변수는 여전히 **XMVECTOR** 유형으로 지정되며, 추가 **XMVECTOR** 매개변수는 **CXMVECTOR** 유형으로 지정됩니다:

```

    XMINLINE XMATRIX XMMatrixTransformation2D( FXMVECTOR ScalingOrigin,
        FLOAT ScalingOrientation, FXMVECTOR
        Scaling, FXMVECTOR RotationOrigin,
        FLOAT Rotation, CXMVECTOR
        Translation);

```

1.6.4 상수 벡터

상수 **XMVECTOR** 인스턴스는 **XMVECTORF32** 유형을 사용해야 합니다. 다음은 DirectX SDK의 CascadedShadowMaps11 샘플에서 가져온 예시입니다:

```

static const XMVECTORF32 g_vHalfVector = { 0.5f, 0.5f, 0.5f, 0.5f };

```

```
static const XMVECTORF32 g_vZero = { 0.0f, 0.0f, 0.0f, 0.0f };
```

```
XMVECTORF32 vRightTop = {
    mViewFrust.RightSlope,
    mViewFrust.TopSlope, 1.0f, 1.0f
};

XMVECTORF32 vLeftBottom = {
    mViewFrust.LeftSlope,
    mViewFrust.BottomSlope, 1.0f, 1.0f
};
```

기본적으로 초기화 구문을 사용할 때마다 XMVECTORF32를 사용합니다. XMVECTORF32는 XMVECTOR 변환 연산자를 가진 16바이트 정렬 구조체로, 다음과 같이 정의됩니다.

```
// 상수용 변환 유형

typedef __declspec(align(16)) struct XMVECTORF32 { union {
float f[4]; XMVECTOR v;

    };

    #if defined(__cplusplus)
        inline operator XMVECTOR() const { return v; }
    #if !defined(XM_NO_INTRINSICS_) && defined(XM_SSE_INTRINSICS_) inline operator __m128i() const
    { return reinterpret_cast<const __m128i*>(&v)[0]; } inline operator __m128d()
        const
    { return reinterpret_cast<const __m128d*>(&v)[0]; } #endif
    #endif // __cplusplus
    XMVECTORF32;
```

XMVECTORU32를 사용하여 정수 데이터의 상수 XMVECTOR를 생성할 수도 있습니다:

```
static const XMVECTORU32 vGrabY = { 0x00000000, 0xFFFFFFFF, 0x00000000, 0x00000000
};
```

1.6.5 과부하 연산자

XMVECTOR는 벡터 덧셈, 뺄셈 및 스칼라 곱셈을 수행하기 위한 여러 개의 오버로드 연산자를 가지고 있습니다. XM_NO_OPERATOR_OVERLOADS를 정의하여 연산자 오버로드를 비활성화 할 수 있습니다. 일부 애플리케이션에서 연산자 오버로드를 비활성화하려는 이유는 성능 때문입니다. 복잡한 표현식을 구성할 때 함수 버전이 더 빠를 수 있기 때문입니다([Oliveira2010] 참조). 이 책에서는 직관적인 연산자 중복 정의 구문을 선호하며 이를 활성화 상태로 유지합니다.

```
// 벡터 연산자
#if defined(__cplusplus) && !defined(XM_NO_OPERATOR_OVERLOADS)

XMVECTOR operator+ (FXMVECTOR V); XMVECTOR operator-
(FXMVECTOR V);

XMVECTOR& operator+= (XMVECTOR& V1, FXMVECTOR V2); XMVECTOR& operator-=
(XMVECTOR& V1, FXMVECTOR V2); XMVECTOR& operator*= (XMVECTOR& V1, FXMVECTOR
V2); XMVECTOR& operator/= (XMVECTOR& V1, FXMVECTOR V2); XMVECTOR& operator*=
(XMVECTOR& V, FLOAT S); XMVECTOR& operator/= (XMVECTOR& V, FLOAT S);
```



```
XMVECTOR operator+ (FXMVECTOR V1, FXMVECTOR V2); XMVECTOR operator-
(FXMVECTOR V1, FXMVECTOR V2); XMVECTOR operator* (FXMVECTOR V1,
FXMVECTOR V2); XMVECTOR operator/ (FXMVECTOR V1, FXMVECTOR V2); XMVECTOR
operator* (FXMVECTOR V, FLOAT S); XMVECTOR operator* (FLOAT S, FXMVECTOR V);
XMVECTOR operator/ (FXMVECTOR V, FLOAT S);
```

```
#endif // __cplusplus && !XM_NO_OPERATOR_OVERLOADS
```

1.6.6 기타

XNA Math 라이브러리는 π 가 포함된 다양한 표현식을 근사하는 데 유용한 다음 상수를 정의합니다:

```
#define XM_PI 3.141592654f
#define XM_2PI 6.283185307f

#define XM_1DIVPI 0.318309886f
#define XM_1DIV2PI 0.159154943f
#define XM_PIDIV2 1.570796327f
#define XM_PIDIV4 0.785398163f
```

또한 라디안과 도 사이의 변환을 위한 다음 인라인 함수를 정의합니다:

```
XMFINLINE FLOAT XMConvertToRadians(FLOAT fDegrees)
{
    return fDegrees * (XM_PI / 180.0f);
}

XMFINLINE FLOAT XMConvertToDegrees(FLOAT fRadians)
{
    return fRadians * (180.0f / XM_PI);
}
```

또한 최소/최대 매크로를 정의합니다:

```
#define XMMin(a, b) (((a) < (b)) ? (a) : (b))
#define XMMax(a, b) (((a) > (b)) ? (a) : (b))
```

1.6.7 세터 함수

XNA Math는 XMVECTOR의 내용을 설정하기 위해 다음과 같은 함수를 제공합니다:

<code>XMVECTOR XMVectorZero();</code>	// 0 벡터를 반환합니다.
<code>XMVECTOR XMVectorSplatOne();</code>	// 벡터 (1, 1, 1, 1)을 반환합니다
<code>XMVECTOR XMVectorSet(FLOAT x, FLOAT y, FLOAT z, FLOAT w);</code>	// 벡터 (x, y, z, w) 반환
<code>XMVECTOR XMVectorReplicate(FLOAT s);</code>	// 벡터(s, s, s, s)를 반환합니다
<code>XMVECTOR XMVectorSplatX(FXMVECTOR V);</code>	// 벡터 (v _x , v _x , v _x , v _x) 반환
<code>XMVECTOR XMVectorSplatY(FXMVECTOR V);</code>	// 벡터 (v _y , v _y , v _y , v _y)를 반환합니다.

XMVECTOR XMVectorSplatZ(FXMVECTOR V);

// 벡터 ($v_z, v_z, v_z,$
 v_z)

다음 프로그램은 이러한 함수 대부분을 보여줍니다.

```
#include <windows.h> // FLOAT 정의용 #include <xnamath.h>
#include <iostream> using
namespace std;
// "<<" 연산자를 오버로드하여 cout를 사용하여
// XMVECTOR 객체를 출력할 수 있도록.
ostream& operator<<(ostream& os, FXMVECTOR v)
{
    XMFLOAT3 dest; XMStore
    Float3(&dest, v);
    os << "(" << dest.x << ", " << dest.y << ", " << dest.z << ")"; return os;
}
int main()
{
    cout.setf(ios_base::boolalpha);
    // SSE2 지원 확인 (Pentium4, AMD K8 이상). if( !XMVerifyCPUSupport() )
    {
        cout << "xna math 지원되지 않음" << endl; return 0;
    }
    XMVECTOR p = XMVectorZero(); XMVECTOR q =
    XMVectorSplatOne();
    XMVECTOR u = XMVectorSet(1.0f, 2.0f, 3.0f, 0.0f);
    XMVECTOR v = XMVectorReplicate(-2.0f); XMVECTOR w =
    XMVectorSplatZ(u);

    cout << "p = " << p << endl; cout << "q = "
    << q << endl; cout << "u = " << u << endl;
    cout << "v = " << v << endl; cout << "w = "
    << w << endl; return 0;
}
```

1.6.8 벡터 함수

XNA Math는 다양한 벡터 연산을 수행하기 위한 다음 함수를 제공합니다. 3차원 버전을 예시로 설명하지만, 2차원과 4차원에 대한 유사한 버전이 존재합니다. 2차원 및 4차원 버전은 3차원 버전과 동일한 이름을 가지며, 각각 3 대신 2와 4가 대체됩니다.

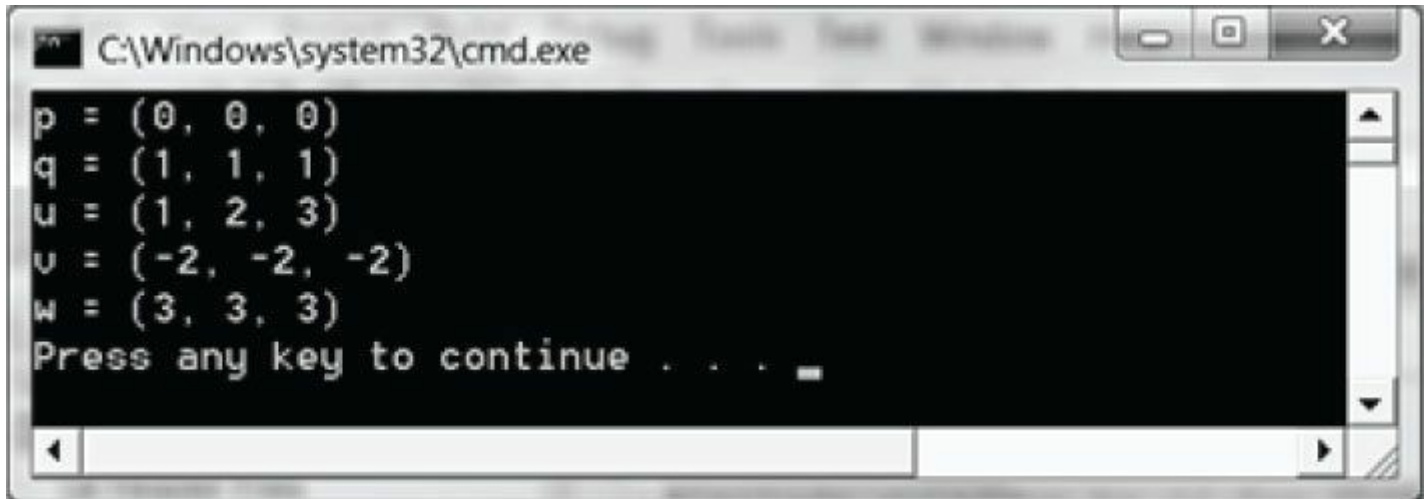


그림 1.18. 이전 프로그램의 출력 결과.

<code>XMVECTOR XMVector3Length(FXMVECTOR V);</code>	<code>//반환값 v </code> <code>// 입력 v</code>
<code>XMVECTOR XMVector3LengthSq(FXMVECTOR V);</code>	<code>//반환값 v ²</code> <code>// 입력 v</code>
<code>XMVECTOR XMVector3Dot(FXMVECTOR V1, FXMVECTOR V2);</code>	<code>// 반환값 v₁ · v₂</code> <code>// 입력 v₁</code> <code>// 입력 v₂</code>
<code>XMVECTOR XMVector3Cross(FXMVECTOR V1, FXMVECTOR V2);</code>	<code>// v₁ × v₂를 반환합니다</code> <code>// 입력 v₁</code> <code>// 입력 v₂</code>
<code>XMVECTOR XMVector3Normalize(FXMVECTOR V);</code>	<code>//반환값v/ v </code> <code>// 입력 v</code>
<code>XMVECTOR XMVector3Orthogonal(FXMVECTOR V);</code>	<code>//v에 직교하는 벡터를 반환합니다//</code> <code>입력 v</code>
<code>XMVECTOR XMVector3AngleBetweenVectors(FXMVECTOR V1, FXMVECTOR V2);</code>	<code>// v(1)과 v(2) 사이의 각도를 반환합니다</code> <code>v₁과 v(2) 사이의 각도를 반환합니다</code> <code>// 입력 v₁</code> <code>// 입력 v₂</code>
<code>VOID XMVector3ComponentsFromNormal(XMVECTOR* pParallel, XMVECTOR* pPerpendicular, FXMVECTOR V, FXMVECTOR Normal);</code>	<code>// projn(v)를 반환합니다</code> <code>// prepn(v)를 반환합니다</code> <code>// 입력 v</code> <code>// 입력 n</code>
<code>BOOL XMVector3Equal(FXMVECTOR V1, FXMVECTOR V2);</code>	<code>// v₁= v₂를 반환합니다</code> <code>// 입력 v₁</code> <code>// 입력 v₂</code>
<code>BOOL XMVector3NotEqual(FXMVECTOR V1, FXMVECTOR V2);</code>	<code>// 반환값: v₁ ≠ v₂</code> <code>// 입력 v₁</code> <code>// 입력 v₂</code>

이러한 함수들은 수학적으로 스칼라를 반환하는 연산(예: 내적 $k = \mathbf{v}_1 \cdot \mathbf{v}_2$)에 대해서도 `XMVECTORSeven`을 반환한다는 점에 유의하십시오. 스칼라 결과는 `XMVECTOR`의 각 성분에 복제됩니다.

Notes 예를 들어, 내적의 경우 반환되는 벡터는 $(\mathbf{v}_1 \cdot \mathbf{v}_2, \mathbf{v}_1 \cdot \mathbf{v}_2, \mathbf{v}_1 \cdot \mathbf{v}_2, \mathbf{v}_1 \cdot \mathbf{v}_2)$ 가 됩니다. 그 이유 중 하나는 스칼라 연산과 SIMD 벡터 연산의 혼합을 최소화하십시오. 계산이 완료될 때까지 모든 것을 SIMD로 유지하는 것이 더 효율적입니다.

다음 데모 프로그램은 이러한 함수 대부분과 일부 오버로드 연산자를 사용하는 방법을 보여줍니다:

```
#include <windows.h> // FLOAT 정의용
```

```

#include <xnamath.h> #include
<iostream> using namespace std;

// cout를 사용하여 XMVECTOR 객체를 출력할 수 있도록 "<<" 연산자를 오버로드합니다.
// XMVECTOR 객체를 출력할 수 있도록 오버로드합니다.
ostream& operator<<(ostream& os, FXMVECTOR v)
{
    XMFLOAT3 dest; XMStore
    Float3(&dest, v);
    os << "(" << dest.x << ", " << dest.y << ", " << dest.z << ")"; return os;
}

int main()
{
    cout.setf(ios_base::boolalpha);
    // SSE2 지원 확인 (Pentium4, AMD K8 이상). if( !XMVerifyCPUSupport() )
    {
cout << "xna math 지원되지 않음" << endl; return 0;
    }
    XMVECTOR n = XMVectorSet(1.0f, 0.0f, 0.0f, 0.0f); XMVECTOR u = XMVectorSet(1.0f, 2.0f,
    3.0f, 0.0f); XMVECTOR v = XMVectorSet(-2.0f, 1.0f, -3.0f, 0.0f); XMVECTOR w =
    XMVectorSet(0.707f, 0.707f, 0.0f, 0.0f);

    // 벡터 덧셈: XMVECTOR operator + XMVECTOR a = u + v;

    // 벡터 뺄셈: XMVECTOR operator - XMVECTOR b = u - v;

    // 스칼라 곱셈: XMVECTOR operator * XMVECTOR c = 10.0f*u;

    // ||u||
    XMVECTOR L = XMVector3Length(u);

    // d = u / ||u||
    XMVECTOR d = XMVector3Normalize(u);

    // s = u dot v
    XMVECTOR s = XMVector3Dot(u, v);

    // e = u x v
    XMVECTOR e = XMVector3Cross(u, v);

    // proj_n(w)와 perp_n(w) 구하기 XMVECTOR
    projW; XMVECTOR perpW;
    XMVector3ComponentsFromNormal(&projW, &perpW, w, n);

    // projW + perpW == w인가?
    bool equal = XMVector3Equal(projW + perpW, w) != 0;
    bool notEqual = XMVector3NotEqual(projW + perpW, w) != 0;

    // projW와 perpW 사이의 각도는 90도여야 합니다.
    XMVECTOR angle Vec = XMVector3Angle BetweenVectors(projW, perpW); float angle Radians =
    XMVectorGetX(angle Vec);
    float angle Degrees = XMConvertToDegrees(angle Radians);

```

```

    cout << "u
    cout << "v
    cout << "w
    cout << "n
    cout << "a = u + v
    cout << "b = u - v
    cout << "c = 10 * u
    cout << "d = u / ||u||
    cout << "e = u x v
    cout << "L = ||u||
    cout << "s = u.v
    cout << "projW
    cout << "perpW
    cout << "projW + perpW == w
    cout << "projW + perpW != w
    cout << "각도

    return 0;
}

```

```

= " << u << endl;
= " << v << endl;
= " << w << endl;
= " << n << endl;
= " << a << endl;
= " << b << endl;
= " << c << endl;
= " << d << endl;
= " << e << endl;
= " << L << endl;
= " << s << endl;
= " << projW << endl;
= " << perpW << endl;
= " << equal << endl;
= " << notEqual << endl;
= " << angle Degrees << endl;

```

```

u = (1, 2, 3)
v = (-2, 1, -3)
w = (0.707, 0.707, 0)
n = (1, 0, 0)
a = u + v = (-1, 3, 0)
b = u - v = (3, 1, 6)
c = 10 * u = (10, 20, 30)
d = u / ||u|| = (0.267261, 0.534522, 0.801784)
e = u x v = (-9, -3, 5)
L = ||u|| = (3.74166, 3.74166, 3.74166)
s = u.v = (-9, -9, -9)
projW = (0.707, 0, 0)
perpW = (0, 0.707, 0)
projW + perpW == w = true
projW + perpW != w = false
angle = 90
Press any key to continue . . .

```

그림 1.19. 위 프로그램의 출력 결과.

XNA 수학 라이브러리에는 정확도는 낮지만 계산 속도가 빠른 추정 메서드도 포함되어 있습니다. 속도를 위해 정확도를 일부 희생할 의향이 있다면 추정 메서드를 사용하십시오. 다음은 추정 함수의 두 가지 예시입니다.

함수 예시:

```
MFINLINE XMVECTOR XMVector3LengthEst(XMVECTOR V);
```

```
// 추정된  $\| \mathbf{v} \|$  반환
```

```
// 입력  $\mathbf{v}$ 
```

```
MFINLINE XMVECTOR XMVector3NormalizeEst(XMVECTOR V);
```

```
//반환값추정된 $\mathbf{v}/\|\mathbf{v}\|$ 
```

```
// 입력  $\mathbf{v}$ 
```

1.6.9 부동 소수점 오차

컴퓨터에서 벡터를 다루는 것과 관련하여 다음 사항을 유의해야 합니다. 부동 소수점 수를 비교할 때는 부동 소수점의 부정확성으로 인해 주의가 필요합니다. 동일할 것으로 예상되는 두 부동 소수점 수가 미세하게 다를 수 있습니다. 예를 들어, 수학적으로 정규화된 벡터의 길이는 1이어야 하지만, 컴퓨터 프로그램에서는 길이가 대략 1에 불과할 수 있습니다. 또한 수학적으로 $1^p = 1$ 이 성립하는 모든 실수 p 에 대해, 우리가 의 수치적 근사값만을 가질 때는 근사값을 제공하면 오차가 증가하므로 수치적 오차도 누적됩니다. 다음의 짧은 프로그램이 이러한 개념을 보여줍니다.

```
#include <windows.h> // FLOAT 정의용 #include <xmath.h>
#include <iostream> using
namespace std; int main()
{

    cout.precision(8);

    // SSE2 지원 확인 (Pentium4, AMD K8 이상). if( !XMVerifyCPUSupport() )
    {
        cout << "xna math 지원되지 않음" << endl; return 0;
    }

    XMVECTOR u = XMVectorSet(1.0f, 1.0f, 1.0f, 0.0f);
    XMVECTOR n = XMVector3Normalize(u);

    float LU = XMVectorGetX(XMVector3Length(n));

    // 수학적으로 길이는 1이어야 합니다. 실제로도 그럴까요? cout << LU << endl;
    if( LU == 1.0f )
        cout << "길이 1" << endl; else
        cout << "길이가 1이 아닙니다" << endl;

    // 1을 어떤 지수로 제곱해도 여전히 1이어야 합니다. 맞나요? float powLU =
    powf(LU, 1.0e 6f);
    cout << "LU^(10^6) = " << powLU << endl;
}
```

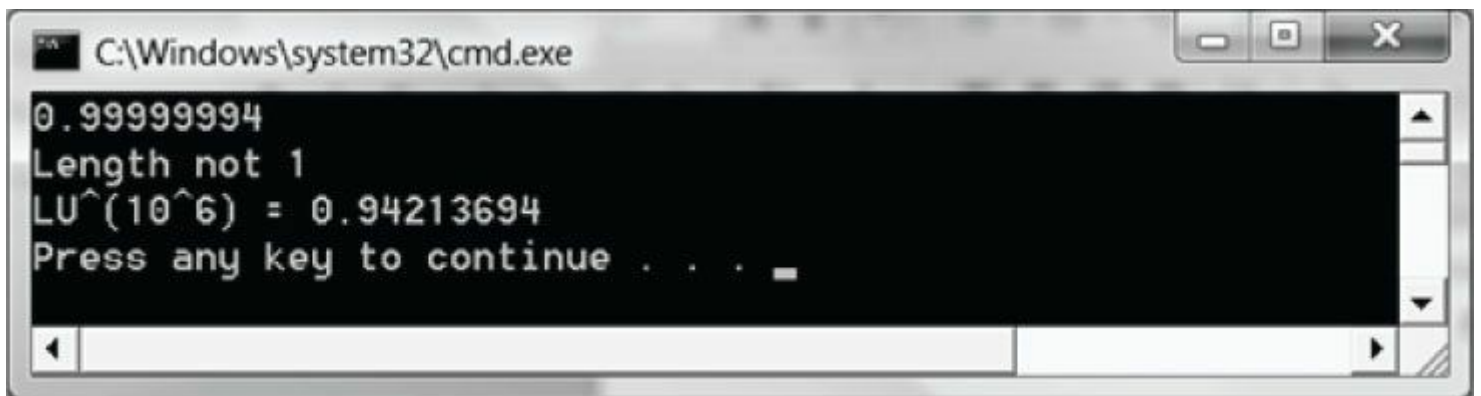


그림 1.20. 위 프로그램의 출력 결과.

부동 소수점의 부정확성을 보정하기 위해, 두 부동 소수점 숫자가 근사적으로 동일인지 테스트합니다. 이를 위해 매우 작은 값인 '에프실론(Epsilon)' 상수를 정의하여 '완충 장치'로 사용합니다. 두 값의 차이가 에프실론보다 작으면 근사적으로 같다고 말합니다. 즉, 에프실론은 부동 소수점 오차에 대한 허용 오차를 제공합니다. 다음 함수는 에프실론을 사용하여 두 부동 소수점 값이 같은지 테스트하는 방법을 보여줍니다:

```
const float Epsilon = 0.001f; bool Equals(float
lhs, float rhs)
{

    // lhs와 rhs 사이의 거리가 EPSILON보다 작은가? return fabs(lhs - rhs) < Epsilon ? true : false;

}
```

XNA Math 라이브러리는 허용 오차 Epsilon 매개변수를 사용하여 벡터의 동일성을 테스트할 때 XMVector3NearEqual 함수를 제공합니다:

```
// 반환값
// |U.x - V.x| <= Epsilon.x &&
// |U.y - V.y| <= Epsilon.y &&
// abs(U.z - V.z) <= Epsilon.z
XMFLOAT3 NearEqual( FXMVECTOR U,
                    FXMVECTOR V,
                    FXMVECTOR Epsilon);
```

1.7 요약

1. 벡터는 크기와 방향을 모두 지닌 물리량을 모델링하는 데 사용됩니다. 기하학적으로 벡터는 방향성 선분으로 표현됩니다. 벡터는 자신의 꼬리가 좌표계의 원점과 일치하도록 평행 이동된 상태를 표준 위치라고 합니다. 표준 위치에 있는 벡터는 좌표계에 대한 머리 부분의 좌표를 지정하여 수치적으로 기술할 수 있습니다.

2. $\mathbf{u} = (u_x, u_y, u_z)$ 및 $\mathbf{v} = (v_x, v_y, v_z)$ 일 때, 다음과 같은 벡터 연산이 성립한다: 합: $\mathbf{u} + \mathbf{v} = (u_x + v_x, u_y + v_y, u_z + v_z)$

뱀셈: $\mathbf{u} \cdot \mathbf{v} = (u_x \cdot v_x, u_y \cdot v_y, u_z \cdot v_z)$ 스칼라 곱셈: $k \mathbf{u} = (ku_x, ku_y, ku_z)$

길이: $|| \mathbf{u} || = \sqrt{x^2 + y^2 + z^2}$
정규화: $\hat{\mathbf{u}} = \frac{\mathbf{u}}{||\mathbf{u}||} = \left(\frac{x}{||\mathbf{u}||}, \frac{y}{||\mathbf{u}||}, \frac{z}{||\mathbf{u}||} \right)$

외적: $\mathbf{u} \cdot \mathbf{v} = || \mathbf{u} || || \mathbf{v} || \cos \theta = u_x v_x + u_y v_y + u_z v_z$

외적: $\mathbf{u} \times \mathbf{v} = (u_y v_z - u_z v_y, u_z v_x - u_x v_z, u_x v_y - u_y v_x)$

3. 우리는 SIMD 연산을 사용하여 코드에서 벡터를 효율적으로 표현하기 위해 XNA Math XMVECTOR 클래스를 사용합니다. 클래스 데이터 멤버의 경우 XMVECTOR3 클래스를 사용하며, 로딩 및 저장 메서드를 통해 XMVECTOR와 XMVECTOR3 간에 변환합니다. 초기화 구문이 필요한 상수 벡터는 XMVECTORF32 유형을 사용해야 합니다.

4. SIMD의 이점을 활용하기 위해 XMVECTOR 유형 함수에 매개변수를 전달할 때 몇 가지 규칙이 있습니다. 플랫폼 독립성을 위해 XMVECTOR 매개변수 전달에는 CXMVECTOR 및 FXMVECTOR 유형을 사용합니다. 이 유형들은 플랫폼에 따라 적절한 유형으로 정의됩니다. XMVECTOR 매개변수 전달 규칙은 다음과 같습니다: 첫 3개의 XMVECTOR 매개변수는 FXMVECTOR 유형이어야 하며, 추가 XMVECTOR 매개변수는 CXMVECTOR 유형이어야 합니다.

5. XMVECTOR 클래스는 벡터 덧셈, 뱀셈 및 스칼라 곱셈을 수행하기 위해 산술 연산자를 오버로드합니다. 또한 XNA Math 라이브러리는 벡터의 길이 계산, 벡터의 제곱 길이 계산, 두 벡터의 내적 계산, 두 벡터의 외적 계산 및 벡터 정규화를 위한 다음과 같은 유용한 함수를 제공합니다:

```
XMVECTOR XMVector3Length(FXMVECTOR V); XMVECTOR
XMVector3LengthSq(FXMVECTOR V);
XMVECTOR XMVector3Dot(FXMVECTOR V1, FXMVECTOR V2); XMVECTOR XMVector3Cross(FXMVECTOR
V1, FXMVECTOR V2);
```

1.8 연습문제

1. $\mathbf{u} = (1, 2)$ 및 $\mathbf{v} = (3, -4)$ 라고 하자. 다음 계산을 수행하고 2차원 좌표계에 대한 벡터를 그려라.

+ $\mathbf{v}$

- $\mathbf{v}$

2 $\mathbf{u} + \mathbf{v}$

2. $\mathbf{u} = (-1, 3, 2)$ 및 $\mathbf{v} = (3, -4, 1)$ 이라 하자. 다음 계산을 수행하라.

+ $\mathbf{v}$

- $\mathbf{v}$

$\mathbf{u} + 2\mathbf{v}$ $2\mathbf{u}$

+ $\mathbf{v}$

3. 이 연습 문제는 벡터 대수가 실수의 많은 훌륭한 성질을 공유함을 보여줍니다(이것은 완전한 목록이 아닙니다). $\mathbf{u} = (u_x, u_y, u_z)$, $\mathbf{v} = (v_x, v_y, v_z)$, $\mathbf{w} = (w_x, w_y, w_z)$ 라고 하자. 또한 c 와 k 가 스칼라라고 하자. 다음 벡터 성질을 증명하라.

+ $\mathbf{v} = \mathbf{v} + \mathbf{u}$ (덧셈의 교환법칙) 덧셈의 교환법칙 덧셈의 교환법칙

+ $(\mathbf{v} + \mathbf{w}) = (\mathbf{u} + \mathbf{v}) + \mathbf{w}$ (덧셈의 결합법칙)

$k(\mathbf{u}) = c(k\mathbf{u})$ (스칼라 곱셈의 결합법칙) $\mathbf{u} + \mathbf{v} = k\mathbf{u} + k\mathbf{v}$ (분배법칙 1)

$(k + c)\mathbf{u} = k\mathbf{u} + c\mathbf{u}$ (분배법칙 2)



벡터 연산의 정의와 실수의 성질을 사용하기만 하면 됩니다. 예를 들어,

$$\begin{aligned}(ck)\mathbf{u} &= (ck)(u_x, u_y, u_z) \\ &= ((ck)u_x, (ck)u_y, (ck)u_z) \\ &= (c(ku_x), c(ku_y), c(ku_z)) \\ &= c(ku_x, ku_y, ku_z) = c(k\mathbf{u})\end{aligned}$$

4. 방정식 $2((1, 2, 3) - \mathbf{x}) - (-2, 0, 4) = -2(1, 2, 3)$ 을 $\mathbf{x}$ 에 대해 풀으시오.

5. $\mathbf{u} = (-1, 3, 2)$ 및 $\mathbf{v} = (3, -4, 1)$ 이라 하자. $\mathbf{u}$ 와 $\mathbf{v}$ 를 정규화하라.

6. k 를 스칼라라 하고 $\mathbf{u} = (u_x, u_y, u_z)$ 라 하자. $\|k\mathbf{u}\| = |k|\|\mathbf{u}\|$ 임을 증명하라.

7. 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 사이의 각도는 직각, 예각, 둔각 중 어느 것인가?

$\mathbf{u} = (1, 1, 1)$, $\mathbf{v} = (2, 3, 4)$

$\mathbf{u} = (1, 1, 0)$, $\mathbf{v} = (-2, 2, 0)$

$\mathbf{u} = (-1, -1, -1)$, $\mathbf{v} = (3, 1, 0)$

8. $\mathbf{u} = (-1, 3, 2)$ 및 $\mathbf{v} = (3, -4, 1)$ 이라 하자. $\mathbf{u}$ 와 $\mathbf{v}$ 사이의 각도 θ 를 구하라.

9. $\mathbf{u} = (u_x, u_y, u_z)$, $\mathbf{v} = (v_x, v_y, v_z)$, $\mathbf{w} = (w_x, w_y, w_z)$ 라 하자. 또한 c 와 k 를 스칼라라 하자. 다음의 내적 성질을 증명하라.

• $\mathbf{v} = \mathbf{v} \cdot \mathbf{u}$

• $(\mathbf{v} + \mathbf{w}) \cdot \mathbf{u} = \mathbf{v} \cdot \mathbf{u} + \mathbf{w} \cdot \mathbf{u}$

$\mathbf{u} \cdot (k\mathbf{v}) = (k\mathbf{u}) \cdot \mathbf{v} = k(\mathbf{u} \cdot \mathbf{v})$

• $\|\mathbf{v}\|^2 = \mathbf{v} \cdot \mathbf{v}$

$$\mathbf{v} \cdot \mathbf{v} = 0$$

 정의만 사용하면 됩니다. 예를 들어,

$$\begin{aligned}\mathbf{v} \cdot \mathbf{v} &= v_x v_x + v_y v_y + v_z v_z \\ &= v_x^2 + v_y^2 + v_z^2 \\ &= \left(\sqrt{v_x^2 + v_y^2 + v_z^2} \right)^2 \\ &= \|\mathbf{v}\|^2\end{aligned}$$

10. 코사인 법칙($c^2 = a^2 + b^2 - 2ab \cos \theta$, 여기서 a, b, c 는 삼각형의 변의 길이이고 θ 는 변 a 와 b 사이의 각도임)을 사용하여 증명하십시오.

$$u_x v_x + u_y v_y + u_z v_z = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

 그림 1.9를 고려하고 $c^2 = \|\mathbf{u} - \mathbf{v}\|^2$, $a^2 = \|\mathbf{u}\|^2$, $b^2 = \|\mathbf{v}\|^2$ 로 설정하고, 이전 연습문제의 내적 성질을 사용하십시오.

11. $\mathbf{n} = (-2, 1)$ 이라 하자. 벡터 $\mathbf{g} = (0, -9.8)$ 을 두 개의 직교하는 벡터의 합으로 분해하되, 하나는 $\mathbf{n}$ 과 평행하고 다른 하나는 $\mathbf{n}$ 과 직교하도록 하라. 또한 2차원 좌표계에 대한 벡터들을 그려라.

12. $\mathbf{u} = (-2, 1, 4)$ 및 $\mathbf{v} = (3, -4, 1)$ 이라 하자. $\mathbf{w} = \mathbf{u} \times \mathbf{v}$ 를 구하고, $\mathbf{w} \cdot \mathbf{u} = 0$ 및 $\mathbf{w} \cdot \mathbf{v} = 0$ 을 증명하라.

13. 다음 점들이 어떤 좌표계에 대해 삼각형을 정의한다고 하자: $\mathbf{A} = (0, 0, 0)$, $\mathbf{B} = (0, 1, 3)$, $\mathbf{C} = (5, 1, 0)$. 이 삼각형에 수직인 벡터를 구하라.

 삼각형의 두 변 위에 두 벡터를 찾고 외적을 사용하라.

14. $\|\mathbf{u} \times \mathbf{v}\| = \|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$ 임을 증명하라.

 $\|\mathbf{u}\| \|\mathbf{v}\| \sin \theta$ 로 시작하여 삼각함수 항등식을 사용하라 $\cos^2 \theta + \sin^2 \theta = 1 \Rightarrow \sin \theta = \sqrt{1 - \cos^2 \theta}$
 $\cos^2 \theta + \sin^2 \theta = 1 \Rightarrow \sin \theta = \sqrt{1 - \cos^2 \theta}$ 을 적용한 후 방정식 1.4를 적용하십시오.

15. $\|\mathbf{u} \times \mathbf{v}\|$ 가 $\mathbf{u}$ 와 $\mathbf{v}$ 가 이루는 평행사변형의 면적을 준다는 것을 증명하라; 그림 1.21을 보라.

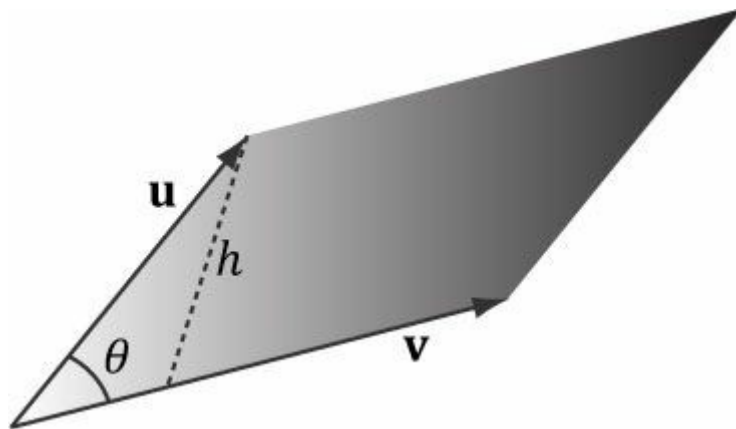


그림 1.21. 두 3차원 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 가 이루는 평행사변형; 이 평행사변형의 밑변은 $\|\mathbf{v}\|$ 이고 높이는 h 이다.

16. $\mathbf{u} \times (\mathbf{v} \times \mathbf{w}) \neq (\mathbf{u} \times \mathbf{v}) \times \mathbf{w}$ 를 만족하는 3차원 벡터 $\mathbf{u}, \mathbf{v}, \mathbf{w}$ 의 예를 제시하라. 이는 외적이 일반적으로 결합법칙을 따르지 않음을 보여준다.

 단순 벡터 $\mathbf{i} = (1, 0, 0)$, $\mathbf{j} = (0, 1, 0)$, $\mathbf{k} = (0, 0, 1)$ 의 조합을 고려해 보십시오.

17. 두 개의 0이 아닌 평행 벡터의 외적은 영벡터를 결과로 한다는 것을 증명하라; 즉, $\mathbf{u} \times \mathbf{k} = \mathbf{0}$ 이다.



단순히 외적의 정의를 사용하면 된다.

18. 그램-슈미트 과정을 사용하여 벡터 집합 $\{(1, 0, 0), (1, 5, 0), (2, 1, -4)\}$ 을 직교 정규화하십시오.

19. 다음 프로그램과 출력을 고려하십시오. 각 XMVECTOR* 함수가 무엇을 하는지 추측한 다음, XNA Math 문서에서 각 함수를 찾아보십시오.

```
#include <windows.h> // FLOAT 정의용 #include <xnamath.h>
#include <iostream> using
namespace std;

// "<<" 연산자를 오버로드하여 cout를 사용하여
// XMVECTOR 객체를 출력할 수 있도록 합니다.
ostream& operator<<(ostream& os, FXMVECTOR v)
{
    XMFLOAT4 dest; XMStore
    Float4(&dest, v);

    os << "(" << dest.x << ", " << dest.y << ", "
<< dest.z << ", " << dest.w << ")"; return os;
}

int main()
{
    cout.setf(ios_base::boolalpha);

    // SSE2 지원 확인 (Pentium4, AMD K8 이상). if( !XMVerifyCPUSupport() )
    {
        cout << "xna math 지원되지 않음" << endl; return 0;
    }

    XMVECTOR p = XMVectorSet(2.0f, 2.0f, 1.0f, 0.0f); XMVECTOR q = XMVectorSet(2.0f, -0.5f,
    0.5f, 0.1f); XMVECTOR u = XMVectorSet(1.0f, 2.0f, 4.0f, 8.0f); XMVECTOR v =
    XMVectorSet(-2.0f, 1.0f, -3.0f, 2.5f);
    XMVECTOR w = XMVectorSet(0.0f, XM_PIDIV4, XM_PIDIV2, XM_PI);

    cout << "XMVectorAbs(v) = " << XMVectorAbs(v) << endl; cout << "XMVectorCos(w) = "
    << XMVectorCos(w) << endl; cout << "XMVectorLog(u) = " << XMVectorLog(u) << endl;
    cout << "XMVectorExp(p) = " << XMVectorExp(p) << endl;

    cout << "XMVectorPow(u, p) = " << XMVectorPow(u, p) << endl; cout << "XMVectorSqrt(u) = "
    << XMVectorSqrt(u) << endl;

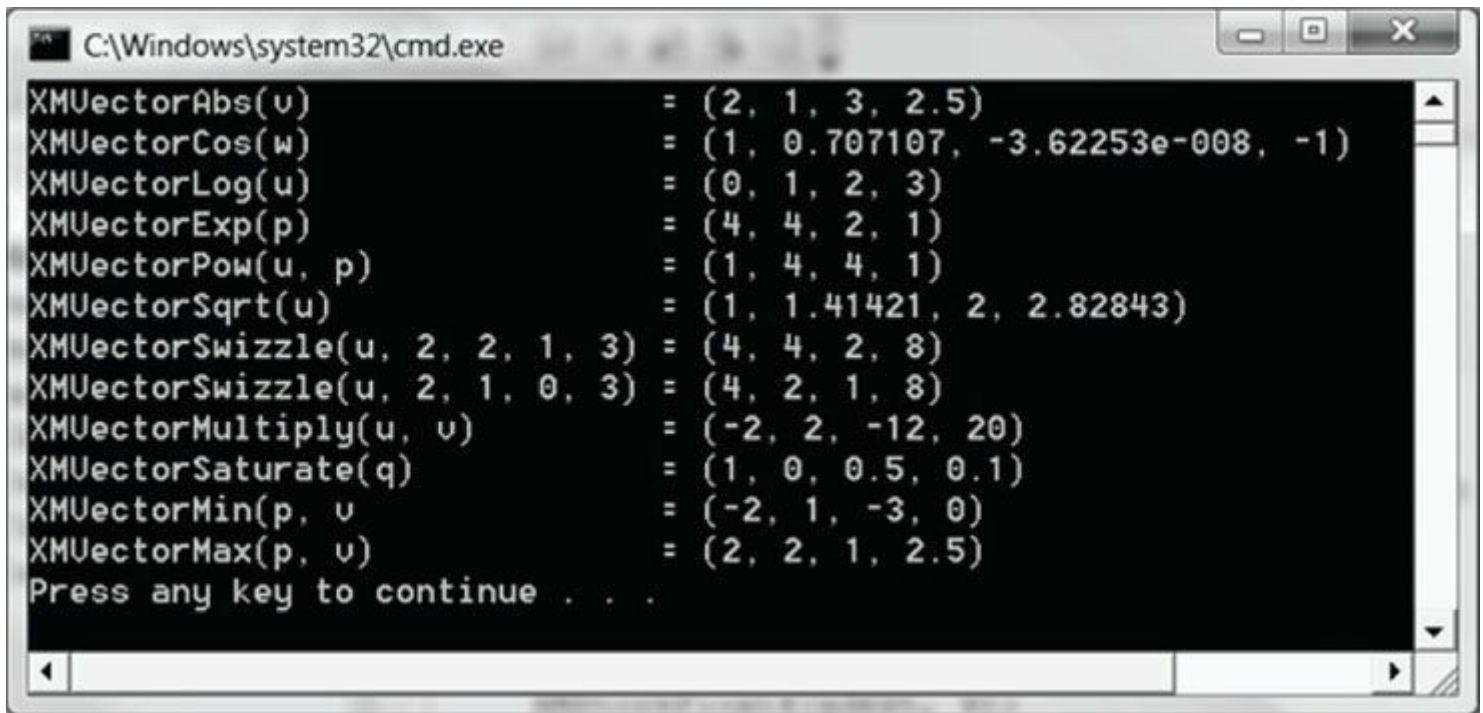
    cout << "XMVectorSwizzle(u, 2, 2, 1, 3) = "
    << XMVectorSwizzle(u, 2, 2, 1, 3) << endl;
    cout << "XMVectorSwizzle(u, 2, 1, 0, 3) = " << endl;
    << XMVectorSwizzle(u, 2, 1, 0, 3) << endl;

    cout << "XMVectorMultiply(u, v) = "
    << XMVectorMultiply(u, v) << endl;
    cout << "XMVectorSaturate(q) = "
    << XMVectorSaturate(q) << endl;
    cout << "XMVectorMin(p, v) = " << XMVectorMin(p, v) << endl;
```

```
cout << "XMVectorMax(p, v) = " << XMVectorMax(p, v) << endl;
```

```
return 0;
```

```
}
```



A screenshot of a Windows command prompt window titled "C:\Windows\system32\cmd.exe". The window has a black background with white text. It displays the output of several XMVector functions. The functions and their results are as follows:

Function	Result
XMVectorAbs(u)	= (2, 1, 3, 2.5)
XMVectorCos(w)	= (1, 0.707107, -3.62253e-008, -1)
XMVectorLog(u)	= (0, 1, 2, 3)
XMVectorExp(p)	= (4, 4, 2, 1)
XMVectorPow(u, p)	= (1, 4, 4, 1)
XMVectorSqrt(u)	= (1, 1.41421, 2, 2.82843)
XMVectorSwizzle(u, 2, 2, 1, 3)	= (4, 4, 2, 8)
XMVectorSwizzle(u, 2, 1, 0, 3)	= (4, 2, 1, 8)
XMVectorMultiply(u, v)	= (-2, 2, -12, 20)
XMVectorSaturate(q)	= (1, 0, 0.5, 0.1)
XMVectorMin(p, v)	= (-2, 1, -3, 0)
XMVectorMax(p, v)	= (2, 2, 1, 2.5)

At the bottom of the window, it says "Press any key to continue . . .".

그림 1.22. 이전 프로그램의 출력 결과.

제2장 행렬 대수

3차원 컴퓨터 그래픽스에서는 스케일링, 회전, 평행 이동과 같은 기하학적 변환을 간결하게 표현하고, 한 프레임에서 다른 프레임으로 점이나 벡터의 좌표를 변환하기 위해 행렬을 사용합니다. 이 장에서는 행렬의 수학적 원리를 탐구합니다.

- 목표:
- 1. 행렬과 그 위에 정의된 연산에 대한 이해를 얻기 위함.
 - 2. 벡터-행렬 곱셈이 어떻게 선형 결합으로 볼 수 있는지 알아보기.
 - 3. 단위 행렬이 무엇인지, 그리고 행렬의 전치, 행렬식, 역행렬이 무엇인지 학습한다.
 - 4. XNA 라이브러리가 제공하는 행렬 연산용 클래스와 함수의 하위 집합에 익숙해지기 위해.

2.1 정의

$m \times n$ 행렬 $\mathbf{M}$ 은 m 개의 행과 n 개의 열을 가진 실수의 직사각형 배열입니다. 행과 열의 개수를 곱한 값이 행렬의 차원을 나타냅니다. 행렬의 숫자들은 요소 또는 엔트리라고 합니다. 행렬 요소는 이중 첨자 표기법 $M_{(ij)}$ 를 사용하여 요소의 행과 열을 지정함으로써 식별합니다. 여기서 첫 번째 첨자는 행을, 두 번째 첨자는 열을 나타냅니다.

예제 2.1

다음 행렬들을 고려하라:

$$\mathbf{A} = \begin{bmatrix} 3.5 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 2 & -5 & \sqrt{2} & 1 \end{bmatrix} \quad \mathbf{B} = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \\ B_{31} & B_{32} \end{bmatrix} \quad \mathbf{u} = [u_1, u_2, u_3] \quad \mathbf{v} = \begin{bmatrix} 1 \\ 2 \\ \sqrt{3} \\ \pi \end{bmatrix}$$

- 1. 행렬 $\mathbf{A}$ 는 4×4 행렬이고, 행렬 $\mathbf{B}$ 는 3×2 행렬이며, 행렬 $\mathbf{u}$ 는 1×3 행렬이고, 행렬 $\mathbf{v}$ 는 4×1 행렬이다.
- 2. 행렬 $\mathbf{A}$ 의 네 번째 행과 두 번째 열에 있는 원소는 $A_{42} = -5$ 로 식별합니다. 행렬 $\mathbf{B}$ 의 두 번째 행과 첫 번째 열에 있는 원소는 $B_{(21)}$ 로 식별합니다.
- 3. 행렬 $\mathbf{u}$ 와 $\mathbf{v}$ 는 각각 단일 행 또는 단일 열만을 포함한다는 점에서 특수한 행렬이다. 이러한 종류의 행렬을 행 벡터 또는 열 벡터라고 부르기도 하는데, 이는 벡터를 행렬 형태로 표현하는 데 사용되기 때문입니다(예: 벡터 표기법 (x, y, z) 와 $[x, y, z]$ 를 자유롭게 교환할 수 있음). 행 벡터와 열 벡터의 경우 행렬의 원소를 나타내기 위해 이중 첨자를 사용할 필요가 없으며, 하나의 첨자만 있으면 충분합니다.

때로는 행렬의 행을 벡터로 생각하기도 합니다. 예를 들어 다음과 같이 쓸 수 있습니다:

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} \leftarrow \mathbf{A}_{1,*} \rightarrow \\ \leftarrow \mathbf{A}_{2,*} \rightarrow \\ \leftarrow \mathbf{A}_{3,*} \rightarrow \end{bmatrix}$$

여기서 $\mathbf{A}_{1,*} = [A_{11}, A_{12}, A_{13}]$, $\mathbf{A}_{2,*} = [A_{21}, A_{22}, A_{23}]$, 그리고 $\mathbf{A}_{3,*} = [A_{31}, A_{32}, A_{33}]$ 이다. 이 표기법에서 첫 번째 인덱스는 행을 지정하며, 두 번째 인덱스에 '*'를 넣어 행 벡터 전체를 참조함을 나타냅니다. 마찬가지로 열에 대해서도 동일한 방식을 적용하고자 합니다.

열에 대해서도 동일하게 처리합니다.

$$\begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ \mathbf{A}_{*,1} & \mathbf{A}_{*,2} & \mathbf{A}_{*,3} \\ \downarrow & \downarrow & \downarrow \end{bmatrix}$$

여기서

$$\mathbf{A}_{*,1} = \begin{bmatrix} A_{11} \\ A_{21} \\ A_{31} \end{bmatrix}, \quad \mathbf{A}_{*,2} = \begin{bmatrix} A_{12} \\ A_{22} \\ A_{32} \end{bmatrix}, \quad \mathbf{A}_{*,3} = \begin{bmatrix} A_{13} \\ A_{23} \\ A_{33} \end{bmatrix}$$

이 표기법에서 두 번째 인덱스는 열을 지정하며, 첫 번째 인덱스에 '*'를 넣어 전체 열 벡터를 참조함을 나타냅니다.

이제 행렬에 대한 등호, 덧셈, 스칼라 곱셈, 뺄셈을 정의한다.

1. 두 행렬은 대응하는 원소들이 같을 때만 같으며, 따라서 두 행렬을 비교하려면 동일한 행과 열의 개수를 가져야 합니다.
2. 두 행렬을 더할 때는 서로 대응되는 원소들을 더합니다. 따라서 행과 열의 개수가 같은 행렬들만 더하는 것이 의미가 있습니다.
3. 스칼라와 행렬의 곱셈은 스칼라를 행렬의 모든 원소와 곱하는 방식으로 정의한다.
4. 뺄셈은 행렬 덧셈과 스칼라 곱셈을 통해 정의한다. 즉, $\mathbf{A} - \mathbf{B} = \mathbf{A} + (-1 \cdot \mathbf{B}) = \mathbf{A} + (-\mathbf{B})$ 이다.

예제 2.2

다음과 같이 정의하자.

$$\mathbf{A} = \begin{bmatrix} 1 & 5 \\ -2 & 3 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 6 & 2 \\ 5 & -8 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 5 \\ -2 & 3 \end{bmatrix}, \quad \mathbf{D} = \begin{bmatrix} 2 & 1 & -3 \\ -6 & 3 & 0 \end{bmatrix}$$

그러면,

$$(i) \quad \mathbf{A} + \mathbf{B} = \begin{bmatrix} 1 & 5 \\ -2 & 3 \end{bmatrix} + \begin{bmatrix} 6 & 2 \\ 5 & -8 \end{bmatrix} = \begin{bmatrix} 1+6 & 5+2 \\ -2+5 & 3+(-8) \end{bmatrix} = \begin{bmatrix} 7 & 7 \\ 3 & -5 \end{bmatrix}$$

$$(ii) \quad \mathbf{A} = \mathbf{C}$$

$$(iii) \quad 3\mathbf{D} = 3 \begin{bmatrix} 2 & 1 & -3 \\ -6 & 3 & 0 \end{bmatrix} = \begin{bmatrix} 3(2) & 3(1) & 3(-3) \\ 3(-6) & 3(3) & 3(0) \end{bmatrix} = \begin{bmatrix} 6 & 3 & -9 \\ -18 & 9 & 0 \end{bmatrix}$$

$$(iv) \quad \mathbf{A} - \mathbf{B} = \begin{bmatrix} 1 & 5 \\ -2 & 3 \end{bmatrix} - \begin{bmatrix} 6 & 2 \\ 5 & -8 \end{bmatrix} = \begin{bmatrix} 1-6 & 5-2 \\ -2-5 & 3-(-8) \end{bmatrix} = \begin{bmatrix} -5 & 3 \\ -7 & 11 \end{bmatrix}$$

덧셈과 스칼라 곱셈이 원소별로 수행되기 때문에, 행렬은 본질적으로 실수로부터 다음과 같은 덧셈과 스칼라 곱셈 성질을 상속받습니다.

1. $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$ 덧셈의 교환법칙
2. $(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \mathbf{A} + (\mathbf{B} + \mathbf{C})$ 덧셈의 결합법칙
3. $r(\mathbf{A} + \mathbf{B}) = r\mathbf{A} + r\mathbf{B}$ 행렬에 대한 스칼라 분배법칙
4. $(r + s)\mathbf{A} = r\mathbf{A} + s\mathbf{A}$ 스칼라에 대한 행렬의 배분법

2.2 행렬 곱셈

2.2.1 정의

A가 $m \times n$ 행렬이고 B가 $n \times p$ 행렬일 때, 곱 AB 는 정의되며 $m \times p$ 행렬 C를 이룬다. 여기서 곱 C의 ij번째 원소는 A의 i번째 행 벡터와 B의 j번째 열 벡터의 내적을 취함으로써 주어지며, 즉

$$C_{ij} = A_{i,*} \cdot B_{*,j} \quad (\text{식 2.1})$$

따라서 행렬 곱 AB 가 정의되기 위해서는 A의 열 개수와 B의 행 개수가 같아야 하며, 즉 A의 행 벡터 차원과 B의 열 벡터 차원이 같아야 함을 유의하십시오. 이 차원들이 일치하지 않는다면, 식 2.1의 내적은 의미가 없게 됩니다.

예제 2.3

A = [1, 2, 3, 4, 5, 6, 7,

$$A = \begin{bmatrix} 1 & 5 \\ -2 & 3 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 2 & -6 \\ 1 & 3 \\ -3 & 0 \end{bmatrix}$$

AB 의 곱은 정의되지 않습니다. 왜냐하면 A의 행 벡터는 차원이 2이고 B의 열 벡터는 차원이 3이기 때문입니다. 특히, A의 첫 번째 행 벡터와 B의 첫 번째 열 벡터의 내적을 취할 수 없습니다. 2차원 벡터와 3차원 벡터의 내적을 취할 수 없기 때문입니다.

예제 2.4

다음과 같이 하자.

$$A = \begin{bmatrix} -1 & 5 & -4 \\ 3 & 2 & 1 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 2 & 1 & 0 \\ 0 & -2 & 1 \\ -1 & 2 & 3 \end{bmatrix}$$

먼저, A 행렬의 열 개수와 B 행렬의 행 개수가 같기 때문에 곱 AB 가 정의되며(그리고 행렬이다)

B의 행 수와 같기 때문이다. 방정식 2.1을 적용하면 다음과 같다:

$$\begin{aligned} AB &= \begin{bmatrix} -1 & 5 & -4 \\ 3 & 2 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 0 \\ 0 & -2 & 1 \\ -1 & 2 & 3 \end{bmatrix} \\ &= \begin{bmatrix} (-1, 5, -4) \cdot (2, 0, -1) & (-1, 5, -4) \cdot (1, -2, 2) & (-1, 5, -4) \cdot (0, 1, 3) \\ (3, 2, 1) \cdot (2, 0, -1) & (3, 2, 1) \cdot (1, -2, 2) & (3, 2, 1) \cdot (0, 1, 3) \end{bmatrix} \\ &= \begin{bmatrix} 2 & -19 & -7 \\ 5 & 1 & 5 \end{bmatrix} \end{aligned}$$

B의 열 개수가 A의 행 개수와 같지 않으므로 곱 BA 는 정의되지 않음을 관찰하십시오. 이는 일반적으로 행렬 곱셈이 교환법칙을 따르지 않음을 보여줍니다. 즉, $AB \neq BA$ 입니다.

이는 일반적으로 행렬 곱셈이 교환법칙을 따르지 않음을 보여줍니다. 즉, $\mathbf{AB} \neq \mathbf{BA}$ 입니다.

2.2.2 벡터-행렬 곱셈

다음 벡터-행렬 곱셈을 고려하라:

$$\mathbf{uA} = \begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = \begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} \uparrow & \uparrow & \uparrow \\ \mathbf{A}_{*,1} & \mathbf{A}_{*,2} & \mathbf{A}_{*,3} \\ \downarrow & \downarrow & \downarrow \end{bmatrix}$$

이 경우 $\mathbf{uA}$ 는 1×3 행 벡터로 평가됨을 알 수 있습니다. 이제 방정식 2.1을 적용하면 다음과 같습니다:

$$\begin{aligned} \mathbf{uA} &= \begin{bmatrix} \mathbf{u} \cdot \mathbf{A}_{*,1} & \mathbf{u} \cdot \mathbf{A}_{*,2} & \mathbf{u} \cdot \mathbf{A}_{*,3} \end{bmatrix} \\ &= \begin{bmatrix} xA_{11} + yA_{21} + zA_{31}, & xA_{12} + yA_{22} + zA_{32}, & xA_{13} + yA_{23} + zA_{33} \end{bmatrix} \\ &= \begin{bmatrix} xA_{11}, xA_{12}, xA_{13} \end{bmatrix} + \begin{bmatrix} yA_{21}, yA_{22}, yA_{23} \end{bmatrix} + \begin{bmatrix} zA_{31}, zA_{32}, zA_{33} \end{bmatrix} \\ &= x \begin{bmatrix} A_{11}, A_{12}, A_{13} \end{bmatrix} + y \begin{bmatrix} A_{21}, A_{22}, A_{23} \end{bmatrix} + z \begin{bmatrix} A_{31}, A_{32}, A_{33} \end{bmatrix} \\ &= x\mathbf{A}_{1,*} + y\mathbf{A}_{2,*} + z\mathbf{A}_{3,*} \end{aligned}$$

따라서,

$$\mathbf{uA} = x\mathbf{A}_{1,*} + y\mathbf{A}_{2,*} + z\mathbf{A}_{3,*} \quad (\text{식 2.2})$$

방정식 2.2는 *선형 결합*의 예시이며, 벡터-행렬 곱 $\mathbf{uA}$ 가 행렬 $\mathbf{A}$ 의 행 벡터들에 대한 스칼라 계수 x, y, z (벡터 $\mathbf{u}$ 에 의해 주어짐)를 가진 선형 결합과 동등함을 나타냅니다. 비록 1×3 행 벡터와 3×3 행렬에 대해 이를 보여주었지만, 이 결과는 일반적으로 성립합니다. 즉, $1 \times n$ 행 벡터 $\mathbf{u}$ 와 $n \times m$ 행렬 $\mathbf{A}$ 에 대해, $\mathbf{uA}$ 는 $\mathbf{u}$ 가 주는 스칼라 계수를 가진 $\mathbf{A}$ 의 행 벡터들의 선형 결합이다:

$$\begin{bmatrix} u_1, \dots, u_n \end{bmatrix} \begin{bmatrix} A_{11} & \dots & A_{1m} \\ \vdots & \ddots & \vdots \\ A_{n1} & \dots & A_{nm} \end{bmatrix} = u_1\mathbf{A}_{1,*} + \dots + u_n\mathbf{A}_{n,*} \quad (\text{eq. 2.3})$$

2.2.3 결합법칙

행렬 곱셈은 몇 가지 훌륭한 대수적 성질을 지닙니다. 예를 들어, 행렬 곱셈은 덧셈에 대해 분배법칙을 따릅니다: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$ 및 $(\mathbf{A} + \mathbf{B})\mathbf{C} = \mathbf{AC} + \mathbf{BC}$. 그러나 특히, 우리는 때때로 행렬 곱셈의 결합법칙을 사용할 것입니다. 이 법칙은 행렬을 곱하는 순서를 선택할 수 있게 해줍니다:

$$(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$$

2.3 행렬의 전치

행렬의 *전치*는 행과 열을 서로 바꾸어 얻습니다. 따라서 $m \times n$ 행렬의 전치는 $n \times m$ 행렬입니다. 행렬 $\mathbf{M}$ 의 전치를 $\mathbf{M}^T$ 로 표기합니다.

예제 2.5

다음 세 행렬의 전치 행렬을 구하시오:

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 8 \\ 3 & 6 & -4 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$$

다시 말해, 전치 행렬은 행과 열을 서로 바꾸어 얻습니다. 따라서

$$\mathbf{A}^T = \begin{bmatrix} 2 & 3 \\ -1 & 6 \\ 8 & -4 \end{bmatrix}, \quad \mathbf{B}^T = \begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}, \quad \mathbf{C}^T = [1 \ 2 \ 3 \ 4]$$

전치 행렬은 다음과 같은 유용한 성질을 가집니다:

1. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$
2. $(c\mathbf{A})^T = c\mathbf{A}^T$
3. $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$
4. $(\mathbf{A}^T)^T = \mathbf{A}$
5. $(\mathbf{A}^{-1})^T = (\mathbf{A}^T)^{-1}$

2.4 단위 행렬

*단위 행렬*이라는 특별한 행렬이 있습니다. 단위 행렬은 대각선을 제외한 모든 요소가 0인 정사각 행렬이며, 대각선 상의 요소들은 모두 1입니다.

예를 들어, 아래는 2×2, 3×3, 4×4 단위 행렬입니다.

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

단위 행렬은 곱셈 항등원 역할을 합니다. 즉, $\mathbf{A}$ 가 $m \times n$ 행렬이고, $\mathbf{B}$ 가 $n \times p$ 행렬이며, $\mathbf{I}$ 가 $n \times n$ 단위 행렬이라면

$$\mathbf{AI} = \mathbf{A} \text{ 및 } \mathbf{IB} = \mathbf{B}$$

즉, 행렬을 단위 행렬로 곱해도 행렬은 변하지 않습니다. 단위 행렬은 행렬에 대한 숫자 1로 생각할 수 있습니다. 특히, $\mathbf{MI}$ 정사각 행렬이라면 단위 행렬과의 곱셈은 교환법칙을 따릅니다:

$$\mathbf{MI} = \mathbf{IM} = \mathbf{M}$$

예제 2.6

다음과 같이 정의하자 $\mathbf{M} = \begin{bmatrix} 1 & 2 \\ 0 & 4 \end{bmatrix}$ 이고 $\mathbf{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ 라고 하자. $\mathbf{MI} = \mathbf{IM} = \mathbf{M}$ 임을 검증하라.

방정식 2.1을 적용하면 다음과 같다:

$$\mathbf{MI} = \begin{bmatrix} 1 & 2 \\ 0 & 4 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} (1,2) \cdot (1,0) & (1,2) \cdot (0,1) \\ (0,4) \cdot (1,0) & (0,4) \cdot (0,1) \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & 4 \end{bmatrix}$$

그리고

$$\mathbf{IM} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 4 \end{bmatrix} = \begin{bmatrix} (1,0) \cdot (1,0) & (1,0) \cdot (2,4) \\ (0,1) \cdot (1,0) & (0,1) \cdot (2,4) \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & 4 \end{bmatrix}$$

따라서 $\mathbf{MI} = \mathbf{IM} = \mathbf{M}$ 이 성립함을 알 수 있다.

예제 2.7

$\mathbf{u} = [-1, 2]$ 이고 $\mathbf{I} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ $\mathbf{uI} = \mathbf{u}$ 임을 검증하라. 방정

$$\mathbf{uI} = [-1, 2] \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = [(-1, 2) \cdot (1, 0) \quad (-1, 2) \cdot (0, 1)] = [-1, 2]$$

식 2.1을 적용하면:

행렬 곱셈이 정의되지 않으므로 $\mathbf{Iu}$ 의 곱을 취할 수 없음을 유의하십시오.

2.5 행렬의 행렬식

행렬식은 정사각 행렬을 입력으로 받아 실수를 출력하는 특수한 함수이다. **정사각 행렬 $\mathbf{A}$ 의 행렬식**은 일반적으로 $\det \mathbf{A}$ 로 표기된다. 행렬식은 상자의 부피와 관련된 기하학적 해석을 가지며, 선형 변환 하에서 부피가 어떻게 변화하는지에 대한 정보를 제공한다는 것이 증명될 수 있다. 또한 행렬식은 크래머의 법칙을 사용하여 선형 방정식 시스템을 풀 때 사용된다. 그러나 본 연구 목적상, 행렬의 역행렬을 구하는 명시적 공식을 제공한다는 점에서 행렬식을 주로 연구하고자 한다 (§2.7 주제). 또한 다음이 증명될 수 있다: 정사각 행렬 $\mathbf{A}$ 가 가역 행렬인 것은 $\det \mathbf{A} \neq 0$ 인 경우에 한한다. 이 사실은 행렬의 가역성을 판단하는 계산적 도구를 제공한다는 점에서 유용하다. 행렬식을 정의하기 전에 먼저 행렬의 소행렬 개념을 소개한다.

2.5.1 소행렬

$n \times n$ 행렬 $\mathbf{A}$ 가 주어졌을 때, **부행렬 $\bar{\mathbf{A}}_{ij}$** 은 $\mathbf{A}$ 의 i 번째 행과 j 번째 열을 삭제하여 얻은 $(n-1) \times (n-1)$ 행렬이다.

예제 2.8

다음 행렬의 부행렬 $\bar{\mathbf{A}}_{11}$, $\bar{\mathbf{A}}_{22}$ 와 $\bar{\mathbf{A}}_{13}$ 를 구하라:

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix}$$

$\bar{\mathbf{A}}_{11}$ 의 경우 첫 번째 행과 첫 번째 열을 제거하여 얻는다:

$$\bar{\mathbf{A}}_{11} = \begin{bmatrix} A_{22} & A_{23} \\ A_{32} & A_{33} \end{bmatrix}$$

$\bar{\mathbf{A}}_{22}$ 의 경우 두 번째 행과 두 번째 열을 제거하여 얻습니다:

$$\bar{\mathbf{A}}_{22} = \begin{bmatrix} A_{11} & A_{13} \\ A_{31} & A_{33} \end{bmatrix}$$

$\bar{\mathbf{A}}_{13}$ 의 경우 첫 번째 행과 세 번째 열을 제거하면 다음과 같이 얻어집니다:

$$\bar{\mathbf{A}}_{13} = \begin{bmatrix} A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix}$$

2.5.2 정의

행렬의 행렬식은 재귀적으로 정의됩니다. 예를 들어, 4×4 행렬의 행렬식은 3×3 행렬의 행렬식으로 정의되며, 3×3 행렬의 행렬식은 2×2 행렬의 행렬식으로 정의되고, 2×2 행렬의 행렬식은 1×1 행렬의 행렬식으로 정의됩니다. (1×1 행렬 $\mathbf{A} = [A_{11}]$ 의 행렬식은 당연히 $\det[A_{11}] = A_{11}$ 로 정의된다).

$\mathbf{A}$ 를 $n \times n$ 행렬이라 하자. 그러면 $n > 1$ 일 때 다음과 같이 정의한다:

$$\det \mathbf{A} = \sum_{j=1}^n A_{1j} (-1)^{1+j} \det \bar{\mathbf{A}}_{1j} \quad (\text{식 2.4})$$

2×2 행렬에 대한 부행렬 $\bar{\mathbf{A}}_{ij}$ 의 정의를 상기하면, 이는 다음 공식을 제공한다:

$$\det \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} = A_{11} \det [A_{22}] - A_{12} \det [A_{21}] = A_{11} A_{22} - A_{12} A_{21}$$

3×3 행렬의 경우, 이는 다음 공식을 제공한다:

$$\det \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} = A_{11} \det \begin{bmatrix} A_{22} & A_{23} \\ A_{32} & A_{33} \end{bmatrix} - A_{12} \det \begin{bmatrix} A_{21} & A_{23} \\ A_{31} & A_{33} \end{bmatrix} + A_{13} \det \begin{bmatrix} A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix}$$

4×4 행렬의 경우, 이로부터 다음 공식이 도출됩니다:

$$\begin{aligned} \det \begin{bmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{bmatrix} \\ = A_{11} \det \begin{bmatrix} A_{22} & A_{23} & A_{24} \\ A_{32} & A_{33} & A_{34} \\ A_{42} & A_{43} & A_{44} \end{bmatrix} - A_{12} \det \begin{bmatrix} A_{21} & A_{23} & A_{24} \\ A_{31} & A_{33} & A_{34} \\ A_{41} & A_{43} & A_{44} \end{bmatrix} \\ + A_{13} \det \begin{bmatrix} A_{21} & A_{22} & A_{24} \\ A_{31} & A_{32} & A_{34} \\ A_{41} & A_{42} & A_{44} \end{bmatrix} - A_{14} \det \begin{bmatrix} A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \\ A_{41} & A_{42} & A_{43} \end{bmatrix} \end{aligned}$$

3D 그래픽스에서는 주로 4×4 행렬을 다루므로, $n > 4$ 에 대한 명시적 공식을 계속 생성할 필요가 없습니다.

예제 2.9

다음 행렬의 행렬식을 구하시오.

$$\mathbf{A} = \begin{bmatrix} 2 & -5 & 3 \\ 1 & 3 & 4 \\ -2 & 3 & 7 \end{bmatrix}$$

다음과 같습니다:

$$\begin{aligned} \det \mathbf{A} &= A_{11} \det \begin{bmatrix} A_{22} & A_{23} \\ A_{32} & A_{33} \end{bmatrix} - A_{12} \det \begin{bmatrix} A_{21} & A_{23} \\ A_{31} & A_{33} \end{bmatrix} + A_{13} \det \begin{bmatrix} A_{21} & A_{22} \\ A_{31} & A_{32} \end{bmatrix} \\ \det \mathbf{A} &= 2 \det \begin{bmatrix} 3 & 4 \\ 3 & 7 \end{bmatrix} - (-5) \det \begin{bmatrix} 1 & 4 \\ -2 & 7 \end{bmatrix} + 3 \det \begin{bmatrix} 1 & 3 \\ -2 & 3 \end{bmatrix} \\ &= 2(3 \cdot 7 - 4 \cdot 3) + 5(1 \cdot 7 - 4 \cdot (-2)) + 3(1 \cdot 3 - 3 \cdot (-2)) \\ &= 2(9) + 5(15) + 3(9) \\ &= 18 + 75 + 27 \\ &= 120 \end{aligned}$$

2.6 행렬의 인접행렬

$\mathbf{A}$ 를 $n \times n$ 행렬이라 하자. 곱 $C_{ij} = (-1)^{i+j} \det \bar{\mathbf{A}}_{ij}$ 는 A_{ij} 의 공인자(cofactor)라 부른다. C_{ij} 를 계산하여 $\mathbf{A}$ 의 모든 원소에 대해 대응하는 행렬 $\mathbf{C}_A$ 의 ij 번째 위치에 배치하면, $\mathbf{A}$ 의 공인자 행렬을 얻는다:

$$\mathbf{C}_A = \begin{bmatrix} C_{11} & C_{12} & \cdots & C_{1n} \\ C_{21} & C_{22} & \cdots & C_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ C_{n1} & C_{n2} & \cdots & C_{nn} \end{bmatrix}$$

$\mathbf{C}_A$ 의 전치를 취하면 $\mathbf{A}$ 의 동반행렬이라 불리는 행렬을 얻으며, 이를 다음과 같이 표기한다:

$$\mathbf{A}^* = \mathbf{C}_A^T \quad (\text{eq. 2.5})$$

다음 절에서는, 동반 행렬을 통해 행렬의 역행렬을 계산하는 명시적 공식을 구할 수 있음을 배웁니다.

2.7 행렬의 역행렬

행렬 대수에서는 나눗셈 연산을 정의하지 않지만, 곱셈 역원 연산은 정의합니다. 다음 목록은 역원에 관한 중요한 정보를 요약합니다:

1. 오직 정사각 행렬만이 역행렬을 가집니다. 따라서 행렬의 역행렬에 대해 이야기할 때는 정사각 행렬을 다루고 있다고 가정합니다.
2. $n \times n$ 행렬 $\mathbf{M}$ 의 역행렬은 $\mathbf{M}^{-1}$ 로 표기되는 $n \times n$ 행렬이다.
3. 모든 정사각 행렬이 역행렬을 가지는 것은 아니다. 역행렬을 가지는 행렬은 가역행렬이라고 하며, 역행렬을 가지지 않는 행렬은 특이행렬이라고 한다.
4. 역행렬이 존재할 때 그 역행렬은 유일하다.
5. 행렬을 그 역행렬로 곱하면 단위 행렬이 된다: $\mathbf{M}\mathbf{M}^{-1} = \mathbf{M}^{-1}\mathbf{M} = \mathbf{I}$. 행렬을 그 자체의 역행렬로 곱하는 경우

행렬 곱셈이 교환법칙을 따르는 경우임을 유의하십시오.

행렬 방정식에서 다른 행렬을 구할 때 행렬의 역행렬이 유용합니다. 예를 들어, 행렬 방정식 $\mathbf{p}' = \mathbf{p}\mathbf{M}$ 이 주어졌다고 가정합니다. 또한 $\mathbf{p}'$ 와 $\mathbf{M}$ 이 주어졌고 $\mathbf{p}$ 를 구하고자 한다고 가정합니다. $\mathbf{M}$ 이 가역행렬(즉, $\mathbf{M}^{-1}$ 이 존재함)이라고 가정하면, 다음과 같이 $\mathbf{p}$ 를 구할 수 있습니다:

$\mathbf{p}' = \mathbf{p}\mathbf{M}$	
$\mathbf{p}'\mathbf{M}^{-1} = \mathbf{p}\mathbf{M}\mathbf{M}^{-1}$	방정식의 양변에 $\mathbf{M}^{-1}$ 을 곱하면
$\mathbf{p}'\mathbf{M}^{-1} = \mathbf{p}\mathbf{I}$	$\mathbf{M}\mathbf{M}^{-1} = \mathbf{I}$, 역함수의 정의에 의해
$\mathbf{p}'\mathbf{M}^{-1} = \mathbf{p}$	$\mathbf{p}\mathbf{I} = \mathbf{p}$, 단위 행렬의 정의에 의해

여기서는 증명하지 않지만 대학 수준의 선형대수학 교재에서 증명해야 할 역행렬 구하기 공식은, 준행렬과 행렬식을 이용해 다음과 같이 표현할 수 있다:

$$\mathbf{A}^{-1} = \frac{\mathbf{A}^*}{\det \mathbf{A}} \tag{식 2.6}$$

예제 2.10

2×2 행렬의 역행렬에 대한 일반 공식을 구하라

$$\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

을 구하고, 이 공식을 사용하여 행렬

$$\mathbf{M} = \begin{bmatrix} 3 & 0 \\ -1 & 2 \end{bmatrix}$$

의 역행렬을 구하라.

다음과 같이 정의된다

$$\det \mathbf{A} = A_{11}A_{22} - A_{12}A_{21}$$

$$\mathbf{C}_\mathbf{A} = \begin{bmatrix} (-1)^{1+1} \det \bar{\mathbf{A}}_{11} & (-1)^{1+2} \det \bar{\mathbf{A}}_{12} \\ (-1)^{2+1} \det \bar{\mathbf{A}}_{21} & (-1)^{2+2} \det \bar{\mathbf{A}}_{22} \end{bmatrix} = \begin{bmatrix} A_{22} & -A_{21} \\ -A_{12} & A_{11} \end{bmatrix}$$

따라서,

$$\mathbf{A}^{-1} = \frac{\mathbf{A}^*}{\det \mathbf{A}} = \frac{\mathbf{C}_\mathbf{A}^T}{\det \mathbf{A}} = \frac{1}{A_{11}A_{22} - A_{12}A_{21}} \begin{bmatrix} A_{22} & -A_{12} \\ -A_{21} & A_{11} \end{bmatrix}$$

이제 이 공식을 적용하여

$$\mathbf{M} = \begin{bmatrix} 3 & 0 \\ -1 & 2 \end{bmatrix}$$

$$\mathbf{M}^{-1} = \frac{1}{3 \cdot 2 - 0 \cdot (-1)} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 1/3 & 0 \\ 1/6 & 1/2 \end{bmatrix}$$

우리의 작업을 확인하기 위해 $\mathbf{M}\mathbf{M}^{-1} = \mathbf{M}^{-1}\mathbf{M} = \mathbf{I}$ 를 검증합니다:

$$\begin{bmatrix} 3 & 0 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1/3 & 0 \\ 1/6 & 1/2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1/3 & 0 \\ 1/6 & 1/2 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ -1 & 2 \end{bmatrix}$$

작은 행렬(4×4 이하 크기)의 경우, 인접 행렬법은 계산 효율적입니다. 더 큰 행렬의 경우 가우스 소거법과 같은 다른 방법이 사용됩니다. 그러나 3D에서 우리가 다루는 행렬들은

 컴퓨터 그래픽스에는 특별한 형태가 존재하여, 일반 행렬의 역행렬을 찾는 데 CPU 사이클을 낭비하지 않도록 사전에 역공식을 결정할 수 있습니다. 따라서 코드에서 방정식 2.6을 적용할 필요가 거의 없습니다.

역행렬에 관한 이 절을 마무리하며, 곱의 역행렬에 대한 다음과 같은 유용한 대수적 성질을 제시합니다:

$$(AB)^{-1} = B^{-1} A^{-1}$$

이 성질은 A와 B가 모두 가역행렬이며 동일한 차원의 정사각 행렬임을 가정한다. $B^{-1}A^{-1}$ 이 AB의 역행렬임을 증명하려면, $(AB)^{(-1)}(B^{-1}A^{-1}) = I$ 및 $(B^{-1}A^{-1})(AB) = I$ 를 보여야 한다.

이는 다음과 같이 수행된다:

$$(AB)(B^{-1}A^{-1}) = A(BB^{-1})A^{-1} = AIA^{-1} = AA^{-1} = I \quad (B^{-1}A^{-1})(AB) = B^{-1}(A^{-1}A)B = B^{-1}IB = B^{-1}B = I$$

2.8 XNA 행렬

점과 벡터를 변환하기 위해 우리는 1×4 행 벡터와 4×4 행렬을 사용합니다. 그 이유는 다음 장에서 설명할 것입니다. 지금은 4×4 행렬을 표현하는 데 사용되는 XNA 수학 유형에 집중하겠습니다.

2.8.1 행렬 유형

XNA 수학에서 4×4 행렬을 표현하기 위해 `XMMATRIX` 클래스를 사용합니다. 이 클래스는 `xnmath.h` 헤더 파일에서 다음과 같이 정의됩니다(추가 설명 포함):

```
// 행렬 유형: 16개의 32비트 부동 소수점 구성 요소로,
// 16바이트 경계에 정렬되고 네 개의 하드웨어 벡터 레지스터에 매핑됨
#if (defined(_XM_X86_) || defined(_XM_X64_)) && defined(_XM_NO_INTRINSICS_) typedef struct _XMMATRIX
#else
typedef _DECLSPEC_ALIGN_16_ struct _XMMATRIX #endif
{
    // 연합체
    {
        // SIMD용 행렬 표현을 위해 4개의 XMVECTOR 사용. XMVECTOR r[4];

        struct
        {
            FLOAT _11, _12, _13, _14; FLOAT _21,
            _22, _23, _24; FLOAT _31, _32, _33, _34;
            FLOAT _41, _42, _43, _44;

        };
        FLOAT m[4][4];
    };

#ifdef __cplusplus
    _XMMATRIX() {}

    // 4개의 행 벡터를 지정하여 행렬 초기화.
    _XMMATRIX(FXMVECTOR R0, FXMVECTOR R1, FXMVECTOR R2, CXMVECTOR R3);

    // 16개 요소를 지정하여 행렬 초기화.
    _XMMATRIX(FLOAT m00, FLOAT m01, FLOAT m02, FLOAT m03, FLOAT m10, FLOAT m11, FLOAT m12,
        FLOAT m13,
        FLOAT m20, FLOAT m21, FLOAT m22, FLOAT m23, FLOAT m30, FLOAT m31,
        FLOAT m32, FLOAT m33);
```

```
// 16개의 부동소수점 배열을 전달하여 행렬을 생성합니다.
_XMMATRIX(CONST FLOAT *pArray);
    FLOAT operator() (UINT Row, UINT Column) CONST { return m[Row][Column]; } FLOAT& operator() (UINT Row, UINT
    Column) { return m[Row][Column]; }

    _XMMATRIX& operator= (CONST _XMMATRIX& M); #ifndef

XM_NO_OPERATOR_OVERLOADS
    _XMMATRIX& operator*= (CONST _XMMATRIX& M);
    _XMMATRIX operator* (CONST _XMMATRIX& M) CONST;
#endif // !XM_NO_OPERATOR_OVERLOADS #endif // __cplusplus

} XMMATRIX;
```

보시다시피, **XMMATRIX**는 SIMD를 사용하기 위해 네 개의 **XMVECTOR** 인스턴스를 사용합니다. 또한 **XMMATRIX**는 행렬 곱셈을 위한 중복 정의 연산자와 **XMMATRIX**의 행과 열(0부터 시작하는 인덱스 사용)을 지정하여 요소에 접근하거나 수정하기 위한 중복 정의 괄호 연산자를 제공합니다.

다양한 생성자 사용 외에도 **XMMatrixSet** 함수를 통해 **XMMATRIX** 인스턴스를 생성할 수 있습니다:

```
XMMATRIX XMMatrixSet(FLOAT m00, FLOAT m01, FLOAT m02, FLOAT m03, FLOAT m10, FLOAT m11, FLOAT
m12, FLOAT m13,
FLOAT m20, FLOAT m21, FLOAT m22, FLOAT m23, FLOAT m30, FLOAT m31,
FLOAT m32, FLOAT m33);
```

클래스에 벡터를 저장할 때 **XMFLOAT2**(2차원), **XMFLOAT3**(3차원), **XMFLOAT4**(4차원)을 사용하는 것처럼, XNA 수학 문서에서는 행렬을 클래스 데이터 멤버로 저장할 때 **XMFLOAT4X4** 유형을 사용할 것을 권장합니다.

```
// 4x4 행렬: 32비트 부동 소수점 구성 요소 typedef struct _XMFLOAT4X4
{
    연합체
    {
        struct
        {
            FLOAT _11, _12, _13, _14; FLOAT _21,
            _22, _23, _24; FLOAT _31, _32, _33, _34;
            FLOAT _41, _42, _43, _44;

        };
        FLOAT m[4][4];
    };

#ifdef __cplusplus

    _XMFLOAT4X4();
    _XMFLOAT4X4(FLOAT m00, FLOAT m01, FLOAT m02, FLOAT m03, FLOAT m10, FLOAT m11, FLOAT m12,
    FLOAT m13,
    FLOAT m20, FLOAT m21, FLOAT m22, FLOAT m23, FLOAT m30, FLOAT m31,
    FLOAT m32, FLOAT m33);
    _XMFLOAT4X4(CONST FLOAT *pArray);

    FLOAT operator() (UINT Row, UINT Column) CONST { return m[Row][Column]; } FLOAT& operator() (UINT Row, UINT
    Column) { return m[Row][Column]; }

    _XMFLOAT4X4& operator= (CONST _XMFLOAT4X4& Float4x4); #endif // __cplusplus

} XMFLOAT4X4;
```

2.8.2 행렬 함수

XNA Math 라이브러리에는 다음과 같은 유용한 행렬 관련 함수가 포함되어 있습니다:

<code>XMMATRIX XMMatrixIdentity();</code>	// 단위 행렬 I 를 반환합니다.
<code>BOOL XMMatrixIsIdentity(CXMMATRIX M);</code>	// M 이 단위 행렬이면 true를 반환합니다 // 입력 M
<code>XMMATRIX XMMatrixMultiply(CXMMATRIX A, CXMMATRIX B);</code>	// 행렬 곱 AB 를 반환합니다 // 입력 A // 입력 B
<code>XMMATRIX XMMatrixTranspose(CXMMATRIX M);</code>	// ^{M_T} 반환 // 입력 M
<code>XMVECTOR XMMatrixDeterminant(CXMMATRIX M);</code>	// (det M , det M , det M , det M) 반환 // 입력 M
<code>XMMATRIX XMMatrixInverse(XMVECTOR* pDeterminant, CXMMATRIX M);</code>	// ^{M⁻¹} 을 반환 // 입력 (det M , det M , det M , det M) // 입력 M

 **Note:** XMMATRIX 매개변수는 CXMMATRIX 유형으로 지정해야 합니다. 이는 Xbox 360 및 Windows 플랫폼에서 XMMATRIX 매개변수가 올바르게 전달되도록 보장합니다.

```
// Xbox 360에서는 레지스터로 전달되도록 XMMATRIX 매개변수를 수정하고,  
// 그 외에는 참조로 전달  
#if defined(_XM_VMX128_INTRINSICS_)    typedef    const  
XMMATRIX CXMMATRIX; #elif defined( cplusplus)  
typedef const XMMATRIX& CXMMATRIX;
```

2.8.3 XNA Matrix 샘플 프로그램

다음 코드는 XMMATRIX 클래스와 이전 섹션에 나열된 대부분의 함수를 사용하는 방법에 대한 몇 가지 예시를 제공합니다.

```
#include <windows.h> // FLOAT 정의용 #include <xnamath.h>
#include <iostream> using namespace std;

// "<<" 연산자를 오버로드하여 cout를 사용하여
// XMVECTOR 및 XMMATRIX 객체를 출력할 수 있도록 합니다. ostream&
operator<<(ostream& os, FXMVECTOR v)
{
    XMFLOAT4 dest; XMStore
    Float4(&dest, v);

    os << "(" << dest.x << ", " << dest.y << ", "
        << dest.z << ", " << dest.w << ")"; return os;
}

ostream& operator<<(ostream& os, CXMMATRIX m)
{
```

```
for(int i = 0; i < 4; ++i)
{
    for(int j = 0; j < 4; ++j) os << m(i,
j) << "\t";
os << endl;
}
return os;
}
```

```
int main()
{
    // SSE2 지원 확인 (Pentium4, AMD K8 이상). if( !XMVerifyCPUSupport() )
    {
        cout << "xna math 지원되지 않음" << endl; return 0;
    }

    XMMATRIX A(1.0f, 0.0f, 0.0f, 0.0f,
                0.0f, 2.0f, 0.0f, 0.0f,
                0.0f, 0.0f, 4.0f, 0.0f,
                1.0f, 2.0f, 3.0f, 1.0f);

    XMMATRIX B = XMMatrixIdentity(); XMMATRIX C = A
    * B;

    XMMATRIX D = XMMatrixTranspose(A);

    XMVECTOR det = XMMatrixDeterminant(A); XMMATRIX E =
    XMMatrixInverse(&det, A);

    XMMATRIX F = A * E;

    cout << "A = " << endl << A << endl;
    cout << "B = " << endl << B << endl;
    cout << "C = A*B = " << endl << C << endl;
    cout << "D = transpose(A) = " << endl << D << endl; cout << "det
= determinant(A) = " << det << endl << endl; cout << "E
= inverse(A) = " << endl << E << endl;
    cout << "F = A*E = " << endl << F << endl;

    return 0;
}
```



```
A =  
1      0      0      0  
0      2      0      0  
0      0      4      0  
1      2      3      1  
  
B =  
1      0      0      0  
0      1      0      0  
0      0      1      0  
0      0      0      1  
  
C = A*B =  
1      0      0      0  
0      2      0      0  
0      0      4      0  
1      2      3      1  
  
D = transpose(A) =  
1      0      0      1  
0      2      0      2  
0      0      4      3  
0      0      0      1  
  
det = determinant(A) = (8, 8, 8, 8)  
  
E = inverse(A) =  
1      0      0      0  
0      0.5    0      0  
0      0      0.25  0  
-1     -1     -0.75  1  
  
F = A*E =  
1      0      0      0  
0      1      0      0  
0      0      1      0  
0      0      0      1  
  
Press any key to continue . . .
```

2.9 요약

1. $m \times n$ 행렬 $\mathbf{M}$ 은 m 개의 행과 n 개의 열을 가진 실수의 직사각형 배열이다. 동일한 차원의 두 행렬은 대응하는 성분이 같을 때만 같다. 동일한 차원의 두 행렬을 더할 때는 대응하는 원소를 더한다. 스칼라와 행렬을 곱할 때는 스칼라를 행렬의 모든 원소와 곱한다.
2. $\mathbf{A}$ 가 $m \times n$ 행렬이고 $\mathbf{B}$ 가 $n \times p$ 행렬일 때, 곱 $\mathbf{AB}$ 는 정의되며 $m \times p$ 행렬 $\mathbf{C}$ 를 이룬다. 여기서 곱 $\mathbf{C}$ 의 ij 번째 원소는 $\mathbf{A}$ 의 i 번째 행 벡터와 $\mathbf{B}$ 의 j 번째 열 벡터의 내적을 취함으로써 주어지며, 즉 $C_{ij} = \mathbf{A}_{i,*} \cdot \mathbf{B}_{*,j}$ 이다.
3. 행렬 곱셈은 교환법칙을 따르지 않습니다(즉, 일반적으로 $\mathbf{AB} \neq \mathbf{BA}$). 행렬 곱셈은 결합법칙을 따릅니다: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$.
4. 행렬의 전치는 행과 열을 서로 바꾸어 엮습니다. 따라서 $m \times n$ 행렬의 전치는 $n \times m$ 행렬입니다. 행렬 $\mathbf{M}$ 의 전치는 $\mathbf{M}^T$ 로 표기합니다.
5. 단위 행렬은 대각선 요소를 제외한 모든 요소가 0이고, 대각선 요소가 모두 1인 정사각 행렬이다.
6. 행렬식 $\det \mathbf{A}$ 는 정사각 행렬을 입력으로 받아 실수를 출력하는 특수한 함수이다. 정사각 행렬 $\mathbf{A}$ 가 가역 행렬인 것은 $\det \mathbf{A} \neq 0$ 일 때에 한한다. 행렬식은 행렬의 역행렬을 계산하는 공식에 사용된다.
7. 행렬과 그 역행렬을 곱하면 단위 행렬이 된다: $\mathbf{MM}^{-1} = \mathbf{M}^{-1}\mathbf{M} = \mathbf{I}$. 행렬의 역행렬은 존재할 경우 유일하다. 오직 정사각 행렬만이 역행렬을 가지며, 정사각 행렬이라 하더라도 가역적이지 않을 수 있다. 행렬의 역행렬은 다음 공식으로 계산할 수 있다: $\mathbf{A}^{-1} = \mathbf{A}^* / \det \mathbf{A}$, 여기서 $\mathbf{A}^*$ 는 $\mathbf{A}$ 의 부행렬의 전치 행렬인 부행렬의 전치 행렬이다.

2.10 연습문제

1. 다음 행렬 방정식에서 $\mathbf{X}$ 를 구하시오:

$$3 \left(\begin{bmatrix} -2 & 0 \\ 1 & 3 \end{bmatrix} - 2\mathbf{X} \right) = 2 \begin{bmatrix} -2 & 0 \\ 1 & 3 \end{bmatrix}$$

2. 다음 행렬 곱셈을 계산하라:

$$\text{a) } \begin{bmatrix} -2 & 0 & 3 \\ 4 & 1 & -1 \end{bmatrix} \begin{bmatrix} 2 & -1 \\ 0 & 6 \\ 2 & -3 \end{bmatrix}, \text{ b) } \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -2 & 0 \\ 1 & 1 \end{bmatrix}, \text{ c) } \begin{bmatrix} 2 & 0 & 2 \\ 0 & -1 & -3 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}$$

3. 다음 행렬들의 전치 행렬을 계산하세요:

$$\text{a) } \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}, \text{ b) } \begin{bmatrix} x & y \\ z & w \end{bmatrix}, \text{ c) } \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{bmatrix}$$

4. 다음 선형 결합을 벡터-행렬 곱으로 표현하세요:
 - a) $\mathbf{v} = 2(1, 2, 3) - 4(-5, 0, -1) + 3(2, -2, 3)$
 - b) $\mathbf{v} = 3(2, -4) + 2(1, 4) - 1(-2, -3) + 5(1, 1)$
5. 다음과 같이 증명하라

$$\mathbf{AB} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} & B_{13} \\ B_{21} & B_{22} & B_{23} \\ B_{31} & B_{32} & B_{33} \end{bmatrix} = \begin{bmatrix} \leftarrow \mathbf{A}_{1,*} \mathbf{B} \rightarrow \\ \leftarrow \mathbf{A}_{2,*} \mathbf{B} \rightarrow \\ \leftarrow \mathbf{A}_{3,*} \mathbf{B} \rightarrow \end{bmatrix}$$

6. 다음과 같이 증명하라.

$$\mathbf{Au} = \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = x\mathbf{A}_{*,1} + y\mathbf{A}_{*,2} + z\mathbf{A}_{*,3}$$

7. 외적(외적)이 행렬 곱으로 표현될 수 있음을 증명하라.

$$\mathbf{u} \times \mathbf{v} = \begin{bmatrix} v_x & v_y & v_z \end{bmatrix} \begin{bmatrix} 0 & u_z & -u_y \\ -u_z & 0 & u_x \\ u_y & -u_x & 0 \end{bmatrix}$$

8. 다음과 같이 하자 $\mathbf{B} = \begin{bmatrix} 1/2 & 0 & -1/2 \\ 0 & -1 & -3 \\ 0 & 0 & 1 \end{bmatrix}$ $\mathbf{A}$ 의 역행렬인가?

9. 다음과 같이 정의하자 $\mathbf{B} = \begin{bmatrix} -2 & 1 \\ 3/2 & 1/2 \end{bmatrix}$ $\mathbf{A}$ 의 역행렬인가?

10. 다음 행렬들의 행렬식을 구하시오:

$$\begin{bmatrix} 21 & -4 \\ 10 & 7 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 7 \end{bmatrix}$$

11. 다음 행렬들의 역행렬을 구하시오:

$$\begin{bmatrix} 21 & -4 \\ 10 & 7 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 7 \end{bmatrix}$$

12. 다음 행렬은 가역적인가?

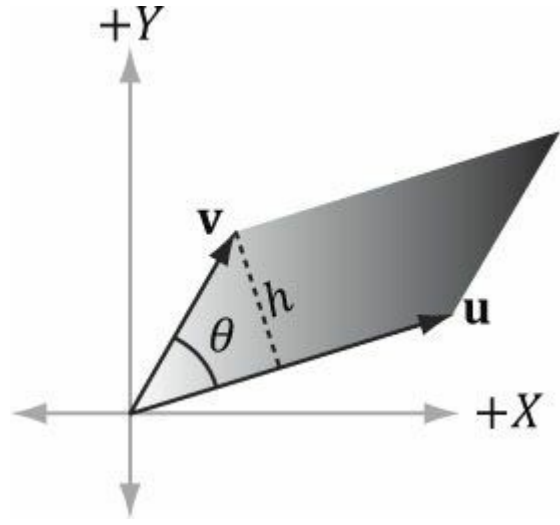
$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 0 \end{bmatrix}$$

13. $\mathbf{A}$ 가 가역행렬이라고 가정할 때, $(\mathbf{A}^{-1})^T = (\mathbf{A}^T)^{-1}$ 임을 증명하시오.

14. $\mathbf{A}$ 와 $\mathbf{B}$ 를 $n \times n$ 행렬이라 하자. 선형대수학 교과서에서 증명된 사실은 $\det(\mathbf{AB}) = \det \mathbf{A} \cdot \det \mathbf{B}$ 이다. 이 사실과 $\det \mathbf{I} = 1$ 이라는 사실을 함께 사용하여 $\mathbf{A}$ 가 가역행렬이라고 가정할 때, 다음을 증명하라.

$$\det \mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}}$$

15. $\begin{bmatrix} u_x & u_y \\ v_x & v_y \end{bmatrix}$ 라는 2차원 행렬식이 $\mathbf{u} = (u_x, u_y)$ 와 $\mathbf{v} = (v_x, v_y)$ 가 이루는 평행사변형의 부호 있는 면적을 준다는 것을 증명하라. $\mathbf{u}$ 를 반시계 방향으로 $\theta \in (0, \pi)$ 만큼 회전시켜 $\mathbf{v}$ 와 일치시킬 수 있으면 결과는 양수이고, 그렇지 않으면 음수이다.
16. 다음으로 이루어진 평행사변형의 면적을 구하시오:
- a) $\mathbf{u} = (3, 0)$ 및 $\mathbf{v} = (1, 1)$
- b) $\mathbf{u} = (-1, -1)$ 및 $\mathbf{v} = (0, 1)$



17. 다음과 같아라. $\mathbf{A} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$, $\mathbf{B} = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$, 그리고 $\mathbf{C} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$. $\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}$ 임을 증명하라. 이는 행렬 곱셈이 결합법칙을 만족함을 보여준다. (사실, 곱셈이 정의된 모든 경우에 행렬 곱셈은 일반 크기의 행렬에 대해 결합법칙을 만족한다.)

18. XNA 수학 라이브러리를 사용하지 않고 (C++에서 배열의 배열만 사용) $m \times n$ 행렬의 전치를 계산하는 컴퓨터 프로그램을 작성하라.

XNA 수학 함수를 사용하지 않고 행렬 연산 수행하기 (C++에서 배열의 배열 사용). 19. 4×4 행렬의 행렬식 및 역행렬을 계산하는 컴퓨터 프로그램 작성하기

제3장 변환

우리는 3차원 세계의 객체를 기하학적으로, 즉 객체의 외부 표면을 근사하는 삼각형들의 집합으로 표현합니다. 객체가 움직이지 않는다면 세상은 흥미롭지 않을 것입니다. 따라서 우리는 기하학적 변환 방법에 관심을 갖게 되며, 기하학적 변환의 예로는 평행 이동, 회전, 크기 변환이 있습니다. 이 장에서는 3차원 공간에서 점과 벡터를 변환하는 데 사용할 수 있는 행렬 방정식을 개발합니다.

목표:

- 1. 선형 변환과 아핀 변환이 행렬로 어떻게 표현되는지 이해한다.
- 2. 확대/축소, 회전, 평행 이동을 위한 좌표 변환을 학습한다.
- 3. 여러 변환 행렬을 행렬 곱셈을 통해 하나의 순수 변환 행렬로 결합하는 방법을 발견한다.
- 4. 한 좌표계에서 다른 좌표계로 좌표를 변환하는 방법과, 이러한 좌표 변환을 행렬로 표현하는 방법을 알아봅니다.
- 5. 변환 행렬을 구성하는 데 사용되는 XNA Math 라이브러리가 제공하는 함수 집합에 익숙해지기 위함입니다.

3.1 선형 변환

3.1.1 정의

수학적 함수 $\tau(\mathbf{v}) = \tau(x, y, z) = (x', y', z')$ 를 고려한다. 이 함수는 3차원 벡터를 입력으로 받아 3차원 벡터를 출력한다. 다음 성질이 성립할 때만 τ 를 *선형 변환*이라고 한다:

1. $\tau(\mathbf{u} + \mathbf{v}) = \tau(\mathbf{u}) + \tau(\mathbf{v})$

2. $\tau(k\mathbf{u}) = k\tau(\mathbf{u})$

(식 3.1)

여기서 $\mathbf{u} = (u_x, u_y, u_z)$ 및 $\mathbf{v} = (v_x, v_y, v_z)$ 는 임의의 3차원 벡터이며, k 는 스칼라입니다.

 선형 변환은 3차원 벡터 이외의 입력 및 출력 값으로 구성될 수 있지만, 3차원 그래픽스 책에서는 이러한 일반성이 필요하지 않습니다.

예제 3.1

함수 $\tau(x, y, z) = (x^2, y^2, z^2)$ 를 정의하자. 예를 들어, $\tau(1, 2, 3) = (1, 4, 9)$ 이다. 이 함수는 선형이 아니다. 왜냐하면 $k = 2$ 이고 $\mathbf{u} = (1, 2, 3)$ 일 때 다음이 성립하지 않기 때문이다:

$\tau(k\mathbf{u}) = \tau(2, 4, 6) = (4, 16, 36)$

그러나

$k \tau(\mathbf{u}) = 2 (1, 4, 9) = (2, 8, 18)$

따라서 방정식 3.1의 성질 2가 만족되지 않는다.

τ 가 선형 함수라면 다음이 성립한다:

$$\begin{aligned}
 \tau(a\mathbf{u} + b\mathbf{v} + c\mathbf{w}) &= \tau(a\mathbf{u} + (b\mathbf{v} + c\mathbf{w})) \\
 &= a\tau(\mathbf{u}) + \tau(b\mathbf{v} + c\mathbf{w}) \\
 &= a\tau(\mathbf{u}) + b\tau(\mathbf{v}) + c\tau(\mathbf{w})
 \end{aligned}
 \tag{식 3.2}$$

이 결과는 다음 절에서 사용할 것이다.

3.1.2 행렬 표현

$\mathbf{u} = (x, y, z)$ 라 하자. 이를 항상 다음과 같이 쓸 수 있음을 관찰하자:

$$\mathbf{u} = (x, y, z) = x\mathbf{i} + y\mathbf{j} + z\mathbf{k} = x(1, 0, 0) + y(0, 1, 0) + z(0, 0, 1)$$

$\mathbb{R}^3$ 작업 좌표축을 따라 향하는 단위 벡터인 $\mathbf{i} = (1, 0, 0)$, $\mathbf{j} = (0, 1, 0)$, $\mathbf{k} = (0, 0, 1)$ 은 3차원 공간 ($\mathbb{R}^3$ 는 모든 3차원 좌표 벡터 (x, y, z) 의 집합을 나타냄)의 *표준 기저 벡터*라고 한다. 이제 τ 를 선형 변환이라 하자. 선형성(즉, 방정식 3.2)에 의해 다음이 성립한다:

$$\tau(\mathbf{u}) = \tau(x\mathbf{i} + y\mathbf{j} + z\mathbf{k}) = x\tau(\mathbf{i}) + y\tau(\mathbf{j}) + z\tau(\mathbf{k}) \tag{식 3.3}$$

이것은 선형 결합에 불과하며, 이전 장에서 배운 바와 같이 벡터-행렬 곱셈으로 표현될 수 있음을 주목하라. 방정식 2.2에 의해 방정식 3.3을 다음과 같이 다시 쓸 수 있다:

$$\begin{aligned}
 \tau(\mathbf{u}) &= x\tau(\mathbf{i}) + y\tau(\mathbf{j}) + z\tau(\mathbf{k}) \\
 &= \mathbf{uA} = \begin{bmatrix} x, & y, & z \end{bmatrix} \begin{bmatrix} \leftarrow \tau(\mathbf{i}) \rightarrow \\ \leftarrow \tau(\mathbf{j}) \rightarrow \\ \leftarrow \tau(\mathbf{k}) \rightarrow \end{bmatrix} = \begin{bmatrix} x, & y, & z \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix}
 \end{aligned}
 \tag{식 3.4}$$

여기서 $\tau(\mathbf{i}) = (A_{11}, A_{12}, A_{13})$, $\tau(\mathbf{j}) = (A_{21}, A_{22}, A_{23})$, 그리고 $\tau(\mathbf{k}) = (A_{31}, A_{32}, A_{33})$ 이다. 이 행렬 $\mathbf{A}$ 를 선형 변환 τ 의 *행렬 표현*이라고 부릅니다.

3.1.3 스케일링

스케일링은 [그림 3.1](#)과 같이 객체의 크기를 변경하는 것을 의미합니다.

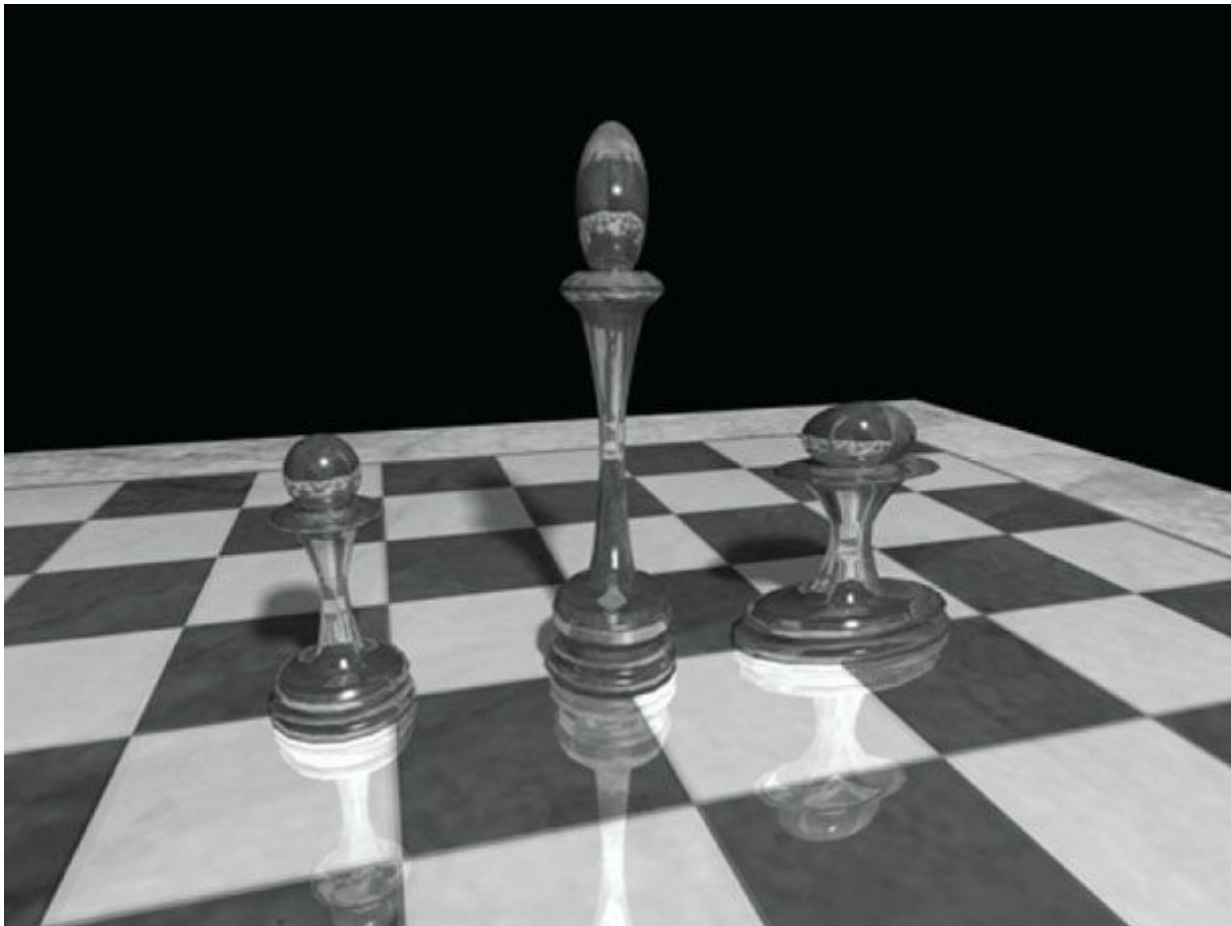


그림 3.1. 왼쪽 폰은 원본 객체입니다. 가운데 폰은 원본 폰을 y축으로 2단위 확대하여 키를 키운 것입니다. 오른쪽 폰은 원본 폰을 x축으로 2단위 확대하여 폭을 넓힌 것입니다.

스케일링 변환은 다음과 같이 정의합니다.

$$S(x, y, z) = (s_x x, s_y y, s_z z)$$

이는 작업 좌표계의 원점을 기준으로 벡터를 x축에서 s_x 단위, y축에서 s_y 단위, z축에서 s_z 단위로 스케일링합니다. 이제 S 가 실제로 선형 변환임을 보여줍니다. 다음이 성립합니다:

$$\begin{aligned} S(\mathbf{u} + \mathbf{v}) &= \left(s_x (u_x + v_x), s_y (u_y + v_y), s_z (u_z + v_z) \right) \\ &= \left(s_x u_x + s_x v_x, s_y u_y + s_y v_y, s_z u_z + s_z v_z \right) \\ &= \left(s_x u_x, s_y u_y, s_z u_z \right) + \left(s_x v_x, s_y v_y, s_z v_z \right) \\ &= S(\mathbf{u}) + S(\mathbf{v}) \\ S(k\mathbf{u}) &= \left(s_x k u_x, s_y k u_y, s_z k u_z \right) \\ &= k \left(s_x u_x, s_y u_y, s_z u_z \right) \\ &= k S(\mathbf{u}) \end{aligned}$$

따라서 방정식 3.1의 두 성질이 모두 만족되므로 S 는 선형이며, 행렬 표현이 존재합니다. 행렬 표현을 구하려면 방정식 3.3과 같이 표준 기저 벡터 각각에 S 를 적용한 후, 결과 벡터들을 행렬의 행에 배치하면 됩니다(방정식 3.4 참조):

$$S(\mathbf{i}) = (s_x \cdot 1, s_y \cdot 0, s_z \cdot 0) = (s_x, 0, 0)$$

$$S(\mathbf{j}) = (s_x \cdot 0, s_y \cdot 1, s_z \cdot 0) = (0, s_y, 0)$$

$$S(\mathbf{k}) = (s_x \cdot 0, s_y \cdot 0, s_z \cdot 1) = (0, 0, s_z)$$

따라서 S 의 행렬 표현은 다음과 같다:

$$\mathbf{S} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix}$$

이 행렬을 **스케일링 행렬**이라고 부릅니다.

스케일링 행렬의 역행렬은 다음과 같이 주어집니다:

$$\mathbf{S}^{-1} = \begin{bmatrix} 1/s_x & 0 & 0 \\ 0 & 1/s_y & 0 \\ 0 & 0 & 1/s_z \end{bmatrix}$$

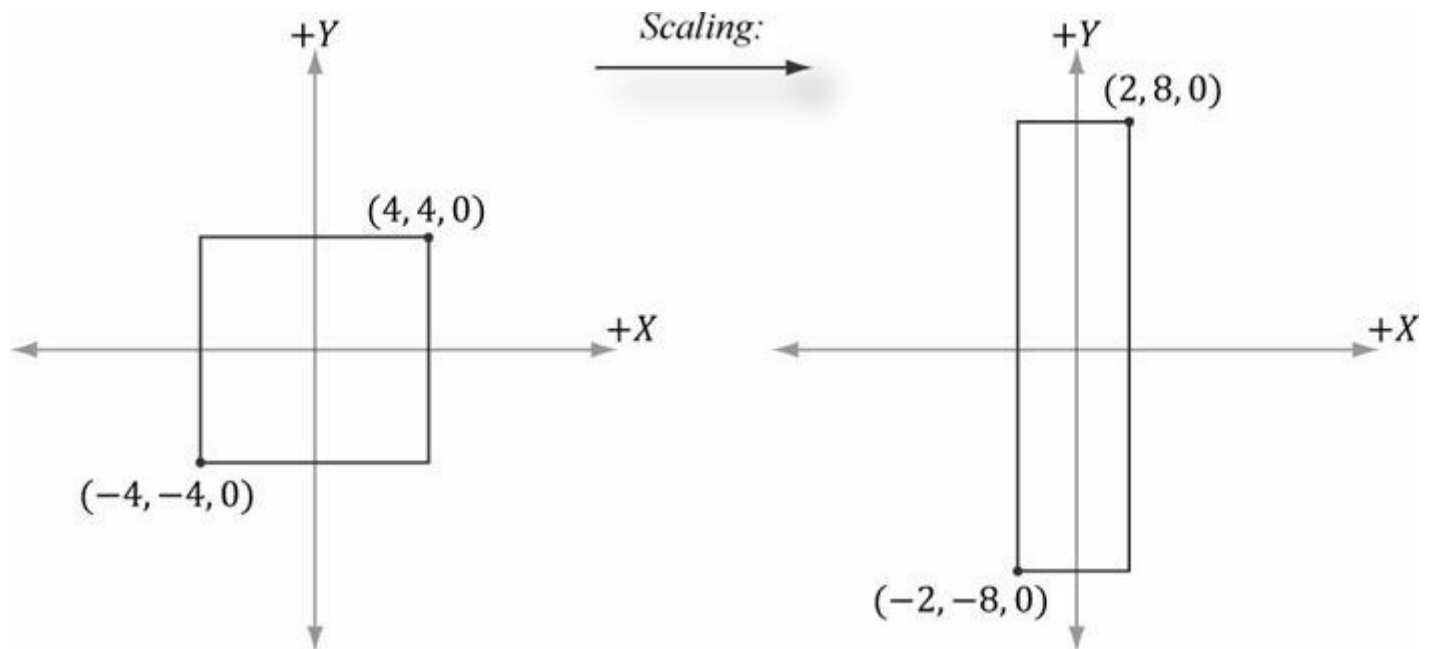


그림 3.2. x축은 1/2 단위로, y축은 2 단위로 스케일링한 모습. 음의 z축 방향을 바라볼 때 기하학적 구조는 $z = 0$ 이므로 기본적으로 2차원임을 유의하십시오.

예제 3.2

최소점 $(-4, -4, 0)$ 과 최대점 $(4, 4, 0)$ 으로 정의된 정사각형이 있다고 가정하자. 이제 이 정사각형을 x축에서 0.5 단위로, y축에서 2.0 단위로 스케일링하고 z축은 변경하지 않으려 한다. 이에 대응하는 스케일링 행렬은 다음과 같다:

$$\mathbf{S} = \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

이제 정사각형을 실제로 스케일링(변환)하기 위해 최소점과 최대점 모두에 이 행렬을 곱합니다:

$$\begin{bmatrix} -4, -4, 0 \end{bmatrix} \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -2, -8, 0 \end{bmatrix} \quad \begin{bmatrix} 4, 4, 0 \end{bmatrix} \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 2, 8, 0 \end{bmatrix}$$

결과는 [그림 3.2](#)에 표시되어 있습니다.

3.1.4 회전

이 절에서는 벡터 $\mathbf{v}$ 를 축 $\mathbf{n}$ 을 중심으로 각도 θ 만큼 회전시키는 방법을 설명합니다. [그림 3.3](#)을 참조하십시오. 각도는 축 $\mathbf{n}$ 을 따라 내려다볼 때 시계 방향으로 측정합니다. 또한 $\|\mathbf{n}\| = 1$ 이라고 가정합니다.

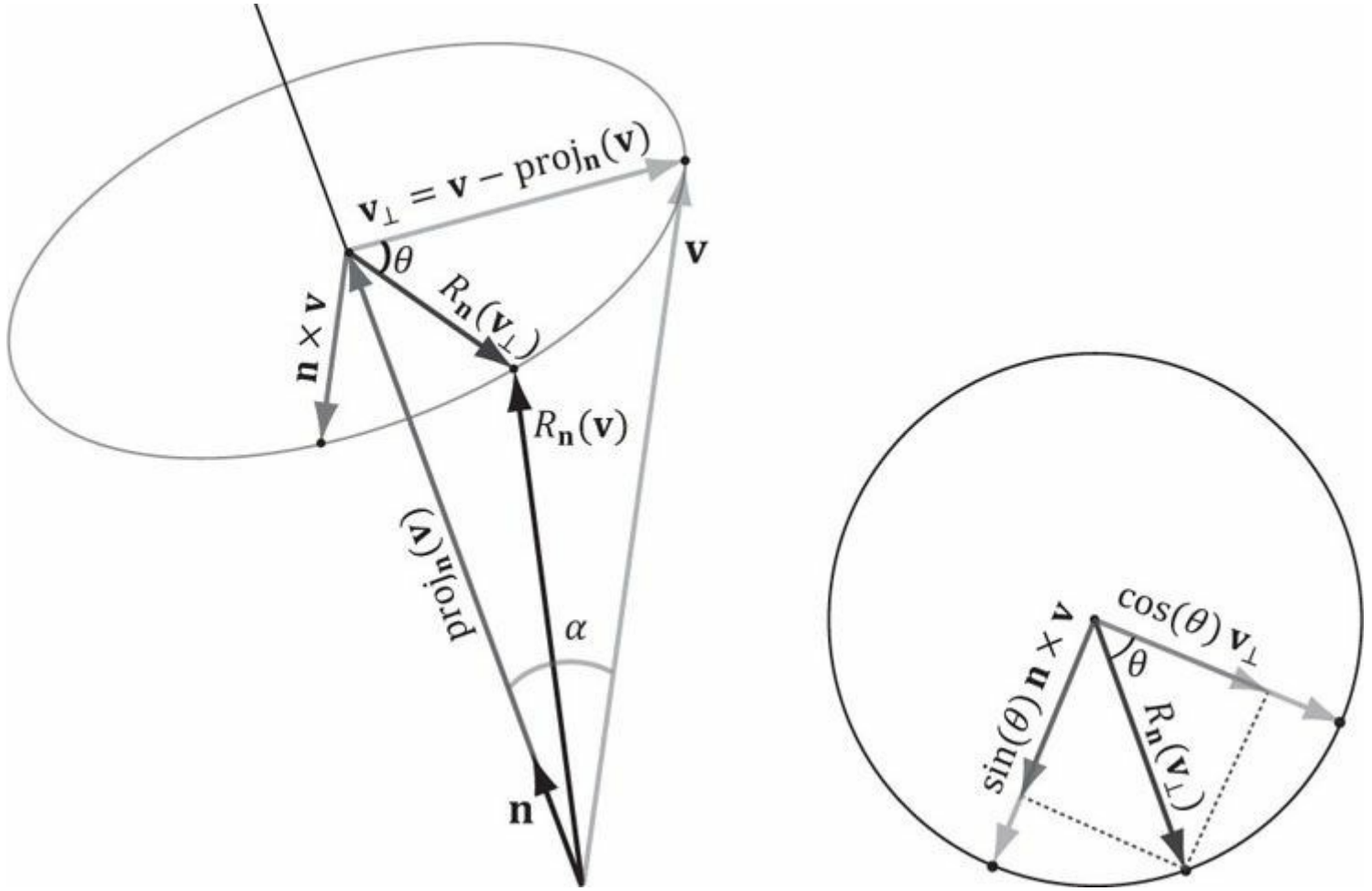


그림 3.3. 벡터 $\mathbf{n}$ 을 중심으로 한 회전의 기하학적 구조.

먼저, 벡터 $\mathbf{v}$ 를 두 부분으로 분해합니다: 하나는 **벡터 $\mathbf{n}$ 과 평행한** 부분이고, 다른 하나는 **벡터 $\mathbf{n}$ 과 직교하는** 부분입니다. 평행한 부분은 단순히 $\text{proj}_{\mathbf{n}}(\mathbf{v})$ 입니다(예제 1.5를 상기하십시오); 직교하는 부분은 $\mathbf{v}_{\perp} = \text{perp}_{\mathbf{n}}(\mathbf{v}) = \mathbf{v} - \text{proj}_{\mathbf{n}}(\mathbf{v})$ 로 주어집니다. (예제 1.5에서 알 수 있듯이, $\mathbf{n}$ 이 단위 벡터이므로 $\text{proj}_{\mathbf{n}}(\mathbf{v}) = (\mathbf{n} \cdot \mathbf{v})\mathbf{n}$ 임을 상기하십시오.) 핵심 관찰점은 $\text{proj}_{\mathbf{n}}(\mathbf{v})$ 중 $\mathbf{n}$ 과 평행한 부분은 회전 하에서 불변이므로, 직교 부분만 어떻게 회전시킬지 알아내면 된다는 점이다. 즉, [그림 3.3](#)에 따라 회전된 벡터 $R_{\mathbf{n}}(\mathbf{v}) = \text{proj}_{\mathbf{n}}(\mathbf{v}) + R_{\mathbf{n}}(\mathbf{v}_{\perp})$ 가 된다.

$R_{\mathbf{n}}(\mathbf{v}_{\perp})$ 을 구하기 위해 회전 평면에 2차원 좌표계를 설정한다. $\mathbf{v}_{\perp}$ 를 하나의 기준 벡터로 사용한다. $\mathbf{v}_{\perp}$ 와 $\mathbf{n}$ 에 직교하는 두 번째 기준 벡터를 얻기 위해 벡터 곱 $\mathbf{n} \times \mathbf{v}$ 를 취한다(왼손 엄지 규칙). [그림 3.3](#)과 [제1장 연습문제 14](#)의 삼각법으로부터 다음을 알 수 있다.

$$\|\mathbf{n} \times \mathbf{v}\| = \|\mathbf{n}\| \|\mathbf{v}\| \sin \alpha = \|\mathbf{v}\| \sin \alpha = \|\mathbf{v}_{\perp}\|$$

여기서 α 는 $\mathbf{n}$ 과 $\mathbf{v}$ 사이의 각도이다. 따라서 두 기준 벡터는 길이가 같으며 회전 원 위에 놓인다. 이제 이 두 기준 벡터를 설정했으므로 삼각법으로부터 다음을 알 수 있다:

$$R_{\mathbf{n}}(\mathbf{v}_{\perp}) = \cos \theta \mathbf{v}_{\perp} + \sin \theta (\mathbf{n} \times \mathbf{v})$$

이를 통해 다음과 같은 회전 공식이 도출됩니다:

$$\begin{aligned}
R_n(\mathbf{v}) &= \text{proj}_n(\mathbf{v}) + R_n(\mathbf{v}_\perp) \\
&= (\mathbf{n} \cdot \mathbf{v})\mathbf{n} + \cos\theta \mathbf{v}_\perp + \sin\theta(\mathbf{n} \times \mathbf{v}) \\
&= (\mathbf{n} \cdot \mathbf{v})\mathbf{n} + \cos\theta(\mathbf{v} - (\mathbf{n} \cdot \mathbf{v})\mathbf{n}) + \sin\theta(\mathbf{n} \times \mathbf{v}) \\
&= \cos\theta \mathbf{v} + (1 - \cos\theta)(\mathbf{n} \cdot \mathbf{v})\mathbf{n} + \sin\theta(\mathbf{n} \times \mathbf{v})
\end{aligned} \tag{식 3.5}$$

이것이 선형 변환임을 증명하는 것은 독자의 연습 문제로 남긴다. 행렬 표현을 구하기 위해, 방정식 3.3과 같이 표준 기저 벡터 각각에 R_n 을 적용한 후, 결과 벡터들을 행렬의 행에 배치한다(방정식 3.4 참조). 최종 결과는 다음과 같다:

$$R_n = \begin{bmatrix} c + (1-c)x^2 & (1-c)xy + sz & (1-c)xz - sy \\ (1-c)xy - sz & c + (1-c)y^2 & (1-c)yz + sx \\ (1-c)xz + sy & (1-c)yz - sx & c + (1-c)z^2 \end{bmatrix}$$

여기서 $c = \cos\theta$, $s = \sin\theta$ 로 정의합니다.

회전 행렬은 흥미로운 성질을 가진다. 각 행 벡터는 길이가 1이다(증명해 보라). 또한 행 벡터들은 서로 직교합니다(확인). 따라서 행 벡터들은 **직교 정규화되어** 있습니다(즉, 서로 직교하고 길이가 1임). 행이 직교 정규화된 행렬을 **직교 행렬**이라고 합니다. 직교 행렬은 역행렬이 실제로 전치 행렬과 같다는 매력적인 특성을 가집니다. 따라서 R_n 의 역행렬은 다음과 같습니다:

$$R_n^{-1} = R_n^T = \begin{bmatrix} c + (1-c)x^2 & (1-c)xy - sz & (1-c)xz + sy \\ (1-c)xy + sz & c + (1-c)y^2 & (1-c)yz - sx \\ (1-c)xz - sy & (1-c)yz + sx & c + (1-c)z^2 \end{bmatrix}$$

일반적으로 직교 행렬은 역행렬을 쉽고 효율적으로 계산할 수 있기 때문에 다루기에 바람직합니다.

특히, 회전을 위해 x-, y-, z-축을 선택할 경우(즉, 각각 $\mathbf{n} = (1, 0, 0)$, $\mathbf{n} = (0, 1, 0)$, $\mathbf{n} = (0, 0, 1)$),

각각 x축, y축, z축을 중심으로 회전하는 다음의 회전 행렬을 얻습니다:

$$R_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & \sin\theta & 0 \\ 0 & -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_y = \begin{bmatrix} \cos\theta & 0 & -\sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, R_z = \begin{bmatrix} \cos\theta & \sin\theta & 0 & 0 \\ -\sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

예제 3.3

최소점 $(-1, 0, -1)$ 과 최대점 $(1, 0, 1)$ 로 정의된 정사각형이 있다고 가정하자. 이제 이 사각형을 y축을 중심으로 시계 방향으로 -30° (즉, 반시계 방향으로 30°) 회전시키고자 한다. 이 경우 $\mathbf{n} = (0, 1, 0)$ 이므로 R_n 이 상당히 단순화된다. 이에 대응하는 y축 회전 행렬은 다음과 같다:

$$R_y = \begin{bmatrix} \cos\theta & 0 & -\sin\theta \\ 0 & 1 & 0 \\ \sin\theta & 0 & \cos\theta \end{bmatrix} = \begin{bmatrix} \cos(-30^\circ) & 0 & -\sin(-30^\circ) \\ 0 & 1 & 0 \\ \sin(-30^\circ) & 0 & \cos(-30^\circ) \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \\ -\frac{1}{2} & 0 & \frac{\sqrt{3}}{2} \end{bmatrix}$$

이제 정사각형을 실제로 회전(변환)시키기 위해, 최소점과 최대점 모두에 이 행렬을 곱합니다:

$$\begin{bmatrix} -1, 0, -1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \\ -\frac{1}{2} & 0 & \frac{\sqrt{3}}{2} \end{bmatrix} \approx \begin{bmatrix} -0.36, 0, -1.36 \end{bmatrix} \quad \begin{bmatrix} 1, 0, 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{3}}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \\ -\frac{1}{2} & 0 & \frac{\sqrt{3}}{2} \end{bmatrix} \approx \begin{bmatrix} 0.36, 0, 1.36 \end{bmatrix}$$

결과는 [그림 3.4](#)에 표시되어 있습니다.

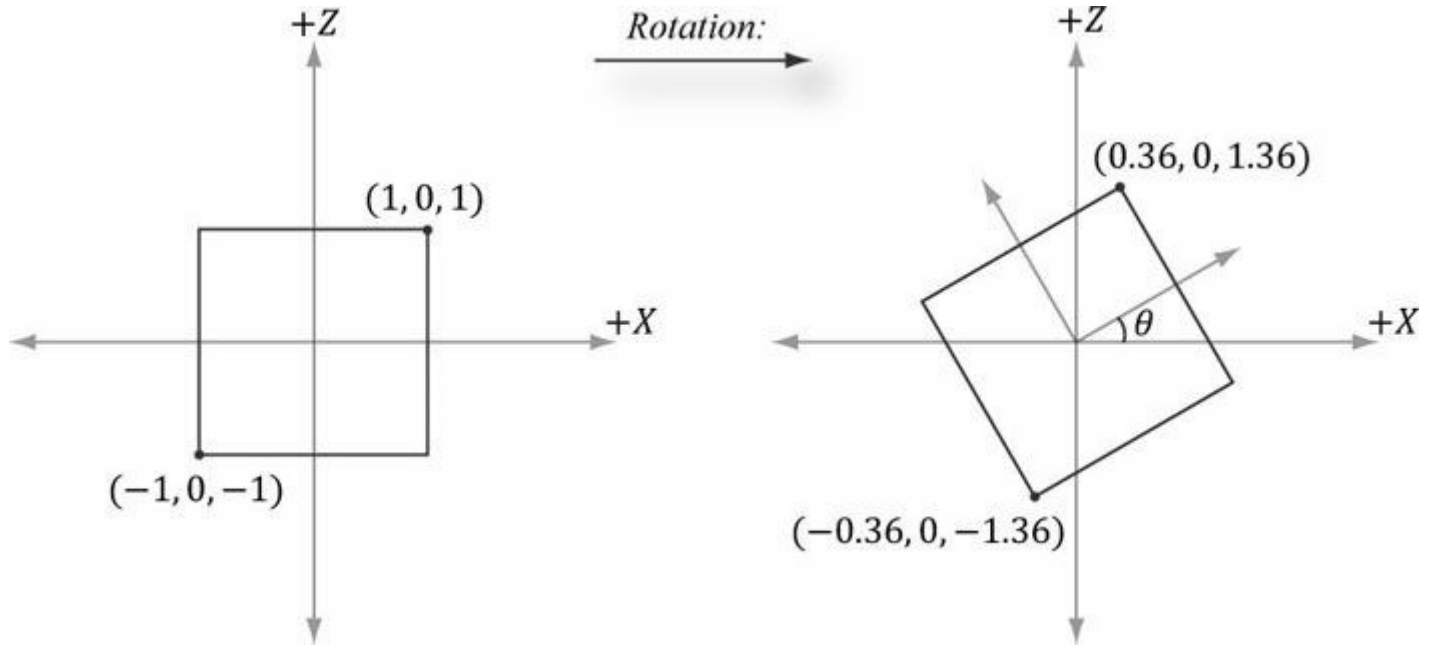


그림 3.4. y축을 중심으로 시계 방향으로 -30° 회전. 양의 y축을 따라 내려다볼 때, $y = 0$ 이므로 기하학적 구조는 기본적으로 2차원임을 유의하십시오.

3.2 아핀 변환

3.2.1 동차 좌표

다음 절에서 보게 되겠지만, 아핀 변환은 선형 변환과 평행 이동의 조합이다. 그러나 벡터에 평행 이동을 적용하는 것은 의미가 없다. 벡터는 위치와 무관하게 방향과 크기만을 나타내기 때문이다. 즉, 벡터는 평행 이동 하에서 불변이어야 한다. 평행 이동은 점(즉, 위치 벡터)에만 적용되어야 합니다. **동차 좌표**는 점과 벡터를 일관되게 처리할 수 있는 편리한 표기법을 제공합니다. 동차 좌표에서는 4-튜플로 확장하며, 네 번째 w-좌표에 무엇을 배치할지는 점인지 벡터인지에 따라 달라집니다. 구체적으로 다음과 같이 표기합니다:

1. 벡터의 경우 $(x, y, z, 0)$
2. 점: $(x, y, z, 1)$

나중에 살펴볼겠지만, 점에 대해 $w = 1$ 로 설정하면 점의 평행 이동이 올바르게 작동하며, 벡터에 대해 $w = 0$ 으로 설정하면 평행 이동에 의해 벡터의 좌표가 변경되는 것을 방지합니다(벡터의 좌표를 평행 이동시키면 방향과 크기가 변하므로 원하지 않음—평행 이동은 벡터의 속성을 변경해서는 안 됨).

Note: 동차 좌표계의 표기법은 [그림 1.17](#)에 나타난 개념과 일치한다. 즉, 두 점 $\mathbf{q} - \mathbf{p} = (q_x, q_y, q_z, 1) - (p_x, p_y, p_z, 1) = (q_x - p_x, q_y - p_y, q_z - p_z, 0)$ 는 벡터를 생성하며, 점과 벡터 $\mathbf{p} + \mathbf{v} = (p_x, p_y, p_z, 1) + (v_x, v_y, v_z, 0) = (p_x + v_x, p_y + v_y, p_z + v_z, 1)$ 는 점으로 표현된다.

3.2.2 정의와 행렬 표현

선형 변환만으로는 우리가 원하는 모든 변환을 설명할 수 없으므로, 더 넓은 범주의 함수인 아핀 변환으로 확장합니다. 아핀 변환은 선형 변환에 평행 이동 벡터 **b**를 더한 것으로, 다음과 같습니다:

$$\alpha(\mathbf{u}) = \tau(\mathbf{u}) + \mathbf{b}$$

또는 행렬 표기법으로:

$$\alpha(\mathbf{u}) = \mathbf{u} \mathbf{A} + \mathbf{b} = \begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} \\ A_{21} & A_{22} & A_{23} \\ A_{31} & A_{32} & A_{33} \end{bmatrix} + \begin{bmatrix} b_x & b_y & b_z \end{bmatrix} = \begin{bmatrix} x' & y' & z' \end{bmatrix}$$

여기서 **A**는 선형 변환의 행렬 표현이다.

w = 1로 동차 좌표로 확장하면, 이를 더 간결하게 다음과 같이 쓸 수 있다:

$$\begin{bmatrix} x & y & z & 1 \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} & A_{13} & 0 \\ A_{21} & A_{22} & A_{23} & 0 \\ A_{31} & A_{32} & A_{33} & 0 \\ b_x & b_y & b_z & 1 \end{bmatrix} = \begin{bmatrix} x' & y' & z' & 1 \end{bmatrix} \tag{식 3.6}$$

식 3.6의 4×4 행렬을 아핀 변환의 행렬 표현이라고 부른다.

b에 의한 덧셈은 본질적으로 평행 이동(즉, 위치 변화)임을 관찰하십시오. 우리는 벡터에 이를 적용하고 싶지 않습니다.

벡터에는 위치가 없기 때문이다. 그러나 우리는 여전히 아핀 변환의 선형 부분만을 벡터에 적용하고자 한다. 벡터의 네 번째 성분 *w* = 0으로 설정하면 **b**에 의한 평행 이동이 적용되지 *않는다*(행렬 곱셈을 통해 확인해 보라).

*Note: 이전 4×4 아핀 변환 행렬의 4번째 열과 행 벡터의 내적은 [x, y, z, w]·[0, 0, 0, 1] = w이므로, 이 행렬은 입력 벡터의 *w* 좌표를 변경하지 않습니다.*

3.2.2 평행 이동

*정규 변환*은 단순히 인자를 반환하는 선형 변환입니다. 즉, *f*(**u**) = **u**입니다. 이 선형 변환의 행렬 표현이 정규 행렬임을 증명할 수 있습니다.

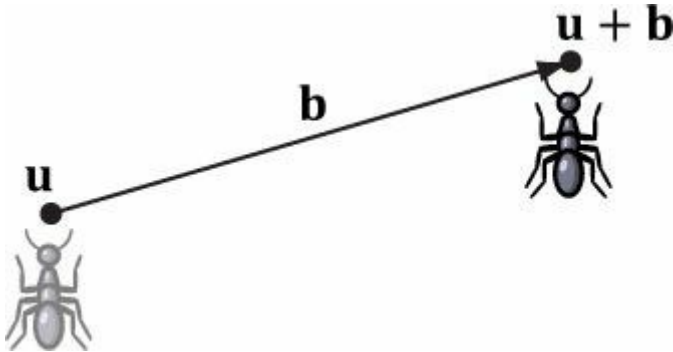
이제, 우리는 번역 변환을 선형 변환이 항등 변환인 아핀 변환으로 정의한다

변환인 아핀 변환으로 정의한다. 즉,

$$\tau(\mathbf{u}) = \mathbf{u} \mathbf{I} + \mathbf{b} = \mathbf{u} + \mathbf{b}$$

보시다시피, 이는 단순히 점 **u**를 **b**만큼 이동(또는 변위)시킵니다. [그림 3.5](#)는 이를 사용하여 객체를 변위시키는 방법을 보여줍니다

—물체 상의 모든 점을 동일한 벡터 **b**만큼 이동시켜 위치를 변경합니다.



*그림 3.5. 개미의 위치를 변위 벡터 **b**만큼 이동시키는 것.*

방정식 3.6에 의해, *τ*는 행렬 표현으로 다음과 같다:

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ b_x & b_y & b_z & 1 \end{bmatrix}$$

이를 **평행 이동 행렬**이라고 한다.

이동 행렬의 역행렬은 다음과 같이 주어진다:

$$\mathbf{T}^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -b_x & -b_y & -b_z & 1 \end{bmatrix}$$

예제 3.4

최소점 (-8, 2, 0)과 최대점 (-2, 8, 0)으로 정의된 정사각형이 있다고 가정하자. 이제 이 정사각형을 x축 방향으로 12 단위, y축 방향으로 -10.0 단위 이동시키고 z축은 그대로 유지하고자 한다. 이에 대응하는 평행 이동 행렬은 다음과 같다:

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 12 & -10 & 0 & 1 \end{bmatrix}$$

이제 정사각형을 실제로 변환(이동)시키려면 최소점과 최대점 모두에 이 행렬을 곱합니다:

$$\begin{bmatrix} -8 & 2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 12 & -10 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 4 & -8 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} -2 & 8 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 12 & -10 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 10 & -2 & 0 & 1 \end{bmatrix}$$

결과는 [그림 3.6](#)에 표시되어 있습니다.

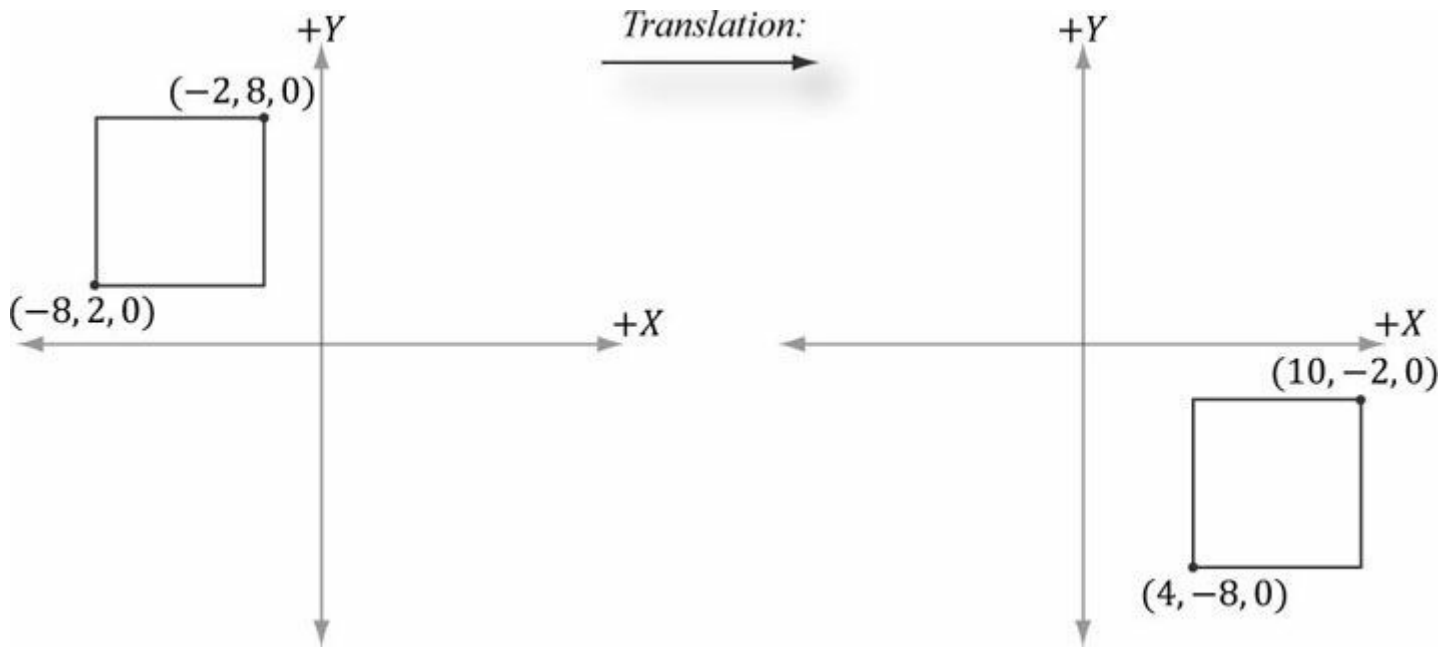


그림 3.6. x축에서 12 단위, y축에서 -10 단위 이동. 음의 z축 방향으로 내려다볼 때 $z = 0$ 이므로 기하학적 구조는 기본적으로 2차원임을 유의하십시오.

변환 행렬 T 를 고려하자. 점/벡터를 $vT = v'$ 의 곱셈으로 변환한다는 점을 상기하라. 점/벡터를 T 로 변환한 후 역변환 T^{-1} 로 다시 변환하면 원래 벡터로 돌아온다: $vT^{T^{-1}} = vI = v$. 즉, 역변환은 변환을 되돌린다. 예를 들어, x축에서 점 5단위만큼 평행 이동시킨 후, 역방향으로 x축에서 -5단위만큼 평행 이동시키면 시작점으로 돌아온다. 마찬가지로, 한 점을 y축을 중심으로 30° 회전시킨 후 역회전인 -30° 로 다시 회전시키면 원래 점으로 돌아옵니다. 요약하면, 변환 행렬의 역행렬은 두 변환의 합성이 기하학적 구조를 변화시키지 않도록 반대 방향의 변환을 수행합니다.

3.2.3 확대 및 회전을 위한 아핀 행렬

$b = 0$ 일 때 아핀 변환은 선형 변환으로 환원됨을 관찰할 수 있다. 모든 선형 변환은 $b = 0$ 인 아핀 변환으로 표현할 수 있다. 이는 다시 말해 모든 선형 변환을 4×4 아핀 행렬로 표현할 수 있음을 의미한다. 예를 들어, 4×4 행렬로 작성된 스케일링 및 회전 행렬은 다음과 같다:

$$S = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R_n = \begin{bmatrix} c + (1-c)x^2 & (1-c)xy + sz & (1-c)xz - sy & 0 \\ (1-c)xy - sz & c + (1-c)y^2 & (1-c)yz + sx & 0 \\ (1-c)xz + sy & (1-c)yz - sx & c + (1-c)z^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

이러한 방식으로, 모든 변환을 4×4 행렬과 1×4 동차 행 벡터로 표현된 점 및 벡터를 사용하여 일관되게 표현할 수 있습니다.

3.2.4 아핀 변환 행렬의 기하학적 해석

이 섹션에서는 아핀 변환 행렬 내부의 숫자들이 기하학적으로 무엇을 의미하는지에 대한 직관을 발전시킵니다. 먼저,

형상을 보존하는 변환인 **강체 변환**을 고려해 보겠습니다. 강체 변환의 실제 사례로는 책상 위의 책을 집어 책상에 올려놓는 행동이 있을 수 있습니다. 이 과정에서 책은 책상에서 책장으로 이동하지만, 동시에 책의 방향(회전)도 변경될 가능성이 매우 높습니다. τ 를 물체를 회전시키려는 방식을 나타내는 회전 변환으로, $\mathbf{b}$ 를 물체를 평행 이동시키려는 방식을 나타내는 변위 벡터로 정의하자. 이 강체 변환은 다음과 같은 아핀 변환으로 표현될 수 있다:

$$\alpha(x, y, z) = \tau(x, y, z) + \mathbf{b} = x\tau(\mathbf{i}) + y\tau(\mathbf{j}) + z\tau(\mathbf{k}) + \mathbf{b}$$

행렬 표기법에서, 벡터에는 평행 이동이 적용되지 않도록 점에 대해 $w = 1$, 벡터에 대해 $w = 0$ 인 동차 좌표를 사용하여 다음과 같이 표기한다:

$$\begin{bmatrix} x & y & z & w \end{bmatrix} \begin{bmatrix} \leftarrow \tau(\mathbf{i}) \rightarrow \\ \leftarrow \tau(\mathbf{j}) \rightarrow \\ \leftarrow \tau(\mathbf{k}) \rightarrow \\ \leftarrow \mathbf{b} \rightarrow \end{bmatrix} = \begin{bmatrix} x' & y' & z' & w \end{bmatrix} \quad (\text{식 3.7})$$

이제 이 방정식이 기하학적으로 무엇을 하는지 보려면 행렬의 행 벡터들을 그래프화하기만 하면 됩니다(그림 3.7 참조). τ 가 회전 변환이므로 길이와 각도를 보존합니다. 특히, τ 가 표준 기저 벡터 $\mathbf{i}$, $\mathbf{j}$, $\mathbf{k}$ 를 새로운 방향 $\tau(\mathbf{i})$, $\tau(\mathbf{j})$, $\tau(\mathbf{k})$ 로 회전시키고 있음을 알 수 있습니다. 벡터 $\mathbf{b}$ 는 원점으로부터의 변위를 나타내는 위치 벡터에 불과하다. 이제 그림 3.7은 $\alpha(x, y, z) = x\tau(\mathbf{i}) + y\tau(\mathbf{j}) + z\tau(\mathbf{k}) + \mathbf{b}$ 가 계산될 때 변환된 점이 기하학적으로 어떻게 얻어지는지 보여준다.

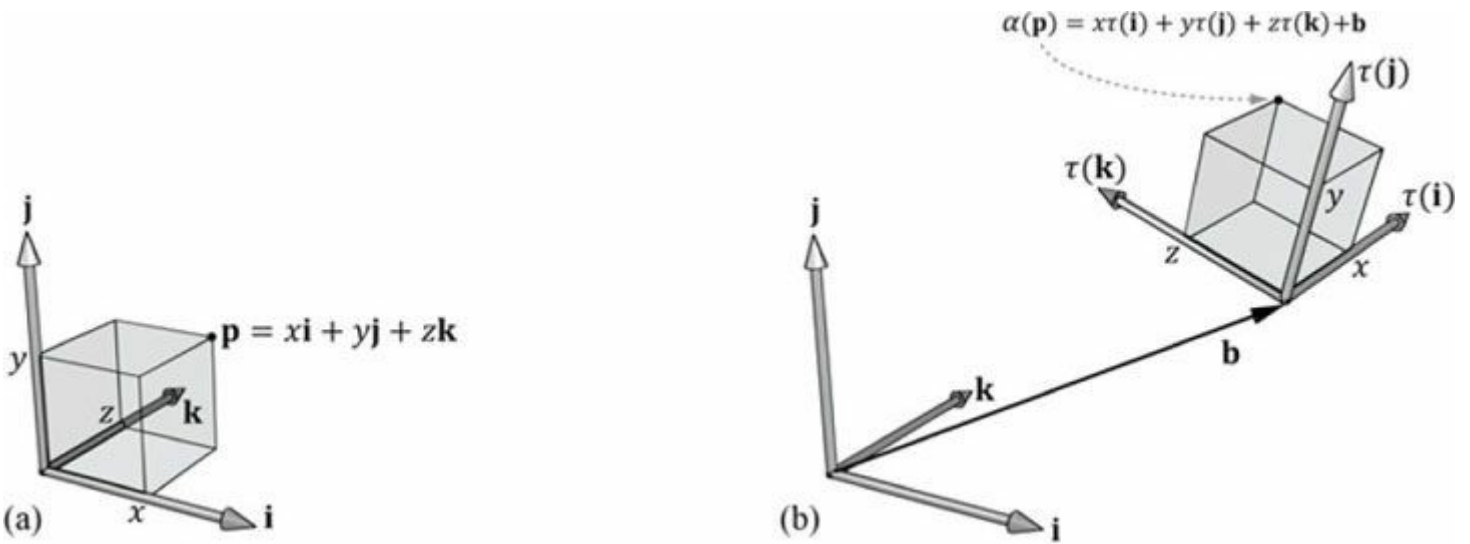


그림 3.7. 아핀 변환 행렬의 행에 대한 기하학적 해석. 변환된 점 $\alpha(\mathbf{p})$ 는 변환된 기준 벡터 $\tau(\mathbf{i})$, $\tau(\mathbf{j})$, $\tau(\mathbf{k})$ 와 오프셋 $\mathbf{b}$ 의 선형 조합으로 주어진다.

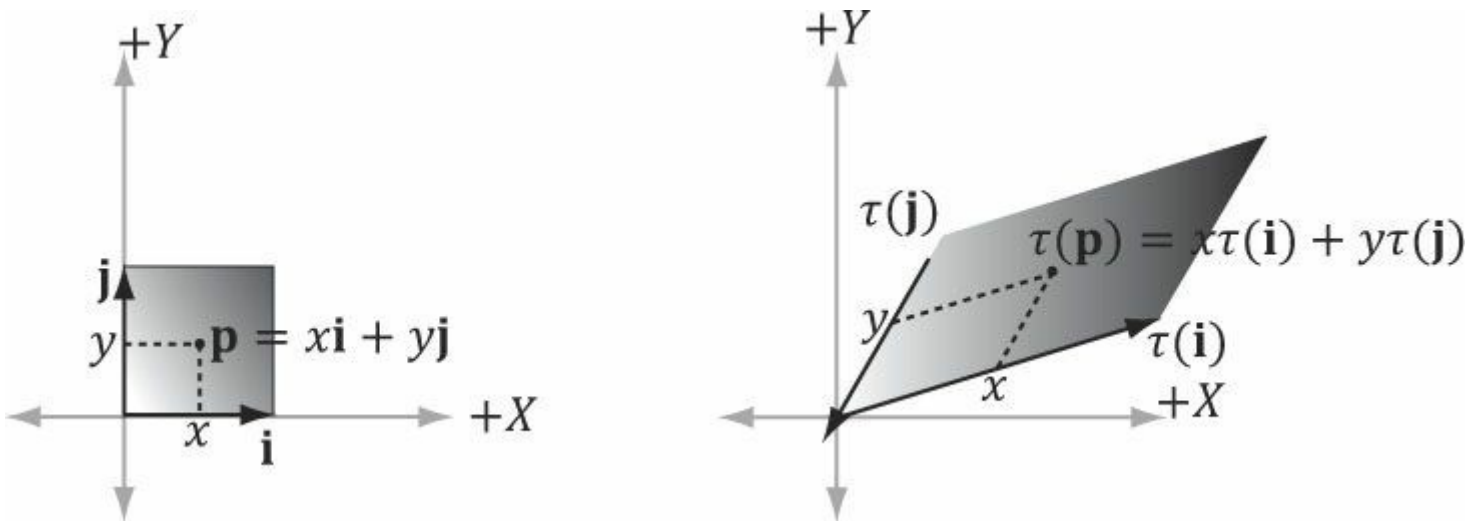


그림 3.8. 정사각형을 평행사변형으로 왜곡하는 선형 변환의 경우, 변환된 점 $\tau(\mathbf{p}) = (x, y)$ 는 변환된 기준 벡터 $\tau(\mathbf{i})$, $\tau(\mathbf{j})$ 의 선형 조합으로 주어진다.

확대/축소 변환이나 기울기 변환에도 동일한 원리가 적용됩니다. [그림 3.8](#)과 같이 정사각형을 평행사변형으로 왜곡하는 선형 변환 τ 를 고려해 보십시오. 왜곡된 점은 단순히 왜곡된 기저 벡터들의 선형 결합입니다.

3.3 변환의 합성

S를 스케일링 행렬, R을 회전 행렬, T를 평행 이동 행렬이라고 하자. $i = 0, 1, \dots, 7$ 에 대해 8개의 꼭짓점 $\mathbf{v}_i$ 로 구성된 정육면체가 있고, 각 꼭짓점에 이 세 변환을 순차적으로 적용하고자 한다고 가정하자. 이를 수행하는 가장 명백한 방법은 단계별로 진행하는 것이다:

$$((\mathbf{v}_i \mathbf{S})\mathbf{R})\mathbf{T} = (\mathbf{v}_i' \mathbf{R})\mathbf{T} = \mathbf{v}_i'' \mathbf{T} = \mathbf{v}_i''' \quad (i = 0, 1, \dots, 7) \text{ 그러나 행렬 곱셈은 결합법칙}$$

을 따르므로, 이를 다음과 같이 동등하게 쓸 수 있다:

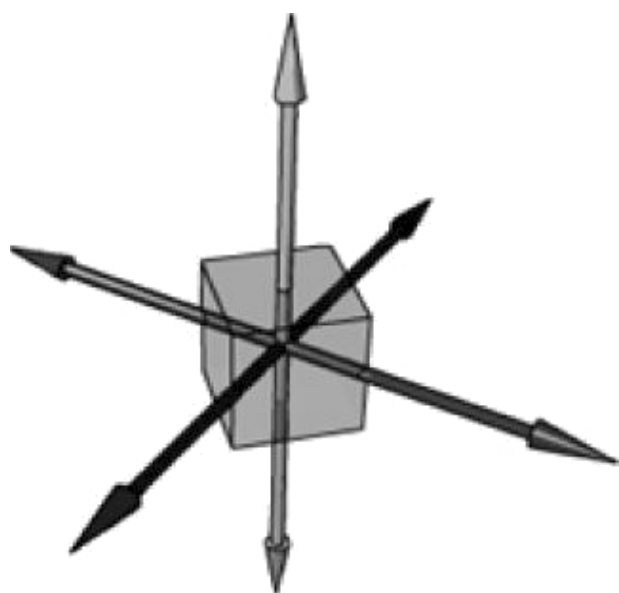
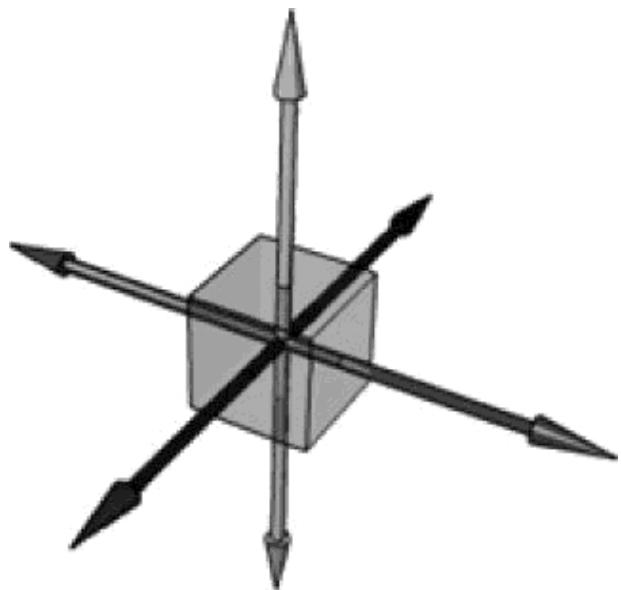
$$\mathbf{v}_i (\mathbf{SRT}) = \mathbf{v}_i''' \quad i = 0, 1, \dots, 7$$

행렬 $\mathbf{C} = \mathbf{SRT}$ 는 세 가지 변환을 하나의 순수 변환 행렬로 통합한 것으로 볼 수 있습니다. 즉, 행렬-행렬 곱셈을 통해 변환들을 연결할 수 있습니다.

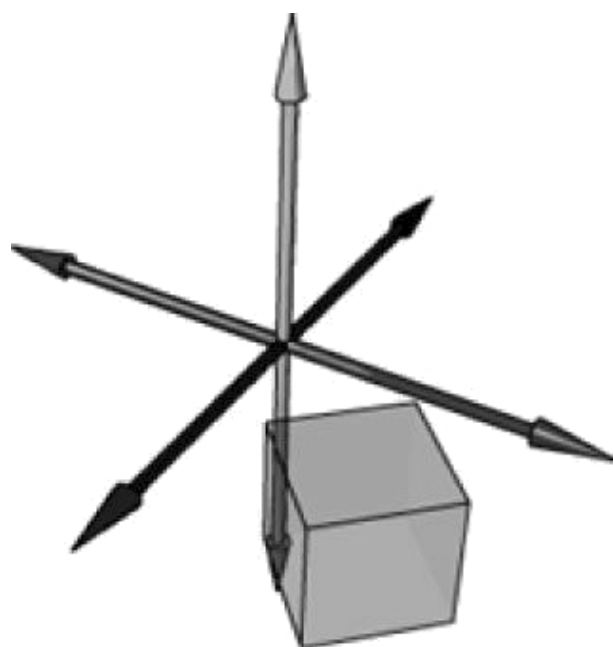
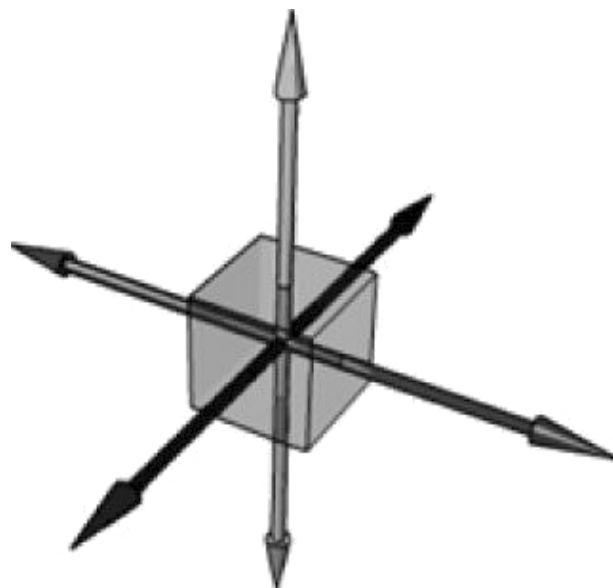
이는 성능에 영향을 미칩니다. 이를 확인하기 위해, 3D 객체가 20,000개의 점으로 구성되어 있고

단계별 접근법을 사용하면 $20,000 \times 3$ 회의 벡터-행렬 곱셈이 필요합니다. 반면 결합 행렬 접근법은 20,000회의 벡터-행렬 곱셈과 2회의 행렬-행렬 곱셈이 필요합니다. 벡터-행렬 곱셈을 크게 절약하는 대가로 추가되는 2회의 행렬-행렬 곱셈은 분명히 저렴한 비용입니다.

다시 한번 강조하건대, 행렬 곱셈은 교환법칙을 따르지 않습니다. 이는 기하학적으로도 관찰됩니다. 예를 들어, 회전()에 이어 평행 이동을 수행하는 경우, 이를 행렬 곱 $\mathbf{RT}$ 로 표현할 수 있지만 동일한 변환을 얻지 못합니다. 동일한 회전에 동일한 평행 이동을 수행하는 경우, 즉 $\mathbf{T}\mathbf{R}$ 과 동일한 변환을 얻게 됩니다. [그림 3.9](#)가 이를 보여줍니다. 변환을 생성하지 않습니다. 즉, 동일한 평행 이동에 동일한 회전을 적용한 경우, 즉 $\mathbf{T}\mathbf{R}$ 과 동일하지 않습니다. [그림 3.9](#)는 이를 보여줍니다.



(a)



(b)

그림 3.9. (a) 먼저 회전한 후 평행 이동. (b) 먼저 평행 이동한 후 회전.

3.4 좌표 변환의 변화

스칼라 100°C 는 섭씨 온도 척도에서 물의 끓는점을 나타냅니다. *같은* 물의 끓는점을 화씨 온도 척도에서 어떻게 표현할까요? 즉, 화씨 척도에서 물의 끓는점을 나타내는 스칼라는 무엇일까요? 이 변환(또는 좌표계 변경)을 위해 섭씨와 화씨 척도의 관계를 알아야 합니다. 둘은 다음과 같이 관련됩니다.

$$T_F = \frac{9}{5}T_C + 32^{\circ}$$

화씨 온도 척도에 대한 끓는 물의 온도는 다음과 같이 주어집니다.

이 예시는 *프레임 A*에 상대적인 어떤 양을 나타내는 스칼라 k 를, *프레임 A*와 *B*의 관계를 알고 있다면 다른 프레임 *B*에 상대적인 *동일한* 양을 나타내는 새로운 스칼라 k' 로 변환할 수 있음을 보여줍니다. 다음 소절에서는 유사한 문제를 다루지만, 스칼라 대신 한 좌표계에 대한 점/벡터의 좌표를 다른 좌표계에 대한 좌표로 변환하는 방법에 관심을 갖습니다(그림 3.10 참조). 한 좌표계의 좌표를 다른 좌표계의 좌표로 변환하는 변환을 *좌표계 변환*이라고 합니다.

좌표계 변환에서는 기하학적 구조 자체가 변하는 것이 아니라, 우리가 좌표계의 기준점을 변경함으로써 기하학적 구조의 좌표 표현을 변경하는 것입니다. 이는 일반적으로 회전, 평행 이동, 크기 변환을 생각할 때 물리적으로 기하학적 구조를 이동하거나 변형시키는 방식으로 생각하는 것과는 대조적입니다.

3D 컴퓨터 그래픽스에서는 여러 좌표계를 사용하므로, 서로 다른 좌표계 간 변환 방법을 알아야 합니다.

위치는 점의 속성이지만 벡터의 속성은 아니므로, 좌표 변환의 변화는 점과 벡터에 대해 다르게 나타납니다.

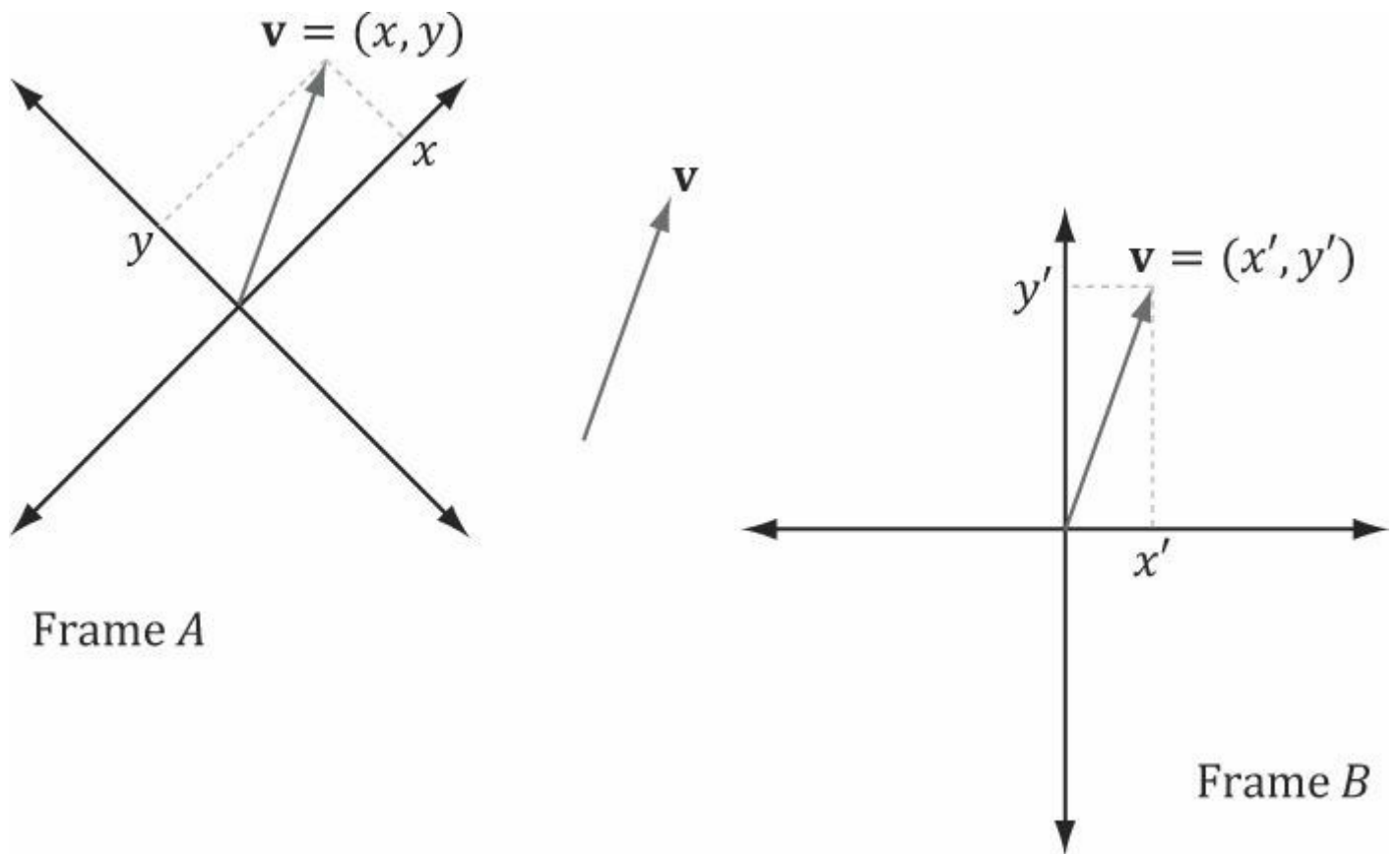


그림 3.10. 동일한 벡터 $\mathbf{v}$ 는 서로 다른 좌표계에 대해 설명될 때 다른 좌표를 가집니다. *좌표계 A*에 대해서는 좌표 (x, y) 를, *좌표계 B*에 대해서는 좌표 (x', y') 를 가집니다.

3.4.1 벡터

그림 3.11을 고려하자. 여기에는 두 개의 좌표계 *A*와 *B*, 그리고 벡터 $\mathbf{p}$ 가 있다. 벡터 $\mathbf{p}$ 의 *좌표계 A*에 대한 좌표 $\mathbf{p}_A = (x, y)$ 가 주어졌다고 가정하자.

즉, 한 좌표계에 대한 벡터의 좌표가 주어졌을 때, 다른 좌표계에 대한 동일한 벡터의 좌표를 어떻게 구할 것인가?

그림 3.11에서 다음과 같이 명확히 알 수 있다.

$$\mathbf{p} = x \mathbf{u} + y \mathbf{v}$$

여기서 $\mathbf{u}$ 와 $\mathbf{v}$ 는 각각 *좌표계 A*의 x 축과 y 축을 따라 향하는 단위 벡터이다. 이전 방정식의 각 벡터를 좌표계 *B* 좌표로 표현하면 다음과 같다:

$$\mathbf{p}_B = x \mathbf{u}_B + y \mathbf{v}_B$$

따라서 $\mathbf{p}_A = (x, y)$ 가 주어지고, 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 의 좌표가 좌표계 B 에 대해 알려져 있다면, 즉 $\mathbf{u}_B = (u_x, u_y)$ 와 $\mathbf{v}_B = (v_x, v_y)$ 를 안다면, 항상 $\mathbf{p}_B = (x', y')$ 를 구할 수 있습니다.

3차원으로 일반화하면, $\mathbf{p}_A = (x, y, z)$ 인 경우

$$\mathbf{p}_B = x \mathbf{u}_B + y \mathbf{v}_B + z \mathbf{w}_B$$

여기서 $\mathbf{u}, \mathbf{v}, \mathbf{w}$ 는 각각 좌표계 A 의 x 축, y 축, z 축을 따라 향하는 단위 벡터이다.

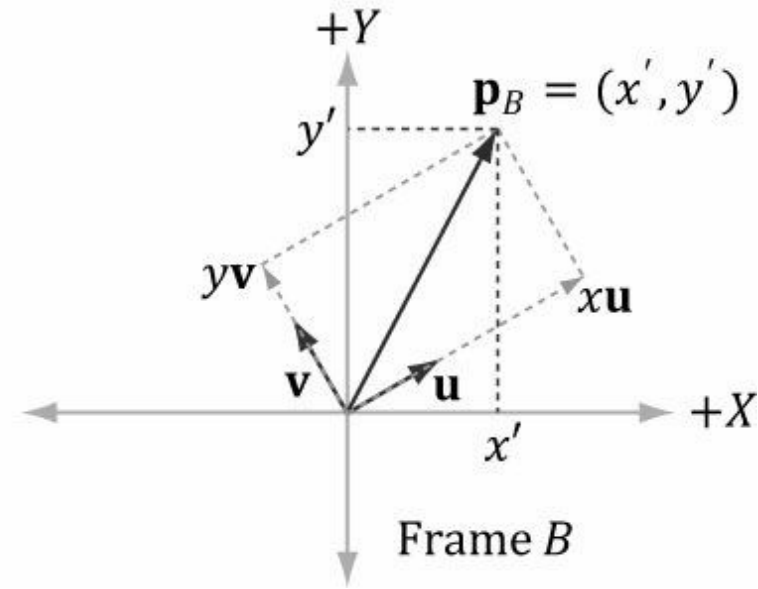
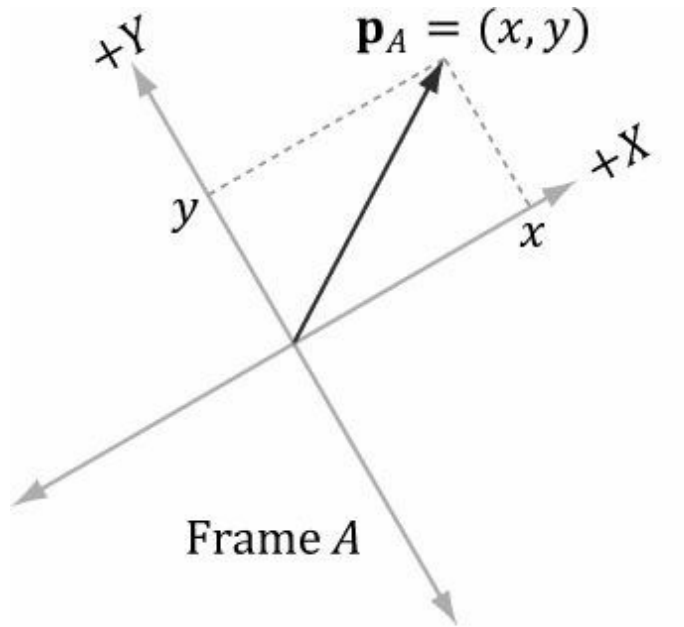


그림 3.11. 좌표계 B 에 대한 $\mathbf{p}$ 의 좌표 구하기의 기하학적 구조.

3.4.2 점

점의 좌표 변환은 벡터의 경우와 약간 다릅니다. 이는 점의 위치가 중요하기 때문에, 그림 3.11에서 벡터를 변환한 것처럼 점을 단순히 평행 이동시킬 수 없기 때문입니다.

그림 3.12는 상황을 보여주며, 점 $\mathbf{p}$ 는 다음 방정식으로 표현될 수 있음을 알 수 있다:

$$\mathbf{p} = x \mathbf{u} + y \mathbf{v} + \mathbf{Q}$$

여기서 $\mathbf{u}$ 와 $\mathbf{v}$ 는 각각 좌표계 A 의 x 축과 y 축을 향하는 단위 벡터이며, $\mathbf{Q}$ 는 좌표계 A 의 원점이다. 위 방정식에서 각 벡터/점을 좌표계 B 좌표로 표현하면 다음과 같다:

$$\mathbf{p}_B = x \mathbf{u}_B + y \mathbf{v}_B + \mathbf{Q}_B$$

따라서, $\mathbf{p}_A = (x, y)$ 가 주어지고 벡터 $\mathbf{u}$ 와 $\mathbf{v}$ 의 좌표, 그리고 좌표계 B 에 대한 원점 $\mathbf{Q}$ 를 안다면, 즉 $\mathbf{u}_B = (u_x, u_y)$, $\mathbf{v}_B = (v_x, v_y)$, $\mathbf{Q}_B = (Q_x, Q_y)$ 를 안다면, 항상 $\mathbf{p}_B = (x', y')$ 를 구할 수 있다.

3차원으로 일반화하면, $\mathbf{p}_A = (x, y, z)$ 일 때,

$$\mathbf{p}_B = x \mathbf{u}_B + y \mathbf{v}_B + z \mathbf{w}_B + \mathbf{Q}_B$$

여기서 $\mathbf{u}, \mathbf{v}, \mathbf{w}$ 는 각각 좌표계 A 의 x 축, y 축, z 축을 향하는 단위 벡터이며, $\mathbf{Q}$ 는 좌표계 A 의 원점이다.

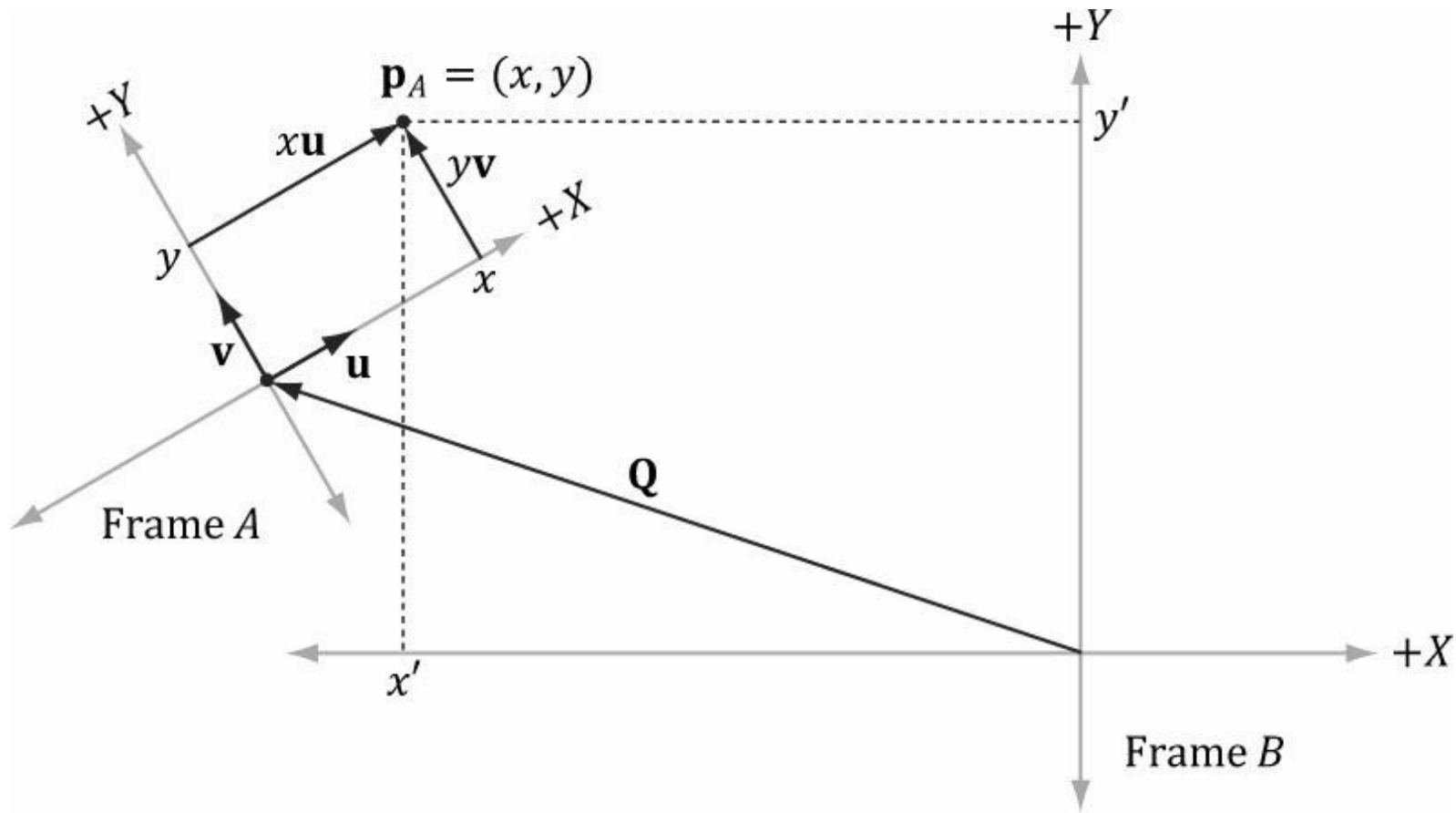


그림 3.12. 기준계 B 에 대한 점 p 의 좌표 구하기의 기하학적 구조.

3.4.3 행렬 표현

지금까지 살펴본 바에 따르면, 벡터 및 점의 좌표 변환은 다음과 같습니다: $(x', y', z') = x \mathbf{u}_B + y \mathbf{v}_B + z \mathbf{w}_B$ 벡터
의 경우

점의 경우: $(x', y', z') = x \mathbf{u}_B + y \mathbf{v}_B + z \mathbf{w}_B + \mathbf{Q}_B$

동차 좌표를 사용하면 벡터와 점을 하나의 방정식으로 처리할 수 있습니다:

$$(x', y', z', w) = x \mathbf{u}_B + y \mathbf{v}_B + z \mathbf{w}_B + w \mathbf{Q}_B \quad (\text{식 3.8})$$

$w = 0$ 일 때, 이 방정식은 벡터에 대한 좌표 변환으로 환원된다; $w = 1$ 일 때, 이 방정식은 점에 대한 좌표 변환으로 환원된다. 방정식 3.8의 장점은 w 좌표를 올바르게 설정하기만 하면 벡터와 점 모두에 적용 가능하다는 점이다. 더 이상 두 개의 방정식(벡터용 하나, 점용 하나)이 필요하지 않다. 방정식 2.3은 방정식 3.8을 행렬 언어로 표현할 수 있음을 말한다:

$$\begin{bmatrix} x' & y' & z' & w \end{bmatrix} = \begin{bmatrix} x & y & z & w \end{bmatrix} \begin{bmatrix} \leftarrow \mathbf{u}_B \rightarrow \\ \leftarrow \mathbf{v}_B \rightarrow \\ \leftarrow \mathbf{w}_B \rightarrow \\ \leftarrow \mathbf{Q}_B \rightarrow \end{bmatrix}$$

$$= \begin{bmatrix} x & y & z & w \end{bmatrix} \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ Q_x & Q_y & Q_z & 1 \end{bmatrix} \quad (\text{식 3.9})$$

$$= x\mathbf{u}_B + y\mathbf{v}_B + z\mathbf{w}_B + w\mathbf{Q}_B$$

여기서 $\mathbf{Q}_B = (Q_x \ Q_y \ Q_z \ 1)$, $\mathbf{u}_B = (u_x \ u_y \ u_z \ 0)$, $\mathbf{v}_B = (v_x \ v_y \ v_z \ 0)$, $\mathbf{w}_B = (w_x \ w_y \ w_z \ 0)$ 및 $\mathbf{w}_{(B)} = (w_{(x)} \ w_y \ w_{(z)} \ 0)$ 는 좌표계 B에 대한 좌표계 A의 원점과 축을 균일 좌표로 설명한다. z , $0)$, $\mathbf{w}_B = (w_x \ , \ w_y \ , \ w_z \ , 0)$ 는 기준계 B에 대한 기준계 A의 원점과 축을 동차 좌표로 나타냅니다. 방정식 3.9의 4×4 행렬을 *좌표 변환 행렬* 또는 *기준계 변환 행렬*이라고 부르며, 이 행렬이 기준계 A의 좌표를 기준계 B의 좌표로 변환(또는 매핑)한다고 말합니다.

3.4.4 결합법칙과 좌표 변환 행렬

이제 세 개의 좌표계 F, G, H 가 있다고 가정하자. 또한 $\mathbf{A}$ 를 F 에서 G 로의 좌표계 변환 행렬로, $\mathbf{B}$ 를 G 에서 H 로의 좌표계 변환 행렬로 정의하자. 벡터의 좌표계 F 에 대한 좌표 $\mathbf{p}_F$ 가 주어졌을 때, 동일한 벡터의 좌표계 H 에 대한 좌표를 구하고자 한다. 즉, $\mathbf{p}_{(H)}$ 를 구하고자 한다. 이를 단계별로 수행하는 한 가지 방법은 다음과 같다:

$$\begin{aligned} (\mathbf{p}_F \mathbf{A}) \mathbf{B} &= \mathbf{p}_H \\ (\mathbf{p}_G) \mathbf{B} &= \mathbf{p}_H \end{aligned}$$

그러나 행렬 곱셈은 결합법칙을 따르므로, $(\mathbf{p}_F \mathbf{A}) \mathbf{B} = \mathbf{p}_H$ 를 다음과 같이 다시 쓸 수 있습니다:

$$\mathbf{p}_F (\mathbf{AB}) = \mathbf{p}_H$$

이러한 의미에서 행렬 곱 $\mathbf{C} = \mathbf{AB}$ 는 F 에서 H 로의 직접적인 좌표계 변환 행렬로 볼 수 있습니다. 이는 $\mathbf{A}$ 와 $\mathbf{B}$ 의 효과를 하나의 순수 행렬로 결합합니다. (이 개념은 함수 합성과 유사합니다.)

이는 성능에 영향을 미칩니다. 이를 확인하기 위해 3차원 객체가 20,000개의 점으로 구성되어 있고, 객체에 두 차례의 연속적인 좌표계 변환을 적용한다고 가정해 보자. 단계별 접근법을 사용하면 $20,000 \times 2$ 회의 벡터-행렬 곱셈이 필요하다. 반면, 결합 행렬 방식을 사용하면 20,000회의 벡터-행렬 곱셈과 두 프레임 변환 행렬을 결합하기 위한 1회의 행렬-행렬 곱셈만 필요합니다. 벡터-행렬 곱셈에서 발생하는 막대한 절감 효과를 고려할 때, 추가되는 1회의 행렬-행렬 곱셈은 매우 저렴한 대가입니다.

다시 말해, 행렬 곱셈은 교환법칙을 따르지 않으므로 AB 와 BA 가 동일한 합성 변환을 나타내지 않을 것으로 예상됩니다. 보다 구체적으로, 행렬을 곱하는 순서는 변환이 적용되는 순서이며, 일반적으로 교환 가능한 과정이 아닙니다.

변환이 적용되는 순서이며, 일반적으로 교환법칙을 따르지 않습니다.

3.4.5 역행렬과 좌표계 변환 행렬

$\mathbf{p}_B$ (좌표계 B에 대한 벡터 $\mathbf{p}$ 의 좌표)가 주어지고, 좌표계 A에서 좌표계 B로의 좌표 변환 행렬 $\mathbf{M}$ 이 주어졌다고 가정하자. 즉, $\mathbf{p}_B = \mathbf{p}_A \mathbf{M}$ 이다. 우리는 $\mathbf{p}_A$ 를 구하고자 한다. 다시 말해, 좌표계 A에서 좌표계 B로의 매핑이 아니라, 좌표계 B에서 좌표계 A로의 매핑을 제공하는 좌표 변환 행렬을 구하고자 한다. 이 행렬을 구하기 위해, $\mathbf{M}$ 이 가역행렬이라고 가정하자(즉, $\mathbf{M}^{-1}$ 존재함). $\mathbf{p}_A$ 는 다음과 같이 구할 수 있다:

$\mathbf{p}_B = \mathbf{p}_A \mathbf{M}$	

$p_B \mathbf{M}^{-1} = p_A \mathbf{M} \mathbf{M}^{-1}$	방정식의 양변에 $\mathbf{M}^{-1}$ 을 곱하면 ¹
$p_B \mathbf{M}^{-1} = p_A \mathbf{I}$	$\mathbf{M} \mathbf{M}^{-1} = \mathbf{I}$, 역행렬의 정의에 의해.
$p_B \mathbf{M}^{-1} = p_A$	$p_A \mathbf{I} = p_A$, 단위 행렬의 정의에 의해.

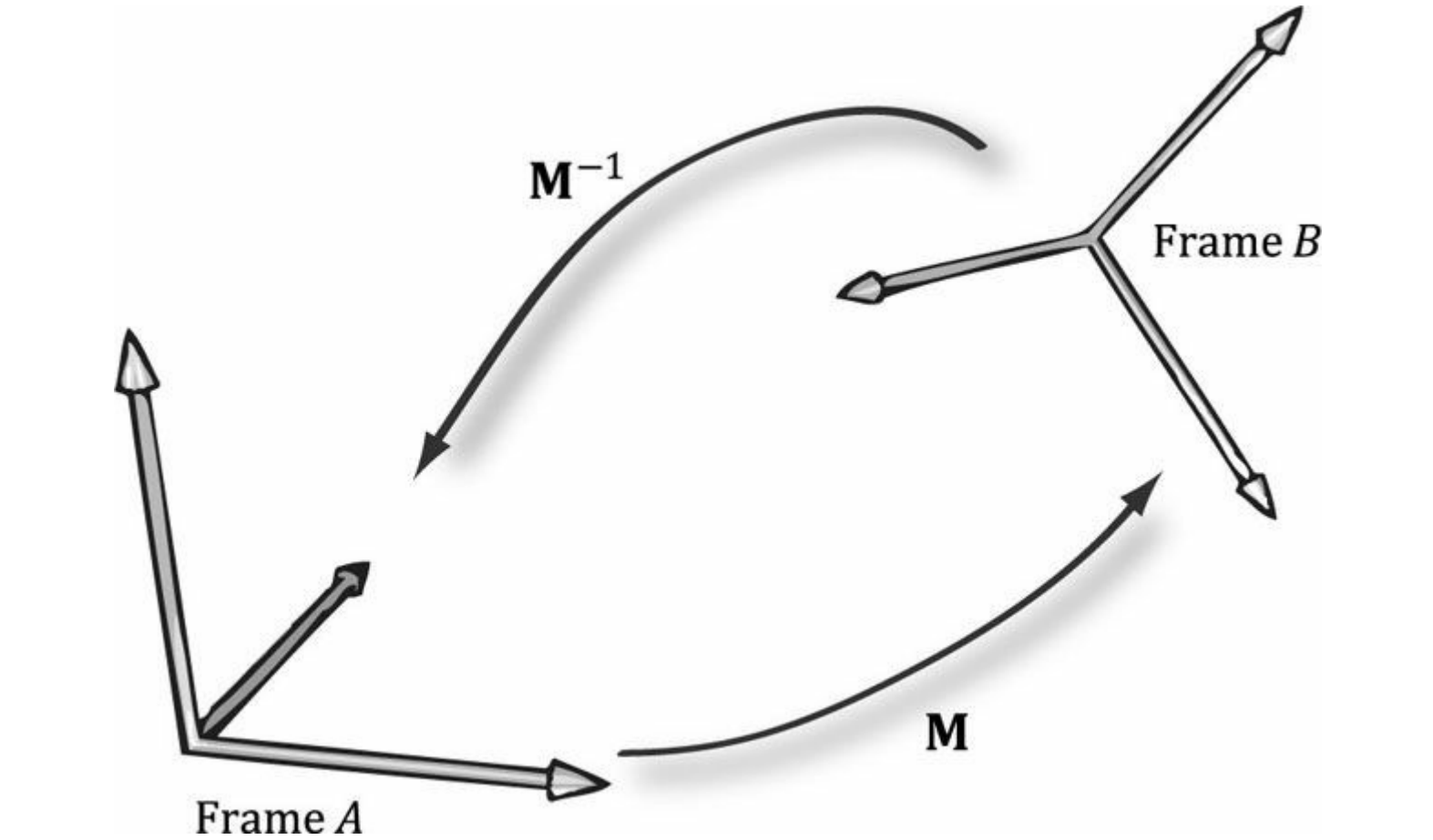


그림 3.13. $\mathbf{M}$ 은 A 를 B 로 매핑하고, $\mathbf{M}^{-1}$ 은 B 를 A 로 매핑한다.

따라서 행렬 $\mathbf{M}^{-1}$ 은 B 에서 A 로의좌표 변환 행렬이다.

그림 3.13은 좌표 변환 행렬과 그 역행렬 간의 관계를 보여준다. 또한 이 책에서 다루는 모든 좌표 변환 이 책에서 수행하는 모든 좌표계 변환은 가역적이므로 역행렬의 존재 여부를 걱정할 필요가 없습니다.

그림 3.14는 행렬 역행렬 성질 $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ 을 좌표 변환 행렬의 관점에서 해석하는 방법을 보여준다.

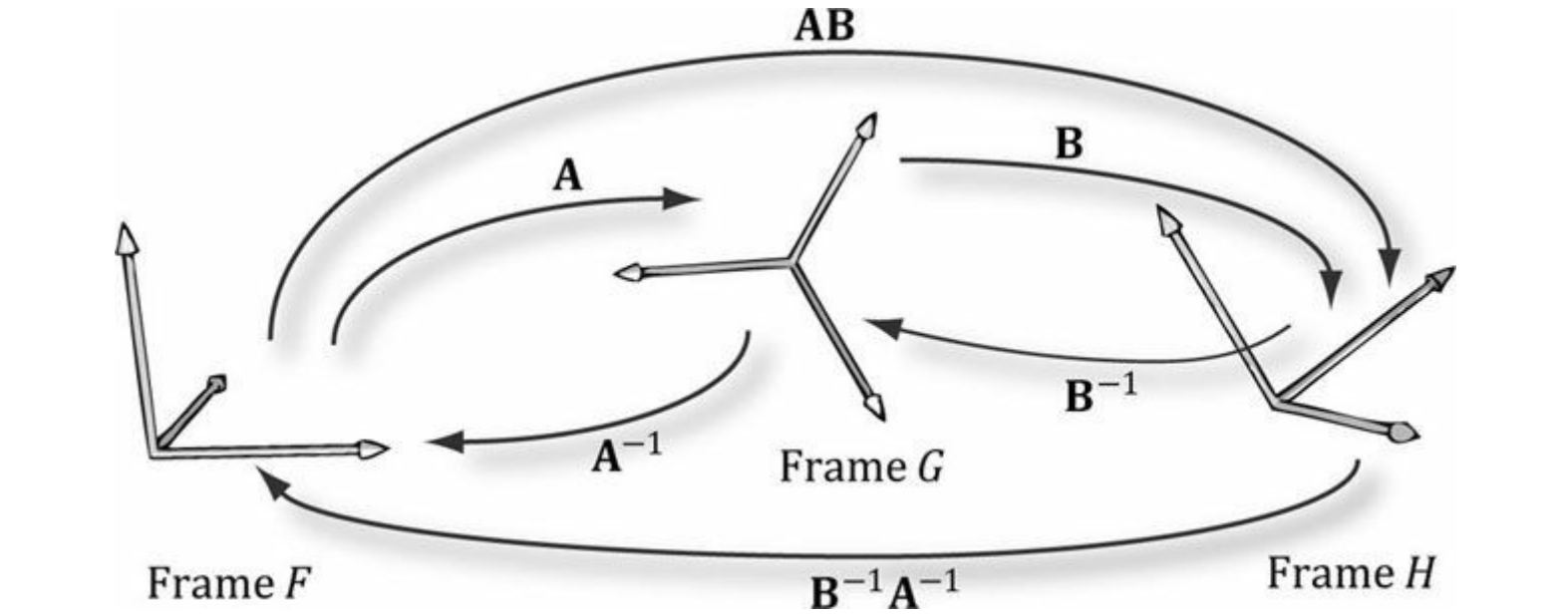


그림 3.14. A는 F에서 G로, B는 G에서 H로, AB는 F에서 직접 H로 매핑한다. B^{-1} 은 H에서 G로, A^{-1} 은 G에서 F로, $B^{-1}A^{-1}$ 은 H에서 직접 F로 매핑한다.

3.5 변환 행렬 대 좌표계 변환 행렬

지금까지 우리는 "능동적" 변환(확대/축소, 회전, 평행 이동)과 좌표계 변환을 구분해 왔습니다. 이 절에서 우리는 수학적으로 이 둘이 동등하며, 능동적 변환을 좌표계 변환으로 해석할 수 있고 그 반대의 경우도 가능함을 살펴볼 것입니다.

그림 3.15는 방정식 3.7의 행(회전 후 평행 이동 아핀 변환 행)과 방정식 3.9의 행(좌표계 변환 행) 사이의 기하학적 유사성을 보여준다.

변환 행렬)과 방정식 3.9(좌표계 변환 행렬)의 행들 사이의 기하학적 유사성을 보여줍니다.

생각해 보면 이는 타당하다. 좌표계 변환에서는 좌표계 간 위치와 방향이 다르기 때문이다.

따라서 한 좌표계에서 다른 좌표계로 변환하기 위한 수학적 공식은 좌표의 회전과 평행 이동을 필요로 하며, 결국 동일한 수학적 형태를 얻게 됩니다. 어느 경우든 동일한 수치를 얻게 되며, 차이는 변환을 해석하는 방식에 있습니다. 어떤 상황에서는 객체 자체는 변하지 않지만 다른 기준 프레임에 상대적으로 표현되기 때문에 좌표 표현만 바뀌는 경우(이 상황은 그림 3.15b에 해당함), 여러 좌표계를 사용하며 시스템 간 변환을 수행하는 것이 더 직관적일 수 있습니다. 반면 기준 프레임을 변경하지 않은 상태에서 좌표계 내 객체를 변환하고자 하는 경우도 있습니다(이 상황은 그림 3.15a에 해당함).

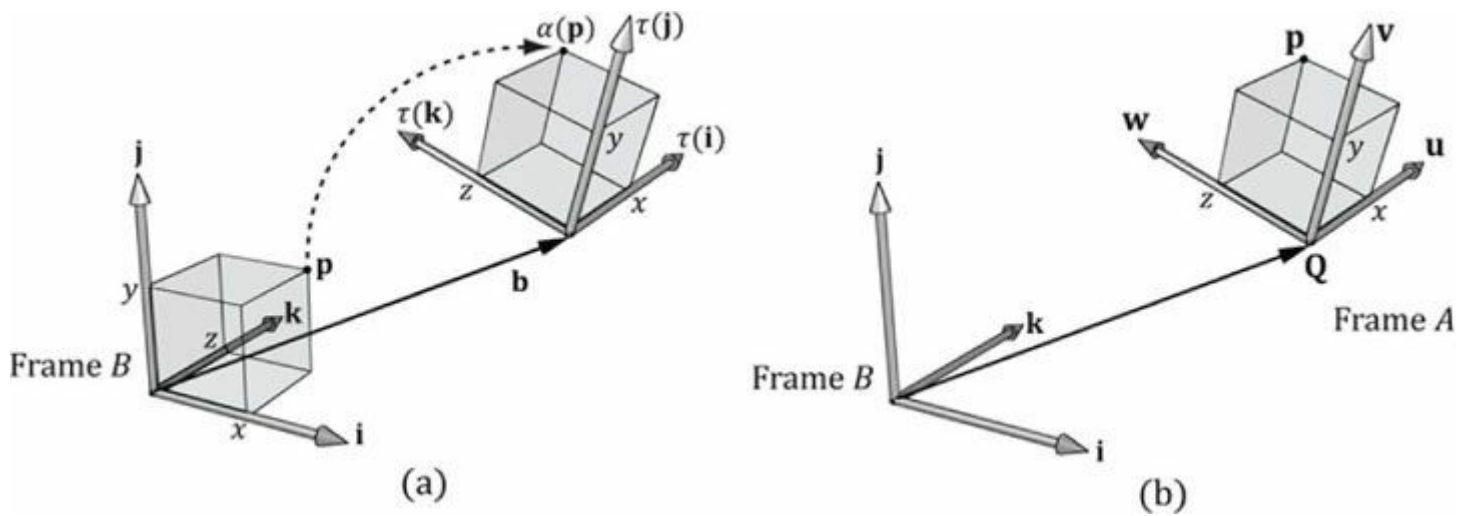


그림 3.15. 여기서 $b = Q$, $\tau(i) = u$, $\tau(j) = v$, $\tau(k) = w$ 임을 알 수 있다. (a) 하나의 좌표계(프레임 B라 함)를 사용하여 정육면체에 아핀 변환을 적용해 프레임 B에 대한 위치와 방향을 변경한다: $\alpha(x, y, z, w) = x\tau(i) + y\tau(j) + z\tau(k) + wb$. (b) 프레임 A와 프레임 B라는 두 좌표계가 존재한다. 프레임 A에 대한 정육면체의 점들은 다음 공식에 의해 프레임 B 좌표로 변환될 수 있다: $p_B = xu_B + yv_B + zw_B + wQ_B$, 여기서 $p_A = (x, y, z, w)$ 이다. 두 경우 모두, 프레임 B에 대한 좌표로 $\alpha(p) = (x', y', z', w) = p_B$ 를 갖습니다.

특히, 이 논의는 활성 변환(확대/축소, 회전, 평행 이동)의 합성을 좌표 변환의 변화로 해석할 수 있음을 보여줍니다. 이는 우리가 종종 세계 공간(제5장) 좌표 변환 행렬을 확대/축소, 회전, 평행 이동 변환의 합성으로 정의할 것이기 때문에 중요합니다.

3.6 XNA 수학 변환 함수

참고를 위해 XNA Math 관련 변환 함수를 요약합니다.

// 스케일링 행렬 생성:

```
XMMATRIX XMMatrixScaling(
    FLOAT Scale X,
    FLOAT Scale Y,
    FLOAT Scale Z);
```

// 스케일링 계수

// 벡터의 구성 요소로부터 스케일링 행렬 생성:

```
XMMATRIX XMMatrixScalingFromVector(
```

FXMVECTOR Scale);

// 스케일링 계수 (s_x, s_y, s_z)

// x축 회전 행렬 생성: Rx XMATRIX XMMatrixRotationX(
 FLOAT Angle);

// 시계 방향으로 θ 각도로 회전

// y축 회전 행렬 생성: Ry XMATRIX XMMatrixRotationY(
 FLOAT Angle);

// 시계 방향 회전 각도 θ

// z축 회전 행렬 생성: Rz XMATRIX XMMatrixRotationZ(
 FLOAT Angle);

// 시계 방향 회전 각도 θ

// 임의의 축 회전 행렬 생성: Rn XMATRIX XMMatrixRotationAxis(
 FXMVECTOR 축,
 FLOAT Angle);

// 회전할 축 n

// 시계 방향 회전 각도 θ

이동 행렬 생성:
XMATRIX XMMatrixTranslation(
 FLOAT OffsetX, FLOAT
 OffsetY,
 FLOAT OffsetZ);

// 변환 계수

벡터의 구성 요소로부터 변환 행렬을 생성합니다.
XMATRIX XMMatrixTranslationFromVector(
 FXMVECTOR 오프셋);

// 평행 이동 계수 (t_x, t_y, t_z)

// 벡터 행렬 곱 vM 계산: XMVECTOR XMVector3Transform(
 FXMVECTOR V, CXMMATRIX M);

// 입력 벡터 v

// 입력 M

// 점 변환 시 $v_w = 1$ 인 벡터 행렬 곱 vM 계산:
XMVECTOR XMVector3TransformCoord(
 FXMVECTOR V,
 CXMMATRIX M);

// 입력 v

// 입력 M

// 벡터 행렬 곱 vM 을 계산합니다. 여기서 $v_w = 0$ 일 때 벡터를 변환합니다.
XMVECTOR XMVector3TransformNormal(
 FXMVECTOR V,
 CXMMATRIX M);

// 입력 v

// 입력 M

마지막 두 함수인 `XMVector3TransformCoord` 와 `XMVector3TransformNormal`의 경우 w 좌표를 명시적으로 설정할 필요가 없습니다  . 이 함수들은 항상

`XMVector3TransformCoord`의 경우 $v_w = 1$, $v_w = 0$ 을, `XMVector3TransformNormal`의 경우 $v_w = 1$, $v_w = 0$ 을 사용합니다.

`XMVector3TransformNormal`의 경우 각각 $v(w) = 1$ 과 $v(w) = 0$ 을 사용합니다.

3.7 요약

1. 기본 변환 행렬—스케일링, 회전, 평행 이동—은 다음과 같이 주어집니다.

$$\mathbf{S} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ b_x & b_y & b_z & 1 \end{bmatrix}$$

$$\mathbf{R}_n = \begin{bmatrix} c + (1-c)x^2 & (1-c)xy + sz & (1-c)xz - sy & 0 \\ (1-c)xy - sz & c + (1-c)y^2 & (1-c)yz + sx & 0 \\ (1-c)xz + sy & (1-c)yz - sx & c + (1-c)z^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. 변환을 표현하기 위해 4×4 행렬을 사용하고, 점과 벡터를 기술하기 위해 1×4 동차 좌표를 사용합니다. 여기서 점은 네 번째 성분을 $w = 1$ 로 설정하여, 벡터는 $w = 0$ 으로 설정하여 나타냅니다. 이러한 방식으로 평행 이동은 점에는 적용되지만 벡터에는 적용되지 않습니다.
3. 행 벡터들이 모두 길이가 1이고 서로 직교하는 행렬을 직교 행렬이라 한다. 직교 행렬은 역행렬이 전치 행렬과 같다는 특별한 성질을 가지며, 이로 인해 역행렬을 쉽고 효율적으로 계산할 수 있다. 모든 회전 행렬은 직교 행렬이다.
4. 행렬 곱셈의 결합 법칙을 통해 여러 변환 행렬을 하나의 변환 행렬로 결합할 수 있으며, 이는 개별 행렬을 순차적으로 적용했을 때의 종합 효과를 나타냅니다.
5. $\mathbf{Q}_B, \mathbf{u}_B, \mathbf{v}_B, \mathbf{w}_B$ 가 각각 기준계 B 에 대한 좌표로 기준계 A 의 원점, x 축, y 축, z 축을 나타낸다고 하자. 벡터/점 $\mathbf{p}$ 가 기준계 A 에 대한 좌표 $\mathbf{p}_A = (x, y, z)$ 를 가질 때, 기준계 B 에 대한 동일한 벡터/점의 좌표는 다음과 같다:

$_B = (x', y', z') = x \mathbf{u}_B + y \mathbf{v}_B + z \mathbf{w}_B$ 벡터(방향과 크기)의 경우

$_B = (x', y', z') = \mathbf{Q}_B + x \mathbf{u}_B + y \mathbf{v}_B + z \mathbf{w}_B$ 위치 벡터(점)에 대하여

이러한 좌표계 변환은 동차 좌표를 사용하여 행렬로 표현할 수 있습니다.

6. 세 개의 좌표계 F, G, H 가 있다고 가정하자. 그리고 A 를 F 에서 G 로의 좌표계 변환 행렬, B 를 G 에서 H 로의 좌표계 변환 행렬이라 하자. 행렬 곱셈을 사용하면, 행렬 $C = AB$ 는 F 에서 H 로의 직접적인 좌표계 변환 행렬로 생각할 수 있다. 즉, 행렬 곱셈은 A 와 B 의 효과를 하나의 순수 행렬로 결합하므로 다음과 같이 쓸 수 있다: $\mathbf{p}_F(AB) = \mathbf{p}_H$.
7. 행렬 M 이 좌표계 A 의 좌표를 좌표계 B 의 좌표로 매핑한다면, 행렬 M^{-1} 은 좌표계 B 의 좌표를 좌표계 A 의 좌표로 매핑한다.
8. 능동적 변환은 좌표 변환의 변화로 해석될 수 있으며, 그 반대의 경우도 마찬가지입니다. 어떤 상황에서는 객체가 불변이지만 다른 기준계에 대해 상대적으로 설명되기 때문에 좌표 표현이 변화하는 경우, 여러 좌표계를 사용하여 객체 자체는 변하지 않으면서 계간 변환을 수행하는 것이 더 직관적일 수 있습니다. 다른 경우에는 기준계를 변경하지 않고 좌표계 내에서 객체를 변환하고자 할 때도 있습니다.

3.8 연습문제

1. $\tau: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ 를 $\tau(x, y, z) = (x + y, x - 3, z)$ 로 정의하자. τ 는 선형 변환인가? 그렇다면 그 표준 행렬 표현을 구하라.
2. $\tau: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ 를 $\tau(x, y, z) = (3x + 4z, 2x - z, x + y + z)$ 로 정의하자. τ 가 선형 변환인가? 그렇다면, 그 표준 행렬 표현을 구하라.
3. 선형 변환 $\tau: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ 이라고 하자. 또한 $\tau(1, 0, 0) = (3, 1, 2)$, $\tau(0, 1, 0) = (2, -1, 3)$, $\tau(0, 0, 1) = (4, 0, 2)$ 라고 하자. $\tau(1, 1, 1)$ 을 구하라.
4. x 축에서 2단위, y 축에서 -3단위로 스케일링하고 z 축은 그대로 유지하는 스케일링 행렬을 구성하세요.
5. $(1, 1, 1)$ 축을 중심으로 30° 회전하는 회전 행렬을 구하세요.
6. x 축으로 4단위, y 축으로 0단위, z 축으로 -9단위 이동하는 평행 이동 행렬을 구성하세요.
7. x 축에서 2단위, y 축에서 -3단위 스케일링하고 z 축은 유지하는 단일 변환 행렬을 생성하십시오.

- 변화 없이 유지한 후, x축에서 4 단위, y축에서 0 단위, z축에서 -9 단위만큼 이동합니다.
- y축을 중심으로 45° 회전한 후 x축에서 -2 단위, y축에서 5 단위, z축에서 1 단위만큼 평행 이동하는 단일 변환 행렬을 구성하세요.
 - 예제 3.2를 다시 수행하되, 이번에는 정사각형을 x축에서 1.5 단위, y축에서 0.75 단위 비율로 확대하고 z축은 변경하지 마십시오. 변환 전후의 기하학적 구조를 그래프로 그려 작업을 확인하십시오.
 - 예제 3.3을 다시 수행하되, 이번에는 정사각형을 y축을 중심으로 시계 방향으로 -45°(즉, 반시계 방향으로 45°) 회전시키십시오. 변환 전후의 기하학적 구조를 그래프로 그려 작업을 확인하십시오.
 - 예제 3.4를 다시 수행하되, 이번에는 정사각형을 x축에서 -5 단위, y축에서 -3.0 단위, z축에서 4.0 단위만큼 평행 이동시킵니다. 변환 전후의 기하학적 구조를 그래프로 그려 작업을 확인하십시오.
 - $R \mathbf{n}(\mathbf{v}) = \cos \theta \mathbf{v} + (1 - \cos \theta)(\mathbf{n} \cdot \mathbf{v})\mathbf{n} + \sin \theta (\mathbf{n} \times \mathbf{v})$ 가 선형 변환임을 증명하고 표준 행렬 표현을 구하라.
 - R_y 의 행들이 직교 정규화됨을 증명하라. 더 많은 계산이 필요한 연습을 원한다면, 독자는 일반 회전 행렬(임의의 축을 중심으로 한 회전 행렬)에 대해서도 이를 수행할 수 있다.
 - 행렬 $\mathbf{M}$ 이 직교 행렬임을 증명하되, $\mathbf{M}^T = \mathbf{M}^{-1}$ 을 만족할 때만 성립함을 증명하라.
 - 다음과 같이 계산하라:

$$[x, y, z, 1] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ b_x & b_y & b_z & 1 \end{bmatrix} \quad \text{and} \quad [x, y, z, 0] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ b_x & b_y & b_z & 1 \end{bmatrix}$$

- 이 평행 이동은 점을 이동시키는가? 이 평행 이동은 벡터를 이동시키는가? 표준 위치에 있는 벡터의 좌표를 평행 이동시키는 것이 왜 의미가 없는가?
- 주어진 스케일링 행렬의 역행렬이 실제로 스케일링 행렬의 역행렬임을 검증하라. 즉, 직접 행렬 곱셈을 수행하여 $\mathbf{S}\mathbf{S}^{-1} = \mathbf{S}^{-1}\mathbf{S} = \mathbf{I}$ 임을 보여라. 마찬가지로, 주어진 평행 이동 행렬의 역행렬이 실제로 평행 이동 행렬의 역행렬임을 검증하라. 즉, $\mathbf{T}\mathbf{T}^{-1} = \mathbf{T}^{-1}\mathbf{T} = \mathbf{I}$ 임을 보여라.
 - 좌표계 A와 B가 있다고 가정하자. $\mathbf{p}_A = (1, -2, 0)$ 과 $\mathbf{q}_A = (1, 2, 0)$ 이 각각 좌표계 A에 상대적인 점과 힘을 나타낸다고 하자. 또한 $\mathbf{Q}_B = (-6, 2, 0)$, $\mathbf{u}_B = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0\right)$, $\mathbf{v}_B = \left(-\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, 0\right)$, $\mathbf{w}_B = (0, 0, 1)$ 이 B 좌표계에 대한 A 좌표계의 좌표를 나타낸다고 하자. A 좌표계를 B 좌표계로 변환하는 좌표 변환 행렬을 구하고, $\mathbf{p}_B = (x, y, z)$ 와 $\mathbf{q}_B = (x, y, z)$ 를 구하라. 답이 타당한지 확인하기 위해 그래프 용지에 그림을 그려 보라.
 - 점 p에 대한 벡터의 선형 조합에 대응하는 것은 아핀 조합이다. $\mathbf{p} = a_1\mathbf{p}_1 + \dots + a_n\mathbf{p}_n$ 여기서 $a_1 + \dots + a_n = 1$ 이며 $\mathbf{p}_1, \dots, \mathbf{p}_n$ 은 점들이다. 스칼라 계수 a_k 는 점 $\mathbf{p}_k$ 가 $\mathbf{p}$ 를 결정하는 데 미치는 영향력을 나타내는 '점' 가중치로 생각할 수 있습니다. 대략적으로 말하면, a_k 가 1에 가까울수록 $\mathbf{p}$ 는 $\mathbf{p}_k$ 에 가까워지고, 음수 a_k 는 $\mathbf{p}$ 를 $\mathbf{p}_k$ 로부터 '밀어냅니다'. (다음 연습문제는 이에 대한 직관을 키우는 데 도움이 될 것입니다.) 이러한 가중치는 중점 좌표(barycentric coordinates)라고도 불립니다. 아핀 조합이 점과 벡터의 합으로 표현될 수 있음을 증명하십시오:

$$\mathbf{p} = \mathbf{p}_1 + a_2 (\mathbf{p}_2 - \mathbf{p}_1) + \dots + a_n (\mathbf{p}_n - \mathbf{p}_1)$$

- 점 $\mathbf{p}_1 = (0, 0, 0)$, $\mathbf{p}_2 = (0, 1, 0)$, $\mathbf{p}_3 = (2, 0, 0)$ 로 정의된 삼각형을 고려하라. 다음 점들을 그래프에 표시하라:

$$\begin{aligned} & .7\mathbf{p}_1 + 0.2\mathbf{p}_2 + 0.1\mathbf{p}_3 \\ & \cdot 0\mathbf{p}_1 + 0.5\mathbf{p}_2 + 0.5\mathbf{p}_3 \quad 0.2\mathbf{p}_1 + 0.6\mathbf{p}_2 + 0.6\mathbf{p}_3 \\ & \cdot 6\mathbf{p}_1 + 0.5\mathbf{p}_2 - 0.1\mathbf{p}_3 \quad 8\mathbf{p}_1 - 0.3\mathbf{p}_2 + 0.5\mathbf{p}_3 \end{aligned}$$

- (a) 부분의 점은 무엇이 특별한가? $\mathbf{p}_2$ 와 점 (1, 0, 0)의 중점 좌표는 $\mathbf{p}_1$, $\mathbf{p}_2$, $\mathbf{p}_3$ 을 기준으로 어떻게 표현되는가?
 $\mathbf{p}_3$ 을 기준으로 한 점 $\mathbf{p}_2$ 와 점 (1, 0, 0)의 중점 좌표는 어떻게 될까요? 중점 좌표 중 하나가 음수일 때 점 $\mathbf{p}$ 가 삼각형에 대해 어디에 위치할지에 대한 추측을 해볼 수 있나요?

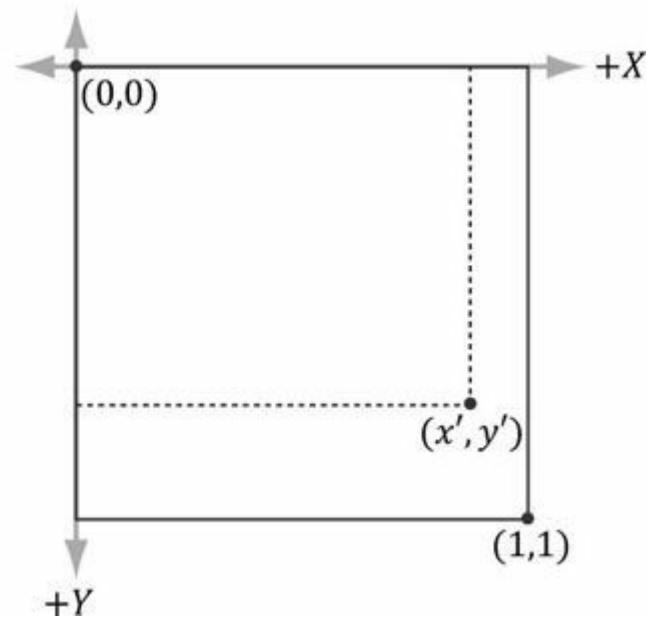
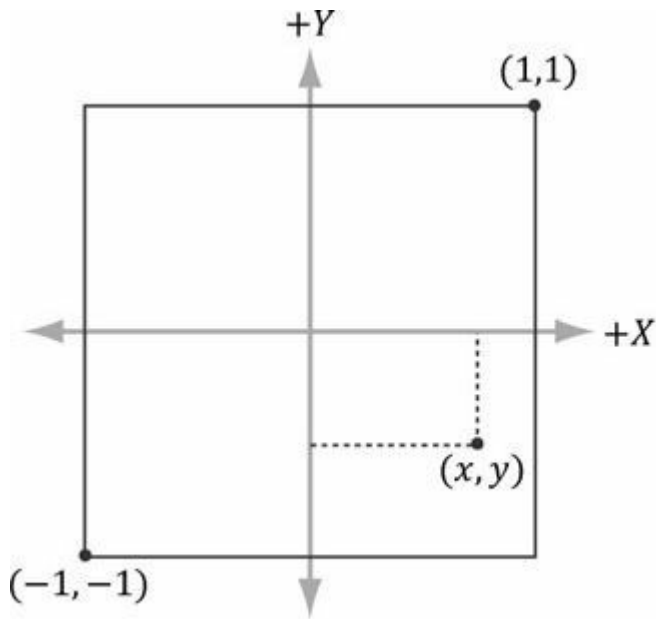


그림 3.16. 좌표계 $A([-1, 1]^2)$ 사각형에서 좌표계 $B([0, 1]^2)$ 사각형, y 축이 좌표계 A 와 반대 방향을 가짐)로의 좌표 변환.

20. 아핀 변환의 정의적 요소 중 하나는 아핀 조합을 보존한다는 것이다. 아핀 변환 $\alpha(\mathbf{u})$ 가 아핀 변환을 보존함을 증명하라. 즉, $\alpha(a_1\mathbf{p}_1 + \dots + a_n\mathbf{p}_n) = a_1\alpha(\mathbf{p}_1) + \dots + a_n\alpha(\mathbf{p}_n)$ 이 성립함을 증명하라. 여기서 $a_1 + \dots + a_n = 1$ 이다.
21. 그림 3.16을 고려하라. 컴퓨터 그래픽스에서 흔히 사용되는 좌표 변환은 프레임 $A([-1, 1]^2)$ 정사각형의 좌표를 프레임 $B([0, 1]^2)$ 정사각형(여기서 y 축은 프레임 A 의 y 축과 반대 방향을 가짐)로 매핑하는 것이다. 프레임 A 에서 프레임 B 로의 좌표 변환이 다음 식으로 주어짐을 증명하라:

$$\begin{bmatrix} x & y & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.5 & 0 & 0 & 0 \\ 0 & -0.5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0.5 & 0.5 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x' & y' & 0 & 1 \end{bmatrix}$$

22. 지난 장에서 행렬의 행렬식이 선형 변환 하에서 직육면체의 부피 변화와 관련이 있다고 언급한 바 있다. 크기 변환 행렬의 행렬식을 구하고 그 결과를 부피 측면에서 해석하라.
23. 사각형을 평행사변형으로 왜곡하는 변환 τ 를 고려하라(예: 그림 3.17 참조). 이는 다음 식으로 주어진다:

$$\tau(x, y) = (3x + y, x + 2y)$$

이 변환의 표준 행렬 표현을 구하고, 변환 행렬의 행렬식이 $\tau(\mathbf{i})$ 와 $\tau(\mathbf{j})$ 가 스팬하는 평행사변형의 면적과 같음을 증명하라.

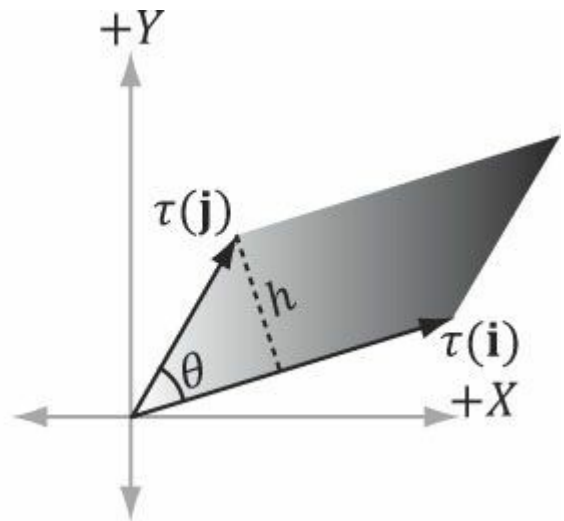
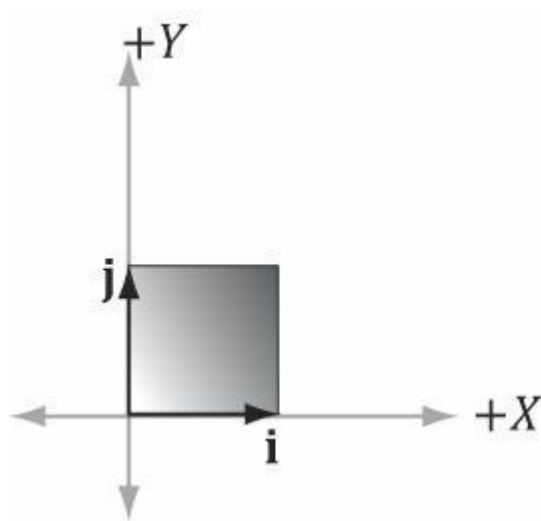


그림 3.17. 정사각형을 평행사변형으로 변환하는 변환.

24. y축 회전 행렬의 행렬식이 1임을 증명하라. 이전 연습 문제를 바탕으로, 그 행렬식이 1이 되는 것이 타당한 이유를 설명하라.

1. 보다 계산 집약적인 연습을 위해, 독자는 일반 회전 행렬(임의의 축을 중심으로 한 회전 행렬)의 행렬식이 1임을 증명할 수 있다.

25. 회전 행렬은 대수적으로 행렬식이 1인 직교 행렬로 특징지을 수 있다. 그림을 재검토하면

- 3.7과 연습문제 24를 함께 보면 이 설명이 타당해집니다. 회전된 기저 벡터 $\tau(i)$, $\tau(j)$, $\tau(k)$ 는 길이가 1이며 서로 직교합니다. 또한 회전은 물체의 크기를 바꾸지 않으므로 행렬의 행렬식은 1이어야 합니다. 두 회전 행렬 $R_1 R_2 = R$ 의 곱이 회전 행렬임을 증명하라. 즉, $RR^T = R^T R = I$ (R 이 직교함을 보임)을 증명하고, $\det R = 1$ 을 증명하라.

26. 회전 행렬에 대해 다음 성질이 성립함을 증명하라:

$$(uR) \cdot (vR) = u \cdot v$$

내적 보존

$$\|uR\| = \|u\|$$

길이의 보존

$$(uR, vR) = \theta(u, v)$$

각도 보존, 여기서 $\theta(x, y)$ 는 x 와 y 사이의 각도를 계산합니다:

$$\theta(x, y) = \cos^{-1} \frac{x \cdot y}{\|x\| \|y\|}$$

이러한 모든 속성이 회전 변환에 대해 타당한 이유를 설명하십시오.

27. 시작점 $p = (0, 0, 0)$ 과 끝점 $q = (0, 0, 1)$ 을 가진 선분을 길이 2의 선분으로 변환하는 스케일링, 회전 및 평행 이동 행렬을 구하십시오. 이 변환된 선분은 벡터 $(1, 1, 1)$ 에 평행하며 시작점은 $(3, 1, 2)$ 입니다.

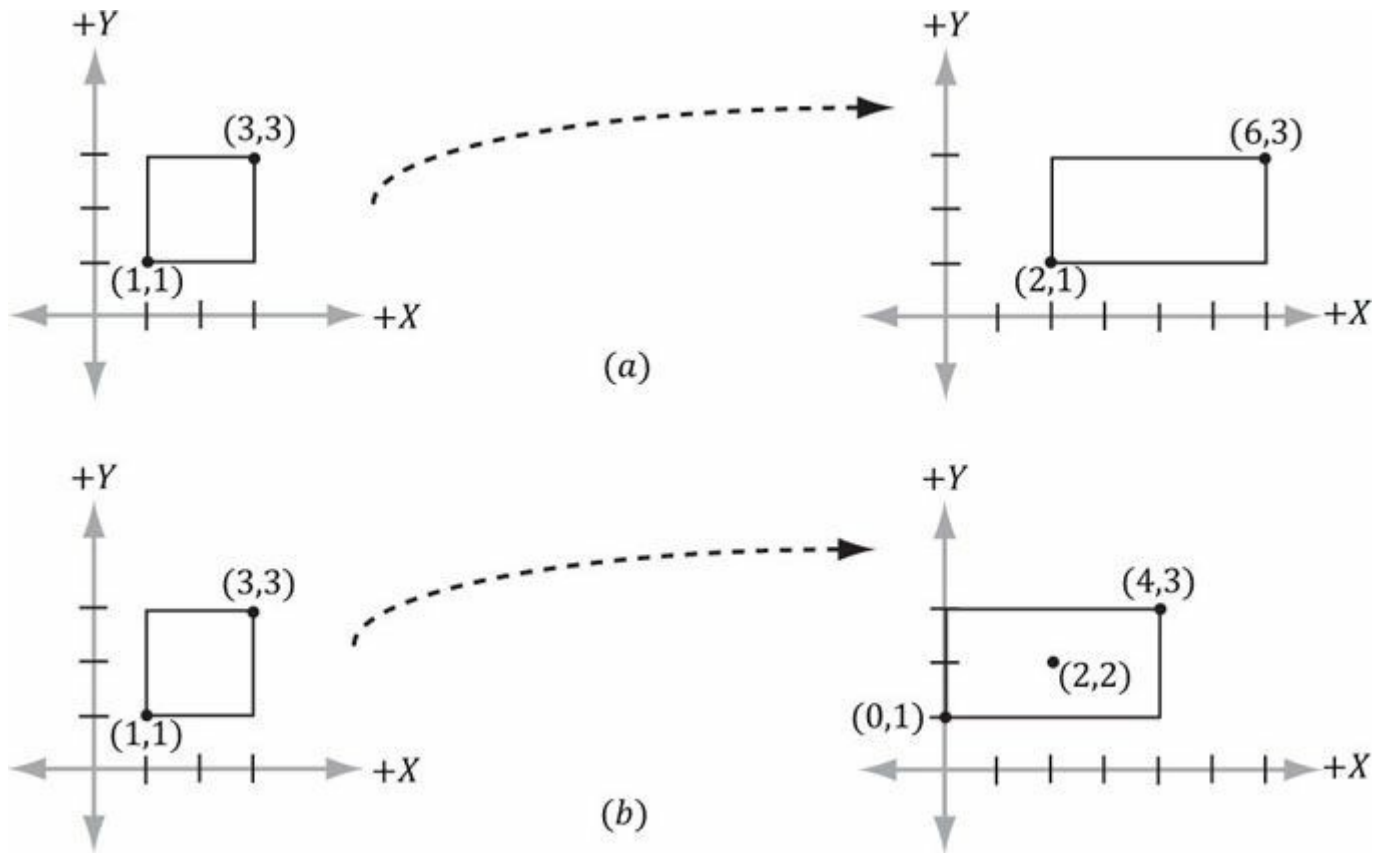


그림 3.18. (a) 원점을 기준으로 x 축에서 2단위 스케일링하면 직사각형이 평행 이동됩니다. (b) 직사각형의 중심을 기준으로 x 축에서 2단위 스케일링하면 평행 이동이 발생하지 않습니다(직사각형은 원래 중심점을 유지합니다).

28. (x, y, z) 위치에 상자가 있다고 가정하자. 정의된 크기 변환은 원점을 기준으로 사용하므로, 이 상자(원점을 중심으로 하지 않음)의 크기를 조정하면 상자가 이동하는 부작용이 발생한다(그림 3.18); 이는 일부 상황에서 바람직하지 않을 수 있다. 상자의 중심점을 기준으로 상자를 스케일링하는 변환을 찾아라. (힌트: 상자 중심을 원점으로 하는 상자 좌표계로 좌표를 변환한 후 상자를 스케일링하고, 다시 원래 좌표계로 변환하라.)

파트 2 *DIRECT3D* 기초

이 부분에서는 이 책의 나머지 부분 전반에 걸쳐 사용되는 기본적인 Direct3D 개념과 기법을 학습합니다. 이러한 기초를 숙지하면 더 흥미로운 애플리케이션 작성으로 나아갈 수 있습니다. 이 부분의 장들에 대한 간략한 설명 다음과 같습니다.

제4장, Direct3D 초기화: 이 장에서는 Direct3D의 개요와 3D 그리기 준비를 위한 초기화 방법을 배웁니다. 표면(Surface), 픽셀 형식(Pixel Format), 페이지 플리핑(Page Flipping), 깊이 버퍼링(Depth Buffering), 멀티샘플링(Multisampling)과 같은 기본 Direct3D 주제들도 소개됩니다. 성능 카운터를 사용해 시간을 측정하는 방법도 배우며, 이를 통해 초당 렌더링되는 프레임 수를 계산합니다. 또한 Direct3D 애플리케이션 디버깅에 대한 몇 가지 팁을 제공합니다. SDK의 프레임워크가 아닌 자체 애플리케이션 프레임워크를 개발하고 사용합니다.

제5장, 렌더링 파이프라인: 이 긴 장에서는 가상 카메라가 보는 것을 바탕으로 세계의 2D 이미지를 생성하는 데 필요한 단계들의 순서인 렌더링 파이프라인에 대한 철저한 소개를 제공합니다. 3D 세계를 정의하고, 가상 카메라를 제어하며, 3D 기하학을 2D 이미지 평면에 투영하는 방법을 배웁니다.

제6장, Direct3D에서의 그리기: 이 장에서는 렌더링 파이프라인 구성, 버텍스 셰이더 및 픽셀 셰이더 정의, 그리기를 위한 기하학적 구조를 렌더링 파이프라인에 제출하는 데 필요한 Direct3D API 인터페이스와 메서드에 중점을 둡니다. 효과 프레임워크도 소개됩니다. 이 장을 마치면 그리드, 상자, 구체 및 원통을 그릴 수 있게 됩니다.

제7장, 조명: 이 장에서는 광원을 생성하고 재질을 통해 빛과 표면 간의 상호작용을 정의하는 방법을 보여줍니다. 특히 정점 셰이더와 픽셀 셰이더를 사용하여 방향광, 점광원, 스포트라이트를 구현하는 방법을 설명합니다.

제8장, 텍스처링: 이 장에서는 2D 이미지 데이터를 3D 프리미티브에 매핑하여 장면의 사실감을 높이는 기술인 텍스처 매핑을 설명합니다. 예를 들어 텍스처 매핑을 사용하면 3D 직사각형에 2D 벽돌 벽 이미지를 적용하여 벽돌 벽을 모델링할 수 있습니다. 다루는 다른 주요 텍스처링 주제에는 텍스처 타일링과 애니메이션 텍스처 변환이 포함됩니다.

제9장, 블렌딩: 블렌딩을 통해 투명도와 같은 다양한 특수 효과를 구현할 수 있습니다. 또한 이미지의 특정 부분이 표시되지 않도록 가리는 데 사용되는 내재적 클립 함수에 대해 논의합니다. 이는 예를 들어 울타리나 문을 구현하는 데 활용될 수 있습니다. 안개 효과 구현 방법도 보여줍니다.

제10장, 스텔실링: 이 장에서는 스텔실처럼 픽셀의 렌더링을 차단할 수 있는 스텔실 버퍼를 설명합니다. 픽셀을 가리는 것은 다양한 상황에서 유용한 도구입니다. 이 장의 개념을 설명하기 위해 스텔실 버퍼를 사용한 평면 반사 및 평면 그림자 구현에 대한 상세한 논의를 포함합니다.

제11장, 지오메트리 셰이더: 이 장에서는 지오메트리 셰이더 프로그래밍 방법을 보여줍니다. 지오메트리 셰이더는 전체 기하학적 프리미티브를 생성하거나 파괴할 수 있다는 점에서 특별합니다. 빌보드, 털 렌더링, 서브디비전, 파티클 시스템 등이 적용 분야입니다. 또한 프리미티브 ID와 텍스처 배열에 대해서도 설명합니다.

제12장, 컴퓨트 셰이더: 컴퓨트 셰이더는 렌더링 파이프라인의 직접적인 일부가 아닌, Direct3D가 제공하는 프로그래밍 가능한 셰이더입니다. 이를 통해 애플리케이션은 그래픽 처리 장치(GPU)를 범용 연산에 활용할 수 있습니다. 예를 들어, 이미징 애플리케이션은 컴퓨트 셰이더로 구현함으로써 GPU를 활용하여 이미지 처리 알고리즘을 가속화할 수 있습니다. 컴퓨트 셰이더는 Direct3D의 일부이므로 Direct3D 리소스에서 읽고 쓰며, 이를 통해 결과를 렌더링 파이프라인에 직접 통합할 수 있습니다. 따라서 일반 목적 계산 외에도 컴퓨트 셰이더는 여전히 3D 렌더링에 적용 가능합니다.

제13장, 테셀레이션 단계: 이 장에서는 렌더링 파이프라인의 테셀레이션 단계를 살펴봅니다. 테셀레이션이란 지오메트리를 더 작은 삼각형으로 세분화한 후 새로 생성된 정점들을 특정 방식으로 오프셋하는 과정을 의미합니다. 삼각형 개수를 늘리는 목적은 메시에 디테일을 추가하기 위함입니다. 본 장의 개념을 설명하기 위해 거리 기반 사각 패치 테셀레이션 방법과 큐빅 베지어 사각 패치 표면 렌더링 방법을 보여줍니다.

제4장 DIRECT3D 초기화

Direct3D 초기화 과정은 기본 Direct3D 유형과 그래픽스 개념에 대한 이해를 요구합니다. 본 장의 첫 번째 섹션은 이러한 요구 사항을 다룹니다. 이후 Direct3D 초기화에 필요한 단계를 상세히 설명합니다. 이어 실시간 그래픽스 애플리케이션에 필요한 정확한 타이밍과 시간 측정을 소개하기 위해 잠시 주제를 전환합니다. 마지막으로, 이 책의 모든 데모 애플리케이션이 따르는 일관된 인터페이스를 제공하기 위해 사용되는 샘플 프레임워크 코드를 살펴봅니다.

목표:

- 1. 3D 하드웨어 프로그래밍에서 Direct3D의 역할을 기본적으로 이해한다.
- 2. COM이 Direct3D에서 수행하는 역할을 이해하기 위해.
- 3. 2D 이미지의 저장 방식, 페이지 플리핑, 깊이 버퍼링, 멀티샘플링과 같은 기본 그래픽스 개념을 학습합니다.
- 4. 고해상도 타이머 판독값을 얻기 위해 성능 카운터 함수를 사용하는 방법을 학습합니다.
- 5. Direct3D를 초기화하는 방법을 알아보기.
- 6. 이 책의 모든 데모에서 사용하는 애플리케이션 프레임워크의 일반적인 구조에 익숙해지기.

4.1 사전 준비 사항

Direct3D 초기화 과정에서는 몇 가지 기본 그래픽 개념과 Direct3D 유형에 익숙해져야 합니다. 다음 섹션에서 주제를 벗어나지 않도록 이 섹션에서 이러한 개념과 유형을 소개합니다.

4.1.1 Direct3D 개요

Direct3D는 3D 하드웨어 가속을 사용하여 3D 세계를 렌더링할 수 있게 해주는 저수준 그래픽 API(응용 프로그램 프로그래밍 인터페이스)입니다. 기본적으로 Direct3D는 그래픽스 하드웨어를 제어하는 소프트웨어 인터페이스를 제공합니다. 예를 들어, 그래픽스 하드웨어에 렌더 타겟(예: 화면)을 지우도록 지시하려면 Direct3D 메서드 `ID3D11DeviceContext::ClearRenderTargetView`를 호출합니다. 애플리케이션과 그래픽스 하드웨어 사이에 Direct3D 계층이 존재한다는 것은

Direct3D 11을 지원하는 장치라면 3D 하드웨어의 세부 사항을 걱정할 필요가 없습니다.

Direct3D 11 지원 그래픽스 디바이스는 몇 가지 예외(예: 멀티샘플링 카운트는 Direct3D 11 하드웨어마다 다를 수 있으므로 여전히 쿼리해야 합니다. 이는 Direct3D 9와 대조적입니다. Direct3D 9에서는 디바이스가 Direct3D 9 기능의 하위 집합만 지원하면 되었기 때문에, Direct3D 9 애플리케이션이 특정 기능을 사용하려면 먼저 사용 가능한 하드웨어가 해당 기능을 지원하는지 확인해야 했습니다. 하드웨어에 구현되지 않은 Direct3D 함수를 호출하면 실패했기 때문입니다. Direct3D 11에서는 Direct3D 11 디바이스가 Direct3D 11 기능 세트를 모두 구현해야 하는 엄격한 요구 사항이 적용되므로 더 이상 디바이스 기능 확인이 필요하지 않습니다.

4.1.2 COM

구성 요소 객체 모델(COM)은 DirectX가 프로그래밍 언어 독립성을 유지하고 하위 호환성을 갖도록 하는 기술입니다. 우리는 일반적으로 COM 객체를 인터페이스라고 부르며, 우리의 목적상 C++ 클래스로 생각하고 사용할 수 있습니다. C++로 DirectX를 프로그래밍할 때 COM의 세부 사항 대부분은 우리에게 숨겨져 있습니다. 우리가 반드시 알아야 할 유일한 점은 COM 인터페이스에 대한 포인터를 특수 함수나 다른 COM 인터페이스의 메서드를 통해 획득한다는 것입니다. C++의 `new` 키워드로 COM 인터페이스를 생성하지 않습니다. 또한 인터페이스 사용을 마친 후에는 삭제(delete) 대신 **Release** 메서드를 호출합니다(모든 COM 인터페이스는 **Release** 메서드를 제공하는 **IUnknown** COM 인터페이스의 기능을 상속받음). COM 객체는 자체 메모리 관리를 수행합니다.

물론 COM에는 훨씬 더 많은 내용이 있지만, DirectX를 효과적으로 사용하기 위해 더 자세한 내용은 필요하지 않습니다.

Note: COM 인터페이스에는 대문자 I가 접두사로 붙습니다. 예를 들어, 2D 텍스처를 나타내는 COM 인터페이스는 `ID3D11Texture2D`라고 합니다.

4.1.3 텍스처와 데이터 리소스 형식

2D 텍스처는 데이터 요소들의 행렬입니다. 2D 텍스처의 한 가지 용도는 2D 이미지 데이터를 저장하는 것으로, 텍스처의 각 요소는 픽셀의 색상을 저장합니다. 그러나 이것이 유일한 용도는 아닙니다. 예를 들어, 노멀 매핑이라는 고급 기법에서는 텍스처의 각 요소가 색상 대신 3D 벡터를 저장합니다. 따라서 텍스처를 이미지 데이터 저장소로 생각하는 것이 일반적이지만, 실제로는 그보다 더 범용적입니다. 1차원 텍스처는 1차원 데이터 요소 배열과 같고, 3차원 텍스처는 3차원 데이터 요소 배열과 같습니다. 후속 장에서 논의될 것처럼, 텍스처는 단순한 데이터 배열 이상입니다. 미프맵 레벨을 가질 수 있으며, GPU는 필터 적용이나 멀티샘플링과 같은 특수 연산을 수행할 수 있습니다. 또한 텍스처는 임의의 데이터 유형을 저장할 수 없습니다. `DXGI_FORMAT` 열거형으로 정의된 특정 데이터 형식만 저장할 수 있습니다. 몇 가지 예시 형식은 다음과 같습니다:

1. `DXGI_FORMAT_R32G32B32_FLOAT`: 각 요소는 세 개의 32비트 부동 소수점 성분을 가집니다.
2. `DXGI_FORMAT_R16G16B16A16_UNORM`: 각 요소는 [0, 1] 범위에 매핑된 네 개의 16비트 구성 요소를 가집니다.
3. `DXGI_FORMAT_R32G32_UINT`: 각 요소는 두 개의 32비트 부호 없는 정수 구성 요소를 가집니다.
4. `DXGI_FORMAT_R8G8B8A8_UNORM`: 각 요소는 [0, 1] 범위에 매핑된 4개의 8비트 부호 없는 부품을 가지고 있습니다.
5. `DXGI_FORMAT_R8G8B8A8_SNORM`: 각 요소는 [-1, 1] 범위에 매핑된 4개의 8비트 부호 있는 구성 요소를 가집니다.
6. `DXGI_FORMAT_R8G8B8A8_SINT`: 각 요소는 [-128, 127] 범위에 매핑된 4개의 8비트 부호 있는 정수 구성 요소를 가집니다.
7. `DXGI_FORMAT_R8G8B8A8_UINT`: 각 요소는 [0, 255] 범위에 매핑된 네 개의 8비트 부호 없는 정수 성분을 가집니다.

R, G, B, A 문자는 각각 빨강(red), 녹색(green), 파랑(blue), 알파(alpha)를 나타냅니다. 색상은 기본 색상인 빨강, 녹색, 파랑의 조합으로 형성됩니다(예: 동일한 빨강과 동일한 녹색은 노랑을 만듭니다). 알파 채널 또는 알파 구성 요소는 일반적으로 투명도를 제어하는 데 사용됩니다. 그러나 앞서 언급했듯이 텍스처는 색상 정보를 저장할 필요가 없습니다. 예를 들어,

`DXGI_FORMAT_R32G32B32_FLOAT`

은 세 개의 부동 소수점 구성 요소를 가지므로 부동 소수점 좌표를 가진 3차원 벡터를 저장할 수 있습니다. 또한 무형식(*typeless*) 형식도 존재합니다. 이 경우 메모리를 예약한 후 텍스처가 파이프라인에 바인딩될 때(C++의 재해석 캐스트(reinterpret cast)와 유사하게) 데이터를 재해석하는 방법을 지정합니다. 예를 들어, 다음 무형식 형식은 4개의 8비트 구성 요소를 가진 요소를 예약하지만 데이터 유형(정수, 부동 소수점, 부호 없는 정수 등)은 지정하지 않습니다:

`DXGI_FORMAT_R8G8B8A8_TYPELESS`

4.1.4 스왑 체인과 페이지 넘기기

애니메이션의 깜빡임을 방지하려면 전체 애니메이션 프레임을 백 버퍼(back buffer)라고 하는 오프 스크린 텍스처에 그려 넣는 것이 가장 좋습니다. 주어진 애니메이션 프레임에 대한 전체 장면이 백 버퍼에 그려지면, 하나의 완전한 프레임으로 화면에 표시됩니다. 이렇게 하면 시청자는 프레임이 그려지는 과정을 보지 않고 완성된 프레임만 보게 됩니다. 이를 구현하기 위해 하드웨어는 두 개의 텍스처 버퍼를 유지합니다. 하나는 *프론트 버퍼*(*front buffer*), 다른 하나는 *백 버퍼*(*back buffer*)라고 합니다. 프론트 버퍼는 모니터에 현재 표시 중인 이미지 데이터를 저장하는 반면, 다음 애니메이션 프레임은 백 버퍼에 그려집니다. 프레임이 백 버퍼에 그려진 후에는 백 버퍼와 프론트 버퍼의 역할이 바뀝니다: 백 버퍼는 다음 애니메이션 프레임의 프론트 버퍼가 되고, 프론트 버퍼는 백 버퍼가 됩니다. 백 버퍼와 프론트 버퍼의 역할을 바꾸는 작업을 *프레젠틱*(*presenting*) *이라고* 합니다. 프레젠틱은 현재 프론트 버퍼의 포인터와 현재 백 버퍼의 포인터를 단순히 교환하기만 하면 되므로 효율적인 작업입니다. [그림 4.1](#)은 이 과정을 보여줍니다.

전방 버퍼와 후방 버퍼는 *스왑 체인*을 구성합니다. Direct3D에서 스왑 체인은 IDXGISwapChain 인터페이스로 표현됩니다. 이 인터페이스는 저장합니다 the 프론트 과 앞 버퍼 텍스처, 및 또한 제공 제공 방법을 제공 크기 조정 버퍼의 크기를 조정 하고

(IDXGISwapChain::ResizeBuffers) 및 프레젠테이션(IDXGISwapChain::Present) 메서드입니다. 이 메서드들에 대해서는 §4.4에서 자세히 설명하겠습니다.

두 개의 버퍼(전면 및 후면)를 사용하는 것을 *더블 버퍼링*이라고 합니다. 두 개 이상의 버퍼를 사용할 수 있으며, 세 개의 버퍼를 사용하는 것은 *트리플 버퍼링*이라고 합니다. 하지만 보통 두 개의 버퍼면 충분합니다.

 *백 버퍼가 텍스처이므로(따라서 요소를 텍셀이라고 불려야 함)에도 불구하고, 백 버퍼의 경우 색상 정보를 저장하기 때문에 요소를 픽셀이라고 부르는 경우가 많습니다. 때로는 색상 정보를 저장하지 않더라도 텍스처의 요소를 픽셀이라고 부르는 경우도 있습니다(예: "노멀 맵의 픽셀").*



Buffer A



Front Buffer Ptr



Buffer B



Back Buffer Ptr

swap



Buffer A



Buffer B



Front Buffer Ptr

Back Buffer Ptr

swap



Buffer A



Front Buffer Ptr



Buffer B



Back Buffer Ptr

그림 4.1. 위에서 아래로, 먼저 현재 백 버퍼 역할을 하는 버퍼 B에 렌더링합니다. 프레임이 완료되면 포인터가 교환되어 버퍼 B가 프론트 버퍼가 되고 버퍼 A가 새로운 백 버퍼가 됩니다. 다음 프레임을 버퍼 A에 렌더링합니다. 프레임이 완료되면 포인터가 교환되어 버퍼 A가 프론트 버퍼가 되고 버퍼 B가 다시 백 버퍼가 됩니다.

4.1.5 깊이 버퍼링

*깊이 버퍼*는 이미지 데이터가 아닌 특정 픽셀에 대한 깊이 정보를 포함하는 텍스처의 예시입니다. 가능한 깊이 값은 0.0부터 1.0까지이며, 0.0은 물체가 시청자에게 가장 가까운 위치를, 1.0은 물체가 시청자에게 가장 먼 위치를 나타냅니다. 깊이 버퍼의 각 요소와 백 버퍼의 각 픽셀 사이에는 일대일 대응 관계가 있습니다(즉, 백 버퍼의 ij 번째 요소는 깊이 버퍼의 ij 번째 요소에 대응합니다). 따라서 백 버퍼의 해상도가 1280×1024 라면 1280×1024 개의 깊이 항목이 존재합니다.



그림 4.2. 서로를 부분적으로 가리는 물체 그룹.

그림 4.2는 일부 객체가 뒤에 있는 객체를 부분적으로 가리는 간단한 장면을 보여줍니다. Direct3D가 어떤 객체의 픽셀이 다른 객체 앞에 있는지 판단하기 위해 *깊이 버퍼링*(*depth buffering*) 또는 *Z-버퍼링*(*z-buffering*)이라는 기법을 사용합니다. 깊이 버퍼링에서는 객체를 그리는 순서가 중요하지 않다는 점을 강조하고자 합니다.

깊이 문제를 해결하기 위해, 장면 속 객체를 가장 먼 것부터 가까운 순서로 그리는 방법을 제안할 수 있습니다. Note: 이렇게 하면 가까운 객체가 먼 객체 위에 그려져 올바른 결과가 렌더링됩니다. 이는 화가가 장면을 그리는 방식과 같습니다. 그러나 이 방법에도 문제가 있습니다—대규모 데이터 세트를 뒤에서 앞으로 정렬하는 작업이 필요하기 때문입니다. 순서와 교차하는 기하학적 구조를 처리합니다. 게다가 그래픽스 하드웨어는 깊이 버퍼링을 무료로 제공합니다.

깊이 버퍼링의 작동 방식을 설명하기 위해 예시를 살펴보자. 그림 4.3을 보라. 이 그림은 관찰자가 보는 볼륨과 그 볼륨의 2D 측면도를 보여준다. 그림에서 우리는 세 개의 서로 다른 픽셀이 뷰 원도우의 픽셀 P 에 렌더링되기 위해 경쟁하는 것을 관찰할 수 있다. (물론 가장 가까운 픽셀이 P 에 렌더링되어야 한다는 것은 알고 있다. 왜냐하면 그 픽셀이 뒤에 있는 픽셀들을 가리기 때문이다. 하지만 컴퓨터는

는 이를 인식하지 못합니다.) 먼저, 렌더링이 시작되기 전에 백 버퍼는 기본 색상(검정색이나 흰색 등)으로 초기화되고, 깊이 버퍼는 기본값(보통 1.0, 픽셀이 가질 수 있는 가장 먼 깊이 값)으로 초기화됩니다. 이제 원통, 구체, 원뿔 순서로 객체가 렌더링된다고 가정합니다. 다음 표는 객체가 그려질 때 픽셀 P 와 그에 대응하는 깊이 값 d 가 어떻게 업데이트되는지 요약한 것입니다; 다른 픽셀들도 유사한 과정을 거칩니다.

Operation	P	d	Description
Clear Operation	Black	1.0	Pixel and corresponding depth entry initialized.
Draw Cylinder	P_3	d_3	Since $d_3 \leq d = 1.0$ the depth test passes and we update the buffers by setting $P = P_3$ and $d = d_3$.
Draw Sphere	P_1	d_1	Since $d_1 \leq d = d_3$ the depth test passes and we update the buffers by setting $P = P_1$ and $d = d_1$.
Draw Cone	P_2	d_2	Since $d_2 > d = d_1$ the depth test fails and we do not update the buffers.

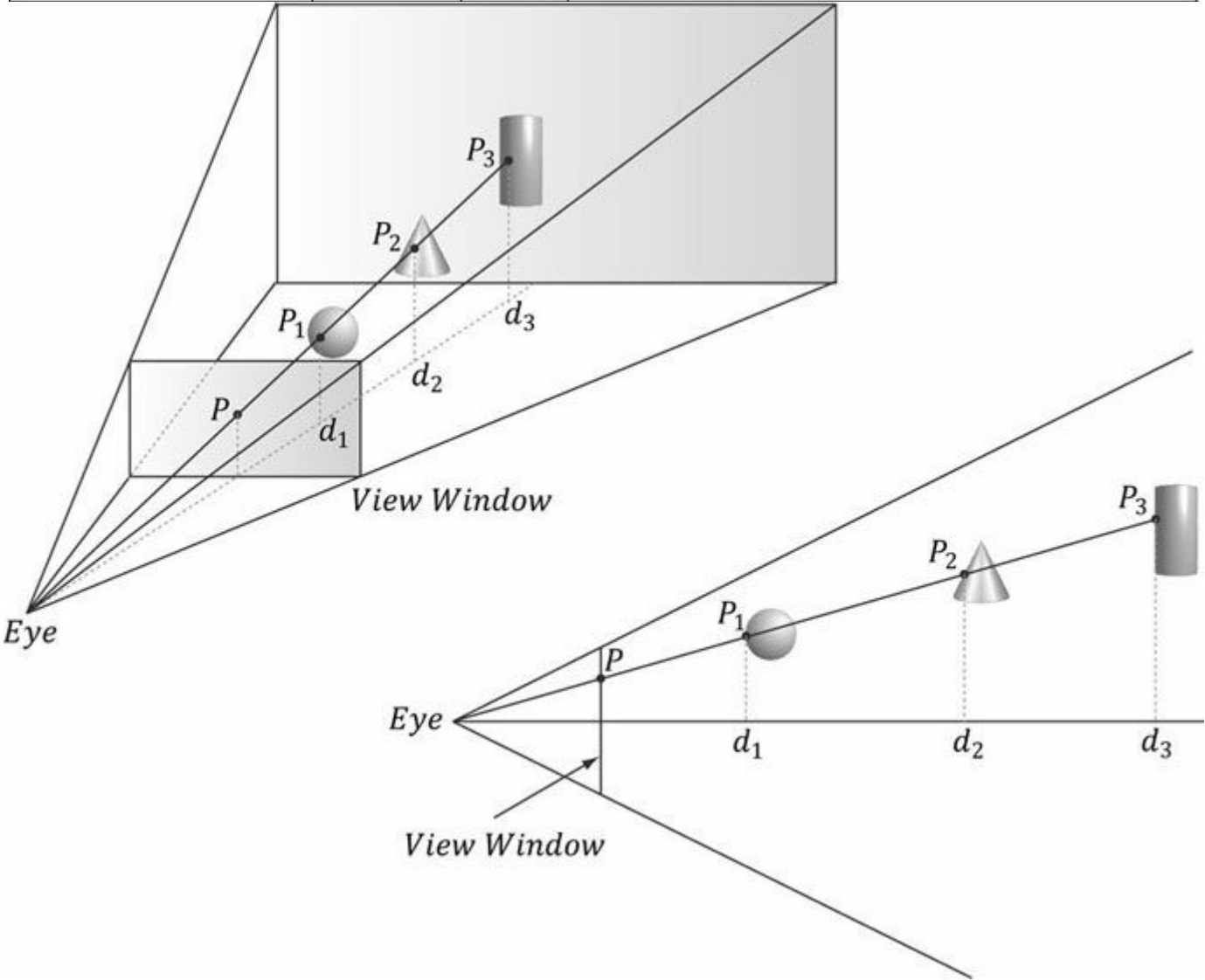


그림 4.3. 뷰 윈도우는 3차원 장면에서 생성한 2차원 이미지(백 버퍼)에 해당한다. 세 개의 서로 다른 픽셀이 픽셀 P 에 투영될 수 있음을 알 수 있습니다. 직관적으로 P_1 이 시청자에게 더 가깝고 다른 두 픽셀을 가리기 때문에 P 에 기록되어야 합니다. 깊이 버퍼 알고리즘은 컴퓨터에서 이를 결정하는 기계적 절차를 제공합니다. 표시된 깊이 값은 시점 중인 3D 장면을 기준으로 한 상대값이지만, 깊이 버퍼에 저장될 때는 실제로 $[0.0, 1.0]$ 범위로 정규화됩니다.

보시다시피, 우리는 깊이 값이 더 작은 픽셀을 발견했을 때만 해당 픽셀과 깊이 버퍼 내 대응 깊이 값을 업데이트합니다. 이렇게 하면 결국 시청자에게 가장 가까운 픽셀이 렌더링됩니다. (아직 확신 이 서지 않는다면, 그리기 순서를 바꿔 이 예제를 다시 살펴보십시오.)

요약하자면, 깊이 버퍼링은 각 픽셀에 대한 깊이 값을 계산하고 깊이 테스트를 수행함으로써 작동합니다. 깊이 테스트는 백 버퍼의 특정 픽셀 위치에 쓰여질 경쟁 픽셀들의 깊이를 비교합니다. 깊이 값이 시청자에게 가장 가까운 픽셀이 승리하며, 해당 픽셀이 백 버퍼에 기록됩니다. 이는 시청자에게 가장 가까운 픽셀 이 뒤에 있는 픽셀들을 가리기 때문에 타당합니다.

깊이 버퍼는 텍스처이므로 특정 데이터 형식으로 생성되어야 합니다. 깊이 버퍼링에 사용되는 형식은 다음과 같습니다:

- 1. **DXGI_FORMAT_D32_FLOAT_S8X24_UINT**: 32비트 부동 소수점 깊이 버퍼를 지정하며, 8비트(부호 없는 정수)는 스텐실 버퍼에 할당되어 [0, 255] 범위로 매핑되고, 24비트는 패딩을 위해 사용되지 않습니다.
- 2. **DXGI_FORMAT_D32_FLOAT**: 32비트 부동 소수점 깊이 버퍼를 지정합니다.
- 3. **DXGI_FORMAT_D24_UNORM_S8_UINT**: [0, 1] 범위에 매핑된 부호 없는 24비트 깊이 버퍼를 지정하며, 8비트(부호 없는 정수)는 [0, 255] 범위에 매핑된 스텐실 버퍼용으로 예약됩니다.
- 4. **DXGI_FORMAT_D16_UNORM**: [0, 1] 범위에 매핑된 부호 없는 16비트 깊이 버퍼를 지정합니다.

애플리케이션은 스텐실 버퍼를 반드시 가질 필요는 없지만, 만약 스텐실 버퍼를 가질 경우 해당 버퍼는 항상 깊이 버퍼에 연결됩니다. 예를 들어, 32비트 형식 DXGI_FORMAT_D24_UNORM_S8_UINT

Note:

은 깊이 버퍼에 24비트를, 스텐실 버퍼에 8비트를 사용합니다. 이러한 이유로 깊이 버퍼는 깊이/스텐실 버퍼라고 부르는 것이 더 적절합니다. 스텐실 버퍼 사용은 고급 주제이므로 제10장에서 설명하겠습니다.

4.1.6 텍스처 리소스 뷰

텍스처는 렌더링 파이프라인의 여러 단계에 바인딩될 수 있습니다. 일반적인 예로는 텍스처를 렌더 타깃(즉, Direct3D가 텍스처에 그리는 경우)과 셰이더 리소스(즉, 셰이더에서 텍스처를 샘플링하는 경우)로 사용하는 것입니다. 이러한 두 가지 목적으로 생성된 텍스처 리소스에는 다음과 같은 바인드 플래그가 지정됩니다:

ID3D11_BIND_RENDER_TARGET|ID3D11_BIND_SHADER_RESOURCE

텍스처가 바인딩될 두 파이프라인 단계를 나타냅니다. 실제로 리소스는 파이프라인 단계에 직접 바인딩되지 않으며, 대신 관련 리소스 뷰가 서로 다른 파이프라인 단계에 바인딩됩니다. 텍스처를 사용하는 각 방식에 대해 Direct3D는 초기화 시점에 해당 텍스처의 리소스 뷰를 생성할 것을 요구합니다. 이는 주로 효율성을 위한 것으로, SDK 문서에서도 지적하듯 "이를 통해 런타임 및 드라이 버에서의 유효성 검사 및 매핑이 뷰 생성 시점에 수행되어 바인딩 시점의 타입 검사를 최소화할 수 있습니다." 따라서 텍스처를 렌더 타깃 및 셰이더 리소스로 사용하는 예시의 경우, 렌더 타깃 뷰 (**ID3D11RenderTargetView**)와 셰이더 리소스 뷰(**ID3D11ShaderResourceView**)라는 두 개의 뷰를 생성해야 합니다. 리소스 뷰는 기본적으로 두 가지 역할을 수행합니다: Direct3D에 리소스의 사용 방식(즉, 파이프라인의 어느 단계에 바인딩할지)을 알리고, 리소스 형식이 생성 시 무형식(typless)으로 지정된 경우 뷰 생성 시 해당 형식을 명시해야 합니다. 따라서 무형식 형식을 사용하면 텍스처 의 요소가 한 파이프라인 단계에서는 부동 소수점 값으로, 다른 단계에서는 정수 값으로 처리될 수 있습니다.

특정 리소스에 대한 뷰를 생성하려면 해당 리소스를 생성할 때 특정 바인드 플래그를 지정해야 합니다. 예를 들어,

리소스가 **ID3D11_BIND_DEPTH_STENCIL** 바인드 플래그(텍스처가 깊이/스텐실 버퍼로 파이프라인에 바인딩될 것임을 나타냄)로 생성되지 않았다면, 해당 리소스에 대한 **ID3D11DepthStencilView**를 생성할 수 없습니다. 시도할 경우 다음과 같은 Direct3D 디버그 오류가 발생합니다:

ID3D11: ERROR: ID3D11Device::CreateDepthStencilView: ID3D11_BIND_DEPTH_STENCIL을 지정하지 않은 리소스에 대해 DepthStencilView를 생성할 수 없습니다.

이 장의 §4.2에서는 렌더 타겟 뷰와 깊이/스텐실 뷰를 생성하는 코드를 살펴볼 기회가 있습니다. 셰이더 리소스 뷰 생성은 8장에서 다룰 예정입니다. 텍스처를 렌더 타겟 및 셰이더 리소스로 사용하는 내용은 이 책의 후반부에 소개됩니다.

2009년 8월 SDK 문서에는 다음과 같이 명시되어 있습니다: "완전한 타입 지정 리소스를 생성하면 해당 리소스가 생성된 형식()으로 제한됩니다. 이를 통해 런타임은 액세스를 최적화할 수 있습니다[...]". 따라서 타입이 지정되지 않은 리소스는 해당 리소스가 제공하는 유연성(여러 뷰를 통해 데이터를 다양한 방식으로 재해석할 수 있는 기능)이 반드시 필요한 경우에만 생성해야 합니다. 그렇지 않으면 완전한 타입 지정 리소스를 생성해야 합니다.

다중 뷰를 통해 데이터를 재해석할 수 있는 능력 이 필요한 경우에만 타입이 없는 리소스를 생성해야 합니다. 그렇지 않으면 완전히 타입화된 리소스를 생성하십시오.

4.1.7 다중 샘플링 이론

모니터의 픽셀은 무한히 작지 않기 때문에 임의의 선을 컴퓨터 모니터에 완벽하게 표현할 수 없습니다. 그림 4.4는 선을 픽셀 행렬로 근사화할 때 발생할 수 있는 "계단 현상"(앨리어싱)을 보여줍니다. 삼각형의 모서리에서도 유사한 앨리어싱 현상이 발생합니다.



그림 4.4. 상단에서는 앨리어싱(픽셀 행렬로 선을 표현하려 할 때 발생하는 계단 현상)을 관찰할 수 있습니다. 하단에서는 앨리어싱이 제거된 선을 볼 수 있는데, 이는 주변 픽셀을 샘플링하여 최종 픽셀 색상을 생성함으로써 더 부드러운 이미지를 만들고 계단 현상을 완화합니다.

모니터 해상도를 높여 픽셀 크기를 축소하면 계단 현상이 거의 눈에 띄지 않을 정도로 문제를 크게 완화할 수 있습니다.

모니터 해상도를 높일 수 없거나 충분하지 않을 경우, 앤티앨리어싱 기법을 적용할 수 있습니다.

슈퍼샘플링은 백 버퍼와 깊이 버퍼를 화면 해상도보다 4배 크게 만드는 방식으로 작동합니다. 이후 3D 장면은 이 더 큰 해상도로 백 버퍼에 렌더링됩니다. 그리고 백 버퍼를 화면에 표시할 때, 백 버퍼는 4픽셀 블록 색상을 평균화하여 평균화된 픽셀 색상을 얻도록 해상도를 낮추는(다운샘플링) 과정을 거칩니다. 결과적으로 슈퍼샘플링은 소프트웨어적으로 해상도를 높이는 방식으로 작동합니다.

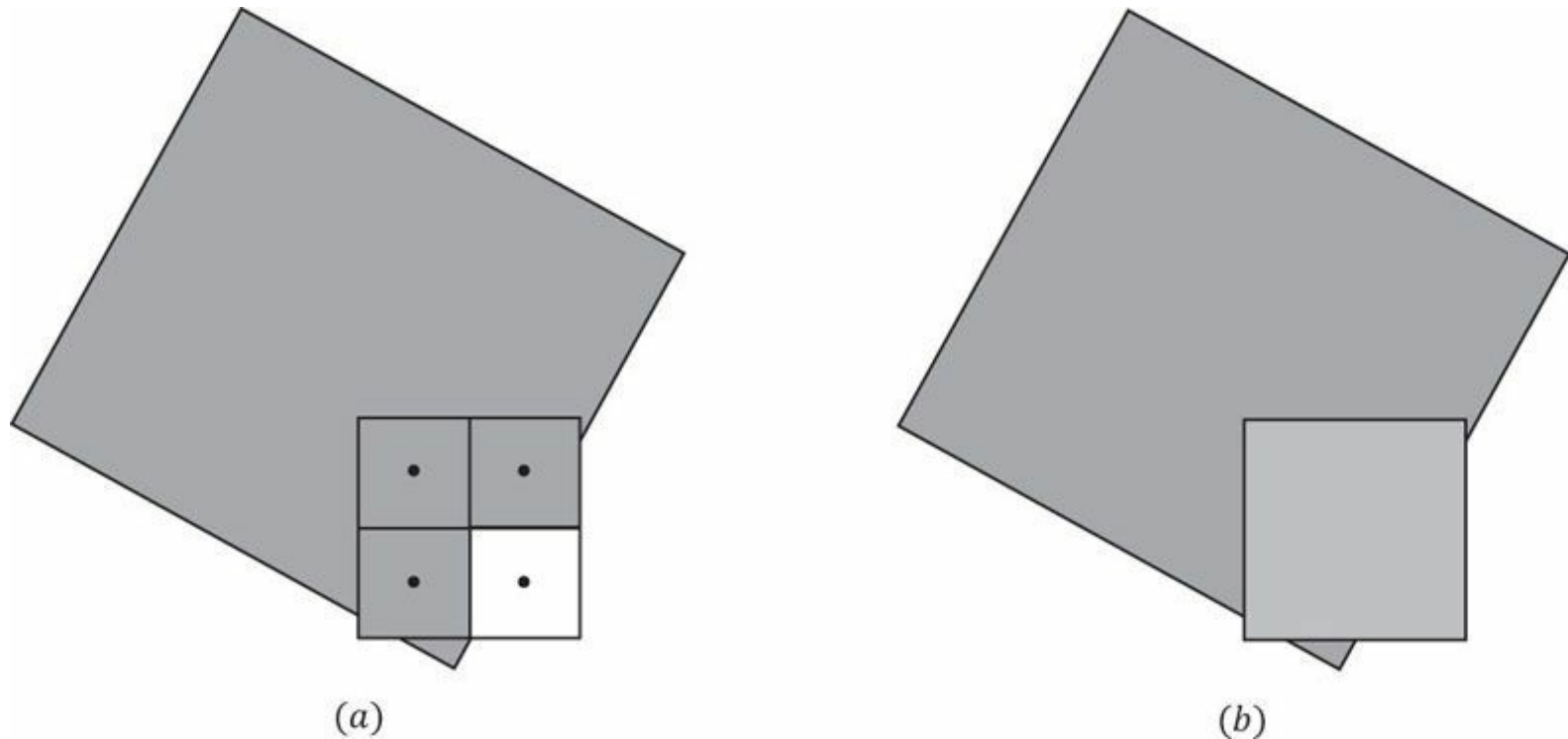


그림 4.5. 다각형의 가장자리를 가로지르는 한 픽셀을 고려합니다. (a) 픽셀 중심에서 평가된 녹색은 다각형에 의해 덮인 세 개의 가시 서브픽셀에 저장됩니다. 4사분면의 서브픽셀은 다각형에 의해 덮이지 않으므로 녹색으로 업데이트되지 않습니다. 이전에 그려진 기하학적 구조나 Clear 작업에서 계산된 이전 색상을 그대로 유지합니다. (b) 해결된 픽셀 색상을 계산하기 위해, 네 개의 서브픽셀(녹색 픽셀 3개와 흰색 픽셀 1개)을 평균하여 다각형 가장자리를 따라 연한 녹색을 얻습니다. 이는 다각형 가장자리의 계단 현상을 희석시켜 더 부드러운 느낌의 이미지를 만들어 냅니다.

슈퍼샘플링은 픽셀 처리량과 메모리 사용량을 4배로 증가시키기 때문에 비용이 많이 듭니다. Direct3D는 멀티샘플링이라는 절충적인 앤티앨리어싱 기법을 지원하는데, 이는 서브픽셀 간에 일부 계산 정보를 공유함으로써 슈퍼샘플링보다 비용을 절감합니다. 4배 멀티샘플링(픽셀당 4개의 서브픽셀)을 사용한다고 가정할 때, 멀티샘플링 역시 화면 해상도보다 4배 큰 백 버퍼와 깊이 버퍼를 사용합니다. 그러나 각 서브픽셀의 이미지 색상을 계산하는 대신,

픽셀 중심에서 한 번만 계산한 후 가시성(심도/스텐실 테스트는 서브픽셀별로 평가됨)과 커버리지(서브픽셀 중심이 다각형 내부 또는 외부에 위치하는가?)에 따라 해당 색상 정보를 서브픽셀과 공유합니다. **그림 4.5**는 예시를 보여줍니다.



Note

슈퍼샘플링과 멀티샘플링의 핵심 차이점을 살펴보자. 슈퍼샘플링에서는 이미지 색상이 서브픽셀 단위로 계산되므로 각 서브픽셀은 서로 다른 색상을 가질 수 있다. 반면 멀티샘플링(**그림 4.5**)에서는

이미지 색상은 픽셀당 한 번씩 계산되며, 해당 색상은 다각형이 덮고 있는 모든 가시 서브픽셀에 복제됩니다. 이미지 색상 계산은 그래픽스 파이프라인에서 가장 비용이 많이 드는 단계 중 하나이므로, 멀티샘플링이 슈퍼샘플링 대비 제공하는 성능 향상 효과는 상당합니다. 반면 슈퍼샘플링은 기술적으로 더 정확하며, 멀티샘플링이 처리하지 못하는 텍스처 밋 셰이더 에일리어싱 현상을 해결합니다.

그림 4.5에서는 균일한 격자 패턴으로 4개의 서브픽셀로 분할된 픽셀을 보여줍니다. 실제 사용되는 패턴(서브픽셀이 배치되는 점)은 하드웨어 벤더마다 다를 수 있습니다. Direct3D는 서브픽셀의 배치를 정의하지 않기 때문입니다. 특정 상황에서는 일부 패턴이 다른 패턴보다 더 나은 성능을 보입니다.

4.1.8 Direct3D의 멀티샘플링

다음 섹션에서는 DXGI_SAMPLE_DESC 구조체를 작성해야 합니다. 이 구조체는 두 개의 멤버를 가지며 다음과 같이 정의됩니다:

```
typedef struct DXGI_SAMPLE_DESC {
    UINT Count;
    UINT Quality;
} DXGI_SAMPLE_DESC, *LPDXGI_SAMPLE_DESC;
```

Count 멤버는 픽셀당 채취할 샘플 수를 지정하며, **Quality** 멤버는 원하는 품질 수준을 지정하는 데 사용됩니다(‘품질 수준’의 의미는 하드웨어 제조사마다 다를 수 있음). 샘플 수가 많거나 품질이 높을수록 렌더링 비용이 증가하므로 품질과 속도 사이의 절충점을 찾아야 합니다. 품질 수준 범위는 텍스처 형식과 픽셀당 샘플 수에 따라 달라집니다. 특정 텍스처 형식과 샘플 수에 대한 품질 수준 수를 조회하려면 다음 메서드를 사용하십시오:

```
HRESULT ID3D11Device::CheckMultisampleQualityLevels(
    DXGI_FORMAT Format, UINT SampleCount, UINT *pNumQualityLevels);
```

이 메서드는 장치에서 형식과 샘플 수 조합을 지원하지 않을 경우 0을 반환합니다. 그렇지 않으면 지정된 조합에 대한 품질 수준 수가 **pNumQualityLevels** 매개변수를 통해 반환됩니다. 텍스처 형식과 샘플 수 조합에 대한 유효한 품질 수준 범위는 0부터 **pNumQualityLevels - 1**까지입니다.

픽셀당 취할 수 있는 최대 샘플 수는 다음으로 정의됩니다:

```
#define D3D11_MAX_MULTISAMPLE_SAMPLE_COUNT ( 32 )
```

그러나 멀티샘플링의 성능 및 메모리 비용을 합리적인 수준으로 유지하기 위해 샘플 수가 4 또는 8인 경우가 일반적입니다. 멀티샘플링을 사용하지 않으려면 샘플 수를 1로, 품질 수준을 0으로 설정하십시오. 모든 Direct3D 11 지원 장치는 모든 렌더 타겟 형식에 대해 4배 멀티샘플링을 지원합니다.



Note

스왑 체인 버퍼와 깊이 버퍼 모두에 대해 **DXGI_SAMPLE_DESC** 구조체를 작성해야 합니다. 백 버퍼와 깊이 버퍼는 동일한 멀티샘플링 설정으로 생성되어야 합니다. 이를 보여주는 샘플 코드는 다음 섹션에서 제공됩니다.

4.1.9 기능 레벨

Direct3D 11은 *기능 레벨* 개념(코드상 **D3D_FEATURE_LEVEL** 열거형으로 표현)을 도입했으며, 이는 대략 Direct3D 버전 9부터 11까지의 다양한 버전에 해당합니다:

```
typedef enum D3D_FEATURE_LEVEL
{
    D3D_FEATURE_LEVEL_9_1      = 0x9100,
    D3D_FEATURE_LEVEL_9_2      = 0x9200,
    D3D_FEATURE_LEVEL_9_3      = 0x9300,
    D3D_FEATURE_LEVEL_10_0     = 0xa000,
```

```
        D3D_FEATURE_LEVEL_10_1 = 0xa100,
        D3D_FEATURE_LEVEL_11_0 = 0xb000,
    } D3D_FEATURE_LEVEL;
```

기능 레벨은 엄격한 기능 집합을 정의합니다(각 기능 레벨이 지원하는 구체적인 기능은 SDK 문서를 참조하십시오). 핵심 개념은 사용자의 하드웨어가 특정 기능 레벨을 지원하지 않을 경우 애플리케이션이 이전 기능 레벨로 대체할 수 있도록 하는 것입니다. 예를 들어, 더 넓은 사용자층을 지원하기 위해 애플리케이션은 Direct3D 11, 10.1, 10, 9.3 레벨 하드웨어를 지원할 수 있습니다. 애플리케이션은 최신에서 가장 오래된 순서로 기능 수준 지원을 확인합니다. 즉, 먼저 Direct3D 11 지원 여부를 확인하고, 다음으로 Direct3D 10.1, Direct3D 10, 마지막으로 Direct3D 9.3을 확인합니다. 이러한 테스트 순서를 용이하게 하기 위해 다음 기능 수준 배열이 사용됩니다(배열의 요소 순서는 기능 수준 테스트 순서를 의미합니다):

```
D3D_FEATURE_LEVEL featureLevels[4] =
{
    D3D_FEATURE_LEVEL_11_0, // 먼저 D3D 11 지원 확인
    D3D_FEATURE_LEVEL_10_1, // 다음으로 D3D 10.1 지원 확인
    D3D_FEATURE_LEVEL_10_0, // 그다음 D3D 10 지원 확인
    D3D_FEATURE_LEVEL_9_3, // 마지막으로 D3D 9.3 지원 확인
};
```

이 배열은 Direct3D 초기화 함수(§4.2.1)에 입력되며, 함수는 배열에서 지원되는 첫 번째 기능 수준을 출력합니다. 예를 들어, Direct3D가 배열에서 지원되는 첫 번째 기능 레벨이 **D3D_FEATURE_LEVEL_10_0**이라고 보고하면, 애플리케이션은 Direct3D 11 및 Direct3D 10.1 기능을 비활성화하고 Direct3D 10 렌더링 경로를 사용할 수 있습니다. 이 책에서는 Direct3D 11에 관한 내용인 만큼 항상 **D3D_FEATURE_LEVEL_11_0** 기능 레벨 지원이 필요합니다. 그러나 실제 애플리케이션은 사용자 기반을 극대화하기 위해 구형 하드웨어 지원에 신경 써야 합니다.

4.2 DIRECT3D 초기화

다음 하위 섹션에서는 Direct3D를 초기화하는 방법을 보여줍니다. Direct3D 초기화 과정은 다음과 같은 단계로 나눌 수 있습니다:

1. **D3D11CreateDevice** 함수를 사용하여 **ID3D11Device** 및 **ID3D11DeviceContext** 인터페이스를 생성합니다.
2. **ID3D11Device::CheckMultisampleQualityLevels** 메서드를 사용하여 4X MSAA 품질 수준 지원을 확인합니다.
3. `swap chain we are going to create by filling out an instance of the DXGI_SWAP_CHAIN_DESC` 구조체.
4. 장치 생성에 사용된 **IDXGIFactory** 인스턴스를 쿼리하고 **IDXGISwapChain** 인스턴스를 생성합니다.
5. 스왑 체인의 백 버퍼에 대한 렌더 타겟 뷰를 생성합니다.
6. 깊이/스텐실 버퍼와 관련 깊이/스텐실 뷰를 생성합니다.
7. 렌더 타겟 뷰와 깊이/스텐실 뷰를 렌더링 파이프라인의 출력 병합 단계에 바인딩하여 Direct3D에서 사용할 수 있도록 합니다.
8. 뷰포트를 설정합니다.

4.2.1 디바이스 및 컨텍스트 생성

Direct3D 초기화는 Direct3D 11 디바이스(**ID3D11Device**)와 컨텍스트(**ID3D11DeviceContext**)를 생성하는 것으로 시작됩니다. 이 두 인터페이스는 주요 Direct3D 인터페이스이며 물리적 그래픽 장치 하드웨어의 소프트웨어 컨트롤러로 생각할 수 있습니다. 즉, 이 인터페이스를 통해 하드웨어와 상호작용하고 작업(예: GPU 메모리에 리소스 할당, 백 버퍼 지우기, 다양한 파이프라인 단계에 리소스 바인딩, 지오메트리 그리기)을 지시할 수 있습니다. 더 구체적으로,

1. **ID3D11Device** 인터페이스는 기능 지원 확인 및 리소스 할당에 사용됩니다.
2. **ID3D11DeviceContext** 인터페이스는 렌더링 상태 설정, 그래픽스 파이프라인에 리소스 바인딩, 렌더링 명령 실행에 사용됩니다.

디바이스와 컨텍스트는 다음 함수로 생성할 수 있습니다:

```
HRESULT D3D11CreateDevice( IDXGIAdapter
    *pAdapter, D3D_DRIVER_TYPE DriverType,
    HMODULE Software,
    UINT Flags,
```



```
CONST D3D_FEATURE_LEVEL *pFeatureLevels, UINT FeatureLevels,  
UINT SDKVersion, ID3D11Device **ppDevice,  
D3D_FEATURE_LEVEL *pFeatureLevel, ID3D11DeviceContext **ppImmediateContext  
);
```

1. **pAdapter**: 생성할 디바이스가 나타나도록 할 디스플레이 어댑터를 지정합니다. 이 매개변수에 null을 지정하면 기본 디스플레이 어댑터가 사용됩니다. 이 책의 샘플 프로그램에서는 항상 기본 어댑터를 사용합니다. 장별 연습 문제에서는 다른 디스플레이 어댑터 사용을 탐구하도록 합니다.

2. **DriverType**: 일반적으로 렌더링에 3D 하드웨어 가속을 사용하려면 이 매개변수에 항상 **D3D_DRIVER_TYPE_HARDWARE**를 지정합니다. 그러나 다음과 같은 추가 옵션도 있습니다:

D3D_DRIVER_TYPE_REFERENCE: 소위 *참조 장치*(reference device)를 생성합니다. 참조 장치는 정확성을 목표로 하는 Direct3D의 소프트웨어 구현체입니다(소프트웨어 구현이므로 매우 느립니다). 참조 장치는 DirectX SDK와 함께 설치되며 개발자만 사용할 수 있습니다. 출시용 애플리케이션에는 사용해서는 안 됩니다. 참조 장치를 사용하는 두 가지 이유는 다음과 같습니다.

하드웨어가 지원하지 않는 코드를 테스트하기 위함입니다. 예를 들어, Direct3D 11을 지원하는 그래픽 카드가 없을 때 Direct3D 11 코드를 테스트하는 경우입니다.

드라이버 버그를 테스트하기 위함입니다. 참조 장치에서는 정상 작동하지만 실제 하드웨어에서는 작동하지 않는 코드가 있다면 하드웨어 드라이버에 버그가 있을 가능성이 높습니다.

D3D_DRIVER_TYPE_SOFTWARE: 3D 하드웨어를 에뮬레이트하는 소프트웨어 드라이버를 생성합니다. 소프트웨어 드라이버를 사용하려면 자체 드라이버를 빌드하거나 타사 소프트웨어 드라이버를 사용해야 합니다. 다음에 설명할 WARP 드라이버를 제외하고 Direct3D는 소프트웨어 드라이버를 제공하지 않습니다.

D3D_DRIVER_TYPE_WARP: 고성능 Direct3D 10.1 소프트웨어 드라이버를 생성합니다. WARP는 Windows Advanced Rasterization Platform의 약자입니다. Direct3D 11을 지원하지 않으므로 우리는 이에 관심이 없습니다.

3. **Software**: 소프트웨어 드라이버를 제공하기 위해 사용됩니다. 렌더링에 하드웨어를 사용하므로 항상 null을 지정합니다. 또한 소프트웨어 드라이버를 사용하려면 반드시 사용 가능한 드라이버가 있어야 합니다.

4. **플래그**: 선택적 장치 생성 플래그(비트 단위로 OR 연산 가능). 두 가지 일반적인 플래그는 다음과 같습니다.

D3D11_CREATE_DEVICE_DEBUG: 디버그 모드 빌드 시 디버그 계층을 활성화하려면 이 플래그를 설정해야 합니다. 디버그 플래그가 지정되면 Direct3D는 디버그 메시지를 VC++ 출력 창으로 전송합니다. [그림 4.6](#)은 출력될 수 있는 오류 메시지 예시를 보여줍니다.

D3D11_CREATE_DEVICE_SINGLETHREADED: Direct3D가 다중 스레드에서 호출되지 않을 것이라고 보장할 수 있는 경우 성능을 향상시킵니다. 이 플래그가 활성화되면 **ID3D11Device::CreateDeferredContext** 메서드가 실패합니다(본 절 끝의 "참고" 참조).

5. **pFeatureLevels**: **D3D_FEATURE_LEVEL** 요소의 배열로, 순서는 가능 수준 지원 테스트 순서를 나타냅니다(§4.1.9 참조). 이 매개변수에 null을 지정하면 지원되는 최대 가능 수준을 선택하도록 지시합니다. 이 책에서는 Direct3D 11만을 대상으로 하므로, 프레임워크에서 이 값이 **D3D_FEATURE_LEVEL_11_0**(즉, Direct3D 11 지원)인지 확인합니다.

6. **FeatureLevels**: **pFeatureLevels** 배열에 포함된 **D3D_FEATURE_LEVEL**의 개수입니다. 이전 매개변수 **pFeatureLevels**에 null을 지정한 경우 0을 지정하십시오.

7. **SDKVersion**: 항상 **D3D11_SDK_VERSION**을 지정하십시오.

8. **ppDevice**: 생성된 장치를 반환합니다.

9. **pFeatureLevel**: **pFeatureLevels** 배열에서 지원되는 첫 번째 가능 수준을 반환합니다(또는 **pFeatureLevels**가 null인 경우 지원되는 가장 높은 가능 수준을 반환합니다). **pFeatureLevels**가 null인 경우 지원되는 최대 피처 레벨).

10. **ppImmediateContext**: 생성된 디바이스 컨텍스트를 반환합니다.

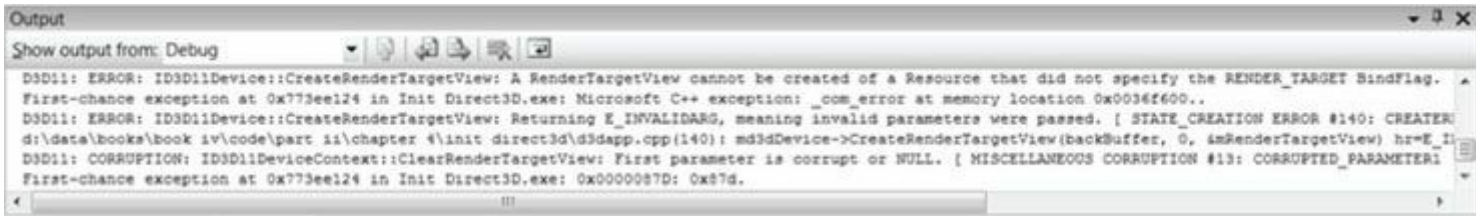


그림 4.6. Direct3D 11 디버그 출력의 예.

이 함수의 호출 예는 다음과 같습니다.

```
UINT createDeviceFlags = 0;

#ifdef(DEBUG) || defined(_DEBUG)
    createDeviceFlags |= D3D11_CREATE_DEVICE_DEBUG; #endif

D3D_FEATURE_LEVEL featureLevel; ID3D11Device* md3dDevice;
ID3D11DeviceContext* md3dImmediateContext; HRESULT hr =
D3D11CreateDevice(
    0,          // 기본 어댑터
    D3D_DRIVER_TYPE_HARDWARE,
    0,          // 소프트웨어 장치 생성 금지
    createDeviceFlags,
    0, 0,       // 기본 기능 레벨 배열
    D3D11_SDK_VERSION,
    & md3dDevice, &
    featureLevel,
    & md3dImmediateContext);

if( FAILED(hr) )
{
    MessageBox(0, L"D3D11CreateDevice 실패.", 0, 0); return false;
}

if( featureLevel != D3D_FEATURE_LEVEL_11_0 )
{
    MessageBox(0, L"Direct3D 기능 레벨 11이 지원되지 않습니다.", 0, 0); return false;
}
```

디바이스 컨텍스트 포인터를 즉시 컨텍스트라고 부르는 점에 유의하십시오:

```
ID3D11DeviceContext* md3dImmediateContext;
```

지연 컨텍스트(ID3D11Device::CreateDeferredContext) 라는 것도 있습니다. 이는 Direct3D 11의 멀티스레딩 지원의 일부입니다. 멀티스레딩은 고급 주제로 간주되어 이 책에서는 다루지 않지만, 기본 개념은 다음과 같습니다.

Note:

1. 즉시 컨텍스트는 메인 렌더링 스레드에 배치합니다.
2. 추가적인 디퍼드 컨텍스트는 별도의 작업 스레드에 배치합니다.

- (a) 각 작업자 스레드는 그래픽 명령을 명령 목록(ID3D11CommandList)에 기록할 수 있습니다.
- (b) 각 작업자 스레드의 명령어 목록은 메인 렌더링 스레드에서 실행될 수 있습니다.

복잡한 렌더링 그래프의 경우 명령어 목록을 조립하는 데 시간이 걸릴 수 있으며, 멀티코어 시스템에서 명령어 목록을 병렬로 조립할 수 있다면 유리합니다.

4.2.2 4X MSAA 품질 지원 확인

이제 장치를 생성했으므로 4X MSAA의 품질 수준 지원 여부를 확인할 수 있습니다. 모든 Direct3D 11 지원 장치는 모든 렌더 타겟 형식에서 4X MSAA를 지원하지만(다만 지원되는 품질 수준은 다를 수 있음), 이를 상기하시기 바랍니다.

```
UINT m4xMsaaQuality;
HR(md3dDevice->CheckMultisampleQualityLevels(DXGI_FORMAT_R8G8B8A8_UNORM, 4,
    &m4xMsaaQuality));
assert(m4xMsaaQuality > 0 );
```

4X MSAA는 항상 지원되므로 반환된 품질은 항상 0보다 커야 합니다. 따라서 우리는 이 조건이 충족됨을 **확인합니다**.

4.2.3 스왑 체인 설명

초기화 과정의 다음 단계는 스왑 체인을 생성하는 것입니다. 이를 위해 먼저 생성할 스왑 체인의 특성을 설명하는 DXGI_SWAP_CHAIN_DESC 구조체의 인스턴스를 작성합니다. 이 구조체는 다음과 같이 정의됩니다:

```
typedef struct DXGI_SWAP_CHAIN_DESC { DXGI_MODE_DESC
    BufferDesc; DXGI_SAMPLE_DESC SampleDesc;
    DXGI_USAGE BufferUsage;
    UINT BufferCount; HWND
    OutputWindow; BOOL Windowed;
    DXGI_SWAP_EFFECT 스왑 효과; UINT 플래그;
} DXGI_SWAP_CHAIN_DESC;
```

DXGI_MODE_DESC 유형은 다음과 같이 정의된 또 다른 구조체입니다:

```
typedef struct DXGI_MODE_DESC
{
    UINT Width; // 원하는 백 버퍼 너비 UINT Height; // 원하는 백 버퍼
    높이
    DXGI_RATIONAL RefreshRate; // 디스플레이 모드 재생률 DXGI_FORMAT Format; // 백 버퍼 픽셀 형식
    DXGI_MODE_SCANLINE_ORDER ScanlineOrdering; // 디스플레이 스캔라인 모드 DXGI_MODE_SCALING Scaling;
    // 디스플레이 스케일링 모드
} DXGI_MODE_DESC;
```

 다음 데이터 멤버 설명에서는 초보자에게 가장 중요한 공통 플래그와 옵션만 다룹니다. 추가 플래그 및 옵션에 대한 설명은 SDK 문서를 참조하십시오.

- 1. BufferDesc:** 이 구조체는 생성하려는 백 버퍼의 속성을 설명합니다. 주요 속성은 너비, 높이 및 픽셀 형식입니다. 다른 멤버에 대한 자세한 내용은 SDK 문서를 참조하십시오.
- 2. SampleDesc:** 멀티샘플 수와 품질 수준; §4.1.8을 참조하십시오.
- 3. BufferUsage:** 백 버퍼에 렌더링할 예정이므로 DXGI_USAGE_RENDER_TARGET_OUTPUT을 지정합니다(즉, 렌더 타겟으로 사용).
- 4. BufferCount:** 스왑 체인에서 사용할 백 버퍼의 개수입니다. 일반적으로 더블 버퍼링에는 하나의 백 버퍼만 사용하지만, 트리플 버퍼링을 위해 두 개를 사용할 수도 있습니다.
- 5. 출력 창:** 렌더링 대상 창에 대한 핸들.
- 6. Windowed:** 창 모드로 실행하려면 true를, 전체 화면 모드로는 false를 지정합니다.
- 7. SwapEffect:** 디스플레이 드라이버가 가장 효율적인 프레젠테이션 방법을 선택하도록 하려면 DXGI_SWAP_EFFECT_DISCARD를 지정합니다.
- 8. Flags:** 선택적 플래그입니다. DXGI_SWAP_CHAIN_FLAG_ALLOW_MODE_SWITCH를 지정하면 애플리케이션이 전체 화면 모드로 전환될 때 현재 백 버퍼 설정에 가장 잘 맞는 디스플레이 모드를 선택합니다. 이 플래그를 지정하지 않으면 애플리케이션이 전체 화면 모드로 전환될 때 현재 데스크톱 디스플레이 모드를 사용합니다. 샘플 프레임워크에서는 이 플래그를 지정하지 않습니다. 전체 화면 모드에서 현재 데스크톱 디스플레이 모드를 사용하는 것이 데모에 적합하기 때문입니다(대부분의 데스크톱 디스플레이는 모니터의 최적 해상도로 설정됨).

다음 코드는 샘플 프레임워크에서 DXGI_SWAP_CHAIN_DESC 구조체를 어떻게 채우는지 보여줍니다:

```
DXGI_SWAP_CHAIN_DESC sd;
sd.BufferDesc.Width = mClientWidth; // 창 클라이언트 영역 크기를 사용 sd.BufferDesc.Height =
mClientHeight; sd.BufferDesc.RefreshRate.Numerator = 60;
sd.BufferDesc.RefreshRate.Denominator = 1; sd.BufferDesc.Format =
DXGI_FORMAT_R8G8B8A8_UNORM;
sd.BufferDesc.ScanlineOrdering = DXGI_MODE_SCANLINE_ORDER_UNSPECIFIED; sd.BufferDesc.Scaling =
DXGI_MODE_SCALING_UNSPECIFIED;

// 4배 MSAA 사용?
```

```

if( mEnable4xMsaa )
{
    sd.SampleDesc.Count = 4;

    // m4xMsaaQuality는 CheckMultisampleQualityLevels()를 통해 반환됩니다. sd.SampleDesc.Quality = m4xMsaaQuality - 1;
}
// MSAA 미사용
시
{
    sd.SampleDesc.Count = 1;
    sd.SampleDesc.Quality = 0;
}
sd.BufferUsage          = DXGI_USAGE_RENDER_TARGET_OUTPUT; sd.BufferCount  = 1;
sd.OutputWindow         = mhMainWnd;
sd.Windowed             = true;
sd.SwapEffect           = DXGI_SWAP_EFFECT_DISCARD; sd.Flags
                        = 0;

```

 런타임에 멀티샘플링 설정을 변경하려면 스왑 체인을 파괴하고 재창조해야 합니다.

백버퍼 에는 DXGI_FORMAT_R8G8B8A8_UNORM 형식(8비트 레드, 그린, 블루, 알파)을 사용합니다. 모니터는 일반적으로 24비트 이상의 색상을 지원하지 않으므로 추가 정밀도는 낭비되기 때문입니다. 알파 채널의 추가 8비트는 모니터에 출력되지 않지만, 백 버퍼에 추가 8비트를 보유하면 특정 특수 효과에 활용될 수 있습니다.

4.2.4 스왑 체인 생성

A 스왑 체인 인터페이스 (IDXGISwapChain) 는 생성됩니다 생성됩니다 IDXGIFactory 인스턴스를 생성된 IDXGIFactory::CreateSwapChain 메서드를 사용하여

IDXGIFactory::CreateSwapChain 메서드를 사용하여

```

HRESULT IDXGIFactory::CreateSwapChain(
    IUnknown *pDevice, // ID3D11Device 포인터.
    DXGI_SWAP_CHAIN_DESC *pDesc, // 스왑 체인 설명에 대한 포인터.
    IDXGISwapChain **ppSwapChain); // 생성된 스왑 체인 인터페이스를 반환합니다.

```

CreateDXGIFactory 함수를 사용하면 IDXGIFactory 인스턴스 포인터를 얻을 수 있습니다(dxgi.lib 링크 필요). 그러나 이 방법으로 IDXGIFactory 인스턴스를 획득한 후 IDXGIFactory::CreateSwapChain을 호출하면 다음과 같은 오류가 발생합니다:

DXGI 경고: IDXGIFactory::CreateSwapChain: 이 함수는 다른 IDXGIFactory의 장치로 호출되고 있습니다.

필요한 수정 사항은 해당 장치를 생성하는 데 사용된 IDXGIFactory 인스턴스를 사용하는 것입니다. 이 인스턴스를 얻으려면 다음 일련의 COM 쿼리를 수행해야 합니다(이는 IDXGIFactory 문서에 설명되어 있음):

```

IDXGIDevice* dxgiDevice = 0;
HR(md3dDevice->QueryInterface(__uuidof(IDXGIDevice),
    (void**)&dxgiDevice));

IDXGIAdapter* dxgiAdapter = 0;
HR(dxgiDevice->GetParent(__uuidof(IDXGIAdapter),
    (void**)&dxgiAdapter));

// 마침내 IDXGIFactory 인터페이스를 얻었습니다. IDXGIFactory*
dxgiFactory = 0;
HR(dxgiAdapter->GetParent(__uuidof(IDXGIFactory),

```

```
(void**))&dxgiFactory));
```

```
// 이제 스왑 체인을 생성합니다. IDXGISwapChain* mSwapChain;
```

```
HR(dxgiFactory->CreateSwapChain(md3dDevice, )&sd, )&mSwapChain));
```

```
// 획득한 COM 인터페이스를 해제합니다(사용이 완료되었기 때문). ReleaseCOM(dxgiDevice);
```

```
ReleaseCOM(dxgiAdapter); ReleaseCOM(dxgiFactory);
```

DXGI(DirectX 그래픽스 인프라스트럭처)는 스왑 체인, 그래픽스 하드웨어 열거, 창 모드와 전체 화면 모드 전환과 같은 그래픽스 관련 작업을 처리하는 Direct3D와 별개의 API입니다. Direct3D와 분리하여 구현한 이유는 다른 그래픽스 API(예: Direct2D) 역시 스왑 체인, 그래픽스 하드웨어 열거, 창 모드와 전체 화면 모드 전환 기능을 필요로 하기 때문입니다. 이를 통해 여러 그래픽스 API가 DXGI API를 활용할 수 있습니다.

4.2.5 렌더 타겟 뷰 생성

§4.1.6에서 언급한 바와 같이, 리소스를 파이프라인 단계에 직접 바인딩하지 않습니다. 대신 리소스에 대한 뷰를 생성한 후 이 뷰를 파이프라인 단계에 바인딩해야 합니다. 특히 백 버퍼를 파이프라인의 출력 병합 단계에 바인딩하기 위해(Direct3D가 여기에 렌더링할 수 있도록), 백 버퍼에 대한 렌더 타겟 뷰를 생성해야 합니다. 다음 예제 코드는 이를 수행하는 방법을 보여줍니다:

```
ID3D11RenderTargetView* mRenderTargetView; ID3D11Texture2D* backBuffer;  
mSwapChain->GetBuffer(0, uuidof(ID3D11Texture2D), reinterpret_cast<void**>(&backBuffer));  
md3dDevice->CreateRenderTargetView(back Buffer, 0, &mRenderTargetView); ReleaseCOM(back Buffer);
```

1. 스왑 체인의 백 버퍼에 대한 포인터는 `IDXGISwapChain::GetBuffer` 메서드를 사용하여 획득합니다. 이 메서드의 첫 번째 매개변수는 획득하려는 특정 백 버퍼를 식별하는 인덱스입니다(여러 개가 존재하는 경우). 데모에서는 백 버퍼를 하나만 사용하며, 인덱스는 0입니다. 두 번째 매개변수는 버퍼의 인터페이스 유형으로, 일반적으로 항상 2D 텍스처(`ID3D11Texture2D`)입니다. 세 번째 매개변수는 백 버퍼에 대한 포인터를 반환합니다.
2. 렌더 타겟 뷰를 생성하려면 `ID3D11Device::CreateRenderTargetView` 메서드를 사용합니다. 첫 번째 매개변수는 렌더 타겟으로 사용될 리소스를 지정하며, 이전 예시에서는 백 버퍼입니다(즉, 백 버퍼에 대한 렌더 타겟 뷰를 생성하는 것입니다). 두 번째 매개변수는 `D3D11_RENDER_TARGET_VIEW_DESC` 구조체에 대한 포인터입니다. 이 구조체는 리소스 내 요소의 데이터 유형(포맷)을 비롯한 여러 정보를 설명합니다. 리소스가 유형 지정된 포맷(즉, 유형이 지정되지 않은 상태가 아님)으로 생성된 경우 이 매개변수는 null일 수 있으며, 이는 리소스가 생성된 포맷으로 이 리소스의 첫 번째 mip맵 레벨(백 버퍼는 하나의 mip맵 레벨만 가짐)에 대한 뷰를 생성하라는 의미입니다. (mip맵에 대해서는 8장에서 설명합니다.) 백 버퍼의 유형을 명시했으므로 이 매개변수에는 null을 지정합니다. 세 번째 매개변수는 생성된 렌더 타겟 뷰 객체에 대한 포인터를 반환합니다.
3. `IDXGISwapChain::GetBuffer` 호출은 백 버퍼에 대한 COM 참조 카운트를 증가시키므로, 코드 조각의 끝에서 사용을 마친 후 이를 해제(`ReleaseCOM`)합니다.

4.2.6 깊이/스텐실 버퍼 생성 및 뷰 설정

이제 깊이/스텐실 버퍼를 생성해야 합니다. §4.1.5에서 설명한 바와 같이, 깊이 버퍼는 깊이 정보(스텐실링을 사용하는 경우 스텐실 정보 포함)를 저장하는 2D 텍스처에 불과합니다. 텍스처를 생성하려면 생성할 텍스처를 설명하는 `D3D11_TEXTURE2D_DESC` 구조체를 작성한 후 `ID3D11Device::CreateTexture2D` 메서드를 호출해야 합니다. `D3D11_TEXTURE2D_DESC` 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_TEXTURE2D_DESC { UINT Width;  
    UINT Height; UINT  
    MipLevels; UINT ArraySize;  
    DXGI_FORMAT Format; DXGI_SAMPLE_DESC SampleDesc;
```

```
        D3D11_USAGE Usage;
        UINT BindFlags;
        UINT CPUAccessFlags;
        UINT MiscFlags;
    } D3D11_TEXTURE2D_DESC;
```

1. **Width:** 텍스처의 너비(텍셀 단위).
2. **Height:** 텍스처의 높이(텍셀 단위).
3. **MipLevels:** mip맵 레벨의 수. mip맵은 텍스처링 장에서 다루어집니다. 깊이/스텐실 버퍼 생성을 위해, 우리의 텍스처는 하나의 mip맵 레벨만 필요합니다.
4. **ArraySize:** 텍스처 배열에 포함된 텍스처의 개수입니다. 깊이/스텐실 버퍼의 경우 텍스처 하나만 필요합니다.
5. **Format:** 텍셀 형식을 지정하는 **DXGI_FORMAT** 열거형 멤버입니다. 깊이/스텐실 버퍼의 경우, §4.1.5에 표시된 형식 중 하나여야 합니다.
6. **SampleDesc:** 멀티샘플 수와 품질 수준입니다. §4.1.7 및 §4.1.8을 참조하십시오. 4X MSAA는 서브픽셀별 색상 및 깊이/스텐실 정보를 저장하기 위해 화면 해상도보다 4배 큰 백 버퍼와 깊이 버퍼를 사용합니다. 따라서 *깊이/스텐실 버퍼에 사용되는 멀티샘플링 설정은 렌더 타겟에 사용되는 설정과 일치해야 합니다.*
7. **사용법:** 텍스처가 어떻게 사용될지 지정하는 **D3D11_USAGE** 열거형의 멤버입니다. 네 가지 사용법 값은 다음과 같습니다: **D3D11_USAGE_DEFAULT**: GPU(그래픽 처리 장치)가 리소스에 읽고 쓸 경우 이 사용법을 지정합니다. CPU는 이 사용법으로 리소스에 읽기/쓰기할 수 없습니다. 깊이/스텐실 버퍼의 경우, **D3D11_USAGE_IMMUTABLE**: 리소스 생성 후 내용이 절대 변경되지 않을 경우 이 사용법을 지정합니다. 리소스가 GPU에 의해 읽기 전용으로 처리되므로 일부 잠재적 최적화가 가능합니다. **D3D11_USAGE_IMMUTABLE**: 리소스 생성 후 내용이 절대 변경되지 않을 경우 이 사용법을 지정합니다. GPU가 리소스를 읽기 전용으로 처리하므로 일부 잠재적 최적화가 가능합니다. CPU와 GPU는 리소스 생성 시 초기화하는 경우를 제외하고 불변 리소스에 쓰기 작업을 수행할 수 없습니다. CPU는 불변 리소스에서 읽기 작업을 수행할 수 없습니다. **D3D11_USAGE_DYNAMIC**: 애플리케이션(CPU)이 리소스의 데이터 내용을 자주 업데이트해야 하는 경우(예: 프레임 단위) 이 사용법을 지정합니다. 이 사용법을 가진 리소스는 GPU가 읽을 수 있고 CPU가 쓸 수 있습니다. CPU에서 GPU 리소스를 동적으로 업데이트하면 성능 저하가 발생합니다. 새 데이터가 CPU 메모리(시스템 RAM)에서 GPU 메모리(비디오 RAM)로 전송되어야 하기 때문입니다. 따라서 동적 사용법은 필요한 경우가 아니면 피해야 합니다. **D3D11_USAGE_STAGING**: 애플리케이션(CPU)이 리소스의 복사본을 읽을 수 있어야 하는 경우(즉, 리소스가 비디오 메모리에서 시스템 메모리로의 데이터 복사를 지원하는 경우) 이 사용법을 지정합니다. GPU에서 CPU 메모리로의 복사는 느린 작업이므로 필요하지 않은 한 피해야 합니다.

8. **BindFlags:** 리소스가 파이프라인에 바인딩될 위치를 지정하는 하나 이상의 플래그를 OR 연산으로 결합한 값입니다. 깊이/스텐실 버퍼의 경우 **D3D11_BIND_DEPTH_STENCIL**이어야 합니다. 텍스처에 대한 다른 바인딩 플래그로는 다음과 같습니다:

D3D11_BIND_RENDER_TARGET: 텍스처가 렌더 타겟으로 파이프라인에 바인딩됩니다.

D3D11_BIND_SHADER_RESOURCE: 텍스처가 파이프라인에 셰이더 리소스로 바인딩됩니다.

9. **CPUAccessFlags:** CPU가 리소스에 접근하는 방식을 지정합니다. CPU가 리소스에 쓰기 작업을 수행해야 하는 경우 **D3D11_CPU_ACCESS_WRITE**를 지정합니다. 쓰기 접근 권한이 있는 리소스는 반드시 **D3D11_USAGE_DYNAMIC** 또는 **D3D11_USAGE_STAGING** 사용 유형을 가져야 합니다. CPU가 버퍼에서 읽어야 하는 경우 **D3D11_CPU_ACCESS_READ**를 지정합니다. 읽기 액세스 권한이 있는 버퍼는 사용 유형 **D3D11_USAGE_STAGING**을 가져야 합니다. 깊이/스텐실 버퍼의 경우 GPU만 깊이/버퍼에 읽고 쓰므로, CPU가 깊이/스텐실 버퍼를 읽거나 쓰지 않으므로 이 값을 0으로 지정할 수 있습니다.

10. **MiscFlags:** 선택적 플래그로, 깊이/스텐실 버퍼에는 적용되지 않으므로 0으로 설정됩니다.

D3D11_USAGE_DYNAMIC 및 **D3D11_USAGE_STAGING** 사용 플래그는 성능 저하가 발생하므로 피해야 한다고 언급했습니다. 공통된 요인은 이 두 플래그 모두 CPU가 관여한다는 점입니다.

Note: CPU와 GPU 메모리 간에 오가는 작업은 성능 저하를 초래합니다. 최대 속도를 위해 그래픽 하드웨어는 모든 리소스를 생성하고 데이터를 GPU로 업로드한 후 해당 리소스가 그대로 유지될 때 가장 효율적으로 작동합니다. GPU에서만 리소스에 읽고 쓰는 경우입니다. 그러나 일부 애플리케이션에서는 이러한 플래그를 피할 수 없어 CPU가 개입해야 하지만, 항상 이러한 플래그 사용을 최소화하도록 노력해야 합니다.

이 책에서는 다양한 옵션(예: 사용 플래그, 바인딩 플래그, CPU 접근 플래그 등)을 적용하여 리소스를 생성하는 여러 사례를 살펴볼 것입니다. 지금은 깊이/스텐실 버퍼 생성에 필요한 값에만 집중하고 모든 옵션을 일일이 걱정하지 마십시오.

또한, 깊이/스텐실 버퍼를 사용하기 전에 파이프라인에 바인딩할 연관된 깊이/스텐실 뷰를 생성해야 합니다. 이 렌더 타겟 뷰를 생성하는 것과 유사한 방식으로 수행됩니다. 다음 코드 예제는 깊이/스텐실 텍스처와 해당 깊이/스텐실 뷰를 생성하는 방법을 보여줍니다:

```
D3D11_TEXTURE2D_DESC depthStencilDesc;
```

```

depthStencilDesc.Width                = mClientWidth;
depthStencilDesc.Height               = mClientHeight;
depthStencilDesc.MipLevels            = 1;
depthStencilDesc.ArraySize           = 1;
depthStencilDesc.Format               = DXGI_FORMAT_D24_UNORM_S8_UINT;

```

```

// 4X MSAA 사용? --스왑 체인의 MSAA 값과 일치해야 함. if( mEnable4xMsaa )
{
    depthStencilDesc.SampleDesc.Count = 4; depthStencilDesc.SampleDesc.Quality = m4xMsaaQuality-1;
}
// MSAA 미사용
{
    depthStencilDesc.SampleDesc.Count = 1;
    depthStencilDesc.SampleDesc.Quality = 0;
}
depthStencilDesc.Usage                = D3D11_USAGE_DEFAULT;
depthStencilDesc.BindFlags            = D3D11_BIND_DEPTH_STENCIL;
depthStencilDesc.CPUAccessFlags       = 0;
depthStencilDesc.MiscFlags            = 0;

```

```

ID3D11Texture2D* mDepthStencilBuffer; ID3D11DepthStencilView*
mDepthStencilView;

```

```

HR(md3dDevice->CreateTexture2D(
    &depthStencilDesc, // 생성할 텍스처의 설명. 0,
    &mDepthStencilBuffer)); // 깊이/스텐실 버퍼에 대한 포인터 반환.

```

```

HR(md3dDevice->CreateDepthStencilView(
    mDepthStencilBuffer, // 뷰를 생성할 리소스. 0,
    &mDepthStencilView)); // 깊이/스텐실 뷰 반환

```

CreateTexture2D의 두 번째 매개변수는 텍스처를 채울 초기 데이터에 대한 포인터입니다. 그러나 이 텍스처는 깊이/스텐실 버퍼로 사용될 예정이므로, 우리가 직접 어떤 데이터로 채울 필요가 없습니다. Direct3D는 깊이 버퍼링 및 스텐실 작업을 수행할 때 깊이/스텐실 버퍼에 직접 기록합니다. 따라서 두 번째 매개변수에는 null을 지정합니다.

CreateDepthStencilView의 두 번째 매개변수는 **D3D11_DEPTH_STENCIL_VIEW_DESC** 포인터입니다. 이 구조체는 리소스 내 요소의 데이터 형식(포맷)을 설명합니다. 리소스가 타입 지정된 포맷(즉, 타입이 지정되지 않은 상태가 아님)으로 생성된 경우 이 매개변수는 null이 될 수 있으며, 이는 뷰 생성을 의미합니다.

이 구조체는 리소스 내 요소의 데이터 형식(포맷)을 설명합니다. 리소스가 타입 지정된 포맷(즉, 타입리스가 아닌)으로 생성된 경우, 이 매개변수는 null로 설정할 수 있습니다. 이는 리소스가 생성된 포맷으로 이 리소스의 첫 번째 맵 레벨(덱스/스텐실 버퍼는 단일 맵 레벨로 생성됨)에 대한 뷰를 생성하라는 의미입니다. (맵에 대해서는 [8장에서](#) 설명합니다.) 깊이/스텐실 버퍼의 유형을 명시했으므로 이 매개변수에는 null을 지정합니다.

4.2.7 뷰를 출력 병합 단계에 바인딩하기

이제 백 버퍼와 깊이 버퍼에 대한 뷰를 생성했으므로, 이 뷰들을 파이프라인의 출력 병합 단계에 바인딩하여 해당 리소스를 파이프라인의 렌더 타깃 및 깊이/스텐실 버퍼로 설정할 수 있습니다:

```

md3dImmediateContext->OMSetRenderTargets(
    1, &mRenderTargetView, mDepthStencilView);

```

첫 번째 매개변수는 바인딩할 렌더 타깃의 수입니다. 여기서는 하나만 바인딩하지만, 여러 렌더 타깃에 동시에 렌더링하기 위해 더 많은 타깃을 바인딩할 수 있습니다(고급 기법). 두 번째 매개변수는 파이프라인에 바인딩할 렌더 타깃 뷰 포인터 배열의 첫 번째 요소에 대한 포인터입니다. 세 번째 매개변수는 파이프라인에 바인딩할 깊이/스텐실 뷰에 대한 포인터입니다.

 *렌더 타겟 뷰 배열은 설정할 수 있지만, 깊이/스텐실 뷰는 하나만 설정할 수 있습니다. 다중 렌더 타겟 사용은 이 책의 제3부에서 다루는 고급 기법입니다.*

4.2.8 뷰포트 설정

일반적으로 우리는 3D 장면을 백 버퍼 전체에 렌더링하는 것을 선호합니다. 그러나 때로는 3D 장면을 백 버퍼의 하위 사각형 영역에만 렌더링하고 싶을 때가 있습니다. [그림 4.7](#)을 참조하십시오.

백 버퍼의 하위 사각형 영역을 *뷰포트(viewport)*라고 하며, 다음 구조체로 정의됩니다:

```
typedef struct D3D11_VIEWPORT { FLOAT
    TopLeftX;
    FLOAT TopLeftY; FLOAT
    Width; FLOAT Height;
    FLOAT MinDepth; FLOAT
    MaxDepth;
} D3D11_VIEWPORT;
```

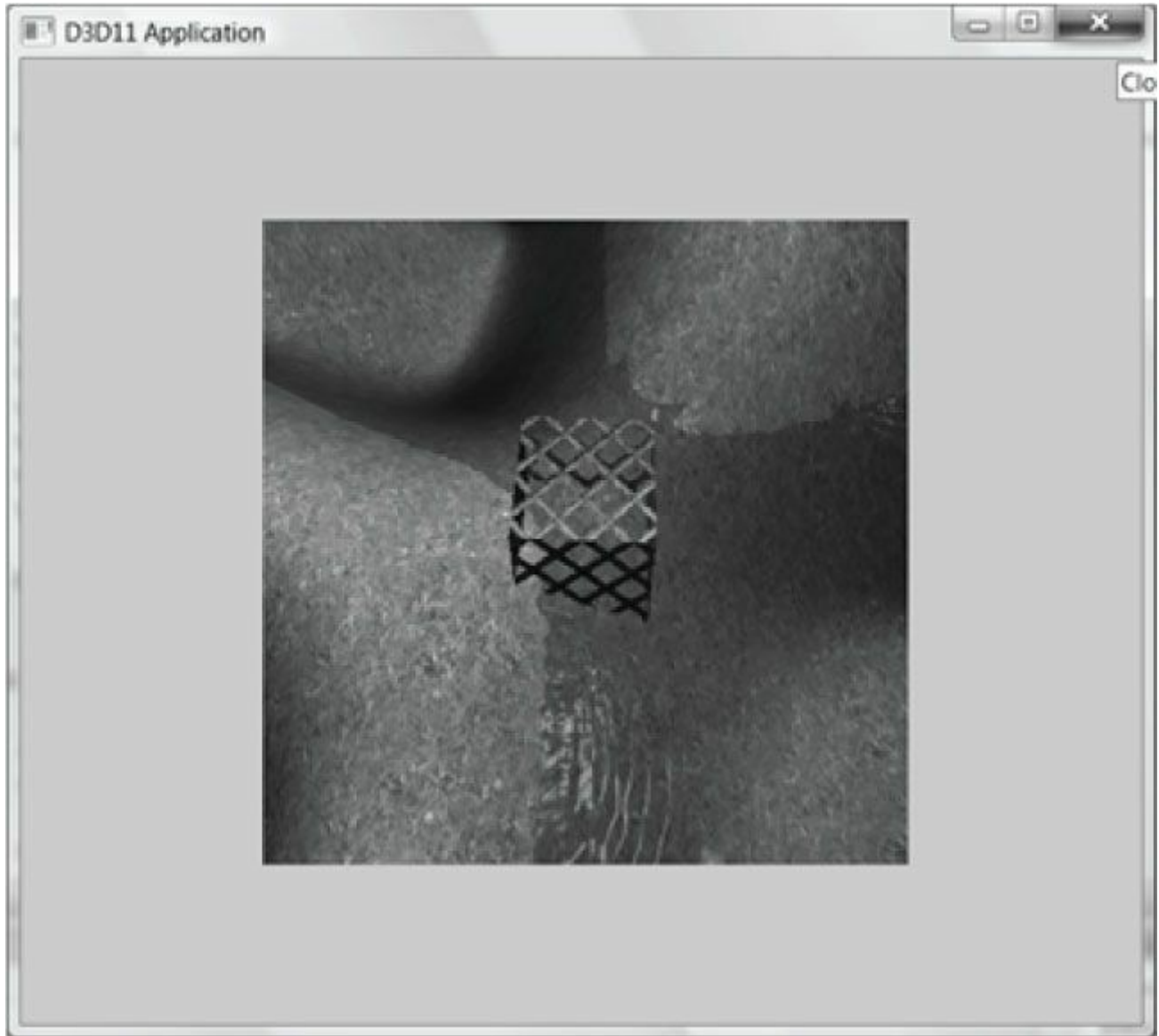


그림 4.7. 뷰포트를 수정함으로써 3D 장면을 백 버퍼의 하위 사각형 영역에 그릴 수 있습니다. 이후 백 버퍼는 창의 클라이언트 영역에 표시됩니다.

처음 네 개의 데이터 멤버는 우리가 그리려는 창의 클라이언트 영역 사각형에 상대적인 뷰포트 사각형을 정의합니다(데이터 멤버가 **float** 형식이므로 소수점 픽셀 좌표를 지정할 수 있음을 유의하십시오). **MinDepth** 멤버는 최소 깊이 버퍼 값을, **MaxDepth**는 최대 깊이 버퍼 값을 지정합니다. Direct3D는 0부터 1까지의 깊이 버퍼 범위를 사용하므로, 특별한 효과를 원하지 않는 한 **MinDepth**와 **MaxDepth**는 각각 해당 값으로 설정해야 합니다.

D3D11_VIEWPORT 구조체를 채운 후에는 **ID3D11DeviceContext::RSSetViewports** 메서드를 사용하여 Direct3D에 뷰포트를 설정합니다. 다음 예제는 전체 백 버퍼에 그리는 뷰포트를 생성하고 설정합니다:

```
D3D11_VIEWPORT vp;
```

```
vp.TopLeftX = 0.0f; vp.TopLeftY = 0.0f;
vp.Width = static_cast<float>(mClientWidth); vp.Height =
static_cast<float>(mClientHeight); vp.MinDepth = 0.0f;
vp.MaxDepth = 1.0f;
```

```
md3dImmediateContext->RSSetViewports(1, &vp);
```

첫 번째 매개변수는 바인딩할 뷰포트의 수이며(복수 사용은 고급 효과를 위한 것입니다), 두 번째 매개변수는 뷰포트 배열에 대한 포인터입니다.

예를 들어, 뷰포트를 사용하여 2인용 게임 모드의 분할 화면을 구현할 수 있습니다. 두 개의 뷰포트를 생성하면 됩니다.

화면의 왼쪽 절반용 하나와 오른쪽 절반용 하나입니다. 그런 다음 플레이어 1의 시점에서 본 3D 장면을 왼쪽 뷰포트에, 플레이어 2의 시점에서 본 3D 장면을 오른쪽 뷰포트에 각각 렌더링합니다.

또는 뷰포트를 사용하여 화면의 하위 영역(subrectangle)에 렌더링하고, 남은 영역을 버튼, 슬라이더, 목록 상자 등의 UI(사용자 인터페이스) 컨트롤로 채울 수도 있습니다.

4.3 타이밍과 애니메이션

애니메이션을 정확하게 구현하려면 시간을 추적해야 합니다. 특히 애니메이션 프레임 간 경과 시간을 측정해야 합니다. 프레임 속도가 높을수록 프레임 간 시간 간격은 매우 짧아지므로 높은 정확도의 타이머가 필요합니다.

4.3.1 성능 타이머

정확한 시간 측정을 위해 성능 타이머(또는 성능 카운터)를 사용합니다. 성능 타이머를 조회하는 Win32 함수를 사용하려면 <windows.h>를 #include해야 합니다.

성능 타이머는 *카운트(counts)*라는 단위로 시간을 측정합니다. QueryPerformanceCounter 함수를 사용하여 성능 타이머의 현재 시간 값(카운트로 측정)을 다음과 같이 얻을 수 있습니다:

QueryPerformanceCounter 함수를 다음과 같이 사용합니다:

```
__int64 currTime; QueryPerformanceCounter((LARGE_INTEGER*)&currTime);
```

이 함수는 매개변수를 통해 현재 시간 값을 반환하며, 해당 값은 64비트 정수 값입니다.

성능 타이머의 주파수(초당 카운트 수)를 얻으려면 QueryPerformanceFrequency 함수를 사용합니다:

```
__int64 countsPerSec; QueryPerformanceFrequency((LARGE_INTEGER*)&countsPerSec);
```

그런 다음 카운트당 초(또는 초의 일부) 수는 단순히 초당 카운트의 역수입니다:

```
mSecondsPerCount = 1.0 / (double)countsPerSec;
```

따라서 시간 측정값 valueInCounts를 초 단위로 변환하려면 변환 계수 mSecondsPerCount를 곱하기만 하면 됩니다: valueInSecs = valueInCounts * mSecondsPerCount;

QueryPerformanceCounter 함수가 반환하는 값 자체는 그다지 흥미롭지 않습니다. 우리가 하는 일은

QueryPerformanceCounter를 사용하여 현재 시간 값을 얻은 다음, 조금 후에 다시 QueryPerformanceCounter를 사용하여 현재 시간 값을 얻습니다. 그런 다음 두 시간 호출 사이에 경과한 시간은 단순히 그 차이입니다. 즉, 우리는 항상 두 타임스탬프 사이의 상대적 차이를 살펴서 시간을 측정하며, 성능 카운터가 반환하는 실제 값 자체를 측정하지 않습니다. 다음 예시는 이 개념을 더 잘 설명합니다:

```
__int64 A = 0; QueryPerformanceCounter((LARGE_INTEGER*)&A);
```

```
/* 작업 수행 */
```

```
__int64 B = 0; QueryPerformanceCounter((LARGE_INTEGER*)&B);
```

따라서 작업 수행에는 (B-A) 카운트가 소요되었으며, 이는 (B-A)*mSecondsPerCount 초에 해당합니다.

 MSDN은 QueryPerformanceCounter에 대해 다음과 같이 언급합니다. "다중 프로세서 컴퓨터에서는 호출되는 프로세서가 중요하지 않아야 합니다. 그러나 기본 입출력 시스템 (BIOS)이나 하드웨어 추상화 계층(HAL)의 버그로 인해 프로세서마다 다른 결과를 얻을 수 있습니다." SetThreadAffinityMask 함수를 사용하면 메인 애플리케이션 스레드가 다른 프로세서로 전환되지 않도록 할 수 있습니다.

4.3.2 게임 타이머 클래스

다음 두 섹션에서는 다음 GameTimer 클래스의 구현에 대해 논의할 것입니다.

```
class GameTimer
{
public:
    GameTimer();

    float GameTime()const; // 초 단위 float DeltaTime()const;
    // 초 단위

    void Reset(); // 메시지 루프 전에 호출. void Start(); // 일시 정지 해제 시 호출.
    void Stop(); // 일시 정지 시 호출. void Tick(); // 매 프레임마다 호출.

private:
    double mSecondsPerCount; double mDeltaTime;

    __int64 mBaseTime;
    __int64 mPausedTime;
    __int64 mStopTime;
    __int64 mPrevTime;
    __int64 m현재시간;

    bool mStopped;
};
```

생성자는 특히 성능 카운터의 주파수를 조회합니다. 다른 멤버 함수들은 다음 두 섹션에서 설명합니다.

```
GameTimer::GameTimer()
: mSecondsPerCount(0.0), mDeltaTime(-1.0), mBaseTime(0), mPausedTime(0), mPrevTime(0),
mCurrTime(0), mStopped(false)
{
    __int64 countsPerSec; QueryPerformanceFrequency((LARGE_INTEGER*)&countsPerSec); mSecondsPerCount = 1.0 /
(double)countsPerSec;
}
```

 GameTimer 클래스와 구현은 샘플 코드의 Common 디렉터리에 있는 GameTimer.h 및 GameTimer.cpp 파일에 있습니다.

4.3.3 프레임 간 경과 시간

애니메이션 프레임을 렌더링할 때, 게임 오브젝트를 시간 경과에 따라 업데이트하기 위해 프레임 간 경과 시간을 알아야 합니다. 프레임 간 경과 시간 계산은 다음과 같이 진행됩니다. i번째 프레임 동안 성능 카운터가 반환한 시간을 t_i 라고 하고, 이전 프레임 동안 성능 카운터가 반환한 시간을 t_{i-1} 이라고 합시다. 그러면 t_{i-1} 측정값과 t_i 측정값 사이의 경과 시간은 $\Delta t = t_i - t_{i-1}$ 입니다. 실시간 렌더링의 경우, 부드러운 애니메이션을 위해 일반적으로 초당 최소 30프레임이 필요합니다(보통 훨씬 더 높은 프레임률을 가집니다). 따라서 $\Delta t = t_i - t_{i-1}$ 은 상대적으로 작은 값이 되는 경향이 있습니다. 다음 코드는 Δt 가 코드 내에서 어떻게 계산되는지 보여줍니다:

```

void GameTimer::Tick()
{
    if(mStopped)
    {
        mDeltaTime = 0.0; return;
    }
    // 이번 프레임의 시간을 가져옵니다.
    __int64 currTime; QueryPerformanceCounter((LARGE_INTEGER*)&currTime); mCurrTime =
    currTime;

    // 현재 프레임과 이전 프레임 간의 시간 차이. mDeltaTime = (mCurrTime -
    mPrevTime)*mSecondsPerCount;

    // 다음 프레임 준비. mPrevTime = mCurrTime;

    // 음수 방지. DXSDK의 CDXUTimer 문서에 따르면
    // 프로세서가 절전 모드로 전환되거나 다른 프로세서로 전환될 경우
    // 프로세서로 전환되면 mDeltaTime이 음수가 될 수 있다고 명시합니다. if(mDeltaTime < 0.0)
    {
        mDeltaTime = 0.0;
    }
}
float GameTimer::DeltaTime()const
{
    return (float)mDeltaTime;
}

```

Tick 함수는 애플리케이션 메시지 루프에서 다음과 같이 호출됩니다:

```

int D3DApp::Run()
{
    MSG msg = {0};

    mTimer.Reset(); while(msg.message != WM_QUIT)
    {
        // 윈도우 메시지가 있으면 처리합니다. if(Peek Message(&msg, 0, 0, 0,
        PM_REMOVE))
        {
            TranslateMessage(&msg); DispatchMessage(&msg);
        }
        // 그렇지 않으면 애니메이션/게임 작업을 수행합니다. else
    {
        mTimer.Tick();
        if(!mAppPaused)
        {
            CalculateFrameStats(); UpdateScene(mTimer.DeltaTime()); DrawScene();

        }
        else    Sleep(100);
    }
}

```

```
    }  
    }  
    return (int)msg.wParam;  
}  

```

이렇게 하면 Δt 가 매 프레임마다 계산되어 UpdateScene 메서드에 전달되므로, 이전 애니메이션 프레임 이후로 경과한 시간에 따라 장면을 업데이트할 수 있습니다. Reset 메서드의 구현은 다음과 같습니다:

```
void GameTimer::Reset()  
{  
    __int64 currTime; QueryPerformanceCounter((LARGE_INTEGER*)&currTime);  
  
    mBaseTime = currTime; mPrevTime  
    = currTime; mStopTime = 0;  
    mStopped = false;  
}
```

표시된 변수 중 일부는 아직 설명되지 않았습니다(§4.3.4 참조). 그러나 이 코드는 **Reset**이 호출될 때 **mPrevTime**을 현재 시간으로 초기화함을 알 수 있습니다. 애니메이션의 첫 번째 프레임에는 이전 프레임이 없으므로 이전 타임스탬프 $t_{i(-1)}$ 도 존재하지 않습니다. 따라서 메시지 루프가 시작되기 전에 Reset 메서드에서 이 값을 초기화해야 합니다.

4.3.4 총 시간

또 다른 유용한 시간 측정값은 일시 정지 시간을 제외하고 애플리케이션 시작 이후 경과한 시간으로, 이를 *총 시간이라고* 부를 것입니다. 다음 상황은 이 측정값이 어떻게 유용할 수 있는지 보여줍니다. 플레이어가 레벨을 완료할 시간이 300초라고 가정해 보겠습니다. 레벨이 시작될 때, 애플리케이션 시작 이후 경과한 시간인 $t_{(start)}$ 를 얻을 수 있습니다. 그런 다음 레벨이 시작된 후, 일정 간격으로 애플리케이션 시작 이후 경과 시간 t 를 확인할 수 있습니다. 만약 $t - t_{start} > 300$ 초(그림 4.8 참조)라면 플레이어가 레벨에 300초 이상 머물렀음을 의미하며 패배합니다. 당연히 이 상황에서는 게임이 플레이어에 대해 일시 정지된 시간은 계산에 포함하지 않아야 합니다.

플레이어에게 불리하게 계산해서는 안 됩니다.

총 시간의 또 다른 응용은 양을 시간의 함수로 애니메이션화할 때입니다. 예를 들어, 우리가 원하는 경우를 가정해 보겠습니다.

$$\begin{cases} x = 10 \cos t \\ y = 20 \\ z = 10 \sin t \end{cases}$$

시간의 함수로 장면을 공전하는 빛을 생성합니다. 그 위치는 다음의 매개변수 방정식으로 표현할 수 있습니다:

여기서 t 는 시간을 나타내며, t (시간)이 증가함에 따라 빛의 좌표가 업데이트되어 빛이 $y = 20$ 평면에서 반지름 10의 원을 그리며 이동합니다. 이러한 종류의 애니메이션에서도 일시 정지된 시간은 계산하지 않습니다. 그림 4.9를 참조하십시오.

총 시간을 구현하기 위해 다음 변수를 사용합니다:

```
__int64 mBaseTime;  
__int64 mPausedTime;  
__int64 mStopTime;
```

§4.3.3에서 살펴본 바와 같이, **mBaseTime**은 **Reset**이 호출될 때 현재 시간으로 초기화됩니다. 이를 애플리케이션이 시작된 시점으로 생각할 수 있습니다. 대부분의 경우 메시지 루프 전에 **Reset**을 한 번만 호출하므로, **mBaseTime**은 애플리케이션의 수명 동안 일정하게 유지됩니다. 변수 **mPausedTime**은 일시 중지 상태에서 경과한 모든 시간을 누적합니다. 이 시간을 누적해야 총 실행 시간에서 일시 중지 시간을 빼서 계산할 수 있습니다. **mStopTime** 변수는 타이머가 중지(일시 중지)된 시점을 제공하며, 이는 일시 중지 시간을 추적하는 데 사용됩니다.

GameTimer 클래스의 두 가지 중요한 메서드는 **Stop**과 **Start**입니다. 이 메서드들은 각각 애플리케이션이 일시 중지될 때와 재개될 때 각각 호출되어야 합니다. 이렇게 해야 **GameTimer**가 일시 정지 시간을 추적할 수 있습니다. 코드 주석은 이 두 메서드의 세부 사항을 설명합니다.

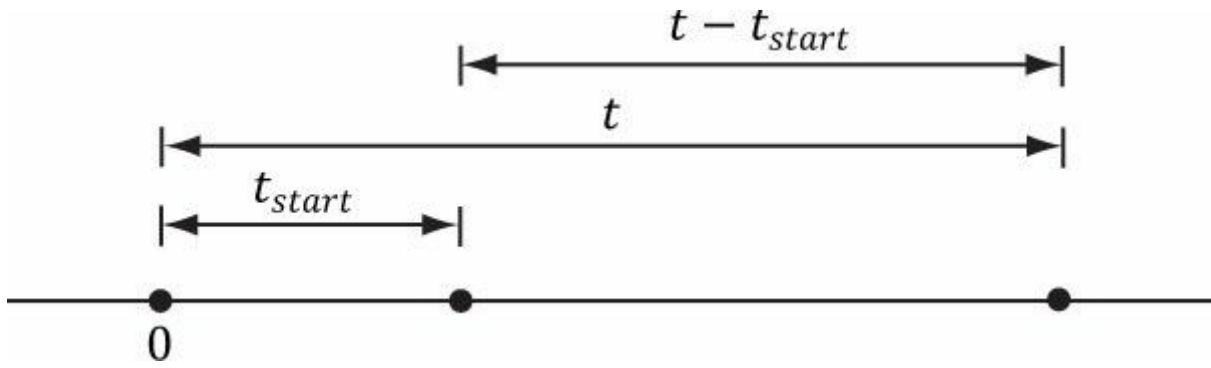


그림 4.8. 레벨 시작 이후 경과 시간 계산. 애플리케이션 시작 시간을 원점(0)으로 선택하고, 해당 기준 프레임에 상대적인 시간 값을 측정함을 유의하십시오.

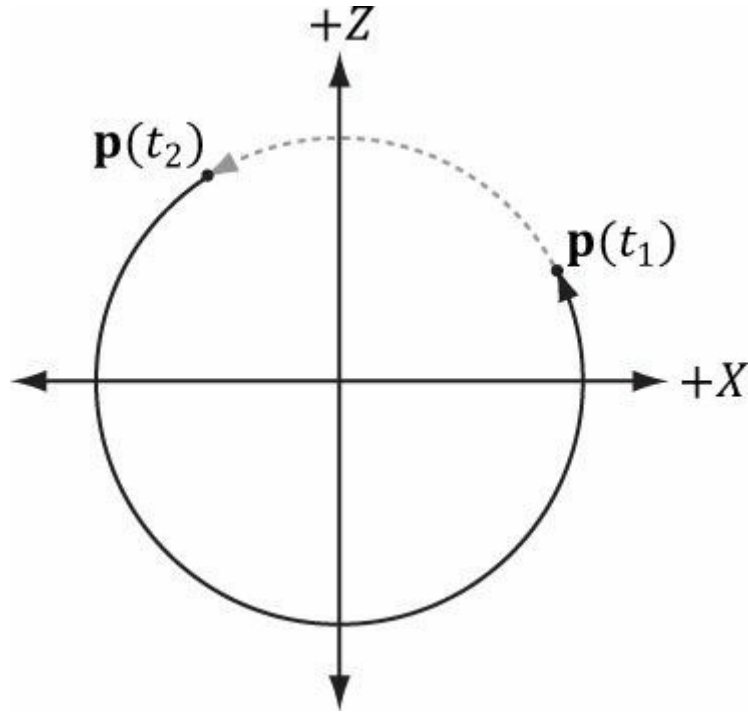


그림 4.9. t_1 에서 일시 정지하고 t_2 에서 일시 정지를 해제하며 정지 시간을 계산한 경우, 일시 정지 해제 시 위치가 $p(t_1)$ 에서 $p(t_2)$ 로 갑자기 점프합니다.

```
void GameTimer::Stop()
{
    // 이미 중지된 상태라면 아무 작업도 수행하지 않습니다. if(!mStopped)
    {
        __int64 currTime; QueryPerformanceCounter((LARGE_INTEGER*)&currTime);

        // 그렇지 않으면, 중지된 시간을 저장하고
        // 타이머가 중지되었음을 나타내는 부울 플래그를 설정합니다. mStopTime = currTime;
        mStopped = true;
    }
}

void GameTimer::Start()
{
    __int64 시작시간; QueryPerformanceCounter((LARGE_INTEGER*)&시작시간);

    // 중지 및 시작 쌍 사이의 경과 시간을 누적합니다.
    //
    // |<-----d----->|
    //   *-----*-----> 시간
    // mStopTime - startTime
```

```
// 타이머가 중지된 상태에서 재개하는 경우... if( mStopped )
{
    // 일시 중지된 시간을 누적합니다. mPausedTime += (startTime -
    mStopTime);

    // 타이머를 다시 시 작 하므로 현재
    // 이전 시간은 일시 중지 중에 발생한 것이므로 유효하지 않습니다.
    // 따라서 현재 시간으로 재설정합니다. mPrevTime =
    startTime;

    // 더 이상 정지 상태가 아님...
    mStopTime = 0; mStopped =
    false;
}
}
```

마지막으로, Reset 호출 이후 경과된 시간(일시정지 시간 제외)을 반환하는 TotalTime 멤버 함수는 다음과 같이 구현됩니다:

```
float GameTimer::TotalTime()const
{
    // 중지된 상태라면, 중지된 이후 경과한 시간은 계산하지 않는다
    // 또한 이전에 이미 일시 정지 상태였다면
    // 정지 상태가 있었다면, 거리 mStopTime - mBaseTime에는 정지 시간이 포함되어
    // 시간을 포함하므로 계산 대상에서 제외해야 합니다. 이를 보정하기 위해
    // mStopTime에서 정지 시간을 빼면 됩니다:
    //
    // 이전 일시 정지 시간
    // |<----->|
    //   *-----*-----*-----*-----> 시간
    // mBaseTime mStopTime mCurrTime

    if(mStopped)
    {
        return (float)((((mStopTime - mPausedTime)-
            mBaseTime)*mSecondsPerCount);
    }

    // mCurrTime - mBaseTime 간격에는 일시 정지 시간이 포함되어 있으며,
    // 이를 보정하기 위해 mCurrTime의 일시 정지 시간을 빼면 됩니다.
    // mCurrTime에서 일시 중지 시간을 빼면 됩니다:
    //
    // (mCurrTime - mPausedTime) - mBaseTime
    //
    // |<---일시정지 시간--->|
    //   *-----*-----*-----*-----> 시간
    // mBaseTime mStopTime startTime mCurrTime

    else
    {
        return (float)((((mCurrTime-mPausedTime)-
            mBaseTime)*mSecondsPerCount);
    }
}
```

우리의 데모 프레임워크는 애플리케이션 시작 이후의 총 시간()과 프레임 간 경과 시간을 측정하기 위해 GameTimer 인스턴스를 생성합니다. 그러나 추가 인스턴스를 생성하여 일반적인 "스톱워치"로 사용할 수도 있습니다. 예를 들어 폭탄이 점화되면 새로운 GameTimer를 시작하고, 총 시간이 5초에 도달하면 폭탄이 폭발했다는 이벤트를 발생시킬 수 있습니다. 5초에 도달하면 폭탄이 폭발했다는 이벤트를 발생시킬 수 있습니다.

4.4 데모 애플리케이션 프레임워크

이 책의 데모는 *d3dUtil.h*, *d3dApp.h* 및 *d3dApp.cpp* 파일의 코드를 사용하며, 해당 파일은 책의 웹사이트에서 다운로드할 수 있습니다. 모든 데모 애플리케이션에서 사용되는 이러한 공통 파일들은 책의 2부 및 3부에서 *Common* 디렉터리에 위치하므로, 각 프로젝트마다 파일을 중복하여 포함하지 않아도 됩니다. *d3dUtil.h* 파일에는 유용한 유틸리티 코드가 포함되어 있으며, *d3dApp.h* 및 *d3dApp.cpp* 파일에는 Direct3D 애플리케이션을 캡슐화하는 데 사용되는 핵심 Direct3D 애플리케이션 클래스 코드가 포함되어 있습니다. 이 장을 읽은 후 독자 여러분께서는 이 파일들을 공부해 보시길 권장합니다. 이 책에서는 해당 파일들의 모든 코드 줄을 다루지 않기 때문입니다(예: 기본적인 Win32 프로그래밍이 이 책의 전제 조건이므로 창 생성 방법을 보여주지 않습니다). 이 프레임워크의 목표는 창 생성 코드와 Direct3D 초기화 코드를 숨기는 것이었습니다. 이 코드를 숨김으로써, 샘플 코드가 설명하려는 특정 세부 사항에만 집중할 수 있어 데모가 덜 산만해질 것이라고 생각합니다.

4.4.1 D3DApp

D3DApp 클래스는 메인 애플리케이션 창 생성, 애플리케이션 메시지 루프 실행, 창 메시지 처리, Direct3D 초기화 기능을 제공하는 기본 Direct3D 애플리케이션 클래스입니다. 또한 이 클래스는 데모 애플리케이션을 위한 프레임워크 함수를 정의합니다. 클라이언트는 **D3DApp**에서 파생된 후 가상 프레임워크 함수를 재정의하고, 파생된 D3DApp 클래스의 인스턴스를 단 하나만 생성해야 합니다. **D3DApp** 클래스는 다음과 같이 정의됩니다:

```
class D3DApp
{
public:
    D3DApp(HINSTANCE hInstance); virtual ~D3DApp();

    HINSTANCE AppInst()const; HWND
        MainWnd()const; float
    AspectRatio()const;

    int Run();

    // 프레임워크 메서드. 파생된 클라이언트 클래스는 이러한 메서드를 재정의하여
    // 특정 애플리케이션 요구 사항을 구현합니다.

    virtual bool Init(); virtual void
    OnResize();
    virtual void UpdateScene(float dt)=0; virtual void
    DrawScene()=0;
    virtual LRESULT MsgProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM
        lParam);

    // 마우스 입력 처리를 위한 편의 오버라이드.
    virtual void OnMouseDown(WPARAM btnState, int x, int y){ } virtual void
    OnMouseUp(WPARAM btnState, int x, int y){ } virtual void OnMouseMove(WPARAM
    btnState, int x, int y){ }

protected:
    bool InitMainWindow(); bool
    InitDirect3D();

    void 프레임 통계 계산(); protected:

    HINSTANCE mhAppInst; // 애플리케이션 인스턴스 핸들 HWND mhMainWnd;

    // 메인 윈도우 핸들

    bool mAppPaused; // 애플리케이션이 일시 정지되었는가? bool
    mMinimized; // 애플리케이션이 최소화되었는가? bool
    mMaximized; // 애플리케이션이 최대화되었는가?
    bool mResizing; // 크기 조정 바가 드래그 중인가? UINT
    m4xMsaaQuality; // 4X MSAA 품질 수준

    // "델타 타임"과 게임 시간 추적용 (§4.3).
```

```
GameTimer mTimer;
```

```
// D3D11 디바이스 (§4.2.1), 페이지 플리핑용 스왑 체인
```

```
// (§4.2.4), 깊이/스텐실 버퍼용 2D 텍스처 (§4.2.6),
```

```
// 렌더 타겟 (§4.2.5) 및 깊이/스텐실 뷰 (§4.2.6), 그리고
```

```
// 뷰포트 (§4.2.8).
```

```
ID3D11Device* md3dDevice; ID3D11DeviceContext*
md3dImmediateContext; IDXGISwapChain* mSwapChain;
ID3D11Texture2D* mDepthStencilBuffer; ID3D11RenderTargetView*
mRenderTargetView; ID3D11DepthStencilView* mDepthStencilView;
D3D11_VIEWPORT mScreenViewport;
```

```
// 다음 변수들은 D3DApp 생성자에서
```

```
// 기본값으로 초기화됩니다. 그러나
```

```
// 파생 클래스 생성자에서 값을 재정의하여 다른 기본값을 선택할 수 있습니다.
```

```
// 창 제목/캡션. D3DApp 기본값은 "D3D11 Application"입니다. std::wstring mMainWndCaption;
```

```
하드웨어 장치 또는 참조 장치? D3DApp은 기본적으로
```

```
// D3D_DRIVER_TYPE_HARDWARE로 설정됩니다.
```

```
D3D_DRIVER_TYPE md3dDriverType;
```

```
// 창의 클라이언트 영역 초기 크기. D3DApp은 기본값으로
```

```
// 800x600입니다. 그러나 이러한 값은 창 크기가 변경될 때
```

```
// 창 크기가 조정됨에 따라 현재 클라이언트 영역 크기를 반영하도록 변경됩니다. int mClientWidth;
```

```
int mClientHeight;
```

```
// 4X MSAA 사용 시 true (§4.1.8). 기본값은 false입니다. bool mEnable4xMsaa;
```

```
};
```

이전 코드에서는 일부 데이터 멤버를 설명하기 위해 주석을 사용했습니다. 메서드에 대해서는 후속 섹션에서 설명합니다.

4.4.2 비 프레임워크 메서드

1. **D3DApp**: 생성자는 데이터 멤버를 기본값으로 초기화합니다.
2. **~D3DApp**: 소멸자는 **D3DApp**이 획득한 COM 인터페이스를 해제합니다.
3. **AppInst**: 사소한 액세스 함수는 응용 프로그램 인스턴스 핸들의 복사본을 반환합니다.
4. **MainWnd**: 단순한 액세스 함수는 메인 윈도우 핸들의 복사본을 반환합니다.
5. **AspectRatio**: 화면 비율은 백 퍼퍼 너비와 높이의 비율로 정의됩니다. 화면 비율은 다음 장에서 사용될 것입니다. 이는 다음과 같이 간단히 구현됩니다:

```
float D3DApp::AspectRatio()const
{
    return static_cast<float>(mClientWidth) / mClientHeight;
}
```

6. **Run**: 이 메서드는 애플리케이션 메시지 루프를 감쌉니다. **Win32 Peek Message** 함수를 사용하여 메시지가 없을 때도 게임 로직을 처리할 수 있도록 합니다. 이 함수의 구현은 §4.3.3에서 설명되었습니다.
7. **InitMainWindow**: 메인 애플리케이션 창을 초기화합니다. 독자가 기본적인 Win32 창 초기화에 익숙하다고 가정합니다.
8. **InitDirect3D**: §4.2에서 논의된 단계를 구현하여 Direct3D를 초기화합니다.
9. **CalculateFrameStats**: 초당 평균 프레임 수와 프레임당 평균 밀리초를 계산합니다. 이 메서드의 구현은 §4.4.4에서 설명합니다.

4.4.3 프레임워크 메서드

이 책의 각 샘플 애플리케이션에서는 **D3DApp**의 다섯 가지 가상 함수를 일관되게 재정의합니다. 이 다섯 가지 함수는

특정 샘플에 특화된 코드를 구현하는 데 사용됩니다. 이러한 설정의 장점은 초기화 코드, 메시지 처리 등이 D3DApp 클래스에서 구현되므로 파생 클래스는 데모 애플리케이션의 특정 코드에만 집중하면 된다는 점입니다. 프레임워크 메서드에 대한 설명은 다음과 같습니다:

- 1. **Init**: 리소스 할당, 객체 초기화, 조명 설정 등 애플리케이션 초기화 코드를 넣기 위해 이 메서드를 사용합니다. D3DApp의 이 메서드 구현은 InitMainWindow와 **InitDirect3D**를 호출하므로, 파생 구현에서는 먼저 **D3DApp** 버전의 이 메서드를 다음과 같이 호출해야 합니다:

```
bool TestApp::Init()
{
    if(!D3DApp::Init()) return false;
    /* 나머지 초기화 코드는 여기에 위치합니다 */
}
```

이렇게 하면 **ID3D11Device**가 나머지 초기화 코드에서 사용할 수 있게 됩니다. (Direct3D 리소스 획득에는 유효한 **ID3D11Device**가 필요합니다.)

- 2. **OnResize**: 이 메서드는 **WM_SIZE** 메시지를 수신할 때 **D3DApp::MsgProc**에 의해 호출됩니다. 창 크기가 변경되면 클라이언트 영역 크기에 따라 일부 Direct3D 속성을 변경해야 합니다. 특히 백 버퍼와 깊이/스텐실 버퍼는 창의 새 클라이언트 영역에 맞게 재생성해야 합니다. 백 버퍼는 **IDXGISwapChain::ResizeBuffers** 메서드를 호출하여 크기를 조정할 수 있습니다. 깊이/스텐실 버퍼는 파괴한 후 새로운 크기에 맞춰 재구성해야 합니다. 또한 렌더 타겟과 깊이/스텐실 뷰도 재구성해야 합니다. D3DApp 구현의 **OnResize**는 백 버퍼와 깊이/스텐실 버퍼 크기 조정에 필요한 코드를 처리합니다. 자세한 내용은 소스 코드를 참조하십시오. 버퍼 외에도 클라이언트 영역의 크기에 따라 달라지는 다른 속성들(예: 투영 행렬)이 있으므로, 이 메서드는 프레임워크의 일부입니다. 창 크기가 변경될 때 클라이언트 코드가 자체 코드를 실행해야 할 수 있기 때문입니다.
- 3. **UpdateScene**: 이 추상 메서드는 매 프레임마다 호출되며, 시간에 따른 3D 애플리케이션 업데이트(예: 애니메이션 실행, 카메라 이동, 충돌 감지, 사용자 입력 확인 등)에 사용되어야 합니다.
- 4. **DrawScene**: 이 추상 메서드는 매 프레임마다 호출되며, 현재 프레임을 백 버퍼에 실제로 그리기 위한 렌더링 명령을 발행하는 곳입니다. 프레임 그리기가 완료되면 **IDXGISwapChain::Present** 메서드를 호출하여 백 버퍼를 화면에 표시합니다.
- 5. **MsgProc**: 이 메서드는 메인 애플리케이션 창의 윈도우 프로시저 함수를 구현합니다. 일반적으로 **D3DApp::MsgProc**가 처리하지 않는(또는 원하는 방식으로 처리하지 않는) 메시지를 처리해야 할 경우에만 이 메서드를 재정의하면 됩니다. 이 메서드의 **D3DApp** 구현은 §4.4.5에서 살펴봅니다. 이 메서드를 재정의할 경우, 처리하지 않는 모든 메시지는 **D3DApp::MsgProc**로 전달해야 합니다.

 *기존 다섯 가지 프레임워크 메서드 외에도, 마우스 버튼이 눌러지거나 놓일 때, 그리고 마우스가 움직일 때 발생하는 이벤트를 처리하기 위해 편의상 세 가지 가상 함수를 추가로 제공합니다.*

```
virtual void OnMouseDown(WPARAM btnState, int x, int y) {} virtual void
OnMouseUp(WPARAM btnState, int x, int y) {} virtual void OnMouseMove(WPARAM btnState,
int x, int y){}
```

이 방법으로 마우스 메시지를 처리하려면 **MsgProc** 메서드를 재정의하는 대신 이 메서드들을 재정의할 수 있습니다. 첫 번째 매개변수는 다양한 마우스 메시지의 **WPARAM** 매개변수와 동일하며, 마우스 버튼 상태(즉, 이벤트 발생 시 누른 마우스 버튼)를 저장합니다. 두 번째와 세 번째 매개변수는 마우스 커서의 클라이언트 영역 (x, y) 좌표입니다.

4.4.4 프레임 통계

게임과 그래픽 애플리케이션에서는 초당 렌더링되는 프레임 수(FPS)를 측정하는 것이 일반적입니다. 이를 위해 특정 시간 간격 *t* 동안 처리된 프레임 수를 단순히 계산(*변수 n에 저장*)합니다. 그러면 시간 간격 *t* 동안의 평균 FPS는 $fps_{avg} = n/t$ 가 됩니다. $t = 1$ 로 설정하면 $fps_{avg} = n/1 = n$ 이 됩니다. 코드에서는 $t = 1$ 초를 사용하는데, 이는 나눗셈을 피할 수 있을 뿐만 아니라 1초가 너무 길지도 짧지도 않아 꽤 괜찮은 평균값을 제공하기 때문입니다. FPS를 계산하는 코드는 **D3DApp::CalculateFrameStats** 메서드에서 제공합니다:

```
void D3DApp::CalculateFrameStats()
{
    // 초당 평균 프레임 수와
    // 한 프레임을 렌더링하는 데 걸리는 평균 시간도 계산합니다. 이 통계는
    // 창 제목 표시줄에 추가됩니다.
```



```
static int frameCnt = 0;
static float timeElapsed = 0.0f; frameCnt++;

// 1초 동안의 평균값을 계산합니다.
if( mTimer.TotalTime() - timeElapsed) >= 1.0f )
{
    float fps = (float)frameCnt; // fps = frameCnt / 1 float mspf = 1000.0f / fps;

    std::wostringstream outs; outs.precision(6);
    outs << mMainWndCaption << L" "
        << L"FPS: " << fps << L" "
        << L"프레임 시간: " << mspf << L" (ms)"; SetWindowText(mhMainWnd,
    outs.str().c_str());

    // 다음 평균값을 위해 초기화. frameCnt
    = 0; timeElapsed += 1.0f;
}
}
```

이 메서드는 프레임을 계산하기 위해 매 프레임마다 호출됩니다.
FPS 계산 외에도, 이전 코드는 프레임 처리에 평균적으로 소요되는 밀리초 수를 계산합니다.

:

```
float mspf = 1000.0f / fps;
```

 **초당 프레임 수는 단순히 FPS의 역수이지만, 초 단위에서 밀리초 단위로 변환하기 위해 1000 밀리초 / 1 초를 곱합니다(1 초당 1000 밀리초임을 기억하십시오).**

이 라인의 목적은 프레임 렌더링에 소요되는 시간을 밀리초 단위로 계산하는 것입니다. 이는 FPS와는 다른 값이지만(단, 이 값은 FPS로부터 도출될 수 있음), 실제로 프레임 렌더링 시간은 FPS보다 유용합니다. 장면을 수정할 때 프레임 렌더링 시간의 증감을 직접 확인할 수 있기 때문입니다. 반면 FPS는 장면을 수정할 때 시간 증가/감소를 즉시 알려주지 않습니다. 더욱이 [Dunlop03]이 논문 *FPS* 대 프레임 시간에서 지적했듯이, FPS 곡선의 비선형성으로 인해 FPS를 사용하면 오해의 소지가 있는 결과를 초래할 수 있습니다. 예를 들어 상황 (1)을 고려해 보겠습니다: 애플리케이션이 1000 FPS로 실행되며 프레임 렌더링에 1ms(밀리초)가 소요된다고 가정합니다. 프레임 속도가 250 FPS로 떨어지면 프레임 렌더링에 4ms가 소요됩니다. 이제 상황 (2)를 살펴보겠습니다: 애플리케이션이 100 FPS로 실행되며 프레임 렌더링에 10ms가 소요된다고 가정합니다. 프레임 속도가 약 76.9 FPS로 떨어지면 프레임 렌더링에 약 13ms가 소요됩니다. 두 상황 모두 프레임당 렌더링 시간이 3ms 증가했으며, 따라서 프레임 렌더링에 소요되는 시간 증가 폭은 동일합니다. FPS를 읽는 것은 그렇게 직관적이지 않습니다. 1000 FPS에서 250 FPS로의 하락은 100 FPS에서 76.9 FPS로의 하락보다 훨씬 더 극적으로 보입니다. 그러나 방금 보여드렸듯이, 실제로는 프레임 렌더링에 소요되는 시간 증가가 동일합니다.

4.4.5 메시지 핸들러

애플리케이션 프레임워크를 위해 구현한 윈도우 프로시저는 최소한의 기능만 수행합니다. 일반적으로 Win32 메시지를 많이 다룰 일은 거의 없습니다. 사실, 애플리케이션 코드의 핵심 부분은 유틸 처리 중(즉, 윈도우 메시지가 존재하지 않을 때)에 실행됩니다. 그럼에도 처리해야 할 중요한 메시지들이 존재합니다. 다만 윈도우 프로시저의 길이를 고려하여 모든 코드를 여기에 포함시키지 않고, 각 메시지 처리의 배경과 필요성을 설명하는 데 중점을 둡니다. 독자 여러분께서는 소스 코드 파일을 다운로드하여 애플리케이션 프레임워크 코드를 숙지하시길 권장합니다. 이는 본 서적의 모든 예제 코드의 기반이 되기 때문입니다.

우리가 처리하는 첫 번째 메시지는 WM_ACTIVATE 메시지입니다. 이 메시지는 애플리케이션이 활성화되거나 비활성화될 때 전송됩니다. 우리는 다음과 같이 구현합니다:

```
case WM_ACTIVATE:
    if(LOWORD(wParam) == WA_INACTIVE)
    {
        mAppPaused = true; mTimer.Stop();
    }
```

```

else
{
    mAppPaused = false; mTimer.Start();
}
return 0;

```

보시다시피, 애플리케이션이 비활성화되면 데이터 멤버 **mAppPaused**를 **true**로 설정하고, 애플리케이션이 활성화되면 데이터 멤버 **mAppPaused**를 **false**로 설정합니다. 또한 애플리케이션이 일시 중지되면 타이머를 중지하고, 애플리케이션이 다시 활성화되면 타이머를 재개합니다. **D3DApp::Run** 구현(§4.3.3)을 다시 살펴보면, 애플리케이션이 일시 중지된 경우 애플리케이션 코드를 업데이트하지 않고 대신 일부 CPU 사이클을 OS에 반환합니다. 이렇게 하면 애플리케이션이 비활성 상태일 때 CPU 사이클을 독점하지 않습니다.

다음으로 처리할 메시지는 **WM_SIZE** 메시지입니다. 이 메시지는 창 크기가 변경될 때 호출된다는 점을 상기하십시오. 주요 이 메시지를 처리하는 이유는 백 버퍼와 깊이/스텐실 버퍼의 크기를 클라이언트 영역 사각형의 크기와 일치시키기 위함입니다(이렇게 하면 늘어나지 않음). 따라서 창 크기가 변경될 때마다 버퍼 크기도 재조정해야 합니다. 버퍼 크기 조정 코드는 **D3DApp::OnResize**에 구현되어 있습니다. 앞서 언급한 바와 같이, 백 버퍼는 **IDXGISwapChain::ResizeBuffers** 메서드를 호출하여 크기를 조정할 수 있습니다. 깊이/스텐실 버퍼는 파괴된 후 새로운 크기에 맞춰 재구성되어야 합니다. 또한 렌더 타겟과 깊이/스텐실 뷰도 재생성해야 합니다. 사용자가 크기 조정 막대를 드래그하는 경우 주의가 필요합니다. 크기 조정 막대 드래그 시 지속적인 **WM_SIZE** 메시지가 전송되는데, 버퍼를 계속해서 크기 조정하는 것은 바람직하지 않기 때문입니다. 따라서 사용자가 드래그로 크기 조정 중이라고 판단되면, 사용자가 크기 조정 막대 드래그를 완료할 때까지 실제로 아무 작업도 수행하지 않습니다(응용 프로그램 일시 정지 제외). 이를 위해 **WM_EXITSIZEMOVE** 메시지를 처리하면 됩니다. 이 메시지는 사용자가 크기 조정 막대를 놓을 때 전송됩니다.

```

// WM_ENTERSIZEMOVE는 사용자가 크기 조정 막대를 잡을 때 전송됩니다. case WM_ENTERSIZEMOVE:
    mAppPaused = true; mResizing
    = true; mTimer.Stop(); return 0;

```

// **WM_EXITSIZEMOVE**는 사용자가 크기 조정 막대를 놓을 때 전송됩니다.

```

// 여기서 새 창 크기에 따라 모든 것을 재설정합니다. case WM_EXITSIZEMOVE:
    mAppPaused = false; mResizing =
    false; mTimer.Start(); OnResize();
    return 0;

```

다음으로 처리하는 세 가지 메시지는 간단하게 구현되므로 코드만 보여드립니다:

```

// WM_DESTROY는 창이 파괴될 때 전송됩니다. case WM_DESTROY:
    PostQuitMessage(0); return 0;

```

// **WM_MENUCHAR** 메시지는 메뉴가 활성화된 상태에서 사용자가

```

// 아무런 니모닉 키나 단축키에도 대응하지 않는 키를 누를 때 전송됩니다. case WM_MENUCHAR:

    // Alt+Enter 시 비프음 발생 금지.
    return MAKELRESULT(0, MNC_CLOSE);

```

// 이 메시지를 처리하여 창이 지나치게 작아지는 것을 방지합니다. case **WM_GETMINMAXINFO**:

```

((MINMAXINFO*)lParam)->ptMinTrack Size.x = 200;
((MINMAXINFO*)lParam)->ptMinTrack Size.y = 200;
return 0;

```

마지막으로, 마우스 입력 가상 함수를 지원하기 위해 다음과 같은 마우스 메시지를 처리합니다:

```

case WM_LBUTTONDOWN:

```

```

case WM_MBUTTONDOWN:
case WM_RBUTTONDOWN:
    OnMouseDown(wParam, GET_X_LPARAM(lParam), GET_Y_LPARAM(lParam)); return 0;

case WM_LBUTTONUP:
case WM_MBUTTONUP:
case WM_RBUTTONUP:
    OnMouseUp(wParam, GET_X_LPARAM(lParam), GET_Y_LPARAM(lParam)); return 0;

case WM_MOUSEMOVE:
    OnMouseMove(wParam, GET_X_LPARAM(lParam), GET_Y_LPARAM(lParam)); return 0;

```

Note: GET_X_LPARAM 및 GET_Y_LPARAM 매크로를 사용하려면 <Windowsx.h>를 #include해야 합니다.

4.4.6 전체 화면으로 전환

우리가 생성한 IDXGISwapChain 인터페이스는 자동으로 ALT-ENTER 키 조합을 감지하여 애플리케이션을 전체 화면 모드로 전환합니다. 전체 화면 모드에서 ALT-ENTER를 누르면 다시 창 모드로 전환됩니다. 모드 전환 중에는 애플리케이션 창 크기가 조정되며, 이 과정에서 WM_SIZE 메시지가 애플리케이션에 전송됩니다. 이를 통해 애플리케이션은 새로운 화면 크기에 맞춰 백 버퍼와 깊이/스텐실 버퍼의 크기를 조정할 수 있습니다. 또한 전체 화면 모드로 전환할 경우 창 스타일이 전체 화면에 적합한 스타일로 변경됩니다. Visual Studio Spy++ 도구를 사용하여 데모 애플리케이션 중 하나에서 ALT-ENTER 실행 시 생성되는 Windows 메시지를 확인할 수 있습니다.

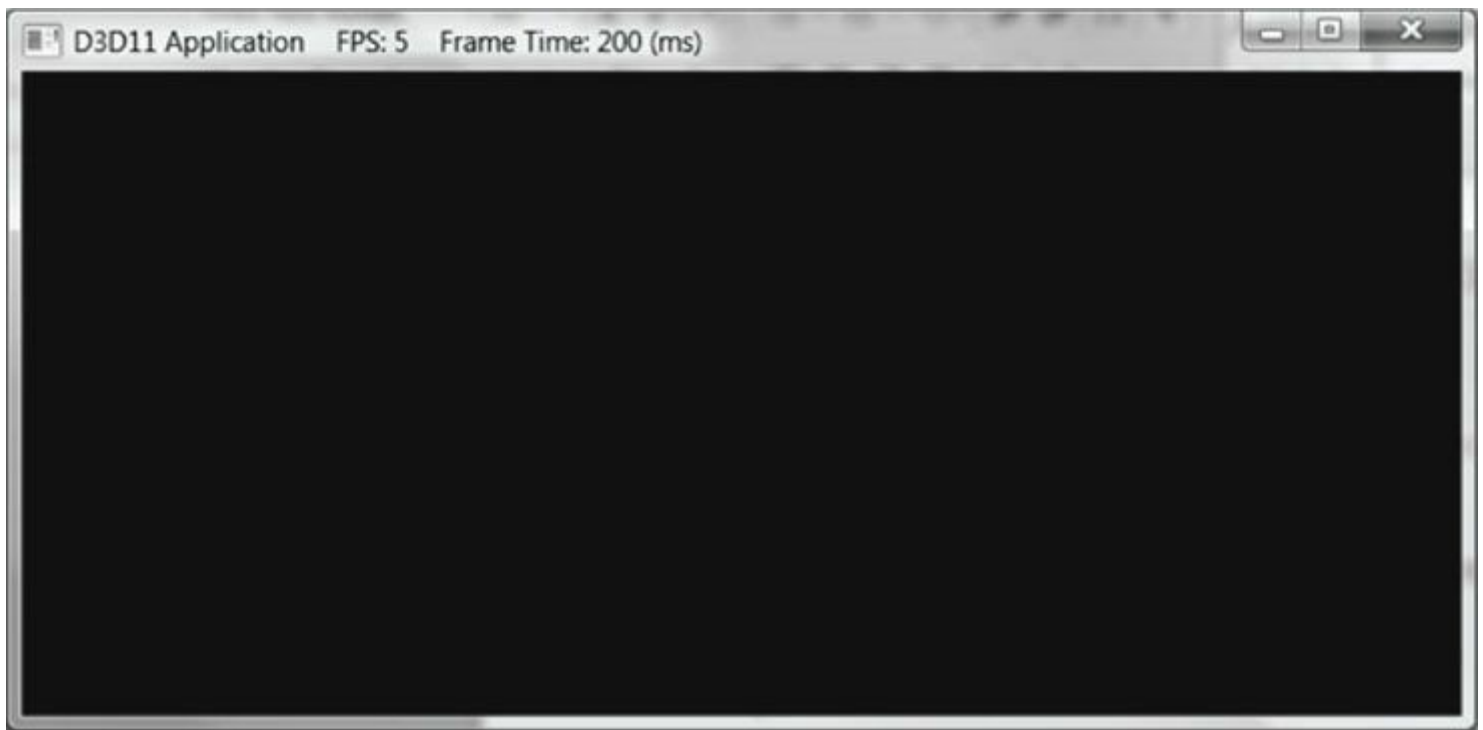


그림 4.10. 제4장 샘플 프로그램의 스크린샷.

연습 문제에서는 필요 시 기본 ALT-ENTER 기능을 비활성화하는 방법을 탐구합니다. 독자는 §4.2.3의 DXGI_SWAP_CHAIN_DESC::Flags 설명을 참고할 수 있습니다.

4.4.7 "Direct3D 초기화" 데모

이제 애플리케이션 프레임워크에 대해 논의했으니, 이를 활용해 간단한 애플리케이션을 만들어 보겠습니다. 부모 클래스 D3DApp이 이 데모에 필요한 대부분의 작업을 수행하므로, 실제 작업은 거의 필요하지 않습니다. 주목할 점은 D3DApp에서 클래스를 파생시키고 프레임워크 함수를 구현하는 방식이며, 여기에 샘플별 특정 코드를 작성하게 됩니다. 이 책의 모든 프로그램은 동일한 템플릿을 따릅니다.

```
#include "d3dApp.h"
```

```

class InitDirect3DApp : public D3DApp
{
public:
    InitDirect3DApp(HINSTANCE hInstance);
    ~InitDirect3DApp();

    bool Init();
    void OnResize();
    void UpdateScene(float dt); void
    DrawScene();

};

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE prevInstance, PSTR cmdLine, int showCmd)
{
    // 디버그 빌드 시 런타임 메모리 검사 활성화.
#ifdef defined(DEBUG) | defined(_DEBUG)
    _CrtSetDbgFlag( _CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF ); #endif

    InitDirect3DApp theApp(hInstance); if( !theApp.Init() )
        return 0;

    return theApp.Run();
}

InitDirect3DApp::InitDirect3DApp(HINSTANCE hInstance)
: D3DApp(hInstance)
{

}

InitDirect3DApp::~InitDirect3DApp()
{
}

bool InitDirect3DApp::Init()
{
    if(!D3DApp::Init()) return false;

    return true;
}

void InitDirect3DApp::OnResize()
{
    D3DApp::OnResize();
}

void InitDirect3DApp::UpdateScene(float dt)
{

}

void InitDirect3DApp::DrawScene()
{
    assert(md3dImmediateContext); assert(mSwapChain);

    // 백 버퍼를 파란색으로 지움. Colors::Blue는 d3dUtil.h에 정의됨. md3dImmediateContext-
    >ClearRenderTargetView(mRenderTargetView,
        reinterpret_cast<const float*>(&Colors::Blue));

```

```
// 깊이 버퍼를 1.0f로, 스텔실 버퍼를 0으로 초기화합니다. md3dImmediateContext-
>ClearDepthStencilView(mDepthStencilView,
    D3D11_CLEAR_DEPTH|D3D11_CLEAR_STENCIL, 1.0f, 0);

// 백 버퍼를 화면에 출력합니다. HR(mSwapChain->Present(0,
0));

}
```

4.5 DIRECT3D 애플리케이션 디버깅

코드를 간결하게 하고 방해 요소를 최소화하기 위해, 이 책에서는 주로 오류 처리를 생략합니다. 그러나 많은 Direct3D 함수가 반환하는 **HRESULT** 반환 코드를 확인하는 매크로는 구현합니다. *d3dUtil.h*에서 매크로는 다음과 같이 정의됩니다:

```
#if defined(DEBUG) | defined(_DEBUG) #ifndef HR
#define HR(x) \
{ \
    HRESULT hr = (x); \
    if(hr이 실패함) \
    { \
        DXTrace(__ FILE__, (DWORD)__ LINE__, hr, L#x, true); \
    } \
}
#endif

#else
#ifndef HR #define HR(x)
(x) #endif
#endif
```

반환된 함수의 반환 코드가 실패를 나타내는 경우, 반환 코드를 **DXTrace** 함수(**#include** **<dxerr.h>** 및 *dxerr.lib* 링크):

```
HRESULT WINAPI DXTraceW(const char* strFile, DWORD dwLine, HRESULT hr, const WCHAR*
strMsg, BOOL bPopMsgBox);
```

이 함수는 오류가 발생한 파일과 줄 번호, 오류에 대한 텍스트 설명, 그리고 오류를 발생시킨 함수의 이름을 표시하는 메시지 상자를 표시합니다. [그림 4.11](#)은 예시를 보여줍니다. **DXTrace**의 마지막 매개변수에 **false**를 지정하면 메시지 상자 대신 디버그 정보가 Visual C++ 출력 창에 출력됩니다. 디버그 모드가 아닐 경우 **HR** 매크로는 아무 작업도 수행하지 않습니다. 또한 **HR**은 함수가 아닌 매크로여야 합니다. 그렇지 않으면 **FILE**과 **LINE**이 함수 **HR**이 호출된 파일 및 줄이 아닌 함수 구현의 파일 및 줄을 참조하게 됩니다.

이 매크로를 사용하려면 **HRESULT**를 반환하는 Direct3D 함수를 다음과 같이 매크로로 감싸기만 하면 됩니다:

```
HR(D3DX11CreateShaderResourceViewFromFile(md3dDevice, L"grass.dds", 0, 0,
&mGrassTexRV, 0 ));
```

이 방법은 데모 디버깅에는 효과적이지만, 실제 애플리케이션은 출시 후 발생할 수 있는 오류(예: 지원되지 않는 하드웨어, 누락된 파일 등)를 처리해야 합니다.

 *L#x*는 **HR** 매크로의 인자 토큰을 유니코드 문자열로 변환합니다. 이를 통해 오류가 발생한 함수 호출을 메시지 상자에 출력할 수 있습니다.

4.6 요약

1. Direct3D는 프로그래머와 그래픽스 하드웨어 사이의 중개자 역할을 한다고 볼 수 있습니다. 예를 들어, 프로그래머는 Direct3D 함수를 호출하여 리소스 뷰를 하드웨어 렌더링 파이프라인에 바인딩하고, 렌더링 파이프라인의 출력을 구성하며, 3D 지오메트리를 그립니다.
2. Direct3D 11에서는 Direct3D 11을 지원하는 그래픽스 디바이스가 몇 가지 예외를 제외하고 전체 Direct3D 11 기능 세트를 지원해야 합니다.
3. COM(Component Object Model)은 DirectX가 언어 독립적이고 하위 호환성을 갖도록 하는 기술입니다. Direct3D 프로그래머는 COM의 세부 사항과 작동 방식을 알 필요가 없으며, COM 인터페이스를 획득하고 해제하는 방법만 알면 됩니다.
4. 1차원 텍스처는 1차원 데이터 요소 배열과 같고, 2차원 텍스처는 2차원 데이터 요소 배열과 같으며, 3차원 텍스처는 3차원 데이터 요소 배열과 같습니다. 텍스처의 요소들은 **DXGI_FORMAT** 열거형의 멤버로 설명되는 형식을 가져야 합니다. 텍스처는 일반적으로 이미지 데이터를 포함하지만, 깊이 정보(예: 깊이 버퍼)와 같은 다른 데이터도 포함할 수 있습니다. GPU는 텍스처에 대해 필터링이나 멀티샘플링과 같은 특수 작업을 수행할 수 있습니다.
5. Direct3D에서는 리소스가 파이프라인에 직접 바인딩되지 않습니다. 대신 리소스 뷰가 파이프라인에 바인딩됩니다. 단일 리소스에 대해 서로 다른 뷰를 생성할 수 있습니다. 이렇게 하면 단일 리소스를 렌더링 파이프라인의 서로 다른 단계에 바인딩할 수 있습니다. 리소스가 유형이 지정되지 않은 형식으로 생성된 경우 뷰 생성 시 유형을 명시해야 합니다.
6. **ID3D11Device** 및 **ID3D11DeviceContext** 인터페이스는 물리적 그래픽스 장치 하드웨어의 소프트웨어 제어로 생각할 수 있습니다. 즉, 이 인터페이스를 통해 하드웨어와 상호작용하고 작업을 지시할 수 있습니다. **ID3D11Device** 인터페이스는 기능 지원 확인 및 리소스 할당을 담당합니다. **ID3D11DeviceContext** 인터페이스는 렌더링 상태 설정, 그래픽스 파이프라인에 리소스 바인딩, 렌더링 명령 발행 등을 담당합니다.

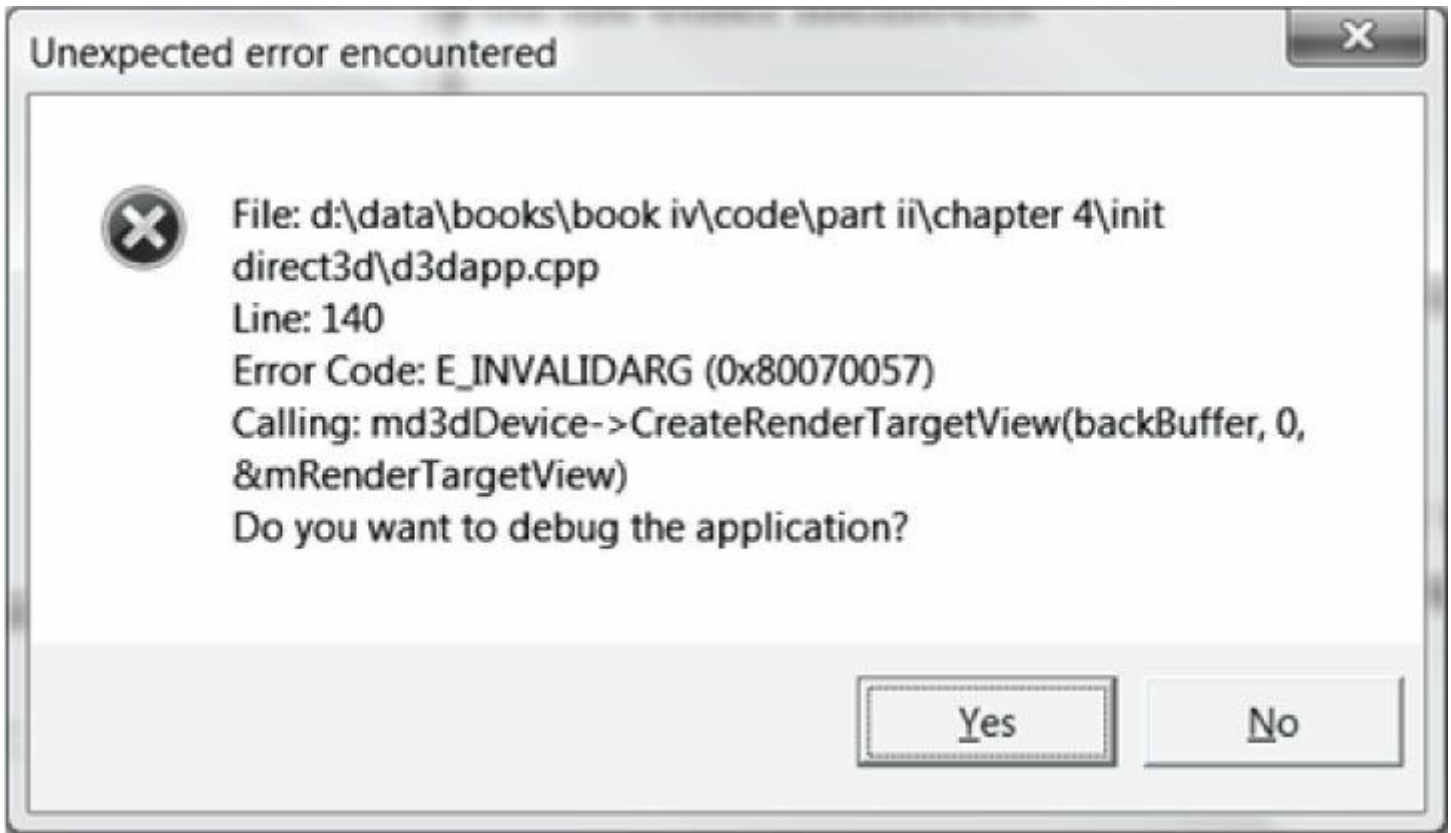


그림 4.11. Direct3D 함수가 오류를 반환할 경우 DXTrace 함수가 표시하는 메시지 상자.

7. 애니메이션에서 깜빡임을 방지하려면 전체 애니메이션 프레임을 백 버퍼(back buffer)라고 하는 오프 스크린 텍스처에 그려 넣는 것이 가장 좋습니다. 주어진 애니메이션 프레임에 대한 전체 장면이 백 버퍼에 그려지면 하나의 완전한 프레임으로 화면에 표시됩니다. 이렇게 하면 시청자가 프레임이 그려지는 과정을 보지 않게 됩니다. 프레임이 백 버퍼에 그려진 후, 백 버퍼와 프론트 버퍼의 역할이 반전됩니다. 백 버퍼는 다음 애니메이션 프레임의 프론트 버퍼가 되고, 프론트 버퍼는 백 버퍼가 됩니다. 백 버퍼와 프론트 버퍼의 역할을 바꾸는 것을 프레젠텐팅(presenting)이라고 합니다. 프론트 버퍼와 백 버퍼는 IDXGISwapChain 인터페이스로 표현되는 스왑 체인(swap chain)을 형성합니다. 두 개의 버퍼(프론트와 백)를 사용하는 것을 더블 버퍼링(double buffering)이라고 합니다.
8. 불투명한 장면 객체를 가정할 때, 카메라에 가장 가까운 점들은 그 뒤에 있는 모든 점을 가립니다. 깊이 버퍼링은 장면에서 카메라에 가장 가까운 점을 결정하는 기술입니다. 이를 통해 장면 객체를 그리는 순서에 대해 걱정할 필요가 없습니다.
9. 성능 카운터는 프레임 간 경과 시간과 같은 작은 시간 차이를 측정하는 데 필요한 정확한 타이밍 측정을 제공하는 고해상도 타이머입니다.

- 성능 타이머는 *카운트(counts)*라는 시간 단위로 작동합니다. `QueryPerformanceFrequency`는 성능 타이머의 초당 카운트 수를 출력하며, 이를 통해 카운트 단위를 초 단위로 변환할 수 있습니다. 성능 타이머의 현재 시간 값(카운트로 측정)은 `QueryPerformanceCounter` 함수로 획득합니다.
- 10.** 초당 평균 프레임 수(FPS)를 계산하려면, 일정 시간 간격 Δt 동안 처리된 프레임 수를 세어야 합니다. Δt 시간 동안 세어진 프레임 수를 n 이라고 하면, 해당 시간 간격의 초당 평균 프레임 수는 $fps_{avg} = n / \Delta t$ 입니다. 프레임 속도는 성능에 대해 오해의 소지가 있는 결론을 내릴 수 있습니다; 프레임 처리 시간은 더 유용한 정보를 제공합니다. 프레임 처리 시간(초)은 프레임 속도의 역수(즉, $1 / fps_{avg}$)입니다.
- 11.** 샘플 프레임워크는 이 책의 모든 데모 애플리케이션이 따르는 일관된 인터페이스를 제공하기 위해 사용됩니다. `d3dUtil.h`, `d3dApp.h` 및 `d3dApp.cpp` 파일에 제공된 코드는 모든 애플리케이션이 구현해야 하는 표준 초기화 코드를 감쌉니다. 이 코드를 감싸 숨김으로써, 샘플은 현재 주제를 시연하는 데 더 집중할 수 있습니다.
- 12.** 디버그 모드 빌드 시에는 디버그 계층을 활성화하기 위해 `D3D11_CREATE_DEVICE_DEBUG` 플래그로 `Direct3D` 디바이스를 생성하십시오. 디버그 플래그가 지정되면 `Direct3D`는 디버그 메시지를 `VC++` 출력 창으로 전송합니다. 또한 디버그 빌드에는 디버그 버전의 `D3DX` 라이브러리(예: `d3dx11d.lib`)를 사용하십시오.

4.7 연습 문제

- 1.** 전체 화면과 창 모드 간 전환을 위한 `ALT-ENTER` 기능을 비활성화하도록 이전 연습 문제의 솔루션을 수정하십시오. `IDXGIFactory::MakeWindowAssociation` 메서드를 사용하고 `DXGI_MWA_NO_WINDOW_CHANGES` 플래그를 지정하여 `DXGI`가 메시지 큐를 모니터링하지 않도록 하십시오. `IDXGIFactory::MakeWindowAssociation` 메서드는 `IDXGIFactory::CreateSwapChain` 호출 후에 호출해야 합니다.
- 2.** 일부 시스템에는 하나 이상의 어댑터(비디오 카드)가 있으며, 애플리케이션은 항상 기본 어댑터를 사용하는 대신 사용자가 사용할 어댑터를 선택할 수 있도록 할 수 있습니다. `IDXGIFactory::EnumAdapters` 메서드를 사용하여 시스템에 있는 어댑터의 수를 확인하십시오.
- 3.** 시스템이 보유한 각 어댑터에 대해 `IDXGIFactory::EnumAdapters`는 채워진 `IDXGIAdapter` 인터페이스에 대한 포인터를 출력합니다. 이 인터페이스를 사용하여 어댑터에 대한 정보를 쿼리할 수 있습니다. `IDXGIAdapter::Check InterfaceSupport` 메서드를 사용하여 시스템의 어댑터가 `Direct3D 11`을 지원하는지 확인할 수 있습니다.
- 4.** 어댑터에는 출력 장치(예: 모니터)가 연결됩니다. `IDXGIAdapter::EnumOutputs` 메서드를 사용하여 특정 어댑터의 출력 장치를 열거할 수 있습니다. 기본 어댑터의 출력 장치 수를 확인하려면 이 메서드를 사용하십시오.
- 5.** 각 출력에는 지정된 픽셀 형식에 대해 지원되는 디스플레이 모드(`DXGI_MODE_DESC`) 목록이 있습니다. 각 출력(`IDXGIOutput`)에 대해, 너비, 높이, 및 재생 속도를 표시합니다. 각 디스플레이 모드의 출력은 지원됩니다. `IDXGIOutput::GetDisplayModeList` 메서드를 사용하여 `DXGI_FORMAT_R8G8B8A8_UNORM` 형식을 지원합니다. 연습 2, 3, 4, 5에 대한 예제 출력은 아래에 나열되어 있습니다. `OutputDebugString` 함수를 사용하여 `VC++` 출력 창에 빠르게 출력하는 데 유용합니다.

```
*** NUM ADAPTERS = 1
*** 어댑터 0에 대해 D3D11 지원됨
*** 기본 어댑터의 출력 수 = 1
*** 너비 = 640 높이 = 480 재생률 = 60000/1000
*** 너비 = 640 높이 = 480 재생률 = 72000/1000
***너비 = 640 높이 = 480 재생률 = 75000/1000
***너비 = 720 높이 = 480 새로그침 = 56250/1000
***너비 = 720 높이 = 480 새로그침 = 56250/1000
***너비 = 720 높이 = 480 새로그침 = 60000/1000
***너비 = 720 높이 = 480 새로그침 = 60000/1000
***너비 = 720 높이 = 480 새로그침 = 72188/1000
***너비 = 720 높이 = 480 새로그침 = 72188/1000
***너비 = 720 높이 = 480 새로그침 = 75000/1000
***너비 = 720 높이 = 480 새로그침 = 75000/1000
***너비 = 720 높이 = 576 새로그침 = 56250/1000
***너비 = 720 높이 = 576 새로그침 = 56250/1000
***너비 = 720 높이 = 576 새로그침 = 60000/1000
***너비 = 720 높이 = 576 새로그침 = 60000/1000
***너비 = 720 높이 = 576 새로그침 = 72188/1000
***너비 = 720 높이 = 576 새로그침 = 72188/1000
***너비 = 720 높이 = 576 새로그침 = 75000/1000
```

***너비 = 720 높이 = 576 새로그침 = 75000/1000

***너비 = 800 높이 = 600 새로그침 = 56250/1000

***너비 = 800 높이 = 600 새로그침 = 60000/1000

***너비 = 800 높이 = 600 새로그침 = 60000/1000

***너비 = 800 높이 = 600 새로그침 = 72188/1000

***너비 = 800 높이 = 600 새로그침 = 75000/1000

***너비 = 848 높이 = 480 새로그침 = 60000/1000

***너비 = 848 높이 = 480 새로그침 = 60000/1000

***너비 = 848 높이 = 480 새로그침 = 70069/1000

***너비 = 848 높이 = 480 새로그침 = 70069/1000

***너비 = 848 높이 = 480 새로그침 = 75029/1000

***너비 = 848 높이 = 480 새로그침 = 75029/1000

***너비 = 960 높이 = 600 새로그침 = 60000/1000

***너비 = 960 높이 = 600 새로그침 = 60000/1000

***너비 = 960 높이 = 600 새로그침 = 70069/1000

***너비 = 960 높이 = 600 새로그침 = 70069/1000

***너비 = 960 높이 = 600 새로그침 = 75029/1000

***너비 = 960 높이 = 600 새로그침 = 75029/1000

***너비 = 1024 높이 = 768 새로그침 = 60000/1000

***너비 = 1024 높이 = 768 새로그침 = 60000/1000

***너비 = 1024 높이 = 768 새로그침 = 70069/1000

***너비 = 1024 높이 = 768 새로그침 = 75029/1000

***너비 = 1152 높이 = 864 새로그침 = 60000/1000

***너비 = 1152 높이 = 864 새로그침 = 60000/1000

***너비 = 1152 높이 = 864 새로그침 = 75000/1000

***너비 = 1280 높이 = 720 새로그침 = 60000/1000

***너비 = 1280 높이 = 720 새로그침 = 60000/1000

***너비 = 1280 높이 = 720 새로그침 = 60000/1001

***너비 = 1280 높이 = 768 새로그침 = 60000/1000

***너비 = 1280 높이 = 768 새로그침 = 60000/1000

***너비 = 1280 높이 = 800 새로그침 = 60000/1000

***너비 = 1280 높이 = 800 새로그침 = 60000/1000

***너비 = 1280 높이 = 960 새로그침 = 60000/1000

***너비 = 1280 높이 = 960 새로그침 = 60000/1000

***너비 = 1280 높이 = 1024 새로그침 = 60000/1000

***너비 = 1280 높이 = 1024 새로그침 = 60000/1000

***너비 = 1280 높이 = 1024 새로그침 = 75025/1000

***너비 = 1360 높이 = 768 새로그침 = 60000/1000

***너비 = 1360 높이 = 768 새로그침 = 60000/1000

***너비 = 1600 높이 = 1200 새로그침 = 60000/1000

6. 뷰포트 설정을 수정하여 백 버퍼의 하위 사각형에 장면을 그리는 실험을 해보세요. 예를 들어, 다음을 시도해 보세요:

```
D3D11_VIEWPORT vp;
vp.TopLeftX = 100.0f;
vp.TopLeftY = 100.0f; vp.Width =
500.0f; vp.Height = 400.0f;
vp.MinDepth = 0.0f; vp.MaxDepth
= 1.0f;
```


CHAPTER 5 렌더링

파이프라인

이 장의 주요 주제는 렌더링 파이프라인입니다. 위치와 방향이 지정된 가상 카메라를 가진 3D 장면의 기하학적 묘사가 주어졌을 때, *렌더링 파이프라인*은 가상 카메라가 보는 것을 바탕으로 2D 이미지를 생성하는 데 필요한 전체 단계 순서를 의미합니다([그림 5.1](#)). 이 장은 주로 이론적 내용으로 구성되어 있으며, 다음 장에서는 Direct3D를 사용하여 그리기 방법을 배우면서 이론을 실제 적용해 보겠습니다. 렌더링 파이프라인을 다루기 전에 두 가지 간단한 내용을 살펴보겠습니다. 첫째, 3D 환상(즉, 평평한 2D 모니터 화면을 통해 3D 세계를 들여다보고 있다는 착각)의 몇 가지 요소를 논의합니다. 둘째, 색상이 수학적으로 그리고 Direct3D 코드에서 어떻게 표현되고 처리되는지 설명합니다.

목표:

1. 2D 이미지에서 현실적인 볼륨감과 공간적 깊이감을 전달하는 데 사용되는 몇 가지 핵심 신호를 발견하기 위함입니다.
2. Direct3D에서 3D 객체를 어떻게 표현하는지 알아보기.
3. 가상 카메라를 모델링하는 방법을 학습합니다.
4. 렌더링 파이프라인—3D 장면의 기하학적 묘사를 받아 2D 이미지를 생성하는 과정—을 이해한다.

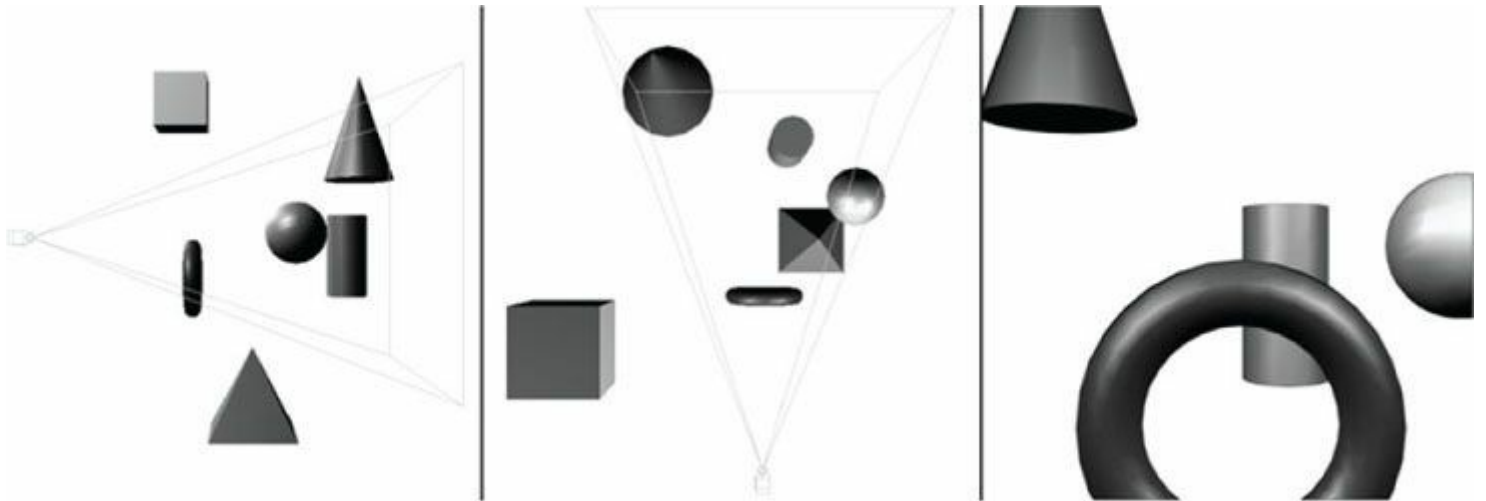


그림 5.1. 왼쪽 이미지는 카메라가 배치되고 조준된 상태에서 3D 세계에 설정된 일부 물체의 측면 뷰를 보여줍니다. 가운데 이미지는 동일한 장면을 위에서 내려다보는 뷰로 보여줍니다. "피라미드" 볼륨은 시청자가 볼 수 있는 공간의 부피를 지정합니다. 이 볼륨 밖에 있는 객체(및 객체의 일부)는 보이지 않습니다. 오른쪽 이미지는 카메라가 "보는" 것을 기반으로 생성된 2D 이미지를 보여줍니다.

5.1 3D 환상

3D 컴퓨터 그래픽스 여정을 시작하기 전에 간단한 질문이 남아 있습니다. 평평한 2D 모니터 화면에 깊이와 부피를 가진 3D 세계를 어떻게 표시할까요? 다행히도 이 문제는 수세기 동안 예술가들이 2D 캔버스에 3D 장면을 그려왔기 때문에 충분히 연구되었습니다. 이 섹션에서는 이미지가 실제로 2차원 평면에 그려졌음에도 3차원처럼 보이게 하는 몇 가지 핵심 기법을 개괄합니다.

만약 곡선 없이 오랫동안 직선으로 뻗어 있는 철로를 마주쳤다고 가정해 보자.

레일들은 선로 전체에 걸쳐 서로 평행하게 유지되지만, 선로 위에 서서 그 길을 따라 내려다보면 두 레일이 당신으로부터 멀어질수록 점점 가까워지다가 결국 무한한 거리에서 한 *지점*으로 수렴하는 것을 관찰할 수 있습니다. 이는 인간의 시각 시스템을 특징짓는 관찰 중 하나입니다: 평행한 시선은 *소실점*으로 수렴합니다; [그림 5.2](#)를 참조하십시오.

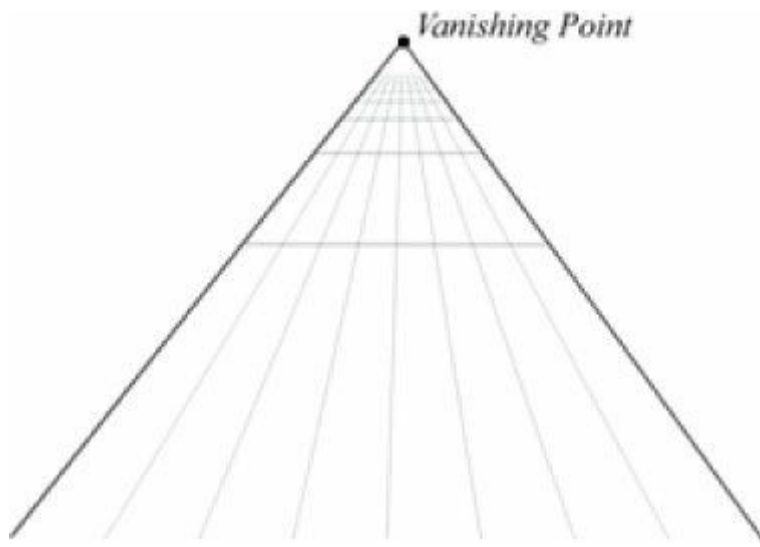


그림 5.2. 평행한 시선은 소실점으로 수렴한다. 예술가들은 이를 때로 선형 원근법이라 부른다.

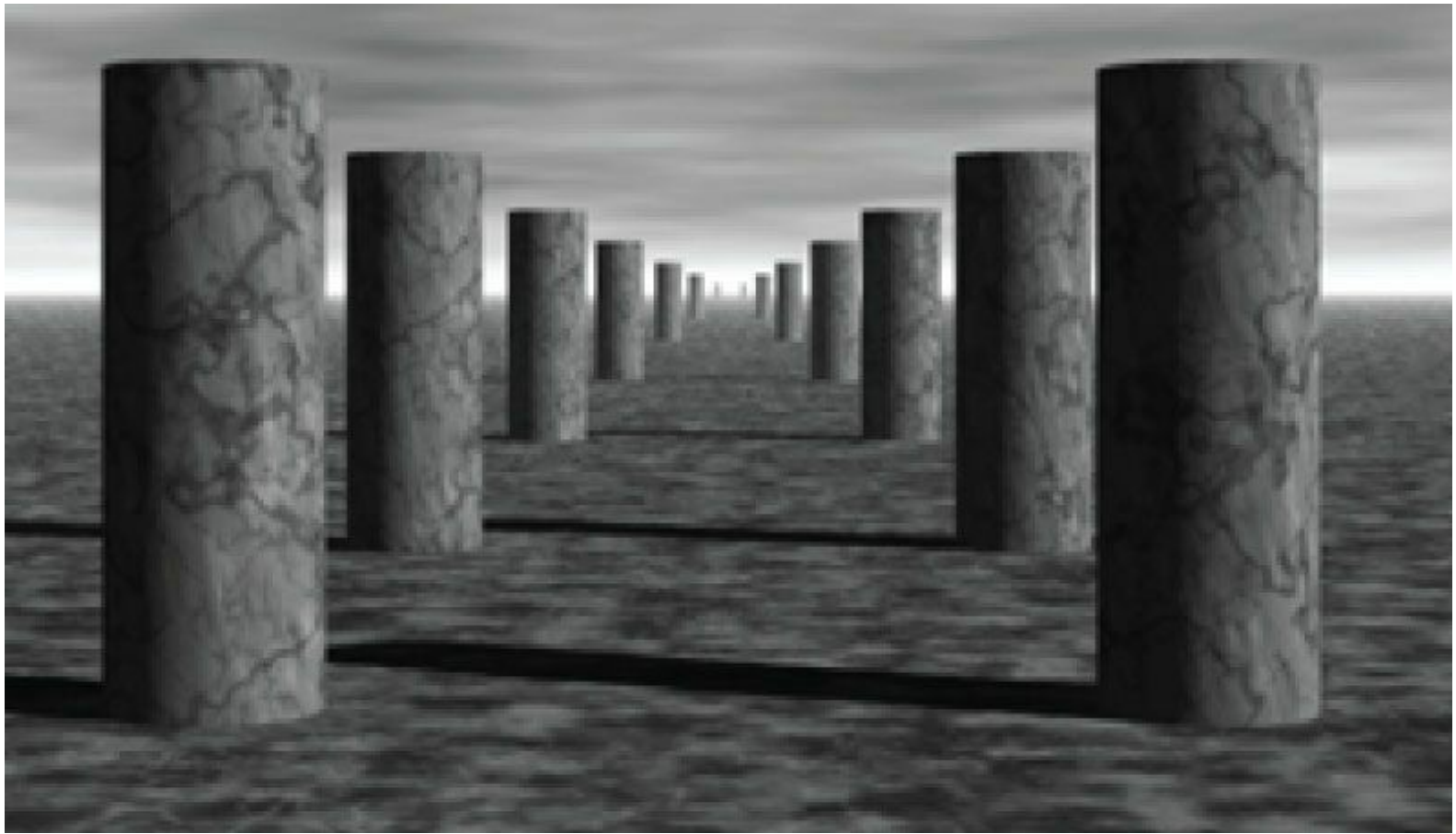


그림 5.3. 여기서는 모든 기둥의 크기가 동일하지만, 관찰자는 깊이에 따라 크기가 줄어드는 현상을 인식한다.

인간이 사물을 인식하는 또 다른 간단한 관찰 결과는 깊이에 따라 사물의 크기가 줄어드는 것처럼 보인다는 점이다. 즉, 가까운 사물은 먼 사물보다 더 크게 보인다. 예를 들어, 언덕 위에 멀리 있는 집은 매우 작게 보이지만, 우리 근처의 나무는 비교적 매우 크게 보인다. [그림 5.3](#)은 평행한 열의 기둥들이 서로 뒤에 배치된 단순한 장면을 보여줍니다. 기둥들은 실제로 모두 같은 크기이지만, 관찰자로부터 깊이가 증가함에 따라 점점 작아져 보입니다. 또한 기둥들이 지평선의 소실점으로 수렴하는 방식에도 주목하십시오.

우리는 모두 *객체 중첩*([그림 5.4](#))을 경험합니다. 이는 불투명한 객체가 그 뒤에 있는 객체의 일부(또는 전체)를 가린다는 사실을 의미합니다. 이것은 장면 내 객체들의 깊이 순서 관계를 전달하는 중요한 시각 현상이다. Direct3D가 깊이 버퍼를 활용해 가려진 픽셀을 식별하고 이를 렌더링하지 않는 방식은 이미 논의한 바 있다([제4장](#)).



그림 5.4. 서로 앞뒤로 위치하여 부분적으로 가려지는(겹치는) 객체 그림.

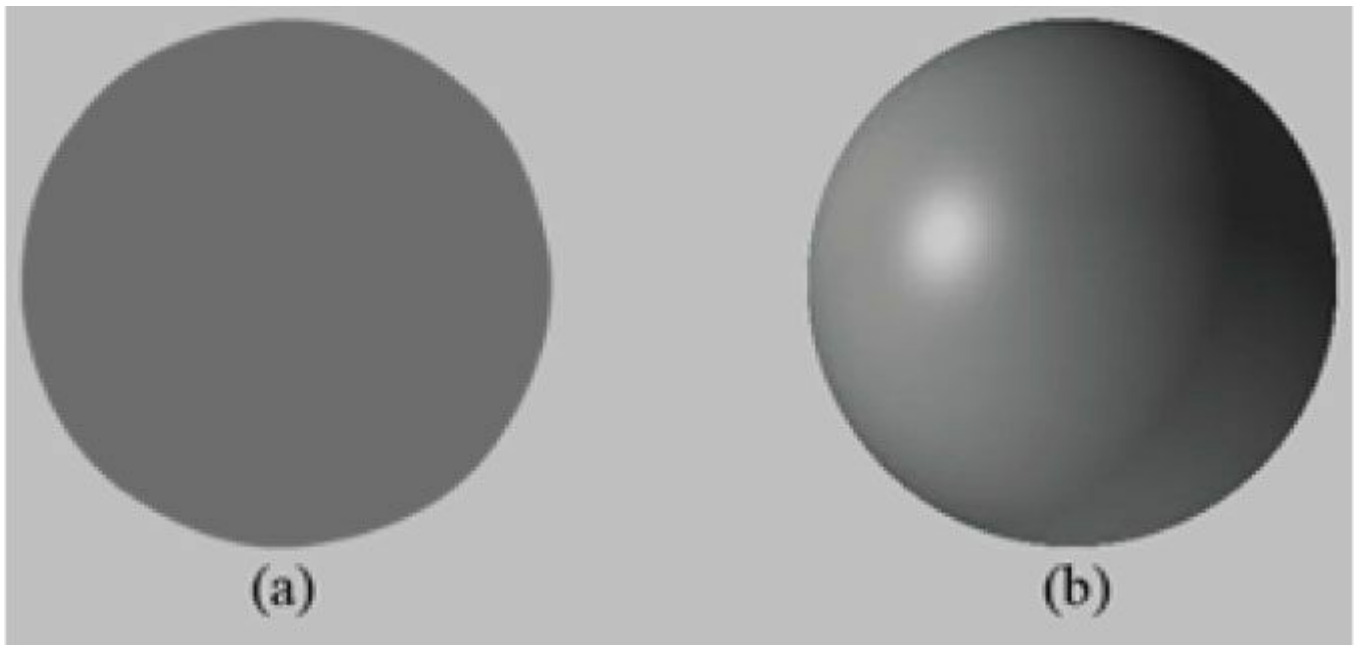


그림 5.5. (a) 2차원처럼 보이는 비조명 구체. (b) 3차원처럼 보이는 조명된 구체.

[그림 5.5](#)를 살펴보자. 왼쪽에는 조명이 없는 구체가, 오른쪽에는 조명이 있는 구체가 있다. 보시다시피 왼쪽의 구체는 상당히 평평해 보인다—아예 구체가 아니라 질감이 있는 2차원 원일 수도 있다! 따라서 조명과 음영은 3차원 물체의 입체적 형태와 부피를 표현하는 데 매우 중요한 역할을 한다.

마지막으로 [그림 5.6](#)은 우주선과 그 그림자를 보여줍니다. 그림자는 두 가지 핵심 목적을 수행합니다. 첫째, 빛의 발원지를 알려줍니다. 장면 속의 광원입니다. 둘째, 우주선이 지면에서 얼마나 높이 떠 있는지 대략적인 개념을 제공합니다.

방금 논의한 관찰 내용은 일상 경험에서 직관적으로 명백할 것입니다. 그럼에도 불구하고 우리가 알고 있는 내용을 명시적으로 밝히고, 3D 컴퓨터 그래픽스를 연구하고 작업할 때 이러한 관찰 사항들을 염두에 두는 것이 도움이 됩니다.



그림 5.6. 우주선과 그 그림자. 그림자는 장면 내 광원의 위치를 암시하며, 우주선이 지면에서 얼마나 높이 떠 있는지도 보여준다.

5.2 모델 표현

고체 3D 객체는 삼각형 메쉬 근사값으로 표현되며, 따라서 삼각형은 우리가 모델링하는 객체의 기본 구성 요소를 이룬다. 그림 5.7이 시사하듯, 우리는 삼각형 메쉬로 어떤 실제 3D 객체든 근사할 수 있다. 일반적으로 객체를 근사하는 데 사용하는 삼각형이 많을수록 더 정밀한 디테일을 모델링할 수 있으므로 근사도가 높아진다. 물론 삼각형을 많이 사용할수록 더 많은 처리 능력이 필요하므로, 애플리케이션 대상 사용자의 하드웨어 성능을 고려하여 균형을 맞춰야 합니다. 삼각형 외에도 선이나 점을 그리는 것이 유용한 경우가 있습니다. 예를 들어, 곡선은 픽셀 두께의 짧은 선분 시퀀스로 그래픽적으로 그릴 수 있습니다.

그림 5.7에서 사용된 다수의 삼각형은 한 가지를 분명히 보여줍니다: 이를 수동으로 일일이 나열하는 것은 매우 번거로운 것입니다.

3D 모델의 삼각형. 가장 단순한 모델을 제외한 모든 경우, 3D 모델러라고 불리는 특수 3D 애플리케이션을 사용하여 3D 객체를 생성하고 조작합니다. 이러한 모델러는 풍부한 도구 세트를 갖춘 시각적이고 상호작용적인 환경에서 사용자가 복잡하고 사실적인 메쉬를 구축할 수 있게 하여 전체 모델링 과정을 훨씬 쉽게 만듭니다. 게임 개발에 사용되는 대표적인 모델러로는 3D Studio Max(<http://usa.autodesk.com/3ds-max/>), LightWave 3D(<http://www.newtek.com/lightwave/>), Maya(<http://usa.autodesk.com/maya/>), Softimage|XSI(<http://www.softimage.com>), Blender(<http://www.blender.org>) 등이 있습니다. (블렌더는 오픈 소스이며 무료라는 점에서 취미로 하는 사람들에게 유리합니다.) 그럼에도 불구하고, 이 책의 첫 번째 부분에서는 3D 모델을 수동으로 직접 생성하거나 수학적 공식(예를 들어 원통과 구체의 삼각형 목록은 파라메트릭 공식으로 쉽게 생성할 수 있음)을 통해 생성할 것입니다. 이 책의 세 번째 부분에서는 3D 모델링 프로그램에서 내보낸 3D 모델을 불러와 표시하는 방법을 보여줍니다.

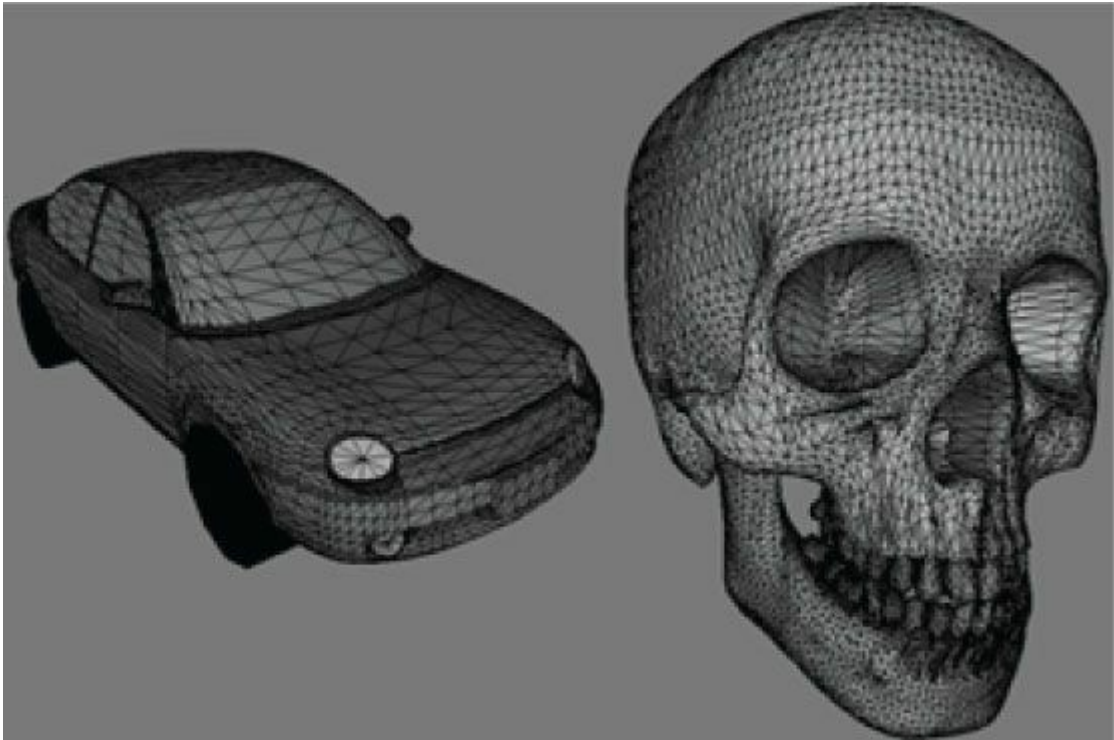


그림 5.7. (왼쪽) 삼각형 메시로 근사화된 자동차. (오른쪽) 삼각형 메시로 근사화된 두개골.

5.3 기본 컴퓨터 컬러

컴퓨터 모니터는 각 픽셀을 통해 빨강, 녹색, 파랑 빛의 혼합을 방출합니다. 이 빛 혼합이 눈에 들어와 망막의 특정 영역에 닿으면 원추 수용체 세포가 자극을 받아 신경 자극이 시신경을 따라 뇌로 전달됩니다. 뇌는 이 신호를 해석하여 색상을 생성합니다. 빛 혼합이 달라지면 세포가 다르게 자극받으며, 이는 다시 뇌에서 다른 색상을 생성합니다. 그림 5.8은 빨강, 녹색, 파랑을 혼합하여 다양한 색상을 얻는 몇 가지 예시와 함께 빨강의 다양한 명도를 보여줍니다. 각 색상 구성 요소에 서로 다른 명도를 적용하고 이를 혼합함으로써, 사실적인 이미지를 표시하는 데 필요한 모든 색상을 표현할 수 있습니다.

RGB(빨강, 녹색, 파랑) 값으로 색상을 표현하는 데 익숙해지는 가장 좋은 방법은 Adobe Photoshop과 같은 페인트 프로그램이나 Win32 ChooseColor 대화 상자(그림 5.9)를 사용하여 다양한 RGB 조합을 실험해 보고 그 결과로 생성되는 색상을 확인해 보는 것입니다.

모니터는 발광할 수 있는 적색, 녹색, 청색 빛의 최대 강도를 가집니다. 빛의 강도를 설명하기 위해 0부터 1까지의 정규화된 범위를 사용하는 것이 유용합니다. 0은 강도가 없음을, 1은 최대 강도를 나타냅니다. 중간 값은 중간 강도를 의미합니다. 예를 들어, (0.25, 0.67, 1.0)이라는 값은 빛의 혼합이 빨간색 빛의 25% 강도, 녹색 빛의 67% 강도, 그리고 파란색 빛의 100% 강도로 구성됨을 의미합니다. 방금 언급한 예시가 시사하듯, 우리는 색상을 3차원 색상 벡터 (r, g, b) 로 표현할 수 있습니다. 여기서 $0 \leq r, g, b \leq 1$ 이며, 각 색상 구성 요소는 혼합물 내 적색, 녹색, 청색 빛의 강도를 나타냅니다.

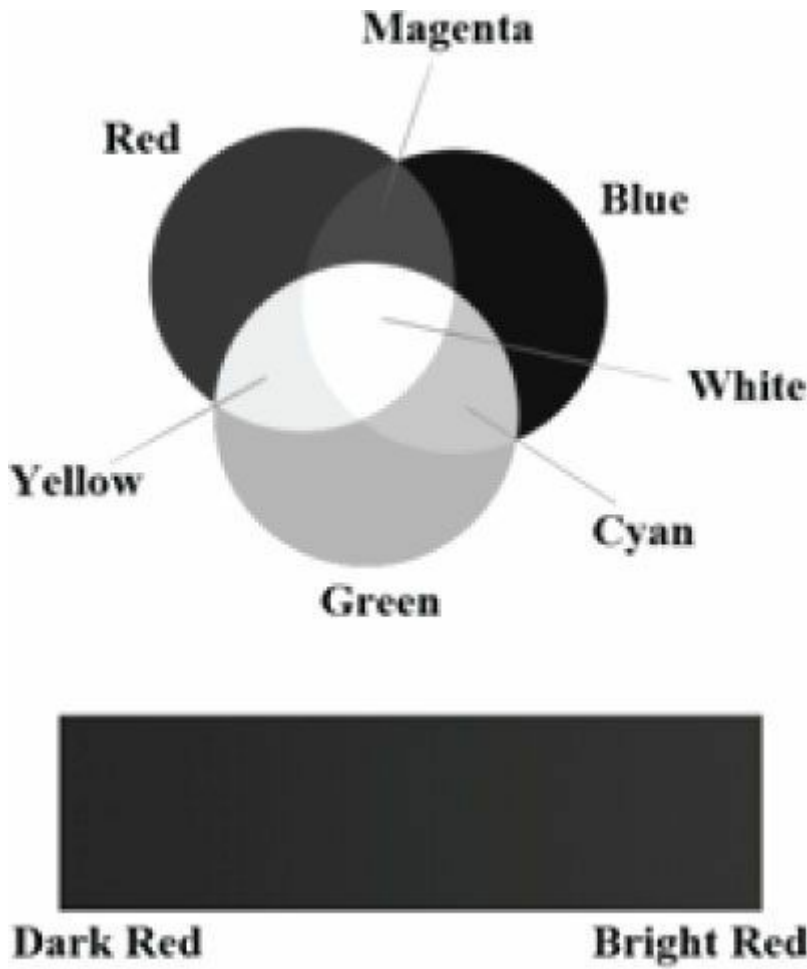


그림 5.8. (위) 순수한 빨강, 녹색, 파랑 색상을 혼합하여 새로운 색상을 얻는 과정. (아래) 빨간색 빛의 강도를 조절하여 얻은 다양한 빨간색 음영.

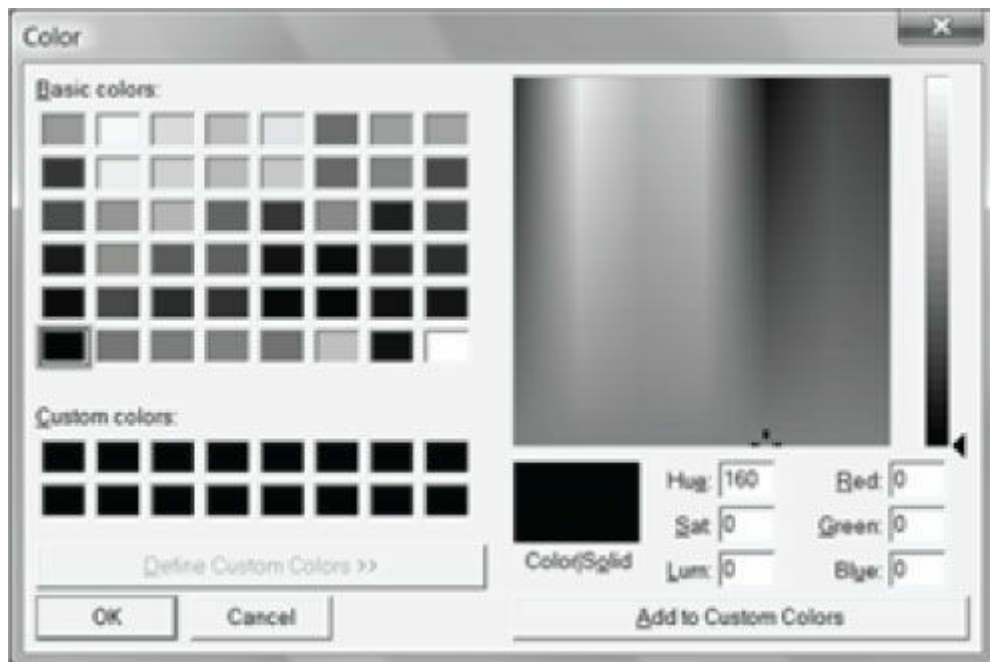


그림 5.9. 색상 선택 대화 상자.

5.3.1 색상 연산

일부 벡터 연산은 색상 벡터에도 적용됩니다. 예를 들어, 색상 벡터를 더하여 새로운 색상을 얻을 수 있습니다:

$$(0.0, 0.5, 0) + (0, 0.0, 0.25) = (0.0, 0.5, 0.25)$$

중간 강도의 녹색과 낮은 강도의 파란색을 결합하면 짙은 녹색을 얻을 수 있습니다.

색상을 빼서 새로운 색상을 얻을 수도 있습니다:

$$(1, 1, 1) - (1, 1, 0) = (0, 0, 1)$$

즉, 흰색에서 빨간색과 녹색 부분을 빼면 파란색이 됩니다.
스칼라 곱셈도 의미가 있습니다. 다음을 고려해 보세요:

$$0.5 (1, 1, 1) = (0.5, 0.5, 0.5)$$

즉, 흰색에서 시작하여 0.5를 곱하면 중간 회색조가 됩니다. 반면 $2(0.25, 0, 0) = (0.5, 0, 0)$ 연산은 적색 성분의 강도를 두 배로 증가시킵니다.
점적산이나 교차적산 같은 표현은 색상 벡터에 적용할 수 없습니다. 그러나 색상 벡터에는

변조(modulation) 또는 *성분별 곱셈(componentwise multiplication)*이라 불리는 특별한 색상 연산을 가집니다. 이는 다음과 같이 정의됩니다:

$$(c_r, c_g, c_b) \otimes (k_r, k_g, k_b) = (c_r k_r, c_g k_g, c_b k_b)$$

이 연산은 주로 조정 방식에서 사용됩니다. 예를 들어, 색상 (r, g, b) 를 가진 입사 광선이 표면에 부딪혔을 때, 이 표면이 적색광의 50%, 녹색광의 75%, 청색광의 25%를 반사하고 나머지는 흡수한다고 가정해 보겠습니다. 그러면 반사된 광선의 색상은 다음과 같이 주어집니다:

$$(r, g, b) \otimes (0.5, 0.75, 0.25) = (0.5r, 0.75g, 0.25b)$$

따라서 표면이 일부 빛을 흡수했기 때문에 광선이 표면에 부딪혔을 때 일부 강도를 잃었음을 알 수 있습니다.
색상 연산을 수행할 때 색상 성분이 $[0, 1]$ 구간을 벗어날 수 있습니다. 예를 들어, $(1, 0.1, 0.6) + (0, 0.3, 0.5) = (1, 0.4, 1.1)$ 과 같은 경우를 예로 들 수 있습니다. 1.0은 색상 구성 요소의 최대 강도를 나타내므로, 이를 초과하는 강도는 불가능합니다. 따라서 1.1은 1.0과 동일한 강도를 가집니다. 따라서 $1.1 \rightarrow 1.0$ 으로 클램핑합니다. 마찬가지로 모니터는 음의 빛을 발산할 수 없으므로, 뿔셈 연산으로 발생할 수 있는 음의 색상 구성 요소는 0.0으로 클램핑해야 합니다.

5.3.2 128비트 컬러

알파 성분이라고 불리는 추가 색상 성분을 포함하는 것이 일반적입니다. 알파 성분은 종종 색상의 불투명도를 나타내는 데 사용되며, 이는 블렌딩(9장 블렌딩)에 유용합니다. (아직 블렌딩을 사용하지 않으므로, 당분간 알파 성분을 1로 설정하세요.)
알파 성분을 포함하면 색상을 4차원 색상 벡터 (r, g, b, a) 로 표현할 수 있으며, 여기서 $0 \leq r, g, b, a \leq 1$ 입니다.
128비트로 색상을 표현하려면 각 구성 요소에 대해 부동 소수점 값을 사용합니다. 수학적으로 색상은 단순히 4차원 벡터이므로, 코드에서 색상을 표현하기 위해 **XMVECTOR** 유형을 사용할 수 있으며, XNA Math 벡터 함수를 사용하여 색상 연산(예: 색상 더하기, 빼기, 스칼라 곱하기)을 수행할 때마다 SIMD 연산의 이점을 얻을 수 있습니다. 성분별 곱셈을 위해 XNA Math 라이브러리는 다음 함수를 제공합니다:

```
XMVECTOR XMColorModulate(// 반환값 (cr, cg, cb, ca) ⊗ (kr, kg, kb, ka) FXMVECTOR C1, // (cr, cg, cb, ca)
```

```
FXMVECTOR C2); // (kr, kg, kb, ka)
```

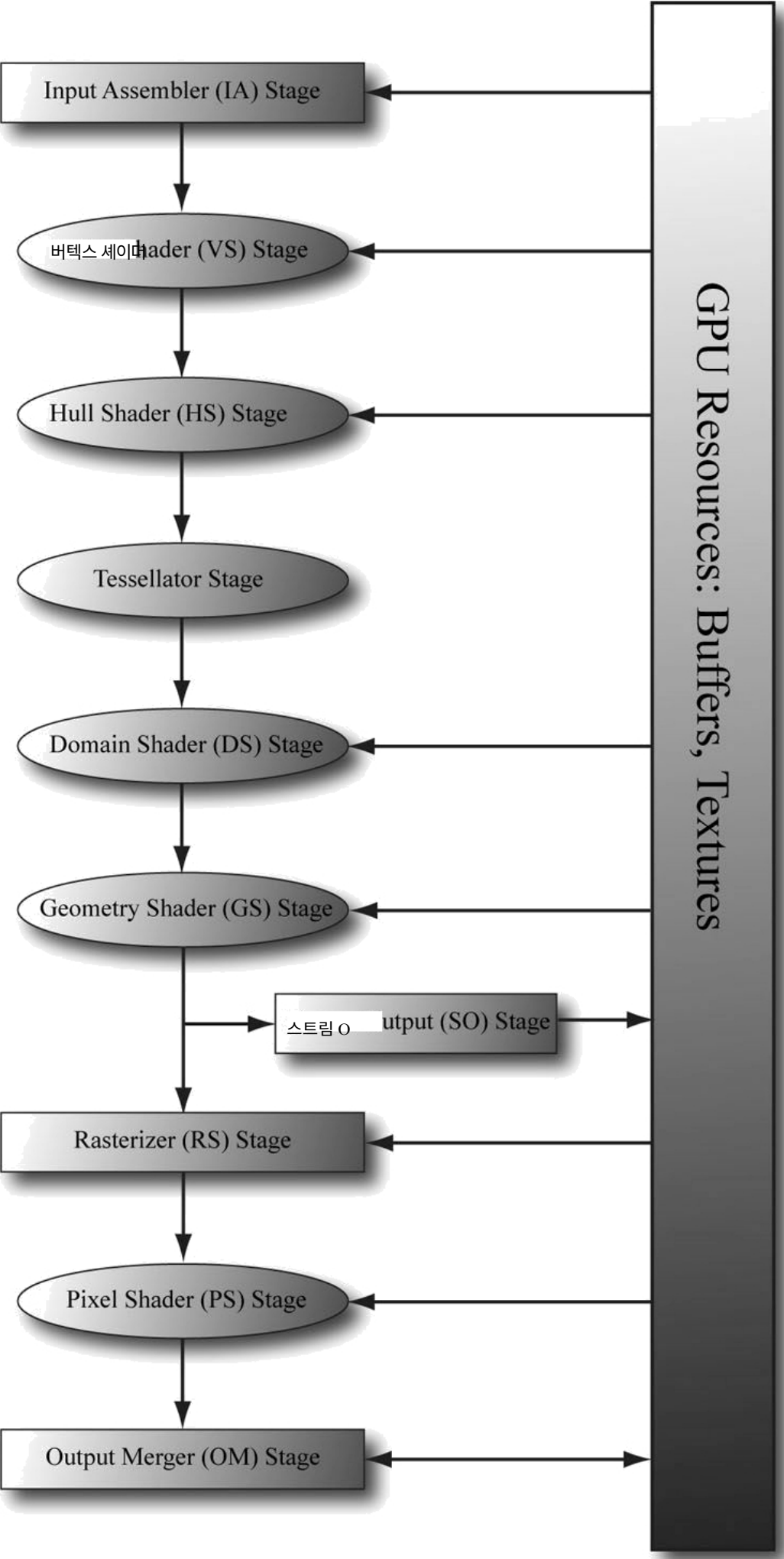
5.3.3 32비트 컬러

32비트로 색상을 표현하려면 각 구성 요소에 1바이트가 할당됩니다. 각 색상에 8비트 바이트가 할당되므로, 각 색상 구성 요소마다 256가지 다른 색조를 표현할 수 있습니다. 0은 강도가 전혀 없는 상태, 255는 최대 강도, 중간 값은 중간 강도를 나타냅니다. 색상 구성요소당 1바이트는 작아 보일 수 있지만, 모든 조합($256 \times 256 \times 256 = 16,777,216$)을 고려하면 수백만 가지의 고유한 색상을 표현할 수 있음을 알 수 있습니다. XNA Math 라이브러리는 32비트 색상을 저장하기 위해 다음과 같은 구조를 제공합니다:

```
// ARGB 색상; 8-8-8-8 비트 부호 없는 정규화 정수 구성 요소
// 32비트 정수로 압축됨. 정규화된 색상은
// 알파, 베타, 그린, 레드 색상 구성 요소에 대해 8비트 부호 없는 정규화된 정수값을 사용하여
// 빨강, 녹색, 파랑 성분.
// 알파 성분은 최상위 비트에 저장되고
// 청색 성분은 최하위 비트에 저장됩니다 (A8R8G8B8):
// [32] aaaaaaaaa rrrrrrrr gggggggg bbbbbbbb [0] typedef struct
_XMCOLOR
{
    union
    {
        struct
```


5.4 렌더링 파이프라인 개요

3차원 장면에 대해 위치와 방향이 지정된 가상 카메라의 기하학적 설명이 주어졌을 때, *렌더링 파이프라인*은 가상 카메라가 보는 것을 기반으로 2차원 이미지를 생성하는 데 필요한 전체 단계 순서를 의미합니다. [그림 5.11](#)은 렌더링 파이프라인을 구성하는 단계들의 다이어그램과 함께 측면에 GPU 메모리 리소스를 보여줍니다. 리소스 메모리 풀에서 특정 단계로 향하는 화살표는 해당 단계가 리소스를 입력으로 접근할 수 있음을 의미합니다. 예를 들어 픽셀 셰이더 단계는 작업을 수행하기 위해 메모리에 저장된 텍스처 리소스에서 데이터를 읽어야 할 수 있습니다. 단계에서 메모리로 향하는 화살표는 해당 단계가 GPU 리소스에 쓰기 작업을 수행함을 의미합니다. 예를 들어, 출력 병합 단계는 백 버퍼 및 깊이/스텐실 버퍼와 같은 텍스처에 데이터를 기록합니다. 출력 병합 단계의 화살표가 양방향(GPU 리소스 읽기 및 쓰기)임을 주목하십시오. 보시다시피 대부분의 스테이지는 GPU 리소스에 쓰지 않습니다. 대신, 그들의 출력은 파이프라인의 다음 스테이지로 입력으로 전달됩니다. 예를 들어, 버텍스 셰이더 스테이지는 입력 어셈블러 스테이지로부터 데이터를 입력으로 받아 자신의 작업을 수행한 후, 그 결과를 지오메트리 셰이더 스테이지로 출력합니다. 후속 섹션에서는 렌더링 파이프라인의 각 스테이지에 대한 개요를 제공합니다.



5.5 입력 어셈블러 단계

입력 어셈블러(IA) 단계는 메모리에서 기하학적 데이터(정점과 인덱스)를 읽고 이를 사용하여 기하학적 프리미티브(예: 삼각형, 선)를 조립합니다. (인덱스는 후속 소절에서 다루지만, 간단히 말해 프리미티브를 형성하기 위해 정점들이 어떻게 배치되어야 하는지를 정의합니다.)

5.5.1 정점

수학적으로 삼각형의 정점은 두 변이 만나는 지점이며, 선분의 정점은 끝점입니다. 단일 점의 경우 그 점 자체가 정점이 됩니다. 그림 5.12는 정점을 도식적으로 보여줍니다.

그림 5.12를 보면 정점이 기하학적 기본 도형 내의 특별한 점에 불과한 것처럼 보일 수 있습니다. 그러나 Direct3D에서 정점은 더 일반적입니다. 본질적으로 Direct3D의 정점은 공간적 위치 외에도 추가 데이터를 포함할 수 있어 더 정교한 렌더링 효과를 구현할 수 있습니다. 예를 들어, 7장에서는 조명 구현을 위해 정점에 법선 벡터를 추가하고, 8장에서는 텍스처링 구현을 위해 정점에 텍스처 좌표를 추가할 것입니다. Direct3D는 사용자 정의 벡스 형식(즉, 벡스의 구성 요소를 정의할 수 있음)을 정의할 수 있는 유연성을 제공하며, 다음 장에서 이를 구현하는 코드를 살펴볼 것입니다. 이 책에서는 수행하는 렌더링 효과에 따라 여러 가지 다른 벡스 형식을 정의할 것입니다.

5.5.2 프리미티브 토폴로지

정점은 *벡스 버퍼*라고 불리는 특별한 Direct3D 데이터 구조를 통해 렌더링 파이프라인에 바인딩됩니다. 벡스 버퍼는 연속된 메모리에 정점 목록을 저장할 뿐입니다. 그러나 이 정점들이 어떻게 조합되어 기하학적 프리미티브를 형성해야 하는지는 명시하지 않습니다. 예를 들어, 벡스 버퍼 내의 두 벡스마다 선분으로 해석해야 할까요, 아니면 세 벡스마다 삼각형으로 해석해야 할까요? 우리는 *원시체 토폴로지*를 지정함으로써 벡스 데이터로부터 기하학적 원시체를 형성하는 방법을 Direct3D에 알려줍니다.

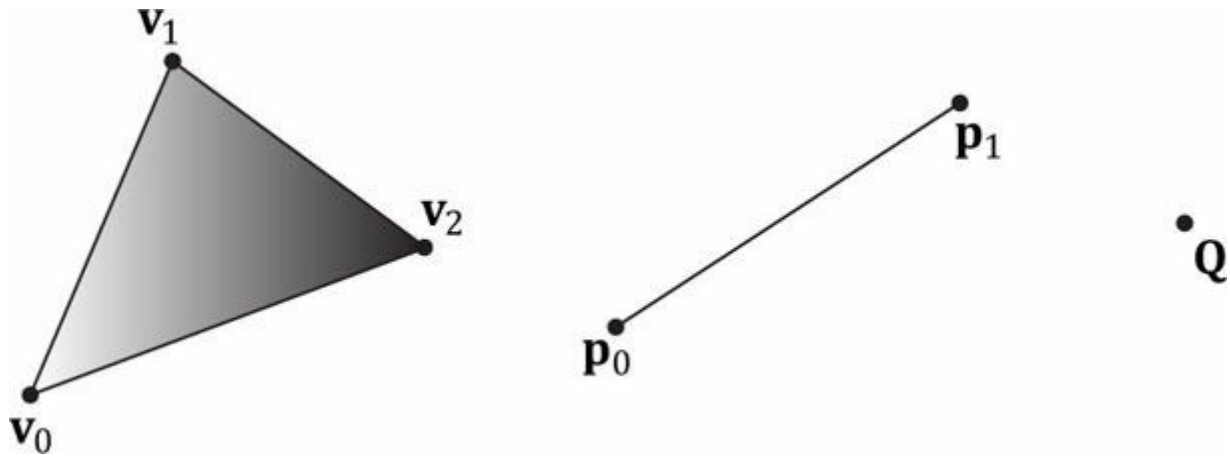


그림 5.12. 세 개의 정점 v_0, v_1, v_2 로 정의된 삼각형; 두 개의 정점 p_0, p_1 로 정의된 선분; 정점 Q 로 정의된 점.

```
void ID3D11DeviceContext::IASetPrimitiveTopology( D3D11_PRIMITIVE_TOPOLOGY
Topology);
```

```
typedef enum D3D11_PRIMITIVE_TOPOLOGY
{
    D3D11_PRIMITIVE_TOPOLOGY_UNDEFINED = 0,
    D3D11_PRIMITIVE_TOPOLOGY_POINTLIST = 1,
    D3D11_PRIMITIVE_TOPOLOGY_LINELIST = 2,
    D3D11_PRIMITIVE_TOPOLOGY_LINESTRIP = 3,
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST = 4,
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP = 5,
    D3D11_PRIMITIVE_TOPOLOGY_LINELIST_ADJ = 10,
    D3D11_PRIMITIVE_TOPOLOGY_LINESTRIP_ADJ = 11,
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST_ADJ = 12,
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP_ADJ = 13,
    D3D11_PRIMITIVE_TOPOLOGY_1_CONTROL_POINT_PATCHLIST = 33,
```

```

D3D11_PRIMITIVE_TOPOLOGY_2_CONTROL_POINT_PATCHLIST=34,
    .
    .
    .
D3D11_PRIMITIVE_TOPOLOGY_32_CONTROL_POINT_PATCHLIST=64,
} D3D11_PRIMITIVE_TOPOLOGY;

```

이후 모든 드로잉 호출은 토폴로지가 변경될 때까지 현재 설정된 프리미티브 토폴로지를 사용합니다. 다음 코드는 이를 보여줍니다.

```

md3dImmediateContext->IASetPrimitiveTopology(
    D3D11_PRIMITIVE_TOPOLOGY_LINELIST);
/* ...라인 리스트를 사용하여 객체 그리기... */

md3dImmediateContext->IASetPrimitiveTopology(
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
/* ...삼각형 목록을 사용하여 객체 그리기... */

md3dImmediateContext->IASetPrimitiveTopology(
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP);
/* ...삼각형 스트립을 사용하여 객체 그리기... */

```

다음 소절에서는 다양한 프리미티브 토폴로지에 대해 자세히 설명합니다. 이 책에서는 몇 가지 예외를 제외하고는 주로 삼각형 리스트만을 사용합니다.

5.5.2.1 점 리스트

점 리스트는 D3D11_PRIMITIVE_TOPOLOGY_POINTLIST로 지정됩니다. 점 리스트를 사용하면 [그림 5.13a](#)와 같이 드로우 호출의 모든 정점이 개별 점으로 그려집니다.

5.5.2.2 선 스트립

라인 스트립은 D3D11_PRIMITIVE_TOPOLOGY_LINESTRIP로 지정됩니다. 라인 스트립을 사용하면 드로우 호출의 정점들이 연결되어 선을 형성합니다([그림 5.13b](#) 참조). 따라서 $n + 1$ 개의 정점이 n 개의 선을 생성합니다.

5.5.2.3 라인 리스트

라인 리스트는 D3D11_PRIMITIVE_TOPOLOGY_LINELIST로 지정됩니다. 라인 리스트를 사용하면 드로우 콜 내의 두 정점마다 개별 선을 형성합니다([그림 5.13c](#) 참조). 따라서 $2n$ 개의 정점은 n 개의 선을 생성합니다. 라인 리스트와 스트립의 차이점은 라인 리스트의 선분들은 연결되지 않을 수 있는 반면, 라인 스트립은 자동으로 연결된 것으로 가정한다는 점입니다. 연결성을 가정함으로써 각 내부 정점이 두 개의 선분에 공유되므로 더 적은 수의 정점을 사용할 수 있습니다.

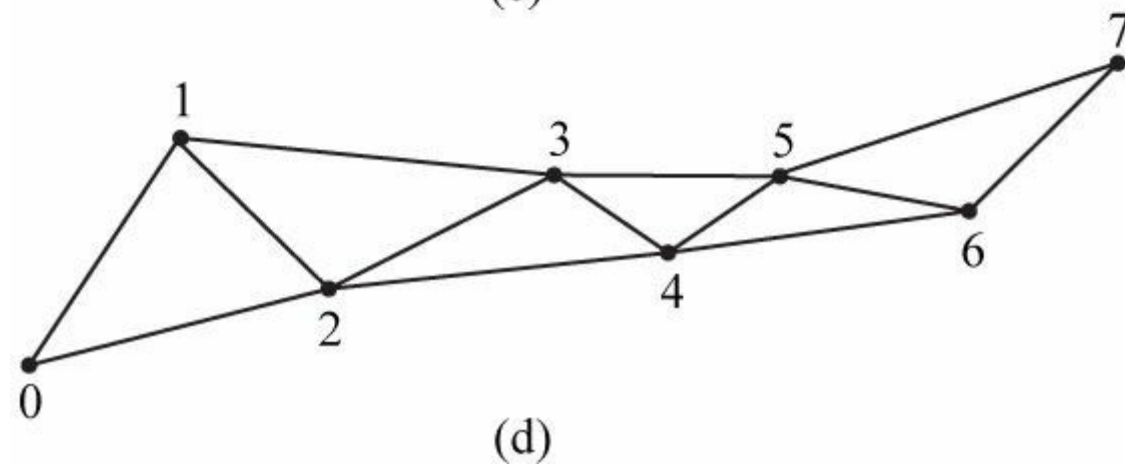
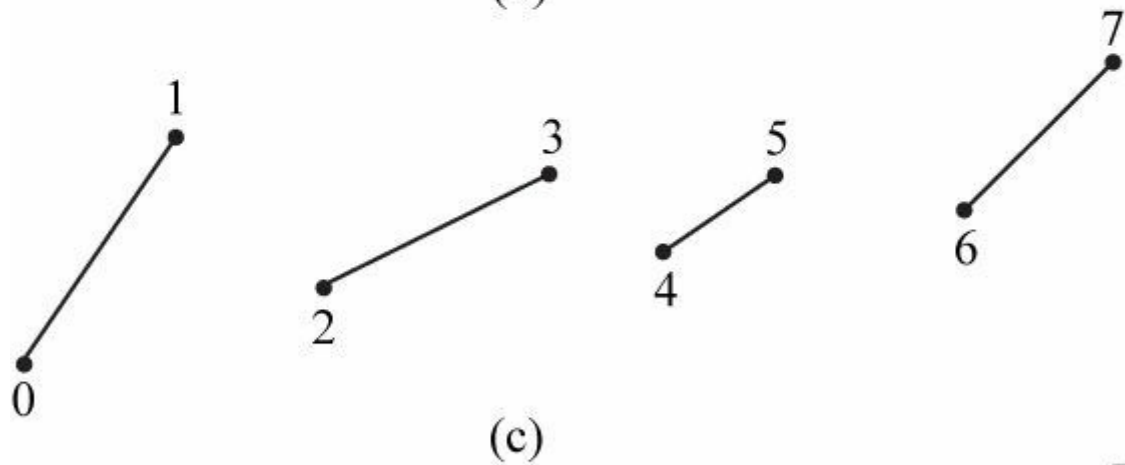
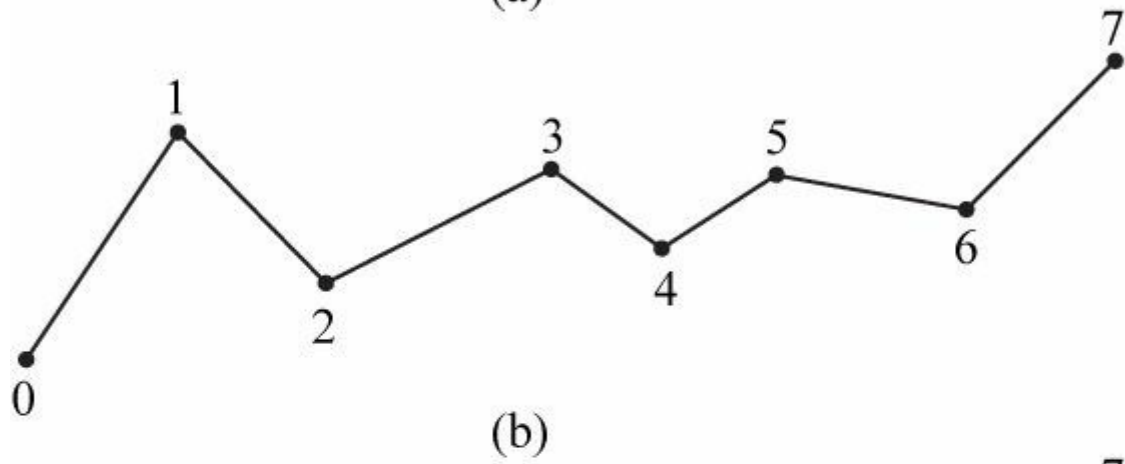
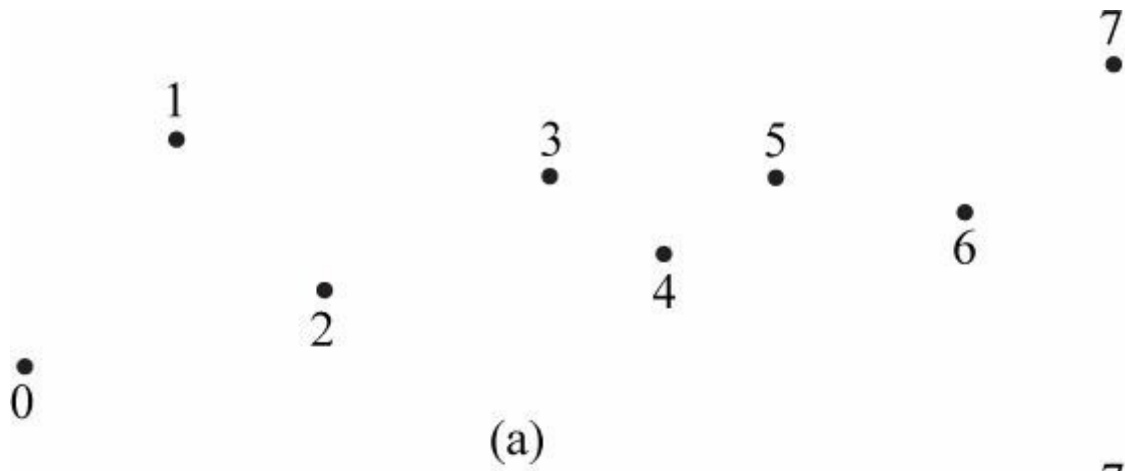


그림 5.13. (a) 점 리스트; (b) 라인 스트립; (c) 라인 리스트; (d) 트라이앵글 스트립.

5.5.2.4 삼각형 스트립

삼각형 스트립은 `D3D11_PRIMITIVE_TOPOLOGY_TRIANGLESTRIP`으로 지정됩니다. 삼각형 스트립에서는 그림 5.13d와 같이 삼각형들이 연결되어 스트립을 형성한다고 가정합니다. 연결성을 가정함으로써 인접한 삼각형들 사이에 정점이 공유되며, n 개의 정점이 $n - 2$ 개의 삼각형을 생성함을 알 수 있습니다.

삼각형 스트립 내 짝수 삼각형의 와인딩 순서는 홀수 삼각형과 달라, 이로 인해 정면 배제(Note) 문제가 발생합니다(\$5.10.2 참조). 이 문제를 해결하기 위해 GPU는 내부적으로

짝수 삼각형의 첫 두 정점 순서를 교환합니다.

삼각형들을 홀수 삼각형들과 마찬가지로 일관되게 정렬되도록 합니다.

5.5.2.5 삼각형 리스트

삼각형 리스트는 `D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST`로 지정됩니다. 삼각형 리스트에서는 드로우 호출마다 세 개의 정점이 개별 삼각형을 형성합니다(그림 5.14a 참조). 따라서 $3n$ 개의 정점은 n 개의 삼각형을 생성합니다. 삼각형 리스트와 스트립의 차이점은 삼각형 리스트의 삼각형은 연결되지 않을 수 있는 반면, 삼각형 스트립은 연결되어 있다고 가정한다는 점입니다.

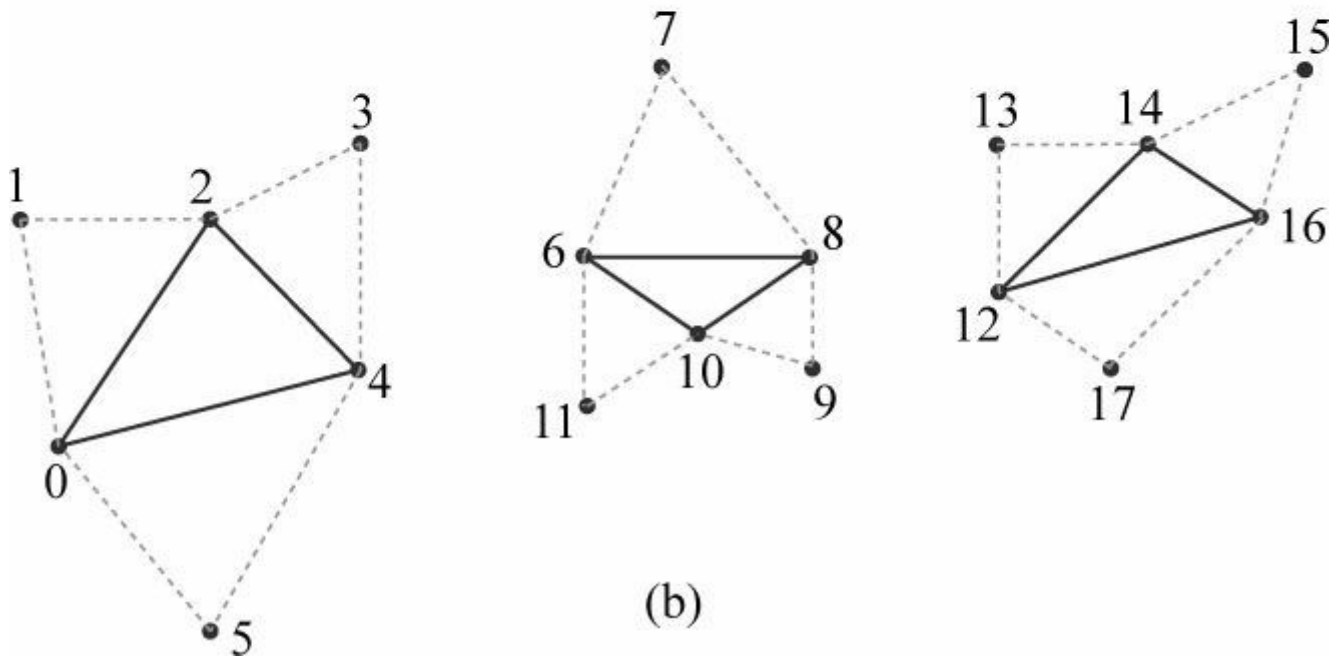
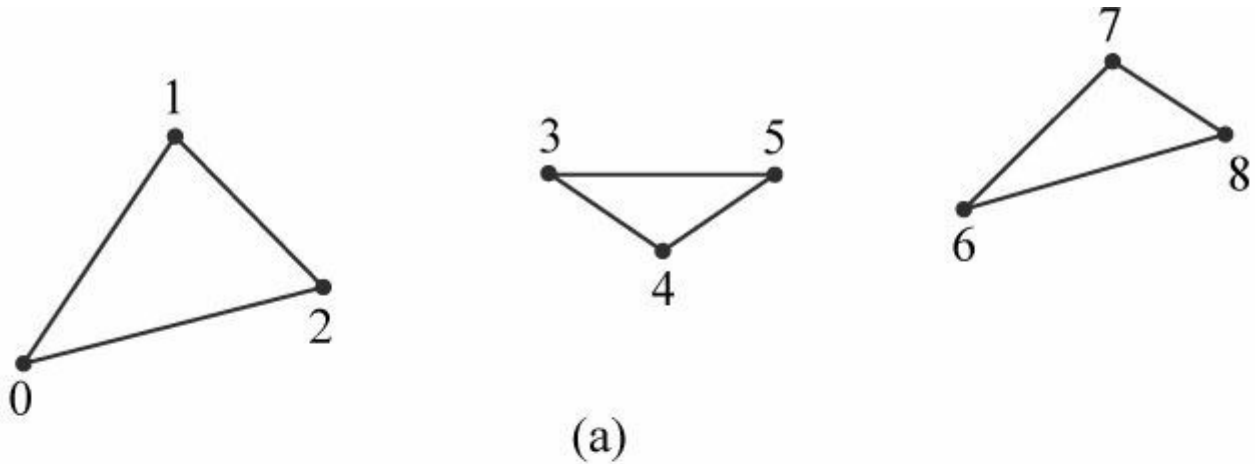


그림 5.14. (a) 삼각형 리스트. (b) 인접성 정보가 포함된 삼각형 리스트—각 삼각형은 자신과 인접 삼각형을 표현하기 위해 6개의 정점이 필요함을 확인하십시오. 따라서 $6n$ 개의 정점은 인접성 정보가 포함된 n 개의 삼각형을 생성합니다.

5.5.2.6 인접성을 가진 프리미티브

인접 삼각형 목록은 각 삼각형마다 *인접 삼각형*이라 불리는 세 개의 이웃 삼각형을 함께 포함하는 목록입니다. 이러한 삼각형이 어떻게 정의되는지 확인하려면 그림 5.14b를 참조하십시오. 이는 특정 지오메트리 셰이딩 알고리즘이 인접 삼각형에 접근해야 하는 지오메트리 셰이더에 사용됩니다. 지오메트리 셰이더가 인접 삼각형을 획득하려면, 해당 삼각형 자체와 함께 인접 삼각형을 버텍스/인덱스 버퍼에 파이프라인으로 제출해야 하며, `D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST_ADJ` 토폴로지를 지정하여 파이프라인이 버텍스 버퍼로부터 삼각형과 그 인접 삼각형을 구성하는 방법을 알 수 있도록 해야 합니다. 인접한 프리미티브의 버텍스는 지오메트리 셰이더의 입력으로만 사용되며

지오메트리 셰이더의 입력으로만 사용되며, 실제로 그려지지 않습니다. 지오메트리 셰이더가 없더라도 인접한 프리미티브들은 여전히 그려지지 않습니다.

인접성을 가진 선 목록, 인접성을 가진 선 스트립, 스트립 인접성을 가진 삼각형 프리미티브를 사용할 수도 있습니다. 자세한 내용은

자세한 내용은 SDK 문서를 참조하십시오.

5.5.2.7 제어점 패치 리스트

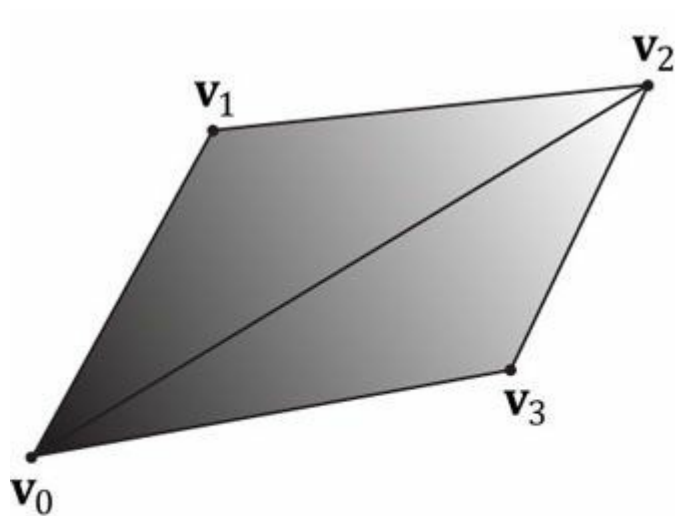
D3D11_PRIMITIVE_TOPOLOGY_N_CONTROL_POINT_PATCHLIST 토폴로지 유형은 정점 데이터를 N 개의 제어점을 가진 패치 리스트로 해석해야 함을 나타냅니다. 이는 렌더링 파이프라인의 (선택적) 테셀레이션 단계에서 사용되므로, 이에 대한 논의는 [13장](#) "테셀레이션 단계"에서 다루도록 하겠습니다.

5.5.3 인덱스

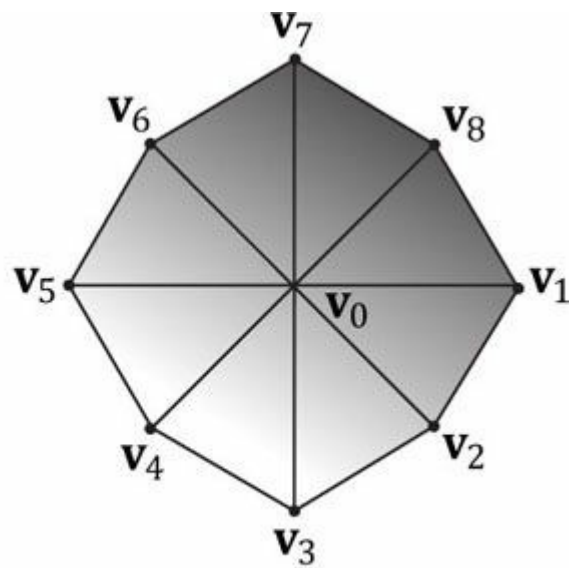
앞서 언급한 바와 같이 삼각형은 3D 솔리드 객체의 기본 구성 요소입니다. 다음 코드는 삼각형 리스트(즉, 세 개의 정점이 삼각형을 형성함)를 사용하여 사각형과 팔각형을 구성하는 데 사용되는 정점 배열을 보여줍니다.

```
Vertex quad[6] = {  
    v0, v1, v2, // 삼각형 0 v0, v2, v3, //  
    삼각형 1  
};  
  
Vertex octagon[24] = {  
    v0, v1, v2, // 삼각형 0 v0, v2, v3, //  
    삼각형 1 v0, v3, v4, // 삼각형 2 v0,  
    v4, v5, // 삼각형 3 v0, v5, v6, // 삼각  
    형 4 v0, v6, v7, // 삼각형 5 v0, v7,  
    v8, // 삼각형 6 v0, v8, v1 // 삼각형 7  
};
```

 삼각형의 꼭짓점을 지정하는 순서는 중요하며, 이를 회전 순서(winding order)라고 합니다. 자세한 내용은 §5.10.2를 참조하십시오.



(a)



(b)

그림 5.15. (a) 두 개의 삼각형으로 구성된 사각형. (b) 여덟 개의 삼각형으로 구성된 팔각형.

그림 5.15에서 볼 수 있듯이, 3차원 물체를 구성하는 삼각형들은 많은 정점을 공유합니다. 더 구체적으로, 그림 5.15a의 사각형 내 각 삼각형은 꼭짓점 v_0 과 v_2 를 공유합니다. 두 꼭짓점을 중복하는 것은 그리 나쁘지 않지만, 팔각형 예시(그림 5.15b)에서는 중복이 더 심합니다. 모든 삼각형이 중심 꼭짓점 v_0 을 중복하고, 팔각형 둘레의 각 꼭짓점은 두 개의 삼각형이 공유합니다. 일반적으로 모델의 디테일과 복잡성이 증가할수록 중복 정점의 수도 증가합니다.

정점을 중복하고 싶지 않은 이유는 두 가지입니다:

1. 메모리 요구량 증가. (동일한 정점 데이터를 왜 여러 번 저장해야 할까요?)
2. 그래픽스 하드웨어의 처리량 증가. (동일한 정점 데이터를 왜 여러 번 처리해야 하는가?)

삼각형 스트립은 지오메트리를 스트립 형태로 구성할 수 있는 경우 중복 정점 문제를 해결하는 데 도움이 될 수 있습니다. 그러나 삼각형 리스트는 더 유연합니다(삼각형이 연결될 필요 없음). 따라서 삼각형 리스트에서 중복 정점을 제거하는 방법을 고안하는 것이 좋습니다.

삼각형 리스트의 중복 정점을 제거하는 방법을 고안할 가치가 있습니다. 해결책은 *인덱스*를 사용하는 것입니다. 작동 방식은 다음과 같습니다: 정점 리스트와 인덱스 리스트를 생성합니다. 정점 리스트는 모든 고유한 정점으로 구성되며, 인덱스 리스트는 삼각형을 형성하기 위해 정점들을 어떻게 조합할지 정의하기 위해 정점 리스트를 참조하는 값들을 포함합니다. [그림 5.16](#)의 도형으로 돌아가면, 사각형의 정점 리스트는 다음과 같이 구성됩니다:

```
Vertex v[4] = {v0, v1, v2, v3};
```

그런 다음 인덱스 리스트는 정점 리스트의 정점들이 어떻게 조합되어 두 개의 삼각형을 형성할지 정의해야 합니다.

```
UINT indexList[6] = {0, 1, 2, // 삼각형 0
0, 2, 3}; // 삼각형 1
```

인덱스 리스트에서 세 개의 요소가 하나의 삼각형을 정의합니다. 따라서 위 인덱스 리스트는 "정점 `v[0]`, `v[1]`, `v[2]`를 사용하여 삼각형 0을 구성하고, 정점 `v[0]`, `v[2]`, `v[3]`을 사용하여 삼각형 1을 구성하라"고 지시합니다. 마찬가지로 원의 정점 목록은 다음과 같이 구성됩니다:

```
정점 v [9] = {v0, v1, v2, v3, v4, v5, v6, v7, v8};
```

그리고 인덱스 리스트는 다음과 같습니다.

```
UINT indexList[24] = { 0, 1, 2, // 삼각
    형 0
    0, 2, 3, // 삼각형 1
    0, 3, 4, // 삼각형 2
    0, 4, 5, // 삼각형 3
    0, 5, 6, // 삼각형 4
    0, 6, 7, // 삼각형 5
    0, 7, 8, // 삼각형 6
    0, 8, 1 // 삼각형 7
```

```
};
정점 목록의 고유 정점들이 처리된 후, 그래픽 카드는 인덱스 목록을 사용하여 정점들을 조합해
```

삼각형을 형성합니다. "중복" 처리가 인덱스 리스트로 옮겨졌지만, 이는 다음과 같은 이유로 문제가 되지 않습니다:

- 1. 인덱스는 단순한 정수이므로 완전한 정점 구조체만큼 많은 메모리를 차지하지 않습니다(정점 구조체는 구성 요소를 추가할수록 커질 수 있음).
- 2. 정점 캐시 정렬이 잘 되어 있다면 그래픽스 하드웨어가 중복 정점을 (너무 자주) 처리할 필요가 없습니다.

5.6 버텍스 셰이더 단계

프리티미티브가 조립된 후, 정점들은 버텍스 셰이더 단계로 공급됩니다. 버텍스 셰이더는 정점을 입력으로 받아 정점을 출력하는 함수로 생각할 수 있습니다. 그러지는 모든 정점은 버텍스 셰이더를 통과하게 됩니다. 실제로 하드웨어에서 다음과 같은 과정이 개념적으로 발생한다고 생각할 수 있습니다:

```
for(UINT i = 0; i < numVertices; ++i)
    outputVertex[i] = VertexShader(inputVertex[i]);
```

버텍스 셰이더 함수는 우리가 구현하는 것이지만, 각 버텍스에 대해 GPU에서 실행되므로 매우 빠릅니다. 변환, 조명, 변위 매핑 등 많은 특수 효과를 버텍스 셰이더에서 수행할 수 있습니다.

입력 버텍스 데이터뿐만 아니라 텍스처와 변환 행렬, 장면 조명 등 GPU 메모리에 저장된 다른 데이터에도 접근할 수 있다는 점을 명심하세요.

이 책에서는 다양한 버텍스 셰이더의 예시를 많이 살펴볼 것입니다. 따라서 책을 다 읽을 때쯤이면

첫 번째 코드 예제에서는 버텍스 셰이더를 사용하여 버텍스를 변환하는 데만 사용할 것입니다. 다음 소절에서는 일반적으로 수행해야 하는 변환 유형을 설명합니다.

5.6.1 로컬 공간과 월드 공간

잠시 상상에 보십시오. 영화 작업을 하고 있으며, 팀이 특수 효과 샷을 위해 기차 장면의 미니어처 버전을 제작해야 한다고 가정합니다. 특히 작은 다리를 만드는 임무를 맡았다고 가정해 봅시다. 이 경우, 장면 한가운데에서 다리를 제작하지는 않을 것입니다. 그곳에서는 작업 각도가 까다로울 뿐만 아니라 장면을 구성하는 다른 미니어처들을 망가뜨리지 않도록 조심해야 하기 때문입니다. 대신 작업대에서 장면과 떨어진 곳에서 다리를 제작할 것입니다. 그런 다음 완성된 다리를 장면 내 정확한 위치와 각도에 배치하게 됩니다.

3D 아티스트들은 3D 객체를 구성할 때 비슷한 작업을 수행합니다. 객체의 기하학적 구조를 좌표계를 기준으로 상대적인 좌표로 구축하는 대신

글로벌 장면 좌표계(*월드 스페이스*)를 사용하기 위해, 이들은 로컬 좌표계(*로컬 스페이스*)를 기준으로 이를 지정합니다. 로컬 좌표계는 일반적으로 객체 근처에 위치하며 객체의 축과 정렬된 편리한 좌표계입니다. 3D 모델의 정점이 로컬 스페이스에서 정의되면, 이는 글로벌 장면에 배치됩니다. 이를 위해 로컬 공간과 월드 공간의 관계를 정의해야 합니다. 이는 로컬 공간 좌표계의 원점과 축을 글로벌 장면 좌표계에 대해 어디에 배치할지 지정하고 좌표 변환을 수행함으로써 이루어집니다([그림 5.16](#) 및 §3.4 참조). 로컬 좌표계에 대한 좌표를 글로벌 장면 좌표계로 변환하는 과정을 *월드 변환*이라고 하며, 이에 대응하는 행렬을 *월드 행렬*이라고 합니다. 장면 내 각 객체는 고유한 월드 행렬을 가집니다. 모든 객체가 로컬 공간에서 월드 공간으로 변환된 후에는 모든 객체의 좌표가 동일한 좌표계(월드 공간)를 기준으로 합니다. 월드 공간에서 직접 객체를 정의하려면, 신본 월드 행렬을 제공하면 됩니다.

각 모델을 자체 국소 좌표계에 대해 정의하는 것은 여러 장점이 있습니다:

- 1. 더 쉽습니다. 예를 들어, 일반적으로 국소 공간에서는 객체가 원점에 중심을 두고 주요 축 중 하나에 대해 대칭적입니다. 또 다른 예로, 정육면체의 꼭짓점은 정육면체 중심을 원점으로 하고 축이 정육면체 면에 직교하도록 국소 좌표계를 선택하면 훨씬 쉽게 지정할 수 있습니다; [그림 5.17](#)을 참조하십시오.

World System

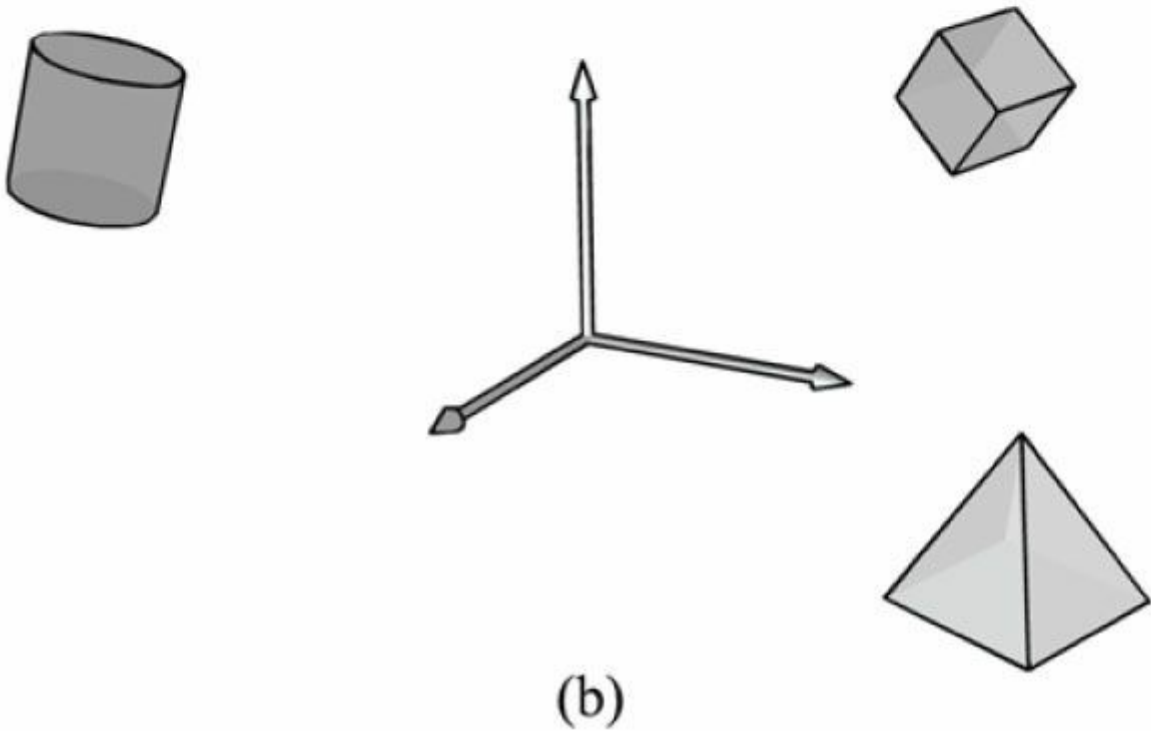
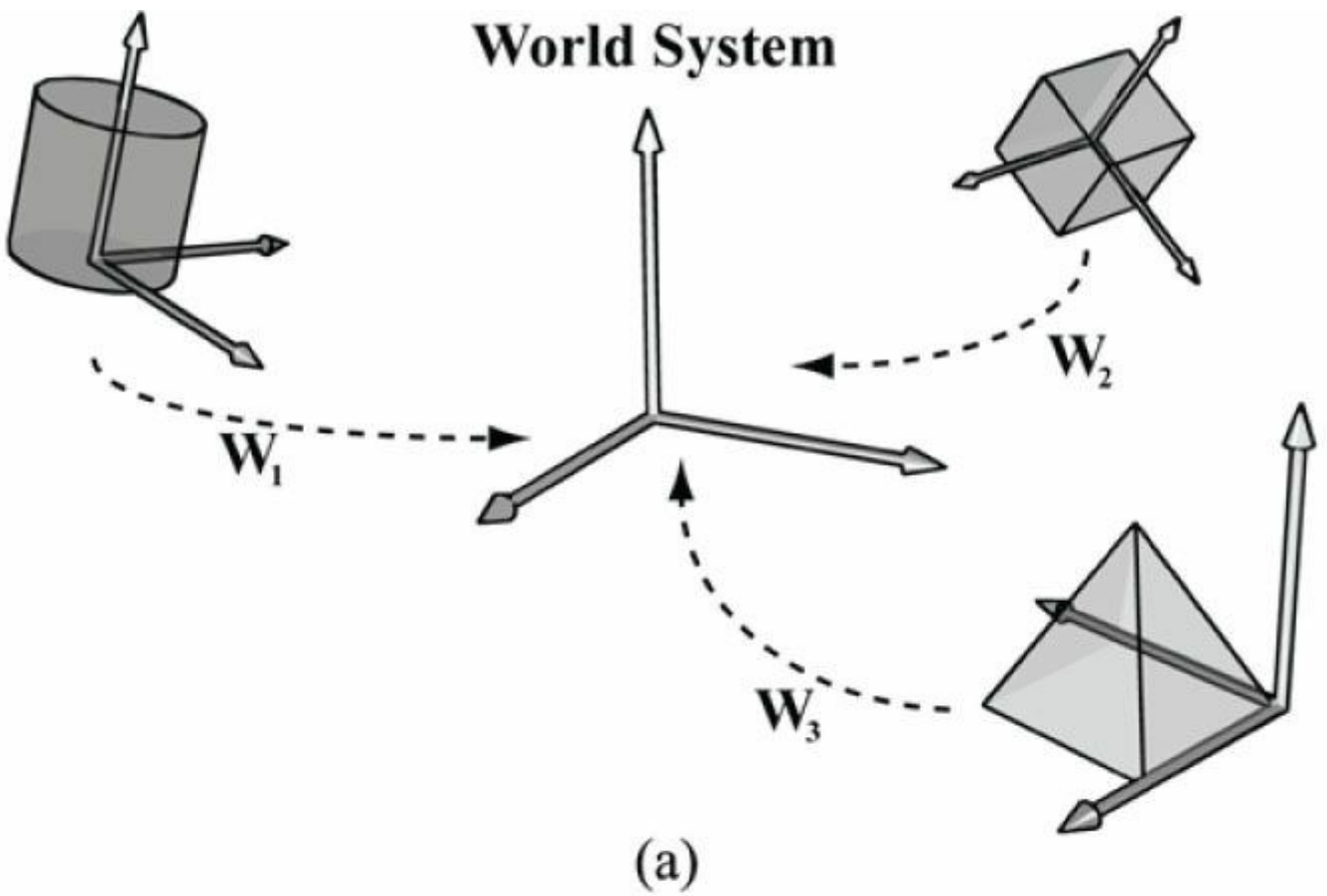


그림 5.16. (a) 각 객체의 꼭짓점은 자체 국소 좌표계에 상대적인 좌표로 정의됩니다. 또한 장면 내에서 객체를 배치하고자 하는 위치를 기준으로, 각 국소 좌표계의 위치와 방향을 세계 공간 좌표계에 상대적으로 정의합니다. 그런 다음 모든 좌표를 세계 공간 좌표계에 상대적으로 만들기 위해 좌표 변환을 수행합니다. (b) 세계 변환 후 객체의 꼭짓점 좌표는 모두 동일한 세계 좌표계에 상대적입니다.

- 객체는 여러 장면에서 재사용될 수 있으며, 이 경우 특정 장면에 대한 객체의 좌표를 하드코딩하는 것은 의미가 없습니다. 대신 로컬 좌표계에 대한 좌표를 저장하고, 각 장면마다 좌표계 변환 행렬을 통해 로컬 좌표계와 월드 좌표계의 관계를 정의하는 것이 더 좋습니다.
- 마지막으로, 동일한 객체를 장면 내에서 여러 번 그리는 경우가 있지만 위치, 방향, 크기가 다를 수 있습니다(예:

나무 객체를 여러 번 재사용하여 숲을 구성할 수 있음). 각 인스턴스에 대해 객체의 정점 및 인덱스 데이터를 복제하는 것은 비효율적입니다. 대신, 로컬 공간에 상대적인 단일 지오메트리 사본(즉, 정점 및 인덱스 목록)을 저장합니다. 그런 다음 객체를 여러 번 렌더링하지만, 매번 다른 월드 매트릭스를 사용하여 월드 공간 내 인스턴스의 위치, 방향 및 크기를 지정합니다. 이를 *인스턴싱*이라고 합니다.

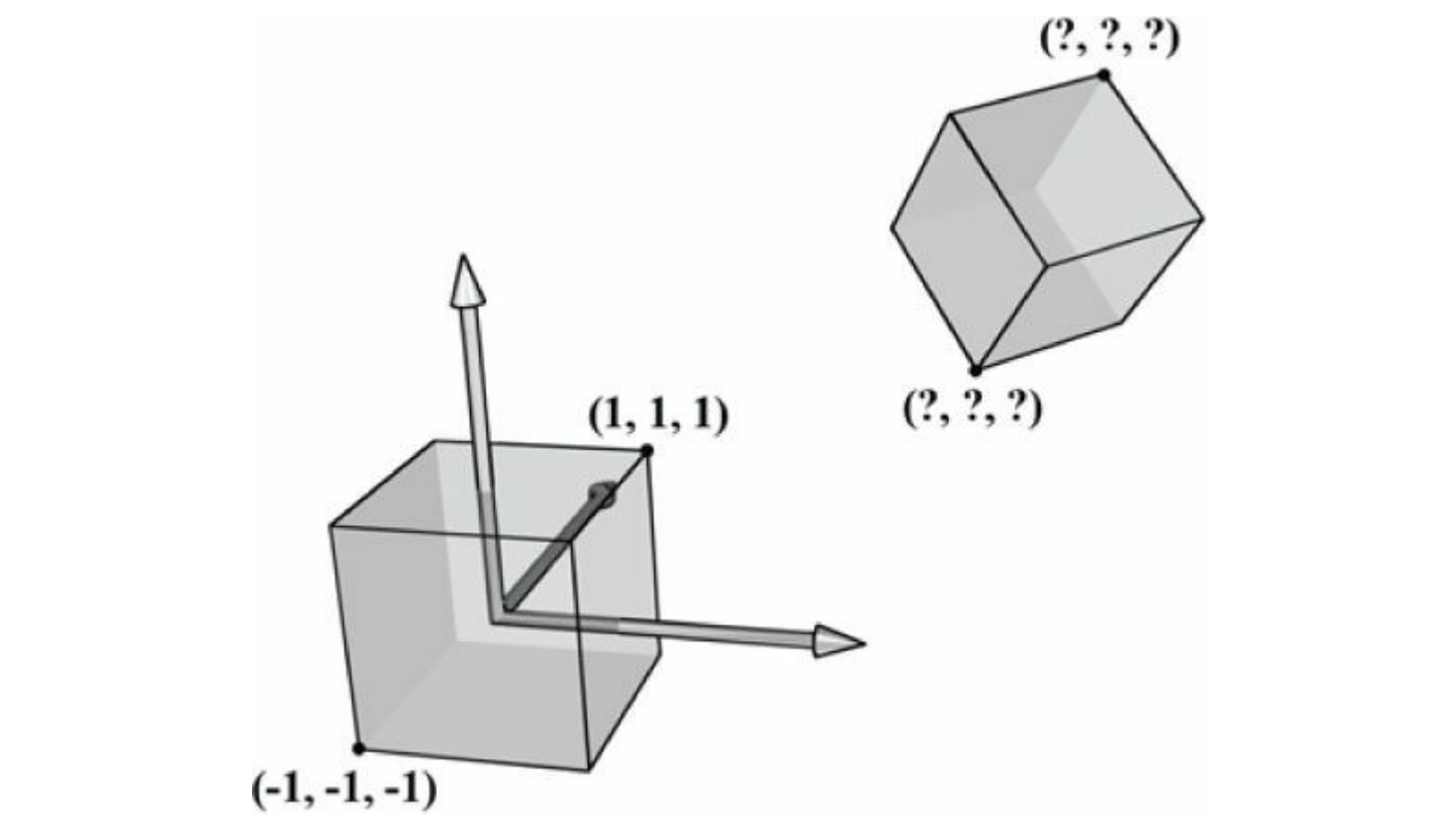


그림 5.17. 정육면체의 꼭짓점은 정육면체가 원점에 중심을 두고 좌표계와 축이 정렬된 경우 쉽게 지정할 수 있습니다. 그러나 정육면체가 좌표계에 대해 임의의 위치와 방향을 가질 때는 좌표 지정이 쉽지 않습니다. 따라서 객체의 기하학적 구조를 구성할 때는 일반적으로 객체 근처에서 객체와 정렬된 편리한 좌표계를 선택하여 그 좌표계를 중심으로 객체를 구축합니다.

§3.4.3에서 보듯이, 물체의 세계 행렬은 세계 공간에 대한 상대 좌표로 그 국소 공간을 기술하고, 이 좌표들을 행렬의 행에 배치함으로써 주어진다. $\mathbf{Q}_w = (Q_x, Q_y, Q_z, 1)$, $\mathbf{u}_w = (u_x, u_y, u_z, 0)$, $\mathbf{v}_w = (v_x, v_y, v_z, 0)$, $\mathbf{w}_w = (w_x, w_y, w_z, 0)$ 가 각각 세계 공간에 대한 동차 좌표를 가진 국소 공간의 원점, x축, y축, z축을 나타낸다면, §3.4.3에서 알 수 있듯이 국소 공간에서 세계 공간으로의 좌표 변환 행렬은 다음과 같습니다:

$$\mathbf{W} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ Q_x & Q_y & Q_z & 1 \end{bmatrix}$$

세계 행렬을 구성하려면 세계 공간에 대한 국소 공간 원점과 축의 좌표를 직접 계산해야 합니다. 이는 때로 쉽지 않거나 직관적이지 않을 수 있습니다. 보다 일반적인 접근법은 $\mathbf{W}$ 를 일련의 변환으로 정의하는 것이다. 예를 들어 $\mathbf{W} = \mathbf{SRT}$ 로 표현할 수 있는데, 이는 객체를 세계 공간으로 스케일링하는 스케일링 행렬 $\mathbf{S}$, 세계 공간에 대한 로컬 공간의 방향을 정의하는 회전 행렬 $\mathbf{R}$, 그리고 세계 공간에 대한 로컬 공간의 원점을 정의하는 평행 이동 행렬 $\mathbf{T}$ 의 곱으로 구성된다. §3.5에서 알 수 있듯이, 이러한 일련의 변환은 좌표계 변환으로 해석될 수 있으며, $\mathbf{W} = \mathbf{SRT}$ 의 행 벡터들은 세계 공간에 대한 로컬 공간의 x축, y축, z축 및 원점의 동차 좌표를 저장합니다.

예시

어떤 로컬 공간에 대해 정의된 단위 정사각형이 있다고 가정하자. 이 정사각형의 최소점과 최대점은 각각 $(-0.5, 0, -0.5)$ 와 $(0.5, 0,$

0.5)입니다. 정사각형이 월드 공간에서 변의 길이가 2가 되도록 하고, 월드 공간의 xz 평면에서 시계 방향으로 45° 회전하며, 월드 공간의 (10, 0, 10) 위치에 배치되도록 하는 월드 매트릭스를 구하십시오. **S**, **R**, **T**, **W**는 다음과 같이 구성합니다:

$$\mathbf{S} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R} = \begin{bmatrix} \sqrt{2}/2 & 0 & -\sqrt{2}/2 & 0 \\ 0 & 1 & 0 & 0 \\ \sqrt{2}/2 & 0 & \sqrt{2}/2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 10 & 0 & 10 & 1 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{SRT} = \begin{bmatrix} \sqrt{2} & 0 & -\sqrt{2} & 0 \\ 0 & 1 & 0 & 0 \\ \sqrt{2} & 0 & \sqrt{2} & 0 \\ 10 & 0 & 10 & 1 \end{bmatrix}$$

이제 여기서 §3.5, 다음과 같이 행에 있는 행은 로컬 좌표계 좌표계 시스템 상대 좌표계 세계 좌표계 공간에 대해 즉, $\mathbf{u}_W = (\sqrt{2}, 0, -\sqrt{2}, 0)$, $\mathbf{v}_W = (0, 1, 0, 0)$, $\mathbf{w}_W = (\sqrt{2}, 0, \sqrt{2}, 0)$, 그리고 $\mathbf{Q}_w = (10, 0, 10, 1)$ 입니다. 좌표를 로컬 공간에서 **W**를 사용해 월드 공간으로 변환하면, 정사각형은 월드 공간에서 원하는 위치에 배치됩니다(그림 5.18 참조).

$$\begin{bmatrix} -0.5 & 0 & -0.5 & 1 \end{bmatrix} \mathbf{W} = \begin{bmatrix} 10 - \sqrt{2} & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} -0.5 & 0 & +0.5 & 1 \end{bmatrix} \mathbf{W} = \begin{bmatrix} 0 & 0 & 10 + \sqrt{2} & 1 \end{bmatrix}$$

$$\begin{bmatrix} +0.5 & 0 & +0.5 & 1 \end{bmatrix} \mathbf{W} = \begin{bmatrix} 10 + \sqrt{2} & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} +0.5 & 0 & -0.5 & 1 \end{bmatrix} \mathbf{W} = \begin{bmatrix} 0 & 0 & 10 - \sqrt{2} & 1 \end{bmatrix}$$

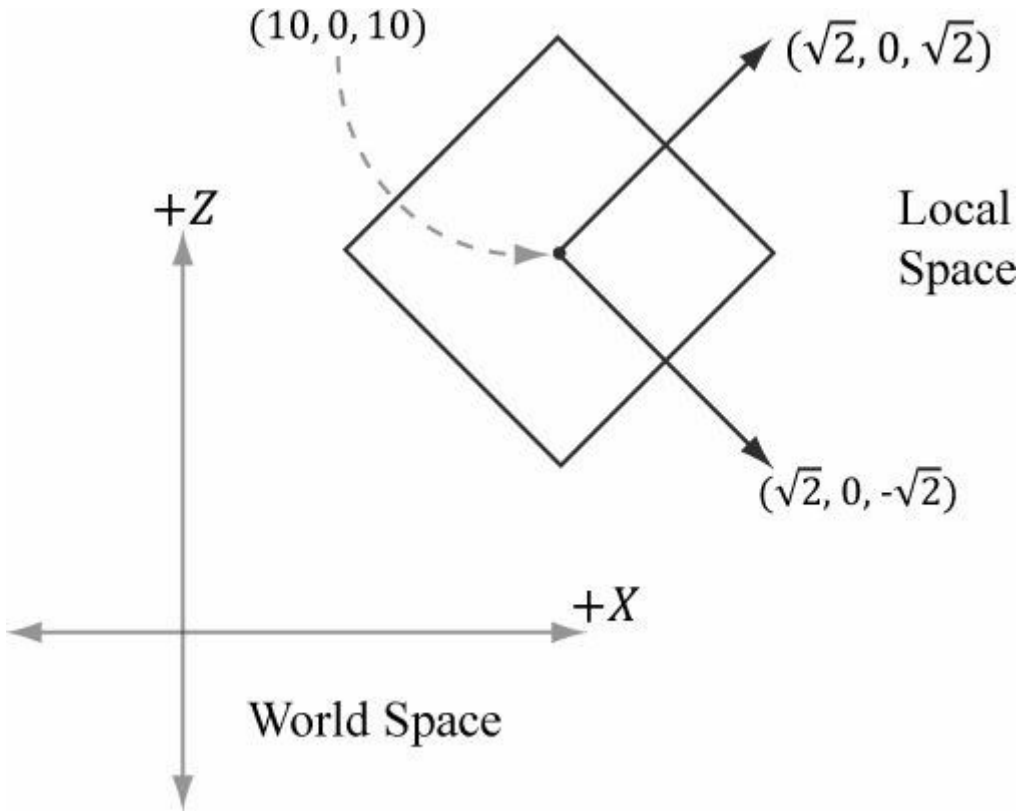


그림 5.18. 세계 행렬의 행 벡터들은 세계 좌표계에 상대적인 좌표로 로컬 좌표계를 기술한다.

이 예제의 핵심은 세계 행렬을 구성하기 위해 $\mathbf{Q}_w, \mathbf{u}_w, \mathbf{v}_w, \mathbf{w}_{(w)}$ 을 직접 계산하는 대신, 일련의 단순한 변환을 합성하여 세계 행렬을 구축할 수 있었다는 점입니다. 이는 $\mathbf{Q}_w, \mathbf{u}_w, \mathbf{v}_w, \mathbf{w}_{(w)}$ 을 직접 구하는 것보다 훨씬 쉬운 경우가 많습니다. 왜냐하면 우리는 단지 세 가지만 묻기만 하면 되기 때문입니다: 세계 공간에서 객체의 크기를 어떻게 할 것인가, 세계 공간에서 객체의 방향을 어떻게 할 것인가, 그리고 세계 공간에서 객체의 위치를 어떻게 할 것인가.

세계 변환을 고려하는 또 다른 방법은 로컬 공간 좌표를 그대로 세계 공간 좌표로 취급하는 것이다 (이는 세계 변환에 단위 행렬을 사용하는 것과 동일합니다). 따라서 객체가 로컬 공간의 중심에 모델링된 경우, 해당 객체는 세계 공간의 중심에 위치하게 됩니다. 일반적으로 세계의 중심은 우리가 모든 객체를 배치하고자 하는 위치가 아닐 것입니다. 따라서 이제 각 객체에 대해 일련의 변환을 적용하여 객체를 세계 공간에서 원하는 위치로 크기 조정, 회전 및 배치합니다. 수학적으로 이는 로컬 공간에서 세계 공간으로의 좌표 변환 행렬을 구축하는 것과 동일한 세계 변환을 제공합니다.

5.6.2 뷰 공간

장면의 2차원 이미지를 형성하기 위해서는 장면 내에 가상 카메라를 배치해야 합니다. 카메라는 시청자가 볼 수 있는 세계의 부피를 지정하며, 따라서 우리가 2차원 이미지를 생성해야 하는 세계의 부피를 결정합니다. 그림 5.19와 같이 카메라에 국소 좌표계(뷰 스페이스, 아이 스페이스 또는 카메라 스페이스라고도 함)를 부착합니다. 즉, 카메라는 원점에 위치하여 양의 z축을 따라 아래를 향하고, x축은 카메라 오른쪽을 향하며, y축은 카메라 위를 향합니다. 렌더링 파이프라인의 후속 단계에서는 장면의 정점들을 세계 공간에 상대적으로 설명하기보다 카메라 좌표계에 상대적으로 설명하는 것이 편리합니다. 세계 공간에서 뷰 공간으로의 좌표 변환을 *뷰 변환*이라고 하며, 이에 대응하는 행렬을 *뷰 행렬*이라고 합니다.

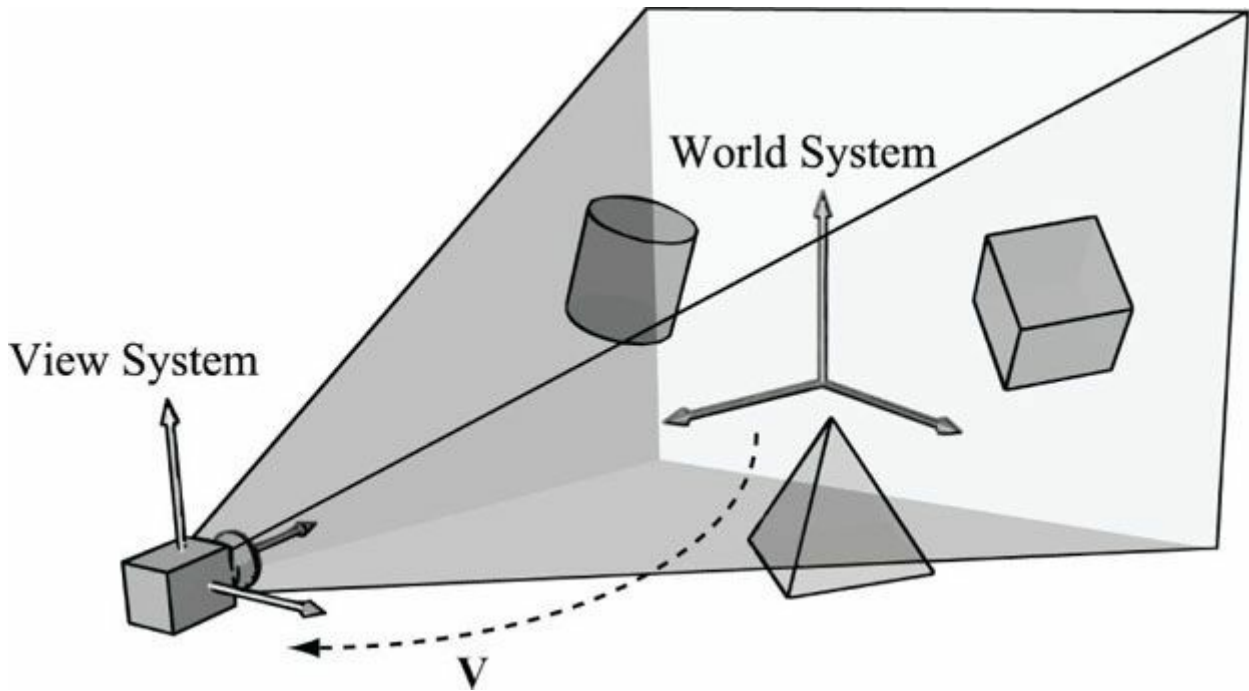


그림 5.19. 정점의 좌표를 월드 공간 상태에서 카메라 공간 상대가 되도록 변환합니다.

$\mathbf{Q}_w = (Q_x, Q_y, Q_z, 1)$, $\mathbf{u}_w = (u_x, u_y, u_z, 0)$, $\mathbf{v}_w = (v_x, v_y, v_z, 0)$, $\mathbf{w}_{(w)} = (w_{(x)}, w_{(y)}, w_{(z)}, 0)$ 및 $\mathbf{w}_w = (w_x, w_y, w_z, 0)$ 은 각각, $v_x, v_y, v_z, 0)$, $\mathbf{w}_w = (w_x, w_y, w_z, 0)$ 가 각각 세계 좌표계에 대한 등차 좌표로 표현된 뷰 공간의 원점, x축, y축, z축을 나타낸다면, §3.4.3에서 알 수 있듯이 뷰 공간에서 세계 공간으로의 좌표 변환 행렬은 다음과 같습니다:

$$\mathbf{W} = \begin{bmatrix} u_x & u_y & u_z & 0 \\ v_x & v_y & v_z & 0 \\ w_x & w_y & w_z & 0 \\ Q_x & Q_y & Q_z & 1 \end{bmatrix}$$

그러나 이는 우리가 원하는 변환이 아니다. 우리는 세계 공간에서 뷰 공간으로의 역변환을 원한다. 그러나 §3.4.5에서 역변환은 단순히 역행렬로 주어진다는 점을 상기하자. 따라서 $\mathbf{W}^{-1}$ 은 세계 공간에서 뷰 공간으로 변환한다.

세계 좌표계와 뷰 좌표계는 일반적으로 위치와 방향만 다르기 때문에, $\mathbf{W} = \mathbf{RT}$ (즉, 세계 행렬은 회전과 평행 이동으로 분해될 수 있음)라는 의미가 직관적으로 이해된다.

$\mathbf{W} = \mathbf{RT}$ (즉, 세계 행렬을 회전과 평행 이동의 순서로 분해할 수 있음)이라는 식이 직관적으로 이해됩니다. 이 형태는 역행렬 계산이 더 쉬워집니다:

$$\mathbf{V} = \mathbf{W}^{-1} = (\mathbf{RT})^{-1} = \mathbf{T}^{-1} \mathbf{R}^{-1} = \mathbf{T}^{-1} \mathbf{R}^T$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -Q_x & -Q_y & -Q_z & 1 \end{bmatrix} \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ -\mathbf{Q} \cdot \mathbf{u} & -\mathbf{Q} \cdot \mathbf{v} & -\mathbf{Q} \cdot \mathbf{w} & 1 \end{bmatrix}$$

따라서 뷰 행렬은 다음과 같은 형태를 가집니다:

$$\mathbf{V} = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ -\mathbf{Q} \cdot \mathbf{u} & -\mathbf{Q} \cdot \mathbf{v} & -\mathbf{Q} \cdot \mathbf{w} & 1 \end{bmatrix}$$

이제 뷰 매트릭스를 구성하는 데 필요한 벡터를 직관적으로 만드는 방법을 보여드리겠습니다. $\mathbf{Q}$ 를 카메라의 위치로, $\mathbf{T}$ 를 카메라가 조준하는 목표점으로 정의합니다. 또한 $\mathbf{j}$ 를 세계 좌표계의 '위' 방향을 나타내는 단위 벡터로 정의합니다. (이 책에서는 세계 xz 평면을 세계 '지면 평면'으로 사용하고 세계 y 축이 '위' 방향을 나타냅니다. 따라서 $\mathbf{j} = (0, 1, 0)$ 은 세계 y 축과 평행한 단위 벡터에 불과합니다. 그러나 이는 단지 관례일 뿐이며, 일부 응용 프로그램에서는 xy 평면을 지면 평면으로, z 축을 "위" 방향으로 선택할 수도 있습니다.) [그림 5.20](#)을 참조하면, 카메라가 바라보는 방향은 다음과 같이 주어집니다:

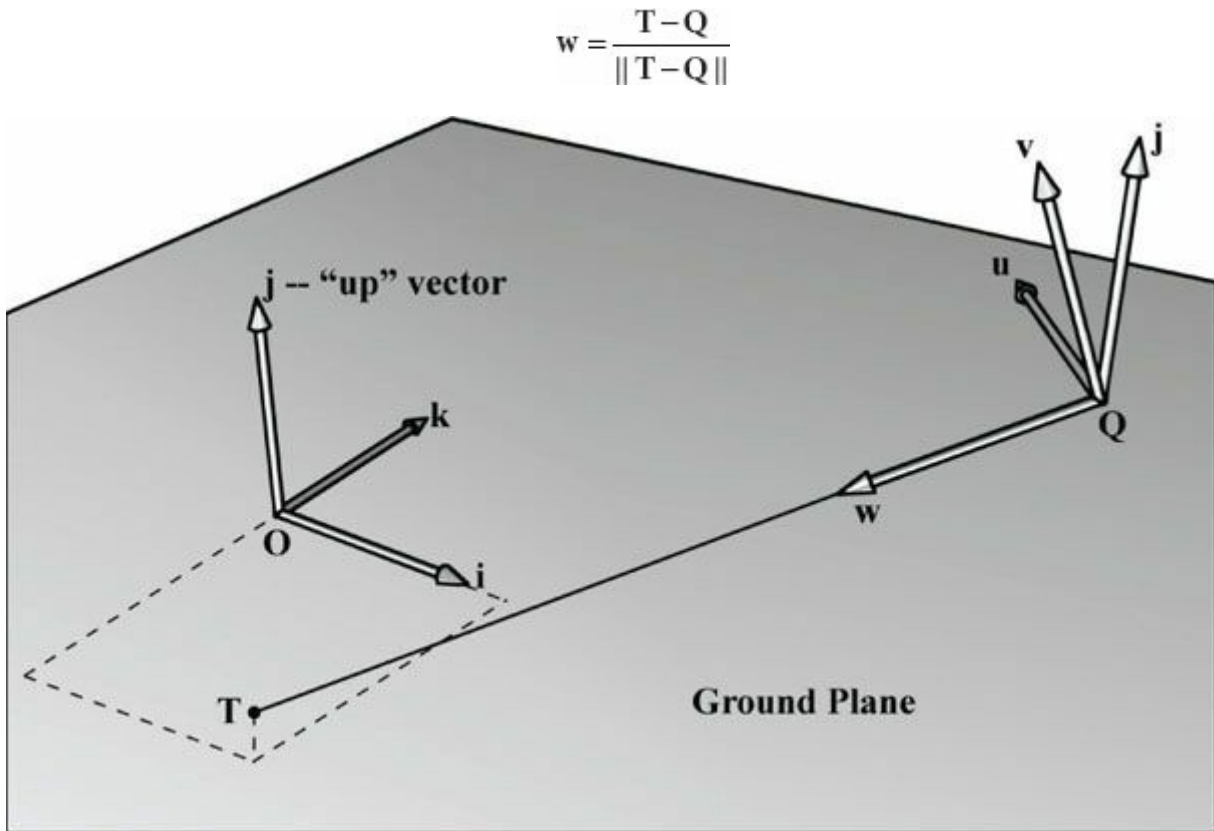


그림 5.20. 카메라 위치, 목표점, 세계 좌표계의 '위' 벡터를 기반으로 카메라 좌표계 구성.

이 벡터는 카메라의 로컬 z 축을 나타냅니다. $\mathbf{w}$ 의 "오른쪽"을 향하는 단위 벡터는 다음과 같습니다:

$$\mathbf{u} = \frac{\mathbf{j} \times \mathbf{w}}{\|\mathbf{j} \times \mathbf{w}\|}$$

이 벡터는 카메라의 로컬 x축을 나타냅니다. 마지막으로 카메라의 로컬 y축을 나타내는 벡터는 다음과 같습니다:

$$\mathbf{v} = \mathbf{w} \times \mathbf{u}$$

$\mathbf{w}$ 와 $\mathbf{u}$ 가 직교하는 단위 벡터이므로, $\mathbf{w} \times \mathbf{u}$ 는 필연적으로 단위 벡터이며, 따라서 정규화할 필요가 없습니다. 따라서 카메라 위치, 목표점, 그리고 세계 좌표계의 '위' 방향이 주어지면, 뷰 매트릭스 형성에 활용될 수 있는 카메라의 국소 좌표계를 도출할 수 있었다. 카메라의 국소 좌표계를 도출할 수 있었으며, 이를 사용하여 뷰 매트릭스를 구성할 수 있습니다. XNA Math 라이브러리는 방금 설명한 과정을 기반으로 뷰 매트릭스를 계산하기 위한 다음 함수를 제공합니다:

```
XMMATRIX XMMatrixLookAtLH( // 결과 뷰 매트릭스 출력 V FXMVECTOR EyePosition, // 입력 카메라 위치
                           // Q FXMVECTOR FocusPosition, // 입력 대상점 T FXMVECTOR UpDirection); // 입력 세계 상향
                           벡터 j
```

일반적으로 월드 좌표계의 y축은 "위" 방향을 나타내므로, "위" 벡터는 거의 항상 $\mathbf{j} = (0, 1, 0)$ 입니다. 예를 들어, 카메라를 월드 공간 상의 점 $(5, 3, -10)$ 에 위치시키고 카메라가 월드 원점 $(0, 0, 0)$ 을 바라보게 하려면 다음과 같이 뷰 매트릭스를 구성할 수 있습니다. 뷰 매트릭스는 다음과 같이 작성하여 생성할 수 있습니다:

```
XMVECTOR pos = XMVectorSet(5, 3, -10, 1.0f);
XMVECTOR target = XMVectorZero();
XMVECTOR up = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);

XMMATRIX V = XMMatrixLookAtLH(pos, target, up);
```

5.6.3 투영 및 동차 클립 공간

지금까지 우리는 세계 좌표계에서의 카메라 위치와 방향을 설명해왔습니다. 그러나 카메라에는 또 다른 구성 요소가 있는데, 바로 카메라가 보는 공간의 부피입니다. 이 부피는 프러스텀(frustum)으로 표현됩니다([그림 5.21](#)).

다음 작업은 원뿔형 투영체 내부의 3차원 기하학을 2차원 투영 창에 투영하는 것입니다. 투영은 평행선이 소실점으로 수렴하는 방식과 동일하게 수행되어야 하며, 물체의 3차원 깊이가 증가할수록 그 투영 크기는 감소합니다. 투시 투영은 이러한 특성을 구현하며, 이는 [그림 5.22](#)에 설명되어 있습니다.

평행선이 한 점으로 수렴하고, 물체의 3D 깊이가 증가할수록 그 투영 크기가 줄어들도록 해야 합니다. 투시 투영은 이를 구현하며, [그림 5.22](#)에 설명되어 있습니다. 정점에서 시점까지의 선을 *정점의 투영선*이라고 부릅니다. 그런 다음 *원근 투영 변환*을 3차원 정점 $\mathbf{v}$ 를 그 투영선이 2차원 투영 평면과 교차하는 점 $\mathbf{v'}$ 로 변환하는 변환으로 정의합니다. $\mathbf{v'}$ 를 $\mathbf{v}$ 의 투영이라고 합니다. 3차원 객체의 투영은 해당 객체를 구성하는 모든 정점의 투영을 의미합니다.

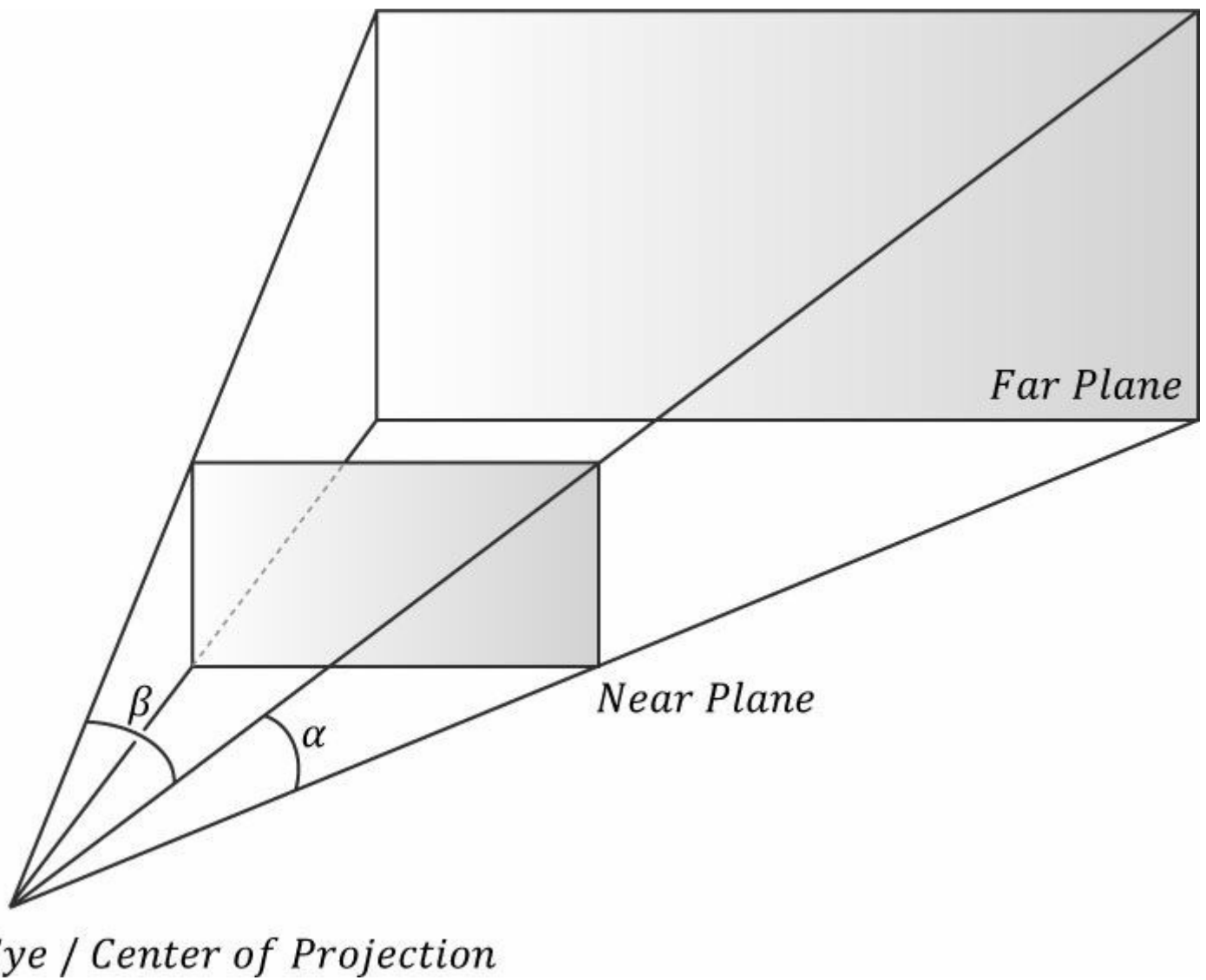


그림 5.21. 원뿔체는 카메라가 "보는" 공간의 부피를 정의합니다.

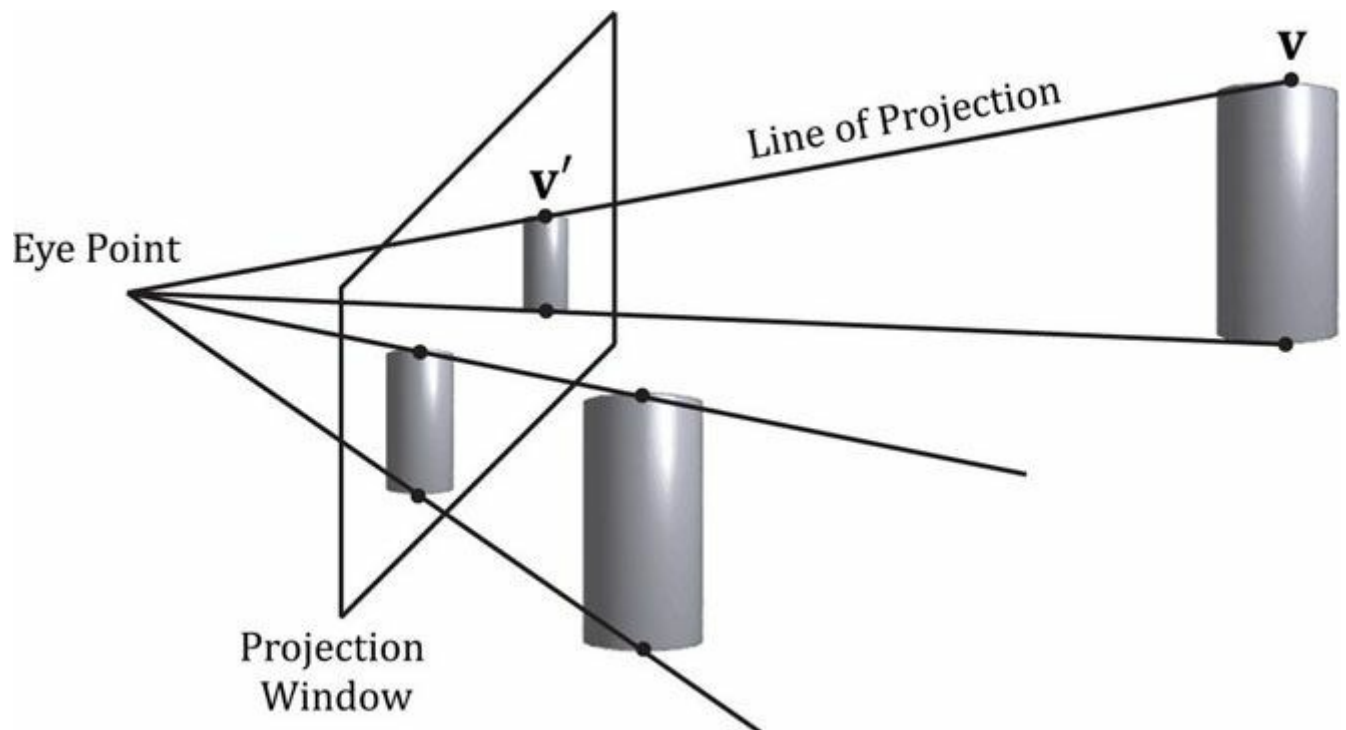


그림 5.22. 3차원 공간에 있는 두 원통은 크기가 같지만 서로 다른 깊이에 배치되어 있습니다. 눈 가까이 있는 원통의 투영은 멀리 있는 원통의 투영보다 큼. 원뿔체 내부의 기하학적 구조는 투영 창에 투영되며, 원뿔체 외부의 기하학적 구조는 투영 평면에 투영되지만 투영 창 밖에 위치하게 됩니다.

5.6.3.1 프러스텀 정의하기

시점 공간에서 투영 중심을 원점에 두고 양의 z 축을 따라 아래를 향하는 원뿔형 시야(frustum)는 다음 네 가지 값으로 정의할 수 있습니다: 근거리 평면 n , 원거리 평면 f , 수직 시야각 a , 종횡비 r . 시점 공간에서 근거리 평면과 원거리 평면은 xy 평면에 평행하므로, 단순히 z 축을 따라 원점으로부터의 거리를 지정하면 됩니다. 종횡비는 $r = w/h$ 로 정의되며, 여기서 w 는 투영 창 너비이고 h 는 투영 창 높이입니다(시점 공간 단위). 투영 창은 본질적으로 시점 공간에서 장면의 2D 이미지입니다. 이 이미지는 결국 백 버퍼에 매핑되므로, 투영 창 치수의 비율이 백 버퍼 치수의 비율과 동일하기를 원합니다. 따라서 백 버퍼 치수의 비율은 일반적으로 종횡비로 지정됩니다(비율이기 때문에 단위가 없습니다). 예를 들어 백 버퍼 치수가 800×600 인 경우 $r = \frac{800}{600} \approx 1.333$ 로 지정합니다. 투영 창과 백 버퍼의 종횡비가 같지 않다면, 투영 창을 백 버퍼에 매핑하기 위해 비균일한 스케일링이 필요하게 되며, 이는 왜곡을 일으킬 수 있습니다(예: 투영 창에 있는 원이 백 버퍼에 매핑될 때 타원으로 늘어나게 될 수 있음).

수평 시야각을 β 로 표기하며, 이는 수직 시야각 α 와 종횡비 r 에 의해 결정됩니다.

r/β 를 찾는 데 어떻게 도움이 되는지 알아보려면 [그림 5.23](#)을 고려하십시오. 실제 투영 창의 크기는 중요하지 않으며, 종횡비만 유지되어야 합니다. 따라서 편리한 높이 2를 선택하면 너비는 다음과 같아야 합니다:

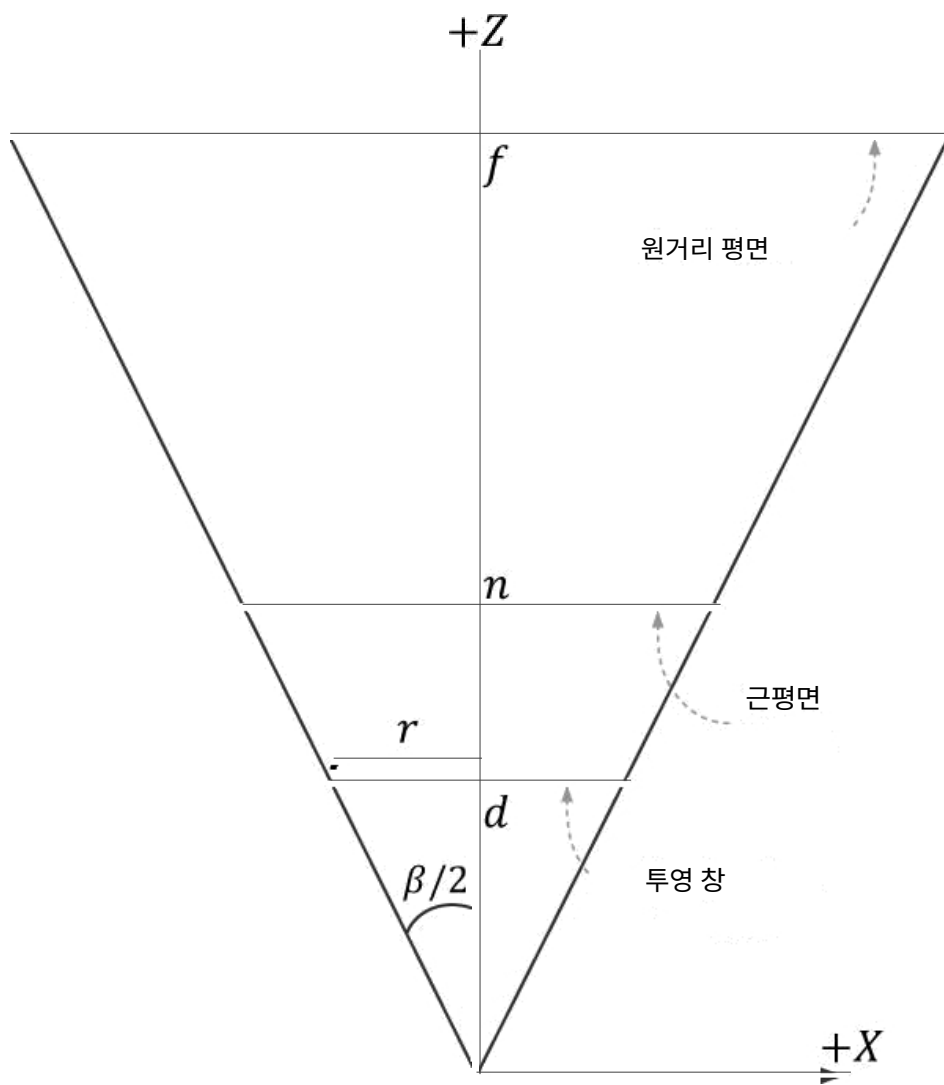
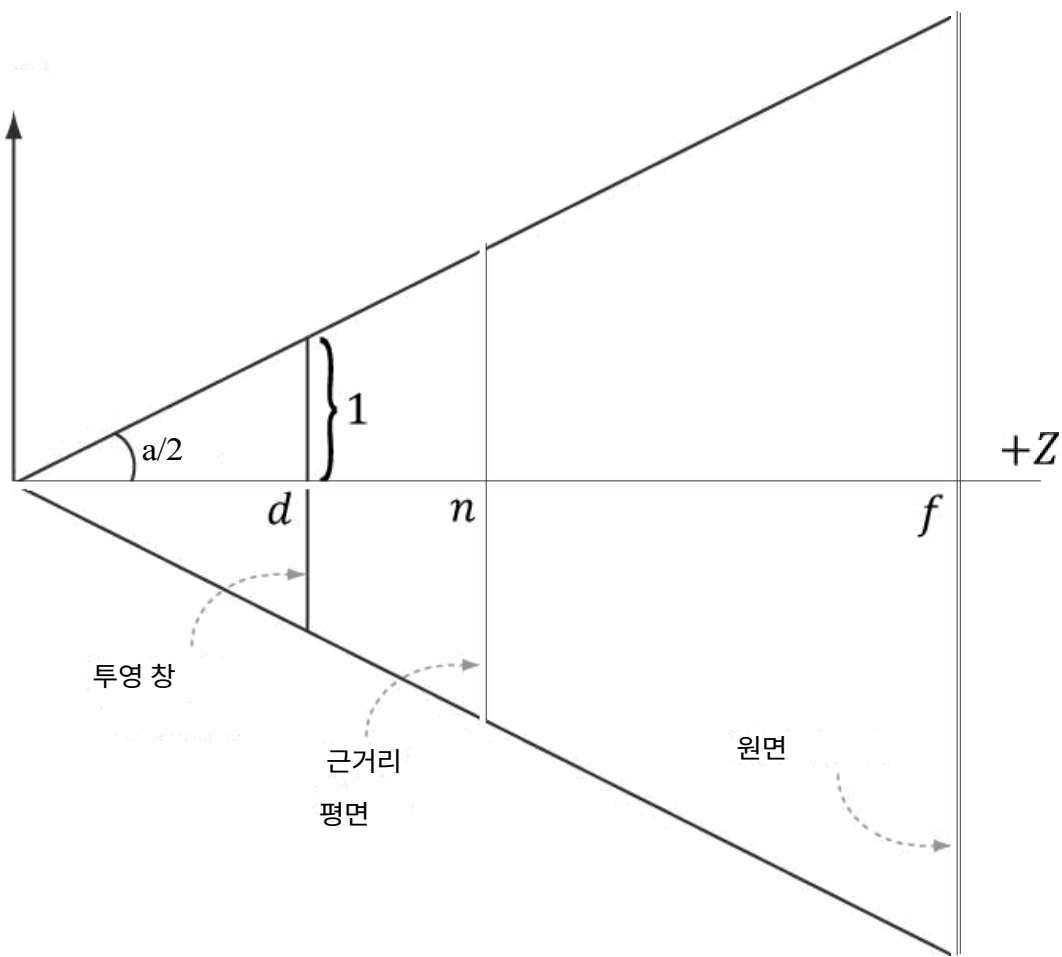


그림 5.23. 수직 시야각 α 와 종횡비 r 을 주어 수평 시야각 β 를 도출함.

$$r = \frac{w}{h} = \frac{w}{2} \Rightarrow w = 2r$$

지정된 수직 시야각 α 를 가지려면 투영 창은 원점으로부터 거리 d 만큼 떨어져 있어야 합니다.

$$\tan\left(\frac{\alpha}{2}\right) = \frac{1}{d} \Rightarrow d = \cot\left(\frac{\alpha}{2}\right)$$

이제 투영 창의 높이가 2일 때 수직 시야각 α 를 갖도록 z 축을 따라 투영 창의 거리 d 를 고정했습니다. 이제 β 를 구할 수 있습니다. 그림 5.23의 xz 평면을 보면 다음과 같습니다:

$$\begin{aligned} \tan\left(\frac{\beta}{2}\right) &= \frac{r}{d} = \frac{r}{\cot\left(\frac{\alpha}{2}\right)} \\ &= r \cdot \tan\left(\frac{\alpha}{2}\right) \end{aligned}$$

따라서 수직 시야각 α 와 종횡비 r 이 주어지면, 항상 수평 시야각 β 를 구할 수 있습니다.

$$\beta = 2 \tan^{-1}\left(r \cdot \tan\left(\frac{\alpha}{2}\right)\right)$$

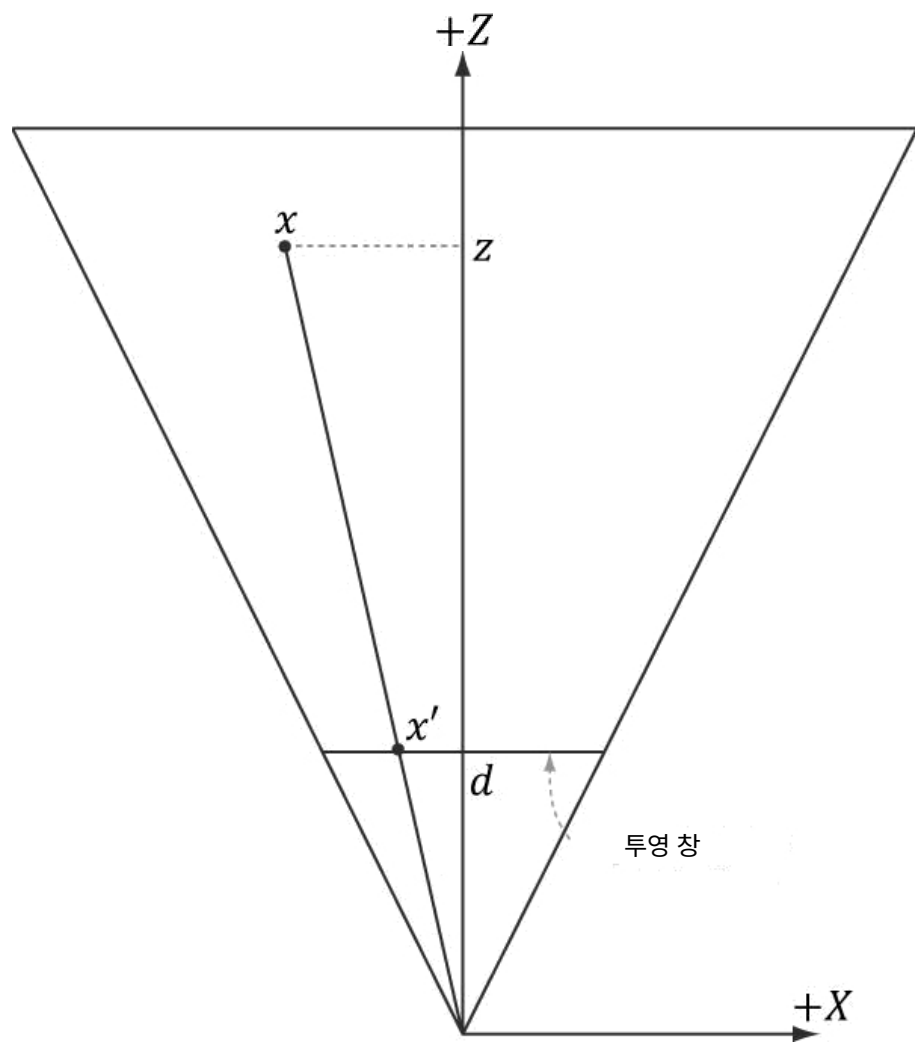
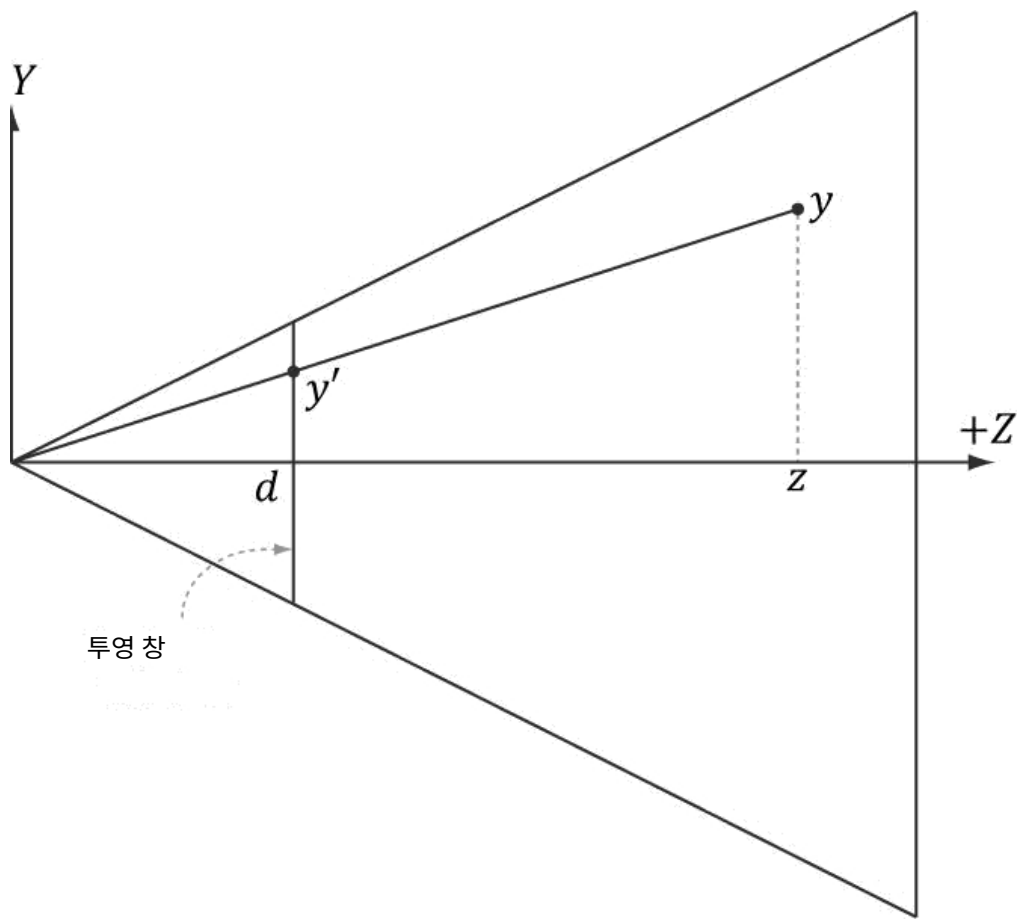
5.6.3.2 정점 투영

그림 5.24를 참조하십시오. 점 (x, y, z) 가 주어졌을 때, 투영면 $z = d$ 상에서의 투영점 (x', y', d) 를 구하고자 합니다. x 좌표와 y 좌표를 각각 따로 고려하고 유사삼각형을 이용하면 다음과 같이 구할 수 있습니다:

$$\frac{x'}{d} = \frac{x}{z} \Rightarrow x' = \frac{xd}{z} = \frac{x \cot(\alpha/2)}{z} = \frac{x}{z \tan(\alpha/2)}$$

그리고

$$\frac{y'}{d} = \frac{y}{z} \Rightarrow y' = \frac{yd}{z} = \frac{y \cot(\alpha/2)}{z} = \frac{y}{z \tan(\alpha/2)}$$



점 (x, y, z) 가 절두체 내에 있을 때만 다음 조건이 성립함을 관찰하라.

$$\begin{aligned} -r &\leq x' \leq r \\ -1 &\leq y' \leq 1 \\ n &\leq z \leq f \end{aligned}$$

5.6.3.3 정규화된 장치 좌표계(NDC)

이전 섹션에서 투영된 점의 좌표는 뷰 공간에서 계산됩니다. 뷰 공간에서 투영 창 의 높이는 2, 너비는 $2r$ (여기서 r 은 종횡비)입니다. 문제는 이 치수가 종횡비에 의존한다는 점입니다. 이는 하드웨어가 나중에 투영 창 의 크기와 관련된 작업(예: 백 버퍼에 매핑)을 수행해야 하므로 하드웨어에 종횡비를 알려줘야 함을 의미합니다. 종횡비에 대한 이 의존성을 제거할 수 있다면 더 편리할 것입니다. 해결책은 투영된 x 좌표를 $[-r, r]$ 구간에서 다음과 같이 $[-1, 1]$ 구간으로 스케일링하는 것입니다:

$$\begin{aligned} -r &\leq x' \leq r \\ -1 &\leq x' / r \leq 1 \end{aligned}$$

이 매핑 후 x 및 y 좌표는 *정규화된 장치 좌표*(NDC)라고 하며(z 좌표는 아직 정규화되지 않음), 점 (x, y, z) 는 다음 조건을 만족할 때만 프러스텀 내에 위치한다.

$$\begin{aligned} -1 &\leq x' / r \leq 1 \\ -1 &\leq y' \leq 1 \\ n &\leq z \leq f \end{aligned}$$

시점 공간에서 NDC 공간으로의 변환은 단위 변환으로 볼 수 있습니다. x 축에서 NDC 1 단위는 시점 공간의 r 단위(즉, $1 \text{ ndc} = r \text{ vs}$)에 해당합니다. 따라서 x 시점 공간 단위가 주어지면 이 관계를 이용해 단위를 변환할 수 있습니다:

$$x \text{ vs} \cdot \frac{1 \text{ ndc}}{r \text{ vs}} = \frac{x}{r} \text{ ndc}$$

투영 공식을 수정하여 투영된 x 및 y 좌표를 NDC 좌표로 직접 구할 수 있습니다:

$$\begin{aligned} x' &= \frac{x}{r z \tan(\alpha / 2)} \\ y' &= \frac{y}{z \tan(\alpha / 2)} \end{aligned} \quad (\text{eq. 5.1})$$

NDC 좌표계에서 투영 창은 높이 2, 너비 2를 가집니다. 따라서 이제 치수가 고정되어 하드웨어는 종횡비를 알 필요가 없지만, 투영된 좌표를 항상 NDC 공간으로 제공해야 하는 것은 우리의 책임입니다(그래픽스 하드웨어는 우리가 그렇게 할 것이라고 가정합니다).

5.6.3.4 행렬을 이용한 투영 방정식 작성

일관성을 위해 투영 변환을 행렬로 표현하고자 합니다. 그러나 식 5.1은 비선형이므로 행렬 표현이 불가능합니다. 여기서 '기법'은 이를 선형 부분과 비선형 부분으로 분리하는 것입니다. 비선형 부분은 z 로 나누는 연산입니다. 다음 절에서 논의하겠지만, z 좌표를 정규화할 예정입니다. 이는 나눗셈에 사용할 원래 z 좌표가 존재하지 않음을 의미합니다. 따라서 변환 전 입력 z 좌표를 저장해야 합니다. 이를 위해 동차 좌표를 활용하여 입력 z 좌표를 출력 w 좌표로 복사합니다. 행렬 곱셈으로 표현하면, $[2][3]$ 요소를 1로, $[3][3]$ 요소를 0(0부터 시작하는 인덱스)으로 설정하는 것입니다. 우리의 투영 행렬은 다음과 같습니다:

$$P = \begin{bmatrix} \frac{1}{r \tan(\alpha/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\alpha/2)} & 0 & 0 \\ 0 & 0 & A & 1 \\ 0 & 0 & B & 0 \end{bmatrix}$$

다음 절에서 결정될 상수 A 와 B 를 행렬에 배치했음을 유의하십시오. 이 상수들은 입력 z 좌표를 정규화된 범위로 변환하는 데 사용됩니다. 임의의 점 $(x, y, z, 1)$ 에 이 행렬을 곱하면 다음과 같습니다:

$$\begin{bmatrix} x, y, z, 1 \end{bmatrix} \begin{bmatrix} \frac{1}{r \tan(\alpha/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\alpha/2)} & 0 & 0 \\ 0 & 0 & A & 1 \\ 0 & 0 & B & 0 \end{bmatrix} = \begin{bmatrix} \frac{x}{r \tan(\alpha/2)}, \frac{y}{\tan(\alpha/2)}, Az + B, z \end{bmatrix}$$

(식 5.2)

투영 행렬(선형 부분)을 곱한 후, 각 좌표를 $w = z$ (비선형 부분)으로 나누어 변환을 완료합니다:

$$\begin{bmatrix} \frac{x}{r \tan(\alpha/2)}, \frac{y}{\tan(\alpha/2)}, Az + B, z \end{bmatrix} \xrightarrow{\text{divide by } w} \begin{bmatrix} \frac{x}{rz \tan(\alpha/2)}, \frac{y}{z \tan(\alpha/2)}, A + \frac{B}{z}, 1 \end{bmatrix}$$

(식 5.3)

한편, 제로로 나누는 오류가 발생할 수 있는지 궁금할 수 있으나, 근평면은 0보다 커야 하므로 그러한 점은 클리핑될 것이다 (§5.9). w 로 나누는 연산은 때로 *투시 분할(perspective divide)* 또는 *동형 분할(homogeneous divide)*이라고 불린다. 투영된 x 및 y 좌표가 방정식 5.1과 일치함을 확인할 수 있다.

5.6.3.5 정규화된 깊이 값

투영 후에는 모든 투영된 점이 이제 2차원 투영 창 위에 놓이게 되므로, 원래의 3차원 z 좌표를 버려도 될 것처럼 보일 수 있습니다. 이 투영 창은 눈으로 보는 2차원 이미지를 형성합니다. 그러나 깊이 버퍼링 알고리즘을 위해 여전히 3차원 깊이 정보가 필요합니다. Direct3D가 투영된 x 및 y 좌표를 정규화된 범위로 요구하는 것처럼, Direct3D는 깊이 좌표도 $[0, 1]$ 의 정규화된 범위로 요구합니다. 따라서 구간 $[n, f]$ 를 $[0, 1]$ 로 매핑하는 순서 보존 함수 $g(z)$ 를 구성해야 합니다.

함수가 순서 보존적이기 때문에, 만약 $z_1, z_2 \in [n, f]$ 이고 $z_1 < z_2$ 라면, $g(z_1) < g(z_2)$ 이다; 따라서 깊이 값들이 변환되었더라도 변환되었더라도 상대적 깊이 관계는 그대로 유지됩니다. 따라서 정규화된 구간 내에서 깊이를 정확히 비교할 수 있으며, 이는 깊이 버퍼링 알고리즘에 필요한 모든 조건을 충족합니다.

$[n, f]$ 를 $[0, 1]$ 로 매핑하는 것은 스케일링과 평행 이동으로 수행할 수 있습니다. 그러나 이 접근법은 현재의 투영 전략에 통합되지 않습니다. 방정식 5.3에서 z 좌표는 다음과 같은 변환을 거칩니다:

$$g(z) = A + \frac{B}{z}$$

이제 다음 제약 조건 하에 A 와 B 를 선택해야 합니다:

조건 1: $g(n) = A + B/n = 0$ (근평면이 0으로 매핑됨) 조건 2: $g(f) = A + B/f = 1$ (원평면이 1로 매핑됨)

조건 1을 B 에 대해 풀면: $B = -An$. 이를 조건 2에 대입하고 A 에 대해 풀면:

$$A + \frac{-An}{f} = 1$$

$$\frac{Af - An}{f} = 1$$

$$Af - An = f$$

$$A = \frac{f}{f - n}$$

따라서,

$$g(z) = \frac{f}{f - n} - \frac{nf}{(f - n)z}$$

g 의 그래프([그림 5.25](#))는 이 함수가 엄격히 증가(순서 보존)하고 비선형적임을 보여줍니다. 또한 대부분의 범위가 근평면에 가까운 깊이 값들에 의해 "소진"됨을 보여줍니다. 결과적으로 대부분의 깊이 값들은 범위의 작은 부분집합으로 매핑됩니다. 이는 깊이 버퍼 정밀도 문제(유한한 수치 표현으로 인해 변환된 깊이 값들 간의 미세한 차이를 컴퓨터가 더 이상 구분할 수 없음)로 이어질 수 있습니다. 일반적인 권고 사항은 근거리 평면과 원거리 평면을 가능한 한 가깝게 설정하여 깊이 정밀도 문제를 최소화하는 것이다.

이제 A 와 B 를 구했으므로 완전한 *원근 투영 행렬*을 다음과 같이 표현할 수 있습니다:

$$P = \begin{bmatrix} \frac{1}{r \tan(\alpha/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\alpha/2)} & 0 & 0 \\ 0 & 0 & \frac{f}{f - n} & 1 \\ 0 & 0 & \frac{-nf}{f - n} & 0 \end{bmatrix}$$

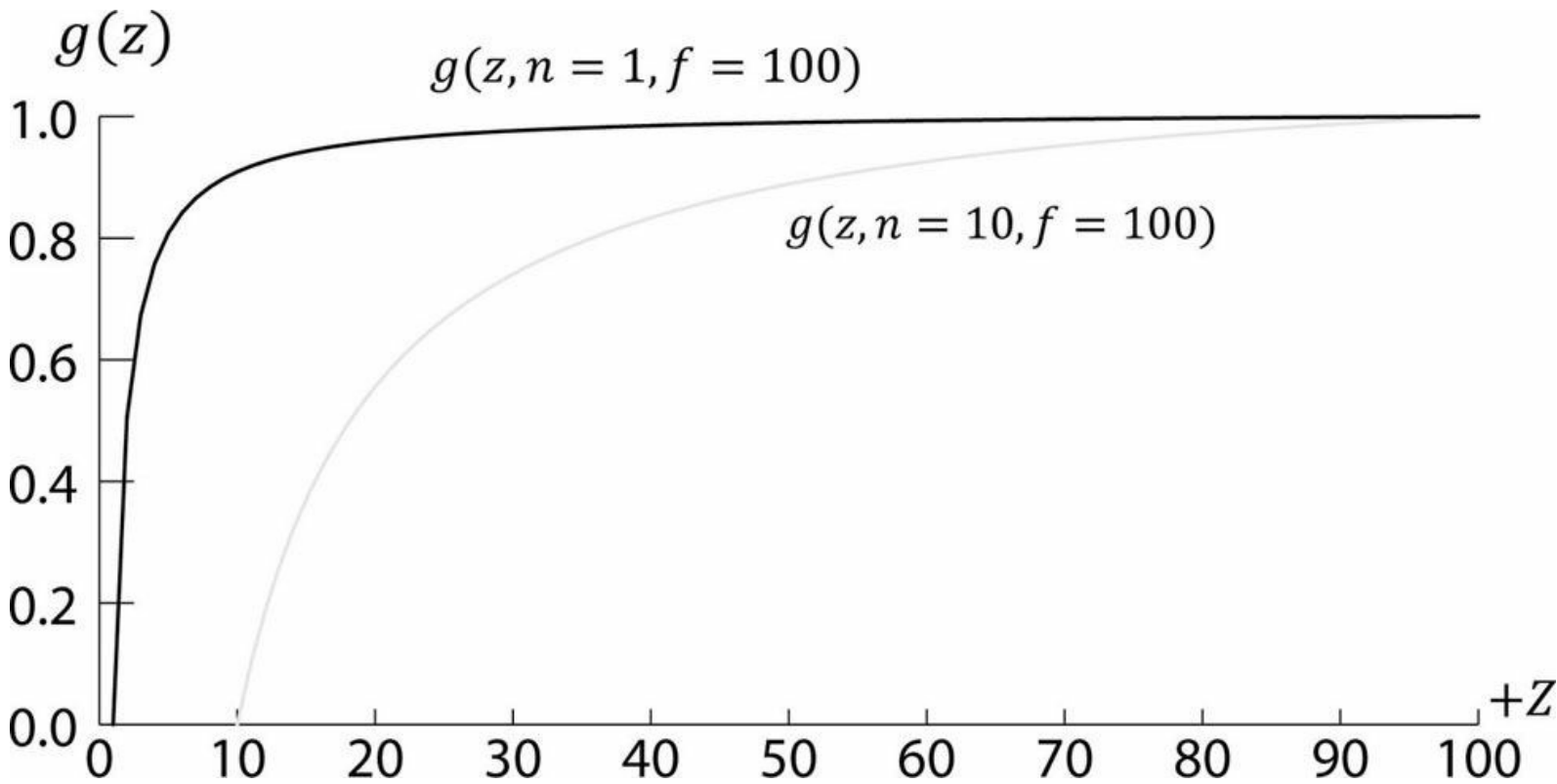


그림 5.25. 다양한 근평면에 대한 $g(z)$ 의 그래프.

투영 행렬을 곱한 후, 원근 분할 전에 기하학적 데이터는 *동차 클립 공간* 또는 *투영 공간*에 있다고 합니다. 원근 분할 후에는 기하학적 데이터가 정규화된 장치 좌표계(NDC)에 있다고 합니다.

5.6.3.6 XMMatrixPerspectiveFovLH

다음 XNA Math 함수를 사용하여 투시 투영 행렬을 생성할 수 있습니다:

```
XMATRIX XMMatrixPerspectiveFovLH( // 투영 행렬 반환 FLOAT FovAngleY, // 수직 시야각(라디안)
    FLOAT AspectRatio, // 종횡비 = 너비 / 높이
    FLOAT NearZ, // 근면 거리 FLOAT FarZ); // 원면 거리
```

다음 코드 조각은 D3DXMatrixPerspectiveFovLH 사용법을 보여줍니다. 여기서 수직 시야각 45°, 근면 $z = 1$, 원면 $z = 1000$ (이 길이는 뷰 공간 기준)을 지정합니다.

```
XMATRIX P = XMMatrixPerspectiveFovLH(0.25f*MathX::Pi, AspectRatio(), 1.0f, 1000.0f);
```

종횡비는 창 종횡비와 일치하도록 설정합니다:

```
float D3DApp::AspectRatio()const
{
    return static_cast<float>(mClientWidth) / mClientHeight;
}
```

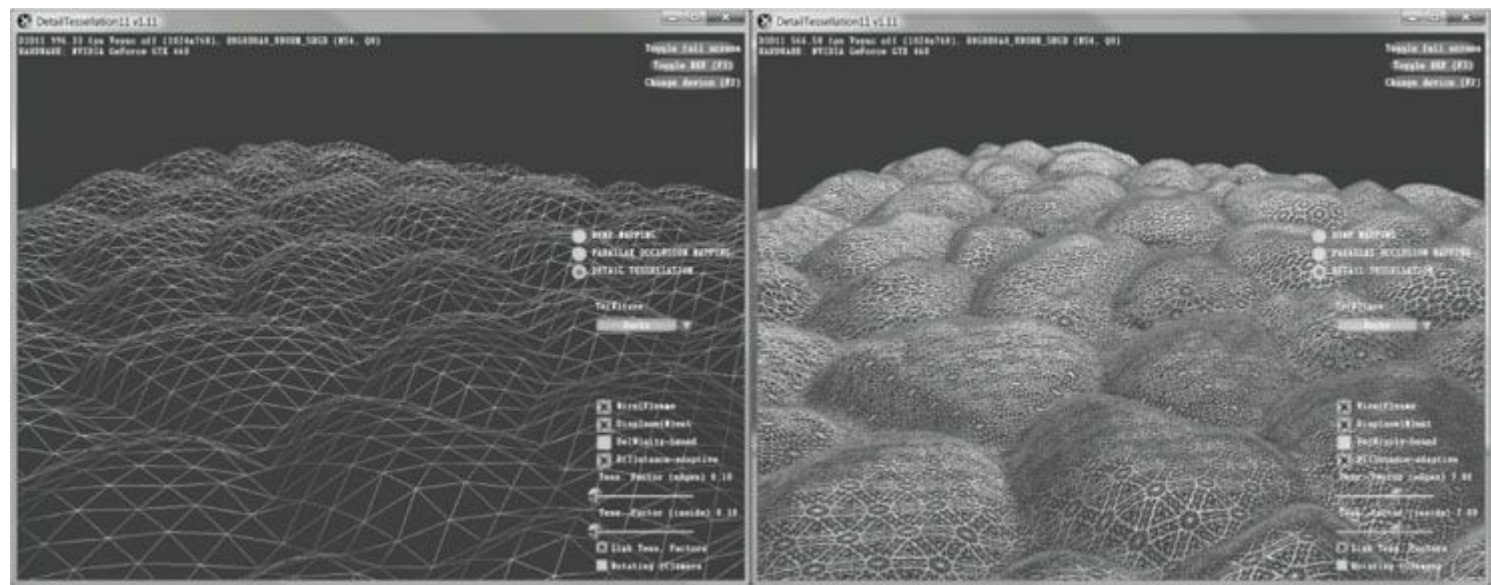
5.7 테셀레이션 단계

테셀레이션은 메시의 삼각형을 세분화하여 새로운 삼각형을 추가하는 것을 의미합니다. 이렇게 생성된 새로운 삼각형들은 오프셋 처리되어 새로운 위치로 이동되며, 이를 통해 더 정밀한 메시 디테일을 구현할 수 있습니다(그림 5.26 참조).

테셀레이션에는 여러 이점이 있습니다:

1. 레벨 오브 디테일(LOD) 메커니즘을 구현할 수 있습니다. 카메라에 가까운 삼각형은 디테일을 추가하기 위해 테셀레이션 처리하고, 카메라에서 먼 삼각형은 테셀레이션하지 않습니다. 이렇게 하면 추가 디테일이 눈에 띄는 부분에만 더 많은 삼각형을 사용하게 됩니다.
2. 메모리에는 단순한 저폴리메쉬(저폴리는 삼각형 수가 적음을 의미)를 유지하고, 추가 삼각형은 실시간으로 생성하므로

테셀레이션 단계는 Direct3D 11에서 새롭게 도입된 기능으로, GPU에서 지오메트리를 테셀레이션하는 방법을 제공합니다. Direct3D 11 이전에는 테셀레이션을 구현하려면 CPU에서 수행한 후, 새로 테셀레이션된 지오메트리를 렌더링을 위해 GPU로 다시 업로드해야 했습니다. 그러나 CPU 메모리에서 GPU 메모리로 새로운 지오메트리를 업로드하는 것은 느릴 뿐만 아니라, 테셀레이션 연산으로 CPU에 부담을 줍니다. 이러한 이유로 Direct3D 11 이전에는 테셀레이션 기법이 실시간 그래픽스에서 널리 사용되지 않았습니다. Direct3D 11은 Direct3D 11을 지원하는 그래픽 카드에서 하드웨어로 완전히 테셀레이션을 수행할 수 있는 API를 제공합니다. 이로 인해 테셀레이션은 훨씬 더 매력적인 기법이 되었습니다. 테셀레이션 단계는 선택 사항입니다(테셀레이션을 원할 때만 사용하면 됩니다). 테셀레이션에 대한 설명은 [13장에서](#) 다루도록 하겠습니다.



5.8 지오메트리 셰이더 단계

지오메트리 셰이더 단계는 선택 사항이며, **11장까지** 사용하지 않으므로 여기서는 간략히 설명하겠습니다. 지오메트리 셰이더는 전체 프리미티브를 입력으로 받습니다. 예를 들어 삼각형 리스트를 그리는 경우, 지오메트리 셰이더의 입력은 삼각형을 정의하는 세 개의 정점이 됩니다. (이 세 정점은 이미 버텍스 셰이더를 통과한 상태임을 유의하십시오.) 지오메트리 셰이더의 주요 장점은 지오메트리를 생성하거나 파괴할 수 있다는 점입니다. 예를 들어 입력된 프리미티브를 하나 이상의 다른 프리미티브로 확장하거나, 특정 조건에 따라 프리미티브를 출력하지 않도록 선택할 수 있습니다. 이는 정점을 생성할 수 없는 버텍스 셰이더와 대조됩니다: 버텍스 셰이더는 하나의 정점을 입력받아 하나의 정점을 출력합니다. 지오메트리 셰이더의 일반적인 예로는 점을 사각형으로 확장하거나 선분을 사각형으로 확장하는 경우가 있습니다.

또한 [그림 5.11](#)의 "스트림-아웃" 화살표를 확인할 수 있습니다. 즉, 지오메트리 셰이더는 정점 데이터를 메모리 버퍼로 스트림-아웃할 수 있으며, 후속 렌더링될 수 있습니다. 이는 고급 기법으로, 후속 장에서 논의될 예정입니다.

Note 지오메트리 셰이더를 떠나는 정점 위치는 동차 클립 공간으로 변환되어야 합니다.

5.9 클리핑

시야 프러스텀(viewing frustum) 외부로 완전히 벗어난 지오메트리는 제거해야 하며, 프러스텀 경계와 교차하는 지오메트리는 내부 부분만 남도록 클리핑해야 합니다. 2D로 설명된 개념은 [그림 5.27](#)을 참조하십시오.

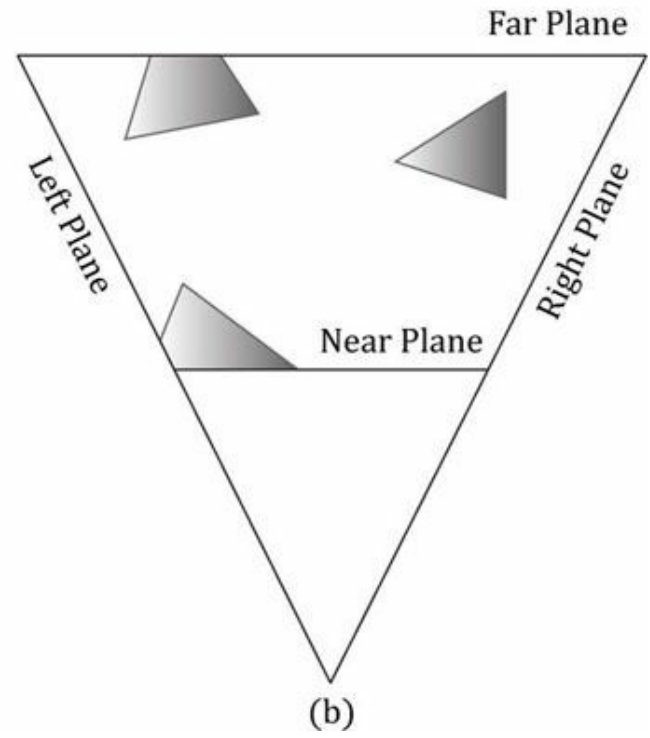
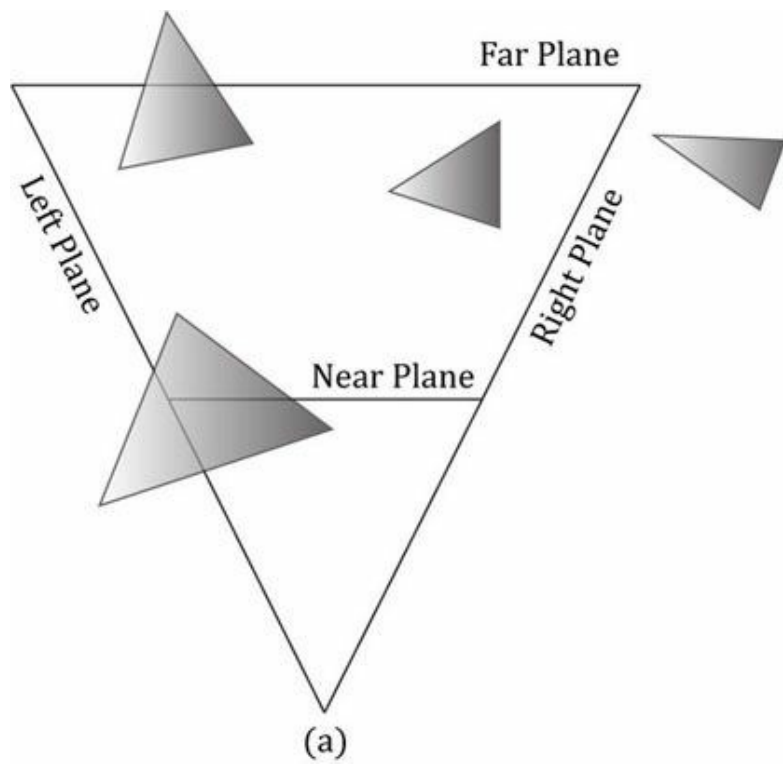


그림 5.27. (a) 클리핑 전. (b) 클리핑 후.

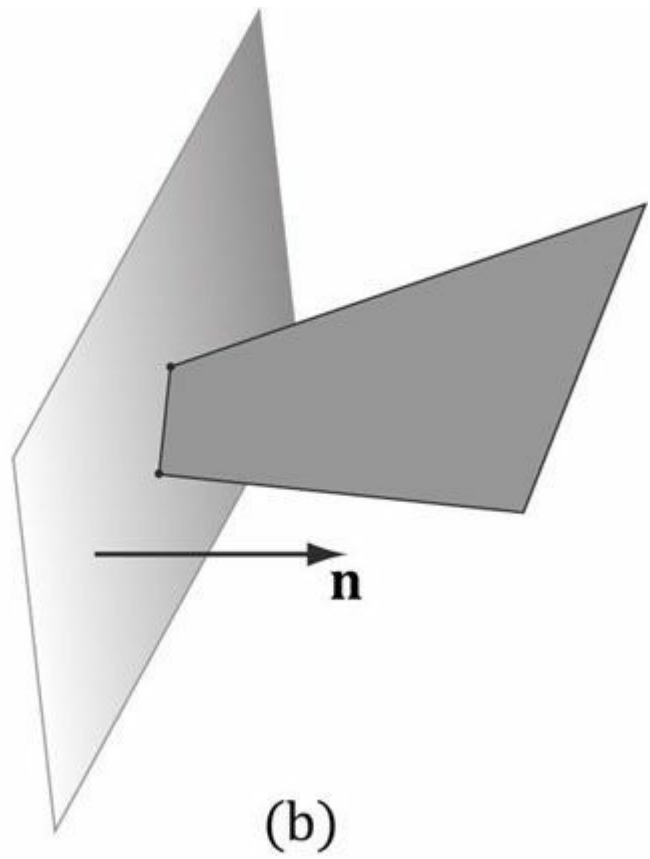
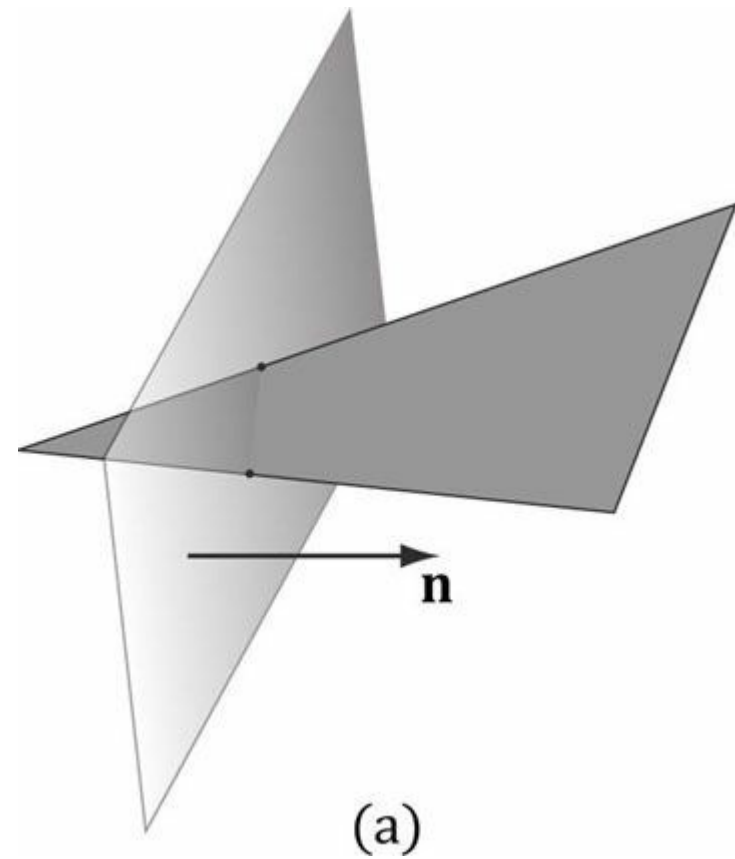


그림 5.28. (a) 평면에 대한 삼각형 클리핑. (b) 클리핑된 삼각형. 클리핑된 삼각형은 삼각형이 아닌 사각형을 유의하십시오. 따라서 하드웨어는 결과 사각형을 삼각형으로 분할해야 하며, 이는 볼록 다각형의 경우 간단히 수행할 수 있습니다.

프러스텀은 상단, 하단, 좌측, 우측, 근거리, 원거리 평면으로 둘러싸인 영역으로 생각할 수 있습니다. 다각형을 프러스텀에 대해 클리핑하려면 각 프러스텀 평면에 대해 하나씩 클리핑합니다. 평면에 대해 다각형을 클리핑할 때(그림 5.28), 평면의 양의 반공간에 있는 부분은 유지되고 음의 반공간에 있는 부분은 제거됩니다. 볼록 다각형을 평면에 대해 클리핑하면 항상 볼록 다각형이 결과로 나타납니다. 하드웨어가 클리핑을 수행해 주므로 여기서는 세부 사항을 다루지 않겠습니다. 대신 널리 알려진 서덜랜드-호지먼 클리핑 알고리즘[Sutherland74]을 참고하시기 바랍니다. 이 알고리즘은 기본적으로 평면과 다각형 변 사이의 교차점을 찾은 후, 정점들을 순서대로 배열하여 새로운 클리핑된 다각형을 형성하는 과정으로 요약됩니다.

[Blinn78]은 4차원 동차 공간에서 클리핑을 수행하는 방법을 설명합니다(그림 5.29). 투시 분할 후 뷰 프러스텀 내의 점들은 정규화된 장치 좌표계에 있으며 다음과 같이 제한됩니다:

$$\left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}, 1 \right)$$

시점 원뿔체 내의 점들은 정규화된 장치 좌표계(NDC)에 있으며 다음과 같이 제한됩니다:

$$\begin{aligned} -1 \leq x/w \leq 1 \\ -1 \leq y/w \leq 1 \\ 0 \leq z/w \leq 1 \end{aligned}$$

따라서 클리핑 공간에서 분할 전, 원뿔체 내부의 4차원 점 (x, y, z, w) 은 다음과 같이 제한됩니다:

$$\begin{aligned} -w \leq x \leq w \\ -w \leq y \leq w \\ 0 \leq z \leq w \end{aligned}$$

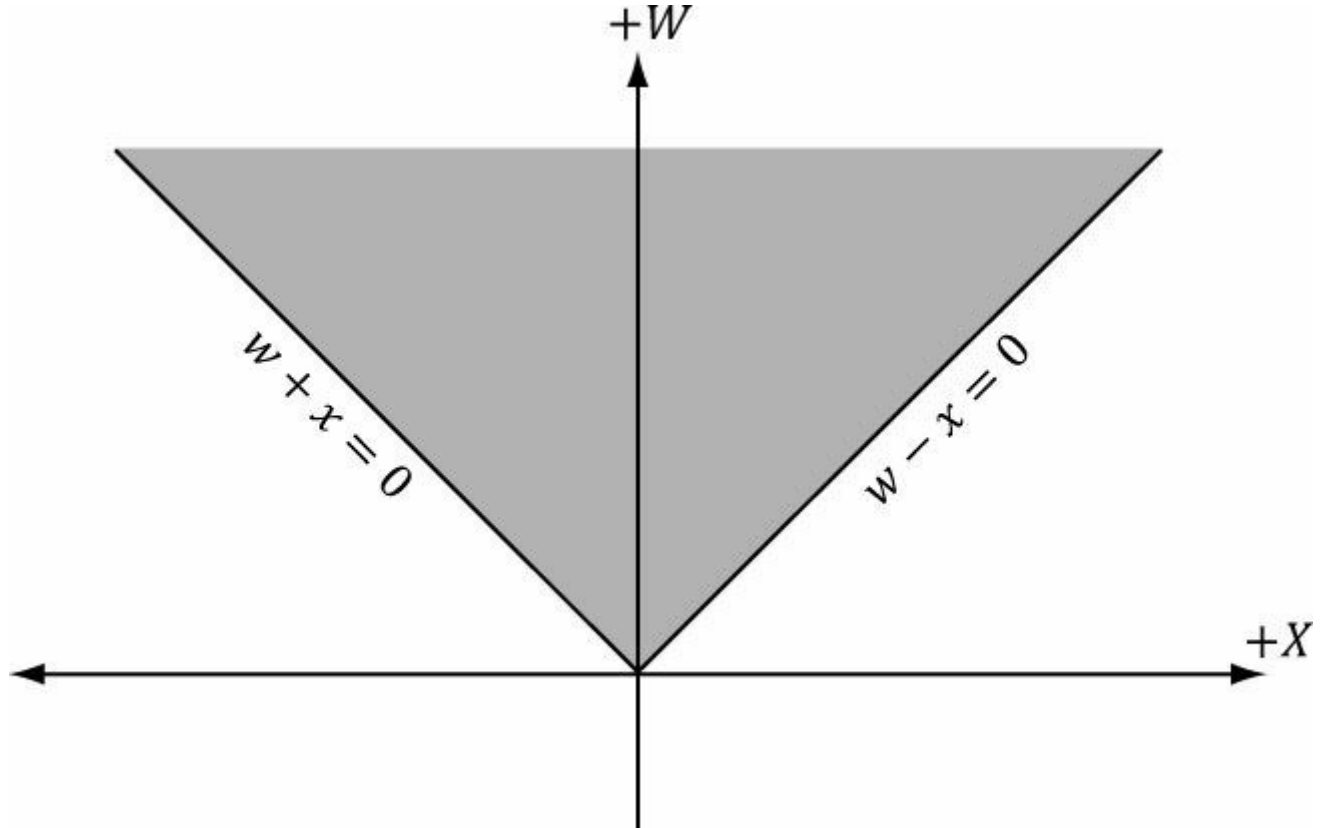


그림 5.29. 동차 클리핑 공간 내 xw 평면의 프러스텀 경계.

즉, 점들은 단순한 4차원 평면들에 의해 경계지워진다:

왼쪽: $w = -x$
 오른쪽: $w = x$
 아래: $w = -y$ 위: $w = y$
 가까운 면: $z = 0$ 면
 먼: $z = w$

동일 공간(homogeneous space)에서 원뿔 단면 평면 방정식을 알면, 클리핑 알고리즘(예: 서덜랜드-호지먼 알고리즘)을 적용할 수 있습니다. 선분/평면 교차 테스트의 수학은 4차원 공간($\mathbb{R}^4$)으로 일반화되므로, 동일 클리핑 공간에서 4차원 점과 4차원 평면을 사용하여 테스트를 수행할 수 있습니다.

5.10 래스터화 단계

래스터화 단계의 주요 작업은 투영된 3D 삼각형으로부터 픽셀 색상을 계산하는 것입니다.

5.10.1 뷰포트 변환

클리핑 후 하드웨어는 투시 분할을 수행하여 동질 클립 공간에서 정규화된 장치 좌표계(NDC)로 변환합니다. 정점들이 NDC 공간에 위치하면, 2D 이미지를 구성하는 2D x 및 y 좌표는 뷰포트라고 불리는 백 버퍼 상의 직사각형으로 변환됩니다(§4.2.8 참조). 이 변환 후 x 및 y 좌표는 픽셀 단위로 표현됩니다. 일반적으로

뷰포트 변환은 깊이 버퍼링에 사용되는 z 좌표를 수정하지 않지만, MinDepth와 D3D11_VIEWPORT 구조체의 MaxDepth 값, MinDepth 및 MaxDepth 값은 0과 1 사이여야 합니다.

5.10.2 뒷면 제거

삼각형은 두 개의 면을 가집니다. 두 면을 구분하기 위해 다음과 같은 규칙을 사용합니다. 삼각형의 정점 순서가 v_0, v_1, v_2 라면 삼각형 **법선 n** 을 다음과 같이 계산합니다:

$$\begin{aligned} \mathbf{e}_0 &= \mathbf{v}_1 - \mathbf{v}_0 \\ \mathbf{e}_1 &= \mathbf{v}_2 - \mathbf{v}_0 \\ \mathbf{n} &= \frac{\mathbf{e}_0 \times \mathbf{e}_1}{\|\mathbf{e}_0 \times \mathbf{e}_1\|} \end{aligned}$$

법선 벡터가 발산하는 면이 *앞면*이고 다른 면이 *뒷면*입니다. [그림 5.30](#)이 이를 설명합니다.

삼각형이 *정면*을 향한다고 말할 때는 관찰자가 삼각형의 앞면을 볼 때를 의미하며, 삼각형이 *뒷면*을 향한다고 말할 때는 관찰자가 삼각형의 뒷면을 보는 경우를 뒷면 삼각형이라고 합니다. [그림 5.30](#)의 관점에서 보면, 왼쪽 삼각형은 앞면 삼각형이고 오른쪽 삼각형은 뒷면 삼각형입니다. 또한 우리의 관점에서 왼쪽 삼각형은 시계 방향으로 배열된 반면 오른쪽 삼각형은 반시계 방향으로 배열됩니다. 이는 우연이 아닙니다. 우리가 선택한 관례(즉, 삼각형 법선을 계산하는 방식)에 따르면, 시계 방향으로 배열된 삼각형(해당 관찰자에 대해)은 정면을 향하고, 반시계 방향으로 배열된 삼각형(해당 관찰자에 대해)은 뒷면을 향합니다.

이제 3D 세계의 대부분의 객체는 폐쇄된 고체 객체입니다. 각 객체에 대해 삼각형을 다음과 같은 방식으로 구성하기로 합시다: 방식으로 삼각형을 구성하기로 합시다. 그러면 카메라가 고체 객체의 뒤쪽을 향한 삼각형을 볼 수 없습니다. 앞쪽을 향한 삼각형이 뒤쪽을 향한 삼각형을 가리기 때문입니다. [그림 5.31](#)은 이를 2D로, [그림 5.32](#)는 3D로 보여줍니다. 앞쪽을 향한 삼각형이 뒤쪽을 향한 삼각형을 가리기 때문에, 뒤쪽 삼각형을 그리는 것은 의미가 없습니다. *백페이스 컬링(Backface culling)*은 파이프라인에서 뒤를 향한 삼각형을 제거하는 과정을 의미합니다. 이를 통해 처리해야 할 삼각형의 양을 절반으로 줄일 수 있습니다.

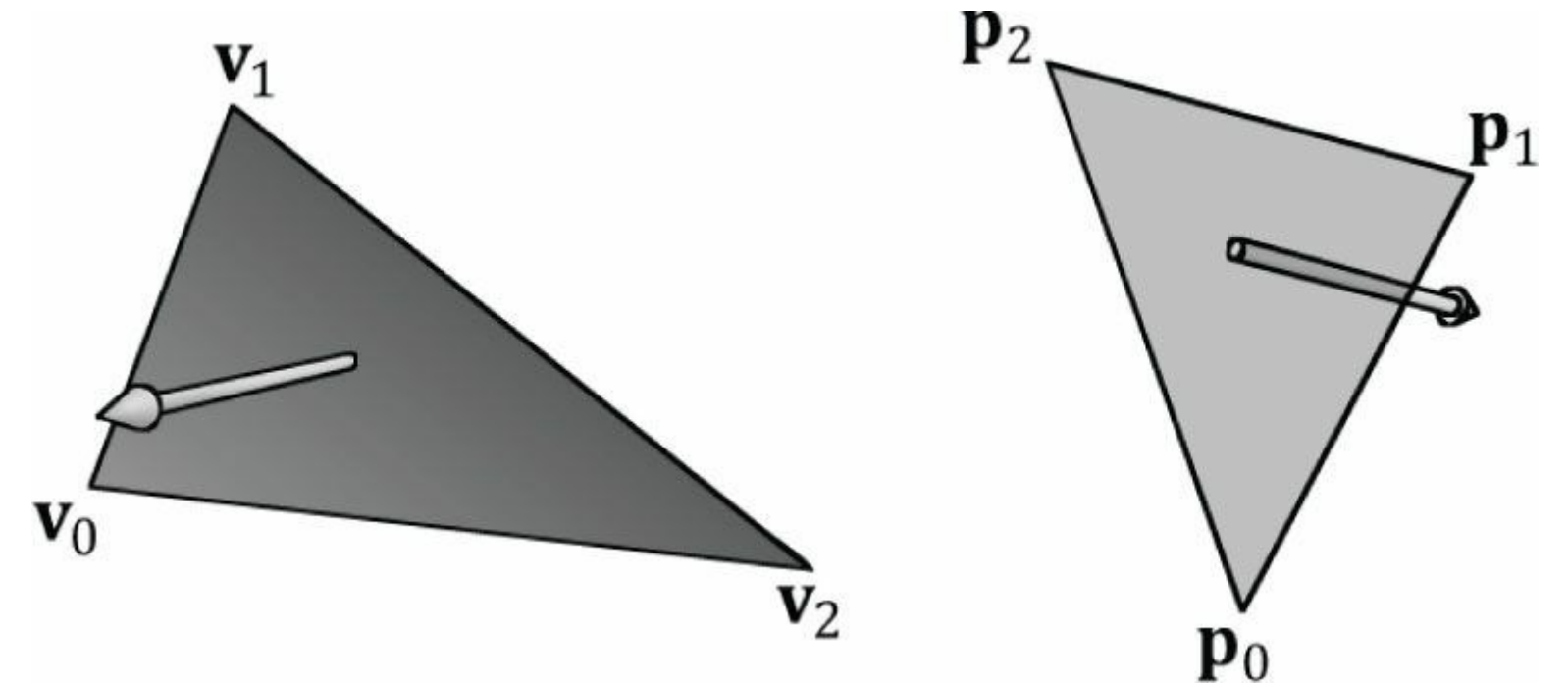
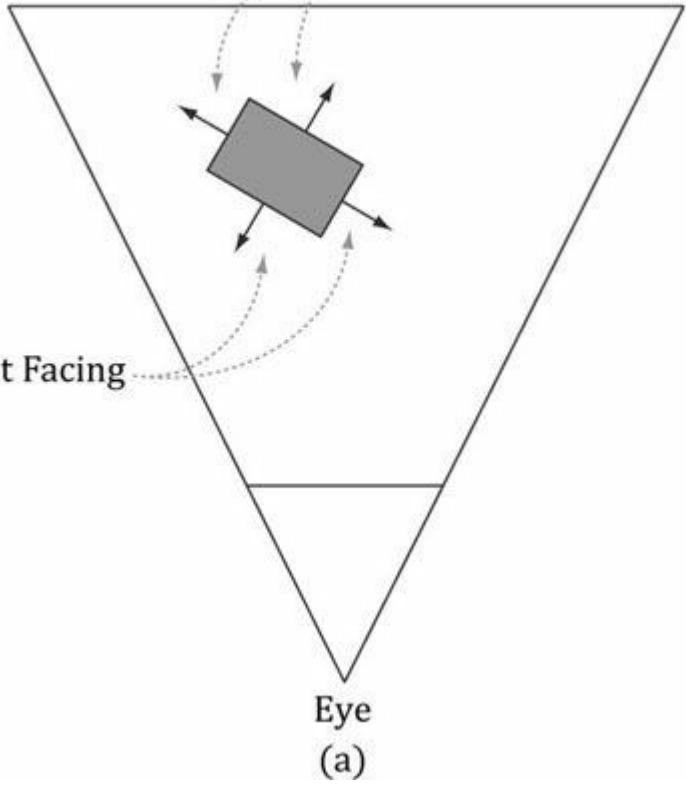


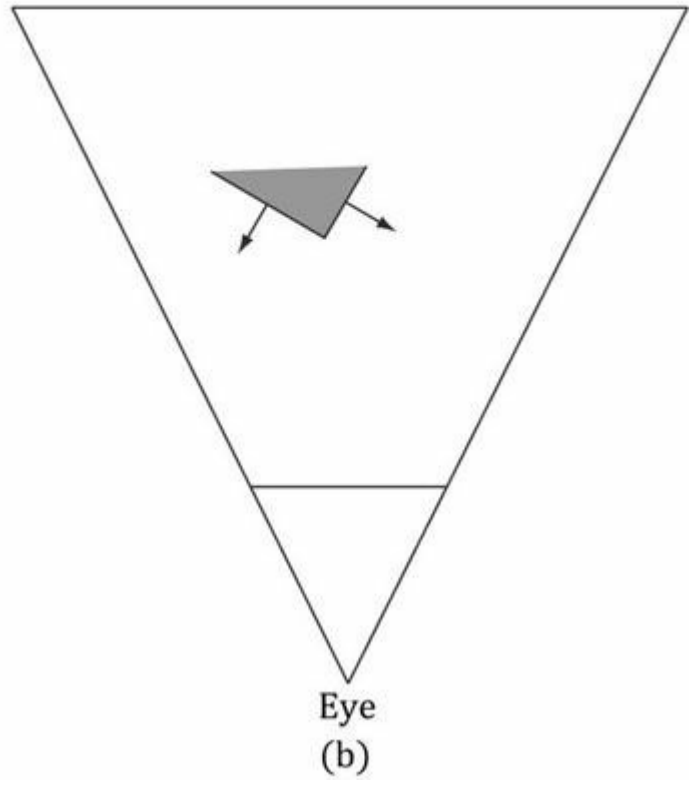
그림 5.30. 왼쪽 삼각형은 우리의 시점에서 정면을 향하고 있으며, 오른쪽 삼각형은 우리의 시점에서 뒷면을 향하고 있습니다.

Back Facing

Front Facing



Eye
(a)



Eye
(b)

그림 5.31. (a) 정면과 후면을 향한 삼각형이 있는 솔리드 오브젝트. (b) 후면 삼각형을 제거한 후의 장면. 후면 삼각형은 정면 삼각형에 의해 가려지기 때문에 후면 제거는 최종 이미지에 영향을 미치지 않음을 유의하십시오.

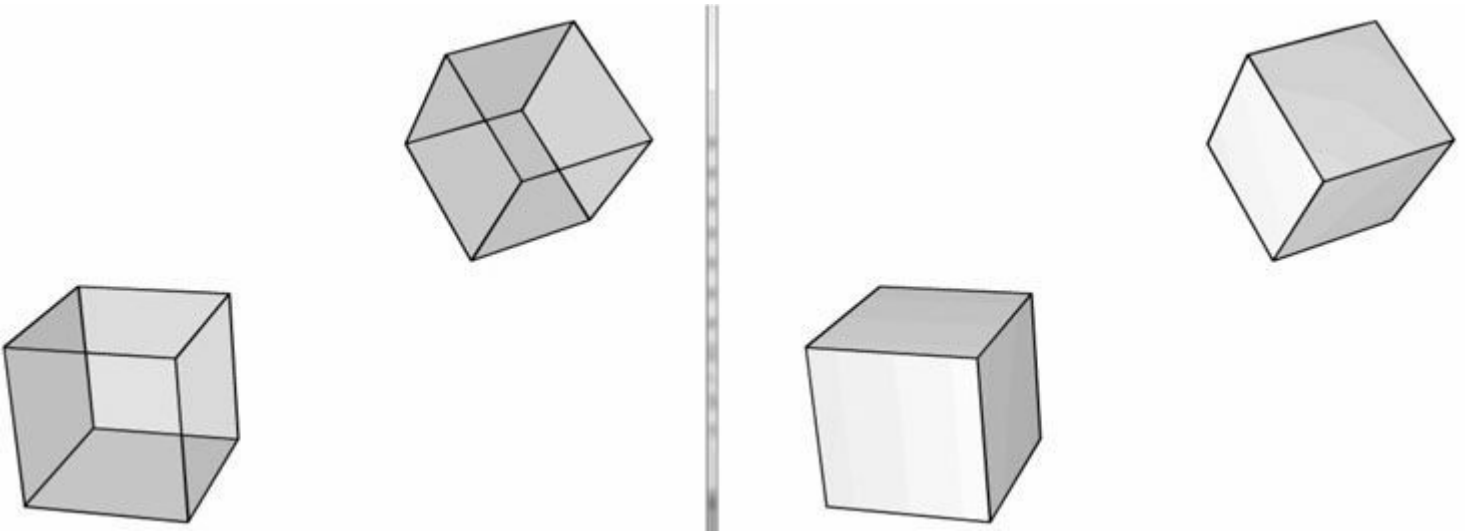


그림 5.32. (왼쪽) 모든 여섯 면을 볼 수 있도록 큐브를 투명하게 그렸습니다. (오른쪽) 큐브를 솔리드 블록으로 그렸습니다. 세 개의 앞면 삼각형이 세 개의 뒷면 삼각형을 가리기 때문에 뒷면 삼각형은 보이지 않습니다. 따라서 뒷면 삼각형은 추가 처리에서 실제로 제거해도 아무도 눈치채지 못합니다.

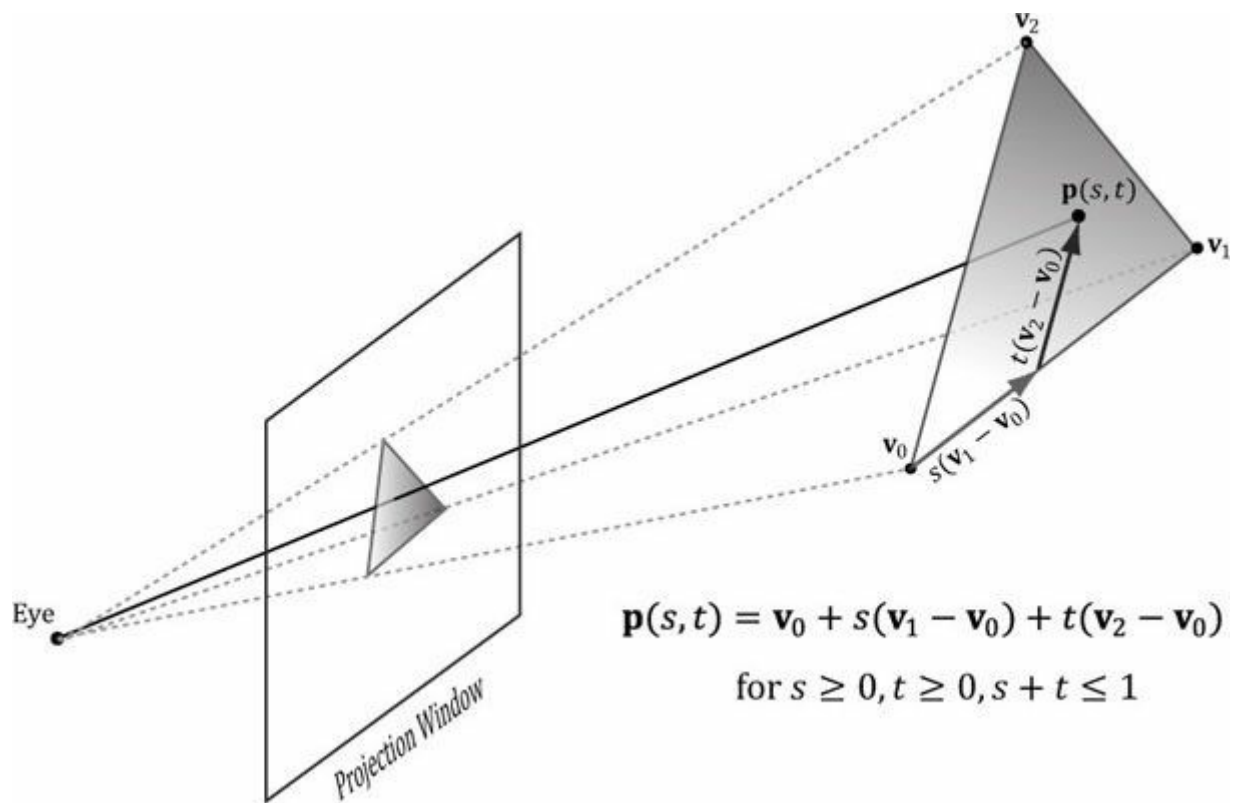
기본적으로 Direct3D는 시계 방향 회전 순서(관찰자 기준)의 삼각형을 정면 삼각형으로, 반시계 방향 회전 순서(관찰자 기준)의 삼각형을 후면 삼각형으로 처리합니다. 그러나 이 규칙은 Direct3D 렌더 상태 설정으로 반전시킬 수 있습니다.

5.10.3 정점 속성 보간

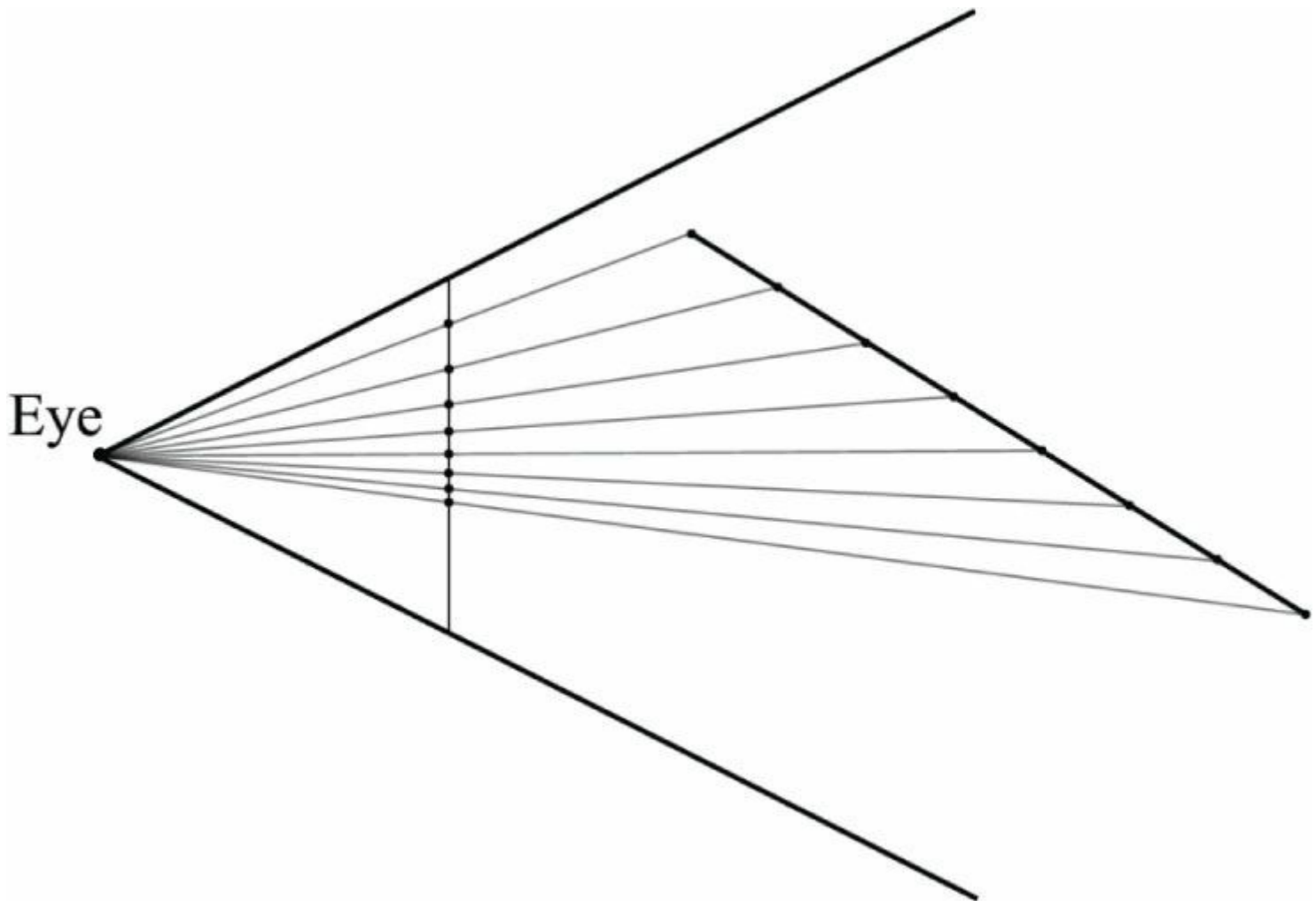
삼각형은 정점을 지정하여 정의합니다. 위치 외에도 정점에 색상, 법선 벡터, 텍스처 좌표 등의 속성을 부여할 수 있습니다. 뷰포트 변환 후에는 삼각형을 덮는 각 픽셀에 대해 이러한 속성을 보간해야 합니다. 정점 속성 외에도 깊이 버퍼링 알고리즘을 위해 각 픽셀이 깊이 값을 가지도록 정점 깊이 값도 보간해야 합니다. 정점 속성은 화면 공간에서 보간되며, 이는 3차원 공간의 삼각형에 걸쳐 속성이 선형적으로 보간되는 방식입니다(그림 5.33). 이를 위해서는 소위 *투시 보정 보간*이 필요합니다. 본질적으로 보간은 정점 값을 사용하여 내부 픽셀의 값을 계산할 수 있게 합니다.

원근 보정 속성 보간의 수학적 세부 사항은 하드웨어가 이를 처리하므로 우리가 신경 쓸 필요가 없습니다

이 작업을 수행하므로, 관심 있는 독자는 [Eberly01]에서 수학적 도출 과정을 확인할 수 있습니다. 그러나 [그림 5.34](#)는 기본적인 원리를 보여줍니다.



그림/5.33. 삼각형 상의 속성값 $p(s, t)$ 는 삼각형의 정점들에서 속성값들 사이를 선형 보간하여 얻을 수 있다.



그림/5.34. 3차원 선분이 투영 창에 투영되고 있다(투영 결과는 화면 공간에서의 2차원 선분이다). 3차원 선분을 따라 균일한 단계 크기를 취하는 것이 2차원 화면 공간에서 비균일한 단계 크기를 취하는 것과 일치함을 알 수 있다. 따라서 3차원 공간에서 선형 보간을 수행하려면 화면 공간에서 비선형 보간을 수행해야 한다.

5.11 픽셀 셰이더 단계

픽셀 셰이더는 GPU에서 실행되는 우리가 작성하는 프로그램입니다. 픽셀 셰이더는 각 픽셀 프래그먼트마다 실행되며, 보관된 버텍스 속성을 입력으로 사용하여 색상을 계산합니다. 픽셀 셰이더는 상수 색상을 반환하는 간단한 작업부터 픽셀별 조명, 반사, 그림자 효과와 같은 복잡한 작업까지 수행할 수 있습니다.

5.12 출력 병합 단계

픽셀 셰이더에 의해 생성된 픽셀 조각들은 렌더링 파이프라인의 출력 병합(OM) 단계로 이동합니다. 이 단계에서 일부 픽셀 조각은 거부될 수 있습니다(예: 깊이 버퍼 또는 스텐실 버퍼 테스트에서). 거부되지 않은 픽셀 조각들은 백 버퍼에 기록됩니다. 블렌딩도 이 단계에서 수행되며, 이때 픽셀은 백 버퍼에 현재 존재하는 픽셀을 완전히 덮어쓰기보다는 블렌딩될 수 있습니다. 투명도와 같은 일부 특수 효과는 블렌딩으로 구현됩니다. 블렌딩에 대해서는 [제9장에서](#) 자세히 다룹니다.

5.13 요약

1. 우리는 실제 생활에서 사물을 보는 방식에 기반한 여러 기법을 활용하여 2D 이미지에 3D 장면을 시뮬레이션할 수 있습니다. 우리는 평행선이 소실점으로 수렴하는 것을 관찰하고, 깊이에 따라 사물의 크기가 줄어들며, 사물이 뒤에 있는 사물을 가리고, 조명과 음영이 3D 사물의 고체 형태와 부피를 묘사하며, 그림자가 광원의 위치를 암시하고 장면 내 다른 사물과 상대적인 사물의 위치를 나타내는 것을 관찰합니다.
2. 우리는 삼각형 메쉬로 물체를 근사화합니다. 각 삼각형은 세 개의 꼭짓점을 지정하여 정의할 수 있습니다. 많은 메쉬에서 꼭짓점은 여러 삼각형이 공유하며, 인덱싱된 리스트를 사용하면 꼭짓점 중복을 피할 수 있습니다.
3. 색상은 빨강, 녹색, 파랑의 강도를 지정하여 표현됩니다. 이 세 가지 색상을 서로 다른 강도로 가산 혼합함으로써 수백만 가지 색상을 표현할 수 있습니다. 빨강, 녹색, 파랑의 강도를 표현하기 위해 0부터 1까지의 정규화된 범위를 사용하는 것이 유용합니다. 0은 강도가 없음을, 1은 최대 강도를, 중간 값은 중간 강도를 나타냅니다. *알파 성분*이라고 불리는 추가적인 색상 성분을 포함시키는 것이 일반적입니다. 알파 성분은 흔히 색상의 불투명도를 나타내는 데 사용되며, 이는 블렌딩에 유용합니다. 알파 성분을 포함하면 색상을 4차원 색상 벡터 (r, g, b, a) 로 표현할 수 있으며, 여기서 $0 \leq r, g, b, a \leq 1$ 입니다. 색상 표현에 필요한 데이터가 4차원 벡터이므로 코드에서 색상을 표현할 때 **XMVECTOR** 유형을 사용할 수 있으며, XNA Math 벡터 함수를 사용하여 색상 연산을 수행할 때마다 SIMD 연산의 이점을 얻을 수 있습니다. 32비트로 색상을 표현하려면 각 구성 요소에 1바이트가 할당됩니다. XNA Math 라이브러리는 32비트 색상을 저장하기 위한 **XMVECTOR** 구조체를 제공합니다. 색상 벡터는 일반 벡터와 마찬가지로 더하고, 빼고, 스케일링할 수 있지만, 구성 요소를 $[0, 1]$ 구간(32비트 색상의 경우 $[0, 255]$)으로 클램핑해야 합니다. 다른 벡터는...
점곱과 교차곱과 같은 연산은 색벡터에 대해 의미가 없습니다. 기호 $\otimes$ 는 성분별 곱셈을 나타내며 다음과 같이 정의됩니다: $(c_1, c_2, c_3, c_4) \otimes (k_1, k_2, k_3, k_4) = (c_1 k_1, c_2 k_2, c_3 k_3, c_4 k_4)$.
4. 3D 장면의 기하학적 묘사와 그 장면에서 위치 및 조준된 가상 카메라가 주어지면, *렌더링 파이프라인*은 가상 카메라가 보는 것을 기반으로 모니터 화면에 표시할 수 있는 2D 이미지를 생성하는 데 필요한 전체 단계의 순서를 의미합니다.
5. 렌더링 파이프라인은 다음과 같은 주요 단계로 나눌 수 있습니다. 입력 어셈블리(IA) 단계; 버텍스 셰이더(VS) 단계; 테셀레이션 단계; 지오메트리 셰이더(GS) 단계; 클리핑 단계; 래스터화 단계(RS); 픽셀 셰이더(PS) 단계; 그리고 출력 병합(OM) 단계.

5.14 연습문제

1. [그림 5.35](#)에 표시된 피라미드의 버텍스와 인덱스 리스트를 구성하세요.
2. [그림 5.36](#)에 표시된 두 도형을 고려하십시오. 객체를 하나의 버텍스 및 인덱스 리스트로 병합하십시오. (여기서 핵심은 두 번째 인덱스 리스트를 첫 번째 리스트에 추가할 때, 추가된 인덱스가 병합된 버텍스 리스트가 아닌 원래 버텍스 리스트의 버텍스를 참조하므로 해당 인덱스를 업데이트해야 한다는 점입니다.)
3. 세계 좌표계에 대해 카메라가 $(-20, 35, -50)$ 위치에 있고 $(10, 0, 30)$ 지점을 바라보고 있다고 가정합니다. 세계 좌표계에서 $(0, 1, 0)$ 방향이 "위" 방향을 나타낸다고 가정하고 뷰 매트릭스를 계산하세요.
4. 시야 원뿔의 수직 시야각이 $\theta = 45^\circ$, 종횡비가 $a = 4/3$, 근면이 $n = 1$, 원면이 $f = 100$ 인 경우, 대응하는 투시 투영 행렬을 구하십시오.
5. 시야 창 높이가 4라고 가정한다. 수직 시야각 $\theta = 60^\circ$ 를 생성하기 위해 시야 창이 원점으로부터 떨어져 있어야 하는 거리 d 를 구하라.

시야각 $\theta = 60^\circ$ 를 생성하기 위해 시야 창이 원점에서 떨어져 있어야 하는 거리 d 를 구하라.

6. 다음 투시 투영 행렬을 고려하십시오:

$$\begin{bmatrix} 1.86603 & 0 & 0 & 0 \\ 0 & 3.73205 & 0 & 0 \\ 0 & 0 & 1.02564 & 1 \\ 0 & 0 & -5.12821 & 0 \end{bmatrix}$$

이 행렬을 구성하는 데 사용된 수직 시야각 α , 종횡비 r , 그리고 근거리 및 원거리 평면 값을 구하라.

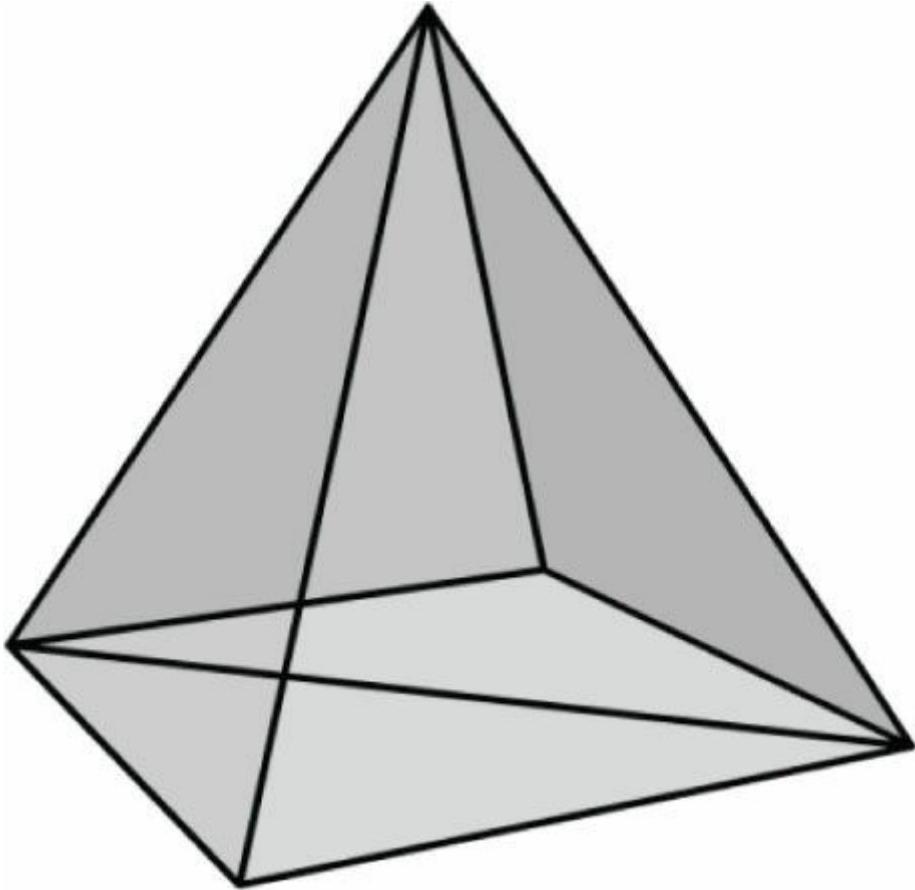


그림 5.35. 피라미드의 삼각형들.

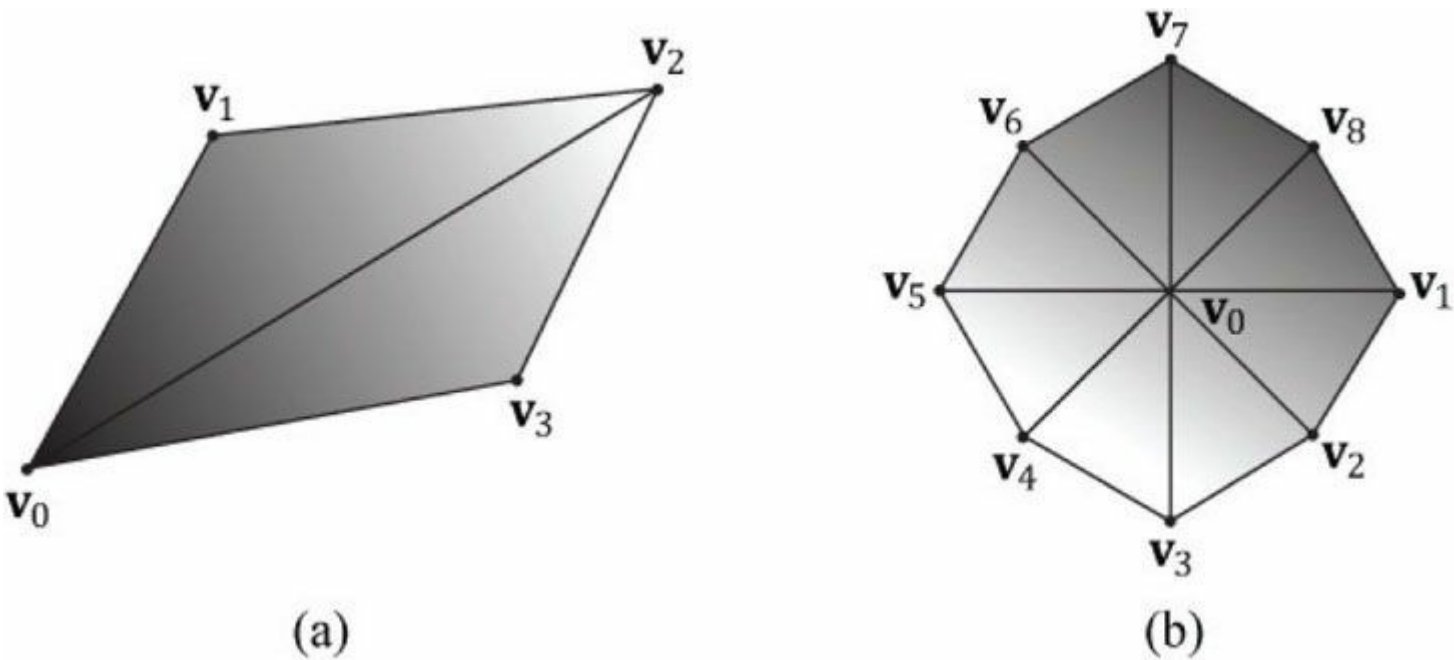


그림 5.36. 연습 2의 도형들.

7. 고정된 A, B, C, D 를 가진 다음과 같은 투시 투영 행렬이 주어졌다고 가정하십시오:

$$\begin{bmatrix} A & 0 & 0 & 0 \\ 0 & B & 0 & 0 \\ 0 & 0 & C & 1 \\ 0 & 0 & D & 0 \end{bmatrix}$$

이 행렬을 구성하는 데 사용된 수직 시야각 α , 종횡비 r , 근거리 평면 및 원거리 평면 값을 A, B, C, D 를 사용하여 구하십시오. 즉, 다음 방정식을 풀어야 합니다:

$$A = \frac{1}{r \tan(\alpha/2)}$$

$$B = \frac{1}{\tan(\alpha/2)}$$

$$C = \frac{f}{f-n}$$

$$D = \frac{-nf}{f-n}$$

이 방정식들을 풀면, 이 책에서 설명한 유형의 모든 투영 행렬로부터 수직 시야각 α , 종횡비 r , 그리고 근거리 및 원거리 평면 값을 추출하는 공식들을 얻을 수 있습니다.

8. 투영 텍스처링 알고리즘에서, 우리는 투영 행렬 이후에 아핀 변환 행렬 $\mathbf{T}$ 를 곱합니다. **투영 행렬** 곱셈 전후에 원근 분할을 수행하는 순서가 중요하지 않음을 증명하십시오. $\mathbf{v}$ 를 4차원 벡터, $\mathbf{P}$ 를 투영 행렬, $\mathbf{T}$ 를 4×4 아핀 변환 행렬이라 하고, w 첨자는 4차원 벡터의 w 좌표를 나타낸다고 하자. 다음을 증명하십시오:

$$\left(\frac{\mathbf{vP}}{(\mathbf{vP})_w}\right)\mathbf{T} = \frac{(\mathbf{vPT})}{(\mathbf{vPT})_w}$$

9. 투영 행렬의 역행렬이 다음 식으로 주어짐을 증명하라:

$$\mathbf{P}^{-1} = \begin{bmatrix} r \tan\left(\frac{\alpha}{2}\right) & 0 & 0 & 0 \\ 0 & \tan\left(\frac{\alpha}{2}\right) & 0 & 0 \\ 0 & 0 & 0 & -\frac{f-n}{nf} \\ 0 & 0 & 1 & \frac{1}{n} \end{bmatrix}$$

10. 시점 공간 내 점의 좌표를 $[x, y, z, 1]$ 이라 하고, 동일한 점의 NDC 공간 내 좌표를 $[x_{ndc}, y_{ndc}, z_{ndc}, 1]$ 이라 하자. 다음 방식으로 NDC 공간에서 시점 공간으로 변환할 수 있음을 증명하라:

$$[x_{ndc}, y_{ndc}, z_{ndc}, 1]\mathbf{P}^{-1} = \left[\frac{x}{z}, \frac{y}{z}, 1, \frac{1}{z}\right] \xrightarrow{\text{divide by } w} [x, y, z, 1]$$

w로 나누는 과정이 필요한 이유를 설명하라. 동차 클립 공간에서 뷰 공간으로 변환할 때도 w로 나누는 과정이 필요할까?

11. 시야 원뿔을 설명하는 또 다른 방법은 근평면에서의 시야 볼륨 너비와 높이를 지정하는 것이다. 근평면에서의 시야 볼륨 너비 w와 높이 h, 그리고 근평면 n과 원평면 f가 주어졌을 때, 투영 행렬이 다음 식으로 주어짐을 보여라:

$$P = \begin{bmatrix} \frac{2n}{w} & 0 & 0 & 0 \\ 0 & \frac{2n}{h} & 0 & 0 \\ 0 & 0 & \frac{f}{f-n} & 1 \\ 0 & 0 & \frac{-nf}{f-n} & 0 \end{bmatrix}$$

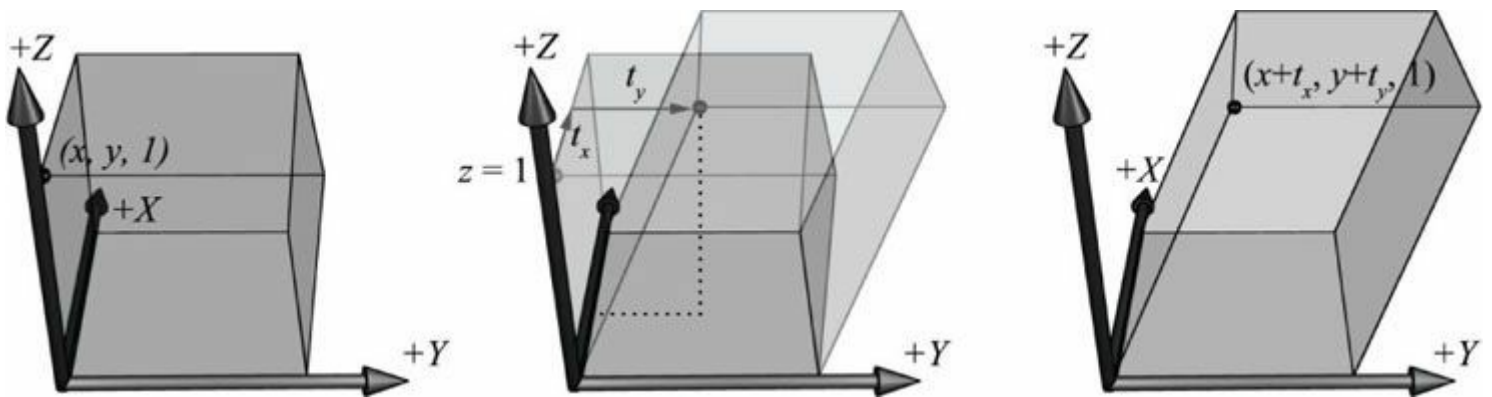


그림 5.37. z좌표에 의해 전단된 x 및 y 좌표. 상자의 상단면은 $z = 1$ 평면에 위치합니다. 이 평면 내의 점들은 전단 변환에 의해 평행 이동됨을 관찰하십시오.

12. 수직 시야각 θ , 중횡비 a , 근면 n , 원면 f 를 가진 뷰 프러스텀이 주어졌을 때, 프러스텀의 8개 꼭짓점 모서리를 구하라.
13. 3차원 전단 변환 $S_{xy}(x, y, z) = (x + zt_x, y + zt_y, z)$ 를 고려하자. 이 변환은 그림 5.37에 설명되어 있다. 이것이 선형 변환이며 다음 행렬 표현을 갖는다는 것을 증명하라:

$$S_{xy} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

14. $z = 1$ 평면 상의 3차원 점, 즉 $(x, y, 1)$ 형태의 점을 고려하자. 이전 연습문제에서 주어진 전단 변환 S_{xy} 로 점 $(x, y, 1)$ 을 변환하는 것은 $z = 1$ 평면에서의 2차원 평행 이동과 동일함을 관찰할 수 있다:

$$\begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} = \begin{bmatrix} x + t_x & y + t_y & 1 \end{bmatrix}$$

2차원 응용을 다루는 경우, 3차원 좌표를 사용할 수 있지만, 우리의 2차원 공간이 항상 $z = 1$ 평면에 위치한다면, $S_{(xy)}$ 를 사용하여 2차원 공간에서 평행 이동을 수행할 수 있습니다.

다음과 같은 일반화를 도출하십시오:

3차원 공간에서 평면이 2차원 공간인 것처럼, 4차원 공간에서 평면은 3차원 공간이다. 우리가 동차 좌표점 $(x, y, z, 1)$ 을 쓸 때 우리는

$w = 1$ 인 4차원 평면 내에 존재하는 3차원 공간에서 작업하고 있는 것이다.

평행 이동 행렬은 4차원 전단 변환 S_{xyz} 의 행렬 표현이다. $(x, y, z, w) = (x + wt_x, y + wt_y, z + wt_z, w)$. 4차원 전단 변환은 $w = 1$ 평면 내 점들을 평행 이동시키는 효과를 가진다.

제6장 DIRECT3D에서의 DIRECT3D

이전 장에서는 렌더링 파이프라인의 개념적, 수학적 측면에 주로 초점을 맞췄습니다. 이번 장에서는 렌더링 파이프라인을 구성하고, 버텍스 셰이더와 픽셀 셰이더를 정의하며, 렌더링 파이프라인에 지오메트리를 제출하여 그리기 위해 필요한 Direct3D API 인터페이스와 메서드에 집중합니다. 이 장을 마치면 다양한 기하학적 도형을 채색하거나 와이어프레임 모드로 그릴 수 있게 될 것입니다.

목표:

- 1. 기하학적 데이터를 정의하고 저장하며 그리기 위한 Direct3D 인터페이스 메서드를 학습합니다.
- 2. 기본적인 버텍스 셰이더와 픽셀 셰이더 작성법을 학습합니다.
- 3. 렌더링 상태를 사용하여 렌더링 파이프라인을 구성하는 방법을 파악합니다.
- 4. 이펙트 프레임워크를 사용하여 셰이더와 렌더 상태를 논리적으로 그룹화하여 렌더링 기법으로 구성하는 방법과, 이펙트 프레임워크를 "셰이더 생성기"로 활용하는 방법을 학습합니다.

6.1 버텍스와 입력 레이아웃

§5.5.1절에서 살펴본 바와 같이 Direct3D의 정점은 공간적 위치 외에도 추가 데이터를 포함할 수 있습니다. 사용자 정의 정점 형식을 생성하려면 먼저 선택한 정점 데이터를 담을 구조체를 만듭니다. 예를 들어, 다음은 두 가지 다른 정점 형식을 보여줍니다. 하나는 위치와 색상으로 구성되고, 다른 하나는 위치, 법선, 텍스처 좌표로 구성됩니다.

```
struct Vertex1
{
    XMFLOAT3 Pos; XMFLOAT4
    Color;
};

struct Vertex2
{
    XMFLOAT3 Pos;
    XMFLOAT3 노멀; XMFLOAT2
    텍스0; XMFLOAT2 텍스1;
};
```

버텍스 구조체를 정의한 후에는 Direct3D가 각 구성 요소를 어떻게 처리해야 하는지 알 수 있도록 버텍스 구조체에 대한 설명을 제공해야 합니다. 이 설명은 *입력 레이아웃*(ID3D11InputLayout) 형태로 Direct3D에 제공됩니다. 입력 레이아웃은 D3D11_INPUT_ELEMENT_DESC 요소 배열로 지정됩니다. D3D11_INPUT_ELEMENT_DESC 배열의 각 요소는 정점 구조체의 하나의 구성 요소를 설명하고 대응합니다. 따라서 정점 구조체가 두 개의 구성 요소를 가지면, 대응하는 D3D11_INPUT_ELEMENT_DESC 배열은 두 개의 요소를 가집니다. D3D11_INPUT_ELEMENT_DESC 배열을 *입력 레이아웃 설명*이라고 합니다. D3D11_INPUT_ELEMENT_DESC 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_INPUT_ELEMENT_DESC { LPCSTR SemanticName;
    UINT SemanticIndex; DXGI_FORMAT
    Format; UINT InputSlot;
    UINT 정렬바이트오프셋; D3D11_INPUT_CLASSIFICATION 입력슬롯분류; UINT 인
    스탠스스테이터단계속도;
```

```
} D3D11_INPUT_ELEMENT_DESC;
```

1. **SemanticName**: 요소에 연결할 문자열입니다. 유효한 변수 이름이면 무엇이든 가능합니다. 시맨틱은 버텍스 구조체의 요소를 버텍스 셰이더 입력 시그니처의 요소와 매핑하는 데 사용됩니다. [그림 6.1](#)을 참조하십시오.
2. **SemanticIndex**: 시맨틱에 연결할 인덱스입니다. 이것의 동기는 [그림 6.1에서도](#) 설명됩니다. 버텍스 구조체에는 하나 이상의 텍스처 좌표 세트가 있을 수 있으므로, 새로운 시맨틱 이름을 도입하기보다는 끝에 인덱스를 부착하여 텍스처 좌표를 구분할 수 있습니다. 셰이더 코드에 인덱스가 지정되지 않은 시맨틱은 기본적으로 인덱스 0으로 설정됩니다. 예를 들어, [그림 6.1에서](#) POSITION은 POSITION0과 동일합니다.

```
struct Vertex
{
    XMFLOAT3 Pos;
    XMFLOAT3 Normal;
    XMFLOAT2 Tex0;
    XMFLOAT2 Tex1;
};

D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 1, DXGI_FORMAT_R32G32_FLOAT, 0, 32,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};

VertexOut VS(float3 iPos : POSITION,
             float3 iNormal : NORMAL,
             float2 iTex0 : TEXCOORD0,
             float2 iTex1 : TEXCOORD1)
```

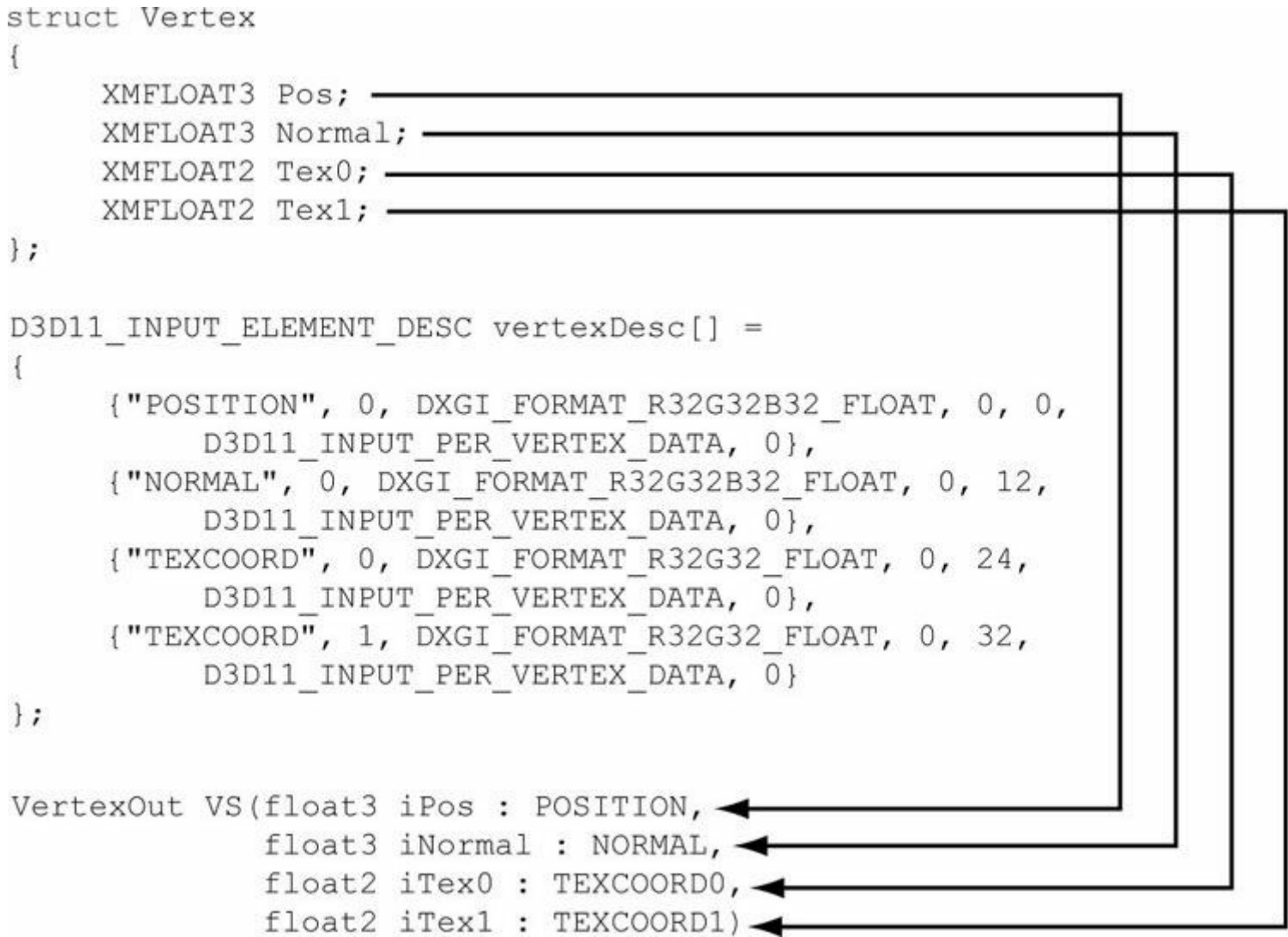


그림 6.1. 버텍스 구조체의 각 요소는 D3D11_INPUT_ELEMENT_DESC 배열의 대응 요소로 설명됩니다. 시맨틱 이름과 인덱스는 버텍스 요소를 버텍스 셰이더의 대응 매개변수에 매핑하는 방법을 제공합니다.

3. **Format**: Direct3D에 이 버텍스 요소의 형식(즉, 데이터 유형)을 지정하는 DXGI_FORMAT 열거형의 멤버입니다. 다음은 일반적으로 사용되는 형식의 몇 가지 예시입니다:

```
DXGI_FORMAT_R32_FLOAT           // 1차원 32비트 부동소수점 스칼라
DXGI_FORMAT_R32G32_FLOAT        // 2D 32비트 부동 소수점 벡터
DXGI_FORMAT_R32G32B32_FLOAT     // 3D 32비트 부동 소수점 벡터
DXGI_FORMAT_R32G32B32A32_FLOAT // 4D 32비트 부동 소수점 벡터

DXGI_FORMAT_R8_UINT             // 1D 8비트 부호 없는 정수 스칼라
DXGI_FORMAT_R16G16_SINT         // 2D 16비트 부호 있는 정수 벡터
DXGI_FORMAT_R32G32B32_UINT     // 3D 32비트 부호 없는 정수 벡터
DXGI_FORMAT_R8G8B8A8_SINT      // 4D 8비트 부호 있는 정수 벡터
DXGI_FORMAT_R8G8B8A8_UINT      // 4D 8비트 부호 없는 정수 벡터
```

- 4. InputSlot:** 이 요소가 유래할 입력 슬롯 인덱스를 지정합니다. Direct3D는 16개의 입력 슬롯(0~15로 인덱싱)을 지원하며, 이를 통해 정점 데이터를 공급할 수 있습니다. 예를 들어, 정점이 위치와 색상 요소로 구성된다면, 두 요소를 단일 입력 슬롯을 통해 공급하거나, 요소를 분할하여 위치 요소를 첫 번째 입력 슬롯을 통해, 색상 요소를 두 번째 입력 슬롯을 통해 공급할 수 있습니다. Direct3D는 서로 다른 입력 슬롯의 요소를 사용하여 정점을 조립합니다. 이 책에서는 하나의 입력 슬롯만 사용하지만, 연습 2에서는 두 개를 사용해 보도록 요청합니다.
- 5. 정렬된 바이트 오프셋:** 단일 입력 슬롯의 경우, 이는 C++ 정점 구조체 시작 지점에서 정점 구성 요소 시작 지점까지의 오프셋(바이트 단위)입니다. 예를 들어, 다음 버텍스 구조에서 요소 **Pos**는 시작 위치가 버텍스 구조의 시작과 일치하므로 0바이트 오프셋을 가집니다; 요소 **Normal**은 **Pos**의 바이트를 건너뛰어 **Normal**의 시작에 도달해야 하므로 12바이트 오프셋을 가집니다; **Tex0** 요소는 **Tex0** 시작 부분에 도달하기 위해 **Pos**와 **Normal**의 바이트를 건너뛰어야 하므로 24바이트 오프셋을 가집니다; **Tex1** 요소는 **Tex1** 시작 부분에 도달하기 위해 **Pos**, **Normal**, **Tex0**의 바이트를 건너뛰어야 하므로 32바이트 오프셋을 가집니다.

```
struct Vertex2
{
    XMFLOAT3 Pos;                // 0바이트 오프셋

    XMFLOAT3 Normal;            // 12바이트 오프셋

    XMFLOAT2 Tex0;              // 24바이트 오프셋

    XMFLOAT2 Tex1;              // 32바이트 오프셋
};
```

- 6. InputSlotClass:** 현재는 D3D11_INPUT_PER_VERTEX_DATA를 지정하십시오; 다른 옵션은 인스턴싱 고급 기법에 사용됩니다.

- 7. InstanceDataStepRate:** 현재는 0으로 지정; 다른 값은 인스턴싱 고급 기법에서만 사용됩니다.

앞서 제시된 두 정점 구조체(Vertex1 및 Vertex2)에 대한 입력 레이아웃 설명은 다음과 같습니다:

```
D3D11_INPUT_ELEMENT_DESC desc1[] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, 12,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};

D3D11_INPUT_ELEMENT_DESC desc2[] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 1, DXGI_FORMAT_R32G32_FLOAT, 0, 32,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};
```

입력 레이아웃 설명이 지정되면 ID3D11Device::CreateInputLayout 메서드를 사용하여 입력 레이아웃을 나타내는 ID3D11InputLayout 인터페이스에 대한 포인터를 얻을 수 있습니다:

```
HRESULT ID3D11Device::CreateInputLayout(
    const D3D11_INPUT_ELEMENT_DESC *pInputElementDescs, UINT NumElements,
    const void *pShaderBytecodeWithInputSignature, SIZE_T BytecodeLength,
    ID3D11InputLayout **ppInputLayout);
```

- 1. pInputElementDescs:** 정점 구조를 설명하는 D3D11_INPUT_ELEMENT_DESC 요소 배열.
- 2. NumElements:** D3D11_INPUT_ELEMENT_DESC 요소 배열의 요소 수. **3. pShaderBytecodeWithInputSignature:** 버텍스 셰이더의 입력 시그니처에 대한 셰이더 바이트 코드에 대한 포인터. **4. BytecodeLength:** 이전 매개 변수로 전달된 버텍스 셰이더 시그니처 데이터의 바이트 크기.
- 5. ppInputLayout:** 생성된 입력 레이아웃에 대한 포인터를 반환합니다.

세 번째 매개변수에 대해 좀 더 설명이 필요합니다. 버텍스 셰이더는 입력 매개변수로 버텍스 요소 목록을 받습니다—소위 *입력 시그니처*입니다. 사용자 정의 버텍스 구조체의 요소들은 버텍스 셰이더의 해당 입력에 매핑되어야 합니다. 그림 6.1이 이를 암시합니다. 입력 레이아웃 생성 시 버텍스 셰이더 입력 시그니처에 대한 설명을 전달함으로써, Direct3D는 입력 레이아웃이 입력 시그니처와 일치하는지 검증하고 생성 시점에 버텍스 구조체에서 셰이더 입력으로의 매핑을 생성할 수 있습니다.

입력 시그니처가 정확히 동일하다면, 입력 레이아웃은 서로 다른 셰이더 간에 재사용될 수 있습니다.

다음과 같은 입력 시그니처와 정점 구조를 가진 경우를 고려하십시오:

```
VertexOut VS(float3 Pos : POSITION, float4 Color : COLOR, float3 Normal : NORMAL) { }
```

```
struct Vertex
{
    XMFLOAT3 Pos; XMFLOAT4
    Color;
};
```

이 코드는 오류를 발생시키며, VC++ 디버그 출력 창에는 다음과 같이 표시됩니다:

D3D11: 오류: ID3D11Device::CreateInputLayout: 제공된 입력 시그니처는 SemanticName/Index: 'NORMAL'/0 요소를 읽을 것으로 예상하지만, 선언에 일치하는 이름이 제공되지 않았습니
다.

이제 정점 구조체와 입력 시그니처의 정점 요소가 일치하지만 유형이 다른 경우를 고려해 보겠습니다:

```
VertexOut VS(int3 Pos : POSITION, float4 Color : COLOR) { } struct Vertex
{
    XMFLOAT3 Pos; XMFLOAT4
    Color;
};
```

Direct3D는 입력 레지스터의 비트를 재해석할 수 있도록 허용하기 때문에 이는 실제로 유효합니다. 그러나 VC++ 디버그 출력 창에는 다음과 같은 경고가 표시됩니다:

D3D11: 경고: ID3D11Device::CreateInputLayout: 제공된 입력 시그니처는 SemanticName/Index: 'POSITION'/0 및 'int32' 유형의 구성 요소를 가진 요소를 읽을 것으로 예상합니다. 그
러나 입력 레이아웃 선언의 일치하는 항목인 element[0]은 불일치하는 형식('R32G32B32_FLOAT')을 지정합니다. 이는 오류가 아닙니다. 동작이 명확히 정의되어 있기 때문입니다: 요소 형
식은 셰이더 레지스터에 표시되기 전에 적용되는 데이터 변환 알고리즘을 결정합니다. 독립적으로, 셰이더 입력 시그니처는 입력 레지스터에 배치된 데이터를 셰이더가 어떻게 해석할지 정
의하며, 저장된 비트 자체는 변경되지 않습니다. 애플리케이션이 버텍스 셰이더 내에서 데이터를 다른 유형으로 재해석하는 것은 유효하므로, 이 경고는 작성자가 의도하지 않은 재해석이 발
생할 경우를 대비하여 발행됩니다.

다음 코드는 ID3D11Device::CreateInputLayout 메서드가 호출되는 방식을 설명하는 예시를 제공합니다. 이 코드에는 아직 다루지 않은 주제(예: ID3D11Effect)가 포함되어 있음을 유의하십시오. 기본적으로 효과(effect)는 하나 이상의 패스(pass)를 캡슐화하며, 각 패스에는 정점 셰이더가 연관됩니다. 따라서 효과에서 패스 설명(D3D11_PASS_DESC)을 얻을 수 있으며, 이를 통해 버텍스 셰이더의 입력 시그니처를 확인할 수 있습니다.

```
ID3DX11Effect* mFX; ID3DX11EffectTechnique* mTech;
ID3D11InputLayout* mInputLayout;

/* ...이펙트 생성... */
mTech = mFX->GetTechniqueByName("Tech"); D3DX11_PASS_DESC passDesc;
mTech->GetPassByIndex(0)->GetDesc(&passDesc); HR(md3dDevice-
>CreateInputLayout(vertexDesc, 4, passDesc.
    pIAInputSignature, passDesc.IAInputSignature Size, &mInputLayout));
```

입력 레이아웃이 생성된 후에도 아직 장치에 바인딩되지 않았습니다. 마지막 단계는 사용하려는 입력 레이아웃을 장치에 바인딩하는 것으로, 다음 코드에서 볼 수 있습니다:

```
ID3D11InputLayout* mInputLayout;
```

/* ...입력 레이아웃 생성... */

md3dImmediateContext->IASetInputLayout(mInputLayout);

그리는 일부 오브젝트가 하나의 입력 레이아웃을 사용하고, 다른 오브젝트가 다른 레이아웃을 필요로 하는 경우, 다음과 같이 코드를 구성해야 합니다:

다음과 같이 코드를 구성해야 합니다:

md3dImmediateContext->IASetInputLayout(mInputLayout1);

/* ...입력 레이아웃 1을 사용하여 객체 그리기... */

md3dImmediateContext->IASetInputLayout(mInputLayout2);

/* ...입력 레이아웃 2를 사용하여 객체 그리기... */

다시 말해, 입력 레이아웃이 장치에 바인딩되면 덮어쓰기 전까지는 변경되지 않습니다.

6.2 버텍스 버퍼

GPU가 버텍스 배열에 접근하려면, ID3D11Buffer 인터페이스로 표현되는 *버퍼*라는 특수한 리소스 구조에 배치되어야 합니다.

정점을 저장하는 버퍼를 *정점 버퍼*라고 합니다. Direct3D 버퍼는 데이터를 저장할 뿐만 아니라, 데이터가 어떻게 접근될지

버텍스 버퍼를 생성하려면 다음 단계를 수행해야 합니다:

- 1. 생성할 버퍼를 설명하는 D3D11_BUFFER_DESC 구조체를 작성합니다.
- 2. 버퍼 내용을 초기화할 데이터를 지정하는 D3D11_SUBRESOURCE_DATA 구조체를 작성합니다.
- 3. 버퍼를 생성하려면 ID3D11Device::CreateBuffer를 호출하십시오.

D3D11_BUFFER_DESC 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_BUFFER_DESC {
    UINT ByteWidth;
    D3D11_USAGE Usage;
    UINT BindFlags;
    UINT CPUAccessFlags;
    UINT MiscFlags;
    UINT StructureByteStride;
} D3D11_BUFFER_DESC;
```

- 1. ByteWidth: 생성할 버텍스 버퍼의 크기(바이트 단위).
- 2. 사용법: 버퍼가 어떻게 사용될지를 지정하는 D3D11_USAGE 열거형의 멤버입니다. 네 가지 사용법 값은 다음과 같습니다:

(a)

D3D11_USAGE_DEFAULT: GPU가 리소스에 읽고 쓸 경우 이 사용법을 지정합니다. CPU는 이 사용법을 가진 리소스에 읽고 쓸 수 없습니다. 매핑 API(예: ID3D11DeviceContext::Map)를 사용합니다. 그러나 ID3D11DeviceContext::UpdateSubresource는 사용할 수 없습니다. ID3D11DeviceContext::Map 메서드는 §6.14에서 설명합니다.

(b)

D3D11_USAGE_IMMUTABLE: 리소스 생성 후 내용이 절대 변경되지 않는 경우 이 사용법을 지정합니다. GPU가 리소스를 읽기 전용으로 처리하므로 일부 잠재적 최적화가 가능합니다. CPU는 리소스 생성 시 초기화 목적으로만 불변 리소스에 쓰기를 수행할 수 있습니다. CPU는 불변 리소스에서 읽기를 수행할 수 없습니다. 불변 리소스는 매핑하거나 업데이트할 수 없습니다. D3D11_USAGE_DYNAMIC: 애플리케이션(CPU)이 리소스의 데이터 내용을 자주(예: 프레임 단위로) 업데이트해야 하는 경우 이 사용법을 지정합니다. 이 사용법을 가진 리소스는 GPU가 읽을 수 있으며, 매핑 API(즉,

(c)	ID3D11DeviceContext::Map)을 사용하여 GPU가 리소스를 읽고 CPU가 리소스에 쓸 수 있습니다. GPU 업데이트
(d)	CPU에서 동적으로 리소스를 할당하면 성능 저하가 발생합니다. 새로운 데이터가 CPU 메모리(즉, 시스템 RAM)에서 GPU 메모리(즉, 비디오 RAM)로 전송되어야 하기 때문입니다. 따라서 동적 사용은 필요한 경우가 아니면 피해야 합니다. D3D11_USAGE_STAGING : 애플리케이션(CPU)이 리소스의 복사본을 읽을 수 있어야 하는 경우(즉, 리소스가 비디오 메모리에서 시스템 메모리로의 데이터 복사를 지원하는 경우) 이 사용법을 지정하십시오. 복사 GPU에서 CPU 메모리로의 복사는 느린 작업이므로 필요하지 않은 경우 피해야 합니다. 리소스는 ID3D11DeviceContext::CopyResource 및 ID3D11DeviceContext::CopySubresourceRegion 메서드를 사용하여 복사할 수 있습니다. §12.3.5에는 CopyResource 의 예가 나와 있습니다.

3. **BindFlags**: 버텍스 버퍼의 경우 **D3D11_BIND_VERTEX_BUFFER**를 지정하십시오.

- CPUAccessFlags**: CPU가 버퍼에 접근하는 방식을 지정합니다. CPU가 이후에 읽기 또는 쓰기 접근이 필요하지 않은 경우 0을 지정합니다.
4. 버퍼가 생성되었습니다. CPU가 버퍼에 쓰기를 통해 업데이트해야 하는 경우 **D3D11_CPU_ACCESS_WRITE**를 지정하십시오. 쓰기 권한이 있는 버퍼는 **D3D11_USAGE_DYNAMIC** 또는 **D3D11_USAGE_STAGING** 사용 유형을 가져야 합니다.

CPU가 버퍼에서 읽어야 하는 경우 **D3D11_CPU_ACCESS_READ**를 지정하십시오. 읽기 권한이 있는 버퍼는 반드시 **D3D11_USAGE_STAGING** 사용 유형을 가져야 합니다. 필요한 경우에만 이러한 플래그를 지정하십시오. 일반적으로 CPU가 Direct3D 리소스에서 데이터를 읽는 작업은 느립니다(GPU는 파이프라인을 통해 데이터를 처리하는 데 최적화되어 있지만, 데이터를 다시 읽는 데는 그렇지 않음). 이로 인해 GPU가 스톱 상태에 빠질 수 있습니다(GPU는 작업을 계속하기 전에 읽기 대상 리소스의 작업이 완료되기를 기다려야 할 수 있음). CPU가 리소스에 쓰는 작업은 더 빠르지만, 업데이트된 데이터를 비디오 메모리로 다시 전송해야 하는 오버헤드는 여전히 존재합니다. 가능하면 이러한 플래그를 전혀 지정하지 않고, GPU만이 읽고 쓰는 비디오 메모리에 리소스를 그대로 두는 것이 가장 좋습니다.

5. **MiscFlags**: 버텍스 버퍼에는 추가 플래그가 필요하지 않습니다. 0을 지정하고 자세한 내용은 SDK 문서의 자세한 내용은 SDK 문서의 **D3D11_RESOURCE_MISC_FLAG** 열거형 유형을 참조하십시오.
6. **StructureByteStride**: 구조체 버퍼에 저장된 단일 요소의 크기(바이트 단위). 이 속성은 구조체 버퍼에만 적용되며, 다른 모든 버퍼에서는 0으로 설정할 수 있습니다. 구조체 버퍼는 동일한 크기의 요소를 저장하는 버퍼입니다.

D3D11_SUBRESOURCE_DATA 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_SUBRESOURCE_DATA { const void *pSysMem;
    UINT SysMemPitch; UINT
    SysMemSlicePitch;
} D3D11_SUBRESOURCE_DATA;
```

1. **pSysMem**: 버텍스 버퍼를 초기화할 데이터를 포함하는 시스템 메모리 배열에 대한 포인터입니다. 버퍼가 *n개의* 버텍스를 저장할 수 있다면, 전체 버퍼를 초기화할 수 있도록 시스템 배열은 최소한 *n개의* 버텍스를 포함해야 합니다.
2. **SysMemPitch**: 버텍스 버퍼에는 사용되지 않습니다.
3. **SysMemSlicePitch**: 버텍스 버퍼에는 사용되지 않습니다.

다음 코드는 원점을 중심으로 한 정육면체의 여덟 꼭짓점으로 초기화된 불변 버텍스 버퍼를 생성합니다. 이 버퍼는 불변입니다. 왜냐하면 정육면체 형상은 생성된 후 변경될 필요가 없기 때문입니다—항상 정육면체로 유지됩니다. 또한 각 꼭짓점에 서로 다른 색상을 할당합니다. 이 꼭짓점 색상들은 이 장 후반부에서 살펴보게 될 것처럼 정육면체에 색을 입히는 데 사용될 것입니다.

```
// d3dUtil.h에 정의된 Colors 네임스페이스.
//
// #define XMGLOBALCONST extern CONST__ declspec(selectany)
// 1. extern을 사용하면 변수의 복사본이 하나만 존재하며,
// 별도의 .obj 파일마다 사본이 생성되지 않도록 합니다.
// 2. __declspec(selectany)를 사용하면 컴파일러가 오류 메시지를 표시하지 않습니다.
// .cpp 파일 내의 여러 정의에 대해 (어느 것이든 선택할 수 있음)
// 선택하고 나머지는 상수이므로 모두 동일하므로 버릴 수 있음).
```

```
네임스페이스 Colors
{
    XMGLOBALCONST XMVECTORF32 White = {1.0f, 1.0f, 1.0f, 1.0f}; XMGLOBALCONST XMVECTORF32 Black =
    {0.0f, 0.0f, 0.0f, 1.0f}; XMGLOBALCONST XMVECTORF32 Red = {1.0f, 0.0f, 0.0f, 1.0f}; XMGLOBALCONST
    XMVECTORF32 Green = {0.0f, 1.0f, 0.0f, 1.0f}; XMGLOBALCONST XMVECTORF32 Blue = {0.0f, 0.0f, 1.0f, 1.0f};
    XMGLOBALCONST XMVECTORF32 Yellow = {1.0f, 1.0f, 0.0f, 1.0f}; XMGLOBALCONST XMVECTORF32 Cyan =
    {0.0f, 1.0f, 1.0f, 1.0f}; XMGLOBALCONST XMVECTORF32 Magenta = {1.0f, 0.0f, 1.0f, 1.0f};
}
```

```
// 원시 정점 데이터 정의 Vertex vertices[] =
{
    { XMFLOAT3(-1.0f, -1.0f, -1.0f), (const float*)&Colors::White },
    { XMFLOAT3(-1.0f, +1.0f, -1.0f), (const float*)&Colors::Black },
    { XMFLOAT3(+1.0f, +1.0f, -1.0f), (const float*)&Colors::Red },
    { XMFLOAT3(+1.0f, -1.0f, -1.0f), (const float*)&Colors::Green },
    { XMFLOAT3(-1.0f, -1.0f, +1.0f), (const float*)&Colors::Blue },
    { XMFLOAT3(-1.0f, +1.0f, +1.0f), (const float*)&Colors::Yellow },
    { XMFLOAT3(+1.0f, +1.0f, +1.0f), (const float*)&Colors::Cyan },
    { XMFLOAT3(+1.0f, -1.0f, +1.0f), (const float*)&Colors::Magenta }
};
```

```
D3D11_BUFFER_DESC vbd;
vbd.Usage = D3D11_USAGE_IMMUTABLE;
vbd.ByteWidth = sizeof(Vertex) * 8; vbd.BindFlags =
D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = 0;

vbd.MiscFlags = 0;
vbd.StructureByteStride = 0;
```

```
D3D11_SUBRESOURCE_DATA vinitData; vinitData.pSysMem =
vertices;
```

```
ID3D11Buffer* mVB; HR(md3dDevice->CreateBuffer(
    &vbd, // 생성할 버퍼 설명 &vinitData, // 버퍼 초기화 데이터 &
    mVB)); // 생성된 버퍼 반환
```

여기서 Vertex 유형과 색상은 다음과 같이 정의됩니다.

```
struct Vertex
{
    XMFLOAT3 Pos; XMFLOAT4
    Color;
};
```

버텍스 버퍼가 생성된 후에는 버텍스를 파이프라인에 입력으로 공급하기 위해 장치의 입력 슬롯에 바인딩해야 합니다. 이는 다음 메서드로 수행됩니다.

```
void ID3D11DeviceContext::IASetVertexBuffers( UINT StartSlot,
    UINT NumBuffers,
    ID3D11Buffer *const *ppVertexBuffers, const UINT *pStrides,
    const UINT *pOffsets);
```

- 1. StartSlot: 버텍스 버퍼 바인딩을 시작할 입력 슬롯입니다. 0부터 15까지 인덱싱된 16개의 입력 슬롯이 있습니다.
- 2. NumBuffers: 입력 슬롯에 바인딩하는 버텍스 버퍼의 개수입니다. 시작 슬롯의 인덱스가 k이고 n개의 버퍼를 바인딩하는 경우, 버퍼는 입력 슬롯 $I_k, I_{(k+1)}, ..., I_{(k+n-1)}$ 에 바인딩됩니다.

- 3. ppVertexBuffers:** 버텍스 버퍼 배열의 첫 번째 요소를 가리키는 포인터입니다.
- 4. pStrides:** 스트라이드 배열의 첫 번째 요소에 대한 포인터(각 정점 버퍼마다 하나씩, i번째 스트라이드는 i번째 정점 버퍼에 해당). 스트라이드는 해당 정점 버퍼 내 요소의 크기(바이트 단위)입니다.
- 5. pOffsets:** 오프셋 배열의 첫 번째 요소에 대한 포인터(각 정점 버퍼마다 하나씩, 여기서 i번째 오프셋은 i번째 정점 버퍼에 해당). 이는 정점 버퍼 시작 위치에서 입력 어셈블리가 정점 데이터 읽기를 시작해야 하는 정점 버퍼 내 위치까지의 오프셋(바이트 단위)입니다. 정점 버퍼 앞부분의 일부 정점 데이터를 건너뛰고 싶을 때 사용합니다.

IASetVertexBuffers 메서드는 다양한 입력 슬롯에 버텍스 버퍼 배열을 설정할 수 있도록 지원하기 때문에 다소 복잡해 보일 수 있습니다. 그러나 대부분의 경우 단일 입력 슬롯만 사용하게 됩니다. 장 끝 연습 문제를 통해 두 개의 입력 슬롯을 다루는 경험을 쌓을 수 있습니다.

버텍스 버퍼는 변경할 때까지 입력 슬롯에 바인딩된 상태로 유지됩니다. 따라서 하나 이상의 버텍스 버퍼를 사용하는 경우 다음과 같이 코드를 구성할 수 있습니다. 버텍스 버퍼를 사용하는 경우 다음과 같이 코드를 구성할 수 있습니다:

```
ID3D11Buffer* mVB1; // Vertex1 유형의 정점 저장 ID3D11Buffer* mVB2; // Vertex2 유형의 정점 저장

/* ...버텍스 버퍼 생성... */

UINT stride = sizeof(Vertex1); UINT offset = 0;
md3dImmediateContext->IASetVertexBuffers(0, 1, &mVB1, &stride, &offset);

/* ...버텍스 버퍼 1을 사용하여 오브젝트 그리기... */ stride =

sizeof(Vertex2);
offset = 0;
md3dImmediateContext->IASetVertexBuffers(0, 1, &mVB2, &stride, &offset);
/* ...버텍스 버퍼 2를 사용하여 객체 그리기... */
```

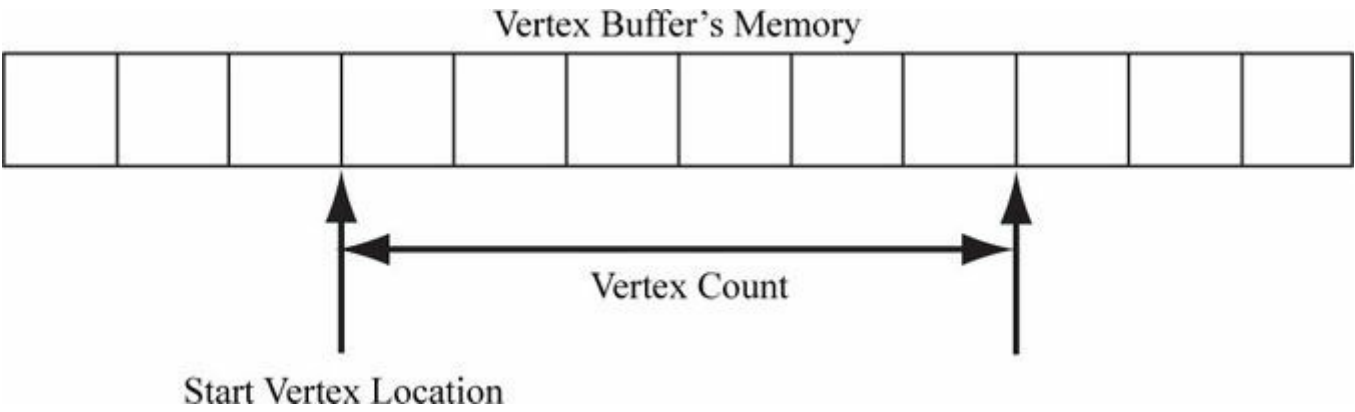


그림 6.2. StartVertexLocation은 드로잉을 시작할 버텍스 버퍼 내 첫 번째 버텍스의 인덱스(0부터 시작)를 지정합니다. VertexCount는 드로잉할 버텍스 수를 지정합니다.

버텍스 버퍼를 입력 슬롯에 설정하는 것만으로는 버텍스가 그려지지 않습니다. 이는 버텍스를 파이프라인에 공급할 준비 상태로 만드는 역할만 합니다. 버텍스를 실제로 그리는 최종 단계는 ID3D11DeviceContext::Draw 메서드로 수행됩니다:

```
void ID3D11DeviceContext::Draw(UINT VertexCount, UINT StartVertexLocation);
```

두 매개변수는 버텍스 버퍼에서 연속적인 부분 집합의 버텍스를 정의하여 렌더링합니다. [그림 6.2](#)를 참조하십시오.

6.3 인덱스와 인덱스 버퍼

인덱스는 GPU가 접근해야 하므로 특별한 리소스 구조인 *인덱스 버퍼*에 배치되어야 합니다. 인덱스 버퍼 생성은 버텍스 버퍼 생성과 매우 유사하지만, 버텍스 대신 인덱스를 저장한다는 점이 다릅니다. 따라서 버텍스 버퍼에 대해 설명한 내용과 유사한 논의를 반복하기보다는 인덱스 버퍼 생성 예시를 보여드리겠습니다:

```
UINT indices[24] = {
    0, 1, 2, // 삼각형 0
    0, 2, 3, // 삼각형 1
```

```

0, 3, 4, // 삼각형 2
0, 4, 5, // 삼각형 3
0, 5, 6, // 삼각형 4
0, 6, 7, // 삼각형 5
0, 7, 8, // 삼각형 6
0, 8, 1 // 삼각형 7
};

```

// 생성할 인덱스 버퍼를 설명합니다. 다음을 확인하세요.

```

// D3D11_BIND_INDEX_BUFFER 바인딩 플래그
D3D11_BUFFER_DESC ibd;
ibd.Usage = D3D11_USAGE_IMMUTABLE;
ibd.ByteWidth = sizeof(UINT) * 24;
ibd.BindFlags = D3D11_BIND_INDEX_BUFFER; ibd.CPUAccessFlags = 0;
ibd.MiscFlags = 0;
ibd.StructureByteStride = 0;

```

```

// 인덱스 버퍼 초기화 데이터 지정. D3D11_SUBRESOURCE_DATA
iinitData; iinitData.pSysMem = indices;

```

```

// 인덱스 버퍼 생성. ID3D11Buffer* mIB;
HR(md3dDevice->CreateBuffer(&ibd, &iinitData, &mIB));

```

버텍스 버퍼나 다른 Direct3D 리소스와 마찬가지로, 인덱스 버퍼를 사용하기 전에 파이프라인에 바인딩해야 합니다. 인덱스 버퍼는 `ID3D11DeviceContext::IASetIndexBuffer` 메서드를 사용하여 입력 어셈블러 단계에 바인딩됩니다. 다음은 호출 예시입니다:

```
md3dImmediateContext->IASetIndexBuffer(mIB, DXGI_FORMAT_R32_UINT, 0);
```

두 번째 매개변수는 인덱스의 형식을 지정합니다. 본 예제에서는 32비트 부호 없는 정수(**DWORD**)를 사용하므로 **DXGI_FORMAT_R32_UINT**를 지정했습니다. 메모리와 대역폭을 절약하고 추가 범위가 필요하지 않다면 16비트 부호 없는 정수를 사용할 수도 있습니다. `IASetIndexBuffer` 메서드에서 형식을 지정하는 것 외에도 **D3D11_BUFFER_DESC::ByteWidth** 데이터 멤버 역시 형식에 따라 달라진다는 점을 기억하십시오. 따라서 문제를 방지하려면 이 둘이 일관되도록 해야 합니다. 인덱스 버퍼에 지원되는 포맷은 **DXGI_FORMAT_R16_UINT**와 **DXGI_FORMAT_R32_UINT**뿐입니다. 세 번째 매개변수는 인덱스 버퍼 시작 위치부터 입력 어셈블리가 데이터 읽기를 시작해야 하는 위치까지의 오프셋(바이트 단위)입니다. 인덱스 버퍼 앞부분의 일부 데이터를 건너뛰고 싶을 때 사용합니다.

마지막으로 인덱스를 사용할 때는 **Draw** 대신 **DrawIndexed** 메서드를 사용해야 합니다:

```

void ID3D11DeviceContext::DrawIndexed(
    UINT IndexCount,
    UINT StartIndexLocation, INT
    BaseVertexLocation);

```

- 1. IndexCount:** 이 드로우 콜에서 사용될 인덱스의 수입니다. 인덱스 버퍼의 모든 인덱스를 사용할 필요는 없습니다. 즉, 연속된 인덱스 부분 집합을 드로우할 수 있습니다.
- 2. 시작 인덱스 위치:** 인덱스 버퍼 내 요소 중 인덱스 읽기를 시작할 시작점을 표시하는 인덱스입니다.
- 3. BaseVertexLocation:** 정점 데이터를 가져오기 전에 이 드로우 콜에서 사용되는 인덱스에 추가될 정수 값입니다.

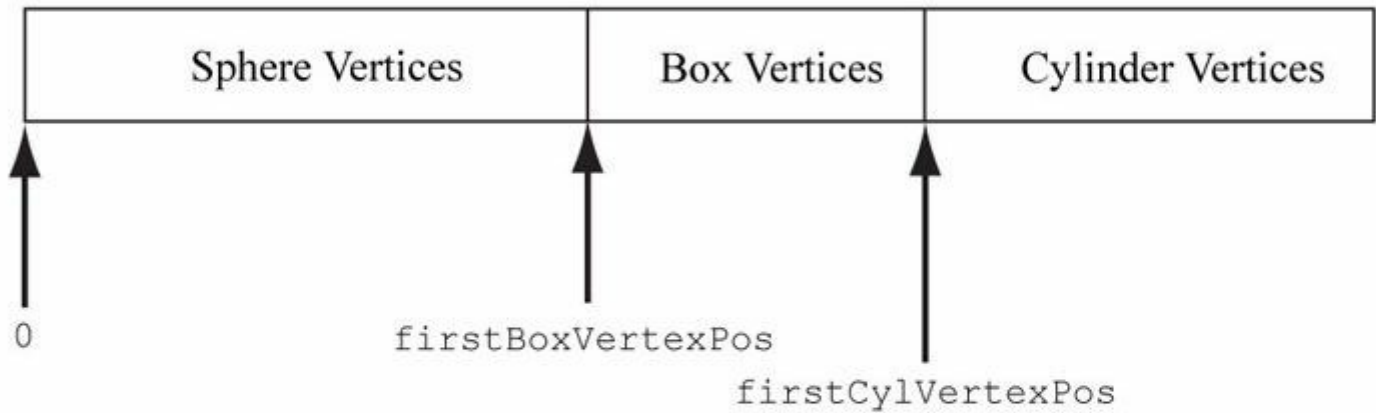
이러한 매개변수를 설명하기 위해 다음과 같은 상황을 고려해 보자. 구, 상자, 원통이라는 세 개의 객체가 있다고 가정하자. 처음에는 각 객체가 자체 정점 버퍼와 자체 인덱스 버퍼를 가진다. 각 로컬 인덱스 버퍼의 인덱스는 해당 로컬 정점 버퍼에 상대적이다. 이제 [그림 6.3과](#) 같이 구, 상자, 원통의 정점과 인덱스를 하나의 글로벌 정점 및 인덱스 버퍼로 연결한다고 가정하자. (버텍스 및 인덱스 버퍼 변경 시 API 오버헤드가 발생하기 때문에 버텍스 및 인덱스 버퍼를 연결할 수 있습니다. 대부분의 경우 이는 병목 현상이 되지 않겠지만, 쉽게 병합할 수 있는 작은 버텍스 및 인덱스 버퍼가 많다면 성능을 위해 그렇게 할 가치가 있을 수 있습니다.) 이 연결 작업 후 인덱스는 더 이상 올바르지 않습니다. 인덱스는 글로벌 버퍼가 아닌 해당 로컬 버텍스 버퍼를 기준으로 위치를 저장하기 때문입니다. 따라서 글로벌 버텍스 버퍼에 올바르게 인덱싱하려면 인덱스를 재계산해야 합니다. 원래 박스 인덱스는 박스의 버텍스가 인덱스 범위를 통과한다는 가정 하에 계산되었습니다.

Sphere = {VB, IB}

Box = {VB, IB}

Cylinder = {VB, IB}

Global Vertex Buffer



Global Index Buffer

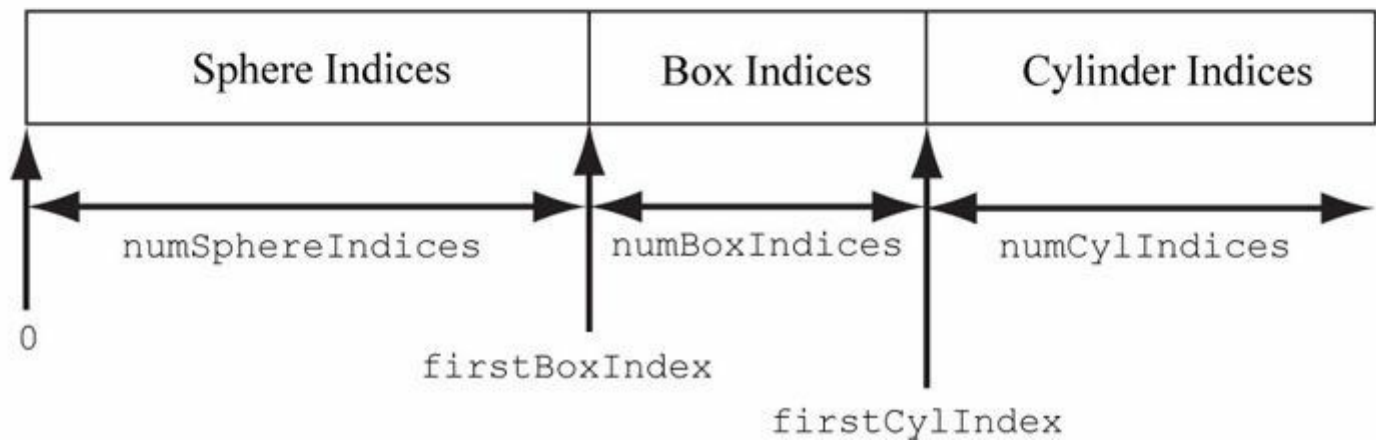


그림 6.3. 여러 정점 버퍼를 하나의 큰 정점 버퍼로 연결하고, 여러 인덱스 버퍼를 하나의 큰 인덱스 버퍼로 연결하는 과정.

0, 1, ..., numBoxVertices-1

그러나 병합 후에는

firstBoxVertexPos, firstBoxVertexPos+1,

...,

firstBoxVertexPos+numBoxVertices-1

따라서 인덱스를 업데이트하려면 모든 박스 인덱스에 `firstBoxVertexPos`를 더해야 합니다. 마찬가지로 모든 실린더 인덱스에 `firstCylVertexPos`를 더해야 합니다. 구체의 인덱스는 변경할 필요가 없습니다(첫 번째 구체 정점 위치가 0이기 때문). 객체의 첫 번째 정점 위치를 글로벌 정점 버퍼에 대한 상대적 *위치로 기본 정점 위치*라고 부를 것입니다. 일반적으로 객체의 새 인덱스는 각 인덱스에 기본 정점 위치를 더하여 계산됩니다.

새로운 인덱스를 직접 계산하는 대신, `DrawIndexed`의 세 번째 매개변수에 베이스 버텍스 위치를 전달하여 Direct3D가 이를 수행하도록 할 수 있습니다.

다음과 같은 세 번의 호출로 구, 직육면체, 원기둥을 차례로 그릴 수 있습니다:

```
md3dImmediateContext->DrawIndexed(numSphereIndices, 0, 0);  
md3dImmediateContext->DrawIndexed(numBoxIndices, firstBoxIndex, firstBoxVertexPos);  
md3dImmediateContext->DrawIndexed(numCylIndices, firstCylIndex, firstCylVertexPos);
```

이 장의 "Shapes" 데모 프로젝트는 이 기법을 사용합니다.

6.4 버텍스 셰이더 예시

아래는 간단한 버텍스 셰이더 구현 예시입니다.

```
cbuffer cbPerObject
{
    float4x4 gWorldViewProj;
};

void VS(float3 iPosL : POSITION, float4 iColor :
    COLOR,
    out float4 oPosH : SV_POSITION, out float4 oColor :
    COLOR)
{
    // 동차 클립 공간으로 변환.
    oPosH = mul(float4(iPosL, 1.0f), gWorldViewProj);

    // 정점 색상을 픽셀 셰이더로 그대로 전달합니다. oColor = iColor;
}
```

셰이더는 HLSL(*고수준 셰이딩 언어*)이라는 언어로 작성되며, C++과 유사한 구문을 사용하므로 배우기 쉽습니다. 부록 B에는 HLSL에 대한 간결한 참조 자료가 수록되어 있습니다. HLSL 및 셰이더 프로그래밍 교육 방식은 예제 중심으로 진행됩니다. 즉, 책을 진행하면서 당면한 데모 구현에 필요한 새로운 HLSL 개념을 단계적으로 소개할 것입니다. 셰이더는 일반적으로 *효과 파일*(.fx)이라는 텍스트 기반 파일로 작성됩니다. 효과 파일은 이 장 후반부에 논의하겠지만, 지금은 버텍스 셰이더에만 집중하겠습니다.

버텍스 셰이더는 **vs**라는 함수입니다. 버텍스 셰이더에는 유효한 함수명을 자유롭게 지정할 수 있습니다. 이 버텍스 셰이더는 네 개의 매개변수를 가집니다. 첫 두 개는 *입력* 매개변수이고, 마지막 두 개는 *출력* 매개변수입니다(out 키워드로 표시됨). HLSL에는 참조나 포인터가 없으므로 함수에서 여러 값을 반환하려면 구조체나 out 매개변수를 사용해야 합니다.

첫 번째 두 입력 매개변수는 버텍스 셰이더의 입력 시그니처를 구성하며, 사용자 정의 버텍스 구조체의 데이터 멤버에 대응합니다.

파라미터 의미론 ":POSITION"과 ":COLOR"은 [그림 6.4](#)에서 보듯이 버텍스 구조체의 요소를 버텍스 셰이더 입력 파라미터에 매핑하는 데 사용됩니다.

출력 매개변수에도 의미론("SV_POSITION" 및 "COLOR")이 부여됩니다. 이는 버텍스 셰이더의 출력을

출력을 다음 단계(지오메트리 셰이더 또는 픽셀 셰이더)의 해당 입력으로 매핑하는 데 사용됩니다. **SV_POSITION** 시맨틱은 특별합니다(SV는 시스템 값을 의미). 이는 정점 위치를 보유하는 버텍스 셰이더 출력 요소를 나타내는 데 사용됩니다. 정점 위치는 클리핑과 같이 다른 정점 속성이 관여하지 않는 작업에 관여하기 때문에 다른 정점 속성과는 다르게 처리되어야 합니다. 시스템 값이 아닌 출력 매개변수의 시맨틱 이름은 유효한 시맨틱 이름이면 무엇이든 가능합니다.

```

struct Vertex
{
    XMFLOAT3 Pos;
    XMFLOAT4 Color;
};

D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, 12,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};

void VS(float3 iPosL : POSITION,
        float4 iColor : COLOR,
        out float4 oPosH : SV_POSITION,
        out float4 oColor : COLOR)
{
    // Transform to homogeneous clip space.
    oPosH = mul(float4(iPosL, 1.0f), gWorldViewProj);

    // Just pass vertex color into the pixel shader.
    oColor = iColor;
}

```

그림 6.4. 각 정점 요소는 D3D11_INPUT_ELEMENT_DESC 배열로 지정된 관련 의미론(semantic)을 가집니다. 정점 셰이더의 각 매개변수에도 연결된 의미론이 있습니다. 의미론은 정점 요소와 정점 셰이더 매개변수를 매칭하는 데 사용됩니다.

첫 번째 줄은 4×4 행렬을 곱하여 정점 위치를 로컬 공간에서 동형 클립 공간으로 변환합니다.

gWorldViewProj:

```

// 동질 클립 공간으로 변환.
oPosH = mul(float4(iPosL, 1.0f), gWorldViewProj);

```

생성자 구문 `float4(iPosL, 1.0f)`는 4차원 벡터를 생성하며, 이는 `float4(iPosL.x, iPosL.y, iPosL.z, 1.0f)`와 동일합니다. 정점의 위치는 벡터가 아닌 점이기에 때문에, 네 번째 성분($w = 1$)에 1을 배치합니다. `float2`와 `float3` 타입은 각각 2차원 및 3차원 벡터를 나타냅니다. 행렬 변수 `gWorldViewProj`는 상수 버퍼에 저장되며, 이에 대해서는 다음 섹션에서 설명하겠습니다. 내장 함수 `mul`은 벡터-행렬 곱셈에 사용됩니다. 참고로 `mul` 함수는 서로 다른 크기의 행렬 곱셈을 위해 오버로드되어 있습니다. 예를 들어 두 개의 4×4 행렬, 두 개의 3×3 행렬, 또는 1×3 벡터와 3×3 행렬을 곱하는 데 사용할 수 있습니다. 셰이더 본문의 마지막 줄은 입력 색상을 출력 매개변수로 복사하여 색상이 파이프라인의 다음 단계로 전달되도록 합니다:

```
oColor = iColor;
```

이전 버텍스 셰이더를 반한 유형과 입력 시그니처에 구조체를 사용해 (긴 매개변수 목록 대신) 동등하게 재작성할 수 있습니다:

```

cbuffer cbPerObject
{
    float4x4 gWorldViewProj;
};

struct VertexIn

```

```
{
    float3 PosL : POSITION; float4 Color :
    COLOR;
};

struct VertexOut
{
    float4 PosH : SV_POSITION; float4 Color :
    COLOR;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.PosL, 1.0f), gWorldViewProj);

    // 정점 색상을 픽셀 셰이더로 전달합니다. vout.Color = vin.Color;

    return vout;
}
```

지오메트리 셰이더가 없는 경우, 버텍스 셰이더는 최소한 투영 변환을 수행해야 합니다. 이는 하드웨어가 버텍스 셰이더를 떠날 때 버텍스가 위치해야 하는 공간을 기대하기 때문입니다(지오메트리 셰이더가 없는 경우).

지오메트리 셰이더가 존재할 경우, 투영 작업은 지오메트리 셰이더로 이관될 수 있습니다.

버텍스 셰이더(또는 지오메트리 셰이더)는 원근 분할을 수행하지 않습니다. 단지 투영 행렬 부분만 처리할 뿐입니다. 원근 분할은 나중에 하드웨어에서 수행됩니다.

6.5 상수 버퍼

이전 섹션의 예시 버텍스 셰이더 위에는 다음과 같은 코드가 있었습니다:

```
cbuffer cbPerObject
{
    float4x4 gWorldViewProj;
};
```

이 코드는 **cbPerObject**라는 **cbuffer** 객체(상수 버퍼)를 정의합니다. 상수 버퍼는 셰이더가 접근할 수 있는 다양한 변수를 저장할 수 있는 데이터 블록입니다. 이 예시에서 상수 버퍼는 **gWorldViewProj**라는 단일 4×4 행렬을 저장하며, 이는 로컬 공간에서 동형 클립 공간으로 점을 변환하는 데 사용되는 세계, 뷰, 투영 행렬을 결합한 것입니다. HLSL에서 4×4 행렬은 내장된 **float4x4** 타입으로 선언됩니다. 예를 들어 3×4 행렬과 2×2 행렬을 선언하려면 각각 **float3x4** 및 **float2x2** 타입을 사용합니다. 상수 버퍼의 데이터는 버텍스별로 변하지 않지만, 효과 프레임워크(§6.9)를 통해 C++ 애플리케이션 코드는 런타임에 상수 버퍼의 내용을 업데이트할 수 있습니다. 이는 C++ 애플리케이션 코드와 효과 코드가 통신할 수 있는 수단을 제공합니다. 예를 들어, 월드 행렬은 오브젝트별로 다르기 때문에 결합된 월드, 뷰, 투영 행렬도 오브젝트별로 달라집니다. 따라서 이전 버텍스 셰이더를 사용하여 여러

개체를 렌더링할 때는 각 개체 렌더링 전에 **gWorldViewProj** 변수를 적절히 업데이트해야 합니다.

일반적인 조언은 내용 업데이트 빈도에 따라 상수 버퍼를 생성하라는 것입니다. 예를 들어,

다음과 같은 상수 버퍼를 생성할 수 있습니다:

```
cbuffer cbPerObject
{
    float4x4 gWVP;
};
cbuffer cbPerFrame
```



```
{
    float3 gLightDirection; float3
    gLightPosition; float4 gLightColor;
};
cbuffer cbRarely
{
    float4 gFogColor; float
    gFogStart; float gFogEnd;
};
```

이 예제에서는 세 개의 상수 버퍼를 사용합니다. 첫 번째 상수 버퍼는 월드, 뷰, 프로젝션 행렬을 결합한 값을 저장합니다. 이 변수는 오브젝트에 따라 달라지므로 오브젝트별로 업데이트해야 합니다. 즉, 프레임당 100개의 오브젝트를 렌더링한다면 이 변수는 프레임당 100번 업데이트됩니다. 두 번째 상수 버퍼는 장면 조명 변수를 저장합니다. 여기서는 조명이 애니메이션 처리된다고 가정하므로 애니메이션 프레임마다 한 번씩 업데이트해야 합니다. 마지막 상수 버퍼는 안개 제어에 사용되는 변수를 저장합니다. 여기서는 장면 안개가 거의 변경되지 않는다고 가정합니다(예: 게임 내 특정 시간대에만 변경될 수 있음).

상수 버퍼를 분할하는 동기는 효율성이다. 상수 버퍼가 업데이트될 때, 그 안의 모든 변수가 업데이트되어야 합니다. 따라서 중복 업데이트를 최소화하기 위해 업데이트 빈도에 따라 그룹화하는 것이 효율적입니다.

6.6 예시 픽셀 셰이더

§5.10.3에서 논의한 바와 같이, 래스터화 과정에서 버텍스 셰이더(또는 지오메트리 셰이더)에서 출력된 버텍스 속성은 삼각형의 픽셀 전체에 걸쳐 보간됩니다. 보간된 값은 이후 픽셀 셰이더의 입력으로 전달됩니다. 지오메트리 셰이더가 없다고 가정할 때, [그림 6.5](#)는 지금까지 버텍스 데이터가 거치는 경로를 보여줍니다.

픽셀 셰이더는 각 픽셀 프래그먼트에 대해 실행되는 함수라는 점에서 버텍스 셰이더와 유사합니다. 픽셀 셰이더 입력값을 주어진다면, 그 작업은 픽셀 셰이더의 역할은 픽셀 프래그먼트에 대한 색상 값을 계산하는 것입니다. 픽셀 프래그먼트가 백 버퍼에 도달하지 못할 수도 있음을 유의해야 합니다. 예를 들어, 픽셀 셰이더에서 클리핑될 수 있으며(HLSL에는 픽셀 프래그먼트를 추가 처리에서 제외시키는 **클리프** 함수가 포함됨), 더 작은 깊이 값을 가진 다른 픽셀 프래그먼트에 의해 가려질 수 있거나, 스텐실 버퍼 테스트와 같은 후속 파이프라인 테스트에서 제외될 수 있습니다. 따라서 백 버퍼상의 한 픽셀은 여러 개의 픽셀 프래그먼트 후보를 가질 수 있습니다. 이것이 "픽셀 프래그먼트"와 "픽셀"의 의미적 차이점입니다. 비록 두 용어가 때때로 혼용되기도 하지만, 문맥상 일반적으로 어떤 의미인지 명확해집니다.

하드웨어 최적화로서, 픽셀 프래그먼트가 픽셀 셰이더에 도달하기 전에 파이프라인에 의해 거부될 수 있습니다(예: 얼리 Z 거부). 이는 깊이 테스트가 먼저 수행되는 단계이며, 깊이 테스트에서 픽셀 프래그먼트가 가려진 것으로 판단되면 픽셀 셰이더가 건너뛰어집니다. 그러나 얼리 Z 거부 최적화를 비활성화할 수 있는 몇 가지 경우가 있습니다. 예를 들어, 픽셀 셰이더가 픽셀의 깊이를 수정하는 경우, 픽셀 셰이더가 깊이를 변경할 수 있으므로 픽셀 셰이더 실행 전에 픽셀의 실제 깊이를 알 수 없기 때문에 픽셀 셰이더를 반드시 실행해야 합니다.



```

struct Vertex
{
    XMFLOAT3 Pos;
    XMFLOAT3 Normal;
    XMFLOAT2 Tex0;
};

D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"NORMAL", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 12,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"TEXCOORD", 0, DXGI_FORMAT_R32G32_FLOAT, 0, 24,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};

void VS(float3 iPosL      : POSITION,
        float3 iNormalL   : NORMAL,
        float2 iTex0      : TEXCOORD,
        out float4 oPosH   : SV_POSITION,
        out float3 oPosW   : POSITION,
        out float3 oNormalW : NORMAL,
        out float2 oTex0   : TEXCOORD0,
        out float  oFog    : TEXCOORD1)
{
}

void PS(float4 posH      : SV_POSITION,
        float3 posW      : POSITION,
        float3 normalW   : NORMAL,
        float2 tex0      : TEXCOORD0,
        float  fog       : TEXCOORD1)
{
}

```

그림 6.5. 각 정점 요소는 D3D11_INPUT_ELEMENT_DESC 배열로 지정된 관련 세미틱을 가집니다. 정점 셰이더의 각 매개변수에도 세미틱이 연결되어 있습니다. 세미엔틱은 버텍스 요소와 버텍스 셰이더 매개변수를 매핑하는 데 사용됩니다. 마찬가지로, 버텍스 셰이더의 각 출력에는 세미엔틱이 연결되어 있으며, 각 픽셀 셰이더 입력 매개변수에도 세미엔틱이 연결되어 있습니다. 이러한 세미엔틱은 버텍스 셰이더 출력을 픽셀 셰이더 입력 매개변수로 매핑하는 데 사용됩니다.

아래는 §6.4에 제시된 버텍스 셰이더에 대응하는 간단한 픽셀 셰이더입니다. 완전성을 위해 버텍스 셰이더를 다시 보여줍니다.

```

cbuffer cbPerObject
{
    float4x4 gWorldViewProj;
};

```

```
void VS(float3 iPos : POSITION, float4 iColor : COLOR, out float4 oPosH :
SV_POSITION,
out float4 oColor : COLOR)
{
    // 동차 클립 공간으로 변환.
    oPosH = mul(float4(iPos, 1.0f), gWorldViewProj);

    // 정점 색상을 픽셀 셰이더로 전달합니다. oColor = iColor;
}

float4 PS(float4 posH : SV_POSITION, float4 color : COLOR) : SV_Target
{
    return pin.Color;
}

이 예제에서 픽셀 셰이더는 보간된 색상 값을 단순히 반환합니다. 픽셀 셰이더의 입력은 정확히
버텍스 셰이더 출력과 정확히 일치해야 한다는 점에 유의하십시오. 이는 필수 조건입니다. 픽셀 셰이더는 4차원 색상 값을 반환하며, 함수 매개변수 목록 뒤에 오는 SV_TARGET 시맨틱은 반환 값의
유형이 렌더 타겟 형식과 일치해야 함을 나타냅니다.
이전 버텍스 셰이더와 픽셀 셰이더를 입력/출력 구조체를 사용하여 동등하게 재작성할 수 있습니다. 표기법은 다음과 같이 달라집니다.
입력/출력 구조체의 멤버에 의미를 부여하고, 출력 매개변수 대신 반환문을 사용하여 출력합니다.
```

```
cbuffer cbPerObject
{
    float4x4 gWorldViewProj;
};

struct VertexIn
{
    float3 Pos : 위치; float4 Color : 색상;
};

struct VertexOut
{
    float4 PosH : SV_POSITION; float4 Color :
COLOR;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.Pos, 1.0f), gWorldViewProj);

    // 정점 색상을 픽셀 셰이더로 전달합니다. vout.Color = vin.Color;

    return vout;
}

float4 PS(VertexOut pin) : SV_Target
{
    return pin.Color;
}
```

6.7 렌더 상태

Direct3D는 기본적으로 상태 기계입니다. 상태는 변경될 때까지 현재 상태를 유지합니다. 예를 들어, §6.1, §6.2 및

§6.3에서 파이프라인의 입력 어셈블러 단계에 바인딩된 입력 레이아웃, 버텍스 버퍼, 인덱스 버퍼는 다른 것을 바인딩할 때까지 그대로 유지된다는 점을 살펴보았습니다. 마찬가지로, 현재 설정된 프리미티브 토폴로지도 변경될 때까지 유효합니다. 또한 Direct3D에는 Direct3D 구성을 위해 사용할 수 있는 설정을 캡슐화하는 상태 그룹이 있습니다:

- 1. ID3D11RasterizerState:** 이 인터페이스는 파이프라인의 래스터화 단계를 구성하는 데 사용되는 상태 그룹을 나타냅니다.
- 2. ID3D11BlendState:** 이 인터페이스는 블렌드 작업을 구성하는 데 사용되는 상태 그룹을 나타냅니다. 이러한 상태들은 블렌드 장에서 논의할 예정입니다. 기본적으로 블렌딩은 비활성화되어 있으므로 당분간은 신경 쓸 필요가 없습니다.
- 3. ID3D11DepthStencilState:** 이 인터페이스는 깊이 테스트와 스텔스 테스트를 구성하는 데 사용되는 상태 그룹을 나타냅니다. 스텔스 버퍼에 관한 장에서 이러한 상태들에 대해 논의할 것입니다. 기본적으로 스텔스링은 비활성화되어 있으므로 당분간은 신경 쓸 필요가 없습니다. 기본 깊이 테스트 설정은 §4.1.5에 설명된 표준 깊이 테스트를 수행하도록 설정되어 있습니다.

현재 우리가 주목해야 할 유일한 상태 블록 인터페이스는 ID3D11RasterizerState 인터페이스입니다. **D3D11_RASTERIZER_DESC** 구조체를 작성한 후 다음 메서드를 호출하여 이 유형의 인터페이스를 생성할 수 있습니다:

```
HRESULT ID3D11Device::CreateRasterizerState(
    const D3D11_RASTERIZER_DESC *pRasterizerDesc, ID3D11RasterizerState
    **ppRasterizerState);
```

첫 번째 매개변수는 생성할 래스터라이저 상태 블록을 설명하는 작성된 D3D11_RASTERIZER_DESC 구조체입니다. 두 번째 매개변수는 생성된 **ID3D11RasterizerState** 인터페이스에 대한 포인터를 반환하는 데 사용됩니다.

D3D11_RASTERIZER_DESC 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_RASTERIZER_DESC {
    D3D11_FILL_MODE FillMode; // 기본값: D3D11_FILL_SOLID D3D11_CULL_MODE CullMode; //
    기본값: D3D11_CULL_BACK BOOL FrontCounterClockwise; // 기본값: false
    INT DepthBias; // 기본값: 0
    FLOAT DepthBiasClamp; // 기본값: 0.0f FLOAT
    SlopeScaledDepthBias; // 기본값: 0.0f BOOL DepthClipEnable; // 기본
    값: true BOOL ScissorEnable; // 기본값: false
    BOOL MultisampleEnable; // 기본값: false BOOL
    AntialiasedLineEnable; // 기본값: false
} D3D11_RASTERIZER_DESC;
```

이 멤버들의 대부분은 고급 기능이거나 자주 사용되지 않으므로, 각 멤버에 대한 설명은 SDK 문서를 참조하시기 바랍니다. 다만, 여기서 논의할 가치가 있는 것은 처음 세 개뿐입니다.

- 1. FillMode:** 와이어프레임 렌더링에는 **D3D11_FILL_WIREFRAME**을, 솔리드 렌더링에는 **D3D11_FILL_SOLID**을 지정합니다. 솔리드 렌더링이 기본값입니다.
- 2. CullMode:** **D3D11_CULL_NONE**을 지정하면 정면 삼각형 커팅(culling)을 비활성화하고, **D3D11_CULL_BACK**을 지정하면 정면 삼각형 커팅을 수행하며, **D3D11_CULL_FRONT**: 정면 삼각형 커팅. 기본적으로 후면 삼각형은 커팅됩니다.
- 3. FrontCounterClockwise:** 카메라 기준 시계 방 향 으 로 배열된 삼각형을 정면 삼각형으로, 카메라 기준 시계 반대 방향으로 배열된 삼각형을 후면 삼각형으로 처리하려면 **false**를 지정하십시오. 카메라 기준 시계 반대 방향으로 배열된 삼각형을 정면 삼각형으로, 시계 방향으로 배열된 삼각형을 후면 삼각형으로 처리하려면 **true**를 지정합니다. 이 상태는 기본적으로 false입니다.

ID3D11RasterizerState 객체가 생성된 후에는 새로운 상태 블록으로 장치를 업데이트할 수 있습니다:

```
void ID3D11DeviceContext::RSSetState( ID3D11RasterizerState
    *pRasterizerState);
```

다음 코드는 백면 제거를 비활성화하는 래스터라이저 상태를 생성하는 방법을 보여줍니다:

```
D3D11_RASTERIZER_DESC rsDesc;
ZeroMemory(&rsDesc, sizeof(D3D11_RASTERIZER_DESC)); rsDesc.FillMode =
D3D11_FILL_SOLID;
rsDesc.CullMode = D3D11_CULL_NONE;
rsDesc.FrontCounterClockwise = false; rsDesc.DepthClipEnable = true;

HR(md3dDevice->CreateRasterizerState(&rsDesc, &mNoCullRS));
```

 **ZeroMemory**를 사용하면 기본값이 0 또는 false 인 다른 속성들을 설정하지 않아도 초기화됩니다. 그러나 속성의 기본값이 0이 아니거나 true 인 경우, 기본값을 적용하려면 해당 속성을 명시적으로 설정해야 합니다.

참고로 애플리케이션에서는 여러 개의 서로 다른 **ID3D11RasterizerState** 객체가 필요할 수 있습니다. 따라서 초기화 시점에 모든 객체를 생성한 후, 애플리케이션 업데이트/드로잉 코드에서 필요에 따라 객체를 전환하면 됩니다. 예를 들어 두 개의 객체가 있고, 첫 번째는 와이어프레임 모드로, 두 번째는 솔리드 모드로 그리려는 경우를 가정해 보겠습니다. 그러면 두 개의 **ID3D11RasterizerState** 객체를 생성하고 객체를 그릴 때 객체 간에 전환하면 됩니다:

```
// 초기화 시 렌더 상태 객체 생성. ID3D11RasterizerState* mWireframeRS;
ID3D11RasterizerState* mSolidRS;
...

// 드로잉 함수 내에서 렌더링 상태 객체를 전환합니다. md3dImmediateContext-
>RSSetState(mSolidRS); DrawObject();

md3dImmediateContext->RSSetState(mWireframeRS); DrawObject();
```

Direct3D는 절대로 상태를 이전 설정으로 복원하지 않는다는 점을 유의해야 합니다. 따라서 객체를 그릴 때 항상 필요한 상태를 설정해야 합니다. 장치의 현재 상태에 대해 잘못된 가정을 하면 잘못된 출력 결과가 발생합니다.

각 상태 블록에는 기본 상태가 있습니다. null을 인수로 **RSSetState** 메서드를 호출하여 기본 상태로 되돌릴 수 있습니다:

```
// 기본 상태 복원. md3dImmediateContext->RSSetState( 0 );
```

일반적으로 애플리케이션은 런타임에 추가 렌더 상태 그룹을 생성할 필요가 없습니다. 따라서 애플리케이션은 초기화 시점에 필요한 모든 렌더링 상태 그룹을 정의하고 생성할 수 있습니다. 또한 런타임에 렌더링 상태 그룹을 수정할 필요가 없으므로, 렌더링 코드에 대해 전역 위기 전용 접근을 제공할 수 있습니다. 예를 들어 모든 렌더링 상태 그룹 객체를 정적 클래스에 배치할 수 있습니다. 이렇게 하면 중복된 렌더링 상태 그룹 객체를 생성하지 않으면서 렌더링 코드의 다양한 부분에서 렌더링 상태 그룹 객체를 공유할 수 있습니다.

6.8 이펙트

*이펙트 프레임워크*는 특정 렌더링 이펙트를 구현하기 위해 함께 작동하는 셰이더 프로그램과 렌더 상태를 구성하는 프레임워크를 제공하는 유틸리티 코드 집합입니다. 예를 들어, 물, 구름, 금속 물체, 애니메이션 캐릭터 렌더링을 위한 서로 다른 이펙트를 가질 수 있습니다. 각 이펙트는 최소한 하나의 버텍스 셰이더, 하나의 픽셀 셰이더, 그리고 이펙트 구현을 위해 작동하는 렌더 상태로 구성됩니다.

이전 버전의 Direct3D에서는 D3D 10 라이브러리와 링크하면 효과 프레임워크가 바로 작동했습니다.

Direct3D 11에서는 효과 프레임워크가 D3DX 라이브러리로 이동되었으며, 별도의 헤더 파일(**d3dx11Effect.h**)을 포함하고 별도의 라이브러리(릴리스 빌드용 **D3DX11Effects.lib**, 디버그 빌드용 **D3DX11EffectsD.lib**)로 링크해야 합니다. 또한 Direct3D 11에서는 효과 라이브러리 코드의 전체 소스 코드(*DirectX SDK\Samples\C++\Effects11*)를 제공합니다. 따라서 필요에 따라 효과 프레임워크를 수정할 수 있습니다. 본서에서는 수정 없이 효과 프레임워크를 그대로 사용할 것입니다. 라이브러리를 사용하려면 먼저 *Effects11* 프로젝트를 릴리스 및 디버그 모드 모두에서 빌드하여 **D3DX11Effects.lib** 및 **D3DX11EffectsD.lib** 파일을 생성해야 합니다. 효과 프레임워크가 업데이트되지 않는 한 이 작업은 한 번만 수행하면 됩니다(예: 새 버전의 DirectX SDK가 이러한 파일을 업데이트할 수 있으므로 최신 버전을 얻으려면 .lib 파일을 다시 빌드해야 할 수 있음). **d3dx11Effect.h** 헤더 파일은 *DirectX SDK\Samples\C++\Effects11\Inc* 디렉터리에서 찾을 수 있습니다. 샘플 프로젝트의 경우, **d3dx11Effect.h**, **D3DX11EffectsD.lib** 및 **D3DX11Effects.lib** 파일을 모든 프로젝트가 코드를 공유하는 Common 디렉터리에 배치합니다(샘플 프로젝트 구성에 대한 설명은 "소개"를 참조하십시오).

6.8.1 효과 파일

우리는 버텍스 셰이더, 픽셀 셰이더, 그리고 어느 정도는 지오메트리 및 테셀레이션 셰이더에 대해서도 논의했습니다. 또한 셰이더에서 접근 가능한 "전역" 변수를 저장하는 데 사용할 수 있는 상수 버퍼에 대해서도 논의했습니다. 이러한 코드는 일반적으로 효과 파일(.fx)에 작성되며, 이는 단순한 텍스트 파일입니다(C++ 코드가 .h/.cpp 파일에 작성되는 것과 같습니다). 셰이더와 상수 버퍼를 포함하는 것 외에도, 효과 파일은

효과에는 또한 하나 이상의 *테크닉*이 포함됩니다. 테크닉은 차례로 하나 이상의 *패스*를 포함합니다.

1. **technique**11: 테크닉은 특정 렌더링 테크닉을 생성하는 데 사용되는 하나 이상의 패스로 구성됩니다. 각 패스마다 지오메트리는 서로 다른 방식으로 렌더링되며, 각 패스의 결과는 원하는 결과를 얻기 위해 어떤 방식으로든 결합됩니다. 예를 들어, 지형 렌더링 테크닉은 멀티패스 텍스처링 테크닉을 사용할 수 있습니다. 멀티패스 테크닉은 각 패스마다 지오메트리를 다시 그리기 때문에 일반적으로 비용이 많이 든다는 점에 유의하십시오. 그러나 일부 렌더링 테크닉을 구현하려면 멀티패스 테크닉이 필요합니다.
2. **패스**: 패스는 버텍스 셰이더, 선택적 지오메트리 셰이더, 선택적 테셀레이션 관련 셰이더, 픽셀 셰이더 및 렌더 상태로 구성됩니다. 이러한 구성 요소는 해당 패스에서 지오메트리를 처리하고 셰이딩하는 방법을 나타냅니다. 픽셀 셰이더 역시 선택적일 수 있음을 유의하십시오(드물게). 예를 들어, 백 버퍼가 아닌 깊이 버퍼에만 렌더링하려는 경우, 픽셀 셰이더로 픽셀을 셰이딩할 필요가 없습니다.

기법은 효과 그룹(effect group)이라는 개념으로 묶을 수도 있습니다. 명시적으로 정의하지 않으면  컴파일러가 효과 파일에 포함된 모든 기법을 담은 익명의 효과 그룹을 생성합니다. 이 책에서는 효과 그룹을 명시적으로 어떤 효과 그룹도 명시적으로 정의하지 않습니다.

아래는 본 장의 데모용 전체 효과 파일입니다:

```
cbuffer cbPerObject
{
    float4x4 gWorldViewProj;
};

struct VertexIn
{
    float3 Pos : 위치; float4 Color : 색상;
};

struct VertexOut
{
    float4 PosH : SV_POSITION; float4 Color :
    COLOR;
};

VertexOut VS(VertexIn vin)
{
    VertexOut vout;

    // 동차 클립 공간으로 변환.
    vout.PosH = mul(float4(vin.Pos, 1.0f), gWorldViewProj);

    // 정점 색상을 픽셀 셰이더로 전달합니다. vout.Color = vin.Color;

    return vout;
}

float4 PS(VertexOut pin) : SV_Target
{
    return pin.Color;
}

technique 11 ColorTech
{
    pass P0
    {
        SetVertexShader( CompileShader( vs_5_0, VS() ) ); SetPixelShader( CompileShader(
        ps_5_0, PS() ) );
    }
}
```

 점과 벡터의 좌표는 다양한 공간(예: 로컬 공간, 월드 공간, 뷰 공간, 동질 공간을 기준으로 지정됩니다. 코드를 읽을 때 점/벡터의 좌표가 어떤 좌표계를 기준으로 하는지 명확하지 않을 수 있습니다. 따라서 우리는 종종 다음과 같은 접미사를 사용하여 공간을 표시합니다: L(로컬 공간), W(월드 공간), V(뷰 공간), H(동질 클립 공간). 다음은 몇 가지 예시입니다: 공간), W(월드 공간), V(뷰 공간), H(동일 클립 공간)을 나타냄). 예시는 다음과 같습니다:

```
float3 iPosL; // 로컬 공간 float3 gEyePosW; // 월드 공간
float3 normalV; // 뷰 공간
float4 posH; // 동차 클립 공간
```

패스는 렌더 상태로 구성됩니다. 즉, 상태 블록은 효과 파일에서 직접 생성 및 설정할 수 있습니다. 이는 효과가 작동하기 위해 특정 렌더 상태를 요구할 때 편리합니다. 반면 일부 효과는 가변적인 렌더 상태 설정으로도 작동할 수 있으며, 이 경우 상태 전환을 용이하게 하기 위해 애플리케이션 수준에서 상태를 설정하는 것이 바람직합니다. 다음 코드는 효과 파일에서 래스터라이저 상태 블록을 생성하고 설정하는 방법을 보여줍니다.

```
RasterizerState Wireframe RS
{
    FillMode = Wireframe; CullMode = Back;
    FrontCounterClockwise = false;

    // 설정하지 않은 속성에 대해 사용되는 기본값.
};

technique11 ColorTech
{
    pass P0
    {
        SetVertexShader( CompileShader( vs_5_0, VS() ) ); SetPixelShader( CompileShader(
ps_5_0, PS() ) );

        SetRasterizerState(Wireframe RS);
    }
}
```

래스터라이저 상태 객체 정의에서 오른쪽 값들은 접두사가 생략된 점을 제외하면(예: D3D11_FILL_ 및 D3D11_CULL_ 생략) C++의 경우와 기본적으로 동일합니다.

 효과(effect)는 일반적으로 외부 .fx 파일에 작성되므로, 소스 코드를 재컴파일하지 않고도 수정할 수 있습니다.

6.8.2 셰이더 컴파일

효과를 만드는 첫 번째 단계는 .fx 파일에 정의된 셰이더 프로그램을 컴파일하는 것입니다. 이는 다음 D3DX 함수를 사용하여 수행할 수 있습니다.

```
HRESULT D3DX11CompileFromFile( LPCTSTR
pSrcFile,
CONST D3D10_SHADER_MACRO *pDefines, LPD3D10INCLUDE pInclude,
LPCSTR pFunctionName, LPCSTR pProfile,
UINT Flags1, UINT
Flags2,
ID3DX11ThreadPump *pPump, ID3D10Blob
**ppShader, ID3D10Blob **ppErrorMsgs,
HRESULT *pHResult);
```

1. **pFileName**: 컴파일하려는 효과 소스 코드가 포함된 .fx 파일의 이름입니다.
2. **pDefines**: 사용하지 않는 고급 옵션입니다. SDK 문서를 참조하십시오. 이 책에서는 항상 null을 지정합니다.
3. **pInclude**: 사용하지 않는 고급 옵션입니다. SDK 문서를 참조하십시오. 이 책에서는 항상 null을 지정합니다.

4. pFunctionName: 셰이더 함수 이름 진입점입니다. 이는 셰이더 프로그램을 개별적으로 컴파일할 때만 사용됩니다. 효과 프레임워크를 사용할 때는 null을 지정하십시오. 효과 파일 내부에서 정의된 테크닉이 셰이더 진입점을 지정하기 때문입니다.

5. pProfile: 사용 중인 셰이더 버전을 지정하는 문자열입니다. Direct3D 11 효과의 경우 셰이더 버전 5.0("fx_5_0")을 사용합니다.

6. Flags1: 셰이더 코드 컴파일 방식을 지정하는 플래그입니다. SDK 문서에는 상당히 많은 플래그가 나열되어 있지만, 이 책에서 사용하는 것은 다음 두 가지뿐입니다:

D3D10_SHADER_DEBUG: 셰이더를 디버깅 모드로 컴파일합니다.

D3D10_SHADER_SKIP_OPTIMIZATION: 컴파일러가 최적화를 건너뛰도록 지시합니다(디버깅에 유용함).

7. Flags2: 사용하지 않는 고급 효과 컴파일 옵션; SDK 문서를 참조하십시오.

8. pPump: 셰이더를 비동기적으로 컴파일하는 고급 옵션으로, 본 문서에서는 사용하지 않습니다. SDK 문서를 참조하십시오. 본 문서에서는 항상 null을 지정합니다.

9. ppShader: 컴파일된 셰이더를 저장하는 **ID3D10Blob** 데이터 구조체에 대한 포인터를 반환합니다.

10. ppErrorMsgs: 컴파일 오류가 있는 경우 해당 오류를 포함하는 문자열을 저장하는 **ID3D10Blob** 데이터 구조체에 대한 포인터를 반환합니다.

11. pHResult: 비동기 컴파일 시 반환된 오류 코드를 얻는 데 사용됩니다. **pPump**에 null을 지정한 경우 null을 지정하십시오.

이 함수는 fx 파일 내부의 셰이더를 컴파일하는 것 외에도 개별 셰이더를 컴파일하는 데에도 사용할 수 있습니다.

-  **Note:**
1. 일부 프로그래머는 효과 프레임워크를 사용하지 않거나(또는 자체적으로 제작하여) 셰이더를 별도로 정의하고 컴파일합니다.

2. 이전 함수에서 "D3D10"에 대한 참조는 오타가 아닙니다. D3D11 컴파일러는 D3D10 컴파일러를 기반으로 구축되었으므로 Direct3D 11 팀은 일부 식별자의 이름을 변경하지 않았습
니다.

3. ID3D10Blob 유형은 두 가지 메서드를 가진 일반적인 메모리 청크입니다.

(a) LPVOID GetBufferPointer: 데이터에 대한 void*를 반환하므로 사용 전에 적절한 유형으로 캐스팅해야 합니다(다음 예시 참조).

(b) SIZE_T GetBufferSize: 버퍼의 바이트 크기를 반환합니다.

이펙트 셰이더가 컴파일된 후에는 다음 함수를 사용하여 ID3DXEffect11 인터페이스로 표현되는 이펙트를 생성할 수 있습니다:

```
HRESULT D3DX11CreateEffectFromMemory( void *pData,
    SIZE_T DataLength, UINT FXFlags,
    ID3D11Device *pDevice,
    ID3DX11Effect **ppEffect);
```

1. pData: 컴파일된 효과 데이터에 대한 포인터.

2. DataLength: 컴파일된 효과 데이터의 바이트 크기.

3. FXFlags: 효과 플래그는 D3DX11CompileFromFile 함수에서 **Flags2**에 지정된 플래그와 일치해야 합니다.

4. pDevice: Direct3D 11 장치에 대한 포인터.

5. ppEffect: 생성된 효과에 대한 포인터를 반환합니다.

다음 코드는 효과를 컴파일하고 생성하는 방법을 보여줍니다:

```
DWORD shaderFlags = 0;
#ifdef DEBUG || defined(_DEBUG) shaderFlags |=
    D3D10_SHADER_DEBUG;
shaderFlags |= D3D10_SHADER_SKIP_OPTIMIZATION; #endif

ID3D10Blob* compiledShader = 0; ID3D10Blob*
compilationMsgs = 0;
HRESULT hr = D3DX11CompileFromFile(L"color.fx", 0, 0, 0, "fx_5_0", shaderFlags,
    0, 0, &compiledShader, &compilationMsgs, 0);

// compilationMsgs에는 오류나 경고가 저장될 수 있습니다. if(compilationMsgs != 0)
{
    MessageBoxA(0, (char*)compilationMsgs->GetBufferPointer(), 0, 0); ReleaseCOM(compilationMsgs);
}
```



```
// compilationMsgs가 없더라도 다른 오류가 없는지 확인합니다.
// 다른 오류가 없는지 확인합니다. if(FAILED(hr))
{
    DXTrace( FILE , (DWORD) LINE , hr, L"D3DX11CompileFromFile", true);
}

ID3DX11Effect* mFX; HR(D3DX11CreateEffectFromMemory(
    compiledShader->GetBufferPointer(), compiledShader-
    >GetBufferSize(),
    0, md3dDevice, &mFX));

// 컴파일된 셰이더 작업 완료. ReleaseCOM(compiledShader);
```

Direct3D 리소스 생성은 비용이 많이 들며, 항상 초기화 시점에 수행해야 하며 런타임에는 절대 수행해서는 안 됩니다.

즉, 입력 레이아웃, 버퍼, 렌더 상태 객체 및 효과 생성은 항상 초기화 시점에 수행해야 합니다.

6.8.3 C++ 애플리케이션에서 효과와의 인터페이스

C++ 애플리케이션 코드는 일반적으로 효과와 통신해야 합니다. 특히 C++ 애플리케이션은 상수 버퍼의 변수를 업데이트해야 하는 경우가 많습니다. 예를 들어, 효과 파일에 다음과 같은 상수 버퍼가 정의되어 있다고 가정해 보겠습니다:

```
cbuffer cbPerObject
{
    float4x4 gWVP; float4
    gColor; float gSize;
    int gIndex;
    bool gOptionOn;
};
```

ID3DX11Effect 인터페이스를 통해 상수 버퍼 내 변수들에 대한 포인터를 얻을 수 있습니다:

```
ID3DX11EffectMatrixVariable* fxWVPVar; ID3DX11EffectVectorVariable* fxColorVar;
ID3DX11EffectScalarVariable* fxSizeVar; ID3DX11EffectScalarVariable* fxIndexVar;
ID3DX11EffectScalarVariable* fxOptionOnVar;
fxWVPVar = mFX->GetVariableByName("gWVP")->AsMatrix(); fxColorVar = mFX-
>GetVariableByName("gColor")->AsVector(); fxSizeVar = mFX->GetVariableByName("gSize")->AsScalar();
fxIndexVar = mFX->GetVariableByName("gIndex")->AsScalar(); fxOptionOnVar = mFX-
>GetVariableByName("gOptionOn")->AsScalar();
```

ID3DX11Effect::GetVariableByName 메서드는 ID3DX11EffectVariable 유형의 포인터를 반환합니다. 이는 일반적인 효과 변수 유형입니다. 특수화된 유형(예: 행렬, 벡터, 스칼라)에 대한 포인터를 얻으려면 적절한 As- 메서드(예: AsMatrix, AsVector, AsScalar)를 사용해야 합니다.

변수에 대한 포인터를 확보한 후에는 C++ 인터페이스를 통해 이를 업데이트할 수 있습니다. 다음은 몇 가지 예시입니다:

```
fxWVPVar->SetMatrix((float*)&M ); // M이 XMMATRIX 유형이라고 가정
fxColorVar->SetFloatVector( (float*)&v );

// v가 XMVECTOR 유형이라고 가정
fxSizeVar->SetFloat( 5.0f );
fxIndexVar->SetInt( 77 ); fxOptionOnVar-
>SetBool( true );
```

이러한 호출은 효과 객체 내부의 캐시를 업데이트하며, 렌더링 패스 적용 시점(§6.8.4)까지 GPU 메모리로 전송되지 않습니다. 이는 비효율적인 다수의 소규모 업데이트 대신 GPU 메모리에 대한 단일 업데이트를 보장합니다.

 **이펙트 변수는 반드시 특수화될 필요는 없습니다. 예를 들어 다음과 같이 작성할 수 있습니다.**

```
ID3DX11EffectVariable* mfxEyePosVar;
mfxEyePosVar = mFX->GetVariableByName("gEyePosW");
...
mfxEyePosVar->SetRawValue(&mEyePos, 0, sizeof(XMFLOAT3));
```

이는 임의의 크기(예: 일반 구조체)의 변수를 설정하는 데 유용합니다. ID3DX11EffectVectorVariable 인터페이스는 4차원 벡터를 가정하므로, 앞서 언급한 바와 같이 3차원 벡터(예: XMFLOAT3)를 사용하려면 ID3DX11EffectVariable을 사용해야 합니다.

상수 버퍼 변수 외에도, 효과에 저장된 테크닉 객체에 대한 포인터를 획득해야 합니다. 이는 다음과 같이 수행됩니다.

```
ID3DX11EffectTechnique* mTech;
mTech = mFX->GetTechniqueByName("ColorTech");
```

이 메서드가 받는 단일 매개변수는 포인터를 얻고자 하는 테크닉의 문자열 이름입니다.

6.8.4 효과를 이용한 그리기

기하학을 그리기 위한 테크닉을 사용하려면, 상수 버퍼의 변수들이 최신 상태인지 확인하기만 하면 됩니다. 그런 다음 테크닉의 각 패스를 순회하며 패스를 적용하고 기하학을 그립니다.

```
// 상수 설정
XMMATRIX world = XMLoadFloat4x4(&mWorld); XMMATRIX view =
XMLoadFloat4x4(&mView); XMMATRIX proj = XMLoadFloat4x4(&mProj);
XMMATRIX worldViewProj = world*view*proj;

mfxWorldViewProj->SetMatrix(reinterpret_cast<float*>(&worldViewProj)); D3DX11_TECHNIQUE_DESC techDesc;
mTech->GetDesc(&techDesc);
for(UINT p = 0; p < techDesc.Passes; ++p)
{
    mTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext);

    // 일부 지오메트리 그리기.
    md3dImmediateContext->DrawIndexed(36, 0, 0);
}
```

패스에서 지오메트리가 렌더링되면 해당 패스에 설정된 셰이더와 렌더 상태로 렌더링됩니다. ID3DX11EffectTechnique::GetPassByIndex 메서드는 지정된 인덱스의 패스를 나타내는 ID3DX11EffectPass 인터페이스에 대한 포인터를 반환합니다. Apply 메서드는 GPU 메모리에 저장된 상수 버퍼를 업데이트하고, 셰이더 프로그램을 파이프라인에 바인딩하며, 패스가 설정하는 모든 렌더 상태를 적용합니다. 현재 버전의 Direct3D 11에서 ID3DX11EffectPass::Apply의 첫 번째 매개변수는 사용되지 않으므로 0을 지정해야 합니다. 두 번째 매개변수는 패스가 사용할 디바이스 컨텍스트에 대한 포인터입니다.

드로우 호출 사이에 상수 버퍼의 변수 값을 변경해야 하는 경우, 업데이트를 위해 Apply를 호출해야 합니다.

기하 구조를 그리기 전 변경 사항:

```
for(UINT i = 0; i < techDesc.Passes; ++i)
{
    ID3DX11EffectPass* pass = mTech->GetPassByIndex(i);

    // 지형 지오메트리에 대한 결합된 월드-뷰-프로젝션 행렬 설정. worldViewProj = landWorld*view*proj;
    mfxWorldViewProj->SetMatrix(reinterpret_cast<float*>(&worldViewProj));
    pass->Apply(0, md3dImmediate Context);
    mLand.draw();

    // 파도 지오메트리에 대한 결합된 월드-뷰-프로젝션 행렬 설정. worldViewProj = wavesWorld*view*proj;
    mfxWorldViewProj->SetMatrix(reinterpret_cast<float*>(&worldViewProj));
```

```
pass->Apply(0, md3dImmediate Context);
mWaves.draw();
}
```

6.8.5 빌드 시점에 이펙트 컴파일하기

지금까지 `D3DX11CompileFromFile` 함수를 통해 런타임에 이펙트를 컴파일하는 방법을 살펴보았습니다. 이 방법은 다소 불편한데, 이펙트 파일 코드에 컴파일 오류가 있을 경우 프로그램을 실행하기 전까지는 이를 알 수 없기 때문입니다. DirectX SDK에 포함된 `fxc` 도구(`DirectX SDK\Utilities\bin\x86` 위치)를 사용하면 이펙트를 오프라인에서 컴파일할 수 있습니다. 또한 VC++ 프로젝트를 수정하여 일반 빌드 프로세스의 일부로 `fxc`를 호출해 효과를 컴파일할 수 있습니다. 다음 단계는 이를 수행하는 방법을 보여줍니다:

1. 소개 부분에서 설명한 대로 프로젝트의 VC++ 디렉터리 탭에서 "실행 파일 디렉터리" 아래에 `DirectX SDK\Utilities\bin\x86` 경로가 포함되어 있는지 확인하십시오.
2. 효과 파일을 프로젝트에 추가합니다.
3. 각 효과에 대해 솔루션 탐색기에서 효과 파일을 마우스 오른쪽 버튼으로 클릭하고 속성을 선택한 다음 사용자 지정 빌드 도구를 추가합니다(그림 6.6 참조).

디버그 모드:

```
fxc /Fc /Od /Zi /T fx_5_0 /Fo "%(RelativeDir)\%(Filename).fxo" "%(FullPath)"
```

릴리스 모드:

```
fxc /T fx_5_0 /Fo "%(RelativeDir)\%(Filename).fxo" "%(FullPath)"
fxc 컴파일러 플래그 전체 목록은 SDK 문서를 참조하십시오. 디버그 모드에 사용하는 세 가지 플래그는 각각
```

“/Fc /Od /Zi”는 각각 어셈블리 리스팅 출력, 최적화 비활성화, 디버그 정보 활성화를 의미합니다.

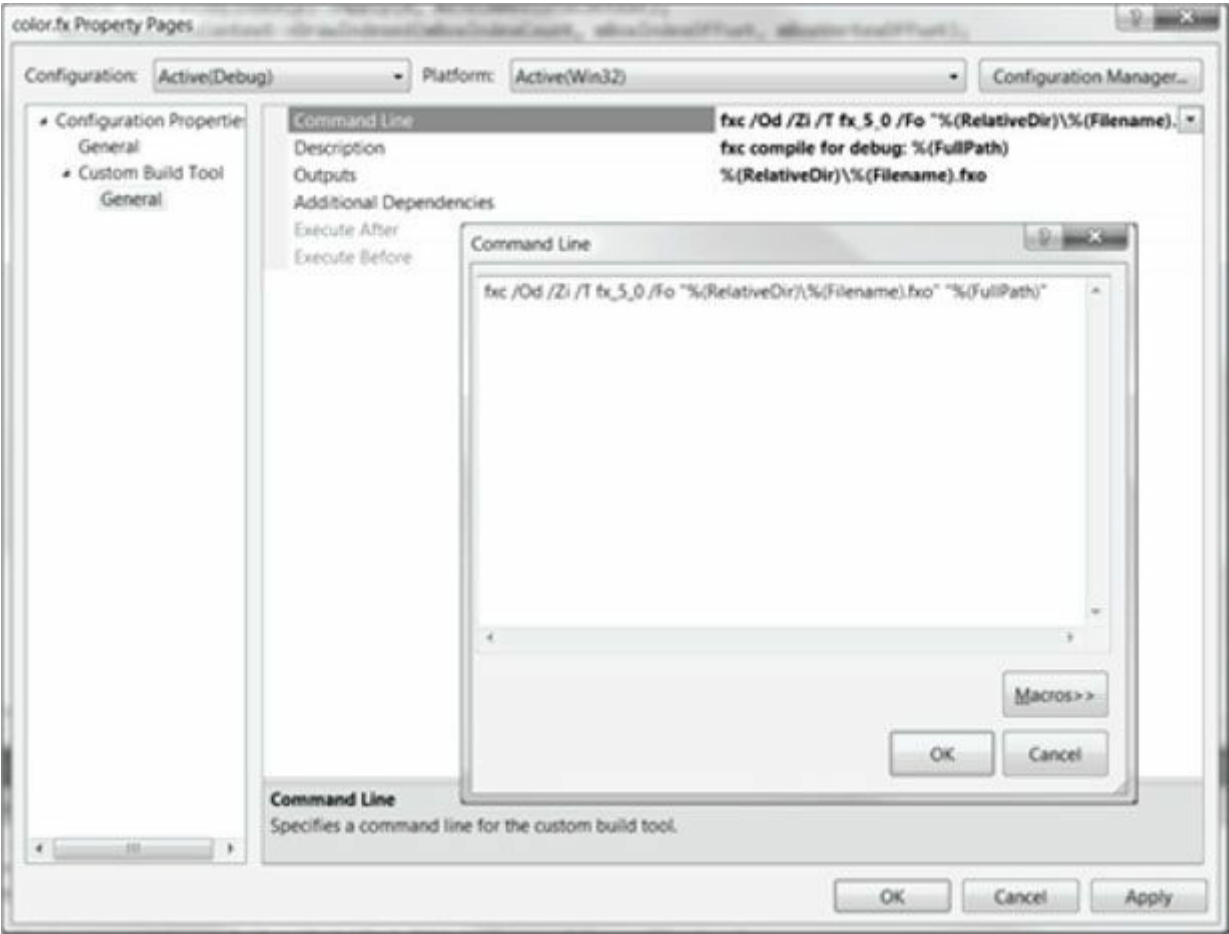


그림 6.6. 프로젝트에 사용자 정의 빌드 도구 추가하기.

셰이더 어셈블리 코드를 수시로 확인하는 것은 예상치 못한 명령어가 생성되지 않았는지 점검하는 데 유용합니다. 예를 들어 HLSL 코드에 조건문이 있다면 어셈블리 코드에 분기 명령어가 있을 것으로 예상할 수 있습니다. GPU에서의 분기는 상당히 비용이 많이 들거나(또는 일부 DirectX 9 하드웨어에서는 지원되지 않았습니다), 따라서 컴파일러가 조건문을 평탄화하여 두 분기 모두를 평가한 후 두 결과 사이를 보간하여 올바른 답을 선택하는 경우가 있습니다. 즉, 다음 코드들은 동일한 결과를 반환합니다:



Note

조건부	Flattened
<pre>float x = 0; // s == 1 (참) 또는 s == 0 (거짓) if(s) { x = sqrt(y); } else { x = 2*y; }</pre>	<pre>float a = 2*y; float b = sqrt(y); float x = a + s*(b-a); // s == 1: // x = a + b - a = b = sqrt(y) // s == 0: // x = a + 0*(b-a) = a = 2*y</pre>

따라서 평탄화된 메서드는 분기 없이 동일한 결과를 제공하지만, 어셈블리 코드를 확인하지 않으면 평탄화가 진행되었는지, 아니면 실제 분기 명령어가 생성되었는지 알 수 없습니다. 핵심은 때로는 실제로 무슨 일이 일어나고 있는지 확인하기 위해 어셈블리 코드를 살펴봐야 한다는 점입니다.

이제 프로젝트를 빌드할 때 *fxc*가 각 이펙트에 대해 호출되어 컴파일된 버전의 이펙트를 *.fxo* 파일로 생성합니다. 또한 *fxc*에서 컴파일 경고나 오류가 발생하면 디버그 출력 창에 표시됩니다. 예를 들어 *color.fx* 이펙트 파일에서 변수 이름을 잘못 지정한 경우:

```
// worldViewProj가 아닌 gWorldViewProj여야 합니다! vout.PosH =
mul(float4(vin.Pos, 1.0f), worldViewProj);
```

그러면 이 하나의 실수(가장 상단의 오류가 수정해야 할 핵심 오류)로 인해 데뷔 출력 창에 꽤 많은 오류가 발생합니다:

```
error X3004: 선언되지 않은 식별자 'worldViewProj'
error X3013: 'mul': 내장 함수는 2개의 매개변수를 받지 않습니다. error X3013: 사용 가능한 내장 함수
는 다음과 같습니다:
error X3013: mul(float, float)
...
```

컴파일 시점에 오류 메시지를 확인하는 것이 런타임보다 훨씬 편리합니다. 이제 빌드 프로세스의 일환으로 효과 파일(.*fxo*)을 컴파일했으므로 런타임에 별도로 컴파일할 필요가 없습니다(즉, *D3DX11CompileFromFile* 호출이 불필요합니다). 그러나 컴파일된 셰이더 데이터를 *.fxo* 파일에서 불러와 *D3DX11CreateEffectFromMemory* 함수에 전달해야 합니다. 이는 다음과 같은 표준 C++ 파일 입력 메커니즘을 사용하여 수행할 수 있습니다:

```
std::ifstream    fin("fx/color.fxo",    std::ios::binary);    fin.seekg(0,

std::ios_base::end);
int size = (int)fin.tellg();
fin.seekg(0, std::ios_base::beg); std::vector<char>
compiledShader(size);

fin.read(&compiledShader[0], size); fin.close();

HR(D3DX11CreateEffectFromMemory(&compiledShader[0], size, 0, md3dDevice, &mFX));
```

런타임에 셰이더 코드를 컴파일하는 "Box" 데모를 제외하고, 우리는 모든 셰이더를 빌드 프로세스의 일부로 컴파일합니다.

6.8.6 "셰이더 생성기"로서의 효과 프레임워크

이 섹션 초반에 효과는 여러 기법을 가질 수 있다고 언급했습니다. 그렇다면 렌더링 효과에 왜 여러 기법이 필요할까요? 그림자 처리를 예로 들어, 구체적인 구현 방식은 생략하겠습니다. 기본적으로 그림자 품질이 높을수록 그림자 처리 기법의 비용도 커집니다. 저사양 및 고사양 그래픽 카드를 모두 사용하는 사용자를 지원하기 위해 저품질, 중품질, 고품질의 그림자 처리 기법을 구현할 수 있습니다. 따라서 하나의 그림자 효과라도 여러 기법을 사용해 구현하게 됩니다. 그림자 효과 파일은 다음과 같을 수 있습니다:

// 상수 버퍼, 버텍스 구조체 등은 생략...

VertexOut VS(VertexIn vin) { /* 구현 세부사항 생략 */ } float4 LowQualityPS(VertexOut pin) :

SV_Target
{

/* 모든 품질 레벨에 공통적으로 적용되는 작업 수행 */

/* 저품질 전용 작업 수행 */

/* 모든 품질 레벨에 공통적인 추가 작업 수행 */

}

float4 MediumQualityPS(VertexOut 핀) : SV_Target

{

/* 모든 품질 레벨에 공통적으로 수행되는 작업 */

/* 중간 품질 수준에 특화된 작업 수행 */

/* 모든 품질 레벨에 공통적인 작업 추가 */

}

float4 HighQualityPS(VertexOut pin) : SV_Target

{

/* 모든 품질 레벨에 공통적으로 수행할 작업 */

/* 고품질 전용 작업 수행 */

/* 모든 품질 레벨에 공통적인 추가 작업 수행 */

}

technique11 ShadowsLow

{

pass P0

{

SetVertexShader(CompileShader(vs_5_0, VS()));

SetPixelShader(CompileShader(ps_5_0, LowQualityPS()));

}

}

technique11 ShadowsMedium

{

P0 통과

{

SetVertexShader(CompileShader(vs_5_0, VS())); SetPixelShader(CompileShader(ps_5_0, MediumQualityPS()));

}

}

technique11 ShadowsHigh

{

pass P0

{

SetVertexShader(CompileShader(vs_5_0, VS())); SetPixelShader(CompileShader(ps_5_0, HighQualityPS()));

}

}

그런 다음 C++ 애플리케이션 코드는 사용자의 그래픽 카드 성능을 감지하고 렌더링에 사용할 적절한 기법을 선택할 수 있습니다.

이전 코드는 세 가지 색도잉 기법 간에 픽셀 셰이더만 다르다고 가정하므로, 모든 픽셀 셰이딩 기법() 이 동일한 버텍스 셰이더를 공유합니다. 그러나 필요한 경우 기법별로 서로 다른 버텍스 셰이더를 가질 수도 있습니다.

필요하다면 각 기법에 다른 버텍스 셰이더를 사용할 수도 있습니다.

이전 구현 방식의 한 가지 성가신 문제는 픽셀 셰이더들이 색도잉 코드에서 차이가 나더라도, 여전히 모두에게 공통적으로 필요한 코드가 있어 이를 복제해야 한다는 점입니다. 조건문을 사용 하자는 제안이 있을 수 있으며, 이는 올바른 방향으로의 한 걸음입니다. 셰이더에서 동적 분기 문은 일부 오버헤드가 발생하므로 정말 필요한 경우에만 사용해야 합니다. 우리가 진정으로 원하는 것 은 컴파일 시점에 필요한 모든 셰이더 변환을 생성하는 조건부 컴파일입니다.

셰이더 코드에 분기 명령어가 전혀 없도록 하는 것입니다. 다행히도 이펙트 프레임워크는 이를 구현할 수 있는 방법을 제공합니다. 새로운 구현을 살펴보겠습니다:

```
// 상수 버퍼, 버텍스 구조체 등은 생략...

VertexOut VS(VertexIn vin) { /* 구현 세부사항 생략 */ }

#define LowQuality 0
#define MediumQuality 1
#define HighQuality 2

float4 PS(VertexOut pin, uniform int gQuality) : SV_Target
{
    /* 모든 품질 수준에 공통적으로 적용되는 작업 수행 */
    if(gQuality == LowQuality)
    {
        /* 저품질 레벨 전용 작업 수행 */
    }
    else if(gQuality == MediumQuality)
    {
        /* 중간 품질 관련 작업 수행 */
    }
    else
    {
        /* 고품질 관련 작업 수행 */
    }
    /* 모든 품질 수준에 공통적으로 적용되는 추가 작업 수행 */
}

technique11 ShadowsLow
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetPixelShader(CompileShader(ps_5_0,
        PS(LowQuality)));
    }
}

technique11 ShadowsMedium
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetPixelShader( CompileShader(ps_5_0,
        PS(MediumQuality)));
    }
}

technique11 ShadowsHigh
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS())); SetPixelShader(CompileShader(ps_5_0,
        PS(HighQuality)));
    }
}
```

픽셀 셰이더에 품질 수준을 나타내는 추가 **유니폼** 매개변수를 추가했음을 확인하십시오. 이 매개변수는 픽셀마다 달라지지 않고 대신 유니폼/상수라는 점에서 다릅니다. 또한 런타임에 변경 하지도 않습니다(상수 버퍼 변수를 변경하는 방식과 다름). 대신 컴파일 시점에 설정하며, 이 값이 컴파일 시점에 알려져 있기 때문에 이펙트 프레임워크가 해당 값에 기반해 다양한 셰이더 변형을 생성할 수 있습니다. 이를 통해 코드 중복 없이(이펙트 프레임워크가 컴파일 시점에 코드를 복제해 줌), 분기 명령어 없이도 저품질, 중간 품질, 고품질 픽셀 셰이더를 생성할 수 있습니다.

셰이더 생성이 흔히 사용되는 다른 예시 몇 가지는 다음과 같습니다:

1. 텍스처 적용? 애플리케이션은 일부 오브젝트에는 텍스처를 적용하고 다른 오브젝트에는 적용하지 않으려 할 수 있습니다. 한 가지 해결책은 텍스처를 적용하는 픽셀 셰이더와 적용하지 않는 픽셀 셰이더, 두 가지를 만드는 것입니다. 또는 셰이더 생성기 메커니즘을 사용하여 두 픽셀 셰이더를 생성한 다음 C++ 애플리케이션이 원하는 기법을 선택하도록 할 수도 있습니다.

```
float4 PS(VertexOut pin, uniform bool gApplyTexture) : SV_Target
{
    /* 공통 작업 수행 */
    if(gApplyTexture)
    {
        /* 텍스처 적용 */
    }
    /* 더 일반적인 작업 수행 */
}

technique11 BasicTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetPixelShader(CompileShader(ps_5_0, PS(false)));
    }
}

technique11 TextureTech
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS()));
        SetPixelShader(CompileShader(ps_5_0, PS(true)));
    }
}
```

2. 사용할 조명 수는? 게임 레벨은 주어진 시점에 1개에서 4개까지의 활성 조명을 지원할 수 있습니다. 사용되는 조명이 많을수록 조명 계산 비용이 증가합니다. 조명 수에 따라 별도의 버텍스 셰이더를 구현하거나, 셰이더 생성기 메커니즘을 사용하여 네 개의 버텍스 셰이더를 생성한 후 C++ 애플리케이션이 현재 활성 조명 수에 따라 원하는 기법을 선택하도록 할 수 있습니다.

```
VertexOut VS(VertexOut pin, uniform int gLightCount)
{
    /* 공통 작업 수행 */
    for(int i = 0; i < gLightCount; ++i)
    {
        /* 조명 작업 수행 */
    }
    /* 더 일반적인 작업을 수행합니다 */
}

technique11 Light1
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS(1)));
        SetPixelShader(CompileShader(ps_5_0, PS()));
    }
}

technique11 Light2
{
    pass P0
    {
        SetVertexShader(CompileShader(vs_5_0, VS(2))); SetPixelShader(
        CompileShader(ps_5_0, PS()));
    }
}
```

```

    }

    technique11 Light3
    {
pass P0
{
    SetVertexShader(CompileShader(vs_5_0, VS(3)));
    SetPixelShader(CompileShader(ps_5_0, PS()));
}
    }

    technique11 Light4
    {
pass P0
{
    SetVertexShader(CompileShader(vs_5_0, VS(4)));
    SetPixelShader(CompileShader(ps_5_0, PS()));
}
    }

```

한 가지 매개변수로만 제한할 필요는 없습니다. 세도우 품질, 텍스처, 조명 수를 모두 결합해야 하므로 버텍스 셰이더와 픽셀 셰이더는 다음과 같이 보일 것입니다:

```

VertexOut VS(VertexOut pin, uniform int gLightCount)
{

float4 PS(VertexOut pin, uniform int
gQuality,
uniform bool gApplyTexture) : SV_Target
{

```

예를 들어, 저품질 그림자, 두 개의 조명, 텍스처 없음 기술을 생성하려면 다음과 같이 작성합니다:

```

    technique11 LowShadowsTwoLightsNoTextures
    {
pass P0
{
    SetVertexShader(CompileShader(vs_5_0, VS(2))); SetPixelShader(CompileShader(ps_5_0,
PS(LowQuality, false)));
}
    }

```

6.8.7 어셈블리 코드의 모습

효과 파일의 어셈블리 출력을 확인해 본 적이 없다면, 이 섹션에서 그 모습을 보여드립니다. 어셈블리 자체는 설명하지 않지만, 어셈블리를 공부해 본 적이 있다면 **mov** 명령어를 알아볼 수 있고, **dp4가** 4차원 내적을 수행한다는 것을 추측할 수도 있을 것입니다. 어셈블리를 이해하지 못하더라도, 이 리스팅은 유용한 정보를 제공합니다. 입력 및 출력 시그니처를 명확히 식별하고, 명령어 수를 대략적으로 알려주는데, 이는 셰이더의 비용/복잡성을 이해하는 데 유용한 지표 중 하나입니다. 또한 컴파일 시 매개변수에 따라 분기 명령어 없이 여러 버전의 셰이더가 실제로 생성되었음을 확인할 수 있습니다. 사용한 이펙트 파일은 §6.8.1에 표시된 것과 동일하지만, 두 가지 기법을 생성하기 위해 간단한 uniform **bool** 매개변수를 추가했습니다:

```

float4 PS(VertexOut pin, uniform bool gUseColor) : SV_Target
{
if(gUseColor)
{
    return pin.Color;
}
else
{
    float4(0, 0, 0, 1);
}
}

```



```

        technique11 ColorTech
        {
pass P0
{
    SetVertexShader(CompileShader(vs_5_0, VS()));
    SetPixelShader(CompileShader(ps_5_0, PS(true)));
}

        }

        technique11 NoColorTech
        {
pass P0
{
    SetVertexShader(CompileShader(vs_5_0, VS()));
    SetPixelShader(CompileShader(ps_5_0, PS(false)));
}

        }

        //
        // FX 버전: fx_5_0
        //
        // 1개의 로컬 버퍼
        //
        cbuffer cbPerObject
        {
float4x4 gWorldViewProj; // 오프셋: 0, 크기: 64
        }

        //
        // 1 그룹
        // fxgroup
        {
//
// 2 기술(들)
//
technique11 ColorTech
{
    패스 P0
    {
        VertexShader = asm {
            //
            // Microsoft (R) HLSL 셰이더 컴파일러 9.29.952.3111에 의해 생성됨
            //
            //
            // 버퍼 정의:
            //
            // cbuffer cbPerObject
            // {
            //
            // float4x4 gWorldViewProj; // 오프셋: 0 크기: 64
            //
            // }
            //
            //
            // 리소스 바인딩:
            //
            // 이름 유형 형식 크기 슬롯 요소
            // -----
            // cbPerObject cbuffer NA NA 0 1
            //

```

```

//
//
// 입력 시그니처:
//
// 이름 인덱스 마스크 레지스터 시스템값 형식 사용
// -----
// 위치 0 xyz 0 없음 부동소수점 xyz
// COLOR 0 xyzw 1 NONE float xyzw
//
//
// 출력 시그니처:
//
// 이름 인덱스 마스크 레지스터 시스템값 형식 사용
// -----
// SV_POSITION 0 xyzw 0 POS float xyzw
// COLOR 0 xyzw 1 NONE float xyzw
// vs_5_0
dcl_globalFlags refactoringAllowed dcl_constantbuffer cb0[4],
immediateIndexed dcl_input v0.xyz
dcl_input v1.xyzw dcl_output_siv o0.xyzw,
position dcl_output o1.xyzw

dcl_temps 1
mov r0.xyz, v0.xyzx mov r0.w,
l(1.000000)
dp4 o0.x, r0.xyzw, cb0[0].xyzw dp4 o0.y,
r0.xyzw, cb0[1].xyzw dp4 o0.z, r0.xyzw,
cb0[2].xyzw dp4 o0.w, r0.xyzw, cb0[3].xyzw
mov o1.xyzw, v1.xyzw

ret

// 약 8개의 명령어 슬롯 사용
};
PixelShader = asm {
//
// Microsoft (R) HLSL 셰이더 컴파일러 9.29.952.3111로 생성됨
//
//
//
//
// 입력 시그니처:
//
// 이름 인덱스 마스크 레지스터 시스템값 형식 사용
// -----
// SV_POSITION 0 xyzw 0 POS float
// COLOR 0 xyzw 1 NONE float xyzw
//
//
// 출력 시그니처:
//
// 이름 인덱스 마스크 레지스터 시스템값 형식 사용
// -----
// SV_Target 0 xyzw 0 TARGET float xyzw
// ps_5_0
dcl_globalFlags refactoringAllowed dcl_input_ps
linear v1.xyzw dcl_output o0.xyzw
mov o0.xyzw, v1.xyzw ret

```

```

        // 약 2개의 명령어 슬롯 사용됨
    };
}
}
technique11 NoColorTech
{
    pass P0
    {
        VertexShader = asm {
            //
            // Microsoft (R) HLSL 셰이더 컴파일러 9.29.952.3111에 의해 생성됨
            //
            //
            // 버퍼 정의:
            //
            // cbuffer cbPerObject
            // {
            //
            // float4x4 gWorldViewProj; // 오프셋: 0 크기: 64
            //
            // }
            //
            //
            // 리소스 바인딩:
            //
            // 이름 유형 형식 크기 슬롯 요소
            // -----
            // cbPerObject cbuffer NA NA 0 1
            //
            //
            //
            // 입력 시그니처:
            //
            // 이름 인덱스 마스크 레지스터 시스템값 형식 사용
            // -----
            // 위치 0 xyz 0 없음 부동소수점 xyz
            // 색상 0 xyzw 1 없음 부동소수점 xyzw
            //
            //
            // 출력 시그니처:
            //
            // 이름 인덱스 마스크 레지스터 시스템값 형식 사용
            // -----
            // SV_POSITION 0 xyzw 0 POS float xyzw
            // COLOR 0 xyzw 1 NONE float xyzw
            // vs_5_0
            dcl_globalFlags refactoringAllowed dcl_constantbuffer cb0[4],
            immediateIndexed dcl_input v0.xyz
            dcl_input v1.xyzw dcl_output_siv o0.xyzw,
            position dcl_output o1.xyzw

            dcl_temps 1
            mov r0.xyz, v0.xyzx mov r0.w,
            l(1.000000)
            dp4 o0.x, r0.xyzw, cb0[0].xyzw dp4 o0.y,
            r0.xyzw, cb0[1].xyzw dp4 o0.z, r0.xyzw,
            cb0[2].xyzw dp4 o0.w, r0.xyzw, cb0[3].xyzw
            mov o1.xyzw, v1.xyzw
        }
    }
}

```

```

        ret
        // 약 8개의 명령어 슬롯 사용
    };
    PixelShader = asm {
        //
        // Microsoft (R) HLSL 셰이더 컴파일러 9.29.952.3111에 의해 생성됨
        //
        //
        //
        // 입력 시그니처:
        //
        // 이름 인덱스 마스크 레지스터 시스템값 형식 사용
        // -----
        // SV_POSITION 0 xyzw 0 POS float
        // COLOR 0 xyzw 1 NONE float
        //
        //
        // 출력 시그니처:
        //
        // 이름 인덱스 마스크 레지스터 시스템값 형식 사용
        // -----
        // SV_Target 0 xyzw 0 TARGET float xyzw
        // ps_5_0
        dcl_globalFlags refactoringAllowed dcl_output
        o0.xyzw
        mov o0.xyzw, l(0,0,0,1.000000) ret
        // 약 2개의 명령어 슬롯 사용
    };
}
}
}

```

6.9 박스 데모

마침내, 우리는 색상 상자를 렌더링하는 간단한 데모를 보여줄 만큼 충분한 내용을 다루었습니다. 이 예제는 본 장에서 지금까지 논의한 모든 내용을 하나의 프로그램에 통합한 것입니다. 독자는 모든 줄을 이해할 때까지 코드를 연구하고 본 장의 이전 섹션을 참조해야 합니다. 이 프로그램은 §6.8.1에 기술된 대로 "color.fx" 효과를 사용한다는 점에 유의하십시오.

```

//*****
// BoxDemo.cpp by Frank Luna (C) 2011 All Rights Reserved.
//
// 색상 박스 렌더링을 보여줍니다.
//
// 조작법:
//
// 마우스 왼쪽 버튼을 누른 상태에서 마우스를 움직여 회전합니다.
//
// 마우스 오른쪽 버튼을 누른 상태로 확대/축소합니다.
//
//*****

#include "d3dApp.h" #include
"d3dx11Effect.h" #include
"MathHelper.h"

struct Vertex
{
    XMFLOAT3 Pos; XMFLOAT4
    Color;

```

```

};

class BoxApp : public D3DApp
{
public:
    BoxApp(HINSTANCE hInstance);
    ~BoxApp();

    bool Init();
    void OnResize();
    void UpdateScene(float dt); void
    DrawScene();

    void OnMouseDown(WPARAM btnState, int x, int y); void
    OnMouseUp(WPARAM btnState, int x, int y); void OnMouseMove(WPARAM
    btnState, int x, int y);

private:
    void BuildGeometryBuffers(); void BuildFX();
    void BuildVertexLayout();

private:
    ID3D11Buffer* mBoxVB; ID3D11Buffer*
    mBoxIB;

    ID3DX11Effect* mFX; ID3DX11EffectTechnique* mTech;
    ID3DX11EffectMatrixVariable* mfxWorldViewProj; ID3D11InputLayout* mInputLayout;

    XMFLOAT4X4 mWorld; XMFLOAT4X4
    mView; XMFLOAT4X4 mProj;

    float mTheta; float
    mPhi; float mRadius;

    POINT mLastMousePos;
};

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE prevInstance, PSTR cmdLine, int showCmd)
{
    // 디버그 빌드 시 런타임 메모리 검사 활성화.
#ifdef _DEBUG
    _CrtSetDbgFlag(_CRTDBG_ALLOC_MEM_DF | _CRTDBG_LEAK_CHECK_DF); #endif

    BoxApp theApp(hInstance); if(!theApp.Init())
        return 0;

    return theApp.Run();
}

BoxApp::BoxApp(HINSTANCE hInstance)
: D3DApp(hInstance), mBoxVB(0), mBoxIB(0), mFX(0), mTech(0), mfxWorldViewProj(0), mInputLayout(0),
  mTheta(1.5f*MathHelper::Pi), mPhi(0.25f*MathHelper::Pi), mRadius(5.0f)
{

```

```

        mMainWndCaption = L"Box 데모";

        mLastMousePos.x = 0;
        mLastMousePos.y = 0;

        XMATRIX I = XMMatrixIdentity();
        XMStoreFloat4x4(&mWorld, I);
        XMStoreFloat4x4(&mView, I);
        XMStoreFloat4x4(&mProj, I);
    }

BoxApp::~BoxApp()
{
    ReleaseCOM(mBoxVB); ReleaseCOM(mBoxIB);
    ReleaseCOM(mFX); ReleaseCOM(mInputLayout);
}

bool BoxApp::Init()
{
    if(!D3DApp::Init()) return false;

    BuildGeometryBuffers(); BuildFX();
    BuildVertexLayout();

    return true;
}

void BoxApp::OnResize()
{
    D3DApp::OnResize();

    // 창 크기가 변경되었으므로 종횡비 업데이트 및
    // 투영 행렬을 재계산합니다.
    XMATRIX P = XMMatrixPerspectiveFovLH(0.25f*MathHelper::Pi, AspectRatio(), 1.0f, 1000.0f);
    XMStoreFloat4x4(&mProj, P);
}

void BoxApp::UpdateScene(float dt)
{
    // 구좌표를 직교좌표로 변환. float x =
    mRadius*sinf(mPhi)*cosf(mTheta); float z =
    mRadius*sinf(mPhi)*sinf(mTheta); float y = mRadius*cosf(mPhi);

    // 뷰 매트릭스 생성.
    XMVECTOR pos = XMVectorSet(x, y, z, 1.0f); XMVECTOR target =
    XMVectorZero();
    XMVECTOR up = XMVectorSet(0.0f, 1.0f, 0.0f, 0.0f);

    XMATRIX V = XMMatrixLook AtLH(pos, target, up); XMStoreFloat4x4(&mView, V);
}

void BoxApp::DrawScene()
{
    md3dImmediateContext->ClearRenderTargetView(mRenderTargetView, reinterpret_cast<const float*>(&Colors::Blue));

```

```

md3dImmediateContext->ClearDepthStencilView(mDepthStencilView, D3D11_CLEAR_DEPTH|D3D11_CLEAR_STENCIL, 1.0f, 0);

md3dImmediateContext->IASetInputLayout(mInputLayout); md3dImmediateContext-
>IASetPrimitiveTopology(
    D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);
UINT stride = sizeof(Vertex); UINT offset = 0;
md3dImmediateContext->IASetVertexBuffers(0, 1, &mBoxVB, &stride, &offset);
md3dImmediateContext->IASetIndexBuffer(mBoxIB, DXGI_FORMAT_R32_UINT, 0);

// 상수 설정
XMMATRIX world = XMLoadFloat4x4(&mWorld); XMMATRIX view =
XMLoadFloat4x4(&mView); XMMATRIX proj = XMLoadFloat4x4(&mProj);
XMMATRIX worldViewProj = world*view*proj;

mfxWorldViewProj->SetMatrix(reinterpret_cast<float*>(&worldViewProj)); D3DX11_TECHNIQUE_DESC techDesc;
mTech->GetDesc(&techDesc);
for(UINT p = 0; p < techDesc.Passes; ++p)
{
    mTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext);

    // 박스용 36개 인덱스.
    md3dImmediateContext->DrawIndexed(36, 0, 0);
}

HR(mSwapChain->Present(0, 0));
}

void BoxApp::OnMouseDown(WPARAM btnState, int x, int y)
{
    mLastMousePos.x = x;
    mLastMousePos.y = y;

    SetCapture(mhMainWnd);
}

void BoxApp::OnMouseUp(WPARAM btnState, int x, int y)
{
    ReleaseCapture();
}

void BoxApp::OnMouseMove(WPARAM btnState, int x, int y)
{
    if( (btnState & MK_LBUTTON) != 0 )
    {
        // 각 픽셀을 1/4도 단위로 변환 float dx = XMConvertToRadians(
            0.25f*static_cast<float>(x - mLastMousePos.x)); float dy =
        XMConvertToRadians(
            0.25f*static_cast<float>(y - mLastMousePos.y));

        // 입력값에 따라 각도를 업데이트하여 카메라를 상자 주위로 회전시킵니다. mTheta += dx;
        mPhi += dy;
        // 각도 mPhi를 제한합니다.
        mPhi = MathHelper::Clamp(mPhi, 0.1f, MathHelper::Pi - 0.1f);
    }
}

```

```

else if( btnState & MK_RBUTTON) != 0 )
{
    // 각 픽셀을 장면 내 0.005 단위로 대응시킵니다. float dx = 0.005f*static_cast<float>(x -
    mLastMousePos.x); float dy = 0.005f*static_cast<float>(y - mLastMousePos.y);

    // 입력값에 따라 카메라 반경을 업데이트합니다. mRadius += dx - dy;

    // 반경 제한 적용.
    mRadius = MathHelper::Clamp(mRadius, 3.0f, 15.0f);
}

mLastMousePos.x = x;
mLastMousePos.y = y;
}

```

```

void BoxApp::BuildGeometryBuffers()
{

    // 버텍스 버퍼 생성 Vertex
    vertices[] =
    {
        { XMFLOAT3(-1.0f, -1.0f, -1.0f), (const float*)&Colors::White },
        { XMFLOAT3(-1.0f, +1.0f, -1.0f), (const float*)&Colors::Black },
        { XMFLOAT3(+1.0f, +1.0f, -1.0f), (const float*)&Colors::Red },
        { XMFLOAT3(+1.0f, -1.0f, -1.0f), (const float*)&Colors::Green },
        { XMFLOAT3(-1.0f, -1.0f, +1.0f), (const float*)&Colors::Blue },
        { XMFLOAT3(-1.0f, +1.0f, +1.0f), (const float*)&Colors::Yellow },
        { XMFLOAT3(+1.0f, +1.0f, +1.0f), (const float*)&Colors::Cyan },
        { XMFLOAT3(+1.0f, -1.0f, +1.0f), (const float*)&Colors::Magenta }
    };

    D3D11_BUFFER_DESC vbd;
    vbd.Usage = D3D11_USAGE_IMMUTABLE;
    vbd.ByteWidth = sizeof(Vertex) * 8; vbd.BindFlags =
    D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = 0;

    vbd.MiscFlags = 0;
    vbd.StructureByteStride = 0; D3D11_SUBRESOURCE_DATA
    vinitData; vinitData.pSysMem = vertices;

    HR(md3dDevice->CreateBuffer(&vbd, &vinitData, &mBoxVB));

    // 인덱스 버퍼 생성 UINT indices[] = {

        // 정면 면 0, 1, 2,

        0, 2, 3,

        // 뒷면 4, 6, 5,

        4, 7, 6,

        // 왼쪽 면 4, 5,

        1,

        4, 1, 0,

        // 오른쪽 면 3, 2,

        6,

        3, 6, 7,

        // 상면

```



```

1, 5, 6,
1, 6, 2,

// 바닥면 4, 0, 3,
4, 3, 7
};

D3D11_BUFFER_DESC ibd;
ibd.Usage = D3D11_USAGE_IMMUTABLE;
ibd.ByteWidth = sizeof(UINT) * 36; ibd.BindFlags =
D3D11_BIND_INDEX_BUFFER; ibd.CPUAccessFlags = 0;
ibd.MiscFlags = 0;
ibd.StructureByteStride = 0; D3D11_SUBRESOURCE_DATA
iinitData; iinitData.pSysMem = indices;
HR(md3dDevice->CreateBuffer(&ibd, &iinitData, &mBoxIB));
}

void BoxApp::BuildFX()
{
    DWORD shaderFlags = 0;
    #if defined(DEBUG) || defined(_DEBUG) shaderFlags |=
    D3D10_SHADER_DEBUG;
    shaderFlags |= D3D10_SHADER_SKIP_OPTIMIZATION; #endif

    ID3D10Blob* compiledShader = 0; ID3D10Blob*
    compilationMsgs = 0;
    HRESULT hr = D3DX11CompileFromFile(L"FX/color.fx", 0, 0, 0, "fx_5_0", shaderFlags,
        0, 0, &compiledShader, &compilationMsgs, 0);

    // compilationMsgs에는 오류나 경고가 저장될 수 있습니다. if(compilationMsgs != 0)
    {
        MessageBoxA(0, (char*)compilationMsgs->GetBufferPointer(), 0, 0); ReleaseCOM(compilationMsgs);
    }

    // compilationMsgs가 없더라도 다른 오류가 없는지 확인합니다.
    // 다른 오류가 없는지 확인합니다. if(FAILED(hr))
    {
        DXTrace( FILE , (DWORD) LINE , hr, L"D3DX11CompileFromFile", true);
    }

    HR(D3DX11CreateEffectFromMemory(    compiledShader-
        >GetBufferPointer(), compiledShader->GetBufferSize(),
        0, md3dDevice, &mFX));

    // 컴파일된 셰이더 작업 완료. ReleaseCOM(compiledShader);

    mTech = mFX->GetTechniqueByName("ColorTech"); mfxWorldViewProj = mFX-
    >GetVariableByName(
        "gWorldViewProj")->AsMatrix();
}

void BoxApp::BuildVertexLayout()

```

```

{
    // 정점 입력 레이아웃 생성. D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
    {
        {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
         D3D11_INPUT_PER_VERTEX_DATA, 0},
        {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, 12,
         D3D11_INPUT_PER_VERTEX_DATA, 0}
    };

    // 입력 레이아웃 생성 D3DX11_PASS_DESC
    passDesc;
    mTech->GetPassByIndex(0)->GetDesc(&passDesc); HR(md3dDevice-
    >CreateInputLayout(vertexDesc, 2,
        passDesc.pIAInputSignature, passDesc.IAInputSignatureSize, &mInputLayout));
}

```

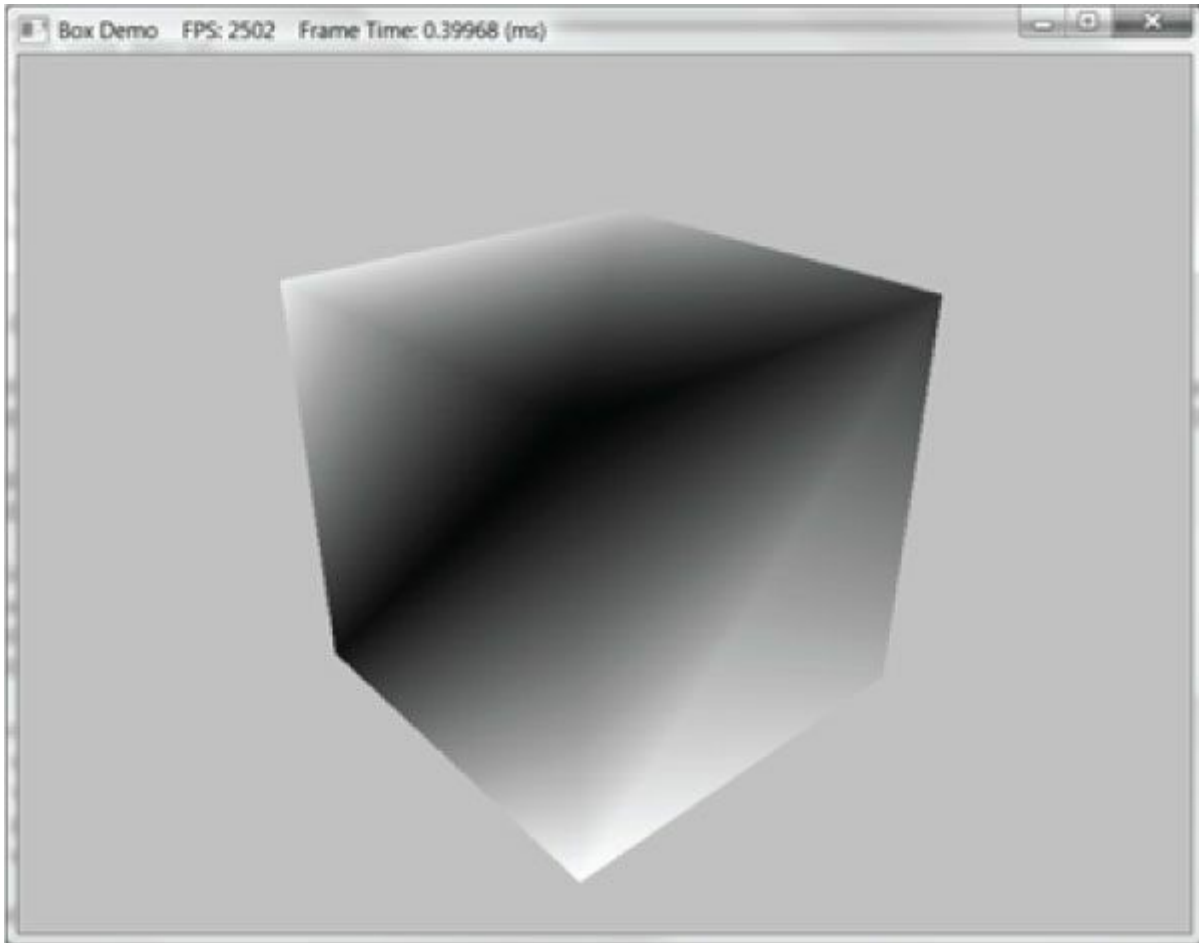


그림 6.7. "Box" 데모의 스크린샷.

6.10 HILLS 데모

이 장에는 "언덕" 데모도 포함되어 있습니다. 이 데모는 박스 데모와 동일한 Direct3D 메서드를 사용하지만, 더 복잡한 지오메트리를 그립니다. 특히, 절차적으로 삼각형 그리드 메시를 구성하는 방법을 보여줍니다. 이러한 지오메트리는 지형 및 물 렌더링 등에 특히 유용합니다.

실수 값을 갖는 "좋은" 함수 $y = f(x, z)$ 의 그래프는 표면이다. xz 평면에 격자를 구성하여 표면을 근사할 수 있다.

xz 평면에 격자를 구성함으로써 표면을 근사할 수 있다. 여기서 각 사각형은 두 개의 삼각형으로 이루어지며, 이후 각 격자점에 함수를 적용한다; [그림 6.8](#)을 참조하라.

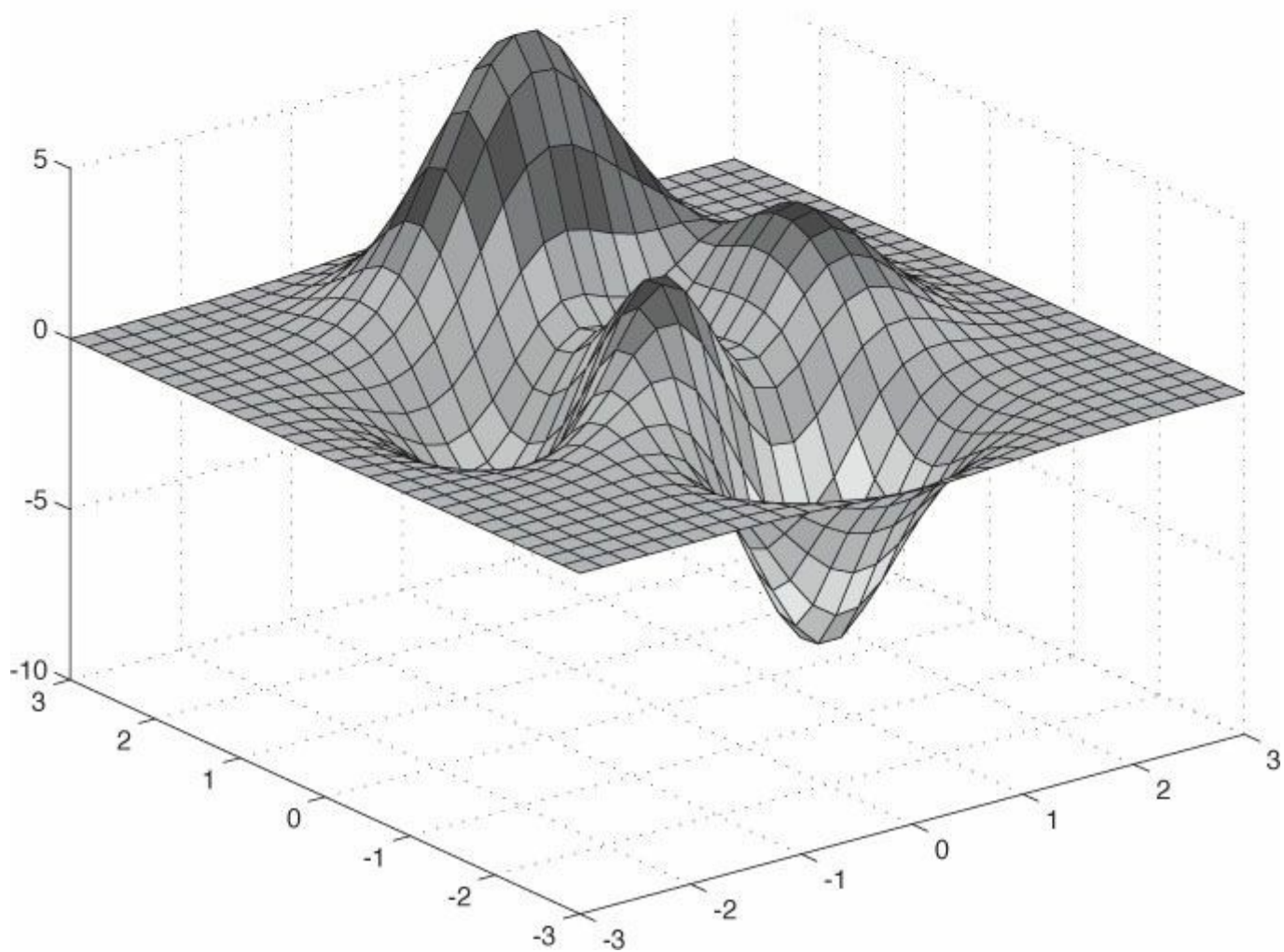
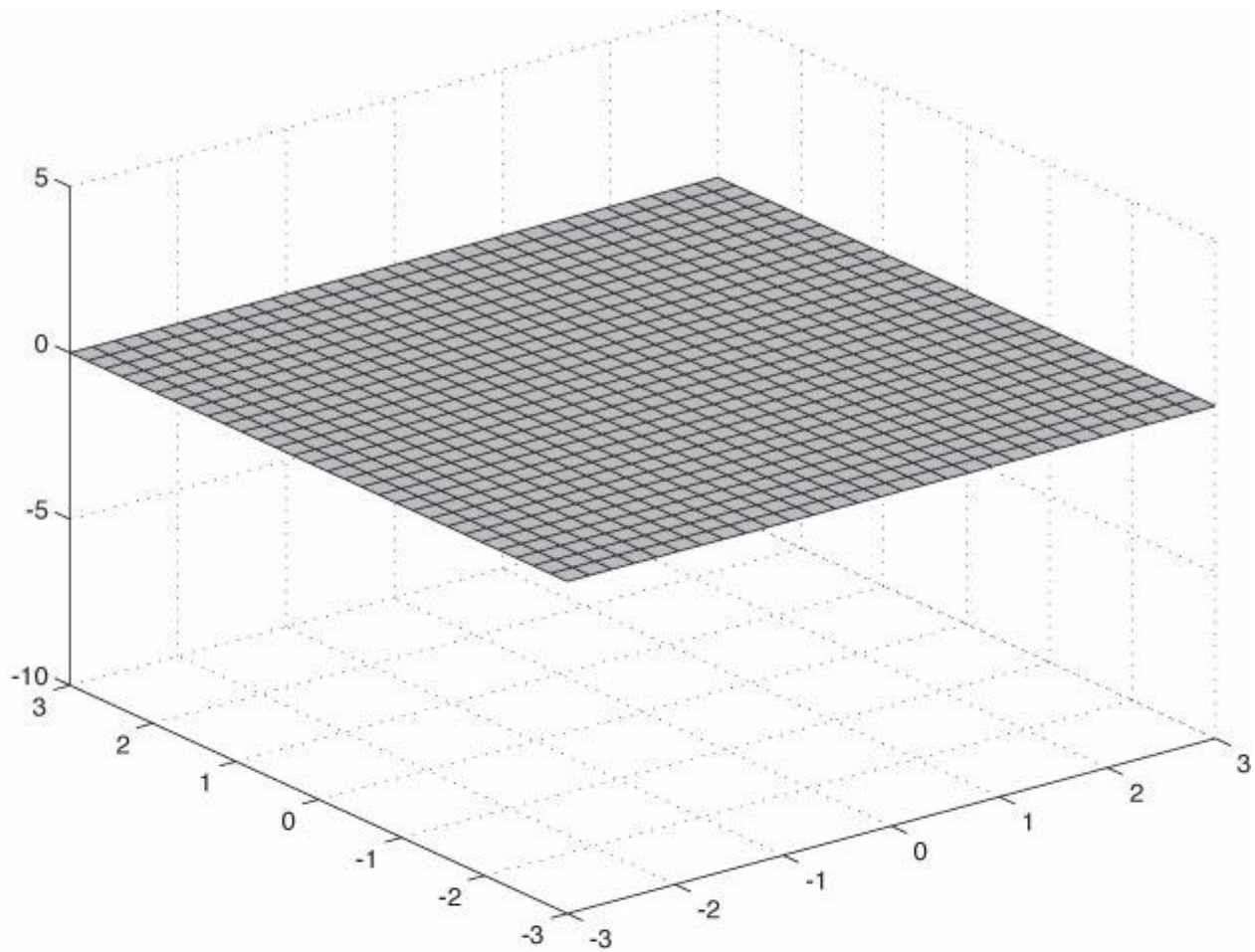


그림 6.8. (위) xz 평면에 격자를 배치한다. (아래) 각 격자점에 함수 $f(x, z)$ 를 적용하여 y 좌표를 구한다. 점 $(x, f(x, z), z)$ 의 집합을 플롯하면 표면의 그래프를 얻을 수 있다.

6.10.1 격자 정점 생성

따라서 주요 과제는 xz 평면에 격자를 구축하는 방법이다. $m \times n$ 개의 정점으로 구성된 격자는 [그림 6.9](#)와 같이 $(m - 1) \times (n - 1)$ 개의 사각형(또는 셀)을 유도한다. 각 셀은 두 개의 삼각형으로 덮이므로 총 $2 \cdot (m - 1) \times (n - 1)$ 개의 삼각형이 존재한다. 격자의 너비가 w 이고 *깊이/가라면*, x축 방향의 셀 간격은 $dx = w / (n - 1)$ 이고 z축 방향의 셀 간격은 $dz = d / (m - 1)$ 입니다. 정점을 생성하기 위해 왼쪽 상단 모서리에서 시작하여 행별로 정점 좌표를 점진적으로 계산합니다. xz 평면에서 ij번째 격자 정점의 좌표는 다음과 같이 주어집니다:

$$\mathbf{v}_{ij} = [-0.5w + j \cdot \mathbf{dx}, 0.0, 0.5d - i \cdot \mathbf{dz}]$$

다음 코드는 격자 정점을 생성합니다:

```
void GeometryGenerator::CreateGrid(float width, float depth, UINT m, UINT n, MeshData&
    meshData)
{
    UINT vertexCount = m*n;
    UINT faceCount = (m-1)*(n-1)*2;

    //
    // 정점 생성.
    //

    float halfWidth = 0.5f * width; float halfDepth =
    0.5f * depth;

    float dx = width / (n-1); float dz =
    depth / (m-1);

    float du = 1.0f / (n-1); float dv =
    1.0f / (m-1);

    meshData.Vertices.resize(vertexCount); for(UINT i = 0; i < m;
    ++i)
    {
        float z = halfDepth - i*dz; for(UINT j =
        0; j < n; ++j)
        {
            float x = -halfWidth + j*dx; meshData.Vertices[i*n+j].Position = XMFLOAT3(x, 0.0f, z);

            // 현재는 무시, 라이팅에 사용됨. meshData.Vertices[i*n+j].Normal = XMFLOAT3(0.0f, 1.0f, 0.0f);
            meshData.Vertices[i*n+j].TangentU = XMFLOAT3(1.0f, 0.0f, 0.0f);

            // 텍스처링에 사용되므로 지금은 무시. meshData.Vertices[i*n+j].TexC.x = j*du;
            meshData.Vertices[i*n+j].TexC.y = i*dv;
        }
    }
}
```

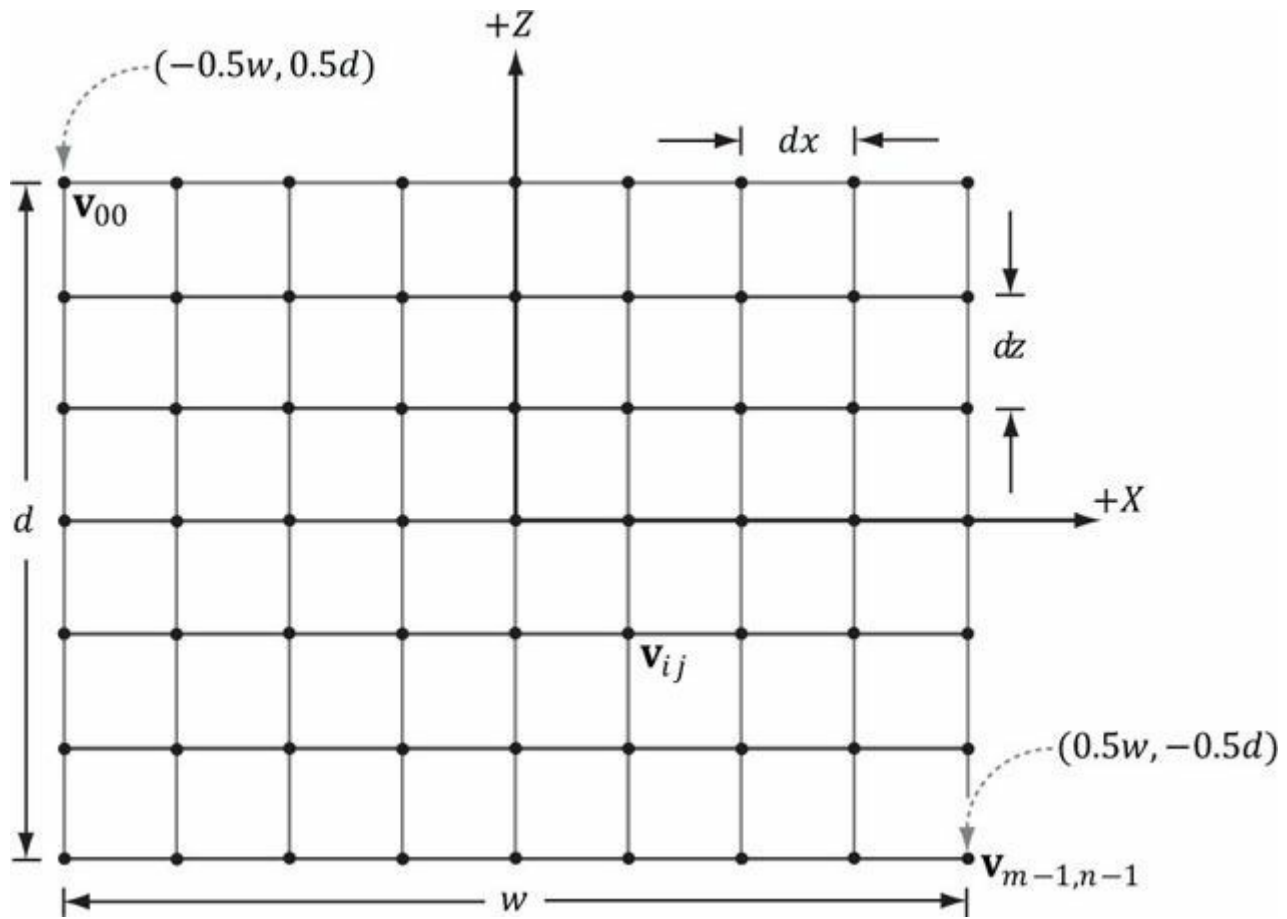


그림 6.9. 그리드 구성.

GeometryGenerator는 그리드, 구체, 원통, 상자 같은 간단한 기하학적 도형을 생성하는 유틸리티 클래스로, 이 책 전체에 걸쳐 데모 프로그램에 사용됩니다. 이 클래스는 시스템 메모리에 데이터를 생성하며, 이후 필요한 데이터를 정점 버퍼와 인덱스 버퍼로 복사해야 합니다. **GeometryGenerator**는 후속 장에서 사용될 일부 정점 데이터를 생성합니다. 현재 데모에서는 이 데이터가 필요하지 않으므로 버텍스 버퍼로 복사하지 않습니다. **MeshData** 구조체는 **GeometryGenerator** 내에 중첩된 간단한 구조체로, 버텍스와 인덱스 목록을 저장합니다:

```
class GeometryGenerator
{
public:
    struct Vertex
    {
        Vertex(){}
        Vertex(const XMFLOAT3& p,
               const XMFLOAT3& n, const
               XMFLOAT3& t, const XMFLOAT2&
               uv)
            : Position(p), Normal(n), TangentU(t), TexC(uv){} Vertex(
            float px, float py, float pz, float nx, float
            ny, float nz, float tx, float ty, float tz,
            float u, float v)
            : 위치(px,py,pz), 법선(nx,ny,nz), 접선U(tx, ty, tz), 텍스처 좌표
              (u,v){}

        XMFLOAT3 위치; XMFLOAT3 법선;
        XMFLOAT3 접선U; XMFLOAT2 텍스처 좌표;
    };

    struct MeshData
    {
        std::vector<Vertex> Vertices;
```

```
std::vector<UINT> 인덱스;
```

```
};
```

```
...
```

```
};
```

6.10.2 그리드 인덱스 생성

정점 계산을 마친 후, 인덱스를 지정하여 그리드 삼각형을 정의해야 합니다. 이를 위해 각 사각형을 왼쪽 위부터 행별로 순차적으로 반복하며, 해당 사각형의 두 삼각형을 정의하는 인덱스를 계산합니다. [그림 6.10](#)을 참조하면, $m \times n$ 정점 그리드에서 두 삼각형의 선형 배열 인덱스는 다음과 같이 계산됩니다:

$$\begin{aligned} \triangle ABC &= (i \cdot n + j, & i \cdot n + j + 1, & (i + 1) \cdot n + j) \\ \triangle CBD &= ((i + 1) \cdot n + j, & i \cdot n + j + 1, & (i + 1) \cdot n + j + 1) \end{aligned}$$

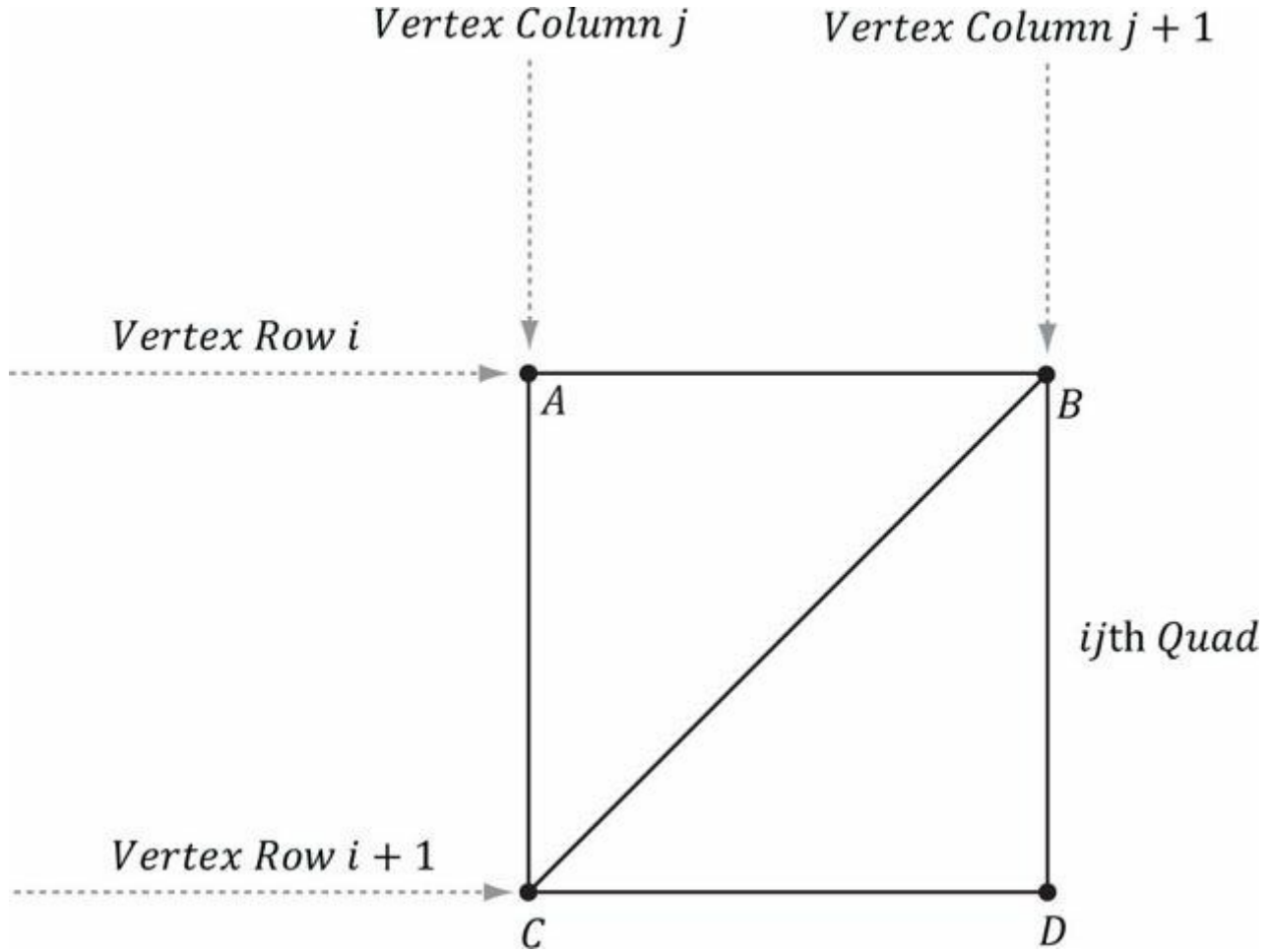


그림 6.10. ij 번째 사각형의 꼭짓점 지수.

해당 코드:

```
meshData.Indices.resize(faceCount*3); // 면당 3개의 인덱스
```

```
// 각 사변형을 순회하며 인덱스 계산. UINT k = 0;
```

```
for(UINT i = 0; i < m-1; ++i)
```

```
{
```

```
    for(UINT j = 0; j < n-1; ++j)
```

```
    {
```

```
        meshData.Indices[k] = i*n+j; meshData.Indices[k+1] =
```

```
        i*n+j+1; meshData.Indices[k+2] = (i+1)*n+j;
```

```
        meshData.Indices[k+3] = (i+1)*n+j+1;
```

```
        meshData.Indices[k+4] = i*n+j+1; meshData.Indices[k+5]
```

```
        = (i+1)*n+j+1; k += 6; // 다음 사각형
```

```
    }  
}  
}
```

6.10.3 높이 함수 적용

그리드를 생성한 후, MeshData 그리드에서 원하는 정점 요소를 추출하고 평평한 그리드를 언덕을 나타내는 표면으로 변환하며, 각 정점의 고도(y-좌표)에 기반하여 색상을 생성할 수 있습니다.

```
// GeometryGenerator.Vertex와 혼동하지 마십시오. struct Vertex  
{  
    XMFLOAT3 Pos; XMFLOAT4  
    Color;  
};  
void HillsApp::BuildGeometryBuffers()  
{  
  
    GeometryGenerator::MeshData grid; GeometryGenerator geoGen;  
    geoGen.CreateGrid(160.0f, 160.0f, 50, 50, grid); mGridIndexCount =  
    grid.Indices.size();  
  
    //  
    // 관심 있는 정점 요소 추출 및  
    // 높이 함수를 각 정점에 적용합니다. 또한 정점 색상을 지정합니다.  
    // 높이에 따라 모래사장이 펼쳐진 해변, 풀이 무성한 낮은 언덕,  
    // 언덕, 그리고 눈 덮인 산봉우리가 있습니다.  
    //  
  
    std::vector<Vertex> vertices(grid.Vertices.size()); for(size_t i = 0; i <  
    grid.Vertices.size(); ++i)  
    {  
        XMFLOAT3 p = grid.Vertices[i].Position;  
  
        p.y = GetHeight(p.x, p.z); vertices[i].Pos =  
        p;  
  
        // 높이에 따라 정점 색상 지정. if( p.y < -10.0f )  
        {  
            // 모래사장 색상.  
            vertices[i].Color = XMFLOAT4(1.0f, 0.96f, 0.62f, 1.0f);  
        }  
        else if(p.y < 5.0f)  
        {  
            // 연한 황록색.  
            vertices[i].Color = XMFLOAT4(0.48f, 0.77f, 0.46f, 1.0f);  
        }  
        else if(p.y < 12.0f)  
        {  
            // 짙은 황록색.  
            vertices[i].Color = XMFLOAT4(0.1f, 0.48f, 0.19f, 1.0f);  
        }  
        else if(p.y < 20.0f)  
        {  
            // 진한 갈색.  
            vertices[i].Color = XMFLOAT4(0.45f, 0.39f, 0.34f, 1.0f);  
        }  
    }  
}
```

```

else
{
    // 하얀 눈.
    vertices[i].Color = XMFLLOAT4(1.0f, 1.0f, 1.0f, 1.0f);
}
}
D3D11_BUFFER_DESC vbd;
vbd.Usage = D3D11_USAGE_IMMUTABLE;
vbd.ByteWidth = sizeof(Vertex) * grid.Vertices.size(); vbd.BindFlags =
D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = 0;
vbd.MiscFlags = 0; D3D11_SUBRESOURCE_DATA vinitData;
vinitData.pSysMem = &vertices[0];
HR(md3dDevice->CreateBuffer(&vbd, &vinitData, &mVB));

//
// 모든 메시의 인덱스를 하나의 인덱스 버퍼에 압축합니다.
//

D3D11_BUFFER_DESC ibd;
ibd.Usage = D3D11_USAGE_IMMUTABLE;
ibd.ByteWidth = sizeof(UINT) * mGridIndexCount; ibd.BindFlags =
D3D11_BIND_INDEX_BUFFER; ibd.CPUAccessFlags = 0;
ibd.MiscFlags = 0; D3D11_SUBRESOURCE_DATA iinitData;
iinitData.pSysMem = &grid.Indices[0];
HR(md3dDevice->CreateBuffer(&ibd, &iinitData, &mIB));
}

```

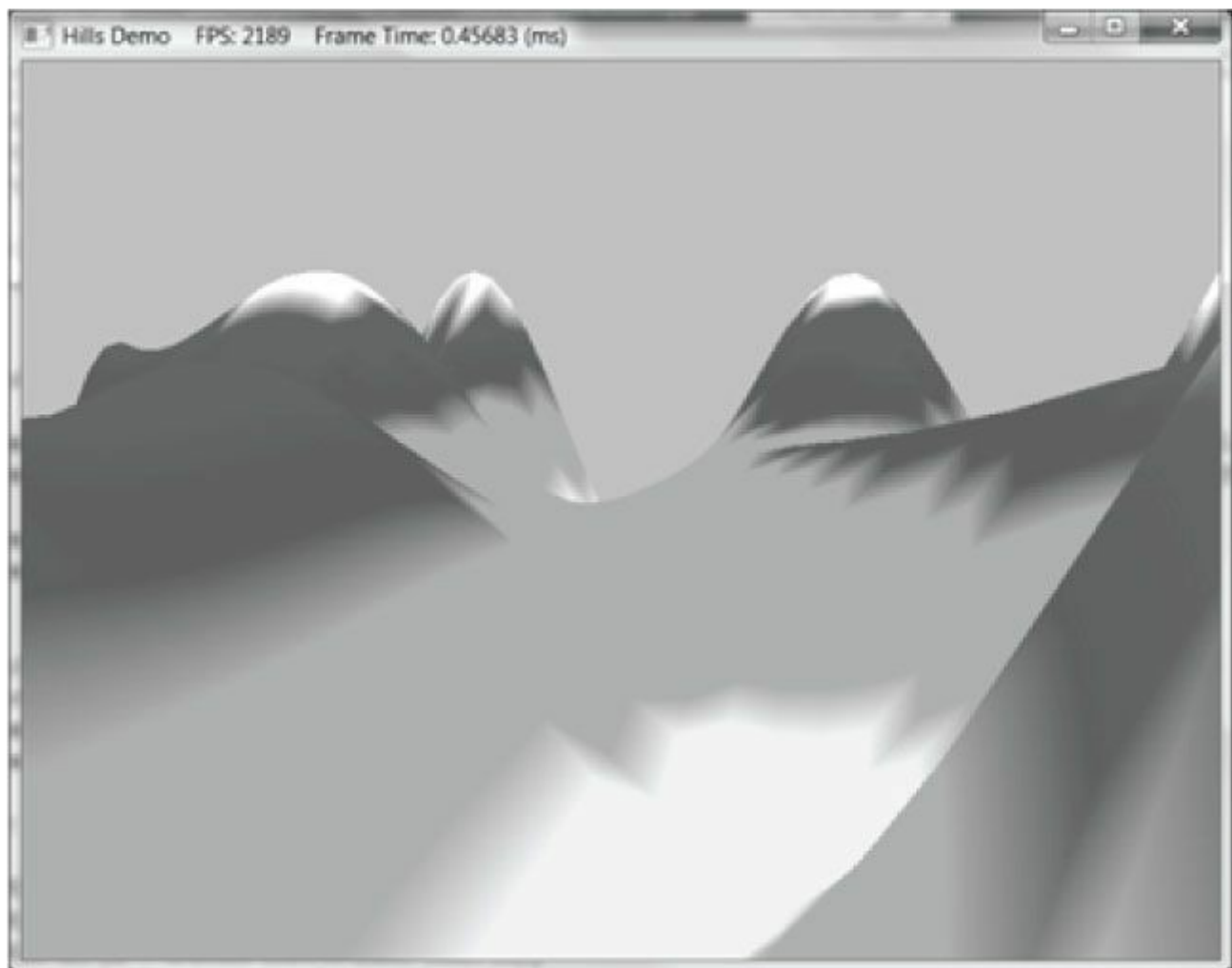


그림 6.11. "Hills Demo" 스크린샷.

이 데모에서 사용한 함수 $f(x, z)$ 는 다음과 같이 정의됩니다:

```
float HillsApp::GetHeight(float x, float z)const
{
    return 0.3f*(z*sinf(0.1f*x) + x*cosf(0.1f*z));
}
```

이 함수의 그래프는 언덕과 계곡이 있는 지형과 유사하게 보입니다(그림 6.11 참조). 데모 프로그램의 나머지 부분은 박스 데모와 매우 유사합니다.

6.11 모양 데모

이 섹션에서는 GeometryGenerator 클래스가 지원하는 다른 두 가지 기하학적 형상인 구체와 원통을 생성하는 방법을 설명합니다. 이러한 형상은 하늘 돔 그리기, 디버깅, 충돌 감지 시각화, 디퍼드 렌더링에 유용합니다. 예를 들어, 디버깅 테스트를 위해 게임 캐릭터를 모두 구체로 렌더링하고 싶을 수 있습니다.

그림 6.12는 이 섹션의 데모 화면을 보여줍니다. 구와 원통을 그리는 방법을 배우는 것 외에도, 여러분은 또한

장면 내에서 여러 개체를 배치하고 그리는 방법(즉, 여러 개의 월드 변환 행렬 생성)에 대한 경험을 쌓습니다. 또한 모든 장면ジオ메트리를 하나의 큰 버텍스 및 인덱스 버퍼에 배치합니다. 그런 다음 DrawIndexed 메서드를 사용하여 개체마다 월드 행렬을 변경해야 하므로 한 번에 하나의 개체를 그립니다. 따라서 DrawIndexed의 StartIndexLocation 및 BaseVertexLocation 매개변수 사용 예시를 확인할 수 있습니다.

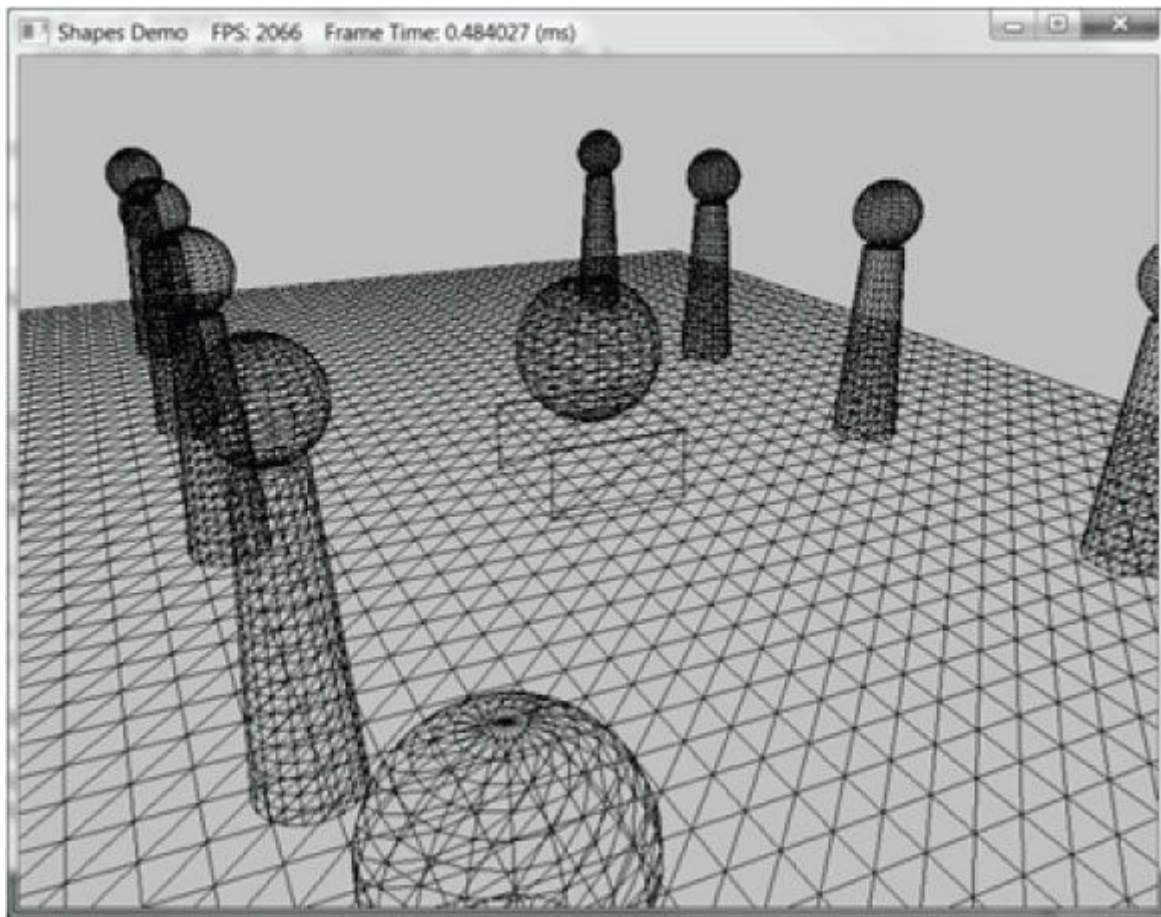


그림 6.12. "Shapes" 데모 스크린샷.

6.11.1 원통형 메쉬 생성

우리는 그림 6.13과 같이 바닥과 상단의 반지름, 높이, 슬라이스 및 스택 수를 지정하여 실린더를 정의합니다. 실린더를 세 부분으로 나눕니다: 1) 측면 형상, 2) 상단 캡 형상, 3) 하단 캡 형상.

6.11.1.1 원통 측면 기하 구조

우리는 원점을 중심으로 y축에 평행한 원통을 생성합니다. 그림 6.13에서 볼 수 있듯이, 모든 정점은 원통의 "링" 위에 위치합니다.

원통에는 *스택 카운트* + 1개의 고리가 있으며, 각 고리에는 *슬라이스 카운트* 개의 고유 정점이 있습니다. 연속된 고리 사이의 반지름 차이는 $\Delta r = (\text{상단 반지름} - \text{하단 반지름}) / \text{스택 카운트}$ 입니다. 지름 0번 링(맨 아래 링)부터 시작할 때, i번째 링의 반지름은 $r_i = \text{bottomRadius} + i\Delta r$ 이며, i번째 링의 높이는 $h_i = -\frac{h}{2} + i\Delta h$ 입니다. 여기서 Δh 는 스택 높이이고 h 는 원통 높이입니다. 따라서 기본 아이디어는 각 링을 순회하며 해당 링 위에 위치한 정점들을 생성하는 것입니다. 이를 구현한 코드는 다음과 같습니다(관련 코드는 굵게 표시):

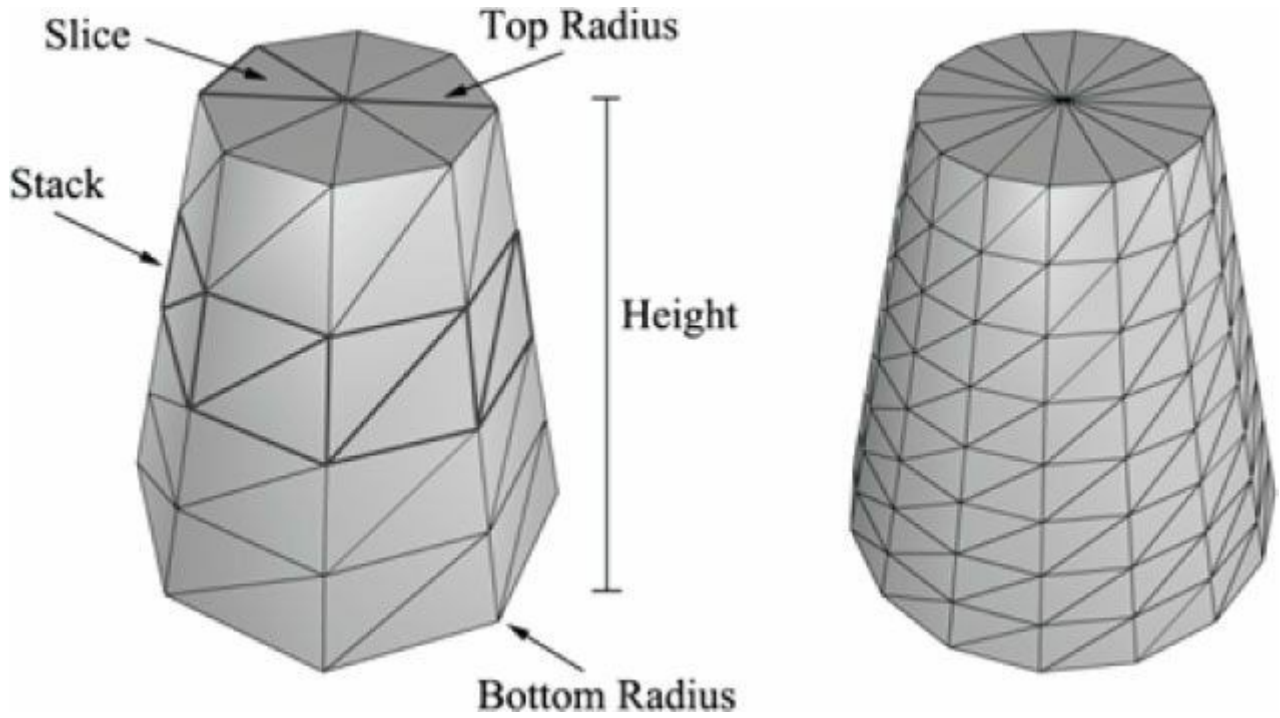


그림 6.13. 이 그림에서 왼쪽 원통은 8개의 슬라이스와 4개의 스택을 가지며, 오른쪽 원통은 16개의 슬라이스와 8개의 스택을 가집니다. 슬라이스와 스택은 삼각형 밀도를 제어합니다. 상단과 하단 반지름이 다를 수 있으므로 "순수한" 원통뿐만 아니라 원뿔 모양의 객체도 생성할 수 있습니다.

```
void GeometryGenerator::CreateCylinder(float bottomRadius, float topRadius, float height, UINT sliceCount, UINT stackCount, MeshData& meshData)
{
    meshData.Vertices.clear(); meshData.Indices.clear();

    //
    // 스택 생성.
    //

    float stack Height = height / stack Count;

    // 각 스택 위로 이동할 때마다 반경을 증가시킬 양
    // 하단부터 상단까지의 레벨.
    float 반경증가량 = (상단반경 - 하단반경) / 스택 개수; UINT 고리개수 = 스택 개수 + 1;

    // 하단에서 시작하여 위로 이동하는 각 스택 링의 정점 계산
    for(UINT i = 0; i < 링 카운트; ++i)
    {
        float y = -0.5f*height + i*stack Height; float r =
        bottomRadius + i*radiusStep;

        // 링의 정점
        float dTheta = 2.0f*XM_PI/sliceCount; for(UINT j = 0; j <=
        sliceCount; ++j)
        {
            Vertex vertex;

            float c = cosf(j*dTheta);
```

```

float s = sinf(j*dTheta);

vertex.Position = XMFLOAT3(r*c, y, r*s);

vertex.TextC.x = (float)j/sliceCount; vertex.TextC.y = 1.0f -
(float)j/stackCount;

// 실린더는 다음과 같이 매개변수화할 수 있으며, 여기서
// v 매개변수를 도입하여
// 텍스처 좌표 v와 동일한 방향을 가지도록 하여,
// 텍스처 좌표 v와 동일한 방향을 가집니다.
//   r0을 하단 반지름, r1을 상단 반지름이라 하자.
//   상단 반지름이라 하자.
//    $y(v) = h - hv, v \in [0,1]$ .
//    $r(v) = r1 + (r0-r1)v$ 
//
//    $x(t, v) = r(v)*\cos(t)$ 
//    $y(t, v) = h - hv$ 
//    $z(t, v) = r(v)*\sin(t)$ 
//
//  $dx/dt = -r(v)*\sin(t)$ 
//  $dy/dt = 0$ 
//  $dz/dt = +r(v)*\cos(t)$ 
//
//  $dx/dv = (r0-r1)*\cos(t)$ 
//  $dy/dv = -h$ 
//  $dz/dv = (r0-r1)*\sin(t)$ 

// 접선U는 단위 길이입니다. vertex.TangentU = XMFLOAT3(-s,
0.0f, c);

float dr = bottomRadius-topRadius; XMFLOAT3 bitangent(dr*c, -
height, dr*s);

XMVECTOR T = XMLoadFloat3(&vertex.TangentU); XMVECTOR B =
XMLoadFloat3(&bitangent);
XMVECTOR N = XMVector3Normalize(XMVector3Cross(T, B)); XMStoreFloat3(&vertex.Normal, N);

meshData.Vertices.push_back(vertex);
}
}

```

 각 링의 첫 번째와 마지막 정점은 위치가 중복되지만, 텍스처 좌표는 중복되지 않음을 확인하십시오. 원통에 텍스처를 올바르게 적용하기 위해 이렇게 해야 합니다.

 실제 메서드 `GeometryGenerator::CreateCylinder`는 향후 데모에 유용할 법선 벡터 및 텍스처 좌표와 같은 추가 정점 데이터를 생성합니다. 지금은 이러한 값들에 대해 걱정하지 마십시오.

그림 6.14에서 각 스택의 모든 슬라이스에 대해 하나의 쿼드(두 개의 삼각형)가 있음을 확인할 수 있습니다. 그림 6.14는 i 번째 스택과 j 번째 슬라이스의 인덱스가 다음과 같이 주어짐을 보여줍니다:

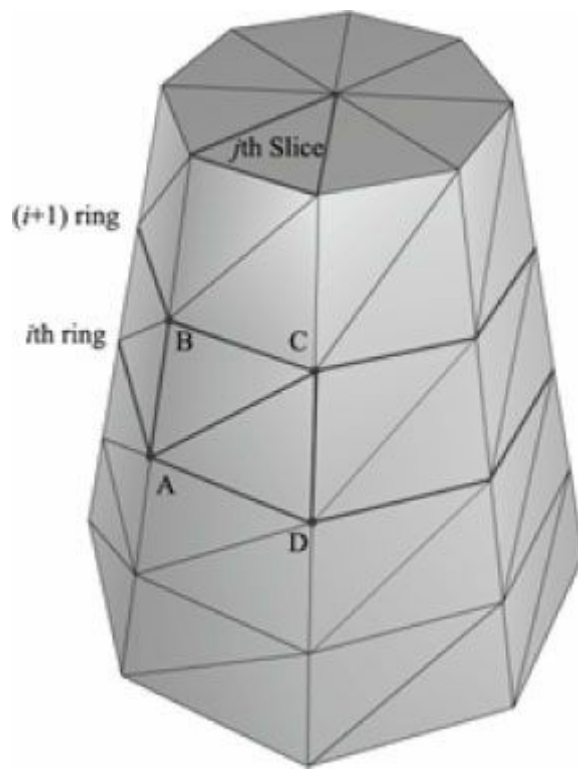


그림 6.14. i 번째 및 $i + 1$ 번째 고리와 j 번째 슬라이스에 포함된 꼭지점 A, B, C, D.

$$\begin{aligned} \triangle ABC &= (i \cdot n + j, & (i + 1) \cdot n + j), & (i + 1) \cdot n + j + 1) \\ \triangle ACD &= (i \cdot n + j, & (i + 1) \cdot n + j + 1, & i \cdot n + j + 1 \end{aligned}$$

여기서 n 은 링당 정점의 수입니다. 따라서 핵심 아이디어는 모든 스택의 모든 슬라이스를 순회하며 위 공식을 적용하는 것입니다.

```
// 링당 첫 번째와 마지막 정점을 중복 처리하므로 1을 더합니다
// 텍스처 좌표가 다르기 때문입니다. UINT ringVertexCount =
sliceCount+1;

// 각 스택에 대한 인덱스 계산. for(UINT i = 0; i <
stackCount; ++i)
{
    for(UINT j = 0; j < sliceCount; ++j)
    {
        meshData.Indices.push_back(i*ringVertexCount + j); meshData.Indices.push_back((i+1)*ringVertexCount + j);
        meshData.Indices.push_back((i+1)*ringVertexCount + j+1); meshData.Indices.push_back(i*ringVertexCount + j);
        meshData.Indices.push_back((i+1)*ringVertexCount + j+1); meshData.Indices.push_back(i*ringVertexCount + j+1);
    }
}

BuildCylinderTopCap(bottomRadius, topRadius, height, sliceCount, stackCount,
meshData);
원통 하단 캡 생성(하단 반지름, 상단 반지름, 높이, 슬라이스 수, 스택 수, 메쉬 데이
터);
}
```

6.11.1.2 캡 지오메트리

캡 지오메트리 생성은 상단 및 하단 링의 슬라이스 삼각형을 생성하여 원을 근사화하는 작업에 해당합니다:

```
void GeometryGenerator::BuildCylinderTopCap(float bottomRadius, float topRadius, float height, UINT
sliceCount,
UINT stackCount, MeshData& meshData)
{
    UINT baseIndex = (UINT)meshData.Vertices.size();
```

```

float y = 0.5f*height;
float dTheta = 2.0f*XM_PI/sliceCount;

// 캡 링 정점 복제: 텍스처 좌표와 법선이 다르기 때문
// 및 법선이 다르기 때문입니다.
for(UINT i = 0; i <= sliceCount; ++i)
{
    float x = topRadius*cosf(i*dTheta); float z =
        topRadius*sinf(i*dTheta);

    // 높이를 기준으로 축소하여 상단 캡을 만들려고 시도
    // 텍스처 좌표 영역을 베이스에 비례하도록 조정합니다.

    float u = x/height + 0.5f; float v =
        z/height + 0.5f;

    meshData.Vertices.push_back( Vertex(x, y, z,
        0.0f, 1.0f, 0.0f,
        1.0f, 0.0f, 0.0f,
        u, v));
}

// 캡 중앙 정점. meshData.Vertices.push_back(
    Vertex(0.0f, y, 0.0f,
    0.0f, 1.0f, 0.0f,
    1.0f, 0.0f, 0.0f,
    0.5f, 0.5f));

// 중심 정점 인덱스.
UINT centerIndex = (UINT)meshData.Vertices.size()-1;

for(UINT i = 0; i < 슬라이스 개수; ++i)
{
    meshData.Indices.push_back(centerIndex); meshData.Indices.push_back(baseIndex +
        i+1); meshData.Indices.push_back(baseIndex + i);
}
}

```

바닥 캡 코드도 유사합니다.

6.11.2 구 메쉬 생성

구체는 반지름과 슬라이스 수, 스택 수를 지정하여 정의합니다([그림 6.15](#) 참조). 구체 생성 알고리즘은 원통 생성 알고리즘과 매우 유사하지만, 각 링의 반지름 변화가 삼각 함수를 기반으로 비선형 방식으로 이루어진다는 점이 다릅니다. `GeometryGenerator::CreateSphere` 코드 연구는 독자에게 맡기겠습니다.

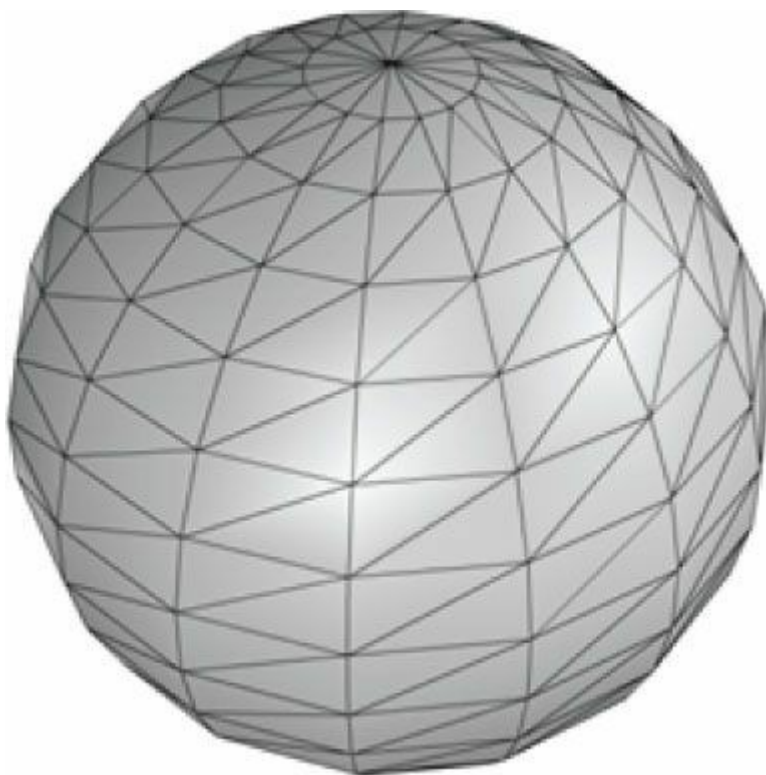


그림 6.15. 테셀레이션 수준을 제어하기 위한 슬라이스와 스택 개념은 구에도 적용됩니다.

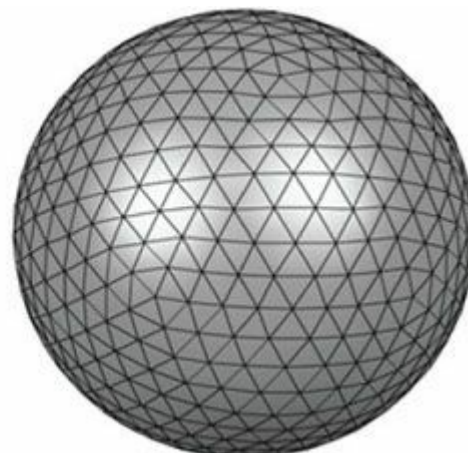
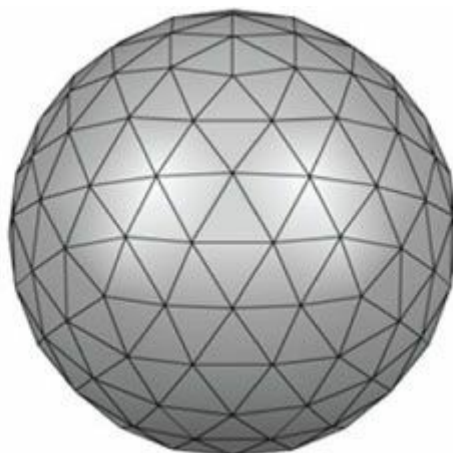


그림 6.16. 반복적인 세분화와 구면 상의 재투영을 통해 지구를 근사화하는 과정.

6.11.3 지오스피어 메쉬 생성

그림 6.15에서 볼 수 있듯이, 구의 삼각형들은 면적이 같지 않습니다. 이는 일부 상황에서는 바람직하지 않을 수 있습니다. 지구는 거의 동일한 면적과 동일한 변 길이를 가진 삼각형을 사용하여 구를 근사화합니다(그림 6.16 참조).

지구를 생성하려면 이코사헤드론으로 시작하여 삼각형을 세분화한 후, 새로운 꼭짓점을 주어진 반지름의 구면에 투영합니다. 주어진 반지름을 가진 구에 투영합니다. 이 과정을 반복하여 테셀레이션을 개선할 수 있습니다.

그림 6.17은 삼각형을 네 개의 동일한 크기의 삼각형으로 분할하는 방법을 보여줍니다. 새로운 꼭짓점은 원본 삼각형의 변을 따라 원래 삼각형의 변을 따라 중간점을 취함으로써 찾을 수 있습니다. 그런 다음 새로운 꼭짓점을 반지름 r 의 구에 투영할 수 있습니다. 이를 위해 꼭짓점을 단위 구에 투영한 후 스칼라 곱셈으로 r 을 곱합니다:

$$\mathbf{v}' = r \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

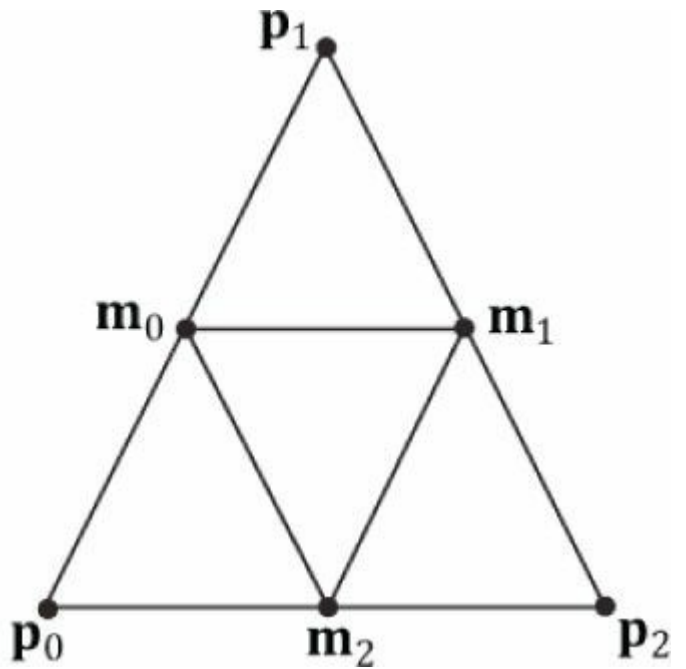
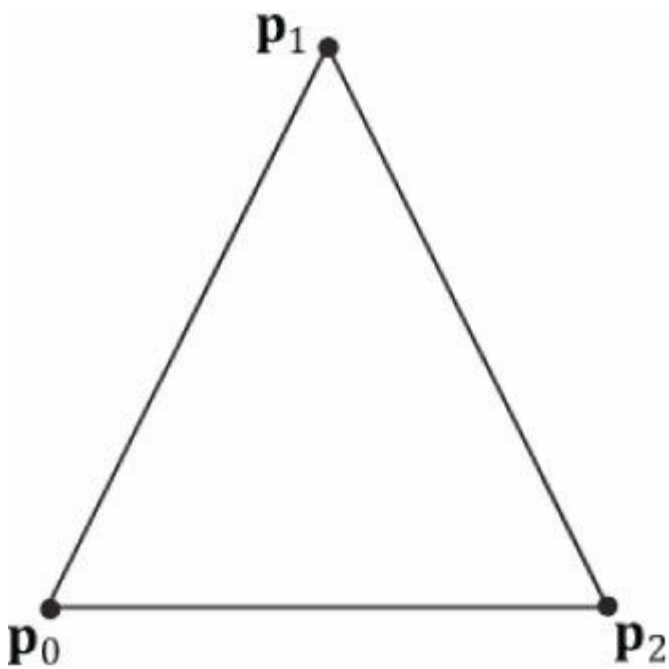


그림 6.17. 삼각형을 면적이 동일한 네 개의 삼각형으로 분할하기.

코드는 다음과 같습니다.

```
void GeometryGenerator::CreateGeosphere(float radius, UINT numSubdivisions, MeshData& meshData)
{
    // 세분화 횟수에 상한을 설정합니다.
    numSubdivisions = MathHelper::Min(numSubdivisions, 5u);

    // 이코사헤드론을 테셀레이션하여 구를 근사화합니다. const float X = 0.525731f;
    const float Z = 0.850651f;

    XMFLOAT3 pos[12] =
    {
        XMFLOAT3(-X, 0.0f, Z), XMFLOAT3(X, 0.0f, Z),
        XMFLOAT3(-X, 0.0f, -Z), XMFLOAT3(X, 0.0f, -Z),
        XMFLOAT3(0.0f, Z, X), XMFLOAT3(0.0f, Z, -X),
        XMFLOAT3(0.0f, -Z, X), XMFLOAT3(0.0f, -Z, -X),
        XMFLOAT3(Z, X, 0.0f), XMFLOAT3(-Z, X, 0.0f),
        XMFLOAT3(Z, -X, 0.0f), XMFLOAT3(-Z, -X, 0.0f)
    };

    DWORD k[60] =
    {
        1,4,0, 4,9,0, 4,5,9, 8,5,4, 1,8,4,
        1,10,8, 10,3,8, 8,3,5, 3,2,5, 3,7,2,
        3,10,7, 10,6,7, 6,11,7, 6,0,11, 6,1,0,
        10,1,6, 11,0,9, 2,11,9, 5,2,9, 11,2,7
    };

    meshData.Vertices.resize(12); meshData.Indices.resize(60);

    for(size_t i = 0; i < 12; ++i) meshData.Vertices[i].Position = pos[i];

    for(size_t i = 0; i < 60; ++i) meshData.Indices[i] = k[i];

    for(size_t i = 0; i < numSubdivisions; ++i) Subdivide(meshData);
}
```



```
// 정점을 구면에 투영하고 크기 조정 for(size_t i = 0; i <
meshData.Vertices.size(); ++i)
{
    // 단위 구에 투영합니다.
    XMVECTOR n = XMVector3Normalize(XMLoadFloat3(
        &meshData.Vertices[i].Position));

    // 구에 투영합니다. XMVECTOR p =
    radius*n;

    XMStoreFloat3(&meshData.Vertices[i].Position, p);
    XMStoreFloat3(&meshData.Vertices[i].Normal, n);

    // 좌표로부터 텍스처 좌표 유도. float theta = MathHelper::AngleFromXY(
        meshData.Vertices[i].Position.x,
        meshData.Vertices[i].Position.z);

    float phi = acosf(meshData.Vertices[i].Position.y / radius);

    meshData.Vertices[i].TexC.x = theta/XM_2PI; meshData.Vertices[i].TexC.y =
    phi/XM_PI;

    // P에 대한 theta의 편미분 meshData.Vertices[i].TangentU.x = -radius*sinf(phi)*sinf(theta);
    meshData.Vertices[i].TangentU.y = 0.0f; meshData.Vertices[i].TangentU.z = +radius*sinf(phi)*cosf(theta);

    XMVECTOR T = XMLoadFloat3(&meshData.Vertices[i].TangentU); XMStoreFloat3(&meshData.Vertices[i].TangentU,
        XMVector3Normalize(T));
}
}
```

6.11.4 데모 코드

이 섹션에서는 "Shapes" 데모와 이전 두 데모 간의 주요 차이점을 살펴봅니다. 다른 지오메트리를 그리는 점은 제외하고, 주요 차이점은 여러 개체를 그리는 것입니다. 각 개체는 월드 공간에 대한 개체의 로컬 공간을 설명하는 월드 매트릭스를 가지며, 이는 월드 내 개체의 위치를 지정합니다. 이 데모에서는 여러 개의 구체와 원통을 그리는 것처럼 보이지만, 실제로는 구체와 원통 지오메트리 의 복사본이 하나만 필요합니다. 동일한 구체와 원통 메시를 여러 번 다시 그리는 것이지만, 서로 다른 월드 매트릭스를 사용합니다. 이를 인스턴싱이라고 합니다. 매트릭스는 초기화 시점에 다음과 같이 생성됩니다:

```
// 로컬 공간에서 월드 공간으로의 변환 정의 XMFLOAT4X4 mSphereWorld[10];

XMFLOAT4X4 mCylWorld[10];
XMFLOAT4X4 mBoxWorld;
XMFLOAT4X4 mGridWorld;

XMFLOAT4X4 mCenterSphere; XMMATRIX I =
XMMatrixIdentity();

XMStoreFloat4x4(&mGridWorld, I);
XMMATRIX boxScale = XMMatrixScaling(2.0f, 1.0f, 2.0f); XMMATRIX boxOffset =
XMMatrixTranslation(0.0f, 0.5f, 0.0f);

XMStoreFloat4x4(&mBoxWorld, XMMatrixMultiply(boxScale, boxOffset)); XMMATRIX centerSphereScale =
XMMatrixScaling(2.0f, 2.0f, 2.0f); XMMATRIX centerSphereOffset = XMMatrixTranslation(0.0f, 2.0f, 0.0f);
XMStoreFloat4x4(&mCenterSphere, XMMatrixMultiply(centerSphereScale, centerSphereOffset));

// 행당 2개의 원통과 구로 구성된 5개의 행을 생성합니다. for(int i = 0; i < 5; ++i)
```



```
{
    XMStoreFloat4x4(&mCylWorld[i*2+0], XMMatrixTranslation(-5.0f, 1.5f, -10.0f
        + i*5.0f));
    XMStoreFloat4x4(&mCylWorld[i*2+1], XMMatrixTranslation(+5.0f, 1.5f, -10.0f + i*5.0f));

    XMStoreFloat4x4(&mSphereWorld[i*2+0], XMMatrixTranslation(-5.0f, 3.5f, -
        10.0f + i*5.0f));
    XMStoreFloat4x4(&mSphereWorld[i*2+1], XMMatrixTranslation(+5.0f, 3.5f, -10.0f +
        i*5.0f));
}
```

모든 메시 정점과 인덱스를 하나의 정점 및 인덱스 버퍼에 압축합니다. 이는 정점 배열과 인덱스 배열을 연결함으로써 이루어집니다. 즉, 객체를 렌더링할 때 정점 및 인덱스 버퍼의 일부만 렌더링하게 됩니다. 지오메트리 일부만 렌더링하기 위해 알아야 할 세 가지 정보가 있습니다([그림 6.3](#) 참조). 연결된 인덱스 버퍼에서 각 객체의 시작 인덱스와 객체당 인덱스 개수를 알아야 합니다. 세 번째로 필요한 것은 연결된 버텍스 버퍼에서 각 객체의 첫 번째 버텍스까지의 인덱스 오프셋입니다. 이는 정점 배열을 연결할 때 상쇄하기 위한 인덱스 오프셋을 적용하지 않았기 때문입니다([5장 연습문제 2](#) 참조). 그러나 각 객체의 첫 번째 정점에 대한 인덱스 오프셋을 알고 있다면, 이를 `ID3D11DeviceContext::DrawIndexed`의 세 번째 매개변수로 전달할 수 있으며, 해당 오프셋이 해당 드로우 콜의 모든

해당 드로우 콜의 모든 인덱스에 추가되어 자동으로 오프셋을 처리해 줍니다.

다음 코드는 지오메트리 버퍼가 생성되는 방식, 필요한 드로잉 양이 캐시되는 방식, 그리고 객체가 그려지는지 보여줍니다.

```
void ShapesApp::BuildGeometryBuffers()
{
    GeometryGenerator::MeshData box;
    GeometryGenerator::MeshData grid;
    GeometryGenerator::MeshData sphere;
    GeometryGenerator::MeshData cylinder;

    GeometryGenerator geoGen; geoGen.CreateBox(1.0f, 1.0f, 1.0f, box);
    geoGen.CreateGrid(20.0f, 30.0f, 60, 40, grid);
    geoGen.CreateSphere(0.5f, 20, 20, sphere);
    geoGen.CreateCylinder(0.5f, 0.3f, 3.0f, 20, 20, cylinder);

    // 연결된 버텍스 버퍼에 각 객체에 대한 버텍스 오프셋을 캐시합니다
    mBoxVertexOffset = 0;
    mGridVertexOffset = box.Vertices.size();
    mSphereVertexOffset = mGridVertexOffset + grid.Vertices.size(); mCylinderVertexOffset = mSphereVertexOffset +
    sphere.Vertices.size();

    // 각 객체의 인덱스 개수를 캐시합니다. mBoxIndexCount =
    box.Indices.size(); mGridIndexCount = grid.Indices.size();
    mSphereIndexCount = sphere.Indices.size();
    mCylinderIndexCount = cylinder.Indices.size();

    // 연결된 인덱스 버퍼 내 각 객체의 시작 인덱스를 캐시합니다.

    // 인덱스 버퍼에 저장합니다.
    mBoxIndexOffset = 0;
    mGridIndexOffset = mBoxIndexCount;
    mSphereIndexOffset = mGridIndexOffset + mGridIndexCount; mCylinderIndexOffset = mSphereIndexOffset +
    mSphereIndexCount;

    UINT 총정점수 = box.Vertices.size() +
        grid.Vertices.size() + sphere.Vertices.size() +
        cylinder.Vertices.size();
```

```

UINT 총인덱스카운트 = mBoxIndexCount +
    mGridIndexCount + mSphereIndexCount +
    mCylinderIndexCount;

//
// 관심 있는 정점 요소 추출 및
// 모든 메시의 정점을 하나의 정점 버퍼에 압축합니다.
//

std::vector<Vertex> vertices(totalVertexCount); XMFLOAT4 black(0.0f, 0.0f, 0.0f, 1.0f);

UINT k = 0;

for(size_t i = 0; i < box.Vertices.size(); ++i, ++k)
{
    vertices[k].Pos = box.Vertices[i].Position; vertices[k].Color = black;
}

for(size_t i = 0; i < grid.Vertices.size(); ++i, ++k)
{
    vertices[k].Pos = grid.Vertices[i].Position; vertices[k].Color = black;
}

for(size_t i = 0; i < sphere.Vertices.size(); ++i, ++k)
{
    vertices[k].Pos = sphere.Vertices[i].Position; vertices[k].Color = black;
}

for(size_t i = 0; i < cylinder.Vertices.size(); ++i, ++k)
{
    vertices[k].Pos = cylinder.Vertices[i].Position; vertices[k].Color = black;
}
D3D11_BUFFER_DESC vbd;
vbd.Usage = D3D11_USAGE_IMMUTABLE;
vbd.ByteWidth = sizeof(Vertex) * totalVertexCount; vbd.BindFlags =
D3D11_BIND_VERTEX_BUFFER; vbd.CPUAccessFlags = 0;
vbd.MiscFlags = 0; D3D11_SUBRESOURCE_DATA vinitData;
vinitData.pSysMem = &vertices[0];
HR(md3dDevice->CreateBuffer(&vbd, &vinitData, &mVB));

//
// 모든 메시의 인덱스를 하나의 인덱스 버퍼에 압축합니다.
//

std::vector<UINT> indices;
indices.insert(indices.end(), box.Indices.begin(), box.Indices.end()); indices.insert(indices.end(), grid.Indices.begin(), grid.Indices.end());
indices.insert(indices.end(), sphere.Indices.begin(),
    sphere.Indices.end());    indices.insert(indices.end(),
cylinder.Indices.begin(),
    cylinder.Indices.end());

D3D11_BUFFER_DESC ibd;

```

```

ibd.Usage = D3D11_USAGE_IMMUTABLE;
ibd.ByteWidth = sizeof(UINT) * totalIndexCount; ibd.BindFlags =
D3D11_BIND_INDEX_BUFFER; ibd.CPUAccessFlags = 0;
ibd.MiscFlags = 0; D3D11_SUBRESOURCE_DATA iinitData;
iinitData.pSysMem = &indices[0];
HR(md3dDevice->CreateBuffer(&ibd, &iinitData, &mIB));
}
void ShapesApp::DrawScene()
{
    md3dImmediateContext->ClearRenderTargetView(mRenderTargetView, reinterpret_cast<const float*>(&Colors::Blue));
    md3dImmediateContext->ClearDepthStencilView(mDepthStencilView, D3D11_CLEAR_DEPTH|D3D11_CLEAR_STENCIL, 1.0f, 0);

    md3dImmediateContext->IASetInputLayout(mInputLayout); md3dImmediateContext-
    >IASetPrimitiveTopology(
        D3D11_PRIMITIVE_TOPOLOGY_TRIANGLELIST);

    md3dImmediateContext->RSSetState(mWireframeRS); UINT stride = sizeof(Vertex);
    UINT offset = 0;
    md3dImmediateContext->IASetVertexBuffers(0, 1, &mVB, &stride, &offset); md3dImmediateContext-
    >IASetIndexBuffer(mIB, DXGI_FORMAT_R32_UINT, 0);

    // 상수 설정

    XMMATRIX 뷰 = XMLoadFloat4x4(&mView); XMMATRIX 프로젝션 =
    XMLoadFloat4x4(&mProj); XMMATRIX 뷰프로젝션 = 뷰*프로젝션;
    D3DX11_TECHNIQUE_DESC 테크닉디스크립션;
    mTech->GetDesc(&techDesc);
    for(UINT p = 0; p < techDesc.Passes; ++p)
    {
        // 그리드 그리기.
        XMMATRIX world = XMLoadFloat4x4(&mGridWorld); mfxWorldViewProj-
        >SetMatrix(
            reinterpret_cast<float*>(&(world*viewProj)));
        mTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); md3dImmediateContext-
        >DrawIndexed(
            mGridIndexCount, mGridIndexOffset, mGridVertexOffset);

        // 박스 그리기.
        world = XMLoadFloat4x4(&mBoxWorld); mfxWorldViewProj-
        >SetMatrix(
            reinterpret_cast<float*>(&(world*viewProj)));
        mTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); md3dImmediateContext-
        >DrawIndexed(
            mBoxIndexCount, mBoxIndexOffset, mBoxVertexOffset);

        // 중심 구체 그리기.
        world = XMLoadFloat4x4(&mCenterSphere); mfxWorldViewProj-
        >SetMatrix(
            reinterpret_cast<float*>(&(world*viewProj)));
        mTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); md3dImmediateContext-
        >DrawIndexed(
            mSphereIndexCount, mSphereIndexOffset, mSphereVertexOffset);

        // 원통 그리기. for(int i = 0; i < 10; ++i)

```

```

    {
        world = XMLoadFloat4x4(&mCylWorld[i]); mfxWorldViewProj->SetMatrix(
            reinterpret_cast<float*>(&(world*viewProj)));
        mTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); md3dImmediateContext->DrawIndexed(mCylinderIndexCount,
            mCylinderIndexOffset, mCylinderVertexOffset);
    }

    // 구를 그림니다. for(int i = 0; i < 10; ++i)
    {
        world = XMLoadFloat4x4(&mSphereWorld[i]); mfxWorldViewProj->SetMatrix(
            reinterpret_cast<float*>(&(world*viewProj)));
        mTech->GetPassByIndex(p)->Apply(0, md3dImmediateContext); md3dImmediateContext->DrawIndexed(mSphereIndexCount,
            mSphereIndexOffset, mSphereVertexOffset);
    }
}

HR(mSwapChain->Present(0, 0));
}

```

6.12 파일에서 지오메트리 로드하기

이 책의 일부 데모에는 박스, 그리드, 구체, 원통만으로도 충분하지만, 더 복잡한 지오메트리를 렌더링하면 일부 데모의 품질이 향상됩니다. 나중에 널리 사용되는 3D 모델링 형식에서 3D 메시를 불러오는 방법을 설명하겠습니다. 그 전에, 두개골 메시([그림 6.18](#))의 지오메트리를 단순한 정점 목록(위치와 법선 벡터만 포함)과 인덱스로 내보냈습니다. 표준 C++ 파일 입출력을 사용하여 파일에서 정점과 인덱스를 간단히 읽어와 정점 및 인덱스 버퍼에 복사할 수 있습니다. 파일 형식은 매우 간단한 텍스트 파일입니다:

```

VertexCount: 31076
삼각형 수: 60339 정점 목록 (위치, 법선)

{
    0.592978 1.92413 -2.62486 0.572276 0.816877 0.0721907
    0.571224 1.94331 -2.66948 0.572276 0.816877 0.0721907
    0.609047 1.90942 -2.58578 0.572276 0.816877 0.0721907
    ...
}
삼각형 목록
{
    0 1 2
    3 4 5
    6 7 8
    ...
}

```

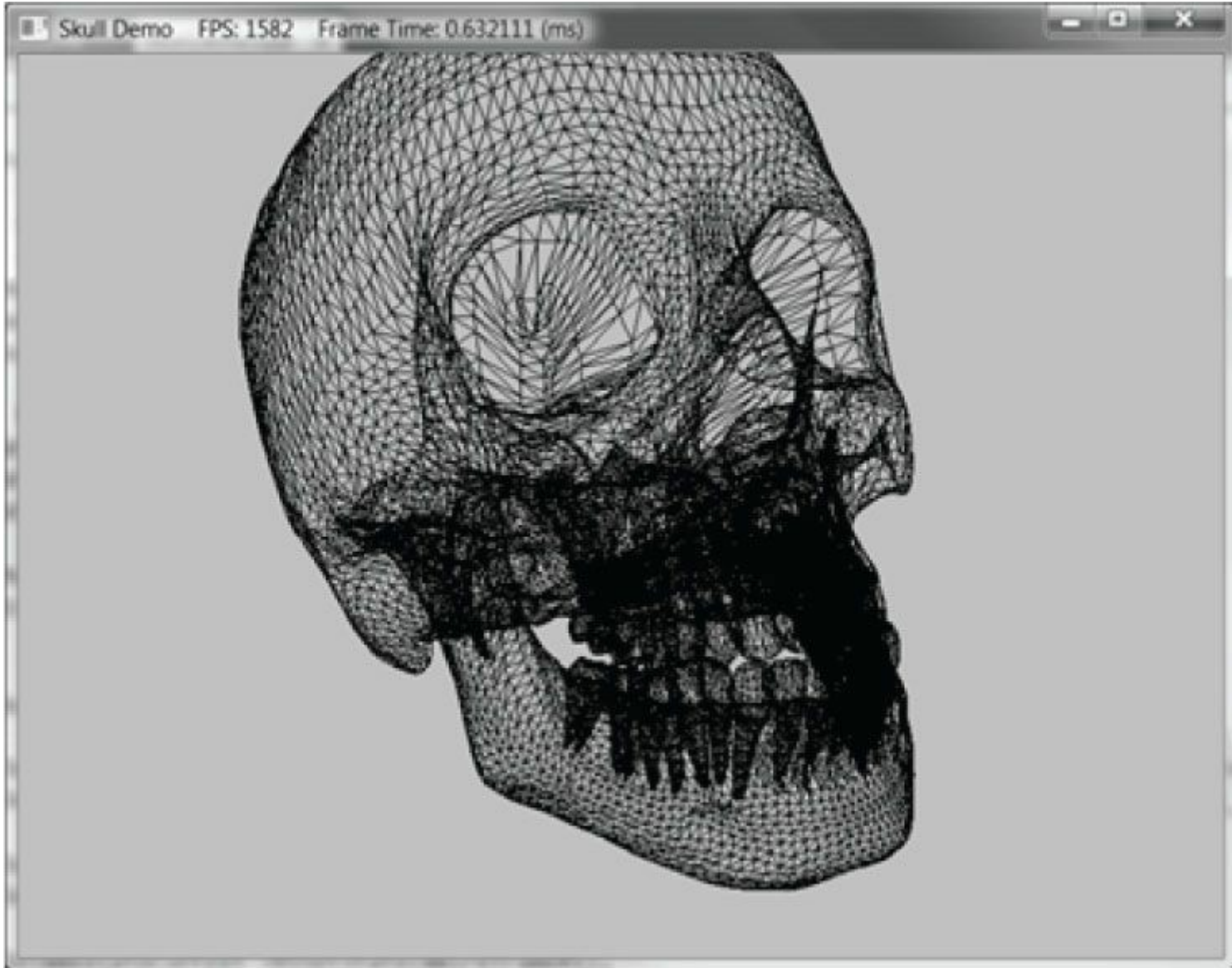


그림 6.18. "Skull" 데모의 스크린샷.

6.13 동적 정점 버퍼

지금까지 우리는 초기화 시점에 고정되는 정적 버퍼를 다루어 왔습니다. 반면 동적 버퍼의 내용은 일반적으로 프레임마다 변경됩니다. 동적 버퍼는 무언가를 애니메이션 처리해야 할 때 주로 사용됩니다. 예를 들어 파동 시뮬레이션을 수행 중이며, 해 함수 $f(x, z, t)$ 에 대한 파동 방정식을 풀고 있다고 가정해 보겠습니다. 이 함수는 $시각 t$ 에 xz 평면 상 각 지점의 파고를 나타냅니다. 이 함수를 이용해 파도를 그릴 경우, 산과 골짜기 표현 시와 마찬가지로 삼각형 격자 메쉬를 사용하고 각 격자점에 $f(x, z, t)$ 를 적용하여 격자점별 파고를 구하게 됩니다. 이 함수는 $시각 t$ 에 의존하므로(즉, 파동 표면이 시간에 따라 변화함), 부드러운 애니메이션을 얻기 위해 그리드 점들에 이 함수를 짧은 시간 후(예: 1/30초마다) 재적용해야 합니다. 따라서 시간이 지남에 따라 삼각형 그리드 메쉬 정점들의 높이를 업데이트하기 위해 동적 정점 버퍼가 필요합니다. 동적 버텍스 버퍼가 필요한 또 다른 상황은 복잡한 물리 및 충돌 감지 기능을 가진 파티클 시스템입니다. 각 프레임마다 CPU에서 물리 및 충돌 감지를 수행하여 파티클의 새 위치를 계산합니다. 파티클 위치가 프레임마다 변하기 때문에, 각 프레임의 렌더링을 위해 파티클 위치를 업데이트하려면 동적 버텍스 버퍼가 필요합니다.

버퍼를 동적으로 만들기 위해서는 **D3D11_USAGE_DYNAMIC** 사용을 지정해야 합니다. 또한

버퍼에 쓰기를 수행할 예정이므로 CPU 액세스 플래그 **D3D11_CPU_ACCESS_WRITE**가 필요합니다:

```
D3D11_BUFFER_DESC vbd;
vbd.Usage = D3D11_USAGE_DYNAMIC; vbd.ByteWidth = sizeof(Vertex) *
mWaves.VertexCount(); vbd.BindFlags = D3D11_BIND_VERTEX_BUFFER;
vbd.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE; vbd.MiscFlags = 0;
```

```
HR(md3dDevice->CreateBuffer(&vbd, 0, &mWavesVB));
```

그런 다음 ID3D11DeviceContext::Map 함수를 사용하여 버퍼 메모리 블록의 시작 부분에 대한 포인터를 얻고 여기에 쓸 수 있습니다:

```
HRESULT ID3D11DeviceContext::Map( ID3D11Resource
    *pResource, UINT Subresource,
    D3D11_MAP MapType, UINT
    MapFlags,
    D3D11_MAPPED_SUBRESOURCE *pMappedResource);
```

- 1. pResource: 읽기/쓰기 목적으로 접근하려는 리소스에 대한 포인터입니다. 버퍼는 Direct3D 11 리소스의 한 유형이지만, 텍스처 리소스와 같은 다른 리소스도 이 방법으로 접근할 수 있습니다.
- 2. 하위 리소스: 리소스에 포함된 하위 리소스에 대한 인덱스입니다. 이 기능의 사용법은 나중에 살펴보겠습니다. 현재 버퍼에는 하위 리소스가 없으므로 0을 지정합니다.
- 3. MapType: 일반적인 플래그는 다음 중 하나입니다:

D3D11_MAP_WRITE_DISCARD: 하드웨어에 버퍼를 버리고 새로 할당된 버퍼의 포인터를 반환하도록 지시합니다. 이는 하드웨어가 버려진 버퍼에서 렌더링을 계속하도록 허용함으로써 하드웨어가 새로 할당된 버퍼에 쓰기 작업을 수행하는 동안 스틀되는 것을 방지합니다.

D3D11_MAP_WRITE_NO_OVERWRITE: 버퍼의 초기화되지 않은 부분에만 쓰기를 수행할 것임을 하드웨어에 알립니다. 이는 초기화되지 않은 부분에 쓰기를 수행하는 동시에 하드웨어가 이전에 쓰여진 지오메트리를 계속 렌더링할 수 있도록 하여 하드웨어가 스틀되는 것을 방지합니다.

3D11_MAP_READ: 스테이징 버퍼와 함께 사용되며, GPU 버퍼의 복사본을 시스템 메모리로 읽어야 할 때 사용됩니다.

- 4. MapFlags: 사용하지 않는 선택적 플래그이므로 0으로 지정합니다. 자세한 내용은 SDK 문서를 참조하십시오.
- 5. pMappedResource: 리소스 데이터에 대한 읽기/쓰기 액세스가 가능한 D3D11_MAPPED_SUBRESOURCE 포인터를 반환합니다.

D3D11_MAPPED_SUBRESOURCE 구조체는 다음과 같이 정의됩니다:

```
typedef struct D3D11_MAPPED_SUBRESOURCE { void *pData;
    UINT RowPitch; UINT
    DepthPitch;
} D3D11_MAPPED_SUBRESOURCE;
```

- 1. pData: 리소스의 원시 메모리 포인터로 읽기/쓰기용입니다. 리소스가 저장하는 적절한 데이터 형식으로 캐스팅해야 합니다.
- 2. RowPitch: 리소스 내 데이터 한 행의 바이트 크기. 예를 들어, 2D 텍스처의 경우 한 행의 바이트 크기입니다.
- 3. DepthPitch: 리소스 내 데이터 페이지의 바이트 크기입니다. 예를 들어, 3D 텍스처의 경우 이는 3D 텍스처의 하나의 2D 이미지 부분 집합의 바이트 크기입니다.

RowPitch와 DepthPitch의 차이는 2D 및 3D 리소스(2D 또는 3D 배열을 생각하십시오)에 있습니다. 버텍스/인덱스 버퍼의 경우

기본적으로 1차원 데이터 배열인 경우, RowPitch와 DepthPitch에는 동일한 값이 할당되며, 이는 정점/인덱스 버퍼의 바이트 크기와 동일합니다.

다음 코드는 "Waves" 데모에서 버텍스 버퍼를 업데이트하는 방법을 보여줍니다:

```
D3D11_MAPPED_SUBRESOURCE mappedData;
HR(md3dImmediateContext->Map(mWavesVB, 0,
    D3D11_MAP_WRITE_DISCARD, 0, &mappedData)); Vertex* v =
reinterpret_cast<Vertex*>(mappedData.pData); for(UINT i = 0; i < mWaves.VertexCount(); ++i)
{
    v[i].Pos = mWaves[i];
    v[i].Color = XMFLLOAT4(0.0f, 0.0f, 0.0f, 1.0f);
}
md3dImmediateContext->Unmap(mWavesVB, 0);
```

버퍼 업데이트를 완료한 후에는 ID3D11DeviceContext::Unmap 함수를 호출해야 합니다.

동적 버퍼를 사용할 때는 일부 오버헤드가 발생합니다. 새로운 데이터가 CPU 메모리에서 GPU 메모리로 다시 전송되어야 하기 때문입니다.

따라서 정적 버퍼가 작동한다면 동적 버퍼보다 정적 버퍼를 우선적으로 사용해야 합니다. 최신 버전의

Direct3D는 동적 버퍼의 필요성을 줄이기 위한 새로운 기능을 도입했습니다. 예를 들어,

1. 간단한 애니메이션은 버텍스 셰이더에서 수행할 수 있습니다.
2. 렌더 투 텍스처 또는 컴퓨트 셰이더와 버텍스 텍스처 페치 기능을 통해, 앞서 설명한 것과 같은 파동 시뮬레이션을 GPU에서 완전히 실행하도록 구현할 수 있습니다.
3. 지오메트리 셰이더는 GPU가 프리미티브를 생성하거나 파괴할 수 있는 기능을 제공하며, 지오메트리 셰이더가 없다면 일반적으로 CPU에서 수행해야 하는 작업입니다.

인덱스 버퍼도 동적으로 사용할 수 있습니다. 그러나 "Waves" 데모에서는 삼각형 토폴로지가 고정되어 있고 버텍스 높이만 변경되므로 버텍스 버퍼만 동적으로 관리하면 됩니다.

이 장의 "파도" 데모는 동적 버텍스 버퍼를 사용하여 본 장 시작 부분에서 설명한 것과 같은 간단한 파도 시뮬레이션을 구현합니다.

이 책에서는 파동 시뮬레이션의 실제 알고리즘 세부 사항(이에 대해서는 [Lengyel02] 참조)보다는 동적 버퍼를 설명하기 위한 프로세스에 더 중점을 둡니다. CPU에서 시뮬레이션을 업데이트한 후 **Map/Unmap**을 사용하여 버텍스 버퍼를 그에 맞게 업데이트합니다.

 "Waves" 데모에서는 파도를 와이어프레임 모드로 렌더링합니다. 이는 조명 없이 솔리드 필 모드에서는 파동 움직임을 확인하기 어렵기 때문입니다.

 이 데모는 렌더 투 텍스처() 기능이나 컴퓨트 셰이더, 버텍스 텍스처 페치(vertex texture fetch) 같은 고급 기법을 활용해 GPU에서 구현할 수 있음을 다시 한번 언급합니다. 다만 해당 주제는 아직 다루지 않았으므로,

우리는 CPU에서 파동 시뮬레이션을 수행하고 동적 버텍스 버퍼를 사용하여 새로운 버텍스를 업데이트합니다.

6.14 요약

1. Direct3D에서 정점은 공간적 위치 외에도 추가 데이터를 포함할 수 있습니다. 사용자 정의 정점 형식을 생성하려면 먼저 선택한 정점 데이터를 저장할 구조체를 정의합니다. 정점 구조체를 정의한 후에는 입력 레이아웃 설명(**D3D11_INPUT_ELEMENT_DESC** 요소의 배열로, 정점 구성 요소마다 하나씩 할당됨)을 정의하여 Direct3D에 이를 설명합니다. 이 배열 설명을 사용하여 **ID3D11Device::CreateInputLayout**으로 **ID3D11InputLayout** 객체를 생성하고, **ID3D11DeviceContext::IASetInputLayout** 메서드를 통해 입력 레이아웃을 IA 단계에 바인딩할 수 있습니다.
2. GPU가 버텍스/인덱스 배열에 접근하려면, **ID3D11Buffer** 인터페이스로 표현되는 *버퍼*라는 특수한 리소스 구조에 배치되어야 합니다.
버텍스를 저장하는 버퍼는 *버텍스 버퍼*라고 하며, 인덱스를 저장하는 버퍼는 *인덱스 버퍼*라고 합니다. Direct3D 버퍼는 데이터를 저장할 뿐만 아니라 데이터에 접근하는 방식과 렌더링 파이프라인에 바인딩될 위치를 정의합니다. 버퍼는 **D3D11_BUFFER_DESC** 및 **D3D11_SUBRESOURCE_DATA** 인스턴스를 작성하고 **ID3D11Device::CreateBuffer**를 호출하여 생성됩니다. 버텍스 버퍼는 **ID3D11DeviceContext::IASetVertexBuffers** 메서드를 사용하여 IA 단계에 바인딩되고, 인덱스 버퍼는 **ID3D11DeviceContext::IASetIndexBuffer** 메서드를 사용하여 IA 단계에 바인딩됩니다. 인덱싱되지 않은 지오메트리는 **ID3D11DeviceContext::Draw**로, 인덱싱된 지오메트리는 **ID3D11DeviceContext::DrawIndexed**로 렌더링할 수 있습니다.
3. 버텍스 셰이더는 HLSL로 작성된 프로그램으로, GPU에서 실행되며 버텍스를 입력으로 받아 버텍스를 출력합니다. 렌더링되는 모든 버텍스는 버텍스 셰이더를 통과합니다. 이를 통해 프로그램머는 버텍스 단위로 특수한 작업을 수행하여 다양한 렌더링 효과를 구현할 수 있습니다. 버텍스 셰이더에서 출력된 값은 파이프라인의 다음 단계로 전달됩니다.
4. 상수 버퍼는 C++ 애플리케이션 코드와 셰이더 프로그램 모두에서 접근할 수 있는 다양한 변수를 저장할 수 있는 데이터 블록입니다. 이를 통해 C++ 애플리케이션은 셰이더와 통신하여 셰이더가 사용하는 상수 버퍼의 값을 업데이트할 수 있습니다. 예를 들어, C++ 애플리케이션은 셰이더가 사용하는 월드-뷰-프로젝션 행렬을 변경할 수 있습니다.
일반적인 조언은 상수 버퍼의 내용을 업데이트해야 하는 빈도에 따라 상수 버퍼를 생성하라는 것입니다. 상수 버퍼를 분할하는 이유는 효율성 때문입니다. 상수 버퍼가 업데이트될 때 그 안의 모든 변수가 업데이트되어야 하므로, 중복 업데이트를 최소화하기 위해 업데이트 빈도에 따라 그룹화하는 것이 효율적입니다.
5. 픽셀 셰이더는 HLSL로 작성된 프로그램으로, GPU에서 실행되며 보간된 버텍스 데이터를 입력받아 색상 값을 출력합니다. 하드웨어 최적화의 일환으로, 픽셀 프래그먼트가 픽셀 셰이더에 도달하기 전에 파이프라인에 의해 거부될 수 있습니다(예: 멀리 Z 거부). 픽셀 셰이더를 통해 프로그래머는 픽셀 단위로 특수 작업을 수행하여 다양한 렌더링 효과를 구현할 수 있습니다. 픽셀 셰이더에서 출력된 값은 파이프라인의 다음 단계로 전달됩니다.
6. 렌더 상태는 기하학적 구조가 렌더링되는 방식에 영향을 미치는 장치의 상태입니다. 렌더 상태는 변경될 때까지 유효하며, 이후 모든 드로잉 작업의 기하학적 구조에 현재 값이 적용됩니다. 모든 렌더 상태에는 초기 기본 상태가 있습니다. Direct3D는 렌더 상태를 세 가지 상태 블록으로 구분합니다: 래스터라이저 상태(**ID3D11RasterizerState**), 블렌드 상태(**ID3D11BlendState**), 깊이/스텐실 상태(**ID3D11DepthStencilState**). 렌더 상태는 C++에서 생성 및 설정할 수 있습니다.

애플리케이션 수준 또는 효과 파일에서.

7. Direct3D 효과(ID3DX11Effect)는 하나 이상의 렌더링 기법을 캡슐화합니다. 렌더링 기법은 특정 방식으로 3D 지오메트리를 렌더링하는 방법을 지정하는 코드를 포함합니다. 렌더링 기법은 하나 이상의 렌더링 패스로 구성됩니다. 각 렌더링 패스마다 지오메트리가 렌더링됩니다. 다중 패스 기법은 원하는 결과를 얻기 위해 지오메트리를 여러 번 렌더링해야 합니다. 각 패스는 정점 셰이더, 선택적 지오메트리 셰이더, 선택적 테셀레이션 셰이더, 픽셀 셰이더, 그리고 해당 패스의 지오메트리 그리기에 사용되는 렌더 상태로 구성됩니다. 정점, 지오메트리, 테셀레이션, 픽셀 셰이더는 효과 파일에 정의된 상수 버퍼의 변수와 텍스처 리소스에 접근할 수 있습니다. 컴파일 시간 매개변수를 사용하여 효과 프레임워크가 주어진 컴파일 시간 인수를 기반으로 셰이더 변형을 생성하도록 할 수 있습니다.
8. 동적 버퍼는 런타임에 버퍼의 내용을 자주 업데이트해야 할 때 사용됩니다(예: 매 프레임 또는 1/30초마다). 동적 버퍼는 D3D11_USAGE_DYNAMIC 사용 설정과 D3D11_CPU_ACCESS_WRITE CPU 액세스 플래그로 생성해야 합니다. 버퍼 업데이트에는 ID3D11DeviceContext::Map 및 ID3D11DeviceContext::Unmap 메서드를 사용합니다.

6.15 연습 문제

1. 다음 정점 구조체에 대한 D3D10_INPUT_ELEMENT_DESC 배열을 작성하세요:

```
struct Vertex
{
    XMFLOAT3 Pos;
    XMFLOAT3 접선; XMFLOAT3 법
    선; XMFLOAT2 텍스0;
    XMFLOAT2 텍스1;
    XMCOLOR Color;
};
```

2. 컬러 큐브 데모를 다시 구현하되, 이번에는 두 개의 정점 버퍼(및 두 개의 입력 슬롯)를 사용하여 파이프라인에 정점을 공급합니다. 하나는 위치 요소를 저장하고 다른 하나는 색상 요소를 저장합니다. D3D11_INPUT_ELEMENT_DESC 배열은 다음과 같이 보일 것입니다:

```
D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 1, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};
```

위치 요소는 입력 슬롯 0에 연결되고, 색상 요소는 입력 슬롯 1에 연결됩니다. 또한 두 요소 모두 D3D11_INPUT_ELEMENT_DESC::AlignedByteOffset이 0임을 유의하십시오. 이는 위치와 색상 요소가 더 이상 단일 입력 슬롯에 인터리브되지 않기 때문입니다.

3. Draw

점 목록(예: 그림 5.13a와 같은 형태). 선 스트립(예: 그림 5.13b

와 같은 형태). 선 목록(예: 그림 5.13c와 같은 형태). 삼각형 스트

립(예: 그림 5.13d와 같은 형태). 삼각형 목록(예: 그림 5.14a와

같은 형태).

삼각형 스트립(예: 그림 5.13d 참조). 삼각형 리스트(예: 그림 5.14a 참

조).

4. 그림 6.19와 같이 피라미드의 정점 및 인덱스 리스트를 구성하고 이를 그려 보십시오. 밑면 정점들은 녹색으로, 꼭대기 정점은 빨간색으로 채색하십시오.
5. "Box" 데모를 실행하고, 우리가 꼭지점에서만 색상을 지정했음을 상기하십시오. 삼각형의 각 픽셀에 대한 픽셀 색상이 어떻게 얻어졌는지 설명하십시오.

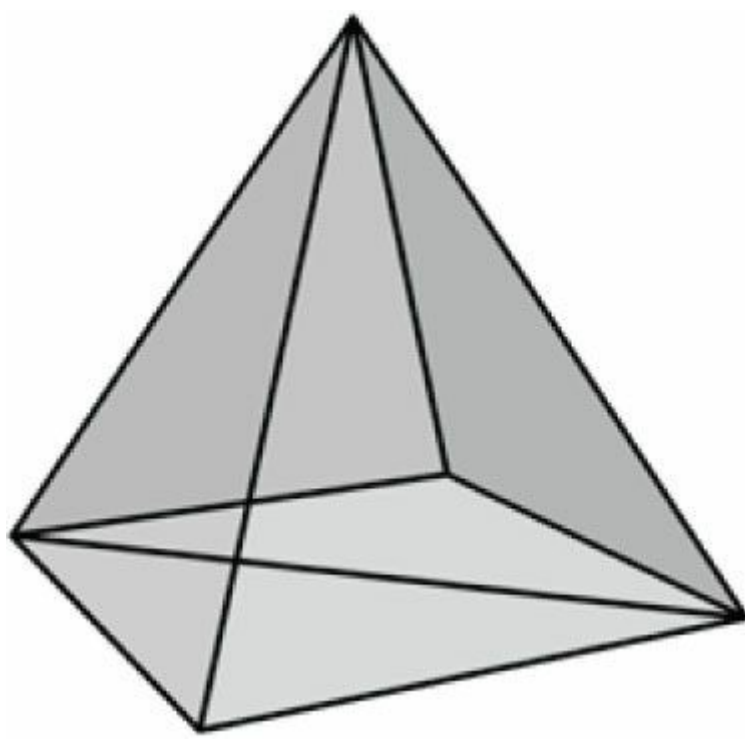


그림 6.19. 피라미드의 삼각형들.

- 컬러 큐브 데모를 수정하여, 월드 공간으로 변환하기 전에 버텍스 셰이더에서 각 버텍스에 다음 변환을 적용하십시오.

```
vin.Pos.xy += 0.5f*sin(vin.Pos.x)*sin(3.0f*gTime); vin.Pos.z *= 0.6f +
0.4f*sin(2.0f*gTime);
```

`gTime` 상수 버퍼 변수는 현재 `GameTimer::TotalTime()` 값에 대응합니다. 이는 사인 함수를 사용하여 점점들을 주기적으로 왜곡함으로써 시간의 함수로서 점점들을 애니메이션 처리합니다.

- 박스(box)와 피라미드(pyramid)의 점점들을 하나의 큰 점점 버퍼로 병합합니다. 또한 박스와 피라미드의 인덱스들을 하나의 큰 인덱스 버퍼로 병합합니다(단, 인덱스 값은 업데이트하지 않습니다). 그런 다음 `ID3D11DeviceContext::DrawIndexed`의 매개변수를 사용하여 박스와 피라미드를 하나씩 차례로 렌더링합니다. 박스와 피라미드가 월드 공간에서 서로 겹치지 않도록 월드 변환 행렬을 사용합니다.
- 컬러 큐브 데모를 수정하여 큐브를 와이어프레임 모드로 렌더링합니다. 두 가지 다른 방법으로 수행합니다: 첫째, C++ 코드에서 `ID3D11DeviceContext::RSSetState`를 호출하여 래스터라이제이션 렌더링 상태를 설정합니다; 둘째, 이펙트 파일에서 이펙트 패스 내 `SetRasterizerState()`를 호출하여 래스터라이제이션 렌더링 상태를 설정합니다.
- 컬러 큐브 데모를 수정하여 백페이스 컬링을 비활성화하십시오(`CullMode = None`). 또한 백페이스 대신 프론트 페이스를 컬링해 보십시오(`CullMode = Front`). 이를 두 가지 다른 방법으로 수행하십시오: 첫째, C++ 코드에서 `ID3D11DeviceContext::RSSetState`를 호출하여 래스터화 렌더링 상태를 설정하는 방법; 둘째, 효과 패스에서 `SetRasterizerState()`를 호출하여 효과 파일에서 래스터화 렌더링 상태를 설정하는 방법입니다. 결과를 와이어프레임 모드로 출력하여 차이를 확인할 수 있도록 하십시오.
- 버텍스 메모리가 상당하다면 128비트 색상 값을 32비트 색상 값으로 줄이는 것이 유용할 수 있습니다. "Box" 데모를 수정하여 버텍스 구조체에서 128비트 색상 값 대신 32비트 색상 값을 사용하세요. 수정된 버텍스 구조체와 해당 버텍스 입력 설명은 다음과 같습니다:

```
struct Vertex
{
    XMFLOAT3 Pos;
    XMCOLOR Color;
};
D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0,
     D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"COLOR", 0, DXGI_FORMAT_R8G8B8A8_UNORM, 0, 12,
     D3D11_INPUT_PER_VERTEX_DATA, 0}
};
```



8비트 색상 구성 요소는 *RGBA* 형식이 아닌 *ABGR* 형식으로 **UINT**에 압축되어야 합니다. 이는 리틀 엔디안 형식에서 바이트 순서가 반전되기 때문입니다(리틀 엔디안에서는 *ABGR* 바이트가 실제로 *RGBA* 순서로 배열됨). 다중 바이트 데이터의 경우, 빅엔디안은 메모리에서 가장 낮은 주소에 최상위 바이트를 저장하고 가장 높은 주소에 최하위 바이트를 저장합니다. 반면 리틀엔디안은 메모리에서 가장 낮은 주소에 최하위 바이트를 저장하고 가장 높은 주소에 최상위 바이트를 저장합니다. **XMColor**가 사용하는 *ARGB*에서 *ABGR*로의 변환은 다음과 같은 함수로 수행할 수 있습니다:

```
static D3DX11_INLINE UINT ArgbToAbgr(UINT argb)
{
    BYTE A = (argb >> 24) & 0xff; BYTE R =
    (argb >> 16) & 0xff; BYTE G = (argb >> 8) &
    0xff; BYTE B = (argb >> 0) & 0xff;

    return (A << 24) | (B << 16) | (G << 8) | (R << 0);
}
```

11. "Skull" 데모를 수정하여 뷰포트를 출력 창의 하위 사각형 영역에만 렌더링하도록 변경하십시오.

12. 다음 C++ 정점 구조를 고려하십시오:

```
struct Vertex
{
    XMFLOAT3 Pos; XMFLOAT4
    Color;
};
```

입력 레이아웃 설명 순서가 정점 구조 순서와 일치해야 합니까? 즉, 다음 정점 선언이 이 정점 구조에 대해 올바른가요? 실험을 통해 확인하세요. 그런 다음 작동한다고 생각하거나 작동하지 않는다고 생각하는 이유를 설명하세요.

```
D3D11_INPUT_ELEMENT_DESC vertexDesc[] =
{
    {"COLOR", 0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0},
    {"POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT, 0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0},
};
```

해당 버텍스 셰이더 구조체의 순서는 C++ 버텍스 구조체의 순서와 일치해야 합니까? 즉, 다음 버텍스 셰이더 구조체가 이전 C++ 버텍스 구조체와 호환됩니까? 실험을 통해 확인하십시오. 그런 다음 작동하거나 작동하지 않는다고 생각하는 이유를 설명하십시오.

```
struct VertexIn
{
    float4 Color : COLOR; float3 Pos :
    POSITION;
};
```

13. *Direct3D 가위 테스트*에 하면 사각형 배열을 제출할 수 있습니다. 가위 테스트는 가위 사각형 외부의 모든 픽셀을 제거합니다. "Shapes" 데모를 수정하여 가위 테스트를 사용하도록 하십시오.

가위 사각형은 다음 메서드로 설정할 수 있습니다:

```
void RSSetScissorRects(UINT NumRects, const D3D11_RECT *pRects);
다음은 호출 예시입니다:
D3D11_RECT rects = {100, 100, 400, 400};
```

```
md3dImmediateContext->RSSetScissorRects(1, &rects);
위 호출은 가위 영역만 설정할 뿐 가위 테스트를 활성화하지는 않습니다. 가위 테스트는
```

D3D11_RASTERIZER_DESC::ScissorEnable을 통해 활성화/비활성화됩니다.

14. "Shape" 데모를 수정하여 **GeometryGenerator::CreateSphere** 대신 **GeometryGenerator::CreateGeosphere**를 사용하도록 하십시오. 0, 1, 2, 3개의 세분화 수준으로 시도해 보십시오.

제7장 조명

[그림 7.1](#)을 살펴보십시오. 왼쪽에는 조명이 없는 구체가, 오른쪽에는 조명이 있는 구체가 있습니다. 보시다시피 왼쪽의 구체는 상당히 평평해 보입니다—아마도 구체라기보다는 2차원 원에 불과할 수도 있습니다. 반면 오른쪽 구체는 3차원적으로 보입니다. 조명과 음영이 물체의 입체 형태와 부피를 인식하는 데 도움을 주기 때문입니다. 실제로 우리가 세상을 시각적으로 인지하는 것은 빛과 물질 간의 상호작용에 달려 있으며, 따라서 사실적인 장면을 생성하는 문제의 상당 부분은 물리적으로 정확한 조명 모델과 관련이 있습니다.

물론 일반적으로 모델이 정확할수록 계산 비용이 더 많이 듭니다. 따라서 현실성과 속도 사이에서 균형을 맞춰야 합니다. 예를 들어 영화의 3D 특수 효과 장면은 게임보다 훨씬 복잡하고 매우 사실적인 조명 모델을 활용할 수 있습니다. 영화의 프레임은 미리 렌더링되기 때문에 몇 시간씩 걸리는 계산도 감당할 수 있기 때문입니다.

예를 들어, 영화의 3D 특수 효과 장면은 게임보다 훨씬 복잡하고 매우 사실적인 조명 모델을 활용할 수 있습니다. 영화의 프레임은 사전 렌더링되기 때문에 프레임 하나를 처리하는 데 몇 시간 또는 며칠이 걸려도 괜찮기 때문입니다. 반면 게임은 실시간 애플리케이션이므로 프레임은 초당 최소 30프레임의 속도로 그려져야 합니다.

이 책에서 설명하고 구현한 조명 모델은 [Möller02]에 기술된 모델을 크게 기반으로 함을 유의하십시오.

목표:

1. 빛과 재료 간의 상호작용에 대한 기본적인 이해를 얻기 위함입니다.
2. 국소 조명과 전역 조명의 차이점을 이해하기 위해.
3. 표면상의 한 점이 "향하는" 방향을 수학적으로 어떻게 표현할 수 있는지 알아내어, 입사광이 표면에 닿는 각도를 결정할 수 있도록 한다.
4. 법선 벡터를 올바르게 변환하는 방법을 학습한다.
5. 주변광, 확산광, 반사광을 구분할 수 있도록 한다.
6. 방향광, 점광원, 스포트라이트 구현 방법을 학습한다.
7. 감쇠 매개변수를 제어하여 깊이에 따른 광도 변화를 이해한다.

7.1 빛과 재료의 상호작용

조명을 사용할 때 우리는 더 이상 정점 색상을 직접 지정하지 않습니다. 대신 재질과 조명을 지정한 후 조명 방정식을 적용합니다. 이 방정식은 빛과 재질의 상호작용을 바탕으로 정점 색상을 계산해 줍니다. 이로 인해 객체의 색상이 훨씬 더 사실적으로 표현됩니다([그림 7.1a](#)와 [7.1b](#)를 다시 비교해 보세요).

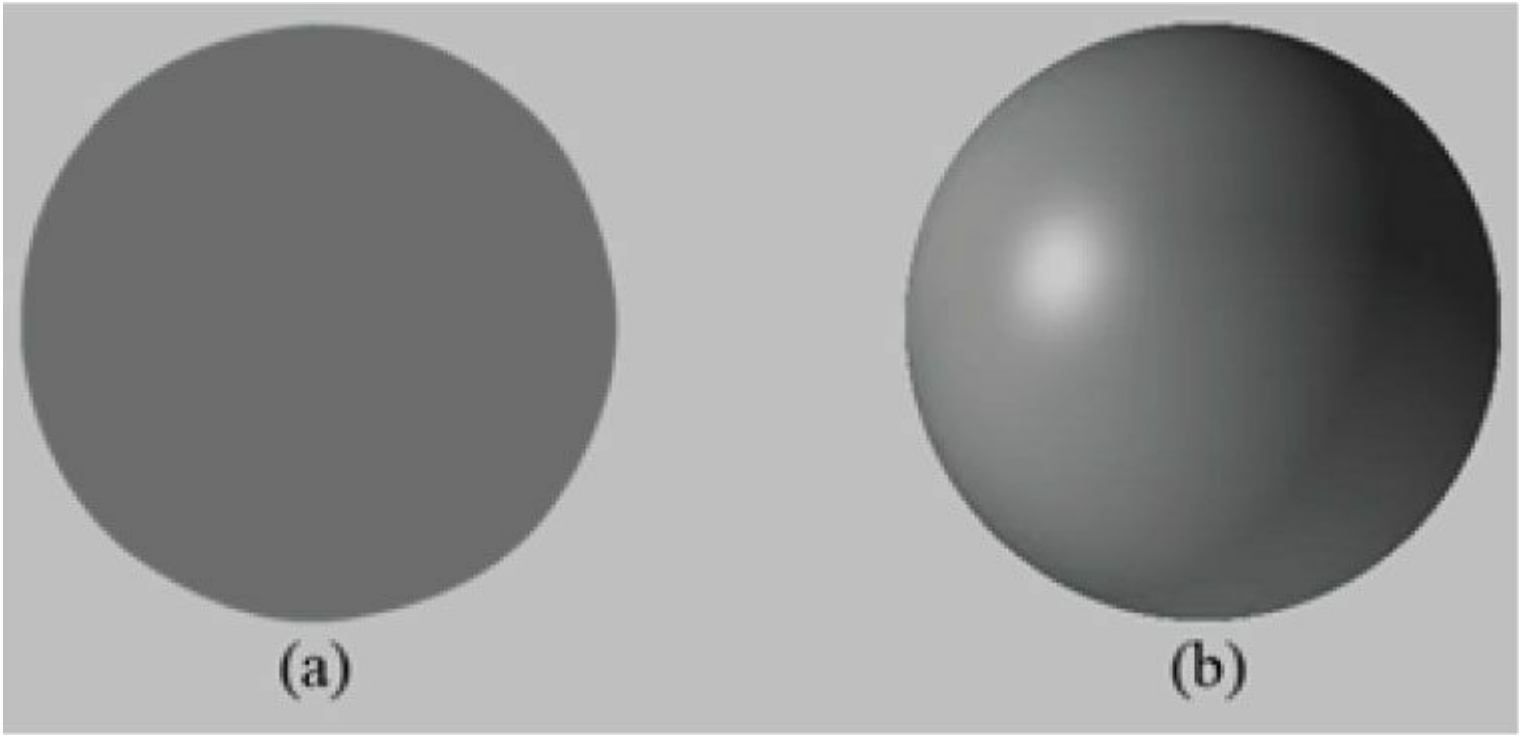


그림 7.1. (a) 조명이 적용되지 않은 구체는 2차원적으로 보인다. (b) 조명이 적용된 구체는 3차원적으로 보인다.

재료는 빛이 물체의 표면과 어떻게 상호작용하는지를 결정하는 속성으로 생각할 수 있습니다. 예를 들어, 표면이 반사하고 흡수하는 빛의 색상, 반사율, 투명도, 광택도 등이 모두 표면 재료를 구성하는 매개변수입니다. 그러나 이 장에서는 표면이 반사하고 흡수하는 빛의 색상과 광택도만 다룹니다.

우리 모델에서 광원은 다양한 강도의 적색, 녹색, 청색 빛을 방출할 수 있습니다. 이를 통해 다양한 빛의 색상을 시뮬레이션할 수 있습니다. 빛이 광원에서 바깥으로 이동하다가 물체와 충돌하면, 그 빛의 일부는 흡수되고 일부는 반사될 수 있습니다(유리 같은 투명한 물체의 경우 빛의 일부가 매질을 통과하지만, 여기서는 투명성은 고려하지 않습니다). 반사된 빛은 이제 새로운 경로를 따라 이동하며 다른 물체에 부딪힐 수 있고, 여기서 다시 일부 빛이 흡수되고 반사됩니다. 한 광선은 완전히 흡수되기 전에 여러 물체에 부딪힐 수 있습니다. 아마도 일부 광선은 결국 눈 안으로 들어와(그림 7.2 참조) 망막의 광수용체 세포(원추세포와 막대세포라고 함)에 도달할 것입니다.

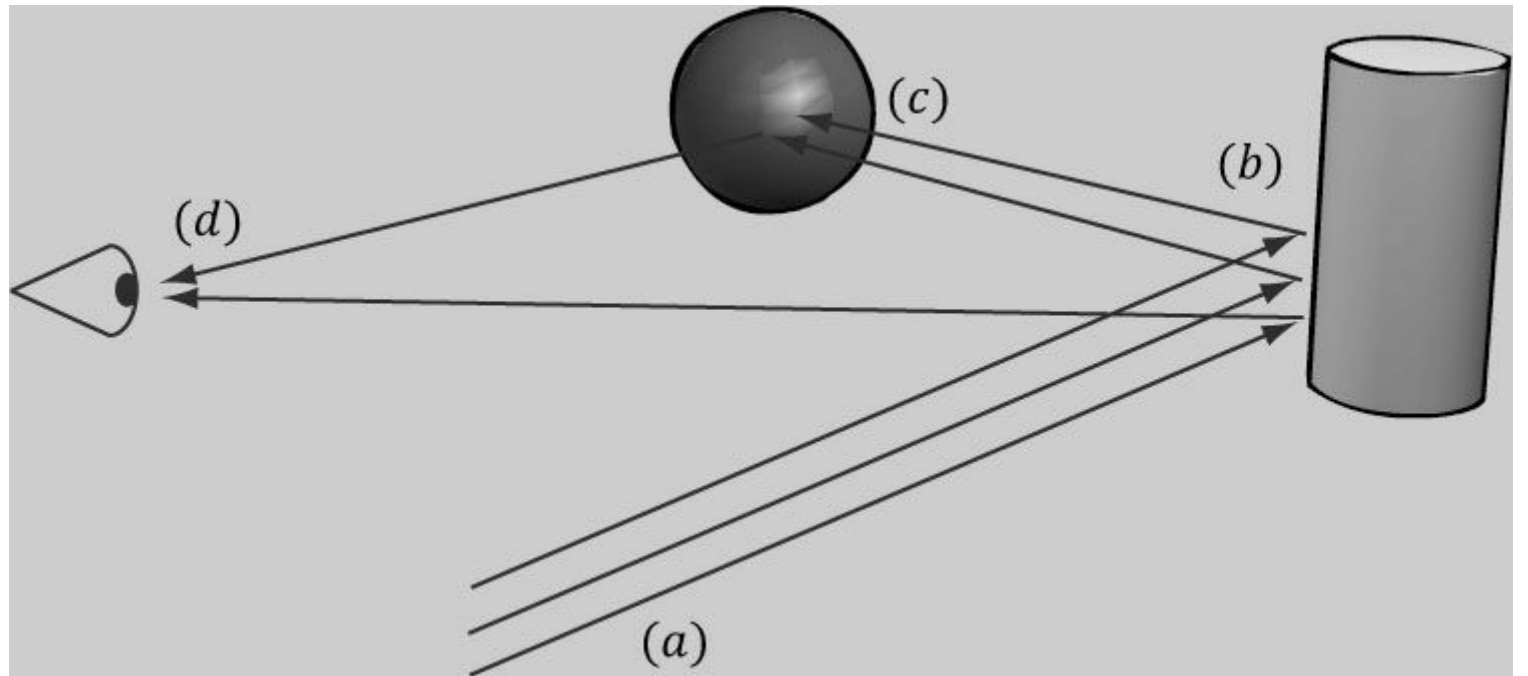


그림 7.2. (a) 입사하는 백색광의 플럭스. (b) 빛이 원통에 부딪히면 일부 광선은 흡수되고 다른 광선은 눈과 구면체 쪽으로 산란된다. (c) 원통에서 구면체 쪽으로 반사된 빛은 다시 흡수되거나 반사되어 눈으로 들어간다. (d) 눈은 입사하는 빛을 받아들이며, 이 빛이 눈이 보는 것을 결정한다.

삼원색 이론([Santrock03] 참조)에 따르면, 망막에는 각각 적색, 녹색, 청색 빛에 민감한(일부 중첩됨) 세 종류의 광수용체가 존재한다. 입사하는 RGB 빛은 빛의 강도에 따라 각기 다른 강도로 해당 광수용체를 자극한다. 광수용체가 자극을 받거나 받지 않을 때, 신경 자극은 시신경을 따라 뇌로 전달되며

뇌로 전달되며, 뇌는 이 광수용체 자극을 바탕으로 머릿속에 이미지를 생성한다. (물론 눈을 감거나 가리면 수용체 세포가 자극을 받지 않아 뇌는 검은색으로 인식한다.)

예를 들어, 다시 [그림 7.2](#)를 살펴보자. 원통의 재질이 적색광의 75%, 녹색광의 75%를 반사하고 흡수한다고 가정하자.

나머지 빛은 구체에 반사되며, 구체는 25%의 적색광을 반사하고 나머지는 흡수합니다. 또한 광원에서 순수한 백색광이 방출된다고 가정합니다. 광선이 실린더에 부딪히면 모든 청색광이 흡수되고 75%의 적색광과 녹색광만 반사됩니다(즉, 중간에서 높은 강도의 노란색). 이 빛은 산란되며, 일부는 눈으로 들어가고 일부는 구체 쪽으로 이동합니다. 눈으로 들어가는 빛은 주로 적색과 녹색 원추세포를 중간 정도의 강도로 자극하므로, 관찰자는 실린더를 중간 밝기의 노란색으로 인식합니다. 이제 나머지 광선은 구체를 향해 이동하여 충돌합니다. 구체는 25%의 적색광을 반사하고 나머지는 흡수합니다. 따라서 희석된 입사 적색광(중간-높은 강도의 적색)은 더욱 희석되어 반사되고, 입사한 모든 녹색광은 흡수됩니다. 이 남은 적색광은 눈 안으로 들어가 주로 적색 원추세포를 낮은 정도로 자극합니다. 따라서 관찰자는 구체가 어두운 적색으로 보이게 됩니다.

이 책에서 우리가(그리고 대부분의 실시간 애플리케이션이) 채택하는 조명 모델은 *국소 조명 모델*이라고 합니다. 국소 모델에서는 모델에서는 각 객체가 다른 객체와 독립적으로 조명되며, 조명 과정에서는 광원에서 직접 방출된 빛만 고려됩니다(즉, 다른 장면 객체에 반사되어 현재 조명 중인 객체에 도달한 빛은 무시됩니다).

[그림 7.3](#)은 이 모델의 결과를 보여줍니다.

반면, 전역 조명 모델은 광원에서 직접 방출된 빛뿐만 아니라

광원은 물론 장면 내 다른 물체에 반사된 간접광까지 고려합니다. 이러한 모델은 물체에 조명을 적용할 때 장면 전체를 종합적으로 고려하기 때문에 전역 조명 모델이라 불립니다. 전역 조명 모델은 일반적으로 실시간 게임에는 비용이 너무 많이 듭니다(하지만 사진처럼 사실적인 장면 생성에 매우 근접합니다). 실시간 전역 조명 기법에 대한 연구가 진행 중입니다.

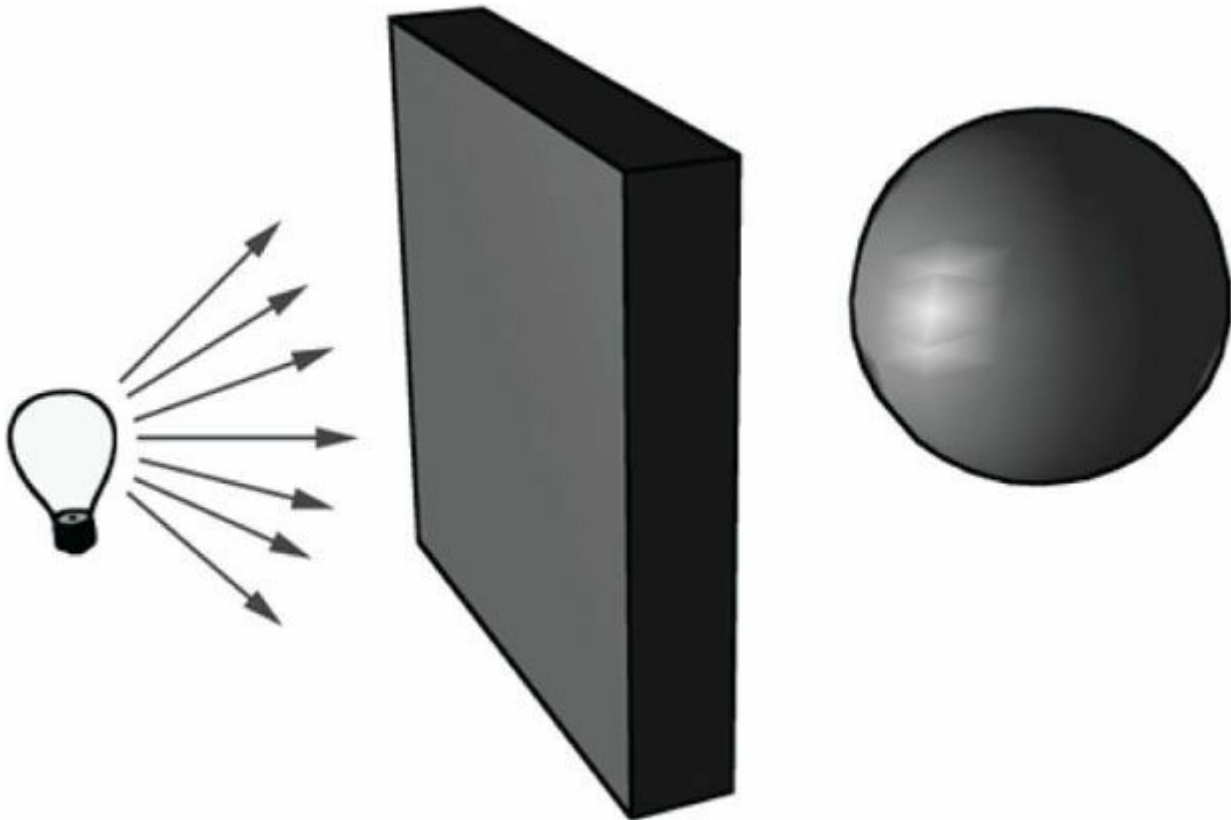


그림 7.3. 물리적으로 벽은 전구에서 방출된 광선을 차단하며, 구체는 벽의 그림자 속에 위치한다. 그러나 로컬 일루미네이션 모델에서는 벽이 존재하지 않는 것처럼 구체가 조명된다.

7.2 법선 벡터

*면 법선*은 다각형이 향하는 방향을 나타내는 단위 벡터입니다(즉, 다각형의 모든 점에 대해 수직임). [그림 7.4a](#)를 참조하십시오. *표면 법선*은 표면 상의 한 점에 대한 접평면에 수직인 단위 벡터입니다. [그림 7.4b](#)를 참조하십시오. 표면 법선이 표면 상의 한 점이 "향하는" 방향을 결정한다는 점을 유의하십시오.

조명 계산을 위해서는 삼각형 메쉬 표면의 각 점에서 표면 법선을 필요로 합니다. 이를 통해 메쉬 표면의 해당 점에 빛이 입사하는 각도를 결정할 수 있기 때문입니다.

표면 법선을 얻기 위해, 우리는 꼭지점(vertex)에서의 표면 법선만 지정합니다(소위 *꼭지점 법선*). 그런 다음 삼각형 메쉬 표면의 각 점에서 표면 법선의 근사값을 얻기 위해, 래스터화 과정에서 이 꼭지점 법선들이 삼각형 전체에 걸쳐 보간됩니다(\$5.10.3을 참고하고 [그림 7.5](#)를 참조하십시오).

법선을 보간하고 픽셀 단위로 조명 계산을 수행하는 것을 픽셀 라이팅 또는 폴 라이팅이라고 합니다. 덜

비싸지만 덜 정확한 방법은 정점별로 조명 계산을 수행하는 것입니다. 그런 다음 정점별 픽셀 조명 (Note:) 계산 결과가 정점 셰이더에서 출력되어 삼각형의 픽셀 전체에 보간됩니다. 픽셀 셰이더에서 정점 셰이더로 계산을 옮기는 것은 품질을 희생하는 일반적인 성능 최적화 기법입니다. 때로는 시각적 차이가 매우 미묘하여 이러한 최적화가 매우 매력적으로 느껴집니다.

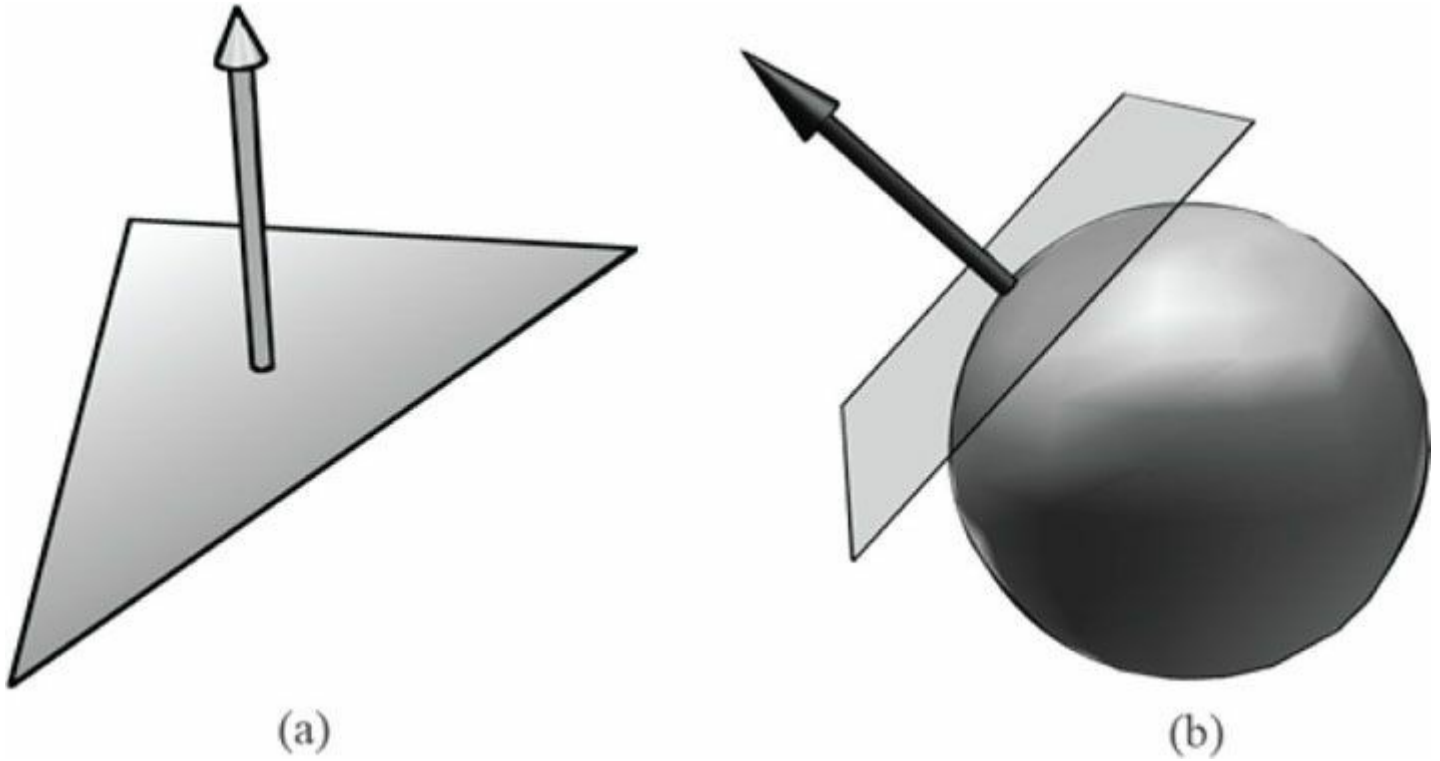


그림 7.4. (a) 면 법선은 면 상의 모든 점에 대해 직교한다. (b) 표면 법선은 표면 상의 한 점의 접평면에 대해 직교하는 벡터이다.

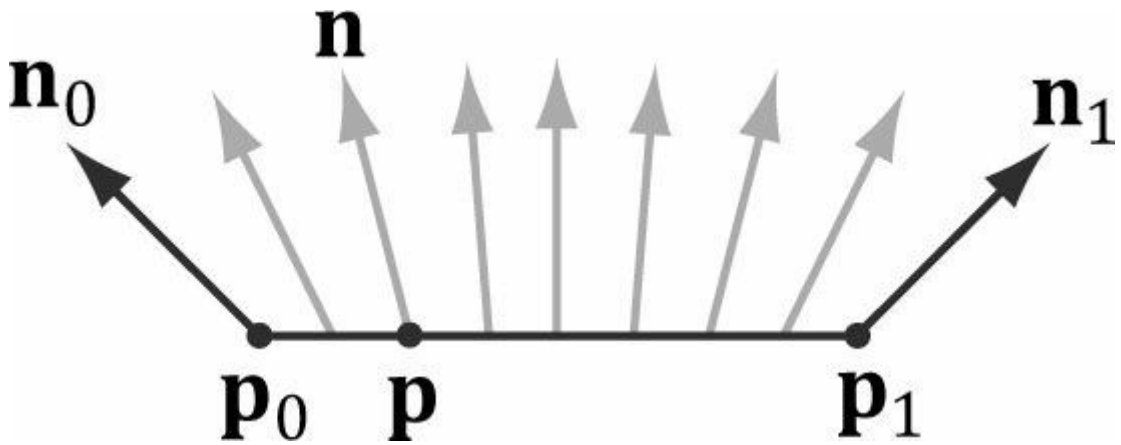


그림 7.5. 선분 정점 p_0 과 p_1 에서 정점 법선 $\mathbf{n}_0$ 과 $\mathbf{n}_1$ 이 정의된다. 선분 내부 점 p 에 대한 법선 벡터 $\mathbf{n}$ 은 정점 법선 사이의 선형 보간(가중 평균)으로 구한다. 즉, $\mathbf{n} = \mathbf{n}_0 + t(\mathbf{n}_1 - \mathbf{n}_0)$ 이며, 여기서 t 는 다음과 같다. $p = p_0 + t(p_1 - p_0)$. 단순화를 위해 선분 상의 정규 보간을 예시로 들었지만, 이 개념은 3차원 삼각형 상의 보간으로 쉽게 일반화됩니다.

7.2.1 법선 벡터 계산

삼각형 $\Delta p_0 p_1 p_2$ 의 면 법선을 구하기 위해, 먼저 삼각형의 변 위에 놓인 두 벡터를 계산합니다:

$$\mathbf{u} = p_1 - p_0 \quad \mathbf{v} = p_2 - p_0$$

그러면 면 법선은 다음과 같습니다:

$$\mathbf{n} = \frac{\mathbf{u} \times \mathbf{v}}{\|\mathbf{u} \times \mathbf{v}\|}$$

다음은 삼각형의 세 꼭짓점으로부터 삼각형의 정면(§5.10.2)의 면 법선을 계산하는 함수입니다.

```
void Compute Normal(const D3DXVECTOR3& p0, const
                    D3DXVECTOR3& p1,
                    const D3DXVECTOR3& p2, D3DXVECTOR3&
                    out)
{
    D3DXVECTOR3 u = p1 - p0; D3DXVECTOR3 v = p2 -
    p0;

    D3DXVec3Cross(&out, &u, &v);
    D3DXVec3Normalize(&out, &out);
}

미분 가능한 표면의 경우, 미적분을 사용하여 표면 상의 점들의 법선을 구할 수 있습니다. 안타깝게도 삼각형 메쉬는 미분 가능하지 않습니다. 삼각형 메쉬에 일반적으로 적용되는 기법은 점 법선 평균화라고 합니다. 메쉬 내 임의의 정점 v에 대한 정점 법선 n은 해당 정점 v를 공유하는 메쉬 내 모든 다각형의 면 법선을 평균화하여 구합니다. 예를 들어, 그림 7.6에서 메쉬 내 네 개의 다각형이 정점 v를 공유하므로, v의 정점 법선은 다음과 같이 주어집니다:
```

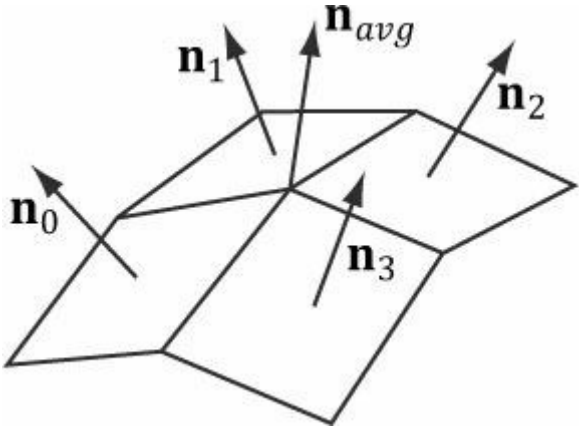


그림 7.6. 중간 정점은 인접한 네 개의 다각형이 공유하므로, 중간 정점 법선은 네 개의 다각형 면 법선을 평균화하여 근사합니다.

$$\mathbf{n}_{avg} = \frac{\mathbf{n}_0 + \mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3}{\|\mathbf{n}_0 + \mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3\|}$$

이전 예시에서는 결과를 정규화하기 때문에 일반적인 평균 계산과 달리 4로 나누지 않아도 됩니다. 또한 더 정교한 평균화 방식을 구성할 수 있다는 점도 유의하십시오. 예를 들어, 다각형의 면적에 따라 가중치를 결정하는 가중 평균을 사용할 수 있습니다(예: 면적이 큰 다각형이 작은 다각형보다 더 큰 가중치를 가짐).

다음 의사 코드는 삼각형 메쉬의 정점 및 인덱스 목록을 기반으로 이 평균화를 구현하는 방법을 보여줍니다:

```
// 입력:
// 1. 정점 배열 (mVertices). 각 정점은
// 위치 성분(pos)과 법선 성분(normal).
// 2. 인덱스 배열(mIndices).

// 메쉬 내 각 삼각형에 대해: for(UINT i = 0; i <
mNumTriangles; ++i)
{

    // i번째 삼각형의 인덱스 UINT i0 =
    mIndices[i*3+0]; UINT i1 = mIndices[i*3+1];
    UINT i2 = mIndices[i*3+2];

    // i번째 삼각형의 정점 Vertex v0 =
    mVertices[i0]; Vertex v1 = mVertices[i1];
```

```
Vertex v2 = mVertices[i2];
```

```
// 면 법선 계산 Vector3 e 0 = v1.pos - v0.pos;
```

```
Vector3 e 1 = v2.pos - v0.pos;
```

```
Vector3 면 법선 = Cross(e 0, e 1);
```

```
// 이 삼각형은 다음 세 꼭짓점을 공유하므로,
```

```
// 따라서 이 면 법선을 해당 정점 법선의 평균값에 추가합니다.
```

```
// 정점 법선의 평균값에 이 면 법선을 더합니다.
```

```
mVertices[i0].normal += face Normal;
```

```
mVertices[i1].normal += face Normal;
```

```
mVertices[i2].normal += face Normal;
```

```
}
```

```
// 각 정점 v에 대해, v를 공유하는 모든 삼각형의 면 법선을 합산했습니다.
```

```
이제 정규화만 하면 됩니다. for(UINT i = 0; i < mNumVertices; ++i)
```

```
mVertices[i].normal = Normalize(&mVertices[i].normal));
```

7.2.2 법선 벡터 변환

그림 7.7a를 보자. 여기에는 법선 벡터 $\mathbf{n}$ 에 직교하는 접선 벡터 $\mathbf{u} = \mathbf{v}_1 - \mathbf{v}_0$ 이 있다. 비균일 스케일링 변환 $\mathbf{A}$ 를 적용하면, 그림 7.7b에서 볼 수 있듯이 변환된 접선 벡터 $\mathbf{u} \mathbf{A} = \mathbf{v}_1 \mathbf{A} - \mathbf{v}_0 \mathbf{A}$ 는 변환된 법선 벡터 $\mathbf{n} \mathbf{A}$ 에 직교하지 않게 된다.

따라서 우리의 문제는 다음과 같습니다. 점과 벡터(비정규)를 변환하는 변환 행렬 $\mathbf{A}$ 가 주어졌을 때, 우리는 다음을 찾고자 합니다:

변환 행렬 $\mathbf{B}$ 는 변환된 접선 벡터가 변환된 법선 벡터와 직교하도록(즉, $\mathbf{u} \mathbf{A} \cdot \mathbf{n} \mathbf{B} = 0$) 법선 벡터를 변환합니다. 이를 위해 먼저 우리가 알고 있는 것부터 시작합시다: 법선 벡터 $\mathbf{n}$ 이 접선 벡터 $\mathbf{u}$ 와 직교한다는 것을 알고 있습니다:

$\mathbf{u} \cdot \mathbf{n} = 0$ $\mathbf{u} \mathbf{n}^T = 0$ $\mathbf{u}(\mathbf{A} \mathbf{A}^{-1}) \mathbf{n}^T = 0$ $(\mathbf{u} \mathbf{A})(\mathbf{A}^{-1} \mathbf{n}^T) = 0$ $(\mathbf{u} \mathbf{A})((\mathbf{A}^{-1} \mathbf{n}^T)^T)^T = 0$ $(\mathbf{u} \mathbf{A})(\mathbf{n}(\mathbf{A}^{-1})^T)^T = 0 \mathbf{u} \mathbf{A} \cdot \mathbf{n}(\mathbf{A}^{-1})^T = 0$ $\mathbf{u} \mathbf{A} \cdot \mathbf{n} \mathbf{B} = 0$	법선 벡터에 수직인 접선 벡터 내적을 행렬 곱셈으로 다시 쓰기 단위 행렬 삽입 $\mathbf{I} = \mathbf{A} \mathbf{A}^{-1}$ 행렬 곱셈의 결합법칙 전치 성질 $(\mathbf{A}^T)^T = \mathbf{A}$ 전치 성질 $(\mathbf{A} \mathbf{B})^T = \mathbf{B}^T \mathbf{A}^T$ 행렬 곱셈을 내적(dot product)으로 재표기 변환된 법선 벡터에 직교하는 변환된 접선 벡터
--	--

따라서 $\mathbf{B} = (\mathbf{A}^{-1})^T$ ($\mathbf{A}$ 의 역전치행렬)은 법선 벡터를 변환하여 해당 변환된 접선 벡터 $\mathbf{u} \mathbf{A}$ 에 수직이 되도록 하는 역할을 수행한다.

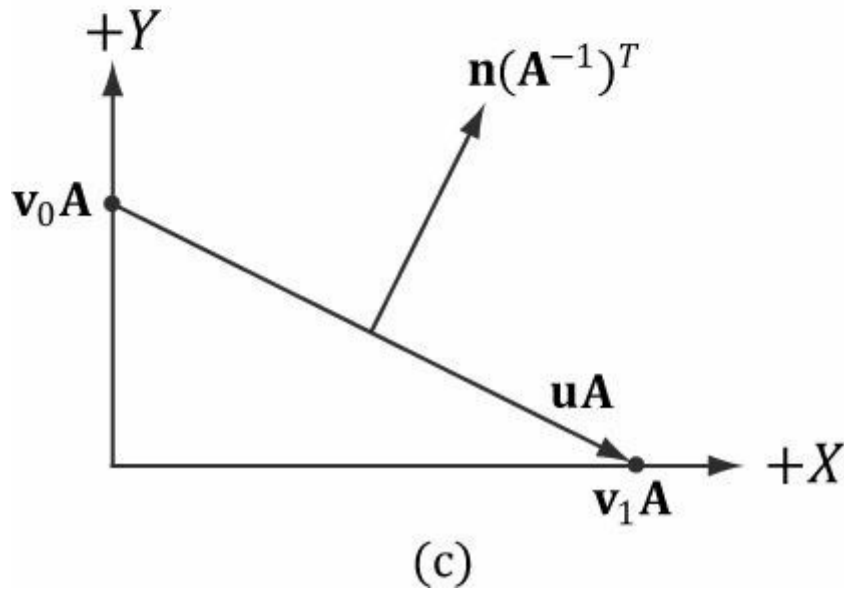
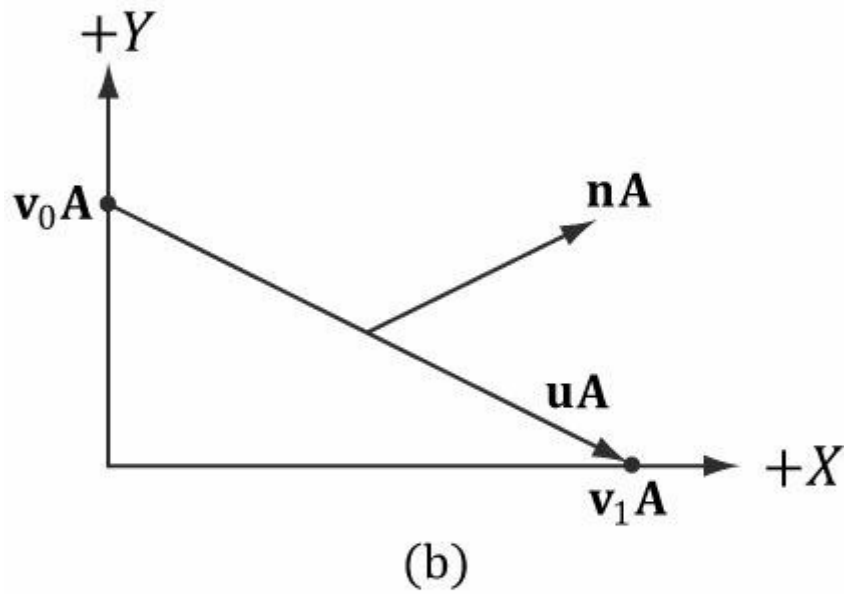
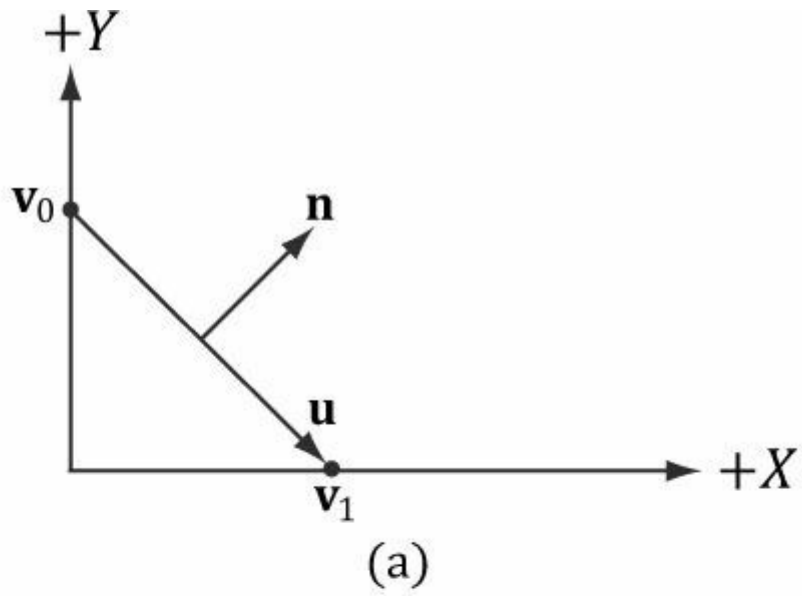


그림 7.7. (a) 변환 전 표면 법선. (b) x축을 2 단위로 스케일링한 후 법선이 더 이상 표면에 수직이 아님. (c) 스케일링 변환의 역전치로 올바르게 변환된 표면 법선.

행렬이 직교 행렬($A^T = A^{-1}$)일 경우, $B = (A^{-1})^T = (A^T)^T = A$ 가 됩니다. 즉, 역행렬을 계산할 필요가 없습니다.

요약하자면, 비균일 변환 또는 전단 변환으로 법선 벡터를 변환할 때는 역전치를 사용하십시오.

역전치를 계산하기 위한 헬퍼 함수를 *MathHelper.h*에 구현합니다:

```
static XMATRIX Inverse Transpose(CXMATRIX M)
```

```

{
    XMMATRIX A = M;
    A.r[3] = XMVectorSet(0.0f, 0.0f, 0.0f, 1.0f);

    XMVECTOR det = XMMatrixDeterminant(A);
    return XMMatrixTranspose(XMMatrixInverse(&det, A));
}

```

우리는 역전치를 사용하여 벡터를 변환하고, 평행 이동은 점에만 적용되기 때문에 행렬에서 평행 이동을 제거합니다. 그러나 §3.2.1에서 알 수 있듯이, 벡터에 대해 $w = 0$ 으로 설정하면(동차 좌표를 사용) 평행 이동에 의해 벡터가 수정되는 것을 방지할 수 있습니다. 따라서 행렬에서 평행 이동을 0으로 설정할 필요가 없습니다. 문제는 역전치분해된 행렬과 비균일 스케일링이 포함되지 않은 다른 행렬(예: 뷰 행렬 $(A^{-1})^T V$)을 연결할 때 발생합니다. $(A^{-1})^T$ 의 4번째 열에 있는 전치된 평행 이동이 곱한 행렬로 "누출"되어 오류를 유발합니다. 따라서 이 오류를 방지하기 위한 예방 조치로 평행 이동을 0으로 설정합니다. 올바른 방법은 노멀을 $((AV)^{-1})^T$ 로 변환하는 것입니다. 다음은 스케일링 및 변위 행렬과, 4번째 열이 $[0, 0, 0, 1]^T$ 이 아닌 경우의 역전치 행렬 예시입니다:

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$(A^{-1})^T = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 2 & 0 & -2 \\ 0 & 0 & 2 & -2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

 역전위 변환을 적용하더라도 법선 벡터는 단위 길이를 잃을 수 있으므로, 변환 후 재규격화가 필요할 수 있다.

7.3 람베르트 코사인 법칙

표면 점에 정면으로 입사하는 빛은 표면 점을 스치듯 입사하는 빛보다 강도가 더 큼니다(그림 7.8 참조). 단면적 dA 를 가진 작은 입사광을 생각해 보십시오.

따라서 핵심은 정점 법선과 빛의 정렬에 따라 다른 강도를 반환하는 함수를 만드는 것이다.

벡터. (광원 벡터는 표면에서 광원까지의 벡터임을 유의하십시오. 즉, 광선이 이동하는 방향과 반대 방향을 향합니다.) 정점 법선 벡터와 광원 벡터가 완벽히 정렬되었을 때(즉, 둘 사이의 각도 θ 가 0° 일 때) 함수는 최대 강도를 반환해야 하며, 정점 법선 벡터와 광원 벡터 사이의 각도가 증가함에 따라 강도가 부드럽게 감소해야 합니다. 만약 $\theta > 90^\circ$ 라면, 빛이 표면의 뒷면에 닿으므로 강도를 0으로 설정합니다. **람베르트 코사인 법칙**이 우리가 찾는 함수를 제공하며, 이는 다음과 같이 주어집니다.

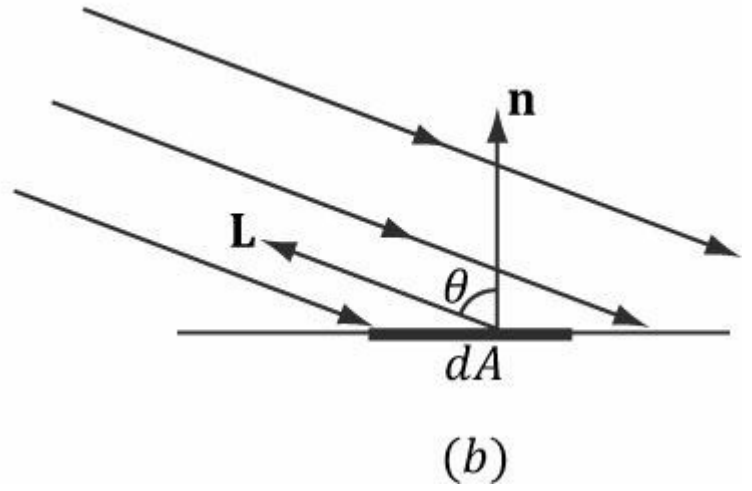
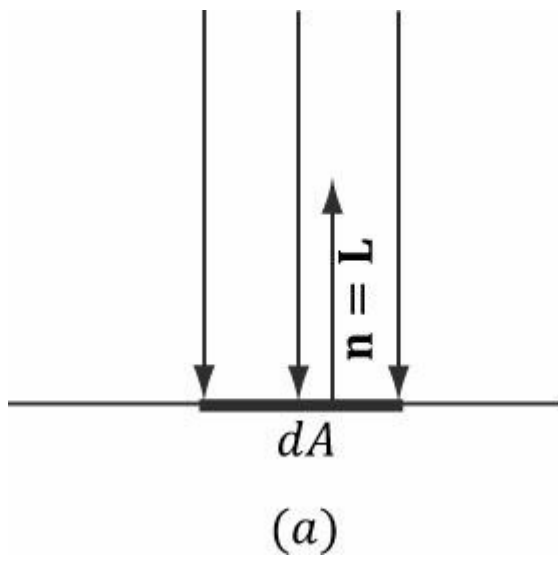


그림 7.8. 작은 면적 요소 dA 를 고려하자. (a) 법선 벡터 $\mathbf{n}$ 과 광선 벡터 $\mathbf{L}$ 이 정렬될 때 면적 dA 는 가장 많은 빛을 받는다. (b) $\mathbf{n}$ 과 $\mathbf{L}$ 사이의 각도 θ 가 증가함에 따라 면적 dA 는 더 적은 빛을 받는다(표면 dA 를 빗나가는 광선들로 묘사됨).

$$f(\theta) = \max(\cos\theta, 0) = \max(\mathbf{L} \cdot \mathbf{n}, 0)$$

여기서 $\mathbf{L}$ 과 $\mathbf{n}$ 은 단위 벡터이다. 그림 6.9는 $f(\theta)$ 의 플롯으로, 0.0에서 1.0(즉, 0%에서 100%)까지의 강도가 θ 에 따라 어떻게 변화하는지 보여준다.

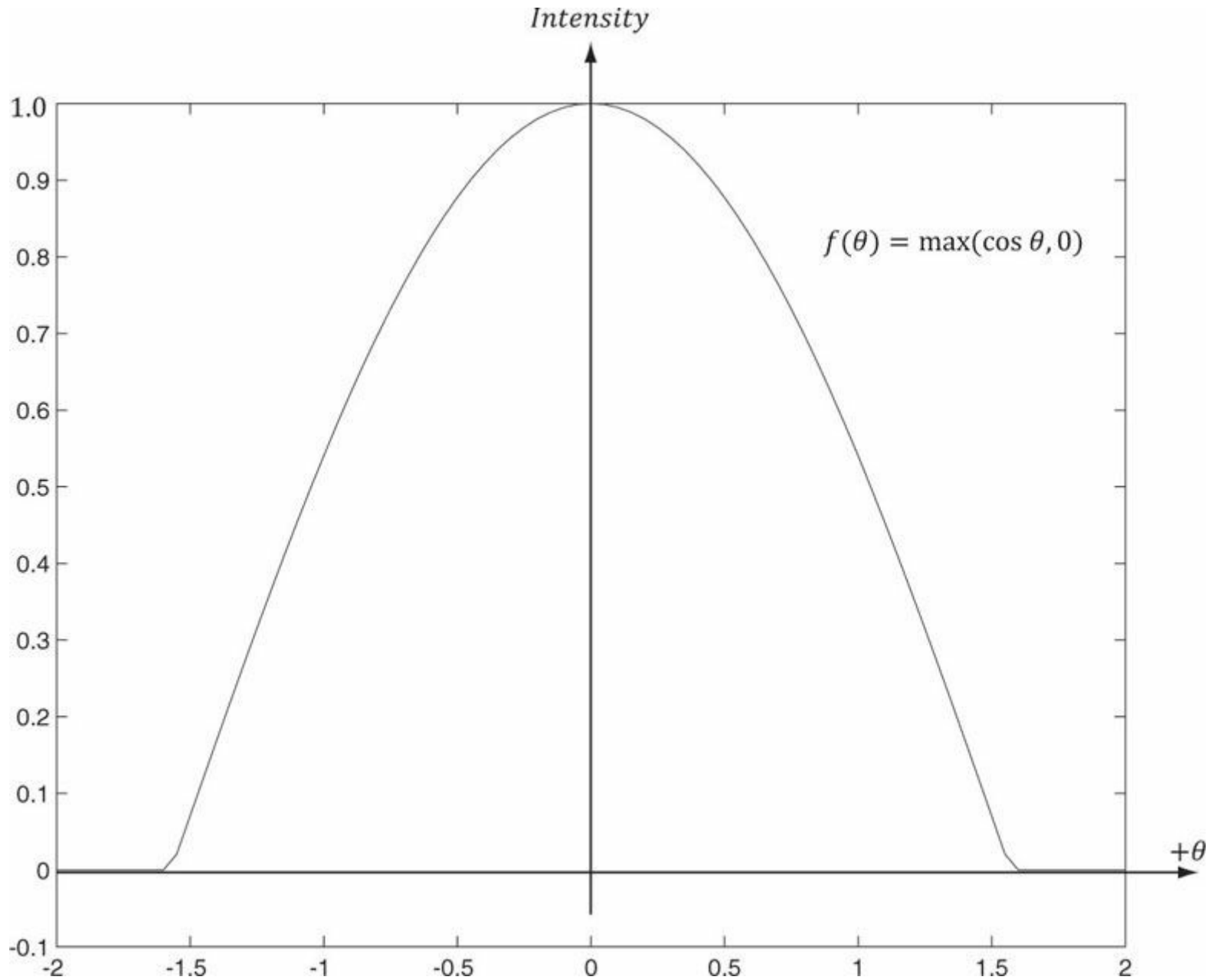


그림 7.9. $-2 \leq \theta \leq 2$ 범위에서 함수 $f(\theta) = \max(\cos \theta, 0) = \max(\mathbf{L} \cdot \mathbf{n}, 0)$ 의 플롯. $\pi/2 \approx 1.57$ 임을 유의하십시오.

7.4 확산 조명

거친 표면을 생각해 보자(그림 7.10 참조). 빛이 이러한 표면의 한 지점에 닿으면 광선이 다양한 무작위 방향으로 산란된다. 이를 *확산 반사*라고 한다. 이러한 빛과 표면의 상호작용을 모델링하기 위한 근사법에서, 우리는 빛이 표면 위 모든 방향으로 균등하게 산란된다고 가정한다. 따라서 반사된 빛은 시점(눈 위치)에 관계없이 눈에 도달하게 된다. 따라서 시점을 고려할 필요가 없으며(즉, 확산 조명 계산은 시점에 독립적임), 표면 상의 한 점의 색상은 시점에 관계없이 항상 동일하게 보입니다.

우리는 확산 조명 계산을 두 부분으로 나눕니다. 첫 번째 부분에서는 확산 빛 색상과 확산 재질을 지정합니다.

색상. 확산 재료는 표면이 반사하고 흡수하는 입사 확산광의 양을 지정하며, 이는 성분별 색상 곱셈으로 처리됩니다. 예를 들어, 표면의 특정 지점이 입사 적색광의 50%, 녹색광의 100%, 청색광의 75%를 반사하고 입사광 색상이 80% 강도의 백색광이라고 가정해 보겠습니다. 따라서 입사 확산광 색상은 $\mathbf{l}_d = (0.8, 0.8, 0.8)$ 로 주어지고 확산 재료 색상은 $\mathbf{m}_d = (0.5, 1.0, 0.75)$ 로 주어집니다. 그러면 해당 점에서 반사되는 빛의 양은 다음과 같이 주어집니다:

$$\mathbf{D} = \mathbf{l}_d \otimes \mathbf{m}_d = (0.8, 0.8, 0.8) \otimes (0.5, 1.0, 0.75) = (0.4, 0.8, 0.6)$$

확산 조명 계산을 완료하기 위해, 우리는 단순히 램버트 코사인 법칙(표면 법선과 광선 벡터 사이의 각도에 따라 표면이 원래 빛의 어느 정도를 수신하는지 제어함)을 포함합니다. $\mathbf{l}_d$ 를 확산광 색상, $\mathbf{m}_d$ 를 확산 재료 색상, $k_d = \max(\mathbf{L} \cdot \mathbf{n}, 0)$ 로 정의하자. 여기서 $\mathbf{L}$ 은 광선 벡터, $\mathbf{n}$ 은 표면 법선 벡터이다. 그러면 확산광의 양은 다음과 같다.